# HTML

MDN Web Docs (Mozilla) — English

Fonte oficial: https://developer.mozilla.org/en-US/docs/Web/HTML
Gerado para o projeto doc-frog

# HTML: HyperText Markup Language

**HTML** (HyperText Markup Language) is the most basic building block of the Web. It defines the meaning and structure of web content. Other technologies besides HTML are generally used to describe a web page's appearance/presentation (CSS) or functionality/behavior (JavaScript).

"Hypertext" refers to text containing links that connect web pages to one another, either within a single website or between websites. Links are a fundamental aspect of the Web. By uploading content to the Internet and linking it to pages created by other people, you become an active participant in the World Wide Web.

HTML uses "markup" to annotate text, images, and other content for display in a Web browser. HTML markup includes special "elements" such as `<head>`, `<title>`, `<body>`, `<header>`, `<footer>`, `<article>`, `<section>`, `<p>`, `<div>`, `<span>`, `<img>`, `<aside>`, `<audio>`, `<canvas>`, `<datalist>`, `<details>`, `<embed>`, `<nav>`, `<search>`, `<output>`, `<progress>`, `<video>`, `<ul>`, `<ol>`, `<li>` and many others.

An HTML element is set off from other text in a document by "tags", which consist of the element name surrounded by `<` and `>`. The name of an element inside a tag is case-insensitive. That is, it can be written in uppercase, lowercase, or a mixture. For example, the `<title>` tag can be written as `<Title>`, `<TITLE>`, or in any other way. However, the convention and recommended practice is to write tags in lowercase.

The articles below can help you learn more about HTML.

### Beginner's tutorials

Our learn web development core modules contain modern, up-to-date tutorials covering HTML fundamentals.

Your first website: Creating the content

> This article provides a brief tour of what HTML is and how to use it, aimed at people who are completely new to web development.

Structuring content with HTML

> This module covers the basics of the HTML language, before looking at key areas such as document structure, links, lists, images, forms, and more.

HTML forms

> Forms are a very important part of the Web — these provide much of the functionality you need for interacting with websites, e.g., registering and logging in, sending feedback, buying products, and more. This module gets you started with creating the client-side/front-end parts of forms.

### Guides

The HTML guides help you build with HTML on the web. They cover topics such as forms, CORS, content preloading, and responsive images.

HTML cheatsheet for syntax and common tasks

> Quick reference for common HTML syntax and tasks.

Using HTML comments `<!-- … -->`

> HTML comments are used to add explanatory notes to the markup or to prevent the browser from interpreting specific parts of the document.

Using HTML form validation and the Constraint Validation API

> HTML5 introduced constraint validation to ease form validation on the client side. Basic constraints can be checked without JavaScript by setting attributes on form elements.

Content categories

> HTML is comprised of several kinds of content, each of which is allowed to be used in certain contexts and is disallowed in others. Similarly, each context has a set of other content categories it can contain and elements that can or can't be used in them. This is a guide to these categories.

Using date and time formats in HTML

> Certain HTML elements use date and/or time values. This guide describes the formats of the strings that specify these values.

Using microdata in HTML

> Microdata is used to nest metadata within existing content on web pages. Search engines and web crawlers can extract and process microdata to provide a richer browsing experience.

Using microformats in HTML

Microformats are standards used to embed semantics and structured data in HTML for use by social web applications, search engines, aggregators, and other tools.

Understanding quirks and standards modes

Historical information on quirks mode and standards mode.

Using responsive images in HTML

Learn about responsive images that work well on devices with widely differing screen sizes, resolutions, and other features, improving performance across different devices.

Media types and formats on the web

The `<audio>` and `<video>` elements allow you to play audio and video media natively within your content without the need for external software support.

### How to

Define terms with HTML

HTML provides several ways to convey description semantics, whether inline or as structured glossaries. This article shows how to properly mark up keywords when defining them.

Use data attributes

HTML5 is designed with extensibility in mind for data that should be associated with a particular element but need not have any defined meaning. `data-*` attributes allow us to store extra information on standard, semantic HTML elements.

Use cross-origin images in a canvas

Some HTML elements that provide support for CORS, such as `<img>` or `<video>`, have a `crossorigin` attribute (`crossOrigin` property), which lets you configure the CORS requests for the element's fetched data.

Add a hitmap on top of an image

Image maps allow hyperlinks to be associated with different parts of an image. This article shows how to create and implement them.

Author fast-loading HTML pages

These tips are based on common knowledge and experimentation. An optimized web page not only provides for a more responsive site for your visitors but also reduces the load on your web servers and internet connection.

Add JavaScript to your web page

This article explains how to add JavaScript code to an HTML file.

### Reference

HTML consists of **elements**, each of which may be modified by some number of **attributes**. HTML documents are connected to each other with **links**. Browse the complete HTML reference documentation.

HTML elements

Reference for all HTML elements.

HTML attributes

Reference for all HTML attributes. Attributes are additional values that configure elements or adjust their behavior in various ways.

Global attributes

Reference for global attributes that may be specified on all HTML elements, *even those not specified in the standard*. This means that any non-standard elements must still permit these attributes, even though those elements make the document HTML5-noncompliant.

### Attributes by element

Input types

Used to create interactive controls for web-based forms.

Script types

Indicates the type of script represented by the element.

meta name

Provides metadata in name-value pairs for the whole page.

### Attribute values

rel keywords

Defines the relationship between a linked resource and the current document.

### Related topics

Applying color to HTML elements using CSS

This article covers most of the ways you use CSS to add color to HTML content, listing what parts of HTML documents can be colored and what CSS properties to use when doing so.

## HTML guides

This page lists the guides for using HTML.

Content categories

Most HTML elements are a member of one or more **content categories** — these categories group elements that share common characteristics. This is a loose grouping (it doesn't actually create a relationship among elements of these categories), but they help define and describe the categories' shared behavior and their associated rules. It's possible for elements to not be a member of *any* of these categories.

HTML cheatsheet for syntax and common tasks

While using HTML it can be very handy to have an easy way to remember how to use HTML tags properly and how to apply them. MDN provides you with extended HTML reference documentation as well as a deep instructional set of HTML guides. However, in many cases we just need some quick hints as we go. That's the whole purpose of the cheat sheet, to give you some quick accurate ready to use code snippets for common usages.

Understanding quirks and standards modes

In the old days of the web, pages were typically written in two versions: One for Netscape Navigator, and one for Microsoft Internet Explorer. When the web standards were made at W3C, browsers could not just start using them, as doing so would break most existing sites on the web. Browsers therefore introduced two modes to treat new standards compliant sites differently from old legacy sites.

Using date and time formats in HTML

Certain HTML elements use date and/or time values. The formats of the strings that specify these values are described in this article.

Using HTML comments <!-- ... -->

An HTML **comment** is used to add explanatory notes to the markup or to prevent the browser from interpreting specific parts of the document.

Using HTML form validation and the Constraint Validation API

The creation of web forms has always been a complex task. While marking up the form itself is easy, checking whether each field has a valid and coherent value is more difficult, and informing the user about the problem may become a headache. HTML5 introduced new mechanisms for forms: it added new semantic types for the `input` element and *constraint validation* to ease the work of checking the form content on the client side. Basic, usual constraints can be checked, without the need for JavaScript, by setting new attributes; more complex constraints can be tested using the Constraint Validation API.

Using microdata in HTML

Microdata is part of the WHATWG HTML Standard and is used to nest metadata within existing content on web pages. Search engines and web crawlers can extract and process microdata from a web page and use it to provide a richer browsing experience for users. Search engines benefit greatly from direct access to this structured data because it allows search engines to understand the information on web pages and provide more relevant results to users. Microdata uses a supporting vocabulary to describe an item and name-value pairs to assign values to its properties. Microdata is an attempt to provide a declarative way of annotating HTML elements with machine-readable tags than the similar approaches of using RDFa and classic microformats.

Using microformats in HTML

*Microformats* are standards used to embed semantics and structured data in HTML, and provide an API to be used by social web applications, search engines, aggregators, and other tools. These minimal patterns of HTML are used for marking up entities that range from fundamental to domain-specific information, such as people, organizations, events, and locations.

Using responsive images in HTML

In this article, we'll learn about the concept of responsive images — images that work well on devices with widely differing screen sizes, resolutions, and other such features — and look at what tools HTML provides to help implement them. This helps to improve performance across different devices.

See also

- ARIA guides

## HTML cheatsheet for syntax and common tasks

While using HTML it can be very handy to have an easy way to remember how to use HTML tags properly and how to apply them. MDN provides you with extended HTML reference documentation as well as a deep instructional set of HTML guides. However, in many cases we just need some quick hints as we go. That's the whole purpose of the cheat sheet, to give you some quick accurate ready to use code snippets for common usages.

> **Note:** HTML tags must be used for their semantic value, not their appearance. It's always possible to totally change the look and feel of a given tag using CSS so, when using HTML, take the time to focus on the meaning rather than the appearance.

### Inline elements

An "element" is a single part of a webpage. Some elements are large and hold smaller elements like containers. Some elements are small and are "nested" inside larger ones. By default, "inline elements" appear next to one another in a webpage. They take up only as much width as they need in a page and fit together horizontally like words in a sentence or books shelved side-by-side in a row. All inline elements can be placed within the `<body>` element.

Inline elements: usage and examples

| Usage | Element | Example |
|---|---|---|
| A link | `<a>` | `<a href="https://example.org">`<br>`A link to example.org</a>.` |
| An image | `<img>` | `<img src="https://developer.mozilla.org/en-US/docs/Web/HTML/Guides/Cheatsheet/beast.png"`<br>`width="50" />` |
| An inline container | `<span>` | `Used to group elements: for example,`<br>`to <span style="color:blue">style`<br>`them</span>.` |
| Emphasize text | `<em>` | `<em>I'm posh</em>.` |
| Italic text | `<i>` | `Mark a phrase in <i>italics</i>.` |
| Bold text | `<b>` | `Bold <b>a word or phrase</b>.` |
| Important text | `<strong>` | `<strong>I'm important!</strong>` |
| Highlight text | `<mark>` | `<mark>Notice me!</mark>` |
| Strikethrough text | `<s>` | `<s>I'm irrelevant.</s>` |
| Subscript | `<sub>` | `H<sub>2</sub>O` |
| Small text | `<small>` | `Used to represent the <small>small`<br>`print </small>of a document.` |
| Address | `<address>` | `<address>Main street 67</address>` |

| Usage | Element | Example |
| --- | --- | --- |
| Textual citation | `<cite>` | `For more monsters,`<br>`see <cite>The Monster Book of Monsters</cite>.` |
| Superscript | `<sup>` | `x<sup>2</sup>` |
| Inline quotation | `<q>` | `<q>Me?</q>, she said.` |
| A line break | `<br>` | `Line 1<br />Line 2` |
| A possible line break | `<wbr>` | `<div style="width: 200px">`<br>`  Llanfair<wbr />pwllgwyngyll<wbr />gogerychwyrndrobwllllantysiliogogogoch.`<br>`</div>` |
| Date | `<time>` | `Used to format the date. For example:`<br>`<time datetime="2020-05-24">`<br>`published on 23-05-2020</time>.` |
| Code format | `<code>` | `This text is in normal format,`<br>`but <code>this text is in code`<br>`format</code>.` |
| Audio | `<audio>` | `<audio controls>`<br>`  <source src="https://developer.mozilla.org/en-US/docs/Web/HTML/Guides/Cheatsheet//shared-assets/audio/t-rex-roar.mp3" type="audio/mpeg" />`<br>`</audio>` |
| Video | `<video>` | `<video controls width="250"`<br>`  src="https://developer.mozilla.org/en-US/docs/Web/HTML/Guides/Cheatsheet//shared-assets/videos/flower.webm" >`<br>`  <a href="https://developer.mozilla.org/shared-assets/videos/flower.webm">Download WebM video</a>`<br>`</video>` |

### Block elements

"Block elements," on the other hand, take up the entire width of a webpage. They also take up a full line of a webpage; they do not fit together side-by-side. Instead, they stack like paragraphs in an essay or toy blocks in a tower.

> **Note:** Because this cheat sheet is limited to a few elements representing specific structures or having special semantics, the `div` element is intentionally not included — because the `div` element doesn't represent anything and doesn't have any special semantics.

| Usage | Element | Example |
| --- | --- | --- |
| A simple paragraph | `<p>` | `<p>I'm a paragraph</p>`<br>`<p>I'm another paragraph</p>` |
| An extended quotation | `<blockquote>` | `They said:`<br>`<blockquote>The blockquote element indicates an extended quotation.</blockquote>` |
| Additional information | `<details>` | `<details>`<br>`  <summary>HTML Cheat Sheet</summary>`<br>`  <p>Inline elements</p>` |

| Usage | Element | Example |
| --- | --- | --- |
| | | Block elements <br></details> |
| An unordered list | <ul> | <ul><br><li>I'm an item</li><br><li>I'm another item</li><br></ul> |
| An ordered list | <ol> | <ol><br><li>I'm the first item</li><br><li>I'm the second item</li><br></ol> |
| A definition list | <dl> | <dl><br><dt>A Term</dt><br><dd>Definition of a term</dd><br><dt>Another Term</dt><br><dd>Definition of another term</dd><br></dl> |
| A horizontal rule | <hr> | before<hr />after |
| Text Heading | <h1>-<h6> | <h1> This is Heading 1 </h1><br><h2> This is Heading 2 </h2><br><h3> This is Heading 3 </h3><br><h4> This is Heading 4 </h4><br><h5> This is Heading 5 </h5><br><h6> This is Heading 6 </h6> |

## Using HTML comments <!-- ... -->

An HTML **comment** is used to add explanatory notes to the markup or to prevent the browser from interpreting specific parts of the document.

Comments start with the string `<!--` and end with the string `-->`, generally with text in between. This text cannot start with the string `>` or `->`, cannot contain the strings `-->` or `--!>`, nor end with the string `<!-`, though `<!` is allowed.

The browser ignores comments as it renders the code. In other words, they are not visible on the page - just in the code. HTML comments are a way for you to write helpful notes about your code or logic.

The above is true for XML comments as well. In addition, in XML, such as in SVG or MathML markup, a comment cannot contain the character sequence `--`.

Comments can be used on a single line, or span multiple lines. They can be used in the following places:

- Before and after the Doctype
- Before and after the `<html>` element
- As the content of most elements except: `<script>`, `<style>`, `<title>`, `<textarea>`, because these elements interpret their content as raw text

> **Note:** While `<script>` elements should not have HTML comments and should use JavaScript comments instead, there was a legacy practice to enclose the whole script content in an HTML comment so ancient browsers that don't support JavaScript don't render it as text. This is now a legacy feature of JavaScript itself and you should not rely on it.

### Syntax

```
<!-- Comment -->
```

### Examples

```
<!-- A one-line comment -->

<!--
A comment
that stretches
```

```
over several
lines
-->

<!-- The comment below disables
   the HTML contained within -->
<!--
<p>
   This content will not be rendered.
</p>
-->
```

### Notes

HTML comments are only permitted as content. You cannot use it within a tag, such as before an HTML attribute.

As with most programming languages that use the `<!-- -->` comment syntax, comments cannot be nested. In other words, the first instance of `-->` that follows an instance of `<!--` closes the comment.

While comments do start with a `<` and end in a `>`, a comment is not an HTML element.

### See also

- Comments in JavaScript
- Comments in CSS
- `Comment` API (`Comment` inherits from `Node`)

## Using HTML form validation and the Constraint Validation API

The creation of web forms has always been a complex task. While marking up the form itself is easy, checking whether each field has a valid and coherent value is more difficult, and informing the user about the problem may become a headache. HTML5 introduced new mechanisms for forms: it added new semantic types for the `<input>` element and *constraint validation* to ease the work of checking the form content on the client side. Basic, usual constraints can be checked, without the need for JavaScript, by setting new attributes; more complex constraints can be tested using the Constraint Validation API.

For a basic introduction to these concepts, with examples, see the Form validation tutorial.

> **Note:** HTML Constraint validation doesn't remove the need for validation on the *server side*. Even though far fewer invalid form requests are to be expected, invalid ones can still be sent in many ways:
>
> - By modifying HTML via the browser's developer tools.
> - By hand-crafting an HTTP request without using the form.
> - By programmatically writing content into the form (certain constraint validations are *only run* for user input, and not if you set the value of a form field using JavaScript).
>
> Therefore, you should always validate form data on the server side, consistent with what is done on the client side.

### Intrinsic and basic constraints

In HTML, basic constraints are declared in two ways:

- By choosing the most semantically appropriate value for the `type` attribute of the `<input>` element, e.g., choosing the `email` type automatically creates a constraint that checks whether the value is a valid email address.
- By setting values on validation-related attributes, allowing basic constraints to be described without the need for JavaScript.

### Semantic input types

The intrinsic constraints for the `type` attribute are:

| Input type | Constraint description | Associated violation |
| --- | --- | --- |
| `<input type="URL">` | The value must be an absolute URL, as defined in the URL Living Standard. | **TypeMismatch** constraint violation |
| `<input type="email">` | The value must be a syntactically valid email address, which generally has the format `username@hostname.tld` but can also be local such as `username@hostname`. | **TypeMismatch** constraint violation |

For the `email` input type, if the `multiple` attribute is set, several values can be set as a comma-separated list. If any of the values in the list do not satisfy the condition described here, the **Type mismatch** constraint violation is triggered.

Note that most input types don't have intrinsic constraints, as some are barred from constraint validation or have a sanitization algorithm transforming incorrect values to a correct default.

### Validation-related attributes

In addition to the `type` attribute described above, the following attributes are used to describe basic constraints:

| Attribute | Input types supporting the attribute | Possible values | Constraint description | Associated violation |
|---|---|---|---|---|
| `pattern` | `text`, `search`, `url`, `tel`, `email`, `password` | A JavaScript regular expression (compiled with the `global`, `ignoreCase`, and `multiline` flags *disabled*) | The value must match the pattern. | **`patternMismatch`** constraint violation |
| `min` | `range`, `number` | A valid number | The value must be greater than or equal to the value. | **`rangeUnderflow`** constraint violation |
| | `date`, `month`, `week` | A valid date | | |
| | `datetime-local`, `time` | A valid date and time | | |
| `max` | `range`, `number` | A valid number | The value must be less than or equal to the value | **`rangeOverflow`** constraint violation |
| | `date`, `month`, `week` | A valid date | | |
| | `datetime-local`, `time` | A valid date and time | | |
| `required` | `text`, `search`, `url`, `tel`, `email`, `password`, `date`, `datetime-local`, `month`, `week`, `time`, `number`, `checkbox`, `radio`, `file`; also on the `<select>` and `<textarea>` elements | *none* as it is a Boolean attribute: its presence means *true*, its absence means *false* | There must be a value (if set). | **`valueMissing`** constraint violation |
| `step` | `date` | An integer number of days | Unless the step is set to the `any` literal, the value must be **min** + an integral multiple of the step. | **`stepMismatch`** constraint violation |
| | `month` | An integer number of months | | |
| | `week` | An integer number of weeks | | |
| | `datetime-local`, `time` | An integer number of seconds | | |
| | `range`, `number` | An integer | | |
| `minlength` | `text`, `search`, `url`, `tel`, `email`, `password`; also on the `<textarea>` element | An integer length | The number of characters (code points) must not be less than the value of the attribute, if non-empty. All newlines are normalized to a single character (as opposed to CRLF pairs) for `<textarea>`. | **`tooShort`** constraint violation |
| `maxlength` | `text`, `search`, `url`, `tel`, `email`, `password`; also on the `<textarea>` element | An integer length | The number of characters (code points) must not exceed the value of the attribute. | **`tooLong`** constraint violation |

### Constraint validation process

Constraint validation is done through the Constraint Validation API either on a single form element or at the form level, on the `<form>` element itself. The constraint validation is done in the following ways:

- By a call to the `checkValidity()` or `reportValidity()` method of a form-associated DOM interface, (`HTMLInputElement`, `HTMLSelectElement`, `HTMLButtonElement`, `HTMLOutputElement` or `HTMLTextAreaElement`), which evaluates the constraints only on this element, allowing a script to get this information. The `checkValidity()` method returns a Boolean indicating whether the element's value passes its constraints. (This is typically done by the user-agent when determining which of the CSS pseudo-classes, `:valid` or `:invalid`, applies.) In contrast, the `reportValidity()` method reports any constraint failures to the user.
- By a call to the `checkValidity()` or `reportValidity()` method on the `HTMLFormElement` interface.
- By submitting the form itself.

Calling `checkValidity()` is called *statically* validating the constraints, while calling `reportValidity()` or submitting the form is called *interactively* validating the constraints.

> **Note:**
>
> - If the `novalidate` attribute is set on the `<form>` element, interactive validation of the constraints doesn't happen.

- Calling the `submit()` method on the `HTMLFormElement` interface doesn't trigger a constraint validation. In other words, this method sends the form data to the server even if it doesn't satisfy the constraints. Call the `click()` method on a submit button instead.
- The `minlength` and `maxlength` constraints are only checked on user-provided input. They are not checked if a value is set programmatically, even when explicitly calling `checkValidity()` or `reportValidity()`.

### Complex constraints using the Constraint Validation API

Using JavaScript and the Constraint API, it is possible to implement more complex constraints, for example, constraints combining several fields, or constraints involving complex calculations.

Basically, the idea is to trigger JavaScript on some form field event (like **onchange**) to calculate whether the constraint is violated, and then to use the method `field.setCustomValidity()` to set the result of the validation: an empty string means the constraint is satisfied, and any other string means there is an error and this string is the error message to display to the user.

### Constraint combining several fields: Postal code validation

The postal code format varies from one country to another. Many countries allow an optional prefix with the country code (like `D-` in Germany, `F-` in France, and `CH-` in Switzerland). Some countries use only a fixed number of digits in postal codes, while others, like the UK, have more complex formats that allow letters at some specific positions.

> **Note:** This is not a comprehensive postal code validation library, but rather a demonstration of the key concepts.

As an example, we will add a script checking the constraint validation for a form:

```
<form>
  <label for="postal-code">Postal Code: </label>
  <input type="text" id="postal-code" />
  <label for="country">Country: </label>
  <select id="country">
    <option value="ch">Switzerland</option>
    <option value="fr">France</option>
    <option value="de">Germany</option>
    <option value="nl">The Netherlands</option>
  </select>
  <input type="submit" value="Validate" />
</form>
```

This displays the following form:

First, we write a function checking the constraint itself:

```
const countrySelect = document.getElementById("country");
const postalCodeField = document.getElementById("postal-code");

function checkPostalCode() {
  // For each country, defines the pattern that the postal code has to follow
  const constraints = {
    ch: [
      "^(CH-)?\\d{4}$",
      "Swiss postal codes must have exactly 4 digits: e.g. CH-1950 or 1950",
    ],
    fr: [
      "^(F-)?\\d{5}$",
      "French postal codes must have exactly 5 digits: e.g. F-75012 or 75012",
    ],
    de: [
      "^(D-)?\\d{5}$",
      "German postal codes must have exactly 5 digits: e.g. D-12345 or 12345",
    ],
    nl: [
      "^(NL-)?\\d{4}\\s*([A-RT-Z][A-Z]|S[BCE-RT-Z])$",
      "Dutch postal codes must have exactly 4 digits, followed by 2 letters except SA, SD and SS",
    ],
  };

  // Read the country id
  const country = countrySelect.value;

  // Build the constraint checker
  const constraint = new RegExp(constraints[country][0], "");
  console.log(constraint);
```

```
  // Check it!
  if (constraint.test(postalCodeField.value)) {
    // The postal code follows the constraint, we use the ConstraintAPI to tell it
    postalCodeField.setCustomValidity("");
  } else {
    // The postal code doesn't follow the constraint, we use the ConstraintAPI to
    // give a message about the format required for this country
    postalCodeField.setCustomValidity(constraints[country][1]);
  }
}
```

Then we link it to the `change` event for the `<select>` and the `input` event for the `<input>`:

```
countrySelect.addEventListener("change", checkPostalCode);
postalCodeField.addEventListener("input", checkPostalCode);
```

### Limiting the size of a file before its upload

Another common constraint is to limit the size of a file to be uploaded. Checking this on the client side before the file is transmitted to the server requires combining the Constraint Validation API, and especially the `field.setCustomValidity()` method, with another JavaScript API, here the File API.

Here is the HTML part:

```
<label for="fs">Select a file smaller than 75 kB: </label>
<input type="file" id="fs" />
```

This displays:

The JavaScript reads the file selected, uses the `File.size()` method to get its size, compares it to the (hard coded) limit, and calls the Constraint API to inform the browser if there is a violation:

```
const fs = document.getElementById("fs");

function checkFileSize() {
  const files = fs.files;

  // If there is (at least) one file selected
  if (files.length > 0) {
    if (files[0].size > 75 * 1000) {
      // Check the constraint
      fs.setCustomValidity("The selected file must not be larger than 75 kB");
      fs.reportValidity();
      return;
    }
  }
  // No custom constraint violation
  fs.setCustomValidity("");
}
```

Finally, we hook the method with the correct event:

```
fs.addEventListener("change", checkFileSize);
```

### Visual styling of constraint validation

Apart from setting constraints, web developers want to control what messages are displayed to the users and how they are styled.

### Controlling the look of elements

The look of elements can be controlled via CSS pseudo-classes.

### :required and :optional CSS pseudo-classes

The `:required` and `:optional` pseudo-classes allow writing selectors that match form elements that have the `required` attribute, or that don't have it.

### :placeholder-shown CSS pseudo-class

See `:placeholder-shown`.

### :valid :invalid CSS pseudo-classes

The `:valid` and `:invalid` pseudo-classes are used to represent <input> elements whose content validates and fails to validate respectively according to the input's type setting. These classes allow the user to style valid or invalid form elements to make it easier to identify elements that are either

formatted correctly or incorrectly.

### Controlling the text of constraint violation

The following items can help with controlling the text of a constraint violation:

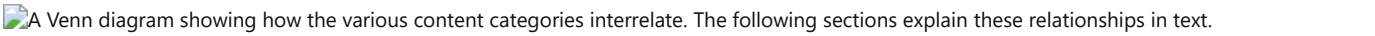
- The `setCustomValidity(message)` method on the following elements:
  - `<fieldset>`. Note: Setting a custom validity message on fieldset elements will not prevent form submission in most browsers.
  - `<input>`
  - `<output>`
  - `<select>`
  - Submit buttons (created with either a `<button>` element with the `submit` type, or an `input` element with the submit type). Other types of buttons do not participate in constraint validation.
  - `<textarea>`
- The `ValidityState` interface describes the object returned by the `validity` property of the element types listed above. It represents various ways that an entered value can be invalid. Together, they help explain why an element's value fails to validate, if it's not valid.

## Content categories

Most HTML elements are a member of one or more **content categories** — these categories group elements that share common characteristics. This is a loose grouping (it doesn't actually create a relationship among elements of these categories), but they help define and describe the categories' shared behavior and their associated rules. It's possible for elements to not be a member of *any* of these categories.

The content categories are used to define the *content model* of elements, in other words, what each element can take as descendants. For example, the `<p>` element can only contain *phrasing content*, while the `<div>` element can contain *flow content*. Some elements, like `<ins>`, have a *transparent* content model.

There are seven main content categories, which can be summarized with the Venn diagram below:


A Venn diagram showing how the various content categories interrelate. The following sections explain these relationships in text.

> **Note:** A more detailed discussion of these content categories and their comparative functionalities is beyond the scope of this article; for that, you may wish to read the relevant portions of the HTML specification.

### Metadata content

Elements belonging to the *metadata content* category modify the presentation or the behavior of the rest of the document, set up links to other documents, or convey other *out-of-band* information. Everything in the `<head>`, including the `<title>`, `<link>`, `<script>`, `<style>`, and the lesser used `<base>`, is metadata content. There is a `<meta>` element for metadata that cannot be represented by these other elements.

The metadata elements are:

- `<base>`
- `<link>`
- `<meta>`
- `<noscript>`
- `<script>`
- `<style>`
- `<template>`
- `<title>`

Some of these elements belong to more than one content category. For example, `<script>` is a member of the metadata, flow, and phrasing content categories, and is a script-supporting element; `<script>` can be used where metadata content, phrasing content, or script-supporting elements are expected.

### Flow content

Flow content is a broad category that encompasses most elements that can go inside the `<body>` element, including heading elements, sectioning elements, phrasing elements, embedding elements, interactive elements, and form-related elements. It also includes text nodes (but not those that only consist of white space characters).

The flow elements are:

- `<a>`
- `<abbr>`
- `<address>`
- `<article>`
- `<aside>`

- <audio>
- <b>
- <bdi>
- <bdo>
- <blockquote>
- <br>
- <button>
- <canvas>
- <cite>
- <code>
- <data>
- <datalist>
- <del>
- <details>
- <dfn>
- <dialog>
- 
- <dl>
- 
- <embed>
- <fieldset>
- <figure>
- <footer>
- <form>
- <geolocation>
- <h1>-<h6>
- <header>
- <hgroup>
- <hr>
- <i>
- <iframe>
- <img>
- <input>
- <ins>
- <kbd>
- <label>
- <main>
- <map>
- <mark>
- <math>
- <menu>
- <meter>
- <nav>
- <noscript>
- <object>
- <ol>
- <output>
- <p>
- <picture>
- <pre>
- <progress>
- <q>
- <ruby>
- <s>
- <samp>
- <script>
- <search>
- <section>
- <select>
- <slot>
- <small>
- 
- 
- <sub>
- <sup>
- <svg>

- `<table>`
- `<template>`
- `<textarea>`
- `<time>`
- `<u>`
- `<ul>`
- `<var>`
- `<video>`
- `<wbr>`
- Autonomous custom elements
- Plain text

A few other elements belong to this category, but only if a specific condition is fulfilled:

- `<area>`, if it is a descendant of a `<map>` element
- `<link>`, if the `itemprop` attribute is present
- `<meta>`, if the `itemprop` attribute is present

### Sectioning content

Sectioning content, a subset of flow content, creates a section in the current outline defining the scope of `<header>` and `<footer>` elements.

The sectioning elements are:

- `<article>`
- `<aside>`
- `<nav>`
- `<section>`

### Heading content

Heading content, a subset of flow content, defines the title of a section. This definition applies both to sections marked by an explicit sectioning content elements and to those implicitly defined by the heading content itself.

The heading elements are:

- `<h1>-<h6>`
- `<hgroup>`

> **Note:** Though likely to contain heading content, the `<header>` is not heading content itself.

### Phrasing content

Phrasing content, a subset of flow content, refers to the text and the markup within a document. Sequences of phrasing content make up paragraphs.

The phrasing elements are:

- `<abbr>`
- `<audio>`
- `<b>`
- `<bdi>`
- `<bdo>`
- `<br>`
- `<button>`
- `<canvas>`
- `<cite>`
- `<code>`
- `<data>`
- `<datalist>`
- `<dfn>`
- `<em>`
- `<embed>`
- `<i>`
- `<iframe>`
- `<img>`
- `<input>`
- `<kbd>`
- `<label>`

- `<mark>`
- `<math>`
- `<meter>`
- `<noscript>`
- `<object>`
- `<output>`
- `<picture>`
- `<progress>`
- `<q>`
- `<ruby>`
- `<s>`
- `<samp>`
- `<script>`
- `<select>`
- `<slot>`
- `<small>`
- `<span>`
- `<strong>`
- `<sub>`
- `<sup>`
- `<svg>`
- `<template>`
- `<textarea>`
- `<time>`
- `<u>`
- `<var>`
- `<video>`
- `<wbr>`
- Autonomous custom elements
- Plain text

A few other elements belong to this category, but only if a specific condition is fulfilled:

- `<a>`, if it contains only phrasing content
- `<area>`, if it is a descendant of a `<map>` element
- `<del>`, if it contains only phrasing content
- `<ins>`, if it contains only phrasing content
- `<link>`, if the `itemprop` attribute is present
- `<map>`, if it contains only phrasing content
- `<meta>`, if the `itemprop` attribute is present

### Embedded content

Embedded content, a subset of flow content, imports another resource or inserts content from another markup language or namespace into the document.

The embedded content elements are:

- `<audio>`
- `<canvas>`
- `<embed>`
- `<iframe>`
- `<img>`
- `<math>`
- `<object>`
- `<picture>`
- `<svg>`
- `<video>`

### Interactive content

Interactive content, a subset of flow content, includes elements that are specifically designed for user interaction.

The interactive content elements are:

- `<button>`
- `<details>`
- `<embed>`
- `<iframe>`

- `<label>`
- `<select>`
- `<textarea>`

Some elements belong to this category only under specific conditions:

- `<a>`, if the `href` attribute is present
- `<audio>`, if the `controls` attribute is present
- `<img>`, if the `usemap` attribute is present
- `<input>`, if the `type` attribute is not in the hidden state
- `<object>`, if the `usemap` attribute is present
- `<video>`, if the `controls` attribute is present

### Palpable content

**Palpable content** is content that is neither empty nor hidden; it is content that is rendered and substantive. Palpable content is not used to define content models but is used to define a general rule: Elements whose content model allows any flow content or phrasing content should have at least one node in its contents that is palpable content and that does not have the `hidden` attribute specified.

The palpable elements are:

- `<a>`
- `<abbr>`
- `<address>`
- `<article>`
- `<aside>`
- `<b>`
- `<bdi>`
- `<bdo>`
- `<blockquote>`
- `<button>`
- `<canvas>`
- `<cite>`
- `<code>`
- `<data>`
- `<del>`
- `<details>`
- `<dfn>`
- `<div>`
- `<em>`
- `<embed>`
- `<fieldset>`
- `<footer>`
- `<figure>`
- `<form>`
- `<iframe>`
- `<img>`
- `<ins>`
- `<kbd>`
- `<label>`
- `<main>`
- `<map>`
- `<mark>`
- `<math>`
- `<meter>`
- `<nav>`
- `<object>`
- `<p>`
- `<picture>`
- `<pre>`
- `<progress>`
- `<q>`
- `<ruby>`
- `<s>`
- `<samp>`
- `<search>`
- `<section>`
- `<select>`

- `<small>`
- `<span>`
- `<strong>`
- `<sub>`
- `<sup>`
- `<svg>`
- `<table>`
- `<textarea>`
- `<time>`
- `<u>`
- `<var>`
- `<video>`
- Autonomous custom elements
- Plain text that is not inter-element whitespace

Some elements belong to this category only under specific conditions:

- `<audio>`, if the `controls` attribute is present
- `<dl>`, if the element's children include at least one name-value group
- `<input>`, if the type attribute is not in the hidden state
- `<ol>`, if its children include at least one `<li>` element
- `<ul>`, if its children include at least one `<li>` element

### Elements without a category

Several elements are not a member of *any* content category. These include:

- `<caption>`
- `<col>`
- `<colgroup>`
- `<dd>`
- `<dt>`
- `<figcaption>`
- `<head>`
- `<html>`
- `<legend>`
- `<li>`
- `<optgroup>`
- `<option>`
- `<param>`
- `<rb>`
- `<rp>`
- `<rt>`
- `<rtc>`
- `<source>`
- `<tbody>`
- `<tfoot>`
- `<th>`
- `<thead>`
- `<tr>`
- `<track>`

### Script-supporting elements

**Script-supporting elements** are elements that don't directly contribute to a document's rendered output. Instead, they serve to support scripts, either by containing or specifying script code directly or by specifying data that will be used by scripts. Nearly all elements, including those that only take specific elements (such as `<ul>`, which takes `<li>` elements), can contain script-supporting elements.

The script-supporting elements are:

- `<script>`
- `<template>`

### Form-associated content

Form-associated content is a subset of flow content comprising elements that have a form owner and can be used everywhere flow content is expected. A form owner is either the containing `<form>` element or the `<form>` whose `id` is specified in the element's `form` attribute.

The form-associated elements are:

- `<button>`
- `<fieldset>`
- `<input>`
- `<object>`
- `<output>`
- `<select>`
- `<textarea>`
- `<img>`

This category contains several sub-categories:

listed

Elements that are listed in the `HTMLFormElement.elements` and `HTMLFieldSetElement.elements` collections. Includes `<button>`, `<fieldset>`, `<input>`, `<object>`, `<output>`, `<select>`, and `<textarea>`.

submittable

Elements that can be used for constructing the form data set when the form is submitted. Includes `<button>`, `<input>`, `<select>`, and `<textarea>`.

resettable

Elements that can be affected when a form is reset. Includes `<input>`, `<output>`, `<select>`, and `<textarea>`.

autocapitalize-and-autocorrect-inheriting

Elements that inherit the `autocapitalize` and `autocorrect` attributes from their form owner. Includes `<button>`, `<fieldset>`, `<input>`, `<output>`, `<select>`, and `<textarea>`.

labelable

Elements that can be associated with `<label>` elements. Includes `<button>`, `<input>` (all types other than `hidden`), `<meter>`, `<output>`, `<progress>`, `<select>`, and `<textarea>`.

### Transparent content model

In addition to the listed content categories, an element's content model may also be defined as "transparent". If an element X's permitted content is "transparent", then we look at the parent of X. We intersect the permitted content of X's parent with the content categories of X, and the result is what "transparent" means in this context. If the parent of X also has a transparent content model, then we continue up the tree until we find a non-transparent content model. When there's no such parent, "transparent" means "flow content".

For example, a `<ruby>` element accepts phrasing content. The `<ins>` element is of category phrasing content when it contains only phrasing content. Therefore, an `<ins>` element can be placed inside a `<ruby>` element. The `<ins>` element's permitted content is "transparent", which when nested in `<ruby>` means "phrasing content". However, `<rt>` elements are not phrasing content. Therefore, an `<rt>` element cannot be nested inside this `<ins>` element, even though both `<rt>` and `<ins>` can be inside `<ruby>`, and `<ins>` is "transparent".

```
<ruby>
  Text before
  <ins>
    <!-- Invalid: rt cannot be placed inside ins here -->
    <rt>Pronunciation</rt>
  </ins>
</ruby>
```

```
<ruby>
  Text before
  <!-- Valid: ins can be inside ruby, and rt can be inside ruby -->
  <ins>Inserted text</ins>
  <rt>Pronunciation</rt>
</ruby>
```

```
<ruby>
  Text before
  <!-- Valid: rt can be inside ruby, and ins can be inside rt -->
  <rt><ins>Pronunciation</ins></rt>
</ruby>
```

Transparent is a *content model*, not a *content category*, so it only defines what an element can contain, not where the element can be placed. That is, when determining an element's children's licitness, you cannot "see through" transparent children. For example, a `<ul>` element accepts only `<li>` elements and script-supporting elements, and does not allow `<del>` or `<ins>`, even if the `<del>` only contains `<li>` elements.

```
<ul>
  <del>
```

```
    <li>Oranges</li>
    <li>Toilet paper</li>
  </del>
  <li>Toothpaste</li>
</ul>
```

```
<ul>
  <li><del>Oranges</del></li>
  <li><del>Toilet paper</del></li>
  <li>Toothpaste</li>
</ul>
```

## Using date and time formats in HTML

Certain HTML elements use date and/or time values. The formats of the strings that specify these values are described in this article.

Elements that use such formats include certain forms of the `<input>` element that let the user choose or specify a date, time, or both, as well as the `<ins>` and `<del>` elements, whose `datetime` attribute specifies the date or date and time at which the insertion or deletion of content occurred.

For `<input>`, the `type` values of inputs whose `value` contains a string representing a date and/or time are:

- `date`
- `datetime-local`
- `month`
- `time`
- `week`

### Examples

Before getting into the intricacies of how date and time strings are written and parsed in HTML, here are some examples that should give you a good idea what the more commonly-used date and time string formats look like.

Example HTML date and time strings

| String | Date and/or time | |
|---|---|---|
| `2005-06-07` | June 7, 2005 | [details] |
| `08:45` | 8:45 AM | [details] |
| `08:45:25` | 8:45 AM and 25 seconds | [details] |
| `0033-08-04T03:40` | 3:40 AM on August 4, 33 | [details] |
| `1977-04-01T14:00:30` | 30 seconds after 2:00 PM on April 1, 1977 | [details] |
| `1901-01-01T00:00Z` | Midnight UTC on January 1, 1901 | [details] |
| `1901-01-01T00:00:01-04:00` | 1 second past midnight Eastern Standard Time (EST) on January 1, 1901 | [details] |

### Basics

Before looking at the various formats of date and time related strings used by HTML elements, it is helpful to understand a few fundamental facts about the way they're defined. HTML uses a variation of the ISO 8601 standard for its date and time strings. It's worth reviewing the descriptions of the formats you're using in order to ensure that your strings are in fact compatible with HTML, as the HTML specification includes algorithms for parsing these strings that is actually more precise than ISO 8601, so there can be subtle differences in how date and time strings are expected to look.

### Character set

Dates and times in HTML are always strings which use the ASCII character set.

### Year numbers

In order to simplify the basic format used for date strings in HTML, the specification requires that all years be given using the modern (or **proleptic**) Gregorian calendar. While user interfaces may allow entry of dates using other calendars, the underlying value always uses the Gregorian calendar.

While the Gregorian calendar wasn't created until the year 1582 (replacing the similar Julian calendar), for HTML's purposes, the Gregorian calendar is extended back to the year 1 C.E. Make sure any older dates account for this.

For the purposes of HTML dates, years are always at least four digits long; years prior to the year 1000 are padded with leading zeroes (`0`), so the year 72 is written as `0072`. Years prior to the year 1 C.E. are not supported, so HTML doesn't support years 1 B.C.E. (1 B.C.) or earlier.

A year is normally 365 days long, except during **leap years**.

### Leap years

A **leap year** is any year which is divisible by 400 *or* the year is divisible by 4 but not by 100. Although the calendar year is normally 365 days long, it actually takes the planet Earth approximately 365.2422 days to complete a single orbit around the sun. Leap years help to adjust the calendar to keep it synchronized with the actual position of the planet in its orbit. Adding a day to the year every four years essentially makes the average year 365.25 days long, which is close to correct.

The adjustments to the algorithm (taking a leap year when the year can be divided by 400, and skipping leap years when the year is divisible by 100) help to bring the average even closer to the correct number of days (365.2425 days). Scientists occasionally add leap seconds to the calendar (seriously) to handle the remaining three ten-thousandths of a day and to compensate for the gradual, naturally occurring slowing of Earth's rotation.

While month `02`, February, normally has 28 days, it has 29 days in leap years.

### Months of the year

There are 12 months in the year, numbered 1 through 12. They are always represented by a two-digit ASCII string whose value ranges from `01` through `12`. See the table in the section Days of the month for the month numbers and their corresponding names (and lengths in days).

### Days of the month

Month numbers 1, 3, 5, 7, 8, 10, and 12 are 31 days long. Months 4, 6, 9, and 11 are 30 days long. Month 2, February, is 28 days long most years, but is 29 days long in leap years. This is detailed in the following table.

The months of the year and their lengths in days

| Month number | Name (English) | Length in days |
|---|---|---|
| **01** | January | 31 |
| **02** | February | 28 (29 in leap years) |
| **03** | March | 31 |
| **04** | April | 30 |
| **05** | May | 31 |
| **06** | June | 30 |
| **07** | July | 31 |
| **08** | August | 31 |
| **09** | September | 30 |
| **10** | October | 31 |
| **11** | November | 30 |
| **12** | December | 31 |

### Week strings

A week string specifies a week within a particular year. A **valid week string** consists of a valid year number, followed by a hyphen character (`-`, or U+002D), then the capital letter `W` (U+0057), followed by a two-digit week of the year value.

The week of the year is a two-digit string between `01` and `53`. Each week begins on Monday and ends on Sunday. That means it's possible for the first few days of January to be considered part of the previous week-year, and for the last few days of December to be considered part of the following week-year. The first week of the year is the week that contains the *first Thursday of the year*. For example, the first Thursday of 1953 was on January 1, so that week—beginning on Monday, December 29—is considered the first week of the year. Therefore, December 30, 1952 occurs during the week `1953-W01`.

A year has 53 weeks if:

- The first day of the calendar year (January 1) is a Thursday **or**
- The first day of the year (January 1) is a Wednesday and the year is a leap year

All other years have 52 weeks.

| Week string | Week and year (Date range) |
|---|---|
| `2001-W37` | Week 37, 2001 (September 10-16, 2001) |
| `1953-W01` | Week 1, 1953 (December 29, 1952-January 4, 1953) |
| `1948-W53` | Week 53, 1948 (December 27, 1948-January 2, 1949) |
| `1949-W01` | Week 1, 1949 (January 3-9, 1949) |
| `0531-W16` | Week 16, 531 (April 13-19, 531) |
| `0042-W04` | Week 4, 42 (January 21-27, 42) |

Note that both the year and week numbers are padded with leading zeroes, with the year padded to four digits and the week to two.

### Month strings

A month string represents a specific month in time, rather than a generic month of the year. That is, rather than representing "January," an HTML month string represents a month and year paired, like "January 1972."

A **valid month string** consists of a valid year number (a string of at least four digits), followed by a hyphen character (-, or U+002D), followed by a two-digit numeric month number, where `01` represents January and `12` represents December.

| Month string | Month and year |
|---|---|
| `17310-09` | September, 17310 |
| `2019-01` | January, 2019 |
| `1993-11` | November, 1993 |
| `0571-04` | April, 571 |
| `0001-07` | July, 1 C.E. |

Notice that all years are at least four characters long; years that are fewer than four digits long are padded with leading zeroes.

### Date strings

A valid date string consists of a month string, followed by a hyphen character (-, or U+002D), followed by a two-digit day of the month.

| Date string | Full date |
|---|---|
| `1993-11-01` | November 1, 1993 |
| `1066-10-14` | October 14, 1066 |
| `0571-04-22` | April 22, 571 |
| `0062-02-05` | February 5, 62 |

### Time strings

A time string can specify a time with precision to the minute, second, or to the millisecond. Specifying only the hour or minute isn't permitted. A **valid time string** minimally consists of a two-digit hour followed by a colon (`:`, U+003A), then a two-digit minute. The minute may optionally be followed by another colon and a two-digit number of seconds. Milliseconds may be specified, optionally, by adding a decimal point character (`.`, U+002E) followed by one, two, or three digits.

There are some additional basic rules:

- The hour is always specified using the 24-hour clock, with `00` being midnight and 11 PM being `23`. No values outside the range `00` – `23` are permitted.
- The minute must be a two-digit number between `00` and `59`. No values outside that range are allowed.
- If the number of seconds is omitted (to specify a time accurate only to the minute), no colon should follow the number of minutes.
- If specified, the integer portion of the number of seconds must be between `00` and `59`. You *cannot* specify leap seconds by using values like `60` or `61`.
- If the number of seconds is specified and is an integer, it must not be followed by a decimal point.
- If a fraction of a second is included, it may be from one to three digits long, indicating the number of milliseconds. It follows the decimal point placed after the seconds component of the time string.

| Time string | Time |
|---|---|
| `00:00:30.75` | 12:00:30.75 AM (30.75 seconds after midnight) |
| `12:15` | 12:15 PM |
| `13:44:25` | 1:44:25 PM (25 seconds after 1:44 PM) |

### Local date and time strings

A valid `datetime-local` string consists of a `date` string and a `time` string concatenated together with either the letter `T` or a space character separating them. No information about the time zone is included in the string; the date and time is presumed to be in the user's local time zone.

When you set the `value` of a `datetime-local` input, the string is **normalized** into a standard form. Normalized `datetime` strings always use the letter `T` to separate the date and the time, and the time portion of the string is as short as possible. This is done by leaving out the seconds component if its value is `:00`.

Examples of valid `datetime-local` strings

| Date/time string | Normalized date/time string | Actual date and time |
|---|---|---|
| `1986-01-28T11:38:00.01` | `1986-01-28T11:38:00.01` | January 28, 1986 at 11:38:00.01 AM |

| Date/time string | Normalized date/time string | Actual date and time |
| --- | --- | --- |
| `1986-01-28 11:38:00.010` | `1986-01-28T11:38:00.01`<br><br>Note that after normalization, this is the same string as the previous `datetime-local` string. The space has been replaced with the `T` character and the trailing zero in the fraction of a second has been removed to make the string as short as possible. | January 28, 1986 at 11:38:00.01 AM |
| `0170-07-31T22:00:00` | `0170-07-31T22:00`<br><br>Note that the normalized form of this date drops the `:00` indicating the number of seconds to be zero, because the seconds are optional when zero, and the normalized string minimizes the length of the string. | July 31, 170 at 10:00 PM |

### Global date and time strings

A global date and time string specifies a date and time as well as the time zone in which it occurs. A **valid global date and time string** is the same format as a local date and time string, except it has a time zone string appended to the end, following the time.

### Time zone offset string

A time zone offset string specifies the offset in either a positive or a negative number of hours and minutes from the standard time base. There are two standard time bases, which are very close to the same, but not exactly the same:

- For dates after the establishment of Coordinated Universal Time (UTC) in the early 1960s, the time base is `Z` and the offset indicates a particular time zone's offset from the time at the prime meridian at 0º longitude (which passes through the Royal Observatory at Greenwich, England).
- For dates prior to UTC, the time base is instead expressed in terms of UT1, which is the contemporary Earth solar time at the prime meridian.

The time zone string is appended immediately following the time in the date and time string. You can specify `Z` as the time zone offset string to indicate that the time is specified in UTC. Otherwise, the time zone string is constructed as follows:

1. A character indicating the sign of the offset: the plus character (`+`, or U+002B) for time zones to the east of the prime meridian or the minus character (`-`, or U+002D) for time zones to the west of the prime meridian.
2. A two-digit number of hours that the time zone is offset from the prime meridian. This value must be between `00` and `23`.
3. An optional colon (`:`) character.
4. A two-digit number of minutes past the hour; this value must be between `00` and `59`.

While this format allows for time zones between -23:59 and +23:59, the current range of time zone offsets is -12:00 to +14:00, and no time zones are currently offset from the hour by anything other than `00`, `30`, or `45` minutes. This may change at more or less anytime, since countries are free to tamper with their time zones at any time and in any way they wish to do so.

Examples of valid global date and time strings

| Global date and time string | Actual global date and time | Date and time at prime meridian |
| --- | --- | --- |
| `2005-06-07T00:00Z` | June 7, 2005 at midnight UTC | June 7, 2005 at midnight |
| `1789-08-22T12:30:00.1-04:00` | August 22, 1789 at a tenth of a second past 12:30 PM Eastern Daylight Time (EDT) | August 22, 1789 at a tenth of a second past 4:30 PM |
| `3755-01-01 00:00+10:00` | January 1, 3755 at midnight Australian Eastern Standard Time (AEST) | December 31, 3754 at 2:00 PM |

### Date issues

Because of data storage and precision issues, you may want to be aware of a few client-side and server-side issues.

### The Y2K38 Problem (often server-side)

JavaScript uses double precision floating points to store dates, as with all numbers, meaning that JavaScript code will not suffer from the Y2K38 problem unless integer coercion/bit-hacks are used because all JavaScript bit operators use 32-bit signed 2s-complement integers.

The problem is with the server side of things: storage of dates greater than 2^31 - 1. To fix this problem, you must store all dates using either unsigned 32-bit integers, signed 64-bit integers, or double-precision floating points on the server. If your server is written in PHP, the fix may require upgrading your PHP to a more recent version, and upgrading your hardware to x86_64 or IA64. If you are stuck with other hardware, you can try to emulate 64-bit hardware inside a 32-bit virtual machine, but most VMs don't support this kind of virtualization, since stability may suffer, and performance will definitely suffer greatly.

### The Y10k Problem (often client-side)

In many servers, dates are stored as numbers instead of as strings--numbers of a fixed size and agnostic of format (aside from endianness). After the year 10,000, those numbers will just be a bit bigger than before, so many servers will not see issues with forms submitted after the year 10,000.

The problem is with the client side of things: parsing of dates with more than 4 digits in the year.

```
<!--midnight of January 1st, 10000: the exact time of Y10K-->
<input type="datetime-local" value="+010000-01-01T05:00" />
```

We need to prepare our code for any number of digits — not just 5. The following JavaScript function programmatically sets the value:

```
function setValue(element, date) {
  const isoString = date.toISOString();
  element.value = isoString.substring(0, isoString.indexOf("T") + 6);
}
```

Why worry about the Y10K problem if it is going to happen many centuries after your death? Exactly because you will already be dead, so the companies using your software will be stuck using your software without any other coder who knows the system well enough to come in and fix it.

### See also

- `<input>`
- `<ins>` and `<del>`: see the `datetime` attribute, which specifies either a date or a local date and time at which the content was inserted or deleted
- The ISO 8601 specification
- Representing dates & times in the JavaScript Guide
- The JavaScript `Date` object
- The `Intl.DateTimeFormat` object for formatting dates and times for a given locale

## Using microdata in HTML

Microdata is part of the WHATWG HTML Standard and is used to nest metadata within existing content on web pages. Search engines and web crawlers can extract and process microdata from a web page and use it to provide a richer browsing experience for users. Search engines benefit greatly from direct access to this structured data because it allows search engines to understand the information on web pages and provide more relevant results to users. Microdata uses a supporting vocabulary to describe an item and name-value pairs to assign values to its properties. Microdata is an attempt to provide a declarative way of annotating HTML elements with machine-readable tags than the similar approaches of using RDFa and classic microformats.

At a high level, microdata consists of a group of name-value pairs. The groups are called items, and each name-value pair is a property. Items and properties are represented by regular elements.

- To create an item, the `itemscope` attribute is used.
- To add a property to an item, the `itemprop` attribute is used on one of the item's descendants.

### Vocabularies

Google and other major search engines support the Schema.org vocabulary for structured data. This vocabulary defines a standard set of type names and property names, for example, Schema.org Music Event indicates a concert performance, with `startDate` and `location` properties to specify the concert's key details. In this case, Schema.org Music Event would be the URL used by `itemtype` and `startDate` and location would be `itemprop`'s that Schema.org Music Event defines.

> **Note:** More about itemtype attributes can be found at https://schema.org/Thing.

Microdata vocabularies provide the semantics or meaning of an *`Item`*. Web developers can design a custom vocabulary or use vocabularies available on the web, such as the widely used schema.org vocabulary. A collection of commonly used markup vocabularies are provided by Schema.org.

Commonly used vocabularies:

- Creative works: CreativeWork, Book, Movie, MusicRecording, Recipe, TVSeries
- Embedded non-text objects: AudioObject, ImageObject, VideoObject
- `Event`
- Health and medical types: Notes on the health and medical types under MedicalEntity
- `Organization`
- `Person`
- `Place`, LocalBusiness, Restaurant
- `Product`, Offer, AggregateOffer
- `Review`, AggregateRating
- `Action`
- `Thing`
- `Intangible`

Major search engine operators like Google, Microsoft, and Yahoo! rely on the schema.org vocabulary to improve search results. For some purposes, an ad hoc vocabulary is adequate. For others, a vocabulary will need to be designed. Where possible, authors are encouraged to re-use existing vocabularies, as this makes content re-use easier.

### Localization

In some cases, search engines covering specific regions may provide locally-specific extensions of microdata. For example, Yandex, a major search engine in Russia, supports microformats such as `hCard` (company contact information), `hRecipe` (food recipe), `hReview` (market reviews) and `hProduct` (product data) and provides its own format for the definition of the terms and encyclopedic articles. This extension was made to solve transliteration problems between the Cyrillic and Latin alphabets. Due to the implementation of additional marking parameters of Schema's vocabulary, the indexation of information in Russian-language web-pages became considerably more successful.

### Global attributes

`itemid` – The unique, global identifier of an item.

`itemprop` – Used to add properties to an item. Every HTML element may have an `itemprop` attribute specified, where an `itemprop` consists of a name and value pair.

`itemref` – Properties that are not descendants of an element with the `itemscope` attribute can be associated with the item using an **itemref**. `itemref` provides a list of element ids (not `itemids`) with additional properties elsewhere in the document.

`itemscope` – The `itemscope` attribute (usually) works along with `itemtype` to specify that the HTML contained in a block is about a particular item. The `itemscope` attribute creates the *`Item`* and defines the scope of the itemtype associated with it. The `itemtype` attribute is a valid URL of a vocabulary (such as schema.org) that describes the item and its properties context.

`itemtype` – Specifies the URL of the vocabulary that will be used to define `itemprop`'s (item properties) in the data structure. The `itemscope` attribute is used to set the scope of where in the data structure the vocabulary set by `itemtype` will be active.

### Example

### HTML

```
<div itemscope itemtype="https://schema.org/SoftwareApplication">
  <span itemprop="name">Angry Birds</span> - REQUIRES
  <span itemprop="operatingSystem">ANDROID</span><br />
  <link
    itemprop="applicationCategory"
    href="https://schema.org/SoftwareApplication" />

  <div
    itemprop="aggregateRating"
    itemscope
    itemtype="https://schema.org/AggregateRating">
    RATING:
    <span itemprop="ratingValue">4.6</span> (
    <span itemprop="ratingCount">8864</span> ratings )
  </div>

  <div itemprop="offers" itemscope itemtype="https://schema.org/Offer">
    Price: $<span itemprop="price">1.00</span>
    <meta itemprop="priceCurrency" content="USD" />
  </div>
</div>
```

### Structured data

| | | | |
|---|---|---|---|
| itemscope | itemtype | SoftwareApplication (https://schema.org/SoftwareApplication) | |
| | itemprop | name | Angry Birds |
| | itemprop | operatingSystem | ANDROID |
| | itemprop | applicationCategory | SoftwareApplication (https://schema.org/SoftwareApplication) |
| itemscope | itemprop[itemtype] | aggregateRating [AggregateRating] | |
| | itemprop | ratingValue | 4.6 |
| | itemprop | ratingCount | 8864 |
| itemscope | itemprop[itemtype] | offers [Offer] | |
| | itemprop | price | 1.00 |
| | itemprop | priceCurrency | USD |

### Result

> **Note:** A handy tool for extracting and verifying microdata structures from HTML is the Schema Markup Validator. Try it on the HTML shown above.

### See also

- Global Attributes

## Using microformats in HTML

*Microformats* are standards used to embed semantics and structured data in HTML, and provide an API to be used by social web applications, search engines, aggregators, and other tools. These minimal patterns of HTML are used for marking up entities that range from fundamental to domain-specific information, such as people, organizations, events, and locations.

- To create a microformats object, `h-*` class names are used in the class attribute.
- To add a property to an object, the `p-*`, `u-*`, `dt-*`, `e-*` class names are used on one of the object's descendants.

Microformats use supporting vocabularies to describe objects and name-value pairs to assign values to their properties. The properties are carried in class attributes that can be added to any HTML element, while the data values re-use HTML element content and semantic attributes.

Microformats2 (sometimes referred to as mf2) is an update to microformats that provides a method of annotating HTML structured syntax and vocabularies than previous approaches of using RDFa and microdata. These previous approaches require learning new attributes.

There are open source parsing libraries for most languages for microformats2.

### How Microformats Work

An author of a webpage can add microformats to their HTML. For example if they wanted to identify themselves they could use an h-card such as:

### HTML Example

```
<a class="h-card" href="https://alice.example.com">Alice Blogger</a>
```

When a parser encounters this data, it will know that this page contains a "card" which describes a person or organization named `Alice Blogger` with a URL of `https://alice.example.com/`. The parser makes this data available via APIs that can be used for different applications. For example, an application could scan a page for a h-card to use as profile information for someone who has signed up to a service.

As in this example, some markup patterns require only a single microformat root class name, which parsers use to find a few generic properties such as `name`, `url`, `photo`.

### Microformats Use Cases

Microformats have numerous use cases. First, the Webmention standard uses microformats to provide a way in which messages and comments can be sent from one site to another. The Webmention specification defines specific attributes that sites may publish and consume to create a rich, interoperable way of publishing messages and comments. Microformats can also be used with Webmentions to enable sending social reactions such as likes, reposts, and bookmarks from one site to another.

Microformats also enable easy syndication across sites. An aggregator could parse a page with published microformats to look for information such as a post title, a post body, and the author of a post. This aggregator could then use the semantic information gathered to render a result on their site. For instance, news aggregators and community posting boards could facilitate submissions and use microformats to exact relevant content from a page. Further, a website could use microformats to send crafted requests to third-parties to publish content, such as social networks.

All major search engines support reading and interpreting microformats. Search engines benefit greatly from direct access to this structured data because it allows them to understand the information on web pages. With this information, search engines can provide more relevant results to users. Some search engines may render special snippets such as star ratings on a search result page based on the data provided in microformats.

In addition to being machine-readable, microformats are designed to be easily read by humans. This approach makes it easy for people to understand and maintain microformats data.

### Microformats Prefixes

All microformats consist of a root, and a collection of properties. Properties are all optional and potentially multivalued - applications needing a singular value may use the first instance of a property. Hierarchical data is represented with nested microformats, typically as property values themselves.

All microformats class names use prefixes. Prefixes are **syntax independent of vocabularies**, which are developed separately.

- **"h-*" for root class names**, e.g., "h-card", "h-entry", "h-feed", and many more. These top-level root classes usually indicate a type and corresponding expected vocabulary of properties. For example:
    - h-card describes a person or organization
    - h-entry describes episodic or date stamped online content like a blog post

    - h-feed describes a stream or feed of posts
    - You can find many more vocabularies on the microformats2 wiki.

- **"p-*" for plain (text) properties**, e.g., "p-name", "p-summary"
    - Generic plain text parsing, element text in general. On certain HTML elements, use special attributes first, e.g., img/alt, abbr/title.
- **"u-*" for URL properties**, e.g., "u-url", "u-photo", "u-logo"
    - Special parsing: element attributes a/href, img/src, object/data etc. attributes over element contents.
- **"dt-*" for datetime properties**, e.g., "dt-start", "dt-end", "dt-bday"
    - Special parsing: time element datetime attribute, value-class-pattern and separate date time value parsing for readability.
- **"e-*" for element tree properties** where the entire contained element hierarchy is the value, e.g., "e-content". The "e-" prefix can also be mnemonically remembered as "element tree", "embedded markup", or "encapsulated markup".

### Some microformats examples

### h-card

The h-card microformat represents a person or organization.

The value of each property is defined in HTML using the class property any element can carry

### Example h-card

```
<p class="h-card">
  <img class="u-photo" src="https://example.org/photo.png" alt="" />
  <a class="p-name u-url" href="https://example.org">Joe Bloggs</a>
  <a class="u-email" href="mailto:jbloggs@example.com">jbloggs@example.com</a>,
  <span class="p-street-address">17 Austerstræti</span>
  <span class="p-locality">Reykjavík</span>
  <span class="p-country-name">Iceland</span>
</p>
```

| Property | Description |
|---|---|
| **`p-name`** | The full/formatted name of the person or organization. |
| **`u-email`** | email address |
| **`u-photo`** | a photo of the person or organization |
| **`u-url`** | home page or other URL representing the person or organization |
| **`u-uid`** | universally unique identifier, preferably canonical URL |
| **`p-street-address`** | street number + name |
| **`p-locality`** | city/town/village |
| **`p-country-name`** | country name |

### Nested h-card example

```
<div class="h-card">
  <a class="p-name u-url" href="https://blog.lizardwrangler.com/">
    Mitchell Baker
  </a>
  (<a class="p-org h-card" href="https://mozilla.org/">Mozilla Foundation</a>)
</div>
```

Parsed JSON:

```
{
  "items": [
    {
      "type": ["h-card"],
      "properties": {
        "name": ["Mitchell Baker"],
        "url": ["https://blog.lizardwrangler.com/"],
        "org": [
          {
            "value": "Mozilla Foundation",
            "type": ["h-card"],
            "properties": {
              "name": ["Mozilla Foundation"],
```

```
            "url": ["https://mozilla.org/"]
          }
        }
      ]
    }
  }
 ]
}
```

In this example, a h-card is specified for both a person and the organization they represent. The person's affiliation with the linked organization is specified using the p-org property.

Note: the nested h-card has implied 'name' and 'url' properties, just like any other root-class-name-only h-card on an `<a href>` would.

### h-entry

The h-entry microformat represents episodic or datestamped content on the web. h-entry is often used with content intended to be syndicated, e.g., blog posts and short notes.

Example h-entry as a blog post:

```
<article class="h-entry">
  <h1 class="p-name">Microformats are amazing</h1>
  <p>
    Published by
    <a class="p-author h-card" href="https://example.com">W. Developer</a> on
    <time class="dt-published" datetime="2013-06-13 12:00:00">
      13<sup>th</sup> June 2013
    </time>
  </p>

  <p class="p-summary">In which I extoll the virtues of using microformats.</p>

  <div class="e-content">
    <p>Blah blah blah</p>
  </div>
</article>
```

### Properties

| Property | Description |
| --- | --- |
| **`p-name`** | entry name/title |
| **`p-author`** | who wrote the entry, optionally embedded h-card |
| **`dt-published`** | when the entry was published |
| **`p-summary`** | short entry summary |
| **`e-content`** | full content of the entry |

### Parsed reply h-entry example

```
<div class="h-entry">
  <p>
    <span class="p-author h-card">
      <a href="https://quickthoughts.jgregorymcverry.com/profile/jgmac1106">
        <img
          class="u-photo"
          alt="Greg McVerry"
          src="https://quickthoughts.jgregorymcverry.com/file/2d6c9cfed7ac8e849f492b5bc7e6a630/thumb.jpg" />
      </a>
      <a
        class="p-name u-url"
        href="https://quickthoughts.jgregorymcverry.com/profile/jgmac1106">
        Greg McVerry
      </a>
    </span>
    Replied to
    <a
      class="u-in-reply-to"
      href="https://developer.mozilla.org/en-US/docs/Web/HTML/Guides/Microformats">
      a post on <strong>developer.mozilla.org</strong>
    </a>
    :
  </p>
  <p class="p-name e-content">
```

```
    Hey thanks for making this microformats resource
  </p>
  <p>
    <a href="https://quickthoughts.jgregorymcverry.com/profile/jgmac1106">
      Greg McVerry
    </a>
    published this
    <a
      class="u-url url"
      href="https://quickthoughts.jgregorymcverry.com/2019/05/31/hey-thanks-for-making-this-microformats-resource">
      <time class="dt-published" datetime="2019-05-31T14:19:09+0000">
        31 May 2019
      </time>
    </a>
  </p>
</div>
```

```
{
  "items": [
    {
      "type": ["h-entry"],
      "properties": {
        "in-reply-to": [
          "https://developer.mozilla.org/en-US/docs/Web/HTML/Guides/Microformats"
        ],
        "name": ["Hey thanks for making this microformats resource"],
        "url": [
          "https://quickthoughts.jgregorymcverry.com/2019/05/31/hey-thanks-for-making-this-microformats-resource"
        ],
        "published": ["2019-05-31T14:19:09+0000"],
        "content": [
          {
            "html": "Hey thanks for making this microformats resource",
            "value": "Hey thanks for making this microformats resource",
            "lang": "en"
          }
        ],
        "author": [
          {
            "type": ["h-card"],
            "properties": {
              "name": ["Greg McVerry"],
              "photo": [
                "https://quickthoughts.jgregorymcverry.com/file/2d6c9cfed7ac8e849f492b5bc7e6a630/thumb.jpg"
              ],
              "url": [
                "https://quickthoughts.jgregorymcverry.com/profile/jgmac1106"
              ]
            },
            "lang": "en",
            "value": "Greg McVerry"
          }
        ]
      },
      "lang": "en"
    }
  ]
}
```

### h-feed

The h-feed is a stream or feed of h-entry posts, like complete posts on a home page or archive pages, or summaries or other brief lists of posts.

### Example h-feed

```
<div class="h-feed">
  <h1 class="p-name">Microformats Blogs</h1>
  <article class="h-entry">
    <h2 class="p-name">Microformats are amazing</h2>
    <p>
      Published by
      <a class="p-author h-card" href="https://example.com">W. Developer</a> on
      <time class="dt-published" datetime="2013-06-13 12:00:00">
        13<sup>th</sup> June 2013
      </time>
    </p>
    <p class="p-summary">
```

```
      In which I extoll the virtues of using microformats.
    </p>
    <div class="e-content"><p>Blah blah blah</p></div>
  </article>
</div>
```

### Properties

| Property | Description |
|---|---|
| **`p-name`** | name of the feed |
| **`p-author`** | author of the feed, optionally embed an h-card |

### Children

| Nested h-entry | |
|---|---|
| objects representing the items of the feed | |

### h-event

The `h-event` is for events on the web. h-event is often used with both event listings and individual event pages.

```
<div class="h-event">
  <h1 class="p-name">Microformats Meetup</h1>
  <p>
    From
    <time class="dt-start" datetime="2013-06-30 12:00">
      30<sup>th</sup> June 2013, 12:00
    </time>
    to <time class="dt-end" datetime="2013-06-30 18:00">18:00</time> at
    <span class="p-location">Some bar in SF</span>
  </p>
  <p class="p-summary">
    Get together and discuss all things microformats-related.
  </p>
</div>
```

### Properties

| Property | Description |
|---|---|
| **`p-name`** | event name (or title) |
| **`p-summary`** | short summary of the event |
| **`dt-start`** | datetime the event starts |
| **`dt-end`** | datetime the event ends |
| **`p-location`** | where the event takes place, optionally embedded h-card |

### Parsed h-event Example

```
<div class="h-event">
  <h2 class="p-name">IndieWeb Summit</h2>
  <time class="dt-start" datetime="2019-06-29T09:00:00-07:00">
    June 29, 2019 at 9:00am (-0700)
  </time>
  <br />through
  <time class="dt-end" datetime="2019-06-30T18:00:00-07:00">
    June 30, 2019 at 6:00pm (-0700)
  </time>
  <br />
  <div class="p-location h-card">
    <div>
      <span class="p-name">Mozilla</span>
    </div>
    <div>
      <span class="p-street-address">1120 NW Couch St</span>,
      <span class="p-locality">Portland</span>,
      <span class="p-region">Oregon</span>,
      <span class="p-country">US</span>
    </div>
    <data class="p-latitude" value="45.52345"></data>
    <data class="p-longitude" value="-122.682677"></data>
  </div>
```

```
  <div class="e-content">Come join us</div>
  <div>
    <span class="p-author h-card">
      <a class="u-url p-name" href="https://aaronparecki.com">Aaron Parecki</a>
    </span>
    Published this
    <a href="https://aaronparecki.com/2019/06/29/1/" class="u-url">event </a>on
    <time class="dt published" datetime="2019-05-25T18:00:00-07:00">
      May 5th, 2019
    </time>
  </div>
</div>
```

```
{
  "items": [
    {
      "type": ["h-event"],
      "properties": {
        "name": ["IndieWeb Summit"],
        "url": ["https://aaronparecki.com/2019/06/29/1/"],
        "author": [
          {
            "type": ["h-card"],
            "properties": {
              "name": ["Aaron Parecki"],
              "url": ["https://aaronparecki.com"]
            },
            "lang": "en",
            "value": "Aaron Parecki"
          }
        ],
        "start": ["2019-06-29T09:00:00-07:00"],
        "end": ["2019-06-30T18:00:00-07:00"],
        "published": ["2019-05-25T18:00:00-07:00"],
        "content": [
          {
            "html": "Come join us",
            "value": "Come join us",
            "lang": "en"
          }
        ],
        "location": [
          {
            "type": ["h-card"],
            "properties": {
              "name": ["Mozilla"],
              "p-street-address": ["1120 NW Couch St"],
              "locality": ["Portland"],
              "region": ["Oregon"],
              "country": ["US"],
              "latitude": ["45.52345"],
              "longitude": ["-122.682677"]
            },
            "lang": "en",
            "value": "Mozilla"
          }
        ]
      },
      "lang": "en"
    }
  ]
}
```

### Microformats rel property examples

There are some microformats that are applied to a page by using a special `rel=` property. These microformats describe a relation between a current document and a linked document. For a full list of these, see the rel property on the microformats wiki.

### rel=author

This attribute states that the linked document represents the author of the current page.

```
<a rel="author" href="https://jamesg.blog">James Gallagher</a>
```

### rel=license

This attribute states that the linked document contains the license under which the current page is published.

```
<a rel="license" href="https://mit-license.org/">MIT License</a>
```

#### rel=nofollow

This attribute states that the linked document should not be given any weight by search engine ranking algorithms that may derive from the current page. This is useful to prevent link graph algorithms from weighing a page higher than it otherwise would after seeing a link to a document.

```
<a rel="nofollow" href="https://jamesg.blog">James Gallagher</a>
```

#### Browser compatibility

Supported in all browsers' support for the class attribute and its DOM API.

#### See also

- class attribute
- Microformat on Wikipedia
- Microformats official website
- Search engines support on Microformats official website
- Microformats on IndieWebCamp

## Understanding quirks and standards modes

In the old days of the web, pages were typically written in two versions: One for Netscape Navigator, and one for Microsoft Internet Explorer. When the web standards were made at W3C, browsers could not just start using them, as doing so would break most existing sites on the web. Browsers therefore introduced two modes to treat new standards compliant sites differently from old legacy sites.

There are now three modes used by the layout engines in web browsers: quirks mode, limited-quirks mode, and no-quirks mode. In **quirks mode**, layout emulates behavior in Navigator 4 and Internet Explorer 5. This is essential in order to support websites that were built before the widespread adoption of web standards. In **no-quirks mode**, the behavior is (hopefully) the desired behavior described by the modern HTML and CSS specifications. In **limited-quirks mode**, there are only a very small number of quirks implemented.

The limited-quirks and no-quirks modes used to be called "almost-standards" mode and "full standards" mode, respectively. These names have been changed as the behavior is now standardized.

#### How do browsers determine which mode to use?

For HTML documents, browsers use a doctype in the beginning of the document to decide whether to handle it in quirks mode or no-quirks mode. To ensure that your page uses no-quirks mode, make sure that your page has a doctype like in this example:

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <title>Hello World!</title>
  </head>
  <body></body>
</html>
```

The doctype shown in the example, `<!doctype html>`, is the simplest possible, and the one recommended by current HTML standards. Earlier versions of the HTML standard recommended other variants, but all existing browsers today will use no-quirks mode for this doctype. There are no valid reasons to use a more complicated doctype. If you do use another doctype, you may risk choosing one which triggers limited-quirks mode or quirks mode.

Put the doctype right at the beginning of your HTML document, before any other content.

The only purpose of `<!doctype html>` is to activate no-quirks mode. Older versions of HTML standard doctypes provided additional meaning, but no browser ever used the doctype for anything other than switching between render modes.

See also a detailed description of when different browsers choose various modes.

#### XHTML

If you serve your page as XHTML using the `application/xhtml+xml` MIME type in the `Content-Type` HTTP header, you do not need a doctype to enable no-quirks mode, as such documents always use no-quirks mode.

If you serve XHTML-like content using the `text/html` MIME type, browsers will read it as HTML, and you will need the doctype to use no-quirks mode.

#### How do I see which mode is used?

If the page is rendered in quirks or limited-quirks mode, Firefox will log a warning to the console tab in the developer tools. If this warning is not shown, Firefox is using no-quirks mode.

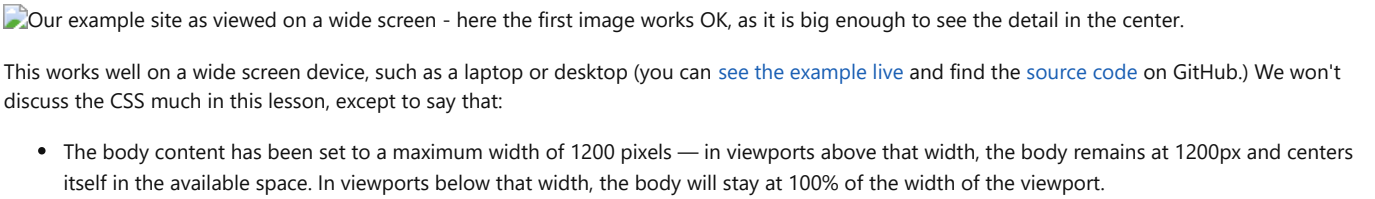
The value of `document.compatMode` in JavaScript will show whether or not the document is in quirks mode. If its value is `"BackCompat"`, the document is in quirks mode. If it isn't, it will have value `"CSS1Compat"`.

## Using responsive images in HTML

In this article, we'll learn about the concept of responsive images — images that work well on devices with widely differing screen sizes, resolutions, and other such features — and look at what tools HTML provides to help implement them. This helps to improve performance across different devices.

### Why responsive images?

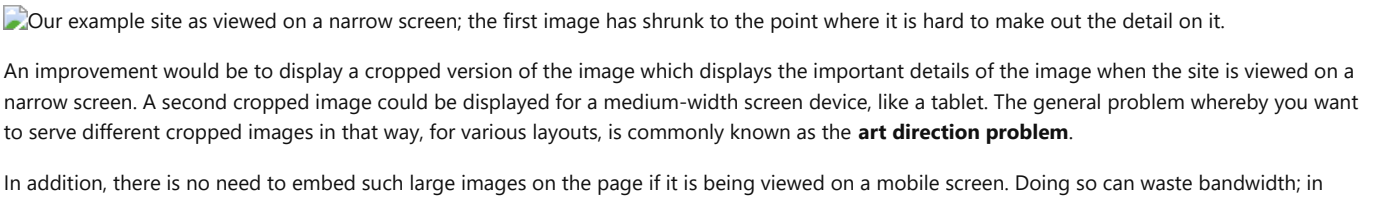
Let's examine a typical scenario. A typical website may contain a header image and some content images below the header. The header image will likely span the whole of the width of the header, and the content image will fit somewhere inside the content column. Here's an example:


Our example site as viewed on a wide screen - here the first image works OK, as it is big enough to see the detail in the center.

This works well on a wide screen device, such as a laptop or desktop (you can see the example live and find the source code on GitHub.) We won't discuss the CSS much in this lesson, except to say that:

- The body content has been set to a maximum width of 1200 pixels — in viewports above that width, the body remains at 1200px and centers itself in the available space. In viewports below that width, the body will stay at 100% of the width of the viewport.
- The header image has been set so that its center always stays in the center of the header, no matter what width the heading is set at. If the site is being viewed on a narrower screen, the important detail in the center of the image (the people) can still be seen, and the excess is lost off either side. It is 200px high.
- The content images have been set so that if the body element becomes smaller than the image, the images start to shrink so that they always stay inside the body, rather than overflowing it.

However, issues arise when you start to view the site on a narrow screen device. The header below looks OK, but it's starting to take up a lot of the screen height for a mobile device. And at this size, it is difficult to see faces of the two people within the first content image.


Our example site as viewed on a narrow screen; the first image has shrunk to the point where it is hard to make out the detail on it.

An improvement would be to display a cropped version of the image which displays the important details of the image when the site is viewed on a narrow screen. A second cropped image could be displayed for a medium-width screen device, like a tablet. The general problem whereby you want to serve different cropped images in that way, for various layouts, is commonly known as the **art direction problem**.

In addition, there is no need to embed such large images on the page if it is being viewed on a mobile screen. Doing so can waste bandwidth; in particular, mobile users don't want to waste bandwidth by downloading a large image intended for desktop users, when a small image would do for their device. Conversely, a small raster image starts to look grainy when displayed larger than its original size (a raster image is a set number of pixels wide and a set number of pixels tall). Ideally, multiple resolutions would be made available to the user's web browser. The browser could then determine the optimal resolution to load based on the screen size of the user's device. This is called the **resolution switching problem**.

To make things more complicated, some devices have high resolution screens that need larger images than you might expect to display nicely. This is essentially the same problem, but in a slightly different context.

You might think that vector images would solve these problems, and they do to a certain degree — they are small in file size and scale well, and you should use them wherever possible. However, they aren't suitable for all image types. Vector images are great for simple graphics, patterns, interface elements, etc., but it starts to get very complex to create a vector-based image with the kind of detail that you'd find in say, a photo. Raster image formats such as JPEGs are more suited to the kind of images we see in the above example.

This kind of problem didn't exist when the web first existed, in the early to mid 90s — back then the only devices in existence to browse the Web were desktops and laptops, so browser engineers and spec writers didn't even think to implement solutions. *Responsive image technologies* were implemented recently to solve the problems indicated above by letting you offer the browser several image files, either all showing the same thing but containing different numbers of pixels (*resolution switching*), or different images suitable for different space allocations (*art direction*).

> **Note:** The new features discussed in this article — `srcset`/`sizes`/`<picture>` — are all supported in modern desktop and mobile browsers.

### How do you create responsive images?

In this section, we'll look at the two problems illustrated above and show how to solve them using HTML's responsive image features. You should note that we will be focusing on `<img>` elements for this section, as seen in the content area of the example above — the image in the site header is only for decoration, and therefore implemented using CSS background images. CSS arguably has better tools for responsive design than HTML, and we'll talk about those in a future CSS module.

#### Resolution switching: Different sizes

So, what is the problem that we want to solve with resolution switching? We want to display identical image content, just larger or smaller depending on the device — this is the situation we have with the second content image in our example. The standard `<img>` element traditionally only lets you point the browser to a single source file:

```
<img src="https://developer.mozilla.org/en-US/docs/Web/HTML/Guides/Responsive_images/elva-fairy-800w.jpg" alt="Elva dressed as a fairy"
/>
```

We can however use two attributes — `srcset` and `sizes` — to provide several additional source images along with hints to help the browser pick the right one. You can see an example of this in our responsive.html example on GitHub (see also the source code):

```
<img
  srcset="elva-fairy-480w.jpg 480w, elva-fairy-800w.jpg 800w"
  sizes="(width <= 600px) 480px,
         800px"
  src="https://developer.mozilla.org/en-US/docs/Web/HTML/Guides/Responsive_images/elva-fairy-800w.jpg"
  alt="Elva dressed as a fairy" />
```

The `srcset` and `sizes` attributes look complicated, but they're not too hard to understand if you format them as shown above, with a different part of the attribute value on each line. Each value contains a comma-separated list, and each part of those lists is made up of three sub-parts. Let's run through the contents of each now:

**`srcset`** defines the set of images we will allow the browser to choose between, and what size each image is. Each set of image information is separated from the previous one by a comma. For each one, we write:

1. An **image filename** (`elva-fairy-480w.jpg`)
2. A space
3. The image's **intrinsic width in pixels** (`480w`) — note that this uses the `w` unit, not `px` as you might expect. An image's intrinsic size is its real size, which can be found by inspecting the image file on your computer (for example, on a Mac you can select the image in Finder and press `Cmd + I` to bring up the info screen).

**`sizes`** defines a set of media conditions (e.g., screen widths) and indicates what image size would be best to choose, when certain media conditions are true — these are the hints we talked about earlier. In this case, before each comma we write:

1. A **media condition** (`(width <= 600px)`) — you'll learn more about these in the CSS topic, but for now let's just say that a media condition describes a possible state that the screen can be in. In this case, we are saying "when the viewport width is 600 pixels or less".
2. A space
3. The **width of the slot** the image will fill when the media condition is true (`480px`)

> **Note:** In `sizes`, you can use any length value. For example, rather than providing an absolute width (for example, `480px`), you can alternatively provide a width relative to the viewport (for example, `50vw`). However, you cannot use a percentage as the slot width. You may have noticed that the last slot width has no media condition (this is the default that is chosen when none of the media conditions are true). The browser ignores everything after the first matching condition, so be careful how you order the media conditions.

So, with these attributes in place, the browser will:

1. Look at screen size, pixel density, zoom level, screen orientation, and network speed.
2. Work out which media condition in the `sizes` list is the first one to be true.
3. Look at the slot size given to that media query.
4. Load the image referenced in the `srcset` list that has the same size as the slot. If there isn't an exact match for the display size, the browser will choose the first image that is bigger than the chosen slot size and scale it down to fit.

And that's it! At this point, if a supporting browser with a viewport width of 480px loads the page, the `(width <= 600px)` media condition will be true, and so the browser chooses the `480px` slot. The `elva-fairy-480w.jpg` will be loaded, as its inherent width (`480w`) is closest to the slot size. The 800px picture is 128KB on disk, whereas the 480px version is only 63KB — a saving of 65KB. Now, imagine if this was a page that had many pictures on it. Using this technique could save mobile users a lot of bandwidth.

> **Note:** When testing this with a desktop browser, if the browser fails to load the narrower images when you've got its window set to the narrowest width, have a look at what the viewport is (you can approximate it by going into the browser's JavaScript console and typing in `document.querySelector('html').clientWidth`). Different browsers have minimum sizes that they'll let you reduce the window width to, and they might be wider than you'd think. When testing it with a mobile browser, you can use tools like Firefox's `about:debugging` page to inspect the page loaded on the mobile using the desktop developer tools.
>
> To see which images were loaded, you can use Firefox DevTools's Network Monitor tab or Chrome DevTools's Network panel. For Chrome, you may also want to disable cache to prevent it from picking already downloaded images.

Older browsers that don't support these features will just ignore them. Instead, those browsers will go ahead and load the image referenced in the `src` attribute as normal.

> **Note:** In the `<head>` of the example linked above, you'll find the line `<meta name="viewport" content="width=device-width">`: this forces mobile browsers to adopt their real viewport width for loading web pages (some mobile browsers lie about their viewport width, and instead load pages at a larger viewport width then shrink the loaded page down, which is not very helpful for responsive images or design).

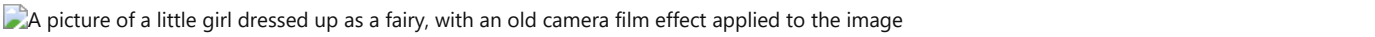
### Resolution switching: Same size, different resolutions

Suppose you have an image that will be rendered at the same real-world size on displays that have different screen resolutions. You can provide a better user experience on high resolution displays by serving a higher resolution version of the image.

To achieve this you can allow the browser to choose an appropriate resolution image by using `srcset` with x-descriptors and without `sizes` — a somewhat easier syntax! You can find an example of what this looks like in srcset-resolutions.html (see also the source code):

```
<img
  srcset="elva-fairy-320w.jpg, elva-fairy-480w.jpg 1.5x, elva-fairy-640w.jpg 2x"
  src="https://developer.mozilla.org/en-US/docs/Web/HTML/Guides/Responsive_images/elva-fairy-640w.jpg"
  alt="Elva dressed as a fairy" />
```

Note that even though the image is always displayed with the same size, on higher resolution displays you get to see more details.


A picture of a little girl dressed up as a fairy, with an old camera film effect applied to the image

In this example, the following CSS is applied to the image so that it will have a width of 320 pixels on the screen (also called CSS pixels):

```
img {
  width: 320px;
}
```

In this case, `sizes` is not needed — the browser works out what resolution the display is that it is being shown on, and serves the most appropriate image referenced in the `srcset`. So if the device accessing the page has a standard/low resolution display, with one device pixel representing each CSS pixel, the `elva-fairy-320w.jpg` image will be loaded (the 1x is implied, so you don't need to include it.) If the device has a high resolution of two device pixels per CSS pixel or more, the `elva-fairy-640w.jpg` image will be loaded. The 640px image is 93KB, whereas the 320px image is only 39KB.

### Art direction

To recap, the **art direction problem** involves wanting to change the image displayed to suit different image display sizes. For example, a web page includes a large landscape shot with a person in the middle when viewed on a desktop browser. When viewed on a mobile browser, that same image is shrunk down, making the person in the image very small and hard to see. It would probably be better to show a smaller, portrait image on mobile, which zooms in on the person. The `<picture>` element allows us to implement just this kind of solution.

Returning to our original not-responsive.html example, we have an image that badly needs art direction:

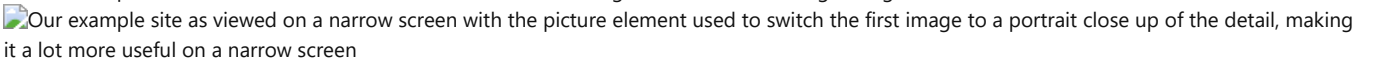
```
<img src="https://developer.mozilla.org/en-US/docs/Web/HTML/Guides/Responsive_images/elva-800w.jpg" alt="Chris standing up holding his
daughter Elva" />
```

Let's fix this, with `<picture>`! Like `<video>` and `<audio>`, the `<picture>` element is a wrapper containing several `<source>` elements that provide different sources for the browser to choose from, followed by the all-important `<img>` element. The code in responsive.html looks like so:

```
<picture>
  <source media="(width < 800px)" srcset="elva-480w-close-portrait.jpg" />
  <source media="(width >= 800px)" srcset="elva-800w.jpg" />
  <img src="https://developer.mozilla.org/en-US/docs/Web/HTML/Guides/Responsive_images/elva-800w.jpg" alt="Chris standing up holding his
daughter Elva" />
</picture>
```

- The `<source>` elements include a `media` attribute that contains a media condition — as with the first `srcset` example, these conditions are tests that decide which image is shown — the first one that returns true will be displayed. In this case, if the viewport width is less than 800px wide, the first `<source>` element's image will be displayed. If the viewport width is 800px or more, it'll be the second one.
- The `srcset` attributes contain the path to the image to display. Just as we saw with `<img>` above, `<source>` can take a `srcset` attribute with multiple images referenced, as well as a `sizes` attribute. So, you could offer multiple images via a `<picture>` element, but then also offer multiple resolutions of each one. Realistically, you probably won't want to do this kind of thing very often.
- In all cases, you must provide an `<img>` element, with `src` and `alt`, right before `</picture>`, otherwise no images will appear. This provides a default case that will apply when none of the media conditions return true (you could actually remove the second `<source>` element in this example), and a fallback for browsers that don't support the `<picture>` element.

This code allows us to display a suitable image on both wide screen and narrow screen displays, as shown below:


Our example site as viewed on a wide screen - here the first image works OK, as it is big enough to see the detail in the center.

Our example site as viewed on a narrow screen with the picture element used to switch the first image to a portrait close up of the detail, making it a lot more useful on a narrow screen

> **Note:** You should use the `media` attribute only in art direction scenarios; when you do use `media`, don't also offer media conditions within the `sizes` attribute.

### Why can't we just do this using CSS or JavaScript?

When the browser starts to load a page, it starts to download (preload) any images before the main parser has started to load and interpret the page's CSS and JavaScript. That mechanism is useful in general for reducing page load times, but it is not helpful for responsive images — hence the need to implement solutions like `srcset`. For example, you couldn't load the `<img>` element, then detect the viewport width with JavaScript, and then dynamically change the source image to a smaller one if desired. By then, the original image would already have been loaded, and you would load the small image as well, which is even worse in responsive image terms.

### Implementing your own responsive images

In this exercise, we're expecting you to be brave and do it alone, mostly. We want you to implement your own suitable art-directed narrow screen/wide screenshot using `<picture>`, and a resolution switching example that uses `srcset`.

1. Write some HTML to contain your code (use `not-responsive.html` as a starting point, if you like).
2. Find a nice wide screen landscape image with some kind of detail contained in it somewhere. Create a web-sized version of it using a graphics editor, then crop it to show a smaller part that zooms in on the detail, and create a second image (about 480px wide is good for this).
3. Use the `<picture>` element to implement an art direction picture switcher!
4. Create multiple image files of different sizes, each showing the same picture.
5. Use `srcset`/`sizes` to create a resolution switcher example, either to serve the same size image at different resolutions depending on the device resolution or to serve different image sizes depending on the viewport widths.

### Summary

That's a wrap for responsive images — we hope you enjoyed playing with these new techniques. As a recap, there are two distinct problems we've been discussing here:

- **Art direction**: The problem whereby you want to serve cropped images for different layouts — for example a landscape image showing a full scene for a desktop layout, and a portrait image showing the main subject zoomed in for a mobile layout. You can solve this problem using the `<picture>` element.
- **Resolution switching**: The problem whereby you want to serve smaller image files to narrow-screen devices, as they don't need huge images like desktop displays do — and to serve different resolution images to high density/low density screens. You can solve this problem using vector graphics (SVG images) and the `srcset` with `sizes` attributes.

### See also

- Learn: Responsive design
- Jason Grigsby's excellent introduction to responsive images
- Responsive Images: If you're just changing resolutions, use srcset — includes more explanation of how the browser works out which image to use
- `<img>`
- `<picture>`
- `<source>`

## How to

This page lists how-to guides for HTML, covering tasks such as adding JavaScript to pages, optimizing load performance, defining terms semantically, working with `data-*` attributes, and more.

Add a hitmap on top of an image

> You can create a clickable image map in HTML using `area` and `map` elements. This article describes how to set up an image map, and some downsides to consider before you build one.

Add JavaScript to your web page

> Take your webpages to the next level by harnessing JavaScript. Learn in this article how to trigger JavaScript right from your HTML documents.

Author fast-loading HTML pages

> An optimized web page not only provides for a more responsive site for your visitors but also reduces the load on your web servers and internet connection. This can be crucial for high volume sites or sites which have a spike in traffic due to unusual circumstances such as breaking news stories. This article describes how to improve page load times based on common knowledge and experimentation.

Define terms with HTML

> HTML provides several ways to convey description semantics, whether inline or as structured glossaries. In this article, we'll cover how to properly mark up keywords when you're defining them.

Use cross-origin images in a canvas

> HTML provides a `crossorigin` attribute for images that, in combination with an appropriate CORS header, allows images defined by the `img` element that are loaded from foreign origins to be used in a `canvas` as if they had been loaded from the current origin.

Use data attributes

> HTML is designed with extensibility in mind for data that should be associated with a particular element but need not have any defined meaning. `data-*` attributes allow us to store extra information on standard, semantic HTML elements without other hacks such as non-standard attributes, or extra properties on DOM.

## Add a hitmap on top of an image

You can create a clickable image map in HTML using `<area>` and `<map>` elements. This article describes how to set up an image map, and some downsides to consider before you build one.

Here are some things you need to know

| | |
|---|---|
| **Prerequisites:** | You should already know how to create a basic HTML document and how to add accessible images to a webpage. |
| **Objective:** | Learn how to make different regions of one image link to different pages. |

> **Warning:** This article discusses client-side image maps only. Do not use server-side image maps, which require the user to have a mouse.

### Image maps, and their drawbacks

When you nest an image inside `<a>`, the entire image links to one webpage. An image map, on the other hand, contains several active regions (called "hotspots") that each link to a different resource.

Formerly, image maps were a popular navigation device, but it's important to thoroughly consider their performance and accessibility ramifications.

> **Warning:** Multiple images referencing the same image map may lead to unexpected browser behavior, severely degrading usability and accessibility. For example, when a user keyboard navigates an image with a re-used image map in Safari and Chromium-based browsers, latter image instances using that same image map are skipped entirely. In Firefox, all image maps get keyboard focus simultaneously and when the user keyboard navigates past the image, the next focused element is the one after the last image instance, effectively skipping everything between the two images.

Text links (perhaps styled with CSS) are preferable to image maps for several reasons: text links are lightweight, maintainable, often more SEO-friendly, and support accessibility needs (e.g., screen readers, text-only browsers, translation services).

### How to insert an image map, properly

#### Step 1: The image

Not just any image is acceptable.

- The image must make it clear what happens when people follow image links. `alt` text is mandatory, of course, but many people never see it.
- The image must clearly indicate where hotspots begin and end.
- Hotspots must be large enough to tap comfortably, at any viewport size. How large is large enough? 72 × 72 CSS pixels is a good minimum, with additional generous gaps between touch targets. The map of the world at 50languages.com (as of time of writing) illustrates the problem perfectly. It's much easier to tap Russia or North America than Albania or Estonia.

You insert your image much the same way as always (with an `<img>` element and `alt` text). If the image is only present as a navigation device, you may write `alt=""`, provided you furnish appropriate `alt` text in the `<area>` elements later on.

You will need a special `usemap` attribute. Come up with a unique name, containing no spaces, for your image map. Then assign that name (preceded by a hash) as the value for the `usemap` attribute:

```
<img src="https://developer.mozilla.org/en-US/docs/Web/HTML/How_to/Add_a_hit_map_on_top_of_an_image/image-map.png" alt=""
usemap="#example-map-1" />
```

#### Step 2: Activate your hotspots

In this step, put all your code inside a `<map>` element. `<map>` only needs one attribute, the same map `name` as you used in your `usemap` attribute above:

```
<map name="example-map-1"> </map>
```

Inside the `<map>` element, we need `<area>` elements. An `<area>` element corresponds to a single hotspot. To keep keyboard navigation intuitive, make sure the source order of `<area>` elements corresponds to the visual order of hotspots.

`<area>` elements are void elements, but do require four attributes:

`shape`

> The `shape` attribute takes one of four values: `circle`, `rect`, `poly`, and `default`. An `<area>` whose `shape` is `default` occupies the entire image, minus any other hotspots you've defined. If there is any overlap between the defined areas, the source order determines which area takes preference. The shape you choose determines the coordinate information you'll need to provide in `coords`.

`coords`

> Coordinates are given in CSS pixels, and its value is dependent on the `shape` selected.
>
> - For a circle, provide the center's x and y coordinates, followed by the length of the radius.
> - For a rectangle, provide the x and y coordinates of the upper-left and bottom-right corners.
> - For a polygon, to provide the x and y coordinates of each corner (so, at least six values).

`href`

> The URL of the resource you're linking to. You may leave this attribute blank if you *don't* want the current area to link anywhere (say, if you're making a hollow circle.)

`alt`

> A mandatory attribute, telling people where the link goes or what it does. `alt` text only displays when the image is unavailable. Please refer to our guidelines for writing accessible link text.
>
> You may write `alt=""` if the `href` attribute is blank *and* the entire image already has an `alt` attribute.

```
<map name="example-map-1">
  <area
    shape="circle"
    coords="200,250,25"
    href="page-2.html"
    alt="circle example" />

  <area
    shape="rect"
    coords="10, 5, 20, 15"
    href="page-3.html"
    alt="rectangle example" />
</map>
```

### Step 3: Make sure it works for everybody

You aren't done until you test image maps rigorously on many browsers and devices. Try following links with your keyboard alone. Try turning images off.

If your image map is wider than about 240px, you'll need to make further adjustments to make your website responsive. It's not enough to resize the image for small screens, because the coordinates stay the same and no longer match the image.

### Learn more

- `<img>`
- `<map>`
- `<area>`
- Online image map editor

## Add JavaScript to your web page

Take your webpages to the next level by harnessing JavaScript. Learn in this article how to trigger JavaScript right from your HTML documents.

| | |
|---|---|
| **Prerequisites:** | You need to be familiar with how to create a basic HTML document. |
| **Objective:** | Learn how to trigger JavaScript in your HTML file, and learn the most important best practices for keeping JavaScript accessible. |

### About JavaScript

JavaScript is a programming language mostly used client-side to make webpages interactive. You *can* create amazing webpages without JavaScript, but JavaScript opens up a whole new level of possibilities.

> **Note:** In this article we're going over the HTML code you need to make JavaScript take effect. If you want to learn JavaScript itself, you can start with our JavaScript basics article. If you already know something about JavaScript or if you have a background with other programming

languages, we suggest you jump directly into our JavaScript Guide.

### How to trigger JavaScript from HTML

Within a browser, JavaScript doesn't do anything by itself. You run JavaScript from inside your HTML webpages. To call JavaScript code from within HTML, you need the `<script>` element. There are two ways to use `script`, depending on whether you're linking to an external script or embedding a script right in your webpage.

### Linking an external script

Usually, you'll be writing scripts in their own .js files. If you want to execute a .js script from your webpage, just use `<script>` with an `src` attribute pointing to the script file, using its URL:

```
<script src="https://developer.mozilla.org/en-US/docs/Web/HTML/How_to/Add_JavaScript_to_your_web_page/path/to/my/script.js"></script>
```

### Writing JavaScript within HTML

You may also add JavaScript code between `<script>` tags rather than providing an `src` attribute.

```
<script>
  console.log("Some code");
</script>
```

That's convenient when you just need a small bit of JavaScript, but if you keep JavaScript in separate files you'll find it easier to

- focus on your work
- write self-sufficient HTML
- write structured JavaScript applications

> **Note:** For both inline scripts and external scripts without the `defer` or `async` attributes, the script is executed immediately when the browser encounters the `<script>` element while parsing the HTML. This means that the script cannot access any HTML elements that appear later in the document. To access such elements, consider moving the script to the end of the document body (just before the closing `</body>` tag), or use the `defer` attribute on external scripts.

### Use scripting accessibly

Accessibility is a major issue in any software development. JavaScript can make your website more accessible if you use it wisely, or it can become a disaster if you use scripting without care. To make JavaScript work in your favor, it's worth knowing about certain best practices for adding JavaScript:

- **Make all content available as (structured) text.** Rely on HTML for your content as much as possible. For example, if you've implemented a nice JavaScript progress bar, make sure to supplement it with matching text percentages inside the HTML. Likewise, your drop-down menus should be structured as unordered lists of links.
- **Make all functionality accessible from the keyboard.**
  - Let users Tab through all controls (e.g., links and form input) in a logical order.
  - If you use pointer events (like mouse events or touch events), duplicate the functionality with keyboard events.
  - Test your site using a keyboard only.
- **Don't set nor even guess time limits.** It takes extra time to navigate with the keyboard or hear content read out. You can hardly ever predict just how long it will take for users or browsers to complete a process (especially asynchronous actions such as loading resources).
- **Keep animations subtle and brief with no flashing.** Flashing is annoying and can cause seizures. Additionally, if an animation lasts more than a couple seconds, give the user a way to cancel it.
- **Let users initiate interactions.** That means, don't update content, redirect, or refresh automatically. Don't use carousels or display popups without warning.
- **Have a plan B for users without JavaScript.** People may have JavaScript turned off to improve speed and security, and users often face network issues that prevent loading scripts. Moreover, third-party scripts (ads, tracking scripts, browser extensions) might break your scripts.
  - At a minimum, leave a short message with `<noscript>` like this: `<noscript>To use this site, please enable JavaScript.</noscript>`
  - Ideally, replicate the JavaScript functionality with HTML and server-side scripting when possible.
  - If you're only looking for simple visual effects, CSS can often get the job done even more intuitively.
  - *Since almost everybody **does** have JavaScript enabled, `<noscript>` is no excuse for writing inaccessible scripts.*

### Learn more

- `<script>`
- `<noscript>`

- Writing JavaScript with Accessibility in Mind by Manuel Matuzovic (2017)
- Accessibility guidelines from W3C

## Author fast-loading HTML pages

An optimized web page not only provides for a more responsive site for your visitors but also reduces the load on your web servers and internet connection. This can be crucial for high volume sites or sites which have a spike in traffic due to unusual circumstances such as breaking news stories. This article describes how to improve page load times based on common knowledge and experimentation.

Optimizing page load performance is not just for content which will be viewed by narrowband dial-up or mobile device visitors. It is just as important for broadband content and can lead to dramatic improvements even for your visitors with the fastest connections.

### Tips

#### Reduce page weight

Page weight is by far the most important factor in page-load performance.

Reducing page weight through the elimination of unnecessary whitespace and comments, commonly known as minimization, and by moving inline script and CSS into external files, can improve download performance with minimal need for other changes in the page structure.

Tools such as HTML Tidy can automatically strip leading whitespace and extra blank lines from valid HTML source. Other tools can "compress" JavaScript by reformatting the source or by obfuscating the source and replacing long identifiers with shorter versions.

#### Minimize the number of files

Reducing the number of files referenced in a web page lowers the number of HTTP connections required to download a page, thereby reducing the time for these requests to be sent, and for their responses to be received.

Depending on a browser's cache settings, it may send a request with the `If-Modified-Since` header for each referenced file, asking whether the file has been modified since the last time it was downloaded. Too much time spent querying the last modified time of the referenced files can delay the initial display of the web page, since the browser must check the modification time for each of these files, before rendering the page.

If you use background images a lot in your CSS, you can reduce the number of HTTP lookups needed by combining the images into one, known as an image sprite. Then you just apply the same image each time you need it for a background and adjust the x/y coordinates appropriately. This technique works best with elements that will have limited dimensions, and will not work for every use of a background image. However, the fewer HTTP requests and single image caching can help reduce page-load time.

#### Use a Content Delivery Network (CDN)

For the purposes of this article, a CDN is a means to reduce the physical distance between your server and your visitor. As the distance between your server origin and visitor increases, the load times will increase. Suppose your website server is located in the United States and it has a visitor from India; the page load time will be much higher for the Indian visitor compared to a visitor from the US.

A CDN is a geographically distributed network of servers that work together to shorten the distance between the user and your website. CDNs store cached versions of your website and serve them to visitors via the network node closest to the user, thereby reducing latency.

Further reading:

- Understanding CDNs

#### Reduce domain lookups

Since each separate domain costs time in a DNS lookup, the page load time will grow along with the number of separate domains appearing in CSS link(s) and JavaScript and image src(es).

This may not always be practical; however, you should always take care to use only the minimum necessary number of different domains in your pages.

#### Cache reused content

Make sure that any content that can be cached, is cached, and with appropriate expiration times.

In particular, pay attention to the `Last-Modified` header. It allows for efficient page caching; by means of this header, information is conveyed to the user agent about the file it wants to load, such as when it was last modified. Most web servers automatically append the `Last-Modified` header to static pages (e.g., `.html`, `.css`), based on the last-modified date stored in the file system. With dynamic pages (e.g., `.php`, `.aspx`), this, of course, can't be done, and the header is not sent.

So, in particular, for pages which are generated dynamically, a little research on this subject is beneficial. It can be somewhat involved, but it will save a lot in page requests on pages which would normally not be cacheable.

More information:

1. HTTP Conditional Get for RSS Hackers

2. HTTP 304: Not Modified
3. HTTP ETag on Wikipedia
4. Caching in HTTP

### Optimally order the components of the page

Download page content first, along with any CSS or JavaScript that may be required for its initial display, so that the user gets the quickest apparent response during the page loading. This content is typically text, and can, therefore, benefit from text compression in transit, thus providing an even quicker response to the user.

Any dynamic features that require the page to complete loading before being used, should be initially disabled, and then only enabled after the page has loaded. This will cause the JavaScript to be loaded after the page contents, which will improve the overall appearance of the page load.

### Reduce the number of inline scripts

Inline scripts can be expensive for page loading since the parser must assume that an inline script could modify the page structure while parsing is in progress. Reducing the use of inline scripts in general, and reducing the use of `document.write()` to output content in particular, can improve overall page loading. Use DOM APIs to manipulate page content, rather than `document.write()`.

### Use modern CSS and valid markup

Use of modern CSS reduces the amount of markup, can reduce the need for (spacer) images, in terms of layout, and can very often replace images of stylized text — that "cost" much more than the equivalent text-and-CSS.

Using valid markup has other advantages. First, browsers will have no need to perform error-correction when parsing the HTML (this is aside from the philosophical issue of whether to allow format variation in user input and then programmatically "correct" or normalize it; or whether, instead, to enforce a strict, no-tolerance input format).

Moreover, valid markup allows for the free use of other tools that can *pre-process* your web pages. For example, HTML Tidy can remove whitespace and optional ending tags; however, it will refuse to run on a page with serious markup errors.

### Chunk your content

Tables for layouts are a legacy method that should not be used anymore. Layouts utilizing floats, positioning, flexbox, or grids should be used instead.

Tables are still considered valid markup but should be used for displaying tabular data. To help the browser render your page quicker, you should use CSS methods such as flexible box layout and grid layout instead of nesting tables.

### Minify and compress SVG assets

SVG produced by most drawing applications often contains unnecessary metadata which can be removed. Configure your servers, apply gzip compression for SVG assets.

### Minify and compress your images

Large images cause your page to take more time to load. Consider compressing your images before adding them to your page, using compression features built into image-manipulation tools such as Photoshop, or using a specialized tool such as Compress JPEG or Tiny PNG.

### Specify sizes for images and tables

If the browser can immediately determine the height and/or width of your images and tables, it will be able to display a web page without having to reflow the content. This not only speeds the display of the page but prevents annoying changes in a page's layout when the page completes loading. For this reason, `height` and `width` should be specified for images, whenever possible.

Tables should use the CSS selector: property combination:

```
table-layout: fixed;
```

and should specify widths of columns using the `<col>` and the `<colgroup>` elements.

### Use lazy loading for images

By default, images are loaded **eagerly**; that is, the image is fetched and rendered as soon as it's processed in the HTML. All eagerly loaded images are rendered before the window's `load` event is sent. Switching to lazy loading of images tells the browser to hold off on loading images until they're about to be needed to draw the visual viewport.

To mark an image for lazy loading, specify its `loading` attribute with a value of `lazy`. With this set, the image will only be loaded when it's needed.

```
<img src="https://developer.mozilla.org/en-US/docs/Web/HTML/How_to/Author_fast-loading_HTML_pages/./images/footerlogo.jpg"
loading="lazy" alt="MDN logo" />
```

Note that lazily-loaded images may not be available when the `load` event is fired. You can determine if a given image is loaded by checking to see if the value of its Boolean `complete` property is `true`.

### Choose your user-agent requirements wisely

To achieve the greatest improvements in page design, make sure that reasonable user-agent requirements are specified for projects. Do not require your content to appear pixel-perfect in all browsers, especially not in down-version browsers.

Ideally, your basic minimum requirements should be based on the consideration of modern browsers that support the relevant standards. This can include recent versions of Firefox, Google Chrome, Opera, and Safari.

Note, however, that many of the tips listed in this article are common-sense techniques which apply to any user agent, and can be applied to any web page, regardless of browser-support requirements.

### Use async and defer, if possible

Make the JavaScript scripts such that they are compatible with both the async and the defer attributes, and use async whenever possible, especially if you have multiple script elements.

With that, the page can stop rendering while JavaScript is still loading. Otherwise, the browser will not render anything that is after the script elements that do not have these attributes.

Note: Even though these attributes do help a lot the first time a page is loaded, you should use them but not assume they will work in all browsers. If you already follow all JavaScript best practices, there is no need to change your code.

### Example page structure

- `<html>`
  - `<head>`
    - `<link>`

      CSS files required for page appearance. Minimize the number of files for performance while keeping unrelated CSS in separate files for maintenance.

    - `<script>`

      JavaScript files for functions **required** during the loading of the page, but not any interaction related JavaScript that can only run after page loads.

      Minimize the number of files for performance while keeping unrelated JavaScript in separate files for maintenance.
  - `<body>`

    User visible page content in small chunks (`<header>`/ `<main>`/ `<table>`) that can be displayed without waiting for the full page to download.

    - `<script>`

      Any scripts which will be used to perform interactivity. Interaction scripts typically can only run after the page has completely loaded and all necessary objects have been initialized. There is no need to load these scripts before the page content. That only slows down the initial appearance of the page load.

      Minimize the number of files for performance while keeping unrelated JavaScript in separate files for maintenance.

### See also

- Book: "Speed Up Your Site" by Andy King
- Learn performance via web.dev (2023)
- Tools for analyzing and optimizing performance: Google PageSpeed Tools and PageSpeed Insights

## Use cross-origin images in a canvas

HTML provides a `crossorigin` attribute for images that, in combination with an appropriate CORS header, allows images defined by the `<img>` element that are loaded from foreign origins to be used in a `<canvas>` as if they had been loaded from the current origin.

See CORS settings attributes for details on how the `crossorigin` attribute is used.

### Security and tainted canvases

Because the pixels in a canvas's bitmap can come from a variety of sources, including images or videos retrieved from other hosts, it's inevitable that security problems may arise.

As soon as you draw into a canvas any data that was loaded from another origin without CORS approval, the canvas becomes **tainted**. A tainted canvas is one which is no longer considered secure, and any attempts to retrieve image data back from the canvas will cause an exception to be thrown.

If the source of the foreign content is an HTML `<img>` or SVG `<svg>` element, attempting to retrieve the contents of the canvas isn't allowed.

If the foreign content comes from an image obtained from either as `HTMLCanvasElement` or `ImageBitMap`, and the image source doesn't meet the same origin rules, attempts to read the canvas's contents are blocked.

Calling any of the following on a tainted canvas will result in an error:

- Calling `getImageData()` on the canvas's context
- Calling `toBlob()`, `toDataURL()` or `captureStream()` on the `<canvas>` element itself

Attempting any of these when the canvas is tainted will cause a `SecurityError` to be thrown. This protects users from having private data exposed by using images to pull information from remote websites without permission.

### Storing an image from a foreign origin

In this example, we wish to permit images from a foreign origin to be retrieved and saved to local storage. Implementing this requires configuring the server as well as writing code for the website itself.

### Web server configuration

The first thing we need is a server that's configured to host images with the `Access-Control-Allow-Origin` header configured to permit cross-origin access to image files.

Let's assume we're serving our site using Apache. Consider the HTML5 Boilerplate Apache server configuration file for CORS images, shown below:

```
<IfModule mod_setenvif.c>
  <IfModule mod_headers.c>
    <FilesMatch "\.(avifs?|bmp|cur|gif|ico|jpe?g|jxl|a?png|svgz?|webp)$">
      SetEnvIf Origin ":" IS_CORS
      Header set Access-Control-Allow-Origin "*" env=IS_CORS
    </FilesMatch>
  </IfModule>
</IfModule>
```

In short, this configures the server to allow graphic files (those with the extensions ".bmp", ".cur", ".gif", ".ico", ".jpg", ".jpeg", ".png", ".svg", ".svgz", and ".webp") to be accessed cross-origin from anywhere on the internet.

### Implementing the save feature

Now that the server has been configured to allow retrieval of the images cross-origin, we can write the code that allows the user to save them to local storage, just as if they were being served from the same domain the code is running on.

The key is to use the `crossorigin` attribute by setting `crossOrigin` on the `HTMLImageElement` into which the image will be loaded. This tells the browser to request cross-origin access when downloading the image data.

### Starting the download

The code that starts the download (say, when the user clicks a "Download" button), looks like this:

```
function startDownload() {
  let imageURL = "https://mdn.github.io/shared-assets/images/examples/mdn.svg";
  let imageDescription = "Logo of a dinosaur in front of a map";

  downloadedImg = new Image();
  downloadedImg.crossOrigin = "anonymous";
  downloadedImg.addEventListener("load", imageReceived);
  downloadedImg.alt = imageDescription;
  downloadedImg.src = imageURL;
}
```

We're using a hard-coded URL (`imageURL`) and associated descriptive text (`imageDescription`) here, but that could easily come from anywhere. To begin downloading the image, we create a new `HTMLImageElement` object by using the `Image()` constructor. The image is then configured to allow cross-origin downloading by setting its `crossOrigin` attribute to `"anonymous"` (that is, allow non-authenticated downloading of the image cross-origin). An event listener is added for the `load` event being fired on the image element, which means the image data has been received. Alternative text is added to the image; while `<canvas>` does not support the `alt` attribute, the value can be used to set an `aria-label` or the canvas's inner content.

Finally, the image's `src` attribute is set to the URL of the image to download; this triggers the download to begin.

### Receiving and saving the image

The code that handles the newly-downloaded image is found in the `imageReceived()` method:

```
function imageReceived() {
  const canvas = document.createElement("canvas");
  const context = canvas.getContext("2d");

  canvas.width = downloadedImg.width;
  canvas.height = downloadedImg.height;
  canvas.innerText = downloadedImg.alt;

  context.drawImage(downloadedImg, 0, 0);
  imageBox.appendChild(canvas);

  try {
    localStorage.setItem("saved-image-example", canvas.toDataURL("image/png"));
  } catch (err) {
    console.error(`Error: ${err}`);
  }
}
```

`imageReceived()` is called to handle the `"load"` event on the `HTMLImageElement` that receives the downloaded image. This event is triggered once the downloaded data is all available. It begins by creating a new `<canvas>` element that we'll use to convert the image into a data URL, and by getting access to the canvas's 2D drawing context (`CanvasRenderingContext2D`) in the variable `context`.

The canvas's size is adjusted to match the received image, the inner text is set to the image description, then the image is drawn into the canvas using `drawImage()`. The canvas is then inserted into the document so the image is visible.

Now it's time to actually save the image locally. To do this, we use the Web Storage API's local storage mechanism, which is accessed through the `localStorage` global. The canvas method `toDataURL()` is used to convert the image into a data:// URL representing a PNG image, which is then saved into local storage using `setItem()`.

### See also

- Using Cross-domain images in WebGL and Chrome 13
- HTML Specification - the `crossorigin` attribute
- Web Storage API

## Define terms with HTML

HTML provides several ways to convey description semantics, whether inline or as structured glossaries. In this article, we'll cover how to properly mark up keywords when you're defining them.

| | |
|---|---|
| **Prerequisites:** | You need to be familiar with how to create a basic HTML document. |
| **Objective:** | Learn how to introduce new keywords and how to build description lists. |

When you need a term defined, you probably go straight to a dictionary or glossary. Dictionaries and glossaries *formally* associate keywords with one or more descriptions, as in this case:

> Blue (*Adjective*)
>
> Of a color like the sky in a sunny day. "*The clear blue sky*"

But we're constantly defining keywords informally, as here:

> **Firefox** is the web browser created by the Mozilla Foundation.

To deal with these use cases, HTML provides tags to mark descriptions and words described, so that your meaning gets across properly to your readers.

### How to mark informal description

In textbooks, the first time a keyword occurs, it's common to put the keyword in bold and define it right away.

We do that in HTML too, except HTML is not a visual medium and so we don't use bold. We use `<dfn>`, which is a special element just for marking the first occurrence of keywords. Note that `<dfn>` tags go around the *word to be defined,* not the definition (the definition consists of the entire paragraph):

```
<p><dfn>Firefox</dfn> is the web browser created by the Mozilla Foundation.</p>
```

> **Note:** Another use for bold is to emphasize content. Bold itself is a concept foreign to HTML, but there are tags for indicating emphasis.

### Special case: Abbreviations

It's best to mark abbreviations specially with `<abbr>`, so that screen readers read them appropriately and so that you can operate on all abbreviations uniformly. Just as with any new keyword, you should define your abbreviations the first time they occur.

```
<p>
  <dfn><abbr>HTML</abbr> (hypertext markup language)</dfn>
  is a description language used to structure documents on the web.
</p>
```

> **Note:** The HTML spec does indeed set aside the `title` attribute for expanding the abbreviation. However, this is not an acceptable alternative for providing an inline expansion. The contents of `title` are completely hidden from your users, unless they're using a mouse and they happen to hover over the abbreviation. The spec duly acknowledges this as well.

### Improve accessibility

`<dfn>` marks the keyword defined, and indicates that the current paragraph defines the keyword. In other words, there's an implicit relationship between the `<dfn>` element and its container. If you want a more formal relationship, or your definition consists of only one sentence rather than the whole paragraph, you can use the `aria-describedby` attribute to associate a term more formally with its definition:

```
<p>
  <span id="ff">
    <dfn aria-describedby="ff">Firefox</dfn>
    is the web browser created by the Mozilla Foundation.
  </span>
  You can download it at <a href="https://www.mozilla.org">mozilla.org</a>
</p>
```

Assistive technology can often use this attribute to find a text alternative to a given term. You can use `aria-describedby` on any tag enclosing a keyword to be defined (not just the `<dfn>` element). `aria-describedby` references the `id` of the element containing the description.

### How to build a description list

Description lists are just what they claim to be: a list of terms and their matching descriptions (e.g., definition lists, dictionary entries, FAQs, and key-value pairs).

> **Note:** Description lists are not suitable for marking up dialog, because conversation does not directly describe the speakers. Here are recommendations for marking up dialog.

The terms described go inside `<dt>` elements. The matching description follows immediately, contained within one or more `<dd>` elements. Enclose the whole description list with a `<dl>` element.

### A simple example

Here's an example describing kinds of food and drink:

```
<dl>
  <dt>jambalaya</dt>
  <dd>
    rice-based entree typically containing chicken, sausage, seafood, and spices
  </dd>

  <dt>sukiyaki</dt>
  <dd>
    Japanese specialty consisting of thinly sliced meat, vegetables, and
    noodles, cooked in sake and soy sauce
  </dd>

  <dt>chianti</dt>
  <dd>dry Italian red wine originating in Tuscany</dd>
</dl>
```

> **Note:** The basic pattern, as you can see, is to alternate `<dt>` terms with `<dd>` descriptions. If two or more terms occur in a row, the following description applies to all of them. If two or more descriptions occur in a row, they all apply to the last given term.

### Improving the visual output

Here's how a graphical browser displays the foregoing list:

If you want the keywords to stand out better, you could try bolding them. Remember, HTML is not a visual medium; we need CSS for all visual effects. The CSS `font-weight` property is what you need here:

```
dt {
  font-weight: bold;
}
```

This produces the slightly more readable result below:

### Learn more

- `<dfn>`
- `<dl>`
- `<dt>`
- `<dd>`
- How to use the aria-describedby attribute

## Use data attributes

HTML is designed with extensibility in mind for data that should be associated with a particular element but need not have any defined meaning. `data-*` attributes allow us to store extra information on standard, semantic HTML elements without other hacks such as non-standard attributes, or extra properties on DOM.

### HTML syntax

The syntax is simple. Any attribute on any element whose attribute name starts with `data-` is a data attribute. Say you have some articles and you want to store some extra information that doesn't have any visual representation. Just use `data` attributes for that:

```
<main>
  <article
    id="electric-cars"
    data-columns="3"
    data-index-number="12314"
    data-parent="cars">
    <!-- Electric car content -->
  </article>

  <article
    id="solar-cars"
    data-columns="3"
    data-index-number="12315"
    data-parent="cars">
    <!-- Solar car content -->
  </article>

  <article
    id="flying-cars"
    data-columns="4"
    data-index-number="12316"
    data-parent="cars">
    <!-- Flying car content -->
  </article>
</main>
```

### JavaScript access

Reading the values of these attributes out in JavaScript is also very simple. You could use `getAttribute()` with their full HTML name to read them, but the standard defines a simpler way: a `DOMStringMap` you can read out via a `dataset` property.

To get a `data` attribute through the `dataset` object, get the property by the part of the attribute name after `data-` (note that dashes are converted to camel case).

```
const article = document.querySelector("#electric-cars");
// The following would also work:
// const article = document.getElementById("electric-cars")

article.dataset.columns; // "3"
article.dataset.indexNumber; // "12314"
article.dataset.parent; // "cars"
```

Each property is a string and can be read and written. In the above case setting `article.dataset.columns = 5` would change that attribute to `"5"`.

You can also use `document.querySelector()` or `document.querySelectorAll()` with data attribute selectors to find one element or all elements that match:

```
// Find all elements with a data-columns attribute
const articles = document.querySelectorAll("[data-columns]");

// Find all elements with data-columns="3"
const threeColumnArticles = document.querySelectorAll('[data-columns="3"]');
// You can then iterate over the results
threeColumnArticles.forEach((article) => {
  console.log(article.dataset.indexNumber);
});
```

### CSS access

Note that, as data attributes are plain HTML attributes, you can even access them from CSS. For example to show the parent data on the article you can use generated content in CSS with the `attr()` function:

```
article::before {
  content: attr(data-parent);
}
```

You can also use the attribute selectors in CSS to change styles according to the data:

```
article[data-columns="3"] {
  width: 400px;
}
article[data-columns="4"] {
  width: 600px;
}
```

Data values are strings. Number values must be quoted in the selector for the styling to take effect.

### Examples

### Style variants

Imagine you have a `callout` class. Now you want to implement different variants, such as "note" and "warning". Traditionally, people just use different class names.

```
<div class="callout callout--note">...</div>
<div class="callout callout--warning">...</div>
```

```
.callout {
  margin: 0.5em 0;
  padding: 0.5em;
  border-radius: 4px;
  border-width: 2px;
  border-style: solid;
}

.callout--note {
  border-color: rgb(15 15 235);
  background-color: rgb(15 15 235 / 0.2);
}
.callout--warning {
  border-color: rgb(235 15 15);
  background-color: rgb(235 15 15 / 0.2);
}
```

With data attributes, here's an alternative you can consider:

```
<div class="callout">...</div>
<div class="callout" data-variant="note">...</div>
<div class="callout" data-variant="warning">...</div>
```

```
.callout {
  margin: 0.5em 0;
  padding: 0.5em;
  border-radius: 4px;
  border-width: 2px;
```

```
  border-style: solid;
}

/* Default style */
.callout:not([data-variant]) {
  border-color: rgb(15 15 15);
  background-color: rgb(15 15 15 / 0.2);
}
.callout[data-variant="note"] {
  border-color: rgb(15 15 235);
  background-color: rgb(15 15 235 / 0.2);
}
.callout[data-variant="warning"] {
  border-color: rgb(235 15 15);
  background-color: rgb(235 15 15 / 0.2);
}
```

There are multiple benefits of this:

- It eliminates a lot of invalid states, such as applying `callout--note` without also adding `callout`, or applying multiple variants simultaneously.
- A separate `data-variant` attribute allows static analysis for valid values via linting or type checking.
- Toggling the variant is more intuitive: you can use `div.dataset.variant = "warning";` instead of manipulating the `classList` which requires multiple steps.

### Associating arbitrary data with DOM elements

Many web apps have JavaScript data as the source-of-truth for their UI state. In these cases, you only add HTML attributes necessary for rendering. Data attributes are useful in the cases where everything is present in the markup, and JavaScript is only needed for handling events, syncing state, etc.

For example, in our carousel with scroll margin example, we have an HTML page already populated with many `<img>` elements. The image's source is initially stored in `data-src` to prevent any request being fired, and the real `src` is only added when the `<img>` scrolls into view. The data (image source) is colocated with the element, and the JavaScript is only responsible for defining behavior.

### Issues

Do not store content that should be visible and accessible in data attributes, because assistive technology may not access them. In addition, search crawlers may not index data attributes' values. Often, if you only intend for the data attribute to be displayed, you can directly manipulate `textContent`.

### See also

- This article is adapted from Using data attributes in JavaScript and CSS on hacks.mozilla.org.
- Custom attributes are also supported in SVG 2; see `HTMLElement.dataset` and `data-*` for more information.
- How to use HTML data attributes (Sitepoint)

## HTML reference

This HTML reference describes all **elements** and **attributes** of HTML, including **global attributes** that apply to all elements.

HTML element reference

> This page lists all the HTML elements, which are created using tags.

HTML attribute reference

> Elements in HTML have attributes; these are additional values that configure the elements or adjust their behavior in various ways to meet the criteria the users want.

Global attributes

> Global attributes are attributes common to all HTML elements; they can be used on all elements, though they may have no effect on some elements.

Content categories

> Every HTML element is a member of one or more content categories — these categories group elements that share common characteristics.

HTML comments

> HTML comments are used to add explanatory notes to the markup or to prevent the browser from interpreting specific parts of the document.

Date and time formats used in HTML

Certain HTML elements allow you to specify dates and/or times as the value or as the value of an attribute. These include the date and time variations of the `<input>` element as well as the `<ins>` and `<del>` elements.

## HTML attribute reference

Elements in HTML have **attributes**; these are additional values that configure the elements or adjust their behavior in various ways to meet the criteria the users want.

### Attribute list

| Attribute Name | Elements | Description |
|---|---|---|
| `accept` | `<form>`, `<input>` | List of types the server accepts, typically a file type. |
| `accept-charset` | `<form>` | The character set, which if provided must be `"UTF-8"`. |
| `accesskey` | Global attribute | Keyboard shortcut to activate or add focus to the element. |
| `action` | `<form>` | The URI of a program that processes the information submitted via the form. |
| `align` | `<caption>`, `<col>`, `<colgroup>`, `<hr>`, `<iframe>`, `<img>`, `<table>`, `<tbody>`, `<td>`, `<tfoot>`, `<th>`, `<thead>`, `<tr>` | Specifies the horizontal alignment of the element. |
| `allow` | `<iframe>` | Specifies a feature-policy for the iframe. |
| `alpha` | `<input>` | Allow the user to select a color's opacity on a `type="color"` input. |
| `alt` | `<area>`, `<img>`, `<input>` | Alternative text in case an image can't be displayed. |
| `aria-*` | All elements | Modifies an element's state or properties in the accessibility tree. Applicability depends on the specific ARIA attribute. |
| `as` | `<link>` | Specifies the type of content being loaded by the link. |
| `async` | `<script>` | Executes the script asynchronously. |
| `autocapitalize` | Global attribute | Sets whether input is automatically capitalized when entered by user |
| `autocomplete` | `<form>`, `<input>`, `<select>`, `<textarea>` | Indicates whether controls in this form can by default have their values automatically completed by the browser. |
| `autoplay` | `<audio>`, `<video>` | The audio or video should play as soon as possible. |
| `background` | `<body>`, `<table>`, `<td>`, `<th>` | Specifies the URL of an image file.<br><br>**Note:** Although browsers and email clients may still support this attribute, it is obsolete. Use CSS `background-image` instead. |
| `bgcolor` | `<body>`, `<col>`, `<colgroup>`, `<marquee>`, `<table>`, `<tbody>`, `<tfoot>`, `<td>`, `<th>`, `<tr>` | Background color of the element.<br><br>**Note:** This is a legacy attribute. Please use the CSS `background-color` property instead. |
| `border` | `<img>`, `<object>`, `<table>` | The border width.<br><br>**Note:** This is a legacy attribute. Please use the CSS `border` property instead. |
| `capture` | `<input>` | From the Media Capture specification, specifies a new file can be captured. |
| `charset` | `<meta>` | Declares the character encoding of the page or script. |
| `checked` | `<input>` | Indicates whether the element should be checked on page load. |
| `cite` | `<blockquote>`, `<del>`, `<ins>`, `<q>` | Contains a URI which points to the source of the quote or change. |
| `class` | Global attribute | Often used with CSS to style elements with common properties. |
| `color` | `<font>`, `<hr>` | This attribute sets the text color using either a named color or a color specified in the hexadecimal #RRGGBB format. |

| Attribute Name | Elements | Description |
| --- | --- | --- |
| | | **Note:** This is a legacy attribute. Please use the CSS `color` property instead. |
| `colorspace` | `<input>` | Defines the color space that is used by a `type="color"` input. |
| `cols` | `<textarea>` | Defines the number of columns in a textarea. |
| `colspan` | `<td>`, `<th>` | The colspan attribute defines the number of columns a cell should span. |
| `content` | `<meta>` | A value associated with `http-equiv` or `name` depending on the context. |
| `contenteditable` | Global attribute | Indicates whether the element's content is editable. |
| `controls` | `<audio>`, `<video>` | Indicates whether the browser should show playback controls to the user. |
| `coords` | `<area>` | A set of values specifying the coordinates of the hot-spot region. |
| `crossorigin` | `<audio>`, `<img>`, `<link>`, `<script>`, `<video>` | How the element handles cross-origin requests |
| `csp` | `<iframe>` | Specifies the Content Security Policy that an embedded document must agree to enforce upon itself. |
| `data` | `<object>` | Specifies the URL of the resource. |
| `data-*` | Global attribute | Lets you attach custom attributes to an HTML element. |
| `datetime` | `<del>`, `<ins>`, `<time>` | Indicates the date and time associated with the element. |
| `decoding` | `<img>` | Indicates the preferred method to decode the image. |
| `default` | `<track>` | Indicates that the track should be enabled unless the user's preferences indicate something different. |
| `defer` | `<script>` | Indicates that the script should be executed after the page has been parsed. |
| `dir` | Global attribute | Defines the text direction. Allowed values are ltr (Left-To-Right) or rtl (Right-To-Left) |
| `dirname` | `<input>`, `<textarea>` | |
| `disabled` | `<button>`, `<fieldset>`, `<input>`, `<optgroup>`, `<option>`, `<select>`, `<textarea>` | Indicates whether the user can interact with the element. |
| `download` | `<a>`, `<area>` | Indicates that the hyperlink is to be used for downloading a resource. |
| `draggable` | Global attribute | Defines whether the element can be dragged. |
| `enctype` | `<form>` | Defines the content type of the form data when the `method` is POST. |
| `enterkeyhint` | `<textarea>`, `contenteditable` | The `enterkeyhint` specifies what action label (or icon) to present for the enter key on virtual keyboards. The attribute can be used with form controls (such as the value of `textarea` elements), or in elements in an editing host (e.g., using `contenteditable` attribute). |
| `elementtiming` | `<img>`, `<image>` elements inside an `<svg>`, poster images of `<video>` elements, elements which have a `background-image`, and elements containing text nodes, such as a `<p>` | Indicates that an element is flagged for tracking by `PerformanceObserver` objects using the `"element"` type. For more details, see the `PerformanceElementTiming` interface. |
| `fetchpriority` | `<img>`, `<link>`, `<script>` | Signals that fetching a particular image early in the loading process has more or less impact on user experience than a browser can reasonably infer when assigning an internal priority. |
| `for` | `<label>`, `<output>` | Describes elements which belongs to this one. |
| `form` | `<button>`, `<fieldset>`, `<input>`, `<object>`, `<output>`, `<select>`, `<textarea>` | Indicates the form that is the owner of the element. |
| `formaction` | `<input>`, `<button>` | Indicates the action of the element, overriding the action defined in the `<form>`. |
| `formenctype` | `<button>`, `<input>` | If the button/input is a submit button (e.g., `type="submit"`), this attribute sets the encoding type to use during form submission. If this attribute is specified, it overrides the `enctype` attribute of the button's form owner. |
| `formmethod` | `<button>`, `<input>` | If the button/input is a submit button (e.g., `type="submit"`), this attribute sets the submission method to use during form submission (`GET`, `POST`, |

| Attribute Name | Elements | Description |
| --- | --- | --- |
| | | etc.). If this attribute is specified, it overrides the `method` attribute of the button's form owner. |
| `formnovalidate` | `<button>`, `<input>` | If the button/input is a submit button (e.g., `type="submit"`), this boolean attribute specifies that the form is not to be validated when it is submitted. If this attribute is specified, it overrides the `novalidate` attribute of the button's form owner. |
| `formtarget` | `<button>`, `<input>` | If the button/input is a submit button (e.g., `type="submit"`), this attribute specifies the browsing context (for example, tab, window, or inline frame) in which to display the response that is received after submitting the form. If this attribute is specified, it overrides the `target` attribute of the button's form owner. |
| `headers` | `<td>`, `<th>` | IDs of the `<th>` elements which applies to this element. |
| `height` | `<canvas>`, `<embed>`, `<iframe>`, `<img>`, `<input>`, `<object>`, `<video>` | Specifies the height of elements listed here. For all other elements, use the CSS `height` property.<br><br>**Note:** In some instances, such as `<div>`, this is a legacy attribute, in which case the CSS `height` property should be used instead. |
| `hidden` | Global attribute | Prevents rendering of given element, while keeping child elements, e.g. script elements, active. |
| `high` | `<meter>` | Indicates the lower bound of the upper range. |
| `href` | `<a>`, `<area>`, `<base>`, `<link>` | The URL of a linked resource. |
| `hreflang` | `<a>`, `<link>` | Specifies the language of the linked resource. |
| `http-equiv` | `<meta>` | Defines a pragma directive. |
| `id` | Global attribute | Often used with CSS to style a specific element. The value of this attribute must be unique. |
| `integrity` | `<link>`, `<script>` | This attribute contains one or more hashes of the resource, and is used to ensure that the content of the resource is what the developer expects it to be, and has not been replaced with a malicious copy in a supply chain attack.<br><br>See Subresource Integrity. |
| `inputmode` | `<textarea>`, `contenteditable` | Provides a hint as to the type of data that might be entered by the user while editing the element or its contents. The attribute can be used with form controls (such as the value of `textarea` elements), or in elements in an editing host (e.g., using `contenteditable` attribute). |
| `ismap` | `<img>` | Indicates that the image is part of a server-side image map. |
| `itemprop` | Global attribute | |
| `kind` | `<track>` | Specifies the kind of text track. |
| `label` | `<optgroup>`, `<option>`, `<track>` | Specifies a user-readable title of the element. |
| `lang` | Global attribute | Defines the language used in the element. |
| `language` | `<script>` | Defines the script language used in the element. |
| `loading` | `<img>`, `<iframe>` | Indicates if the element should be loaded lazily (`loading="lazy"`) or loaded immediately (`loading="eager"`). |
| `list` | `<input>` | Identifies a list of pre-defined options to suggest to the user. |
| `loop` | `<audio>`, `<marquee>`, `<video>` | Indicates whether the media should start playing from the start when it's finished. |
| `low` | `<meter>` | Indicates the upper bound of the lower range. |
| `max` | `<input>`, `<meter>`, `<progress>` | Indicates the maximum value allowed. |
| `maxlength` | `<input>`, `<textarea>` | Defines the maximum number of characters allowed in the element. |
| `minlength` | `<input>`, `<textarea>` | Defines the minimum number of characters allowed in the element. |

| Attribute Name | Elements | Description |
| --- | --- | --- |
| media | <a>, <area>, <link>, <source>, <style> | Specifies a hint of the media for which the linked resource was designed. |
| method | <form> | Defines which HTTP method to use when submitting the form. Can be GET (default) or POST. |
| min | <input>, <meter> | Indicates the minimum value allowed. |
| multiple | <input>, <select> | Indicates whether multiple values can be entered in an input of the type email or file. |
| muted | <audio>, <video> | Indicates whether the audio will be initially silenced on page load. |
| name | <button>, <form>, <fieldset>, <iframe>, <input>, <object>, <output>, <select>, <textarea>, <map>, <meta>, <param> | Name of the element. For example used by the server to identify the fields in form submits. |
| novalidate | <form> | This attribute indicates that the form shouldn't be validated when submitted. |
| open | <details>, <dialog> | Indicates whether the contents are currently visible (in the case of a <details> element) or whether the dialog is active and can be interacted with (in the case of a <dialog> element). |
| optimum | <meter> | Indicates the optimal numeric value. |
| pattern | <input> | Defines a regular expression which the element's value will be validated against. |
| ping | <a>, <area> | The ping attribute specifies a space-separated list of URLs to be notified if a user follows the hyperlink. |
| placeholder | <input>, <textarea> | Provides a hint to the user of what can be entered in the field. |
| playsinline | <video> | A Boolean attribute indicating that the video is to be played "inline"; that is, within the element's playback area. Note that the absence of this attribute does not imply that the video will always be played in fullscreen. |
| poster | <video> | A URL indicating a poster frame to show until the user plays or seeks. |
| preload | <audio>, <video> | Indicates whether the whole resource, parts of it or nothing should be preloaded. |
| readonly | <input>, <textarea> | Indicates whether the element can be edited. |
| referrerpolicy | <a>, <area>, <iframe>, <img>, <link>, <script> | Specifies which referrer is sent when fetching the resource. |
| rel | <a>, <area>, <link> | Specifies the relationship of the target object to the link object. |
| required | <input>, <select>, <textarea> | Indicates whether this element is required to fill out or not. |
| reversed | <ol> | Indicates whether the list should be displayed in a descending order instead of an ascending order. |
| role | Global attribute | Defines an explicit role for an element for use by assistive technologies. |
| rows | <textarea> | Defines the number of rows in a text area. |
| rowspan | <td>, <th> | Defines the number of rows a table cell should span over. |
| sandbox | <iframe> | Stops a document loaded in an iframe from using certain features (such as submitting forms or opening new windows). |
| scope | <th> | Defines the cells that the header test (defined in the th element) relates to. |
| selected | <option> | Defines a value which will be selected on page load. |
| shape | <a>, <area> | |
| size | <input>, <select> | Defines the width of the element (in pixels). If the element's type attribute is text or password then it's the number of characters. |
| sizes | <link>, <img>, <source> | |
| slot | Global attribute | Assigns a slot in a shadow DOM shadow tree to an element. |
| span | <col>, <colgroup> | |
| spellcheck | Global attribute | Indicates whether spell checking is allowed for the element. |
| src | <audio>, <embed>, <iframe>, <img>, <input>, <script>, <source>, <track>, | The URL of the embeddable content. |

| Attribute Name | Elements | Description |
| --- | --- | --- |
| | `<video>` | |
| `srcdoc` | `<iframe>` | |
| `srclang` | `<track>` | |
| `srcset` | `<img>`, `<source>` | One or more responsive image candidates. |
| `start` | `<ol>` | Defines the first number if other than 1. |
| `step` | `<input>` | |
| `style` | Global attribute | Defines CSS styles which will override styles previously set. |
| `summary` | `<table>` | |
| `tabindex` | Global attribute | Overrides the browser's default tab order and follows the one specified instead. |
| `target` | `<a>`, `<area>`, `<base>`, `<form>` | Specifies where to open the linked document (in the case of an `<a>` element) or where to display the response received (in the case of a `<form>` element) |
| `title` | Global attribute | Text to be displayed in a tooltip when hovering over the element. |
| `translate` | Global attribute | Specify whether an element's attribute values and the values of its `Text` node children are to be translated when the page is localized, or whether to leave them unchanged. |
| `type` | `<button>`, `<input>`, `<embed>`, `<object>`, `<ol>`, `<script>`, `<source>`, `<style>`, `<menu>`, `<link>` | Defines the type of the element. |
| `usemap` | `<img>`, `<input>`, `<object>` | |
| `value` | `<button>`, `<data>`, `<input>`, `<li>`, `<meter>`, `<option>`, `<progress>`, `<param>` | Defines a default value which will be displayed in the element on page load. |
| `width` | `<canvas>`, `<embed>`, `<iframe>`, `<img>`, `<input>`, `<object>`, `<video>` | For the elements listed here, this establishes the element's width.<br><br>**Note:** For all other instances, such as `<div>`, this is a legacy attribute, in which case the CSS `width` property should be used instead. |
| `wrap` | `<textarea>` | Indicates whether the text should be wrapped. |

### Content versus IDL attributes

In HTML, most attributes have two faces: the **content attribute** and the **IDL (Interface Definition Language) attribute**.

The content attribute is the attribute as you set it from the content (the HTML code) and you can set it or get it via `element.setAttribute()` or `element.getAttribute()`. The content attribute is always a string even when the expected value should be an integer. For example, to set an `<input>` element's `maxlength` to 42 using the content attribute, you have to call `setAttribute("maxlength", "42")` on that element.

The IDL attribute is also known as a JavaScript property. These are the attributes you can read or set using JavaScript properties like `element.foo`. The IDL attribute is always going to use (but might transform) the underlying content attribute to return a value when you get it and is going to save something in the content attribute when you set it. In other words, the IDL attributes, in essence, reflect the content attributes.

Most of the time, IDL attributes will return their values as they are really used. For example, the default `type` for `<input>` elements is "text", so if you set `input.type="foobar"`, the `<input>` element will be of type text (in the appearance and the behavior) but the "type" content attribute's value will be "foobar". However, the `type` IDL attribute will return the string "text".

IDL attributes are not always strings; for example, `input.maxlength` is a number (a signed long). When using IDL attributes, you read or set values of the desired type, so `input.maxlength` is always going to return a number and when you set `input.maxlength`, it wants a number. If you pass another type, it is automatically converted to a number as specified by the standard JavaScript rules for type conversion.

IDL attributes can reflect other types such as unsigned long, URLs, booleans, etc. Unfortunately, there are no clear rules and the way IDL attributes behave in conjunction with their corresponding content attributes depends on the attribute. Most of the time, it will follow the rules laid out in the specification, but sometimes it doesn't. HTML specifications try to make this as developer-friendly as possible, but for various reasons (mostly historical), some attributes behave oddly (`select.size`, for example) and you should read the specifications to understand how exactly they behave.

### Boolean Attributes

Some content attributes (e.g., `required`, `readonly`, `disabled`) are called boolean attributes. If a boolean attribute is present, its value is **true**, and if it's absent, its value is **false**.

HTML defines restrictions on the allowed values of boolean attributes: If the attribute is present, its value must either be the empty string (equivalently, the attribute may have an unassigned value), or a value that is an ASCII case-insensitive match for the attribute's canonical name, with no leading or trailing whitespace. The following examples are valid ways to mark up a boolean attribute:

```
<div itemscope>This is valid HTML but invalid XML.</div>
<div itemscope=itemscope>This is also valid HTML but invalid XML.</div>
<div itemscope="">This is valid HTML and also valid XML.</div>
<div itemscope="itemscope">
  This is also valid HTML and XML, but perhaps a bit verbose.
</div>
```

To be clear, the values `"true"` and `"false"` are not allowed on boolean attributes. To represent a false value, the attribute has to be omitted altogether. This restriction clears up some common misunderstandings: With `checked="false"` for example, the element's `checked` attribute would be interpreted as **true** because the attribute is present.

### Event handler attributes

> **Warning:** The use of event handler content attributes is discouraged. The mix of HTML and JavaScript often produces unmaintainable code, and the execution of event handler attributes may also be blocked by content security policies.

> **Warning:** While not visible by calling the `Function.prototype.toString()` method on the handler, event handler attributes will implicitly wrap code inside of 2 `with` statements, and may produce unexpected results. For example:
>
> ```
> Bad Example
> ```
>
> Essentially becomes:
>
> ```
> function onclick(event) {
>   with (this.ownerDocument) {
>     with (this) {
>       console.log(new URL(location)); // 'URL' now resolves to document.URL instead of window.URL
>       // TypeError: URL is not a constructor
>     }
>   }
> }
> ```

In addition to the attributes listed in the table above, global event handlers — such as `onclick` — can also be specified as content attributes on all elements.

All event handler attributes accept a string. The string will be used to synthesize a JavaScript function like `function name(/*args*/) {body}`, where `name` is the attribute's name, and `body` is the attribute's value. The handler receives the same parameters as its JavaScript event handler counterpart — most handlers receive only one `event` parameter, while `onerror` receives five: `event`, `source`, `lineno`, `colno`, `error`. This means you can, in general, use the `event` variable within the attribute.

```
<div onclick="console.log(event)">Click me!</div>
<!-- The synthesized handler has a name; you can reference itself -->
<div onclick="console.log(onclick)">Click me!</div>
```

### See also

- HTML elements

## accept HTML attribute

The **`accept`** attribute takes as its value a comma-separated list of one or more file types, or unique file type specifiers, describing which file types to allow.

### Try it

```
<label for="movie">Choose a movie to upload:</label>

<input type="file" id="movie" name="movie" accept="video/*" />

<label for="poster">Choose a poster:</label>

<input type="file" id="poster" name="poster" accept="image/png, image/jpeg" />
```

```
label {
  display: block;
```

```
  margin-top: 1rem;
}

input {
  margin-bottom: 1rem;
}
```

### Overview

The accept property is an attribute of the file `<input>` type. It was supported on the `<form>` element, but was removed in favor of file.

Because a given file type may be identified in more than one manner, it's useful to provide a thorough set of type specifiers when you need files of specific type, or use the wild card to denote a type of any format is acceptable.

For instance, there are a number of ways Microsoft Word files can be identified, so a site that accepts Word files might use an `<input>` like this:

```
<input
  type="file"
  id="docpicker"
  accept=".doc,.docx,application/msword,application/vnd.openxmlformats-officedocument.wordprocessingml.document" />
```

Whereas if you're accepting a media file, you may want to include any format of that media type:

```
<input type="file" id="soundFile" accept="audio/*" />
<input type="file" id="videoFile" accept="video/*" />
<input type="file" id="imageFile" accept="image/*" />
```

The `accept` attribute doesn't validate the types of the selected files; it provides hints for browsers to guide users towards selecting the correct file types. It is still possible (in most cases) for users to toggle an option in the file chooser that makes it possible to override this and select any file they wish, and then choose incorrect file types.

Because of this, you should make sure that expected requirement is validated server-side.

### Examples

When set on a file input type, the native file picker that opens up should only enable selecting files of the correct file type. Most operating systems lighten the files that don't match the criteria and aren't selectable.

```
<p>
  <label for="soundFile">Select an audio file:</label>
  <input type="file" id="soundFile" accept="audio/*" />
</p>
<p>
  <label for="videoFile">Select a video file:</label>
  <input type="file" id="videoFile" accept="video/*" />
</p>
<p>
  <label for="imageFile">Select some images:</label>
  <input type="file" id="imageFile" accept="image/*" multiple />
</p>
```

Note the last example allows you to select multiple images. See the `multiple` attribute for more information.

### Unique file type specifiers

A **unique file type specifier** is a string that describes a type of file that may be selected by the user in an `<input>` element of type `file`. Each unique file type specifier may take one of the following forms:

- A valid case-insensitive filename extension, starting with a period (".") character. For example: `.jpg`, `.pdf`, or `.doc`.
- A valid MIME type string, with no extensions.
- The string `audio/*` meaning "any audio file".
- The string `video/*` meaning "any video file".
- The string `image/*` meaning "any image file".

The `accept` attribute takes as its value a string containing one or more of these unique file type specifiers, separated by commas. For example, a file picker that needs content that can be presented as an image, including both standard image formats and PDF files, might look like this:

```
<input type="file" accept="image/*,.pdf" />
```

### Using file inputs

### A basic example

```
<div>
  <label for="file">Choose file to upload</label>
  <input type="file" id="file" name="file" multiple />
</div>
```

This produces the following output:

Regardless of the user's device or operating system, the file input provides a button that opens up a file picker dialog that allows the user to choose a file.

Including the `multiple` attribute, as shown above, specifies that multiple files can be chosen at once. The user can choose multiple files from the file picker in any way that their chosen platform allows (e.g., by holding down `Shift` or `Control`, and then clicking). If you only want the user to choose a single file per `<input>`, omit the `multiple` attribute.

### Limiting accepted file types

Often you won't want the user to be able to pick any arbitrary type of file; instead, you often want them to select files of a specific type or types. For example, if your file input lets users upload a profile picture, you probably want them to select web-compatible image formats, such as JPEG or PNG.

Acceptable file types can be specified with the `accept` attribute, which takes a comma-separated list of allowed file extensions or MIME types. Some examples:

- `accept="image/png"` or `accept=".png"` — Accepts PNG files.
- `accept="image/png, image/jpeg"` or `accept=".png, .jpg, .jpeg"` — Accept PNG or JPEG files.
- `accept="image/*"` — Accept any file with an `image/*` MIME type. (Many mobile devices also let the user take a picture with the camera when this is used.)
- `accept=".doc,.docx,.xml,application/msword,application/vnd.openxmlformats-officedocument.wordprocessingml.document"` — accept anything that smells like an MS Word document.

Let's look at a more complete example:

```
<div>
  <label for="profile_pic">Choose file to upload</label>
  <input
    type="file"
    id="profile_pic"
    name="profile_pic"
    accept=".jpg, .jpeg, .png" />
</div>
```

### See also

- Using files from web applications
- File API

## autocomplete HTML attribute

The HTML `autocomplete` attribute lets web developers specify what if any permission the user agent has to provide automated assistance in filling out form field values, as well as guidance to the browser as to the type of information expected in the field.

It is available on `<input>` elements that take a text or numeric value as input, `<textarea>` elements, `<select>` elements, and `<form>` elements.

### Try it

```
<label for="firstName">First Name:</label>
<input name="firstName" id="firstName" type="text" autocomplete="given-name" />

<label for="lastName">Last Name:</label>
<input name="lastName" id="lastName" type="text" autocomplete="family-name" />

<label for="email">Email:</label>
<input name="email" id="email" type="email" autocomplete="off" />
```

```
label {
  display: block;
  margin-top: 1rem;
}
```

### Description

The `autocomplete` attribute provides a hint to the user agent specifying how to, or indeed whether to, prefill a form control. The attribute value is either the keyword `off` or `on`, or an ordered list of space-separated tokens.

```
<input autocomplete="off" />
<input autocomplete="on" />
<textarea autocomplete="shipping street-address"></textarea>
<input autocomplete="section-user1 billing postal-code" />
```

If an `<input>`, `<select>` or `<textarea>` element has no `autocomplete` attribute, the browser will use the `autocomplete` attribute of the element's **owning form**. The owning form is either the `<form>` matching the `id` specified by the `form` attribute of the element (if present) or, more commonly, the `<form>` the element is nested in.

> **Note:** In order to provide autocompletion, user-agents might require `<input>`/`<select>`/`<textarea>` elements to:
>
> 1. Have a `name` and/or `id` attribute
> 2. Be descendants of a `<form>` element
> 3. Be owned by a form with a submit button

If the same list of tokens is used in more than one form control, the user-agent will autocomplete all occurrences of the same `autocomplete` value with the same data value.

Some tokens may be used more than once with potentially different expected values, such as the `postal-code` token in a form that contains both shipping and billing addresses. Including multiple different tokens in a space-separated list causes the associated form controls to be given unique autocomplete values: in this case, `autocomplete="shipping postal-code"` and `autocomplete="billing postal-code"`.

Some autocomplete values may need to be re-used multiple times. For example, a form may contain multiple shipping addresses and therefore multiple occurrences of `"shipping postal-code"` while still expecting different values. To make the autocomplete value unique in these cases, the first token in the space-separated list of tokens can be a `section-*` token, where the token's first eight characters are always the string "section-", followed by an alphanumeric string. All form fields given the `section-*` token with the same alphanumeric string belong to the same **named group**.

If including the `autocomplete` attribute on hidden input elements (`<input type="hidden">`), its value must be an ordered list of space-separated tokens; the `on` and `off` keywords are not allowed.

The source of the suggested values is generally up to the browser; typically values come from past values entered by the user, but they may also come from pre-configured values. For instance, a browser might let the user save their name, address, phone number, and email addresses for autocomplete purposes. The browser may also offer the ability to save encrypted credit card information, for autocompletion following an authentication procedure.

> **Note:** The `autocomplete` attribute also controls whether Firefox will — unlike other browsers — persist the dynamic disabled state and (if applicable) dynamic checkedness of an `<input>` element, `<textarea>` element, or entire `<form>` across page loads. The persistence feature is enabled by default. Setting the value of the `autocomplete` attribute to `off` disables this feature. This works even when the `autocomplete` attribute would normally not apply by virtue of its `type`. See Firefox bug 654072.

### Value

The attribute value is either the keyword `off` or `on`, or a space-separated `<token-list>` that describes the meaning of the autocompletion value.

`off`

> The browser is not permitted to automatically enter or select a value for this field. It is possible that the document or application provides its own autocomplete feature, or that security concerns require that the field's value not be automatically entered.
>
> > **Note:** In most modern browsers, setting `autocomplete` to `"off"` will not prevent a password manager from asking the user if they would like to save username and password information, or from automatically filling in those values in a site's login form. See Managing autofill for login fields.

`on`

> The browser is allowed to automatically complete the input. No guidance is provided as to the type of data expected in the field, so the browser may use its own judgment.

`<token-list>`

> An ordered set of space-separated tokens consisting of autofill detail tokens preceded by optional sectioning and either billing or shipping grouping tokens. Phone numbers, email addresses, and messaging protocol tokens are preceded by a token identifying the type of recipient.

See the WHATWG Standard for more detailed information.

### Token list tokens

The `<token-list>` options include, in order:

1. Group naming token
2. Grouping identifier
3. Detail tokens
4. Web authorization

### Named groups

To create a named group of form fields, the optional `section-*` token can be used. If present, this token must be the first token in the space-separated list of tokens.

`section-*`

Defines the name for a group of form controls. A token whose first eight characters are the string "section-", case-insensitive, followed by additional characters. All form controls that start with the same token belong to the named group.

### Grouping identifier

An optional `shipping` or `billing` grouping identifier

`shipping`

The field identified by subsequent tokens is part of the shipping address or contact information

`billing`

The field identified by subsequent tokens is part of the billing address or contact information

### Detail tokens

Each space-separated detail token list includes either a recipient type with digital contact information, in that order, or a space-separated token list of other tokens.

### Recipient type

The tokens that identify the type of recipient include:

`home`

The contact type identified by subsequent tokens is for contacting the recipient at their residence.

`work`

The contact type identified by subsequent tokens is for contacting the recipient at their work.

`mobile`

The contact type identified by subsequent tokens is for contacting the recipient regardless of location.

`fax`

The recipient identified by subsequent tokens is for a fax machine.

`pager`

The recipient identified by subsequent tokens is for a pager or beeper.

### Digital contact tokens

The token or group of tokens for telephone numbers or a number's component parts, phone extensions, email addresses, or instant messaging protocols.

`tel`

A full telephone number, including the country code. If you need to break the phone number up into its components, you can use these values for those fields:

`tel-country-code`

The country code, such as "1" for the United States, Canada, and other areas in North America and parts of the Caribbean.

`tel-national`

The entire phone number without the country code component, including a country-internal prefix. For the phone number "1-855-555-6502", this field's value would be "855-555-6502".

`tel-area-code`

The area code, with any country-internal prefix applied if appropriate.

`tel-local`

The phone number without the country or area code. This can be split further into two parts, for phone numbers which have an exchange number and then a number within the exchange. For the phone number "555-6502", use `tel-local-prefix` for "555" and `tel-local-suffix` for "6502".

`tel-extension`

A telephone extension code within the phone number, such as a room or suite number in a hotel or an office extension in a company.

`email`

An email address.

`impp`

A URL for an instant messaging protocol endpoint, such as `xmpp:username@example.net`.

### Other tokens

When the form field is not a phone number, email address, or instant messaging protocol, the space-separated list of tokens is not preceded by a contact type:

`name`

The field expects the value to be a person's full name. Using `name` rather than breaking the name down into its components is generally preferred because it avoids dealing with the wide diversity of human names and how they are structured; however, you can use the following `autocomplete` values if you do need to break the name down into its components:

`honorific-prefix`

The prefix or title, such as "Mrs.", "Mr.", "Miss", "Ms.", "Dr.", or "Mlle.".

`given-name`

The given (or "first") name.

`additional-name`

The middle name.

`family-name`

The family (or "last") name.

`honorific-suffix`

The suffix, such as "Jr.", "B.Sc.", "PhD.", "MBASW", or "IV".

`nickname`

A nickname or handle.

`username`

A username or account name.

`new-password`

A new password. When creating a new account or changing passwords, this should be used for an "Enter your new password" or "Confirm new password" field, as opposed to a general "Enter your current password" field that might be present. This may be used by the browser both to avoid accidentally filling in an existing password and to offer assistance in creating a secure password.

`current-password`

The user's current password.

`one-time-code`

A one-time password (OTP) for verifying user identity that is used as an additional factor in a sign-in flow. Most commonly this is a code received via some out-of-channel mechanism, such as SMS, email, or authenticator application.

`organization-title`

A job title, or the title a person has within an organization, such as "Senior Technical Writer", "President", or "Assistant Troop Leader".

`organization`

A company or organization name, such as "Acme Widget Company" or "Girl Scouts of America".

`street-address`

A street address. This can be multiple lines of text, and should fully identify the location of the address within its second administrative level (typically a city or town), but should not include the city name, ZIP or postal code, or country name.

`address-line1`, `address-line2`, `address-line3`

Each individual line of the street address. These should only be present if the `street-address` is not present.

`address-level4`

The finest-grained administrative level, in addresses which have four levels.

`address-level3`

The third administrative level, in addresses with at least three administrative levels.

`address-level2`

The second administrative level, in addresses with at least two of them. In countries with two administrative levels, this would typically be the city, town, village, or other locality in which the address is located.

`address-level1`

The first administrative level in the address. This is typically the province in which the address is located. In the United States, this would be the state. In Switzerland, the canton. In the United Kingdom, the county.

`country`

A country or territory code.

`country-name`

A country or territory name.

`postal-code`

A postal code (in the United States, this is the ZIP code).

`cc-name`

The full name as printed on or associated with a payment instrument such as a credit card. Using a full name field is preferred, typically, over breaking the name into pieces.

`cc-given-name`

A given (first) name as given on a payment instrument like a credit card.

`cc-additional-name`

A middle name as given on a payment instrument or credit card.

`cc-family-name`

A family name, as given on a credit card.

`cc-number`

A credit card number or other number identifying a payment method, such as an account number.

`cc-exp`

A payment method expiration date, typically in the form "MM/YY" or "MM/YYYY".

`cc-exp-month`

The month in which the payment method expires.

`cc-exp-year`

The year in which the payment method expires.

`cc-csc`

The security code for the payment instrument; on credit cards, this is the 3-digit verification number on the back of the card.

`cc-type`

The type of payment instrument (such as "Visa" or "Master Card").

`transaction-currency`

The currency in which the transaction is to take place.

`transaction-amount`

The amount, given in the currency specified by `transaction-currency`, of the transaction, for a payment form.

`language`

A preferred language, given as a valid BCP 47 language tag.

`bday`

A birth date, as a full date.

`bday-day`

The day of the month of a birth date.

`bday-month`

The month of the year of a birth date.

`bday-year`

The year of a birth date.

`sex`

A gender identity (such as "Female", "Fa'afafine", "Hijra", "Male", "Nonbinary"), as freeform text without newlines.

`url`

A URL, such as a home page or company website address as appropriate given the context of the other fields in the form.

`photo`

The URL of an image representing the person, company, or contact information given in the other fields in the form.

### Web authorization token

With `<input>` and `<textarea>`, the `webauthn` token can be included last to indicate the user agent should show public key credentials when the user is interacting with the control.

`webauthn`

Passkeys generated by the Web Authentication API, as requested by a conditional `navigator.credentials.get()` call (i.e., one that includes `mediation: 'conditional'`). If included, this is the last token in the space-separated token list. See Sign in with a passkey through form autofill for more details.

### Accessibility

Using appropriate `autocomplete` values helps users with cognitive disabilities, motor impairments, and other conditions fill out forms more quickly and accurately by reducing the need to type and remember information. When the browser can identify the purpose of a form field through its `autocomplete` value, it can offer stored data such as names, addresses, and payment details, which benefits all users but is especially important for those who find manual data entry difficult.

Providing valid autocomplete tokens also satisfies WCAG 2.2 Success Criterion 1.3.5: Identify Input Purpose (Level AA), which requires that the purpose of input fields collecting user information can be programmatically determined. This enables user agents and assistive technologies to apply personalized presentations, such as displaying familiar icons next to fields, to help users understand and complete forms.

### Avoid disabling autocomplete

Setting `autocomplete="off"` prevents the browser from offering stored data for a field. While developers sometimes use this to prevent autofill for security reasons (such as one-time codes), it removes a feature that many users depend on. Users with cognitive disabilities may rely on autocomplete to recall personal information, and users with motor impairments benefit from reduced typing. Browsers may also ignore `autocomplete="off"` on login fields to support password managers.

### Avoid invalid autocomplete values

Using invalid or non-standard values (such as made-up strings to circumvent autofill) has a similar effect: the browser cannot match the field to any known purpose, so it cannot offer relevant suggestions. This also fails the WCAG requirement above, since the input purpose is no longer programmatically determinable. If you need to disable autofill for a specific field, consider using `autocomplete="off"` only where genuinely necessary (such as for CAPTCHA or one-time token fields) rather than applying it broadly across a form.

### Examples

```
<div>
  <label for="cc-number">Enter your credit card number</label>
  <input name="cc-number" id="cc-number" autocomplete="off" />
</div>
```

### Administrative levels in addresses

The four administrative level fields (`address-level1` through `address-level4`) describe the address in terms of increasing levels of precision within the country in which the address is located. Each country has its own system of administrative levels, and may arrange the levels in different orders when addresses are written.

`address-level1` always represents the broadest administrative division; it is the least-specific portion of the address short of the country name.

### Form layout flexibility

Given that different countries write their address in different ways, with each field in different places within the address, and even different sets and numbers of fields entirely, it can be helpful if, when possible, your site is able to switch to the layout expected by your users when presenting an address entry form, given the country the address is located within.

### Variations

The way each administrative level is used will vary from country to country. Below are some examples; this is not meant to be an exhaustive list.

### United States

A typical home address within the United States looks like this:

432 Anywhere St Exampleville CA 95555

In the United States, the least-specific portion of the address is the state, in this case "CA" (the official US Postal Service shorthand for "California"). Thus `address-level1` is the state, or "CA" in this case.

The second-least specific portion of the address is the city or town name, so `address-level2` is "Exampleville" in this example address.

United States addresses do not use levels 3 and up.

### United Kingdom

Address input forms in the UK should contain one or two address levels and one, two or three address lines, depending on the address. A complete address would look like so:

103 Frogmarch Street Upper-Wapping Winchelsea Whereshire TN99 8ZZ

The address levels are:

- `address-level1`: The county — "Whereshire" in this case.
- `address-level2`: The post town — "Winchelsea" in this case.
- `address-line2`: The locality — "Upper-Wapping" in this case.
- `address-line1`: The house/street particulars — "103 Frogmarch Street".

The postcode is separate. Note that you can actually use just the postcode and `address-line1` to successfully deliver mail in the UK, so they should be the only mandatory items, but usually people tend to provide more details.

### China

China can use as many as three administrative levels: the province, the city, and the district.

The 6 digit postal code is not always needed but when supplied it is placed separately with a label for clarity. For example:

北京市东城区建国门北大街 8 号华润大厦 17 层 1708 单元 邮编：100005

### Japan

An address in Japan is typically **written in one line**, in an order from the least-specific to more-specific portions (in **reverse order to the United States**). There are two or three administrative levels in an address. Additional line can be used to show building names and room numbers. The postal code is separate. For example:

〒 381-0000 長野県長野市某町 123

"〒" and following seven digits shows the postal code.

`address-level1` is used for prefectures or the Tokyo Metropolis; "長野県" (Nagano Prefecture) is in this case. `address-level2` is typically used for cities, counties, towns and villages; "長野市" (Nagano City) in this case. "某町 123" is `address-line1` which consists of an area name and a lot number.

### Browser compatibility

### See also

- The `<input>` element
- The `<select>` element
- The `<textarea>` element
- The `<form>` element
- HTML forms
- All global attributes

## capture HTML attribute

The **`capture`** attribute specifies that, optionally, a new file should be captured, and which device should be used to capture that new media of a type defined by the `accept` attribute.

Values include `user` and `environment`. The capture attribute is supported on the file input type.

The `capture` attribute takes as its value a string that specifies which camera to use for capture of image or video data, if the accept attribute indicates that the input should be of one of those types.

| Value | Description |
| --- | --- |
| `user` | The user-facing camera and/or microphone should be used. |
| `environment` | The outward-facing camera and/or microphone should be used |

> **Note:** Capture was previously a Boolean attribute which, if present, requested that the device's media capture device(s) such as camera or microphone be used instead of requesting a file input.

### Try it

```
<label for="selfie">Take a picture of your face:</label>

<input type="file" id="selfie" name="selfie" accept="image/*" capture="user" />

<label for="picture">Take a picture using back facing camera:</label>

<input
  type="file"
  id="picture"
  name="picture"
  accept="image/*"
  capture="environment" />
```

```
label {
  display: block;
  margin-top: 1rem;
}

input {
  margin-bottom: 1rem;
}
```

### Examples

When set on a file input type, operating systems with microphones and cameras will display a user interface allowing the selection from an existing file or the creating of a new one.

```
<p>
  <label for="soundFile">What does your voice sound like?:</label>
  <input type="file" id="soundFile" capture="user" accept="audio/*" />
</p>
<p>
  <label for="videoFile">Upload a video:</label>
  <input type="file" id="videoFile" capture="environment" accept="video/*" />
```

```
</p>
<p>
  <label for="imageFile">Upload a photo of yourself:</label>
  <input type="file" id="imageFile" capture="user" accept="image/*" />
</p>
```

Note these work better on mobile devices; if your device is a desktop computer, you'll likely get a typical file picker.

### See also

- Using files from web applications
- File API
- `HTMLInputElement.files`

## content HTML attribute

The **`content`** attribute specifies the value of a metadata name defined by the `<meta>` `name` attribute. It takes a string as its value, and the expected syntax varies depending on the `name` value used.

### Value

The types of values that a `content` attribute accepts depends on the `name` value. For details on specific formats and types, see the `<meta>` `name` attribute page.

### Examples

### Setting a document meta description

The following `<meta>` tag uses `name=description` to set a "meta description" for a document. The `content` attribute provides the value for the metadata:

```
<meta
  name="description"
  content="The HTML reference describes all elements and attributes of HTML, including global attributes that apply to all elements." />
```

### See also

- `<meta>` `name` attribute

## crossorigin HTML attribute

The **`crossorigin`** attribute, valid on the `<audio>`, `<img>`, `<link>`, `<script>`, and `<video>` elements, provides support for CORS, defining how the element handles cross-origin requests, thereby enabling the configuration of the CORS requests for the element's fetched data. Depending on the element, the attribute can be a CORS settings attribute.

The `crossorigin` content attribute on media elements is a CORS settings attribute.

These attributes are enumerated, and have the following possible values:

`anonymous`

> Request uses CORS headers and credentials flag is set to `'same-origin'`. There is no exchange of **user credentials** via cookies, client-side TLS certificates or HTTP authentication, unless destination is the same origin.

`use-credentials`

> Request uses CORS headers, credentials flag is set to `'include'` and **user credentials** are always included.

`""`

> Setting the attribute name to an empty value, like `crossorigin` or `crossorigin=""`, is the same as `anonymous`.

An invalid keyword and an empty string will be handled as the `anonymous` keyword.

By default (that is, when the attribute is not specified), CORS is not used at all. The user agent will not ask for permission for full access to the resource and in the case of a cross-origin request, certain limitations will be applied based on the type of element concerned:

| Element | Restrictions |
| --- | --- |
| `img`, `audio`, `video` | When resource is placed in `<canvas>`, element is marked as *tainted*. |
| `script` | Access to error logging via `window.onerror` will be limited. |
| `link` | Request with no appropriate `crossorigin` header may be discarded. |

> **Note:** The `crossorigin` attribute is not supported for `rel="icon"` in Chromium-based browsers. See the open Chromium issue.

### Examples

#### <code>crossorigin</code> with the <code><script></code> element

You can use the following `<script>` element to tell a browser to execute the `https://example.com/example-framework.js` script without sending user-credentials.

```
<script
  src="https://example.com/example-framework.js"
  crossorigin="anonymous"></script>
```

#### Web manifest with credentials

The `use-credentials` value must be used when fetching a manifest that requires credentials, even if the file is from the same origin.

```
<link rel="manifest" href="https://developer.mozilla.org/app.webmanifest" crossorigin="use-credentials" />
```

### Browser compatibility

### See also

- Cross-Origin Resource Sharing (CORS)
- HTML attribute: `rel`

## dirname HTML attribute

The **`dirname`** attribute can be used on the `<textarea>` element and several `<input>` types and describes the directionality of the element's text content during form submission. The browser uses this attribute's value to determine whether text the user has entered is left-to-right or right-to-left oriented. When used, the element's text directionality value is included in form submission data along with the `dirname` attribute's value as the name of the field.

### Usage notes

The `dirname` attribute can be used on any `<textarea>` element, or any `<input>` element with hidden, text, search, tel, url, email, password, submit, reset or button type.

The format of the submitted data is `{dirname_value}={direction}` where `{dirname_value}` is the value of the `dirname` attribute and `{direction}` is the directionality of the text. For example, if the user enters "Hello" in an element with the attributes `name="comment"` and `dirname="comment-direction"`, the URL-encoded form submission data for `GET` requests will be `comment=Hello&comment-direction=ltr`. The directionality is one of the following:

`rtl`

: The text entered by the user is in a right-to-left writing direction.

`ltr`

: The text entered by the user is in a left-to-right writing direction.

If no text directionality is specified, the user agent will use the directionality of the parent element containing the form, and if that is not specified, the default directionality of the user agent.

### Examples

#### Textarea element directionality

In this example, the `dir="auto"` attribute on the textarea element allows the text directionality to be determined automatically based on the text entered by the user:

```
<form method="get" action="https://www.example.com/submit">
  <textarea name="comment" dir="auto" dirname="comment-direction">سيب</textarea>
  <button type="submit">Send my greetings</button>
</form>
```

When the user submits the form, the user agent includes two fields, one called `comment` with the value "سيب", and one called `comment-direction` with the value "rtl". The URL-encoded submission body looks like this:

```
https://www.example.com/submit?comment=%D8%B3%D9%8A%D8%A8&comment-direction=rtl
```

#### Input element directionality

In this example, the `dir="auto"` attribute on the input element allows the text directionality to be determined automatically based on the text entered by the user:

```
<form method="get" action="https://www.example.com/submit">
  <input
    type="text"
    name="comment-input"
    dir="auto"
    dirname="comment-direction"
    value="Hello" />
  <button type="submit">Send my greetings</button>
</form>
```

When the user submits the form, the user agent includes two fields, one called `comment-input` with the value "Hello", and one called `comment-direction` with the value "ltr":

```
https://www.example.com/submit?comment-input=Hello&comment-direction=ltr
```

### Inheriting directionality

The following `<input>` and `<textarea>` elements do not have a `dir` attribute, so they inherit the explicit directionality of their parent element, which is `rtl`:

```
<div dir="rtl">
  <form method="get" action="https://www.example.com/submit">
    <input
      type="text"
      name="user"
      dirname="user-direction"
      value="LTR Username" />
    <textarea name="comment" dirname="comment-direction">LTR Comment</textarea>
    <button type="submit">Post Comment</button>
  </form>
</div>
```

The URL-encoded submission body looks like this:

```
https://www.example.com/submit?user=LTR+Username&user-direction=rtl&comment=LTR+Comment&comment-direction=rtl
```

### Browser compatibility

### See also

- `dir` attribute
- `<input>`
- `<textarea>`

## disabled HTML attribute

The Boolean **`disabled`** attribute, when present, makes the element not mutable, focusable, or even submitted with the form. The user can neither edit nor focus on the control, nor its form control descendants.

### Try it

```
<form>
  <label for="name">Name:</label>
  <input id="name" name="name" type="text" />

  <label for="emp">Employed:</label>
  <select id="emp" name="emp" disabled>
    <option>No</option>
    <option>Yes</option>
  </select>

  <label for="empDate">Employment Date:</label>
  <input id="empDate" name="empDate" type="date" disabled />

  <label for="resume">Resume:</label>
  <input id="resume" name="resume" type="file" />
</form>
```

```
label {
  display: block;
```

```
  margin-top: 1em;
}

*:disabled {
  background-color: dimgrey;
  color: linen;
  opacity: 1;
}
```

### Overview

If the `disabled` attribute is specified on a form control, the element and its form control descendants do not participate in constraint validation. Often browsers gray out such controls and it won't receive any browsing events, like mouse clicks or focus-related ones.

The `disabled` attribute is supported by `<button>`, `<fieldset>`, `<optgroup>`, `<option>`, `<select>`, `<textarea>` and `<input>`.

This Boolean disabled attribute indicates that the user cannot interact with the control or its descendant controls. If this attribute is not specified, the control inherits its setting from the containing element, for example `fieldset`; if there is no containing element with the `disabled` attribute set, and the control itself does not have the attribute, then the control is enabled. If declared on an `<optgroup>`, the select is still interactive (unless otherwise disabled), but none of the items in the option group are selectable.

> **Note:** If a `<fieldset>` is disabled, the descendant form controls are all disabled, with the exception of form controls within the `<legend>`.

When a supporting element has the `disabled` attribute applied, the `:disabled` pseudo-class also applies to it. Conversely, elements that support the `disabled` attribute but don't have the attribute set match the `:enabled` pseudo-class.

This Boolean attribute prevents the user from interacting with the button. If this attribute isn't set, the button can still be disabled from a containing element, for example `<fieldset>`; if there is no containing element with the `disabled` attribute set, then the button is enabled.

Firefox will, unlike other browsers, persist the dynamic disabled state of a `<button>` across page loads. Use the `autocomplete` attribute to control this feature.

### Attribute interactions

The difference between `disabled` and `readonly` is that read-only controls can still function and are still focusable, whereas disabled controls cannot receive focus and are not submitted with the form and generally do not function as controls until they are enabled.

Because a disabled field cannot have its value changed, `required` does not have any effect on inputs with the `disabled` attribute also specified. Additionally, since the elements become immutable, most other attributes, such as `pattern`, have no effect, until the control is enabled.

> **Note:** The `required` attribute is not permitted on inputs with the `disabled` attribute specified.

### Usability

Browsers display disabled form controls grayed as disabled form controls are immutable, won't receive focus or any browsing events, like mouse clicks or focus-related ones, and aren't submitted with the form.

If present on a supporting elements, the `:disabled` pseudo class will match. If the attribute is not included, the `:enabled` pseudo class will match. If the element doesn't support the disabled attribute, the attribute will have no effect, including not leading to being matched by the `:disabled` and `:enabled` pseudo classes.

### Constraint validation

If the element is `disabled`, then the element's value cannot receive focus and cannot be updated by the user, and does not participate in constraint validation.

### Examples

When form controls are disabled, many browsers will display them in a lighter, grayed-out color by default. Here are examples of a disabled checkbox, radio button, `<option>` and `<optgroup>`, as well as some form controls that are disabled via the disabled attribute set on the ancestor `<fieldset>` element. The `<option>`s are disabled, but the `<select>` itself is not. We could have disable the entire `<select>` by adding the attribute to that element rather than its descendants.

```
<fieldset>
  <legend>Checkboxes</legend>
  <p>
    <label>
      <input type="checkbox" name="ch-box" value="regular" /> Regular
    </label>
  </p>
```

```
  <p>
    <label>
      <input type="checkbox" name="ch-box" value="disabled" disabled /> disabled
    </label>
  </p>
</fieldset>

<fieldset>
  <legend>Radio buttons</legend>
  <p>
    <label> <input type="radio" name="radio" value="regular" /> Regular </label>
  </p>
  <p>
    <label>
      <input type="radio" name="radio" value="disabled" disabled /> disabled
    </label>
  </p>
</fieldset>

<p>
  <label
    >Select an option:
    <select>
      <optgroup label="Group 1">
        <option>Option 1.1</option>
      </optgroup>
      <optgroup label="Group 2">
        <option>Option 2.1</option>
        <option disabled>Option 2.2</option>
        <option>Option 2.3</option>
      </optgroup>
      <optgroup label="Group 3" disabled>
        <option>Disabled 3.1</option>
        <option>Disabled 3.2</option>
        <option>Disabled 3.3</option>
      </optgroup>
    </select>
  </label>
</p>

<fieldset disabled>
  <legend>Disabled fieldset</legend>
  <p>
    <label>
      Name: <input type="radio" name="radio" value="regular" /> Regular
    </label>
  </p>
  <p>
    <label>Number: <input type="number" /></label>
  </p>
</fieldset>
```

### Browser compatibility

### See also

- `:disabled` and `:enabled`

## elementtiming HTML attribute

The **`elementtiming`** attribute is used to indicate that an element is flagged for tracking by `PerformanceObserver` objects using the `"element"` type. For more details, see the `PerformanceElementTiming` interface.

This attribute may be applied to `<img>`, `<image>` elements inside an `<svg>`, poster images of `<video>` elements, elements which have a `background-image`, and elements containing text nodes, such as a `<p>`.

In the DOM, this attribute is reflected as `Element.elementTiming`.

### Usage notes

The value given for `elementtiming` becomes an identifier for the observed element.

```
<img alt="alt" src="https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Attributes/elementtiming/img.jpg" elementtiming="label
for element" />
```

Good contenders for elements you might want to observe are:

- The main image for an article.
- A blog post title
- Images in a carousel for a shopping site.
- The poster image for the main video on a page.

### Examples

```
<img
  alt="Alt for a main blog post image"
  src="https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Attributes/elementtiming/my-massive-image.jpg"
  elementtiming="Main image" />

<p elementtiming="important-text">Some very important information.</p>
```

### See also

- `PerformanceElementTiming`
- `Element.elementTiming`

## fetchpriority HTML attribute

The **`fetchpriority`** attribute allows a developer to signal that fetching a particular image early in the loading process has more or less impact on user experience than a browser can reasonably infer when assigning an internal priority. This in turn allows the browser to increase or decrease the priority, and potentially load the image earlier or later than it would otherwise.

This attribute may be applied to `<img>`, `<link>`, and `<script>` elements. It also has an SVG counterpart.

The fetch priority can be used to complement preloading, allowing a developer to boost the priority ahead of less-impactful resources that have a higher default priority. For example, if a developer knows that a particular image significantly contributes to the website's Largest Contentful Paint (LCP) they could add `<link rel="preload">` for the image and then further boost the priority using the `fetchpriority` attribute.

Note that both the internal priority of any fetch operation, and the impact of `fetchpriority` on the priority, are entirely browser dependent.

This attribute is enumerated and can have one of the following values:

`high`

: Fetches the external resource at a high priority relative to other external resources.

`low`

: Fetches the external resource at a low priority relative to other external resources.

`auto`

: Doesn't set a preference for the fetch priority. It is used if no value or an invalid value is set. This is the default.

### Usage notes

The attribute should be used sparingly, as excessive or incorrect prioritization can degrade performance.

### Browser compatibility

### See also

- SVG `fetchpriority` attribute

## for HTML attribute

The **`for`** attribute is an allowed attribute for `<label>` and `<output>`. When used on a `<label>` element it indicates the form element that this label describes. When used on an `<output>` element it allows for an explicit relationship between the elements that represent values which are used in the output.

### Try it

```
<p>
  <label>First Name (no "for" attribute):</label>
  <input id="first" type="text" value="Jane" />
</p>
<p>
  <label for="last">Last Name (w/ "for" attribute):</label>
  <input id="last" type="text" value="Doe" />
</p>
<p id="result">
  <strong id="result-label">Full Name:</strong>
```

```
  <output for="first last" aria-labelledby="result-label" id="output"></output>
</p>
```

```
label[for="paragraph"] {
  color: rebeccapurple;
}

#result {
  text-align: center;
}

#result-label {
  font-size: 16pt;
}

#result-label,
#output {
  display: block;
}
```

```
const firstNameEl = document.getElementById("first");
const lastNameEl = document.getElementById("last");
const outputEl = document.getElementById("output");

function updateOutput() {
  const value = `${firstNameEl.value} ${lastNameEl.value}`;
  outputEl.innerText = value;
}

updateOutput();
firstNameEl.addEventListener("input", updateOutput);
lastNameEl.addEventListener("input", updateOutput);
```

### Usage notes

When used as an attribute of `<label>`, the `for` attribute has a value which is the `id` of the form element it relates to.

```
<label for="username">Your name</label> <input type="text" id="username" />
```

When used as an attribute of `<output>`, the `for` attribute has a value which is a space separated list of the `id` values of the elements which are used to create the output.

```
<input type="range" id="b" name="b" value="50" /> +
<input type="number" id="a" name="a" value="10" /> =
<output name="result" for="a b">60</output>
```

### Examples

See examples of usage on the element pages for `<label>` and `<output>`.

### Browser compatibility

## form HTML attribute

The `form` HTML attribute associates a form-associated element with a `<form>` element within the same document. This attribute applies to the `<button>`, `<fieldset>`, `<input>`, `<object>`, `<output>`, `<select>`, and `<textarea>` elements.

### Values

The `id` of the `<form>` element that the element should be associated with.

### Usage notes

By default, form controls are associated with their nearest ancestor `<form>` element, while form controls that are not nested within a `<form>` are not associated with any form. The `form` attribute enables overriding these default behaviors.

The `form` attribute of the `<button>`, `<fieldset>`, `<input>`, `<object>`, `<output>`, `<select>`, and `<textarea>` elements allows you to specify an explicit form owner, enabling you to associate form controls located anywhere within a document with any `<form>` element located in the same document.

When a form is submitted, the names and values of the `<form>` element's associated controls are submitted, whether or not they are physically nested within that `<form>`, and even if they are nested in a different `<form>`.

A control's `form` attribute takes as its value the `id` of the `<form>` element you want to associate the control with. All other values set for the `form` attribute are ignored.

While setting the attribute value to the `id` of the nearest ancestor `<form>` isn't necessary, explicitly defining the association between a form control and its nearest ancestor form ensures the form control will not be disassociated from its form if scripts or malformed HTML result in that specific `<form>` not being the nearest form ancestor of the control.

### Associating with a non-ancestor form

The `form` attribute can be used to associate a form control nested in one `<form>` with another `<form>`.

In this code example, the username `<input>` is nested within the `internalForm`, but the `form` attribute disassociates the control from its ancestor form in which it is nested, and associates it with the `externalForm` instead:

```
<form id="externalForm"></form>
<form id="internalForm">
  <label for="username">Username:</label>
  <input form="externalForm" type="text" name="username" id="username" />
</form>
```

In this case, the username will be submitted when the `externalForm` is submitted, while the `internalForm` has no associated form controls.

### Non-inheritance of the <code>form</code> attribute

The `form` attribute only associates the element on which it is set. The attribute behavior is not inherited. For example, when the `form` attribute is set on a `<fieldset>` element, it only associates the `<fieldset>`; it does **not** automatically associate the form controls nested within that `<fieldset>`.

In this example, the `<fieldset>` and `username` `<input>` are associated with the `exampleForm`, and included in the `HTMLFormControlsCollection` of the `HTMLFormElement.elements` property, but the `password` is not. Only the `username` will be included when the `exampleForm` is submitted:

```
<form id="exampleForm"></form>

<fieldset form="exampleForm">
  <legend>Login information</legend>
  <label
    >Username: <input form="exampleForm" type="text" name="username"
  /></label>
  <label>Password: <input type="password" name="password" /></label>
</fieldset>
```

Each nested element needs its own `form` attribute or must be nested inside the form. You can check which elements are associated with a form via JavaScript, using HTMLFormElement.elements.

### Form submission

Including the `form` attribute does not mean the element will be submitted with the form. Only submittable elements, including `<button>`, `<input>`, `<select>`, and `<textarea>`, have their name and values submitted when their associated `<form>` is submitted.

In this case, even though the `<output>` is implicitly then explicitly associated with the `calcForm`, the `result` is not submitted along with `a` and `b` when `calcForm` is submitted. It is, however, part of the form's `HTMLFormControlsCollection`.

```
<form id="calcForm">
  <label>First number: <input id="a" value="2" type="number" /></label>
  <label>Second number: <input id="b" value="3" type="number" /></label>
  <label>Sum: <output name="result" for="a b" form="calcForm">5</output></label>
</form>
```

## Examples

### Basic example

This example demonstrates how form-associated elements can be associated with a `<form>` element using the `form` attribute, even if they are not explicitly nested inside it. All of the form-associated elements shown in this example are associated with the `loginForm` either implicitly (by being directly nested inside the form), or explicitly via the `form` attribute. When the login form is submitted, the names and values of each submittable element will be included.

```
<form id="loginForm">
  <label>Username: <input type="text" name="username" /></label>
</form>

<label
  >Password: <input form="loginForm" type="password" name="password"
/></label>
<label>
  Choose an option:
  <select form="loginForm" name="options">
```

```
    <option value="A">A</option>
    <option value="B">B</option>
  </select>
</label>
<label
  >Description:
  <textarea form="loginForm" rows="4" name="description">
Hello, World!</textarea>
</label>
<button form="loginForm" type="submit" name="submitLogin" value="Login">
  Submit
</button>
```

#### Element associated with a different form

In this example, we have two `<form>` elements: `parentForm` and `targetForm`. The `<button>` inside `parentForm` has its `form` attribute set to `targetForm`, disassociating it from its nearest ancestor `parentForm`, while associating it with the `targetForm`. When the submit button is activated, it submits the `targetForm`, not its `parentForm` ancestor.

```
<form id="targetForm">
  <input type="text" name="targetInput" />
</form>
<form id="parentForm">
  <button form="targetForm" type="submit" name="submitTarget" value="Target">
    Submit target form
  </button>
</form>
```

#### Browser compatibility

#### See also

- Overriding default form behaviors

### integrity HTML attribute

The **`integrity`** attribute provides a mechanism for a developer to assert that a linked script or stylesheet must have a particular content. The browser will check that the resource does in fact have that content, and refuse to load the resource if it does not.

This is a defense against a supply-chain attack in which an attacker gets access to the domain that serves the script or stylesheet, and changes the expected resource for a malicious one.

#### Description

The attribute can be applied to `<script>` or `<link>` elements only.

The attribute consists of one or more components, each of which consists of:

- An identifier for a cryptographic hash function. Three hash functions are supported. In increasing order of strength, these are: SHA-256, SHA-384, and SHA-512.
- The result of hashing the resource contents using the specified hash function.

When the browser downloads a resource with the `integrity` attribute set, it will first select the set of hashes that were generated using the strongest hash function present. That is, if the attribute contains values generated with SHA-256 and SHA-384, it will only use the hashes generated using SHA-384.

The browser will then calculate the hash of the resource contents using the specified function, and compare the result with all the specified values: if the actual value matches any of the specified values, then the browser will load the resource, otherwise it will refuse to load the resource.

For more details, see our guide to Subresource Integrity.

#### Values

The value of this attribute consists of a whitespace-separated list of components, each of which has one of the following forms:

- `sha256-HASH_VALUE`
- `sha384-HASH_VALUE`
- `sha512-HASH_VALUE`

In each case, the part preceding `-` identifies the hash function used, and `HASH_VALUE` is the base64 encoding of the result of hashing the resource using the specified hash function.

#### Examples

### Including different hash functions

The following `<script>` element includes an `integrity` attribute containing three values, one calculated using SHA-256, one calculated using SHA-384, and one calculated using SHA-512.

The browser will select the value that was calculated using the strongest algorithm that the browser supports. Since all modern browsers support SHA-512, this means that the browser will select the `sha512-` value. It will hash the file contents using SHA-512 and compare the result with the `sha512-` value, and load the file only if they match.

In this case providing multiple values enables a website to work with browsers that may not support all the hash functions.

```
<script
  src="https://cdn.example.com/script.js"
  integrity="
  sha256-NmUxNTFiMDUzZGIwZjcwZDIyYTc5NTA4ZmQyNT
  sha384-Tk2Yjg3YmYzMWNkZTdhMTFkM2FlNDg4ZjE3MzEzNTk3ZDlh
  sha512-OGUwYThkZDc2YzFlZGI5MDEzZmZhMGFkMGQ0OTQ3MzZkNGYZTEzODk2"
  crossorigin="anonymous"></script>
```

Note that in this and subsequent examples, we've truncated the example hash values, for brevity.

### Including different hash values

The following `<script>` element includes an `integrity` attribute containing two different values, both calculated using the SHA-512 algorithm. These different values reflect alternative contents for the linked file.

If the SHA-512 hash of the linked file matches either of the given values, then the browser will load it.

This enables the server at `cdn.example.com` to respond with one of two versions of the file.

```
<script
  src="https://cdn.example.com/script.js"
  integrity="
  sha512-ZmQ5NjNiYWJjYTM3MjRhMGI4MTQzNWRmZTZkZGYyMzQyOGYYTZkYjBm
  sha512-OGUwYThkZDc2YzFlZGI5MDEzZmZhMGFkMGQ0OTQ3MzZkNGYZTEzODk2"
  crossorigin="anonymous"></script>
```

### Including <code>integrity</code> on <code><link></code> elements

The following `<link>` element loads a stylesheet and includes an `integrity` attribute containing six values, reflecting two possible contents for the linked file, each calculated using three different hash functions.

The browser will select the set of values that were calculated using the strongest hash function that it supports: in modern browsers, this will be the two `sha512-` values.

It will then calculate the hash of the downloaded file using SHA-512, and compare the result with both the `sha512-` values: if either of them match, then the browser will load the resource.

```
<link
  rel="stylesheet"
  href="https://cdn.example.com/style.css"
  integrity="
  sha256-NmUxNTFiMDUzZGIwZjcwZDIyYTc5NTA4ZmQyNT
  sha256-OTcyMGZkY2Y3NGZhZjUwNWU5NGQ0ZWJhYWVhND
  sha384-Tk2Yjg3YmYzMWNkZTdhMTFkM2FlNDg4ZjE3MzEzNTk3ZDlh
  sha384-ZTdhYjQ2NTE5OTg0Yjc2ZDU2MDMxMDUxY2Y5NDMxYzI5NjA
  sha512-OGUwYThkZDc2YzFlZGI5MDEzZmZhMGFkMGQ0OTQ3MzZkNGYZTEzODk2
  sha512-IxZTcwZjE2ZjU3MzE4NWM5ODU4ZmJkYjBlYzBhYzFkYzU0OGJmM2ZkN"
  crossorigin="anonymous" />
```

### Browser compatibility

### See also

- Subresource Integrity
- Supply chain attacks

## max HTML attribute

The **`max`** attribute defines the maximum value that is acceptable and valid for the input containing the attribute. If the `value` of the element is greater than this, the element fails validation. This value must be greater than or equal to the value of the `min` attribute. If the `max` attribute is present but is not specified or is invalid, no `max` value is applied. If the `max` attribute is valid and a non-empty value is greater than the maximum allowed by the `max` attribute, constraint validation will prevent form submission.

The max attribute is valid for the numeric input types, including the date, month, week, time, datetime-local, number and range types, and both the `<progress>` and `<meter>` elements. It is a number that specifies the most positive value a form control to be considered valid.

If the value exceeds the max value allowed, the `validityState.rangeOverflow` will be true, and the control will be matched by the `:out-of-range` and `:invalid` pseudo-classes.

### Syntax

Syntax for `max` values by input `type`

| Input type | Syntax | Example |
|---|---|---|
| date | `yyyy-mm-dd` | `<input type="date" max="2019-12-25" step="1">` |
| month | `yyyy-mm` | `<input type="month" max="2019-12" step="12">` |
| week | `yyyy-W##` | `<input type="week" max="2019-W23" step="">` |
| time | `HH:mm` | `<input type="time" max="17:00" step="900">` |
| datetime-local | `yyyy-mm-ddTHH:mm` | `<input type="datetime-local" max="2019-12-25T23:59">` |
| number | <number> | `<input type="number" min="0" step="5" max="100">` |
| range | <number> | `<input type="range" min="60" step="5" max="100">` |

> **Note:** When the data entered by the user doesn't adhere to the maximum value set, the value is considered invalid in constraint validation and will match the `:invalid` and `:out-of-range` pseudo-classes.

See Client-side validation and `rangeOverflow` for more information.

For the `<progress>` element, the `max` attribute describes how much work the task indicated by the `progress` element requires. If present, must have a value greater than zero and be a valid floating point number. For the `<meter>` element, the `max` attribute defines the upper numeric bound of the measured range. This must be greater than the minimum value (`min` attribute), if specified. In both cases, if omitted, the value defaults to 1.

Syntax for `max` values for other elements

| Input type | Syntax | Example |
|---|---|---|
| `<progress>` | <number> | `<progress id="file" max="100" value="70"> 70% </progress>` |
| `<meter>` | <number> | `<meter id="fuel" min="0" max="100" low="33" high="66" optimum="80" value="40"> at 40/100</meter>` |

### Accessibility concerns

Provide instructions to help users understand how to complete the form and use individual form controls. Indicate any required and optional input, data formats, and other relevant information. When using the `max` attribute, ensure this maximum requirement is understood by the user. Providing instructions within the `<label>` may be sufficient. If providing instructions outside of labels, which allows more flexible positioning and design, consider using `aria-labelledby` or `aria-describedby`.

### Browser compatibility

### See also

- `step`
- `min`
- Constraint validation
- Form validation
- `validityState.rangeOverflow`
- `:out-of-range`
- `<input>`
- date, month, week, time, datetime-local, number and range types, and the `<meter>`

## maxlength HTML attribute

The **`maxlength`** attribute defines the maximum string length that the user can enter into an `<input>` or `<textarea>`. The attribute must have an integer value of 0 or higher.

The length is measured in UTF-16 code units, which is often but not always equal to the number of characters. If no `maxlength` is specified, or an invalid value is specified, the input has no maximum length.

Any `maxlength` value must be greater than or equal to the value of `minlength`, if present and valid. The input will fail constraint validation if the length of the text value of the field is greater than maxlength UTF-16 code units long. Constraint validation is only applied when the value is changed by the user.

### Try it

```
<label for="name">Product name:</label>
<input
  id="name"
  name="name"
  type="text"
  value="Shampoo"
  minlength="3"
  maxlength="20"
  required />

<label for="description">Product description:</label>
<textarea
  id="description"
  name="description"
  minlength="10"
  maxlength="40"
  required></textarea>
```

```
label {
  display: block;
  margin-top: 1em;
}

input:valid,
textarea:valid {
  background-color: palegreen;
}
```

### Description

### Constraint validation

While the browser will generally prevent user from entering more text than the maxlength attribute allows, should the length be longer than the maxlength allows, the read-only `tooLong` property of a `ValidityState` object will be true.

### Examples

```
<input type="password" maxlength="4" />
```

### Browser compatibility

### See also

- `minlength`
- `size`
- `pattern`
- Constraint validation
- Form validation
- `<input>`

## min HTML attribute

The **`min`** attribute defines the minimum value that is acceptable and valid for the input containing the attribute. If the `value` of the element is less than this, the element fails validation. This value must be less than or equal to the value of the `max` attribute.

Some input types have a default minimum. If the input has no default minimum and a value is specified for `min` that can't be converted to a valid number (or no minimum value is set), the input has no minimum value.

It is valid for the input types including: date, month, week, time, datetime-local, number and range types, and the `<meter>` element.

### Syntax

Syntax for `min` values by input `type`

| Input type | Syntax | Example |
|---|---|---|
| date | `yyyy-mm-dd` | `<input type="date" min="2019-12-25" step="1">` |
| month | `yyyy-mm` | `<input type="month" min="2019-12" step="12">` |
| week | `yyyy-W##` | `<input type="week" min="2019-W23" step="">` |
| time | `HH:mm` | `<input type="time" min="09:00" step="900">` |
| datetime-local | `yyyy-mm-ddTHH:mm` | `<input type="datetime-local" min="2019-12-25T19:30">` |
| number | <number> | `<input type="number" min="0" step="5" max="100">` |

| Input type | Syntax | Example |
| --- | --- | --- |
| range | <number> | `<input type="range" min="60" step="5" max="100">` |

> **Note:** When the data entered by the user doesn't adhere to the min value set, the value is considered invalid in constraint validation and will match the `:invalid` and `:out-of-range` pseudo-classes.

See Client-side validation and `rangeUnderflow` for more information.

For the `<meter>` element, the `min` attribute defines the lower numeric bound of the measured range. This must be less than the maximum value (`max` attribute), if specified. In both cases, if omitted, the value defaults to 1.

Syntax for `min` values for other elements

| Input type | Syntax | Example |
| --- | --- | --- |
| `<meter>` | <number> | `<meter id="fuel" min="0" max="100" low="33" high="66" optimum="80" value="40"> at 40/100</meter>` |

### Impact on step

The value of `min` and `step` define what are valid values, even if the `step` attribute is not included, as `step` defaults to `0`.

We add a big red border around invalid inputs:

```
input:invalid {
  border: solid red 3px;
}
```

Then define an input with a minimum value of 7.2, omitting the step attribute, wherein it defaults to 1.

```
<input id="myNumber" name="myNumber" type="number" min="7.2" value="8" />
```

Because `step` defaults to 1, valid values include `7.2`, `8.2`, `9.2`, and so on. The value 8 is not valid. As we included an invalid value, supporting browsers will show the value as invalid.

If not explicitly included, `step` defaults to 1 for `number` and `range`, and 1 unit type (second, week, month, day) for the date/time input types.

### Accessibility concerns

Provide instructions to help users understand how to complete the form and use individual form controls. Indicate any required and optional input, data formats, and other relevant information. When using the `min` attribute, ensure this minimum requirement is understood by the user. Providing instructions within the `<label>` may be sufficient. If providing instructions outside of labels, which allows more flexible positioning and design, consider using `aria-labelledby` or `aria-describedby`.

### Browser compatibility

### See also

- `step`
- `max`
- other meter attributes: `low`, `high`, `optimum`
- Constraint validation
- Form validation
- `validityState.rangeUnderflow`
- `:out-of-range`
- `<input>`
- date, month, week, time, datetime-local, number and range types, and the `<meter>`

## minlength HTML attribute

The **`minlength`** attribute defines the minimum string length that the user can enter into an `<input>` or `<textarea>`. The attribute must have an integer value of 0 or higher.

The length is measured in UTF-16 code units, which is often but not always equal to the number of characters. If no `minlength` is specified, or an invalid value is specified, the input has no minimum length. This value must be less than or equal to the value of maxlength, otherwise the value will never be valid, as it is impossible to meet both criteria.

The input will fail constraint validation if the length of the text value of the field is less than minlength UTF-16 code units long, with `validityState.tooShort` returning `true`. Constraint validation is only applied when the value is changed by the user. Once submission fails, some browsers will display an error message indicating the minimum length required and the current length.

`minlength` does not imply `required`; an input only violates a `minlength` constraint if the user has input a value. If an input is not `required`, an empty string can be submitted even if `minlength` is set.

### Try it

```
<label for="name">Product name:</label>
<input
  id="name"
  name="name"
  type="text"
  value="Shampoo"
  minlength="3"
  maxlength="20"
  required />

<label for="description">Product description:</label>
<textarea
  id="description"
  name="description"
  minlength="10"
  maxlength="40"
  required></textarea>
```

```
label {
  display: block;
  margin-top: 1em;
}

input:valid,
textarea:valid {
  background-color: palegreen;
}
```

### Examples

By adding `minlength="5"`, the value must either be empty or five characters or longer to be valid.

```
<label for="fruit">Enter a fruit name that is at least 5 letters long</label>
<input type="text" minlength="5" id="fruit" />
```

We can use pseudoclasses to style the element based on whether the value is valid. The value will be valid as long as it is either null (empty) or five or more characters long. *Lime* is invalid, *lemon is valid*.

```
input {
  border: 2px solid currentColor;
}
input:invalid {
  border: 2px dashed red;
}
input:invalid:focus {
  background-image: linear-gradient(pink, lightgreen);
}
```

### Browser compatibility

### See also

- `maxlength`
- `size`
- `pattern`
- Constraint validation
- Form validation
- `<input>`

## multiple HTML attribute

The Boolean **`multiple`** attribute, if set, means the form control accepts one or more values. The attribute is valid for the email and file input types and the `<select>`. The manner by which the user opts for multiple values depends on the form control.

### Try it

```
<label for="recipients">Where should we send the receipt?</label>
<input id="recipients" name="recipients" type="email" multiple />
```

```
<label for="shakes">Which shakes would you like to order?</label>
<select id="shakes" name="shakes" multiple>
  <option>Vanilla Shake</option>
  <option>Strawberry Shake</option>
  <option>Chocolate Shake</option>
</select>

<label for="payment">How would you like to pay?</label>
<select id="payment" name="payment">
  <option>Credit card</option>
  <option>Bank Transfer</option>
</select>
```

```
label {
  display: block;
  margin-top: 1em;
}

input,
select {
  width: 100%;
}

input:invalid {
  background-color: lightpink;
}
```

### Overview

Depending on the type, the form control may have a different appearance if the `multiple` attribute is set. For the file input type, the native messaging the browser provides differs. In Firefox, the file input reads "No files selected" when the attribute is present and "No file selected" when it is not. Most browsers display a scrolling list box for a `<select>` control with the `multiple` attribute set and a single line dropdown when the attribute is omitted. The email input displays the same whether or not the `multiple` attribute is included, but will match the `:invalid` pseudo-class if more than one comma-separated email address is included if the attribute is not present.

When `multiple` is set on the email input type, the user can include zero (if not also `required`), one or more comma-separated email addresses.

```
<input type="email" multiple name="emails" id="emails" />
```

If and only if the `multiple` attribute is specified, the value can be a list of properly-formed comma-separated email addresses. Any trailing and leading whitespace is removed from each address in the list.

When `multiple` is set on the file input type, the user can select one or more files. The user can choose multiple files from the file picker in any way that their chosen platform allows (e.g., by holding down `Shift` or `Control`, and then clicking).

```
<input type="file" multiple name="uploads" id="uploads" />
```

When the attribute is omitted, the user can only select a single file per `<input>`.

The `multiple` attribute on the `<select>` element represents a control for selecting zero or more options from the list of options. Otherwise, the `<select>` element represents a control for selecting a single `<option>` from the list of options.

```
<select multiple name="dwarfs" id="dwarfs">
  <option>Grumpy</option>
  <option>Happy</option>
  <option>Sleepy</option>
  <option>Bashful</option>
  <option>Sneezy</option>
  <option>Dopey</option>
  <option>Doc</option>
</select>
```

When `multiple` is specified, most browsers will show a scrolling list box instead of a single line dropdown.

Multiple selected options are submitted using the `URLSearchParams` array convention, i.e., `name=value1&name=value2`.

### Accessibility concerns

Provide instructions to help users understand how to complete the form and use individual form controls. Indicate any required and optional input, data formats, and other relevant information. When using the `multiple` attribute, inform the user that multiple values are allowed and provide directions on how to provide multiple values, such as "separate email addresses with a comma."

Setting `size="1"` on a multiple select (i.e., `<select multiple size="1">`) renders a drop-down that lets users select multiple options. Some browsers don't expand the option list when the control is active, displaying multiple options in a menu list the height of a single-line box. This harms usability. When including a multiple-select, inform users they can select more than one option, even when multiple options are displayed.

### Examples

#### email input

```
<label for="emails">Who do you want to email?</label>
<input
  type="email"
  multiple
  name="emails"
  id="emails"
  list="dwarf-emails"
  required
  size="64" />

<datalist id="dwarf-emails">
  <option value="grumpy@woodworkers.com">Grumpy</option>
  <option value="happy@woodworkers.com">Happy</option>
  <option value="sleepy@woodworkers.com">Sleepy</option>
  <option value="bashful@woodworkers.com">Bashful</option>
  <option value="sneezy@woodworkers.com">Sneezy</option>
  <option value="dopey@woodworkers.com">Dopey</option>
  <option value="doc@woodworkers.com">Doc</option>
</datalist>
```

If and only if the `multiple` attribute is specified, the value can be a list of properly-formed comma-separated email addresses. Any trailing and leading whitespace is removed from each address in the list. If the `required` attribute is present, at least one email address is required.

Some browsers support the appearance of the `list` of options from the associated `<datalist>` for subsequent email addresses when `multiple` is present. Others do not.

#### file input

When `multiple` is set on the file input type, the user can select one or more files:

```
<p>
  <label for="uploads"> Choose the images you want to upload: </label>
  <input
    type="file"
    id="uploads"
    name="uploads"
    accept=".jpg, .jpeg, .png, .svg, .gif"
    multiple />
</p>
<p>
  <label for="text">Pick a text file to upload: </label>
  <input type="file" id="text" name="text" accept=".txt" />
</p>
```

Note the difference in appearance between the example with `multiple` set and the other `file` input without.

If these inputs are included in a form submitted using `method="get"`, each selected file's name is added to URL parameters as `?uploads=img1.jpg&uploads=img2.svg`. To submit the files as multipart form data, use `method="post"` and `enctype="multipart/form-data"`. See the `<form>` element and sending form data for more information.

#### select

The `multiple` attribute on the `<select>` element represents a control for selecting zero or more options from the list of options. Otherwise, the `<select>` element represents a control for selecting a single `<option>` from the list of options. The control generally has a different appearance based on the presence of the multiple attribute, with most browsers displaying a scrolling list box instead of a single line dropdown when the attribute is present.

```
<p>
  <label for="dwarfs">Select the dwarf woodsman you like:</label>
  <select multiple name="dwarfs" id="dwarfs">
    <option>grumpy@woodworkers.com</option>
    <option>happy@woodworkers.com</option>
    <option>sleepy@woodworkers.com</option>
    <option>bashful@woodworkers.com</option>
    <option>sneezy@woodworkers.com</option>
    <option>dopey@woodworkers.com</option>
```

```
    <option>doc@woodworkers.com</option>
  </select>
</p>
<p>
  <label for="favoriteOnly">Select your favorite:</label>
  <select name="favoriteOnly" id="favoriteOnly">
    <option>grumpy@woodworkers.com</option>
    <option>happy@woodworkers.com</option>
    <option>sleepy@woodworkers.com</option>
    <option>bashful@woodworkers.com</option>
    <option>sneezy@woodworkers.com</option>
    <option>dopey@woodworkers.com</option>
    <option>doc@woodworkers.com</option>
  </select>
</p>
```

Note the difference in appearance between the two form controls.

```
/* uncomment this CSS to make the multiple the same height as the single */

/*
select[multiple] {
  height: 1.5em;
  vertical-align: top;
}
select[multiple]:focus,
select[multiple]:active {
  height: auto;
}
*/
```

There are a few ways to select multiple options in a `<select>` element with a `multiple` attribute. Depending on the operating system, mouse users can hold the `Ctrl`, `Command`, or `Shift` keys and then click multiple options to select/deselect them. Keyboard users can select multiple contiguous items by focusing on the `<select>` element, selecting an item at the top or bottom of the range they want to select using the `Up` and `Down` cursor keys to go up and down the options. The selection of non-contiguous is not as well-supported: items should be able to be selected and deselected by pressing `Space`, but support varies between browsers.

### Browser compatibility

### See also

- `<input>`
- `<select>`
- Allowing multiple email addresses

## pattern HTML attribute

The **`pattern`** attribute specifies a regular expression the form control's value should match. If a non-`null` value doesn't conform to the constraints set by the `pattern` value, the `ValidityState` object's read-only `patternMismatch` property will be true.

### Try it

```
<label for="username">Username: (3-16 characters)</label>
<input
  id="username"
  name="username"
  type="text"
  value="Sasha"
  pattern="\w{3,16}"
  required />

<label for="pin">PIN: (4 digits)</label>
<input id="pin" name="pin" type="password" pattern="\d{4,4}" required />
```

```
label {
  display: block;
  margin-top: 1em;
}

input:valid {
  background-color: palegreen;
}

input:invalid {
```

```
  background-color: lightpink;
}
```

### Overview

The `pattern` attribute is an attribute of the text, tel, email, url, password, and search input types.

The `pattern` attribute, when specified, is a regular expression which the input's `value` must match for the value to pass constraint validation. It must be a valid JavaScript regular expression, as used by the `RegExp` type, and as documented in our guide on regular expressions.

The pattern's regular expression is compiled with the `'v'` flag. This makes the regular expression unicode-aware, and also changes how character classes are interpreted. This allows character class set intersection and subtraction operations, and in addition to `]` and `\`, the following characters must be escaped using a `\` backslash if they represent literal characters: `(`, `)`, `[`, `{`, `}`, `/`, `-`, `|`. Before mid-2023, the `'u'` flag was specified instead; if you're updating older code, read the `unicodeSets` reference.

The pattern's regular expression must match the entire input's `value`, rather than matching a substring - as if a `^(?:` were implied at the start of the pattern and `)$` at the end.

No forward slashes should be specified around the pattern text. No regular expression is applied if the attribute value is absent, an empty string, or invalid.

Some of the input types supporting the pattern attribute, notably the email and url input types, have expected value syntaxes that must be matched. If the pattern attribute isn't present, and the value doesn't match the expected syntax for that value type, the `ValidityState` object's read-only `typeMismatch` property will be true.

### Constraint validation

If the input's value is not the empty string and the value does not match the entire regular expression, there is a constraint violation reported by the `ValidityState` object's `patternMismatch` property being `true`.

> **Note:** If the `pattern` attribute is specified with no value, its value is implicitly the empty string. Thus, **any non-empty** input `value` will result in constraint violation.

### Usability and accessibility considerations

When including a `pattern`, provide a description of the pattern in visible text near the control. Additionally, include a `title` attribute which gives a description of the pattern. User agents may use the title contents during constraint validation to tell the user that the pattern is not matched. Some browsers show a tooltip with title contents, improving usability for sighted users. Additionally, assistive technology may read the title aloud when the control gains focus, but this should not be relied upon for accessibility.

Only relying on the `title` attribute for the visual display of text content is discouraged, as many user agents do not expose the attribute in an accessible manner. Although some browsers show a tooltip when an element with a title is hovered, that leaves out keyboard-only and touch-only users. This is one of the several reasons you must include information informing users how to fill out the control to match the requirements.

While `title`s are used by some browsers to populate error messaging, because browsers sometimes also show the title as text on hover, it therefore shows in non-error situations, so be careful not to word titles as if an error has occurred.

### Examples

### Matching a phone number

Given the following:

```
<p>
  <label>
    Enter your phone number in the format (123) - 456 - 7890 (<input
      name="tel1"
      type="tel"
      pattern="[0-9]{3}"
      placeholder="###"
      aria-label="3-digit area code"
      size="2" />) -
    <input
      name="tel2"
      type="tel"
      pattern="[0-9]{3}"
      placeholder="###"
      aria-label="3-digit prefix"
      size="2" />
    -
    <input
      name="tel3"
```

```
      type="tel"
      pattern="[0-9]{4}"
      placeholder="####"
      aria-label="4-digit number"
      size="3" />
  </label>
</p>
```

Here we have 3 sections for a north American phone number with an implicit label encompassing all three components of the phone number, expecting 3-digits, 3-digits and 4-digits respectively, as defined by the `pattern` attribute set on each.

If the values are too long or too short, or contain characters that aren't digits, the `patternMismatch` will be true. When `true`, the element matches the `:invalid` CSS pseudo-classes.

```
input:invalid {
  border: red solid 3px;
}
```

If we had used `minlength` and `maxlength` attributes instead, we may have seen `validityState.tooLong` or `validityState.tooShort` being true.

### Specifying a pattern

You can use the `pattern` attribute to specify a regular expression that the inputted value must match in order to be considered valid (see Validating against a regular expression for a crash course on using regular expressions to validate inputs).

The example below restricts the value to 4-8 characters and requires that it contain only lower-case letters.

```
<form>
  <div>
    <label for="uname">Choose a username: </label>
    <input
      type="text"
      id="uname"
      name="name"
      required
      size="45"
      pattern="[a-z]{4,8}"
      title="4 to 8 lowercase letters" />
    <span class="validity"></span>
    <p>Usernames must be lowercase and 4-8 characters in length.</p>
  </div>
  <div>
    <button>Submit</button>
  </div>
</form>
```

This renders like so:

### See also

- Constraint validation
- Forms: Data form validation
- Regular Expressions

## placeholder HTML attribute

The **`placeholder`** attribute defines the text displayed in a form control when the control has no value. The placeholder text should provide a brief hint to the user as to the expected type of data that should be entered into the control.

Effective placeholder text includes a word or short phrase that hints at the expected data type, not an explanation or prompt. The placeholder must not be used instead of a `<label>`. As the placeholder is not visible if the value of the form control is not null, using `placeholder` instead of a `<label>` for a prompt harms usability and accessibility.

The `placeholder` attribute is supported by the following input types: `text`, `search`, `url`, `tel`, `email`, and `password`. It is also supported by the `<textarea>` element. The example below shows the `placeholder` attribute in use to explain the expected format of an input field.

> **Note:** Except in `<textarea>` elements, the `placeholder` attribute can't include any line feeds (LF) or carriage returns (CR). If either is included in the value, the placeholder text will be clipped.

### Accessibility concerns

Placeholders should only be used to show an example of the type of data that should be entered into a form; never as a replacement for a `<label>` element; doing so harms accessibility and user experience.

The `<label>` text is visually and programmatically associated with its corresponding form control. Screen readers don't announce placeholder content by default, but they do announce label content; it's the label that informs assistive technology users what data should be entered in the control. Labels also improve user experience for users of pointing devices: When a user clicks, touches, or taps a `<label>`, focus is moved to the label's associated form control.

Placeholders cannot be relied upon as a replacement for a label even for those not relying on assistive technology. Placeholder text is displayed at lower color contrast than the default form control text. This is by design, as you don't want users to be confused by what is placeholder text versus what is a filled-in form field. However, this lack of contrast can cause issues for low-vision users. Additionally, placeholder text disappears from form fields when users start entering text. If the placeholder text contains instructional information or examples that disappear, it can be confusing to users with cognitive issues and can make the form inaccessible if the placeholder contained the label.

### Example

#### HTML

```
<label for="name">Enter your name:</label>
<input type="text" id="name" name="name" placeholder="e.g. Mike Shinoda" />
```

### Result

### Browser compatibility

### See also

- HTML `title`
- CSS `:placeholder-shown` pseudo-class selector
- CSS `::placeholder` pseudo-element selector

## readonly HTML attribute

The Boolean **`readonly`** attribute, when present, makes the element not mutable, meaning the user cannot edit the control.

### Try it

```
<label for="firstName">First Name:</label>
<input id="firstName" name="firstName" type="text" value="Adam" />

<label for="age">Age:</label>
<input id="age" name="age" type="number" value="42" readonly />

<label for="hobbies">Hobbies:</label>
<textarea id="hobbies" name="hobbies" readonly>Baseball</textarea>
```

```
label {
  display: block;
  margin-top: 1em;
}

input:read-only,
textarea:read-only {
  background-color: silver;
}
```

### Overview

If the `readonly` attribute is specified on an input element, because the user cannot edit the input, the element does not participate in constraint validation.

The `readonly` attribute is supported by textual form controls, including:

- `<input>` elements of type:
  - `text`
  - `search`
  - `tel`
  - `url`
  - `email`
  - `password`
  - `date`
  - `month`

    - `week`
    - `time`
    - `datetime-local`
    - `number`
- `<textarea>`

The attribute is not relevant to all other elements, including `<select>` and `<button>`. It also does not apply to non-textual input elements, including:

- `hidden`
- `range`
- `color`
- `checkbox`
- `radio`
- `file`
- `submit`
- `image`
- `reset`
- `button`

Inputs that support the `readonly` attribute but don't have the attribute set match the `:read-write` pseudo-class. All other elements match the `:read-only` pseudo-class.

### Attribute interactions

The difference between `disabled` and `readonly` is that read-only controls can still function and are still focusable, whereas disabled controls cannot receive focus and are not submitted with the form and generally do not function as controls until they are enabled.

Because a read-only field cannot have its value changed by a user interaction, `required` does not have any effect on inputs with the `readonly` attribute also specified.

The only way to modify dynamically the value of the readonly attribute is through a script.

> **Note:** The `required` attribute is not permitted on inputs with the `readonly` attribute specified.

### Usability

Browsers display the `readonly` attribute.

### Constraint validation

If the element is read-only, then the element's value cannot be updated by the user, and does not participate in constraint validation.

### Example

### HTML

```
<div class="group">
  <input type="text" value="Some value" readonly id="text" />
  <label for="text">Text box</label>
</div>
<div class="group">
  <input type="date" value="2020-01-01" readonly id="date" />
  <label for="date">Date</label>
</div>
<div class="group">
  <input type="email" value="Some value" readonly id="email" />
  <label for="email">Email</label>
</div>
<div class="group">
  <input type="password" value="Some value" readonly id="pwd" />
  <label for="pwd">Password</label>
</div>
<div class="group">
  <textarea readonly id="ta">Some value</textarea>
  <label for="ta">Message</label>
</div>
```

### Result

### Browser compatibility

### See also

- `:read-only` and `:read-write`
- `<input>`
- `<select>`

### rel HTML attribute

The **`rel`** attribute defines the relationship between a linked resource and the current document. Valid on `<link>`, `<a>`, `<area>`, and `<form>`, the supported values depend on the element on which the attribute is found.

The type of relationships is given by the value of the `rel` attribute, which, if present, must have a value that is an unordered set of unique space-separated keywords. Differently from a `class` name, which does not express semantics, the `rel` attribute must express tokens that are semantically valid for both machines and humans. The current registries for the possible values of the `rel` attribute are the IANA link relation registry, the HTML Living Standard, and the freely-editable existing-rel-values page in the microformats wiki, as suggested by the Living Standard. If a `rel` attribute not present in one of the three sources above is used some HTML validators (such as the W3C Markup Validation Service) will generate a warning.

The following table lists some of the most important existing keywords. Every keyword within a space-separated value should be unique within that value.

| `rel` value | Description | `<link>` | `<a>` and `<area>` | `<form>` |
|---|---|---|---|---|
| `alternate` | Alternate representations of the current document. | Link | Link | Not allowed |
| `author` | Author of the current document or article. | Link | Link | Not allowed |
| `bookmark` | Permalink for the nearest ancestor section. | Not allowed | Link | Not allowed |
| `canonical` | Preferred URL for the current document. | Link | Not allowed | Not allowed |
| `compression-dictionary` | Link to a compression dictionary that can be used to compress future downloads for resources on this site. | Link | Not allowed | Not allowed |
| `dns-prefetch` | Tells the browser to preemptively perform DNS resolution for the target resource's origin. | External Resource | Not allowed | Not allowed |
| `external` | The referenced document is not part of the same site as the current document. | Not allowed | Annotation | Annotation |
| `expect` | When used with `blocking="render"`, allows the page to be render-blocked until the essential parts of the document are parsed so it will render consistently. | Link | Not allowed | Not allowed |
| `help` | Link to context-sensitive help. | Link | Link | Link |
| `icon` | An icon representing the current document. | External Resource | Not allowed | Not allowed |
| `license` | Indicates that the main content of the current document is covered by the copyright license described by the referenced document. | Link | Link | Link |
| `manifest` | Web app manifest. | Link | Not allowed | Not allowed |
| `me` | Indicates that the current document represents the person who owns the linked content. | Link | Link | Not allowed |
| `modulepreload` | Tells to browser to preemptively fetch the script and store it in the document's module map for later evaluation. Optionally, the module's dependencies can be fetched as well. | External Resource | Not allowed | Not allowed |
| `next` | Indicates that the current document is a part of a series and that the next document in the series is the referenced document. | Link | Link | Link |
| `nofollow` | Indicates that the current document's original author or publisher does not endorse the referenced document. | Not allowed | Annotation | Annotation |
| `noopener` | Creates a top-level browsing context that is not an auxiliary browsing context if the hyperlink would create either of those, to begin with (i.e., has an appropriate `target` attribute value). | Not allowed | Annotation | Annotation |
| `noreferrer` | No `Referer` header will be included. Additionally, has the same effect as `noopener`. | Not allowed | Annotation | Annotation |
| `opener` | Creates an auxiliary browsing context if the hyperlink would otherwise create a top-level browsing context that is not an auxiliary browsing context (i.e., has `"_blank"` as `target` attribute value). | Not allowed | Annotation | Annotation |

| `rel` value | Description | `<link>` | `<a>` and `<area>` | `<form>` |
|---|---|---|---|---|
| `pingback` | Gives the address of the pingback server that handles pingbacks to the current document. | External Resource | Not allowed | Not allowed |
| `preconnect` | Specifies that the user agent should preemptively connect to the target resource's origin. | External Resource | Not allowed | Not allowed |
| `prefetch` | Specifies that the user agent should preemptively fetch and cache the target resource as it is likely to be required for a followup navigation. | External Resource | Not allowed | Not allowed |
| `preload` | Specifies that the user agent must preemptively fetch and cache the target resource for current navigation according to the potential destination given by the `as` attribute (and the priority associated with the corresponding destination). | External Resource | Not allowed | Not allowed |
| `prerender` | Specifies that the user agent should preemptively fetch the target resource and process it in a way that helps deliver a faster response in the future. This feature is superseded by the Speculation Rules API. | External Resource | Not allowed | Not allowed |
| `prev` | Indicates that the current document is a part of a series and that the previous document in the series is the referenced document. | Link | Link | Link |
| `privacy-policy` | Gives a link to information about the data collection and usage practices that apply to the current document. | Link | Link | Not allowed |
| `search` | Gives a link to a resource that can be used to search through the current document and its related pages. | Link | Link | Link |
| `stylesheet` | Imports a style sheet. | External Resource | Not allowed | Not allowed |
| `tag` | Gives a tag (identified by the given address) that applies to the current document. | Not allowed | Link | Not allowed |
| `terms-of-service` | Link to the agreement, or terms of service, between the document's provider and users who wish to use the document. | Link | Link | Not allowed |

The `rel` attribute is relevant to the `<link>`, `<a>`, `<area>`, and `<form>` elements, but some values only relevant to a subset of those elements. Like all HTML keyword attribute values, these values are case-insensitive.

The `rel` attribute has no default value. If the attribute is omitted or if none of the values in the attribute are supported, then the document has no particular relationship with the destination resource other than there being a hyperlink between the two. In this case, on `<link>` and `<form>`, if the `rel` attribute is absent, has no keywords, or if not one or more of the space-separated keywords above, then the element does not create any links. `<a>` and `<area>` will still create links, but without a defined relationship.

### Value

`alternate`

Indicates an alternate representation of the current document. Valid for `<link>`, `<a>`, and `<area>`, the meaning depends on the values of the other attributes.

- With the `stylesheet` keyword on a `<link>`, it creates an alternate stylesheet.

```
<!-- a persistent style sheet -->
<link rel="stylesheet" href="default.css" />
<!-- alternate style sheets -->
<link
  rel="alternate stylesheet"
  href="highcontrast.css"
  title="High contrast" />
```

- With an `hreflang` attribute that differs from the document language, it indicates a translation.
- With the `type` attribute value of `"application/rss+xml"` or `"application/atom+xml"`, it creates a hyperlink referencing a syndication feed.

```
<link
  rel="alternate"
  type="application/atom+xml"
  href="posts.xml"
  title="Blog" />
```

- Otherwise, it creates a hyperlink referencing an alternate representation of the current document, whose nature is given by the `hreflang` and `type` attributes.

- If `hreflang` is given alongside `alternate`, and the value of `hreflang` is different from the current document's language, it indicates that the referenced document is a translation.
- If `type` is given alongside `alternate`, it indicates that the referenced document is an alternative format (such as a PDF).
- The `hreflang` and `type` attributes may both be given alongside `alternate`.

```
<link
  rel="alternate"
  href="https://developer.mozilla.org/fr/html/print"
  hreflang="fr"
  type="text/html"
  media="print"
  title="French HTML (for printing)" />
<link
  rel="alternate"
  href="https://developer.mozilla.org/fr/pdf"
  hreflang="fr"
  type="application/pdf"
  title="French PDF" />
```

`author`

Indicates the referenced document provides further information about the author of the current document or article. Relevant for `<link>`, `<a>`, and `<area>` elements.

With `<a>` and `<area>`, it indicates the linked document (or `mailto:`) provides information about the author of the nearest `<article>` ancestor if there is one, otherwise the entire document.

With `<link>`, it represents the author of the entire document.

> **Note:** For historical reasons, the obsolete attribute value `rev="made"` is treated as `rel="author"`.

`bookmark`

Relevant as the `rel` attribute value for the `<a>` and `<area>` elements. Gives a permalink for the nearest ancestor `<article>` element, if there is one. If there is no ancestor `<article>` element, gives a permalink for the section the linking element is most closely associated with.

`canonical`

Valid for `<link>`, it defines the preferred URL for the current document, which helps search engines reduce duplicate content.

`compression-dictionary`

Valid for `<link>`, it defines a compression dictionary that can be used to compress future downloads for resources on this site so the download sizes of those resources are smaller than standard compression.

`dns-prefetch`

Relevant for the `<link>` element both in the `<body>` and `<head>`, it tells the browser to preemptively perform DNS resolution for the target resource's origin. Useful for resources the user will likely need, it helps reduce latency and thereby improves performance when the user does access the resources as the browser preemptively performed DNS resolution for the origin of the specified resource. See dns-prefetch described in resource hints.

`external`

Relevant to `<form>`, `<a>`, and `<area>`, it indicates the referenced document is not part of the current site. This can be used with attribute selectors to style external links in a way that indicates to the user that they will be leaving the current site.

`expect`

Allows the page to be render-blocked until the essential parts of the document are parsed so it will render consistently. Note that render-blocking occurs only when supplemented with the `blocking="render"` attribute.

> **Note:** See Stabilizing page state to make cross-document transitions consistent for more information on its use.

`help`

Relevant to `<form>`, `<link>`, `<a>`, and `<area>`, the `help` keyword indicates that the linked to content provides context-sensitive help, providing information for the parent of the element defining the hyperlink, and its children. When used within `<link>`, the help is for the whole document. When included with `<a>` and `<area>` and supported, the default `cursor` will be `help` instead of `pointer`.

`icon`

Valid with `<link>`, the linked resource represents the icon, a resource for representing the page in the user interface, for the current document.

The most common use for the `icon` value is the favicon:

```
<link rel="icon" href="favicon.ico" />
```

If there are multiple `<link rel="icon">`s, the browser uses their `media`, `type`, and `sizes` attributes to select the most appropriate icon. If several icons are equally appropriate, the last one is used. If the most appropriate icon is later found to be inappropriate, for example because it uses an unsupported format, the browser proceeds to the next-most appropriate, and so on.

> **Note:** The `crossorigin` attribute is not supported for `rel="icon"` in Chromium-based browsers. See the open Chromium issue.

> **Note:** Apple's iOS does not use this link type, nor the `sizes` attribute, like others mobile browsers do, to select a webpage icon for Web Clip or a start-up placeholder. Instead it uses the non-standard `apple-touch-icon` and `apple-touch-startup-image` respectively.

> **Note:** The `shortcut` link type is often seen before `icon`, but this link type is non-conforming, ignored and **web authors must not use it anymore**.

`license`

Valid on the `<a>`, `<area>`, `<form>`, `<link>` elements, the `license` value indicates that the hyperlink leads to a document describing the licensing information; that the main content of the current document is covered by the copyright license described by the referenced document. If not inside the `<head>` element, the standard doesn't distinguish between a hyperlink applying to a specific part of the document or to the document as a whole. Only the data on the page can indicate this.

```
<link rel="license" href="#license" />
```

> **Note:** Although recognized, the synonym `copyright` is incorrect and must be avoided.

`manifest`

Web app manifest. Requires the use of the CORS protocol for cross-origin fetching.

`modulepreload`

Useful for improved performance, and relevant to the `<link>` anywhere in the document, setting `rel="modulepreload"` tells the browser to preemptively fetch the script (and dependencies) and store it in the document's module map for later evaluation. `modulepreload` links can ensure network fetching is done with the module ready (but not evaluated) in the module map before it is necessarily needed. See also `modulepreload`.

`next`

Relevant to `<form>`, `<link>`, `<a>`, and `<area>`, the `next` values indicates that the current document is a part of a series, and that the next document in the series is the referenced document. When included in a `<link>`, browsers may assume that document will be fetched next, and treat it as a resource hint.

`nofollow`

Relevant to `<form>`, `<a>`, and `<area>`, the `nofollow` keyword tells search engine spiders to ignore the link relationship. The nofollow relationship may indicate the current document's owner does not endorse the referenced document. It is often included by Search Engine Optimizers pretending their link farms are not spam pages.

`noopener`

Relevant to `<form>`, `<a>`, and `<area>`, creates a top-level browsing context that is not an auxiliary browsing context if the hyperlink would create either of those to begin with (i.e., has an appropriate `target` attribute value). In other words, it makes the link behave as if `window.opener` were null and `target="_parent"` were set.

This is the opposite of `opener`.

`noreferrer`

Relevant to `<form>`, `<a>`, and `<area>`, including this value makes the referrer unknown (no `Referer` header will be included), and creates a top-level browsing context as if `noopener` were also set.

`opener`

Creates an auxiliary browsing context if the hyperlink would otherwise create a top-level browsing context that is not an auxiliary browsing context (i.e., has `"_blank"` as `target` attribute value). Effectively, the opposite of noopener.

`pingback`

Gives the address of the pingback server that handles pingbacks to the current document. See the Pingback specification.

`preconnect`

Provides a hint to the browser suggesting that it open a connection to the linked website in advance, without disclosing any private information or downloading any content, so that when the link is followed the linked content can be fetched more quickly.

`prefetch`

Specifies that the user agent should preemptively fetch and cache the target resource as it is likely to be required for a followup navigation. See prefetch for more information.

`preload`

Specifies that the user agent must preemptively fetch and cache the target resource for current navigation according to the potential destination given by the `as` attribute (and the priority associated with the corresponding destination). See the page for the `preload` value.

`prerender`

Specifies that the user agent should preemptively fetch the target resource and process it in a way that helps deliver a faster response in the future, for example by fetching its subresources or performing some rendering. This feature is superseded by the Speculation Rules API.

`prev`

Similar to the `next` keyword, relevant to `<form>`, `<link>`, `<a>`, and `<area>`, the `prev` values indicates that the current document is a part of a series, and that the link references a previous document in the series is the referenced document.

Note: The synonym `previous` is incorrect and should not be used.

`privacy-policy`

Valid for `<a>`, `<area>`, and `<link>` elements, the `privacy-policy` value indicates that the referenced document is the Privacy Policy which describes the data collection and usage practices of the current document.

`search`

Relevant to `<form>`, `<link>`, `<a>`, and `<area>` elements, the `search` keywords indicates that the hyperlink references a document whose interface is specially designed for searching in the current document, site, and related resources, providing a link to a resource that can be used to search.

If the `type` attribute is set to `application/opensearchdescription+xml` the resource is an OpenSearch plugin that can be easily added to the interface of Firefox.

`stylesheet`

Valid for the `<link>` element, it imports an external resource to be used as a stylesheet. The `type` attribute is not needed if it's a `text/css` stylesheet, as that is the default value. If it's not a stylesheet of type `text/css` it is best to declare the type.

While this attribute defines the link as being a stylesheet, the interaction with other attributes and other key terms within the rel value impact whether the stylesheet is downloaded and/or used.

When used with the `alternate` keyword, it defines an alternative style sheet. In this case, include a non-empty `title`.

The external stylesheet will not be used or even downloaded if the media does not match the value of the `media` attribute.

Requires the use of the CORS protocol for cross-origin fetching.

`tag`

Valid for the `<a>`, and `<area>` elements, it gives a tag (identified by the given address) that applies to the current document. The tag value denotes that the link refers to a document describing a tag applying to the document on which it is located. This link type is not meant for tags within a tag cloud, as those tags apply to a group of pages, whereas the `tag` value of the `rel` attribute is for a single document.

`terms-of-service`

Valid for `<a>`, `<area>`, and `<link>` elements, the `terms-of-service` value indicates that the referenced document is the Terms of Service that describes the agreements between the current document's provider and users who wish to use the document provided.

### Non-standard values

`apple-touch-icon`

Specifies the icon for a web application on an iOS device.

### Browser compatibility

### See also

- `HTMLLinkElement.relList`
- `HTMLAnchorElement.relList`
- `HTMLAreaElement.relList`

## rel="alternate stylesheet" HTML attribute value

The **`alternate stylesheet`** keyword pair, when used as a value for the `rel` attribute of the `<link>` element, indicates that the target resource is an *alternative style sheet*. Specifying **alternative style sheets** in a web page allows users to see multiple versions of a page based on their needs or preferences.

> **Note:** This feature is not well supported in browsers without an extension. To offer alternative presentations that work with a user's existing preferences, see the CSS media features `prefers-color-scheme` and `prefers-contrast`.

Firefox lets users select alternate stylesheets using the *View* > *Page Style* submenu, which displays the values of the `title` attributes. Other browsers require an extension to enable this functionality. The web page can also provide its own user interface to let users switch styles.

### Examples

#### Specifying alternate stylesheets

Alternate stylesheets are specified using `<link>` elements with `rel="alternate stylesheet"` and `title="…"` attributes. For example:

```
<link href="reset.css" rel="stylesheet" />

<link href="default.css" rel="stylesheet" title="Default Style" />
<link href="fancy.css" rel="alternate stylesheet" title="Fancy" />
<link href="basic.css" rel="alternate stylesheet" title="Basic" />
```

In this example, the styles "Default Style", "Fancy", and "Basic" will be listed in the Firefox *Page Style* submenu with "Default Style" pre-selected. When the user selects a different style, the page will immediately be re-rendered using that style sheet.

No matter what style is selected, the rules from the `reset.css` stylesheet will always be applied.

### Try it out

Try a working example here.

### Details

Any stylesheet in a document falls into one of the following categories:

- **Persistent** (has `rel="stylesheet"`, no `title=""`): always applies to the document.
- **Preferred** (has `rel="stylesheet"`, with `title="…"` specified): applied by default, but disabled if an alternate stylesheet is selected. **There can only be one preferred stylesheet**, so providing stylesheets with different title attributes will cause some of them to be ignored.
- **Alternate** (`rel="alternate stylesheet"`, with `title="…"` specified): disabled by default, can be selected.

In cases where a stylesheet menu exists, when style sheets are referenced with a `title` attribute on the `<link rel="stylesheet">` or `<style>` element, the title becomes one of the choices offered to the user. Style sheets linked with the same `title` are part of the same choice. Style sheets linked without a `title` attribute are always applied.

Use `rel="stylesheet"` to link to the default style, and `rel="alternate stylesheet"` to link to alternative style sheets. This tells the browser which style sheet title should be selected by default, and makes that default selection apply in browsers that do not support alternate style sheets.

### See also

- CSS
- Using dynamic styling information

## rel="compression-dictionary" HTML attribute value

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`compression-dictionary`** keyword for the `rel` attribute of the `<link>` element is a link that browsers can use to download a compression dictionary that can be used to compress future downloads for resources on this site so the download sizes of those resources are smaller than standard compression.

**Note:** if the website has a `Content-Security-Policy` header, the `connect-src` directive (or `default-src`, if `connect-src` is not set) must permit the location of the dictionary resource to avoid the request being blocked.

See the Compression Dictionary Transport guide for more information.

### See also

- Compression Dictionary Transport guide

#### rel="dns-prefetch" HTML attribute value

The **`dns-prefetch`** keyword for the `rel` attribute of the `<link>` element is a hint to browsers that the user is likely to need resources from the target resource's origin, and therefore the browser can likely improve the user experience by preemptively performing DNS resolution for that origin.

See Using dns-prefetch for more details.

### See also

- Speculative loading for a comparison of `<link rel="dns-prefetch">` and other similar performance improvement features.

#### rel="manifest" HTML attribute value

The **`manifest`** keyword for the `rel` attribute of the `<link>` element indicates that the target resource is a Web app manifest.

#### rel="me" HTML attribute value

The **`me`** keyword for the `rel` attribute of the `<link>` and `<a>` elements indicates that the current resource is represented by the linked party. The `me` value was introduced in the XHTML Friends Network (XFN) specification.

```
<link rel="me" href="example.com" />
```

The `rel="me"` attribute is used in RelMeAuth and Web sign in specifications as a way to enable a person to identify themselves to a web service using their domain name or a particular URL.

### Browser compatibility

Browser compatibility is irrelevant because `rel="me"` is used by web services and not by browsers.

#### rel="modulepreload" HTML attribute value

The **`modulepreload`** keyword, for the `rel` attribute of the `<link>` element, provides a declarative way to preemptively fetch a module script, parse and compile it, and store it in the document's module map for later execution.

Preloading allows modules and their dependencies to be downloaded early, and can also significantly reduce the overall download and processing time. This is because it allows pages to fetch modules in parallel, instead of sequentially as each module is processed and its dependencies are discovered. Note however that you can't just preload everything! What you choose to preload must be balanced against other operations that might then be starved, adversely affecting user experience.

Links with `rel="modulepreload"` are similar to those with `rel="preload"`. The main difference is that `preload` just downloads the file and stores it in the cache, while `modulepreload` gets the module, parses and compiles it, and puts the results into the module map so that it is ready to execute.

When using `modulepreload` the fetch request mode is always `cors`, and the `crossorigin` property is used to determine the request credential mode. If `crossorigin` is set to `anonymous` or `""` (default), then the credentials mode is `same-origin`, and user credentials such as cookies and authentication are only sent for requests with the `same-origin`. If `crossorigin` is set to `use-credentials` then the credentials mode is `include`, and user credentials for both single- and cross-origin requests.

The `as` attribute is optional for links with `rel="modulepreload"`, and defaults to `"script"`. It can be set to `"script"`, `"style"`, `"json"`, `"text"`, or any script-like destination, such as `"audioworklet"`, `"paintworklet"`, `"serviceworker"`, `"sharedworker"`, or `"worker"`. The full set of allowed values is defined in the HTML spec's module preload destination mapping. For the list of script-like destinations, refer to the Fetch spec. An `Event` named "error" is fired on the element if any other destination is used.

A browser *may* additionally also choose to automatically fetch any dependencies of the module resource. Note however that this is a browser-specific optimization — the only approach to ensure that all browsers will try to preload a module's dependencies is to individually specify them! Further, the events named `load` or `error` fire immediately following success or failure of loading the *specified* resources. If dependencies are automatically fetched, no additional events are fired in the main thread (although you might monitor additional requests in a service worker or on the server).

### Examples

Consider the basic-modules example (live version), introduced in the JavaScript modules guide.

This has a file structure as shown below, consisting of the top level module `main.js`, which statically imports two dependency modules `modules/canvas.js` and `modules/square.js` using the `import` statement.

```
index.html
main.js
modules/
    canvas.js
    square.js
```

The HTML for the example below shows how `main.js` is fetched in a `<script>` element. Only after `main.js` has loaded does the browser discover and fetch the two dependency modules.

```
<!doctype html>
<html lang="en-US">
  <head>
    <meta charset="utf-8" />
    <title>Basic JavaScript module example</title>
    <script type="module" src="https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Attributes/rel/modulepreload/main.js">
</script>
  </head>
  <body></body>
</html>
```

The HTML below updates the example to use `<link>` elements with `rel="modulepreload"` for the main file and each of the dependency modules. This is much faster because the three modules all start downloading asynchronously and in parallel before they are needed. By the time `main.js` has been parsed and its dependencies are known, they have already been fetched and downloaded.

```
<!doctype html>
<html lang="en-US">
  <head>
    <meta charset="utf-8" />
    <title>Basic JavaScript module example</title>
    <link rel="modulepreload" href="main.js" />
    <link rel="modulepreload" href="modules/canvas.js" />
    <link rel="modulepreload" href="modules/square.js" />
    <script type="module" src="https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Attributes/rel/modulepreload/main.js">
</script>
  </head>
  <body></body>
</html>
```

### See also

- Speculative loading for a comparison of `<link rel="modulepreload">` and other similar performance improvement features.
- Preloading modules on web.dev

## rel="noopener" HTML attribute value

The **`noopener`** keyword for the `rel` attribute of the `<a>`, `<area>`, and `<form>` elements instructs the browser to navigate to the target resource without granting the new browsing context access to the document that opened it — by not setting the `Window.opener` property on the opened window (it returns `null`).

This is especially useful when opening untrusted links, in order to ensure they cannot tamper with the originating document via the `Window.opener` property (see About rel=noopener for more details), while still providing the `Referer` HTTP header (unless `noreferrer` is used as well).

Note that when `noopener` is used, nonempty target names other than `_top`, `_self`, and `_parent` are all treated like `_blank` in terms of deciding whether to open a new window/tab.

> **Note:** Setting `target="_blank"` on `<a>`, `<area>` and `<form>` elements implicitly provides the same `rel` behavior as setting `rel="noopener"` which does not set `window.opener`.

### Browser compatibility

## rel="noreferrer" HTML attribute value

The **`noreferrer`** keyword for the `rel` attribute of the `<a>`, `<area>`, and `<form>` elements instructs the browser, when navigating to the target resource, to omit the `Referer` header and otherwise leak no referrer information — and additionally to behave as if the `noopener` keyword were also specified.

### Browser compatibility

### rel="preconnect" HTML attribute value

The **preconnect** keyword for the `rel` attribute of the `<link>` element is a hint to browsers that the user is likely to need resources from the target resource's origin, and therefore the browser can likely improve the user experience by preemptively initiating a connection to that origin. Preconnecting speeds up future loads from a given origin by preemptively performing part or all of the handshake (DNS+TCP for HTTP, and DNS+TCP+TLS for HTTPS origins).

`<link rel="preconnect">` will provide a benefit to any future cross-origin HTTP request, navigation or subresource. It has no benefit on same-origin requests because the connection is already open.

If a page needs to make connections to many third-party domains, preconnecting them all can be counterproductive. The `<link rel="preconnect">` hint is best used for only the most critical connections. For the others, just use `<link rel="dns-prefetch">` to save time on the first step — the DNS lookup.

#### Examples

```
<link rel="preconnect" href="https://example.com" />
```

You can also implement preconnect as an HTTP `Link` header, for example:

```
Link: <https://example.com>; rel="preconnect"
```

#### See also

- Speculative loading for a comparison of `<link rel="preconnect">` and other similar performance improvement features.

### rel="prefetch" HTML attribute value

The **prefetch** keyword for the `rel` attribute of the `<link>` element provides a hint to browsers that the user is likely to need the target resource for future navigations, and therefore the browser can likely improve the user experience by preemptively fetching and caching the resource. `<link rel="prefetch">` is used for same-site navigation resources, or for subresources used by same-site pages.

The result is kept in the HTTP cache on disk. Because of this it is useful for prefetching subresources, even if they are not used by the current page. You could also use it to prefetch the next document the user will likely visit on the site. However, as a result you need to be careful with headers — for example certain Cache-Control headers could block prefetching (for example `no-cache` or `no-store`).

> **Note:** Because of such limitations, you are advised to use the Speculation Rules API for document prefetches instead, where it is supported.

`<link rel="prefetch">` is functionally equivalent to a `fetch()` call with a `priority: "low"` option set on it, except that the former will generally have an even lower priority, and it will have a `Sec-Purpose: prefetch` header set on the request. Note that in general browsers will give prefetch resources a lower priority than preload ones (e.g., requested via `<link rel="preload">`) — the current page is more important than the next.

The fetch request for a `prefetch` operation results in an HTTP request that includes the HTTP header `Sec-Purpose: prefetch`. A server might use this header to change the cache timeouts for the resources, or perform other special handling. The request will also include the `Sec-Fetch-Dest` header with the value set to `empty`.

The `Accept` header in the request will match the value used for normal navigation requests. This allows the browser to find the matching cached resources following navigation.

#### Examples

##### Basic prefetch

```
<link rel="prefetch" href="main.js" />
```

##### Navigation and subresource prefetches

Prefetching can be used to fetch both HTML and sub-resources for a possible next navigation. A common use case is to have a website landing page that fetches more "heavy-weight" resources used by the rest of the site.

```
<link rel="prefetch" href="https://developer.mozilla.org/app/style.css" />
<link rel="prefetch" href="https://developer.mozilla.org/landing-page" />
```

##### The effects of cache partitioning

Many browsers now implement some form of cache partitioning, which makes `<link rel="prefetch">` useless for resources intended for use by different top-level sites. This includes the main document when navigating cross-site. So, for example, the following prefetch:

```
<link rel="prefetch" href="https://news.example/article" />
```

Would not be accessible from `https://aggregator.example/`.

### See also

- Speculative loading for a comparison of `<link rel="prefetch">` and other similar performance improvement features.

## rel="preload" HTML attribute value

The `preload` value of the `<link>` element's `rel` attribute lets you declare fetch requests in the HTML's `<head>`, specifying resources that your page will need very soon, which you want to start loading early in the page lifecycle, before browsers' main rendering machinery kicks in. This ensures they are available earlier and are less likely to block the page's render, improving performance. Even though the name contains the term *load*, it doesn't load and execute the script but only schedules it to be downloaded and cached with a higher priority.

### The basics

You most commonly use `<link>` to load a CSS file to style your page with:

```
<link rel="stylesheet" href="styles/main.css" />
```

Here however, we will use a `rel` value of `preload`, which turns `<link>` into a preloader for any resource we want. You will also need to specify:

- The path to the resource in the `href` attribute.
- The type of resource in the `as` attribute.

An example might look like this (see our JS and CSS example source, and also live):

```
<head>
  <meta charset="utf-8" />
  <title>JS and CSS preload example</title>

  <link rel="preload" href="style.css" as="style" />
  <link rel="preload" href="main.js" as="script" />

  <link rel="stylesheet" href="style.css" />
</head>

<body>
  <h1>bouncing balls</h1>
  <canvas></canvas>

  <script src="https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Attributes/rel/preload/main.js" defer></script>
</body>
```

Here we preload our CSS and JavaScript files so they will be available as soon as they are required for the rendering of the page later on. This example is trivial, as the browser probably discovers the `<link rel="stylesheet">` and `<script>` elements in the same chunk of HTML as the preloads, but the benefits can be seen much more clearly the later resources are discovered and the larger they are. For example:

- Resources that are pointed to from inside CSS, like fonts or images.
- Resources that JavaScript can request, like imported scripts.

`preload` has other advantages too. Using `as` to specify the type of content to be preloaded allows the browser to:

- Store in the cache for future requests, reusing the resource if appropriate.
- Apply the correct content security policy to the resource.
- Set the correct `Accept` request headers for it.

### What types of content can be preloaded?

Many content types can be preloaded. The possible `as` attribute values are:

- `fetch`: Resource to be accessed by a fetch or XHR request, such as an ArrayBuffer, WebAssembly binary, or JSON file.
- `font`: Font file.
- `image`: Image file.
- `script`: JavaScript file.
- `style`: CSS stylesheet.
- `track`: WebVTT file.

> **Note:** `font` and `fetch` preloading requires the `crossorigin` attribute to be set; see CORS-enabled fetches below.

> **Note:** There's more detail about these values and the web features they expect to be consumed by in the HTML spec — see Link type "preload".

> Also note that the full list of values the `as` attribute can take is governed by the HTML spec — see Link type "preload" destinations.

### Including a MIME type

`<link>` elements can accept a `type` attribute, which contains the MIME type of the resource the element points to. This is especially useful when preloading resources — the browser will use the `type` attribute value to work out if it supports that resource, and will only download it if so, ignoring it if not.

```
<head>
  <meta charset="utf-8" />
  <title>Image preload example</title>

  <link rel="preload" href="flower.avif" as="image" type="image/avif" />
</head>
<body>
  <picture>
    <source src="https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Attributes/rel/preload/flower.avif" type="image/avif" />
    <source src="https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Attributes/rel/preload/flower.webp" type="image/webp" />
    <img src="https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Attributes/rel/preload/flower.jpg" />
  </picture>
</body>
```

The code in the example above causes the `image/avif` image to be preloaded only in supporting browsers — and for users who have `image/avif` support in their browsers, causes the `image/avif` image to actually be used (since it's the first `<source>` specified). That makes the image download hopefully smaller for users who have `image/avif` support in their browsers.

Note that for users whose browsers have both `image/avif` and `image/webp` support, if in that code a `<link rel="preload" href="flower.webp" as="image" type="image/webp">` element were also specified, then *both* the `image/avif` and `image/webp` images would be preloaded — even though only one of them would actually be used.

Therefore, specifying preloading for multiple types of the same resource is discouraged. Instead, the best practice is to specify preloading only for the type the majority of your users are likely to actually use. That's why the code in the example above doesn't specify preloading for the `image/webp` image.

However, the lack of preloading doesn't prevent the `image/webp` image from actually being used by those who need it: for users whose browsers don't have `image/avif` support but do have `image/webp` support, the code in the example above does still cause the `image/webp` image to be used — but it does so without also causing it to also be preloaded unnecessarily for the majority of other users.

### CORS-enabled fetches

When preloading resources that are fetched with CORS enabled (e.g., `fetch()`, `XMLHttpRequest` or fonts), special care needs to be taken to setting the `crossorigin` attribute on your `<link>` element. The attribute needs to be set to match the resource's CORS and credentials mode, even when the fetch is not cross-origin.

As mentioned above, one interesting case where this applies is font files. Because of various reasons, these have to be fetched using anonymous-mode CORS (see Font fetching requirements).

Let's use this case as an example. You can see the full example source code on GitHub (also see it live):

```
<head>
  <meta charset="utf-8" />
  <title>Web font example</title>

  <link
    rel="preload"
    href="fonts/cicle_fina-webfont.woff2"
    as="font"
    type="font/woff2"
    crossorigin />
  <link
    rel="preload"
    href="fonts/zantroke-webfont.woff2"
    as="font"
    type="font/woff2"
    crossorigin />

  <link href="style.css" rel="stylesheet" />
</head>
<body>
  …
</body>
```

Not only are we providing the MIME type hints in the `type` attributes, but we are also providing the `crossorigin` attribute to make sure the preload's CORS mode matches the eventual font resource request.

### Including media

One nice feature of `<link>` elements is their ability to accept `media` attributes. These can accept media types or full-blown media queries, allowing you to do responsive preloading!

Let's look at an example (see it on GitHub — source code, live example):

```
<head>
  <meta charset="utf-8" />
  <title>Responsive preload example</title>

  <link
    rel="preload"
    href="bg-image-narrow.png"
    as="image"
    media="(width <= 600px)" />
  <link
    rel="preload"
    href="bg-image-wide.png"
    as="image"
    media="(width > 600px)" />

  <link rel="stylesheet" href="main.css" />
</head>
<body>
  <header>
    <h1>My site</h1>
  </header>

  <script>
    const mediaQueryList = window.matchMedia("(width <= 600px)");
    const header = document.querySelector("header");

    if (mediaQueryList.matches) {
      header.style.backgroundImage = 'url("bg-image-narrow.png")';
    } else {
      header.style.backgroundImage = 'url("bg-image-wide.png")';
    }
  </script>
</body>
```

We include `media` attributes on our `<link>` elements so that a narrow image is preloaded if the user has a narrow viewport, and a wider image is loaded if they have a wide viewport. We use `Window.matchMedia` / `MediaQueryList` to do this (see Testing media queries for more).

This same technique also applies to other resource types. For example, when used with fonts, preloading makes it more likely the font will be available at render time, reducing the chance of a flash of unstyled text (FOUT).

This doesn't have to be limited to images, or even files of the same type — think big! You could perhaps preload and display a simplified SVG diagram if the user is on a narrow screen where bandwidth and CPU is potentially more limited, or preload a complex chunk of JavaScript then use it to render an interactive 3D model if the user's resources are more plentiful.

### Scripting and preloads

> **Note:** Use `<link rel="modulepreload">` instead if you are working with JavaScript modules.

Another nice thing about these preloads is that you can execute them with script. For example, here we create a `HTMLLinkElement` instance, then attach it to the DOM:

```
const preloadLink = document.createElement("link");
preloadLink.href = "myscript.js";
preloadLink.rel = "preload";
preloadLink.as = "script";
document.head.appendChild(preloadLink);
```

This means that the browser will preload the `myscript.js` file, but not actually use it yet. To use it, you could do this:

```
const preloadedScript = document.createElement("script");
preloadedScript.src = "myscript.js";
document.body.appendChild(preloadedScript);
```

This is useful when you want to preload a script, but then defer execution until exactly when you need it.

#### See also

- Speculative loading for a comparison of `<link rel="preload">` and other similar performance improvement features.
- Preload: What Is It Good For? by Yoav Weiss

### rel="prerender" HTML attribute value

> **Non-standard:** This feature is not standardized. We do not recommend using non-standard features in production, as they have limited browser support, and may change or be removed. However, they can be a suitable alternative in specific cases where no standard option exists.

The **`prerender`** keyword for the `rel` attribute of the `<link>` element is a hint to browsers that the user might need the target resource for the next navigation, and therefore the browser can likely improve the user experience by preemptively fetching and processing the resource — for example, by fetching its subresources or performing some rendering in the background offscreen.

This feature is superseded by the Speculation Rules API.

#### See also

- Speculative loading for a comparison of `<link rel="prerender">` and other similar performance improvement features.

## required HTML attribute

The Boolean **`required`** attribute, if present, indicates that the user must specify a value for the input before the owning form can be submitted.

The `required` attribute is supported by `text`, `search`, `url`, `tel`, `email`, `password`, `date`, `month`, `week`, `time`, `datetime-local`, `number`, `checkbox`, `radio`, `file`, `<input>` types along with the `<select>` and `<textarea>` form control elements. If present on any of these input types and elements, the `:required` pseudo class will match. If the attribute is not included, the `:optional` pseudo class will match.

The attribute is not supported on, or relevant to, range and color input types, as both have default values. Type `color` defaults to `#000000`. Type `range` defaults to the midpoint between `min` and `max` — with `min` and `max` defaulting to 0 and 100 respectively in most browsers if not declared. `required` is also not supported on the hidden input type — users cannot be expected to fill out a hidden form field. Finally, `required` is not supported on any button input types, including image.

In the case of a same named group of radio buttons, if a single radio button in the group has the `required` attribute, a radio button in that group must be checked, although it doesn't have to be the one on which the attribute is applied. To improve code maintenance, it is recommended to either include the `required` attribute in every same-named radio button in the group, or else in none.

In the case of a same named group of checkbox input types, only the checkboxes with the `required` attribute are required.

> **Note:** Setting `aria-required="true"` tells a screen reader that an element (any element) is required, but has no bearing on the optionality of the element.

#### Description

#### Attribute interactions

Because a read-only field cannot be changed, `required` does not have any effect on inputs with the `readonly` attribute also specified.

#### Usability

When including the `required` attribute, provide a visible indication near the control informing the user that the `<input>`, `<select>` or `<textarea>` is required. In addition, target required form controls with the `:required` pseudo-class, styling them in a way to indicate they are required. This improves usability for sighted users. Assistive technology should inform the user that the form control is mandatory based on the required attribute, but adding `aria-required="true"` doesn't hurt, in case the browser / screen reader combination does not support `required` yet.

#### Constraint validation

If the element is required and the element's value is the empty string, then the element is suffering from `valueMissing` and the element will match the `:invalid` pseudo class.

#### Accessibility concerns

Provide an indication to users informing them the form control is required. Ensure the messaging is multi-faceted, such as through text, color, markings, and attribute, so that all users understand the requirements whether they have color blindness, cognitive differences, or are using a screen reader.

#### Example

### HTML

```
<form>
  <div class="group">
    <input type="text" />
    <label>Normal</label>
  </div>
  <div class="group">
    <input type="text" required />
    <label>Required</label>
  </div>
  <input type="submit" />
</form>
```

### Result

### Browser compatibility

### See also

- `validityState.valueMissing`
- `:required` and `:optional`
- `<input>`
- `<select>`

## size HTML attribute

The **`size`** attribute defines the width of the `<input>` and the height of the `<select>` element. For an `input` element, it defines the number of characters that the user agent allows the user to see when editing the value. For a `select` element, it defines the number of options that should be shown to the user. This must be a valid non-negative integer greater than zero.

If no `size` is specified, or an invalid value is specified, the input has no size declared, and the form control will be the default width based on the user agent. If CSS targets the element with properties impacting the width, CSS takes precedence.

The `size` attribute has no impact on constraint validation.

### Try it

```
<label for="firstName">First Name:</label>
<input id="firstName" name="firstName" type="text" size="10" />

<label for="lastName">Last Name:</label>
<input id="lastName" name="lastName" type="text" size="20" />

<label for="fruit">Favorite fruit:</label>
<select id="fruit" name="fruit" size="2">
  <option>Orange</option>
  <option>Banana</option>
  <option>Apple</option>
</select>
```

```
label {
  display: block;
  margin-top: 1rem;
}
```

### Examples

By adding `size` on some input types, the width of the input can be controlled. Adding size on a select changes the height, defining how many options are visible in the closed state.

```
<label for="fruit">Enter a fruit</label>
<input type="text" size="15" id="fruit" />
<label for="vegetable">Enter a vegetable</label>
<input type="text" id="vegetable" />

<select name="fruits" size="5">
  <option>banana</option>
  <option>cherry</option>
  <option>strawberry</option>
  <option>durian</option>
  <option>blueberry</option>
</select>
```

```
<select name="vegetables" size="5">
  <option>carrot</option>
  <option>cucumber</option>
  <option>cauliflower</option>
  <option>celery</option>
  <option>collard greens</option>
</select>
```

### Browser compatibility

### See also

- `maxlength`
- `minlength`
- `pattern`

## step HTML attribute

The **`step`** attribute is a number that specifies the granularity that the value must adhere to or the keyword `any`. It is valid for the numeric input types, including the date, month, week, time, datetime-local, number and range types.

The `step` sets the *stepping interval* when clicking up and down spinner buttons, moving a slider left and right on a range, and validating the different date types. If not explicitly included, `step` defaults to 1 for `number` and `range`, and 1 unit type (minute, week, month, day) for the date/time input types. The value must be a positive number - integer or float — or the special value `any`, which means no stepping is implied and any value is allowed (barring other constraints, such as `min` and `max`).

Only values which are a whole number of steps from the step base are valid. The step base is `min` if specified, `value` otherwise, or `0` if neither is provided (except for `week`, which has a default step base of −259,200,000, representing the start of week `1970-W01`).

### Syntax

Default values for step

| Input type | Value | Example |
|---|---|---|
| date | 1 (day) | `<input type="date" min="2019-12-25" step="1">` |
| month | 1 (month) | `<input type="month" min="2019-12" step="12">` |
| week | 1 (week) | `<input type="week" min="2019-W23" step="2">` |
| time | 60 (seconds) | `<input type="time" min="09:00" step="900">` |
| datetime-local | 60 (seconds) | `<input type="datetime-local" min="2019-12-25T19:30" step="900">` |
| number | 1 | `<input type="number" min="0" step="0.1" max="10">` |
| range | 1 | `<input type="range" min="0" step="2" max="10">` |

If `any` is not explicitly set, valid values for the `number`, date/time input types, and `range` input types are equal to the basis for stepping - the `min` value and increments of the step value, up to the `max` value, if specified. The following example results in any even integer, 10 or greater, being valid:

```
<input type="number" min="10" step="2" />
```

If `step` is omitted, any integer is valid but floats like 4.2 are not valid as `step` defaults to 1. For 4.2 to be valid:

- either `step` would have to be set to `any`, 0.1, or 0.2,
- or the `min` value would have to be a number ending in .2, such as 0.2, 1.2, or -5.2.

### Examples

#### <code>min</code> impact on step

The value of `min` defines valid values, even if the `step` attribute is not included. This is because `step` defaults to `1` for the `number` input type.

In this example, we add a big red border around invalid inputs:

```
input:invalid {
  border: solid red 3px;
}
```

We then define an input with a minimum value of 1.2 and a step value of 2:

```
<input id="myNumber" name="myNumber" type="number" step="2" min="1.2" />
```

Valid values include 1.2, 3.2, 5.2, 7.2, 9.2, 11.2, and so on. Only floats with an odd-numbered integer part and a decimal part of .2 are valid. The number spinner, if present, generates valid float values of 1.2 and greater, in increments of 2.

> **Note:** When the data entered by the user doesn't adhere to the stepping configuration, the value is considered invalid in constraint validation and will match the `:invalid` and `:out-of-range` pseudoclasses.

See Client-side validation and `stepMismatch` for more information.

### Accessibility concerns

Provide instructions to help users understand how to complete the form and use individual form controls. Indicate any required and optional input, data formats, and other relevant information. When using the `min` attribute, ensure this minimum requirement is understood by the user. Providing instructions within the `<label>` may be sufficient. If providing instructions outside of labels, which allows more flexible positioning and design, consider using `aria-labelledby` or `aria-describedby`.

### See also

- `max`
- `min`
- Constraint validation
- Form validation
- `validityState.stepMismatch`
- `:out-of-range`
- `<input>`
- date, month, week, time, datetime-local, number and range types, and the `<meter>`

## HTML elements reference

This page lists all the HTML elements, which are created using tags.

They are grouped by function to help you find what you have in mind easily. An alphabetical list of all elements is provided in the sidebar on every element's page as well as this one.

> **Note:** For more information about the basics of HTML elements and attributes, see Anatomy of an HTML element.

### Main root

| Element | Description |
|---|---|
| `<html>` | Represents the root (top-level element) of an HTML document, so it is also referred to as the *root element*. All other elements must be descendants of this element. |

### Document metadata

Metadata contains information about the page. This includes information about styles, scripts and data to help software (search engines, browsers, etc.) use and render the page. Metadata for styles and scripts may be defined in the page or linked to another file that has the information.

| Element | Description |
|---|---|
| `<base>` | Specifies the base URL to use for all relative URLs in a document. There can be only one such element in a document. |
| `<head>` | Contains machine-readable information (metadata) about the document, like its title, scripts, and style sheets. |
| `<link>` | Specifies relationships between the current document and an external resource. This element is most commonly used to link to CSS but is also used to establish site icons (both "favicon" style icons and icons for the home screen and apps on mobile devices) among other things. |
| `<meta>` | Represents metadata that cannot be represented by other HTML meta-related elements, like `<base>`, `<link>`, `<script>`, `<style>` and `<title>`. |
| `<style>` | Contains style information for a document or part of a document. It contains CSS, which is applied to the contents of the document containing this element. |
| `<title>` | Defines the document's title that is shown in a browser's title bar or a page's tab. It only contains text; HTML tags within the element, if any, are also treated as plain text. |

### Sectioning root

| Element | Description |
|---|---|
| `<body>` | Represents the content of an HTML document. There can be only one such element in a document. |

### Content sectioning

Content sectioning elements allow you to organize the document content into logical pieces. Use the sectioning elements to create a broad outline for your page content, including header and footer navigation, and heading elements to identify sections of content.

| Element | Description |
| --- | --- |
| `<address>` | Indicates that the enclosed HTML provides contact information for a person or people, or for an organization. |
| `<article>` | Represents a self-contained composition in a document, page, application, or site, which is intended to be independently distributable or reusable (e.g., in syndication). Examples include a forum post, a magazine or newspaper article, a blog entry, a product card, a user-submitted comment, an interactive widget or gadget, or any other independent item of content. |
| `<aside>` | Represents a portion of a document whose content is only indirectly related to the document's main content. Asides are frequently presented as sidebars or call-out boxes. |
| `<footer>` | Represents a footer for its nearest ancestor sectioning content or sectioning root element. A `<footer>` typically contains information about the author of the section, copyright data, or links to related documents. |
| `<header>` | Represents introductory content, typically a group of introductory or navigational aids. It may contain some heading elements but also a logo, a search form, an author name, and other elements. |
| `<h1>`, `<h2>`, `<h3>`, `<h4>`, `<h5>`, `<h6>` | Represent six levels of section headings. `<h1>` is the highest section level and `<h6>` is the lowest. |
| `<hgroup>` | Represents a heading grouped with any secondary content, such as subheadings, an alternative title, or a tagline. |
| `<main>` | Represents the dominant content of the body of a document. The main content area consists of content that is directly related to or expands upon the central topic of a document, or the central functionality of an application. |
| `<nav>` | Represents a section of a page whose purpose is to provide navigation links, either within the current document or to other documents. Common examples of navigation sections are menus, tables of contents, and indexes. |
| `<section>` | Represents a generic standalone section of a document, which doesn't have a more specific semantic element to represent it. Sections should always have a heading, with very few exceptions. |
| `<search>` | Represents a part that contains a set of form controls or other content related to performing a search or filtering operation. |

### Text content

Use HTML text content elements to organize blocks or sections of content placed between the opening `<body>` and closing `</body>` tags. Important for accessibility and SEO, these elements identify the purpose or structure of that content.

| Element | Description |
| --- | --- |
| `<blockquote>` | Indicates that the enclosed text is an extended quotation. Usually, this is rendered visually by indentation. A URL for the source of the quotation may be given using the `cite` attribute, while a text representation of the source can be given using the `<cite>` element. |
| `<dd>` | Provides the description, definition, or value for the preceding term (`<dt>`) in a description list (`<dl>`). |
| `<div>` | The generic container for flow content. It has no effect on the content or layout until styled in some way using CSS (e.g., styling is directly applied to it, or some kind of layout model like flexbox is applied to its parent element). |
| `<dl>` | Represents a description list. The element encloses a list of groups of terms (specified using the `<dt>` element) and descriptions (provided by `<dd>` elements). Common uses for this element are to implement a glossary or to display metadata (a list of key-value pairs). |
| `<dt>` | Specifies a term in a description or definition list, and as such must be used inside a `<dl>` element. It is usually followed by a `<dd>` element; however, multiple `<dt>` elements in a row indicate several terms that are all defined by the immediate next `<dd>` element. |
| `<figcaption>` | Represents a caption or legend describing the rest of the contents of its parent `<figure>` element. |
| `<figure>` | Represents self-contained content, potentially with an optional caption, which is specified using the `<figcaption>` element. The figure, its caption, and its contents are referenced as a single unit. |
| `<hr>` | Represents a thematic break between paragraph-level elements: for example, a change of scene in a story, or a shift of topic within a section. |
| `<li>` | Represents an item in a list. It must be contained in a parent element: an ordered list (`<ol>`), an unordered list (`<ul>`), or a menu (`<menu>`). In menus and unordered lists, list items are usually displayed using bullet points. In ordered lists, they are usually displayed with an ascending counter on the left, such as a number or letter. |
| `<menu>` | A semantic alternative to `<ul>`, but treated by browsers (and exposed through the accessibility tree) as no different than `<ul>`. It represents an unordered list of items (which are represented by `<li>` elements). |
| `<ol>` | Represents an ordered list of items — typically rendered as a numbered list. |
| `<p>` | Represents a paragraph. Paragraphs are usually represented in visual media as blocks of text separated from adjacent blocks by blank lines and/or first-line indentation, but HTML paragraphs can be any structural grouping of related content, such as images or form fields. |

| Element | Description |
| --- | --- |
| `<pre>` | Represents preformatted text which is to be presented exactly as written in the HTML file. The text is typically rendered using a non-proportional, or monospaced, font. Whitespace inside this element is displayed as written. |
| `<ul>` | Represents an unordered list of items, typically rendered as a bulleted list. |

### Inline text semantics

Use the HTML inline text semantic to define the meaning, structure, or style of a word, line, or any arbitrary piece of text.

| Element | Description |
| --- | --- |
| `<a>` | Together with its `href` attribute, creates a hyperlink to web pages, files, email addresses, locations within the current page, or anything else a URL can address. |
| `<abbr>` | Represents an abbreviation or acronym. |
| `<b>` | Used to draw the reader's attention to the element's contents, which are not otherwise granted special importance. This was formerly known as the Boldface element, and most browsers still draw the text in boldface. However, you should not use `<b>` for styling text or granting importance. If you wish to create boldface text, you should use the CSS `font-weight` property. If you wish to indicate an element is of special importance, you should use the `<strong>` element. |
| `<bdi>` | Tells the browser's bidirectional algorithm to treat the text it contains in isolation from its surrounding text. It's particularly useful when a website dynamically inserts some text and doesn't know the directionality of the text being inserted. |
| `<bdo>` | Overrides the current directionality of text, so that the text within is rendered in a different direction. |
| `<br>` | Produces a line break in text (carriage-return). It is useful for writing a poem or an address, where the division of lines is significant. |
| `<cite>` | Used to mark up the title of a creative work. The reference may be in an abbreviated form according to context-appropriate conventions related to citation metadata. |
| `<code>` | Displays its contents styled in a fashion intended to indicate that the text is a short fragment of computer code. By default, the content text is displayed using the user agent's default monospace font. |
| `<data>` | Links a given piece of content with a machine-readable translation. If the content is time- or date-related, the `<time>` element must be used. |
| `<dfn>` | Used to indicate the term being defined within the context of a definition phrase or sentence. The ancestor `<p>` element, the `<dt>`/`<dd>` pairing, or the nearest section ancestor of the `<dfn>` element, is considered to be the definition of the term. |
| `<em>` | Marks text that has stress emphasis. The `<em>` element can be nested, with each nesting level indicating a greater degree of emphasis. |
| `<i>` | Represents a range of text that is set off from the normal text for some reason, such as idiomatic text, technical terms, and taxonomical designations, among others. Historically, these have been presented using italicized type, which is the original source of the `<i>` naming of this element. |
| `<kbd>` | Represents a span of inline text denoting textual user input from a keyboard, voice input, or any other text entry device. By convention, the user agent defaults to rendering the contents of a `<kbd>` element using its default monospace font, although this is not mandated by the HTML standard. |
| `<mark>` | Represents text which is marked or highlighted for reference or notation purposes due to the marked passage's relevance in the enclosing context. |
| `<q>` | Indicates that the enclosed text is a short inline quotation. Most modern browsers implement this by surrounding the text in quotation marks. This element is intended for short quotations that don't require paragraph breaks; for long quotations use the `<blockquote>` element. |
| `<rp>` | Used to provide fall-back parentheses for browsers that do not support the display of ruby annotations using the `<ruby>` element. One `<rp>` element should enclose each of the opening and closing parentheses that wrap the `<rt>` element that contains the annotation's text. |
| `<rt>` | Specifies the ruby text component of a ruby annotation, which is used to provide pronunciation, translation, or transliteration information for East Asian typography. The `<rt>` element must always be contained within a `<ruby>` element. |
| `<ruby>` | Represents small annotations that are rendered above, below, or next to base text, usually used for showing the pronunciation of East Asian characters. It can also be used for annotating other kinds of text, but this usage is less common. |
| `<s>` | Renders text with a strikethrough, or a line through it. Use the `<s>` element to represent things that are no longer relevant or no longer accurate. However, `<s>` is not appropriate when indicating document edits; for that, use the `<del>` and `<ins>` elements, as appropriate. |
| `<samp>` | Used to enclose inline text which represents sample (or quoted) output from a computer program. Its contents are typically rendered using the browser's default monospaced font (such as Courier or Lucida Console). |
| `<small>` | Represents side-comments and small print, like copyright and legal text, independent of its styled presentation. By default, it renders text within it one font size smaller, such as from `small` to `x-small`. |

| Element | Description |
| --- | --- |
| `<span>` | A generic inline container for phrasing content, which does not inherently represent anything. It can be used to group elements for styling purposes (using the `class` or `id` attributes), or because they share attribute values, such as `lang`. It should be used only when no other semantic element is appropriate. `<span>` is very much like a div element, but div is a block-level element whereas a `<span>` is an inline-level element. |
| `<strong>` | Indicates that its contents have strong importance, seriousness, or urgency. Browsers typically render the contents in bold type. |
| `<sub>` | Specifies inline text which should be displayed as subscript for solely typographical reasons. Subscripts are typically rendered with a lowered baseline using smaller text. |
| `<sup>` | Specifies inline text which is to be displayed as superscript for solely typographical reasons. Superscripts are usually rendered with a raised baseline using smaller text. |
| `<time>` | Represents a specific period in time. It may include the `datetime` attribute to translate dates into machine-readable format, allowing for better search engine results or custom features such as reminders. |
| `<u>` | Represents a span of inline text which should be rendered in a way that indicates that it has a non-textual annotation. This is rendered by default as a single solid underline but may be altered using CSS. |
| `<var>` | Represents the name of a variable in a mathematical expression or a programming context. It's typically presented using an italicized version of the current typeface, although that behavior is browser-dependent. |
| `<wbr>` | Represents a word break opportunity—a position within text where the browser may optionally break a line, though its line-breaking rules would not otherwise create a break at that location. |

### Image and multimedia

HTML supports various multimedia resources such as images, audio, and video.

| Element | Description |
| --- | --- |
| `<area>` | Defines an area inside an image map that has predefined clickable areas. An *image map* allows geometric areas on an image to be associated with hyperlink. |
| `<audio>` | Used to embed sound content in documents. It may contain one or more audio sources, represented using the `src` attribute or the source element: the browser will choose the most suitable one. It can also be the destination for streamed media, using a `MediaStream`. |
| `<img>` | Embeds an image into the document. |
| `<map>` | Used with `<area>` elements to define an image map (a clickable link area). |
| `<track>` | Used as a child of the media elements, audio and video. It lets you specify timed text tracks (or time-based data), for example to automatically handle subtitles. The tracks are formatted in WebVTT format (`.vtt` files)—Web Video Text Tracks. |
| `<video>` | Embeds a media player which supports video playback into the document. You can also use `<video>` for audio content, but the audio element may provide a more appropriate user experience. |

### Embedded content

In addition to regular multimedia content, HTML can include a variety of other content, even if it's not always easy to interact with.

| Element | Description |
| --- | --- |
| `<embed>` | Embeds external content at the specified point in the document. This content is handled by the browser or an external application. |
| `<iframe>` | Represents a nested browsing context, embedding another document into the current one. |
| `<object>` | Represents an external resource, which can be treated as an image, a nested browsing context, or a resource to be handled by a plugin. |
| `<picture>` | Contains zero or more `<source>` elements and one `<img>` element to offer alternative versions of an image for different display/device scenarios. |
| `<source>` | Specifies multiple media resources for the picture, the audio element, or the video element. It is a void element, meaning that it has no content and does not have a closing tag. It is commonly used to offer the same media content in multiple file formats in order to provide compatibility with a broad range of browsers given their differing support for image file formats and media file formats. |

### SVG and MathML

You can embed SVG and MathML content directly into HTML documents, using the `<svg>` and `<math>` elements.

| Element | Description |
| --- | --- |
| `<svg>` | Container defining a new coordinate system and viewport. It is used as the outermost element of SVG documents, but it can also be used to embed an SVG fragment inside an SVG or HTML document. |
| `<math>` | The top-level element in MathML. Every valid MathML instance must be wrapped in it. In addition, you must not nest a second `<math>` element in another, but you can have an arbitrary number of other child elements in it. |

### Scripting

To create dynamic content and Web applications, HTML supports the use of scripting languages, most prominently JavaScript. Certain elements support this capability.

| Element | Description |
| --- | --- |
| `<canvas>` | Container element to use with either the canvas scripting API or the WebGL API to draw graphics and animations. |
| `<noscript>` | Defines a section of HTML to be inserted if a script type on the page is unsupported or if scripting is currently turned off in the browser. |
| `<script>` | Used to embed executable code or data; this is typically used to embed or refer to JavaScript code. The `<script>` element can also be used with other languages, such as WebGL's GLSL shader programming language and JSON. |

### Demarcating edits

These elements let you provide indications that specific parts of the text have been altered.

| Element | Description |
| --- | --- |
| `<del>` | Represents a range of text that has been deleted from a document. This can be used when rendering "track changes" or source code diff information, for example. The `<ins>` element can be used for the opposite purpose: to indicate text that has been added to the document. |
| `<ins>` | Represents a range of text that has been added to a document. You can use the `<del>` element to similarly represent a range of text that has been deleted from the document. |

### Table content

The elements here are used to create and handle tabular data.

| Element | Description |
| --- | --- |
| `<caption>` | Specifies the caption (or title) of a table. |
| `<col>` | Defines one or more columns in a column group represented by its implicit or explicit parent `<colgroup>` element. The `<col>` element is only valid as a child of a `<colgroup>` element that has no `span` attribute defined. |
| `<colgroup>` | Defines a group of columns within a table. |
| `<table>` | Represents tabular data—that is, information presented in a two-dimensional table comprised of rows and columns of cells containing data. |
| `<tbody>` | Encapsulates a set of table rows (`<tr>` elements), indicating that they comprise the body of a table's (main) data. |
| `<td>` | A child of the `<tr>` element, it defines a cell of a table that contains data. |
| `<tfoot>` | Encapsulates a set of table rows (`<tr>` elements), indicating that they comprise the foot of a table with information about the table's columns. This is usually a summary of the columns, e.g., a sum of the given numbers in a column. |
| `<th>` | A child of the `<tr>` element, it defines a cell as the header of a group of table cells. The nature of this group can be explicitly defined by the `scope` and `headers` attributes. |
| `<thead>` | Encapsulates a set of table rows (`<tr>` elements), indicating that they comprise the head of a table with information about the table's columns. This is usually in the form of column headers (`<th>` elements). |
| `<tr>` | Defines a row of cells in a table. The row's cells can then be established using a mix of `<td>` (data cell) and `<th>` (header cell) elements. |

### Forms

HTML provides several elements that can be used together to create forms that the user can fill out and submit to the website or application. Further information about this available in the HTML forms guide.

| Element | Description |
| --- | --- |
| `<button>` | An interactive element activated by a user with a mouse, keyboard, finger, voice command, or other assistive technology. Once activated, it performs an action, such as submitting a form or opening a dialog. |
| `<datalist>` | Contains a set of `<option>` elements that represent the permissible or recommended options available to choose from within other controls. |

| Element | Description |
| --- | --- |
| `<fieldset>` | Used to group several controls as well as labels (`<label>`) within a web form. |
| `<form>` | Represents a document section containing interactive controls for submitting information. |
| `<input>` | Used to create interactive controls for web-based forms to accept data from the user; a wide variety of types of input data and control widgets are available, depending on the device and user agent. The `<input>` element is one of the most powerful and complex in all of HTML due to the sheer number of combinations of input types and attributes. |
| `<label>` | Represents a caption for an item in a user interface. |
| `<legend>` | Represents a caption for the content of its parent `<fieldset>`. |
| `<meter>` | Represents either a scalar value within a known range or a fractional value. |
| `<optgroup>` | Creates a grouping of options within a `<select>` element. |
| `<option>` | Used to define an item contained in a `<select>`, an `<optgroup>`, or a `<datalist>` element. As such, `<option>` can represent menu items in popups and other lists of items in an HTML document. |
| `<output>` | Container element into which a site or app can inject the results of a calculation or the outcome of a user action. |
| `<progress>` | Displays an indicator showing the completion progress of a task, typically displayed as a progress bar. |
| `<select>` | Represents a control that provides a menu of options. |
| `<selectedcontent>` | Displays the content of the currently selected `<option>` inside a closed `<select>` element. |
| `<textarea>` | Represents a multi-line plain-text editing control, useful when you want to allow users to enter a sizeable amount of free-form text, for example, a comment on a review or feedback form. |

### Interactive elements

HTML offers a selection of elements that help to create interactive user interface objects.

| Element | Description |
| --- | --- |
| `<details>` | Creates a disclosure widget in which information is visible only when the widget is toggled into an "open" state. A summary or label must be provided using the `<summary>` element. |
| `<dialog>` | Represents a dialog box or other interactive component, such as a dismissible alert, inspector, or subwindow. |
| `<geolocation>` | Creates an interactive control for the user to share their geolocation data with the page. |
| `<summary>` | Specifies a summary, caption, or legend for a details element's disclosure box. Clicking the `<summary>` element toggles the state of the parent `<details>` element open and closed. |

### Web Components

Web Components is an HTML-related technology that makes it possible to, essentially, create and use custom elements as if it were regular HTML. In addition, you can create custom versions of standard HTML elements.

| Element | Description |
| --- | --- |
| `<slot>` | Part of the Web Components technology suite, this element is a placeholder inside a web component that you can fill with your own markup, which lets you create separate DOM trees and present them together. |
| `<template>` | A mechanism for holding HTML that is not to be rendered immediately when a page is loaded but may be instantiated subsequently during runtime using JavaScript. |

### Obsolete and deprecated elements

> **Warning:** These are old HTML elements that are deprecated and should not be used. **You should never use them in new projects, and you should replace them in old projects as soon as you can.** They are listed here for completeness only.

| Element | Description |
| --- | --- |
| `<acronym>` | Allows authors to clearly indicate a sequence of characters that compose an acronym or abbreviation for a word. |
| `<big>` | Renders the enclosed text at a font size one level larger than the surrounding text (`medium` becomes `large`, for example). The size is capped at the browser's maximum permitted font size. |
| `<center>` | Displays its block-level or inline contents centered horizontally within its containing element. |
| `<content>` | An obsolete part of the Web Components suite of technologies—was used inside of Shadow DOM as an insertion point, and wasn't meant to be used in ordinary HTML. It has now been replaced by the `<slot>` element, which creates a point in the DOM at which a shadow DOM can be inserted. Consider using `<slot>` instead. |
| `<dir>` | Container for a directory of files and/or folders, potentially with styles and icons applied by the user agent. Do not use this obsolete element; instead, you should use the `<ul>` element for lists, including lists of files. |

| Element | Description |
| --- | --- |
| `<fencedframe>` | Represents a nested browsing context, like `<iframe>` but with more native privacy features built in. |
| `<font>` | Defines the font size, color and face for its content. |
| `<frame>` | Defines a particular area in which another HTML document can be displayed. A frame should be used within a `<frameset>`. |
| `<frameset>` | Used to contain `<frame>` elements. |
| `<image>` | An ancient and poorly supported precursor to the `<img>` element. It should not be used. |
| `<marquee>` | Used to insert a scrolling area of text. You can control what happens when the text reaches the edges of its content area using its attributes. |
| `<menuitem>` | Represents a command that a user is able to invoke through a popup menu. This includes context menus, as well as menus that might be attached to a menu button. |
| `<nobr>` | Prevents the text it contains from automatically wrapping across multiple lines, potentially resulting in the user having to scroll horizontally to see the entire width of the text. |
| `<noembed>` | An obsolete, non-standard way to provide alternative, or "fallback", content for browsers that do not support the embed element or do not support the type of embedded content an author wishes to use. This element was deprecated in HTML 4.01 and above in favor of placing fallback content between the opening and closing tags of an `<object>` element. |
| `<noframes>` | Provides content to be presented in browsers that don't support (or have disabled support for) the `<frame>` element. Although most commonly-used browsers support frames, there are exceptions, including certain special-use browsers including some mobile browsers, as well as text-mode browsers. |
| `<param>` | Defines parameters for an `<object>` element. |
| `<plaintext>` | Renders everything following the start tag as raw text, ignoring any following HTML. There is no closing tag, since everything after it is considered raw text. |
| `<rb>` | Used to delimit the base text component of a ruby annotation, i.e., the text that is being annotated. One `<rb>` element should wrap each separate atomic segment of the base text. |
| `<rtc>` | Embraces semantic annotations of characters presented in a ruby of `<rb>` elements used inside of `<ruby>` element. `<rb>` elements can have both pronunciation (`<rt>`) and semantic (`<rtc>`) annotations. |
| `<shadow>` | An obsolete part of the Web Components technology suite that was intended to be used as a shadow DOM insertion point. You might have used it if you have created multiple shadow roots under a shadow host. Consider using `<slot>` instead. |
| `<strike>` | Places a strikethrough (horizontal line) over text. |
| `<tt>` | Creates inline text which is presented using the user agent default monospace font face. This element was created for the purpose of rendering text as it would be displayed on a fixed-width display such as a teletype, text-only screen, or line printer. |
| `<xmp>` | Renders text between the start and end tags without interpreting the HTML in between and using a monospaced font. The HTML2 specification recommended that it should be rendered wide enough to allow 80 characters per line. |

### See also

- `Element` interface

## <a> HTML anchor element

The **`<a>`** HTML element (or *anchor* element), with its `href` attribute, creates a hyperlink to web pages, files, email addresses, locations in the same page, or anything else a URL can address.

Content within each `<a>` *should* indicate the link's destination. If the `href` attribute is present, pressing the enter key while focused on the `<a>` element will activate it.

### Try it

```
<p>You can reach Michael at:</p>

<ul>
  <li><a href="https://example.com">Website</a></li>
  <li><a href="mailto:m.bluth@example.com">Email</a></li>
  <li><a href="tel:+123456789">Phone</a></li>
</ul>
```

```
li {
  margin-bottom: 0.5rem;
}
```

### Attributes

This element's attributes include the global attributes.

`attributionsrc`

Specifies that you want the browser to send an `Attribution-Reporting-Eligible` header. On the server-side this is used to trigger sending an `Attribution-Reporting-Register-Source` header in the response, to register a navigation-based attribution source.

The browser stores the source data associated with the navigation-based attribution source (as provided in the `Attribution-Reporting-Register-Source` response header) when the user clicks the link. See the Attribution Reporting API for more details.

There are two versions of this attribute that you can set:

- Boolean, i.e., just the `attributionsrc` name. This specifies that you want the `Attribution-Reporting-Eligible` header sent to the same server as the `href` attribute points to. This is fine when you are handling the attribution source registration on the same server.
- Value containing one or more URLs, for example:

  ```
  attributionsrc="https://a.example/register-source
  https://b.example/register-source"
  ```

  This is useful in cases where the requested resource is not on a server you control, or you just want to handle registering the attribution source on a different server. In this case, you can specify one or more URLs as the value of `attributionsrc`. When the resource request occurs, the `Attribution-Reporting-Eligible` header will be sent to the URL(s) specified in `attributionsrc` in addition to the resource origin. These URLs can then respond with the `Attribution-Reporting-Register-Source` to complete registration.

  > **Note:** Specifying multiple URLs means that multiple attribution sources can be registered on the same feature. You might for example have different campaigns that you are trying to measure the success of, which involve generating different reports on different data.

`<a>` elements cannot be used as attribution triggers, only sources.

`download`

Causes the browser to treat the linked URL as a download. Can be used with or without a `filename` value:

- Without a value, the browser will suggest a filename/extension, generated from various sources:
  - The `Content-Disposition` HTTP header
  - The final segment in the URL path
  - The media type (from the `Content-Type` header, the start of a `data:` URL, or `Blob.type` for a `blob:` URL)
- `filename`: defining a value suggests it as the filename. `/` and `\` characters are converted to underscores (_). Filesystems may forbid other characters in filenames, so browsers will adjust the suggested name if necessary. This adjustment happens when the download is triggered and is not observable via the `HTMLAnchorElement.download` property.

> **Note:**
>
> - `download` only works for same-origin URLs, same-origin `blob:`, and `data:` schemes. For other URLs, the `download` attribute is ignored and a normal navigation happens, although the target page may still use `Content-Disposition` to instruct a download.
> - How browsers treat downloads varies by browser, user settings, and other factors. The user may be prompted before a download starts, or the file may be saved automatically, or it may open automatically, either in an external application or in the browser itself.
> - The `download` attribute, when honored, overrides the `Content-Disposition: inline` behavior, but the `filename` parameter of `Content-Disposition` overrides the name in the `download` attribute.

`href`

The URL that the hyperlink points to. Links are not restricted to HTTP-based URLs — they can use any URL scheme supported by browsers:

- Telephone numbers with `tel:` URLs
- Email addresses with `mailto:` URLs
- SMS text messages with `sms:` URLs
- Executable code with `javascript:` URLs
- While web browsers may not support other URL schemes, websites can with `registerProtocolHandler()`

Moreover other URL features can locate specific parts of the resource, including:

- Sections of a page with document fragments
- Specific text portions with text fragments

- Pieces of media files with media fragments

`hreflang`

Hints at the human language of the linked URL. No built-in functionality. Allowed values are the same as the global `lang` attribute.

`interestfor`

Defines the `<a>` element as an **interest invoker**. Its value is the `id` of the target element, which will be affected in some way (normally shown or hidden) when interest is shown or lost on the invoker element (for example, by hovering/unhovering or focusing/blurring it). See Using interest invokers for more details and examples.

`ping`

A space-separated list of URLs. When the link is followed, the browser will send `POST` requests with the body `PING` to the URLs. Typically for tracking.

`referrerpolicy`

How much of the referrer to send when following the link.

- `no-referrer`: The `Referer` header will not be sent.
- `no-referrer-when-downgrade`: The `Referer` header will not be sent to origins without TLS (HTTPS).
- `origin`: The sent referrer will be limited to the origin of the referring page: its scheme, host, and port.
- `origin-when-cross-origin`: The referrer sent to other origins will be limited to the scheme, the host, and the port. Navigations on the same origin will still include the path.
- `same-origin`: A referrer will be sent for same origin, but cross-origin requests will contain no referrer information.
- `strict-origin`: Only send the origin of the document as the referrer when the protocol security level stays the same (HTTPS→HTTPS), but don't send it to a less secure destination (HTTPS→HTTP).
- `strict-origin-when-cross-origin` (default): Send a full URL when performing a same-origin request, only send the origin when the protocol security level stays the same (HTTPS→HTTPS), and send no header to a less secure destination (HTTPS→HTTP).
- `unsafe-url`: The referrer will include the origin *and* the path (but not the fragment, password, or username). **This value is unsafe**, because it leaks origins and paths from TLS-protected resources to insecure origins.

`rel`

The relationship of the linked URL as space-separated link types.

`target`

Where to display the linked URL, as the name for a *browsing context* (a tab, window, or `<iframe>`). The following keywords have special meanings for where to load the URL:

- `_self`: The current browsing context. (Default)
- `_blank`: Usually a new tab, but users can configure browsers to open a new window instead.
- `_parent`: The parent browsing context of the current one. If no parent, behaves as `_self`.
- `_top`: The topmost browsing context. To be specific, this means the "highest" context that's an ancestor of the current one. If no ancestors, behaves as `_self`.
- `_unfencedTop`: Allows embedded fenced frames to navigate the top-level frame (i.e., traversing beyond the root of the fenced frame, unlike other reserved destinations). Note that the navigation will still succeed if this is used outside of a fenced frame context, but it will not act like a reserved keyword.

> **Note:** Setting `target="_blank"` on `<a>` elements implicitly provides the same `rel` behavior as setting `rel="noopener"` which does not set `window.opener`.

`type`

Hints at the linked URL's format with a MIME type. No built-in functionality.

### Deprecated attributes

`charset`

Hinted at the character encoding of the linked URL.

> **Note:** This attribute is deprecated and **should not be used by authors**. Use the HTTP `Content-Type` header on the linked URL.

`coords`

Used with the `shape` attribute. A comma-separated list of coordinates.

`name`

Was required to define a possible target location in a page. In HTML 4.01, `id` and `name` could both be used on `<a>`, as long as they had identical values.

> **Note:** Use the global attribute `id` instead.

`rev`

Specified a reverse link; the opposite of the `rel` attribute. Deprecated for being very confusing.

`shape`

The shape of the hyperlink's region in an image map.

> **Note:** Use the `<area>` element for image maps instead.

### Accessibility

#### Strong link text

**The content inside a link should indicate where the link goes**, even out of context.

#### Inaccessible, weak link text

A sadly common mistake is to only link the words "click here" or "here":

```
<p>Learn more about our products <a href="https://developer.mozilla.org/products">here</a>.</p>
```

#### Result

#### Accessible, strong link text

Luckily, this is an easy fix, and it's actually shorter than the inaccessible version!

```
<p>Learn more <a href="https://developer.mozilla.org/products">about our products</a>.</p>
```

#### Result

Assistive software has shortcuts to list all links on a page. However, strong link text benefits all users — the "list all links" shortcut emulates how sighted users quickly scan pages.

#### onclick events

Anchor elements are often abused as fake buttons by setting their `href` to `#` or `javascript:void(0)` to prevent the page from refreshing, then listening for their `click` events.

These bogus `href` values cause unexpected behavior when copying/dragging links, opening links in a new tab/window, bookmarking, or when JavaScript is loading, errors, or is disabled. They also convey incorrect semantics to assistive technologies, like screen readers.

Use a `<button>` instead. In general, **you should only use a hyperlink for navigation to a real URL**.

#### External links and linking to non-HTML resources

Links that open in a new tab/window via `target="_blank"`, or links that point to a download file should indicate what will happen when the link is followed.

People experiencing low vision conditions, navigating with the aid of screen reading technology, or with cognitive concerns may be confused when a new tab, window, or application opens unexpectedly. Older screen-reading software may not even announce the behavior.

#### Link that opens a new tab/window

```
<a target="_blank" href="https://www.wikipedia.org">
  Wikipedia (opens in new tab)
</a>
```

#### Result

#### Link to a non-HTML resource

If an icon is used to signify link behavior, make sure it has an `alt` attribute to describe its purpose. In case the icon is missing, the `alt` attribute's content will still convey the link's behavior.

```
<p>
  <a href="https://www.wikipedia.org/" target="_blank">
    Wikipedia
    <img src="https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/a/new-tab.svg" width="14" alt="(Opens in new tab)" />
  </a>
  <br />
  <a href="2017-annual-report.ppt">
    2017 annual report
    <img src="https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/a/powerpoint.svg" width="14" alt="(PowerPoint file)"
/>
  </a>
</p>
<p>
  <a href="https://www.wikipedia.org/" target="_blank">
    Wikipedia
    <img src="https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/a/missing-icon.svg" width="14" alt="(Opens in new
tab)" />
  </a>
  <br />
  <a href="2017-annual-report.ppt">
    2017 annual report
    <img src="https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/a/missing-icon.svg" width="14" alt="(PowerPoint
file)" />
  </a>
</p>
```

### Result

- WebAIM: Links and Hypertext - Hypertext Links
- MDN / Understanding WCAG, Guideline 3.2
- G200: Opening new windows and tabs from a link only when necessary
- G201: Giving users advanced warning when opening a new window

### Skip links

A **skip link** is a link placed as early as possible in `<body>` content that points to the beginning of the page's main content. Usually, CSS hides a skip link offscreen until focused.

```
<body>
  <a href="#content" class="skip-link">Skip to main content</a>

  <header>…</header>

  <!-- The skip link jumps to here -->
  <main id="content"></main>
</body>
```

```
.skip-link {
  position: absolute;
  top: -3em;
  background: white;
}
.skip-link:focus {
  top: 0;
}
```

### Result

Skip links let keyboard users bypass content repeated throughout multiple pages, such as header navigation.

Skip links are especially useful for people who navigate with the aid of assistive technology such as switch control, voice command, or mouth sticks/head wands, where the act of moving through repetitive links can be laborious.

- WebAIM: "Skip Navigation" Links
- How-to: Use Skip Navigation links
- MDN / Understanding WCAG, Guideline 2.4 explanations
- Understanding Success Criterion 2.4.1

### Size and proximity

### Size

Interactive elements, like links, should provide an area large enough that it is easy to activate them. This helps a variety of people, including those with motor control issues and those using imprecise inputs such as a touchscreen. A minimum size of 44×44 CSS pixels is recommended.

Text-only links in prose content are exempt from this requirement, but it's still a good idea to make sure enough text is hyperlinked to be easily activated.

- Understanding Success Criterion 2.5.5: Target Size
- Target Size and 2.5.5
- Quick test: Large touch targets

#### Proximity

Interactive elements, like links, placed in close visual proximity should have space separating them. Spacing helps people with motor control issues, who may otherwise accidentally activate the wrong interactive content.

Spacing may be created using CSS properties like `margin`.

- Hand tremors and the giant-button-problem

### Examples

#### Linking to an absolute URL

#### HTML

```
<a href="https://www.mozilla.com">Mozilla</a>
```

#### Result

#### Linking to relative URLs

#### HTML

```
<a href="https://developer.mozilla.org//example.com">Scheme-relative URL</a>
<a href="#mdn-Web-HTML">Origin-relative URL</a>
<a href="p">Directory-relative URL</a>
<a href="./p">Directory-relative URL</a>
<a href="../p">Parent-directory-relative URL</a>
```

#### Result

#### Linking to an element on the same page

```
<!-- <a> element links to the section below -->
<p><a href="#Section_further_down">Jump to the heading below</a></p>

<!-- Heading to link to -->
<h2 id="Section_further_down">Section further down</h2>
```

#### Result

> **Note:** You can use `href="#top"` or the empty fragment (`href="#"`) to link to the top of the current page, as defined in the HTML specification.

#### Linking to an email address

To create links that open in the user's email program to let them send a new message, use the `mailto:` scheme:

```
<a href="mailto:nowhere@mozilla.org">Send email to nowhere</a>
```

#### Result

For details about `mailto:` URLs, such as including a subject or body, see Email links or RFC 6068.

#### Linking to telephone numbers

```
<a href="tel:+49.157.0156">+49 157 0156</a>
<a href="tel:+1(800)555-0123">(800) 555-0123</a>
```

#### Result

`tel:` link behavior varies with device capabilities:

- Cellular devices autodial the number.
- Most operating systems have programs that can make calls, like Skype or FaceTime.

- Websites can make phone calls with `registerProtocolHandler`, such as `web.skype.com`.
- Other behaviors include saving the number to contacts, or sending the number to another device.

See RFC 3966 for syntax, additional features, and other details about the `tel:` URL scheme.

### Using the download attribute to save a <canvas> as a PNG

To save a `<canvas>` element's contents as an image, you can create a link where the `href` is the canvas data as a `data:` URL created with JavaScript and the `download` attribute provides the file name for the downloaded PNG file:

#### Example painting app with save link

#### HTML

```
<p>
  Paint by holding down the mouse button and moving it.
  <a href="" download="my_painting.png">Download my painting</a>
</p>

<canvas width="300" height="300"></canvas>
```

#### CSS

```
html {
  font-family: sans-serif;
}
canvas {
  background: white;
  border: 1px dashed;
}
a {
  display: inline-block;
  background: #6699cc;
  color: white;
  padding: 5px 10px;
}
```

#### JavaScript

```
const canvas = document.querySelector("canvas");
const c = canvas.getContext("2d");
c.fillStyle = "hotpink";
let isDrawing;

function draw(x, y) {
  if (isDrawing) {
    c.beginPath();
    c.arc(x, y, 10, 0, Math.PI * 2);
    c.closePath();
    c.fill();
  }
}

canvas.addEventListener("mousemove", (event) =>
  draw(event.offsetX, event.offsetY),
);
canvas.addEventListener("mousedown", () => (isDrawing = true));
canvas.addEventListener("mouseup", () => (isDrawing = false));

document
  .querySelector("a")
  .addEventListener(
    "click",
    (event) => (event.target.href = canvas.toDataURL()),
  );
```

#### Result

### Security and privacy

`<a>` elements can have consequences for the security and privacy of users. See `Referer` header: privacy and security concerns for information.

Using `target="_blank"` without `rel="noreferrer"` and `rel="noopener"` makes the website vulnerable to `window.opener` API exploitation attacks, although note that, in newer browser versions setting `target="_blank"` implicitly provides the same protection as setting `rel="noopener"`. See browser compatibility for details.

### Technical summary

| | |
|---|---|
| **Content categories** | Flow content, phrasing content, interactive content, palpable content. |
| **Permitted content** | Transparent, except that no descendant may be interactive content or an `<a>` element, and no descendant may have a specified tabindex attribute. |
| **Tag omission** | None, both the starting and ending tag are mandatory. |
| **Permitted parents** | Any element that accepts flow content, but not other `<a>` elements. |
| **Implicit ARIA role** | `link` when `href` attribute is present, otherwise `generic` |
| **Permitted ARIA roles** | When `href` attribute is present:<br>• `button`<br>• `checkbox`<br>• `menuitem`<br>• `menuitemcheckbox`<br>• `menuitemradio`<br>• `option`<br>• `radio`<br>• `switch`<br>• `tab`<br>• `treeitem`<br>When `href` attribute is not present:<br>• any |
| **DOM interface** | `HTMLAnchorElement` |

### See also

- `<link>` is similar to `<a>`, but for metadata hyperlinks that are invisible to users.
- `:link` is a CSS pseudo-class that will match `<a>` elements with URL in `href` attribute that was not yet visited by the user.
- `:visited` is a CSS pseudo-class that will match `<a>` elements with URL in `href` attribute that was visited by the user in the past.
- `:any-link` is a CSS pseudo-class that will match `<a>` elements with `href` attribute.
- Text fragments are user-agent instructions added to URLs that allow content authors to link to specific text on a page, without IDs being required.

## <abbr> HTML abbreviation element

The **`<abbr>`** HTML element represents an abbreviation or acronym.

When including an abbreviation or acronym, provide a full expansion of the term in plain text on first use, along with the `<abbr>` to mark up the abbreviation. This informs the user what the abbreviation or acronym means.

The optional `title` attribute can provide an expansion for the abbreviation or acronym when a full expansion is not present. This provides a hint to user agents on how to announce/display the content while informing all users what the abbreviation means. If present, `title` must contain this full description and nothing else.

### Try it

```
<p>
  You can use <abbr>CSS</abbr> (Cascading Style Sheets) to style your
  <abbr>HTML</abbr> (HyperText Markup Language). Using style sheets, you can
  keep your <abbr>CSS</abbr> presentation layer and <abbr>HTML</abbr> content
  layer separate. This is called "separation of concerns."
</p>
```

```
abbr {
  font-style: italic;
  color: chocolate;
}
```

### Attributes

This element only supports the global attributes. The `title` attribute has a specific semantic meaning when used with the `<abbr>` element; it *must* contain a full human-readable description or expansion of the abbreviation. This text is often presented by browsers as a tooltip when the mouse cursor is hovered over the element.

Each `<abbr>` element you use is independent of all others; providing a `title` for one does not automatically attach the same expansion text to others with the same content text.

### Usage notes

#### Typical use cases

It's certainly not required that all abbreviations be marked up using `<abbr>`. There are, though, a few cases where it's helpful to do so:

- When an abbreviation is used and you want to provide an expansion or definition outside the flow of the document's content, use `<abbr>` with an appropriate `title`.
- To define an abbreviation which may be unfamiliar to the reader, present the term using `<abbr>` and inline text providing the definition. Include a `title` attribute only when the inline expansion or definition is not available.
- When an abbreviation's presence in the text needs to be semantically noted, the `<abbr>` element is useful. This can be used, in turn, for styling or scripting purposes.
- You can use `<abbr>` in concert with `<dfn>` to establish definitions for terms which are abbreviations or acronyms. See the example Defining an abbreviation below.

#### Grammar considerations

In languages with grammatical number (that is, languages where the number of items affects the grammar of a sentence), use the same grammatical number in your `title` attribute as inside your `<abbr>` element. This is especially important in languages with more than two numbers, such as Arabic, but is also relevant in English.

#### Default styling

The purpose of this element is purely for the convenience of the author and all browsers display it inline (`display: inline`) by default, though its default styling varies from one browser to another:

Some browsers add a dotted underline to the content of the element. Others add a dotted underline while converting the contents to small caps. Others may not style it differently than a `<span>` element. To control this styling, use `text-decoration` and `font-variant`.

#### Accessibility

Spelling out the acronym or abbreviation in full the first time it is used on a page is beneficial for helping people understand it, especially if the content is technical or industry jargon.

Only include a `title` if expanding the abbreviation or acronym in the text is not possible. Having a difference between the announced word or phrase and what is displayed on the screen, especially if it's technical jargon the reader may not be familiar with, can be jarring.

```
<p>
  JavaScript Object Notation (<abbr>JSON</abbr>) is a lightweight
  data-interchange format.
</p>
```

This is especially helpful for people who are unfamiliar with the terminology or concepts discussed in the content, people who are new to the language, and people with cognitive concerns.

### Examples

#### Marking up an abbreviation semantically

To mark up an abbreviation without providing an expansion or description, use `<abbr>` without any attributes, as seen in this example.

#### HTML

```
<p>Using <abbr>HTML</abbr> is fun and easy!</p>
```

#### Result

#### Styling abbreviations

You can use CSS to set a custom style to be used for abbreviations, as seen in this basic example.

#### HTML

```
<p>Using <abbr>CSS</abbr>, you can style your abbreviations!</p>
```

#### CSS

```
abbr {
  font-variant: all-small-caps;
```

```
}
```

#### Result

#### Providing an expansion

Adding a `title` attribute lets you provide an expansion or definition for the abbreviation or acronym.

#### HTML

```
<p>Ashok's joke made me <abbr title="Laugh Out Loud">LOL</abbr> big time.</p>
```

#### Result

#### Defining an abbreviation

You can use `<abbr>` in tandem with `<dfn>` to more formally define an abbreviation, as shown here.

#### HTML

```
<p>
  <dfn id="html"><abbr title="HyperText Markup Language">HTML</abbr> </dfn> is a
  markup language used to create the semantics and structure of a web page.
</p>

<p>
  A <dfn id="spec">Specification</dfn> (<abbr>spec</abbr>) is a document that
  outlines in detail how a technology or API is intended to function and how it
  is accessed.
</p>
```

#### Result

#### Technical summary

| | |
|---|---|
| **Content categories** | Flow content, phrasing content, palpable content |
| **Permitted content** | Phrasing content |
| **Tag omission** | None, both the starting and ending tag are mandatory. |
| **Permitted parents** | Any element that accepts phrasing content |
| **Implicit ARIA role** | No corresponding role |
| **Permitted ARIA roles** | Any |
| **DOM Interface** | `HTMLElement` |

#### See also

- Using the `<abbr>` element

### <acronym> HTML acronym or abbreviation element

The **`<acronym>`** HTML element allows authors to clearly indicate a sequence of characters that compose an acronym or abbreviation for a word.

> **Warning:** Don't use this element. Use the `<abbr>` element instead.

#### Attributes

This element only has global attributes, which are common to all elements.

#### DOM Interface

This element implements the `HTMLElement` interface.

#### Examples

```
<p>
  The <acronym title="World Wide Web">WWW</acronym> is only a component of the
  Internet.
</p>
```

#### Result

### Default styling

Though the purpose of this tag is purely for the convenience of the author, its default styling varies from one browser to another:

- Opera, Firefox, Chrome, and some others add a dotted underline to the content of the element.
- A few browsers not only add a dotted underline, but also put it in small caps; to avoid this styling, adding something like `font-variant: none` in the CSS takes care of this case.

It is therefore recommended that web authors either explicitly style this element, or accept some cross-browser variation.

### See also

- The `<abbr>` HTML element

## <address> HTML contact address element

The **`<address>`** HTML element indicates that the enclosed HTML provides contact information for a person or people, or for an organization.

### Try it

```
<p>Contact the author of this page:</p>

<address>
  <a href="mailto:jim@example.com">jim@example.com</a><br />
  <a href="tel:+14155550132">+1 (415) 555-0132</a>
</address>
```

```
a[href^="mailto"]::before {
  content: "✉ ";
}

a[href^="tel"]::before {
  content: "📞 ";
}
```

### Attributes

This element only includes the global attributes.

### Usage notes

The contact information provided by an `<address>` element's contents can take whatever form is appropriate for the context, and may include any type of contact information that is needed, such as a physical address, URL, email address, phone number, social media handle, geographic coordinates, and so forth. The `<address>` element should include the name of the person, people, or organization to which the contact information refers. However, this element should not contain more information than the contact information, like a publication date (which belongs in a `<time>` element).

`<address>` can be used in a variety of contexts, such as providing a business's contact information in the page header, or indicating the author of an article by including an `<address>` element within the `<article>`. The `<address>` element can only be used to represent the contact information for its nearest `<article>` or `<body>` element ancestor. Typically an `<address>` element can be placed inside the `<footer>` element of the current section, if any.

### Examples

This example demonstrates the use of `<address>` to demarcate the contact information for an article's author.

```
<address>
  You can contact author at
  <a href="http://www.example.com/contact">www.example.com</a>.<br />
  If you see any bugs, please
  <a href="mailto:webmaster@example.com">contact webmaster</a>.<br />
  You may also want to visit us:<br />
  Mozilla Foundation<br />
  331 E Evelyn Ave<br />
  Mountain View, CA 94041<br />
  USA
</address>
```

### Result

Although it renders text with the same default styling as the `<i>` or `<em>` elements, it is more appropriate to use `<address>` when dealing with contact information, as it conveys additional semantic information.

### Technical summary

| | |
|---|---|
| **Content categories** | Flow content, palpable content. |
| **Permitted content** | Flow content, but with no nested `<address>` element, no heading content (`<hgroup>`, h1, h2, h3, h4, h5, h6), no sectioning content (`<article>`, `<aside>`, `<section>`, `<nav>`), and no `<header>` or `<footer>` element. |
| **Tag omission** | None, both the starting and ending tag are mandatory. |
| **Permitted parents** | Any element that accepts flow content, but always excluding `<address>` elements (according to the logical principle of symmetry, if `<address>` tag, as a parent, cannot have nested `<address>` element, then the same `<address>` content cannot have `<address>` tag as its parent). |
| **Implicit ARIA role** | `group` |
| **Permitted ARIA roles** | Any |
| **DOM interface** | `HTMLElement` Prior to Gecko 2.0 (Firefox 4), Gecko implemented this element using the `HTMLSpanElement` interface |

### See also

- Others section-related elements: `<body>`, `<nav>`, `<article>`, `<aside>`, h1, h2, h3, h4, h5, h6, `<hgroup>`, `<footer>`, `<section>`, `<header>`;
- Sections and outlines of an HTML document.

## <area> HTML image map area element

The **`<area>`** HTML element defines an area inside an image map that has predefined clickable areas. An *image map* allows geometric areas on an image to be associated with hypertext links.

This element is used only within a `<map>` element.

### Try it

```
<map name="infographic">
  <area
    shape="poly"
    coords="129,0,260,95,129,138"
    href="https://developer.mozilla.org/docs/Web/HTTP"
    alt="HTTP" />
  <area
    shape="poly"
    coords="260,96,209,249,130,138"
    href="https://developer.mozilla.org/docs/Web/HTML"
    alt="HTML" />
  <area
    shape="poly"
    coords="209,249,49,249,130,139"
    href="https://developer.mozilla.org/docs/Web/JavaScript"
    alt="JavaScript" />
  <area
    shape="poly"
    coords="48,249,0,96,129,138"
    href="https://developer.mozilla.org/docs/Web/API"
    alt="Web APIs" />
  <area
    shape="poly"
    coords="0,95,128,0,128,137"
    href="https://developer.mozilla.org/docs/Web/CSS"
    alt="CSS" />
</map>
<img
  usemap="#infographic"
  src="https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/area//shared-assets/images/examples/mdn-info.png"
  alt="MDN infographic" />
```

```
img {
  display: block;
  margin: 0 auto;
  width: 260px;
  height: 260px;
}
```

### Attributes

This element's attributes include the global attributes.

`alt`

A text string alternative to display on browsers that do not display images. The text should be phrased so that it presents the user with the same kind of choice as the image would offer when displayed without the alternative text. This attribute is required only if the `href` attribute is used.

`coords`

The `coords` attribute details the coordinates of the `shape` attribute in size, shape, and placement of an `<area>`. This attribute must not be used if `shape` is set to `default`.

- `rect`: the value is `x1,y1,x2,y2`. The value specifies the coordinates of the top-left and bottom-right corner of the rectangle. For example, in `<area shape="rect" coords="0,0,253,27" href="#" target="_blank" alt="Mozilla">` the coordinates are `0,0` and `253,27`, indicating the top-left and bottom-right corners of the rectangle, respectively.
- `circle`: the value is `x,y,radius`. Value specifies the coordinates of the circle center and the radius. For example: `<area shape="circle" coords="130,136,60" href="#" target="_blank" alt="MDN">`
- `poly`: the value is `x1,y1,x2,y2,..,xn,yn`. Value specifies the coordinates of the edges of the polygon. If the first and last coordinate pairs are not the same, the browser will add the last coordinate pair to close the polygon

The values are numbers of CSS pixels. Our shape generator can help you generate the `coords` syntax by selecting points on an image you upload.

`download`

This attribute, if present, indicates that the linked resource is intended to be downloaded rather than displayed in the browser. See `<a>` for a full description of the `download` attribute.

`href`

The hyperlink target for the area. Its value is a valid URL. This attribute may be omitted; if so, the `<area>` element does not represent a hyperlink.

`interestfor`

Defines the `<area>` element as an **interest invoker**. Its value is the `id` of the target element, which will be affected in some way (normally shown or hidden) when interest is shown or lost on the invoker element (for example, by hovering/unhovering or focusing/blurring it). See Using interest invokers for more details and examples.

`ping`

Contains a space-separated list of URLs to which, when the hyperlink is followed, `POST` requests with the body `PING` will be sent by the browser (in the background). Typically used for tracking.

`referrerpolicy`

A string indicating which referrer to use when fetching the resource:

- `no-referrer`: The `Referer` header will not be sent.
- `no-referrer-when-downgrade`: The `Referer` header will not be sent to origins without TLS (HTTPS).
- `origin`: The sent referrer will be limited to the origin of the referring page: its scheme, host, and port.
- `origin-when-cross-origin`: The referrer sent to other origins will be limited to the scheme, the host, and the port. Navigations on the same origin will still include the path.
- `same-origin`: A referrer will be sent for same origin, but cross-origin requests will contain no referrer information.
- `strict-origin`: Only send the origin of the document as the referrer when the protocol security level stays the same (HTTPS→HTTPS), but don't send it to a less secure destination (HTTPS→HTTP).
- `strict-origin-when-cross-origin` (default): Send a full URL when performing a same-origin request, only send the origin when the protocol security level stays the same (HTTPS→HTTPS), and send no header to a less secure destination (HTTPS→HTTP).
- `unsafe-url`: The referrer will include the origin *and* the path (but not the fragment, password, or username). **This value is unsafe**, because it leaks origins and paths from TLS-protected resources to insecure origins.

`rel`

For anchors containing the `href` attribute, this attribute specifies the relationship of the target object to the link object. The value is a space-separated list of link types. The values and their semantics will be registered by some authority that might have meaning to the document author. The default relationship, if no other is given, is void. Use this attribute only if the `href` attribute is present.

`shape`

The shape of the associated hot spot. The specifications for HTML defines the values `rect`, which defines a rectangular region; `circle`, which defines a circular region; `poly`, which defines a polygon; and `default`, which indicates the entire region beyond any defined shapes.

`target`

A keyword or author-defined name of the browsing context to display the linked resource. The following keywords have special meanings:

- `_self` (default): Show the resource in the current browsing context.
- `_blank`: Show the resource in a new, unnamed browsing context.
- `_parent`: Show the resource in the parent browsing context of the current one, if the current page is inside a frame. If there is no parent, acts the same as `_self`.
- `_top`: Show the resource in the topmost browsing context (the browsing context that is an ancestor of the current one and has no parent). If there is no parent, acts the same as `_self`.

Use this attribute only if the `href` attribute is present.

> **Note:** Setting `target="_blank"` on `<area>` elements implicitly provides the same `rel` behavior as setting `rel="noopener"` which does not set `window.opener`. See browser compatibility for support status.

### Examples

#### Image with clickable areas

```
<map name="primary">
  <area
    shape="circle"
    coords="75,75,75"
    href="left.html"
    alt="Click to go Left" />
  <area
    shape="circle"
    coords="275,75,75"
    href="right.html"
    alt="Click to go Right" />
</map>
<img
  usemap="#primary"
  src="https://dummyimage.com/350x150"
  alt="350 x 150 pic" />
```

### Technical summary

| | |
|---|---|
| **Content categories** | Flow content, phrasing content. |
| **Permitted content** | None; it is a void element. |
| **Tag omission** | Must have a start tag and must not have an end tag. |
| **Permitted parents** | Any element that accepts phrasing content. The `<area>` element must have an ancestor `<map>`, but it need not be a direct parent. |
| **Implicit ARIA role** | `link` when `href` attribute is present, otherwise `generic` |
| **Permitted ARIA roles** | No `role` permitted |
| **DOM interface** | `HTMLAreaElement` |

## <article> HTML article contents element

The **`<article>`** HTML element represents a self-contained composition in a document, page, application, or site, which is intended to be independently distributable or reusable (e.g., in syndication). Examples include: a forum post, a magazine or newspaper article, or a blog entry, a product card, a user-submitted comment, an interactive widget or gadget, or any other independent item of content.

### Try it

```
<article class="forecast">
  <h1>Weather forecast for Seattle</h1>
  <article class="day-forecast">
    <h2>03 March 2018</h2>
    <p>Rain.</p>
  </article>
  <article class="day-forecast">
    <h2>04 March 2018</h2>
    <p>Periods of rain.</p>
  </article>
  <article class="day-forecast">
    <h2>05 March 2018</h2>
    <p>Heavy rain.</p>
  </article>
</article>
```

```
.forecast {
  margin: 0;
  padding: 0.3rem;
  background-color: #eeeeee;
}

.forecast > h1,
.day-forecast {
  margin: 0.5rem;
  padding: 0.3rem;
  font-size: 1.2rem;
}

.day-forecast {
  background: right/contain content-box border-box no-repeat
    url("/shared-assets/images/examples/rain.svg") white;
}

.day-forecast > h2,
.day-forecast > p {
  margin: 0.2rem;
  font-size: 1rem;
}
```

### Attributes

This element only includes the global attributes.

### Usage notes

A given document can have multiple articles in it; for example, on a blog that shows the text of each article one after another as the reader scrolls, each post would be contained in an `<article>` element, possibly with one or more `<section>`s within.

- Each `<article>` should be identified, typically by including a heading (`<h1>` - `<h6>` element) as a child of the `<article>` element.
- When an `<article>` element is nested, the inner element represents an article related to the outer element. For example, the comments of a blog post can be `<article>` elements nested in the `<article>` representing the blog post.
- Author information of an `<article>` element can be provided through the `<address>` element, but it doesn't apply to nested `<article>` elements.
- The publication date and time of an `<article>` element can be described using the `datetime` attribute of a `<time>` element.

### Examples

```
<article class="film_review">
  <h2>Jurassic Park</h2>
  <section class="main_review">
    <h3>Review</h3>
    <p>Dinos were great!</p>
  </section>
  <section class="user_reviews">
    <h3>User reviews</h3>
    <article class="user_review">
      <h4>Too scary!</h4>
      <p>Way too scary for me.</p>
      <footer>
        <p>
          Posted on
          <time datetime="2015-05-16 19:00">May 16</time>
          by Lisa.
        </p>
      </footer>
    </article>
    <article class="user_review">
      <h4>Love the dinos!</h4>
      <p>I agree, dinos are my favorite.</p>
      <footer>
        <p>
          Posted on
          <time datetime="2015-05-17 19:00">May 17</time>
          by Tom.
        </p>
      </footer>
    </article>
  </section>
  <footer>
    <p>
      Posted on
```

```
      <time datetime="2015-05-15 19:00">May 15</time>
      by Staff.
    </p>
  </footer>
</article>
```

### Result

### Technical summary

| | |
|---|---|
| **Content categories** | Flow content, sectioning content, palpable content |
| **Permitted content** | Flow content. |
| **Tag omission** | None, both the starting and ending tag are mandatory. |
| **Permitted parents** | Any element that accepts flow content. Note that an `<article>` element must not be a descendant of an `<address>` element. |
| **Implicit ARIA role** | `article` |
| **Permitted ARIA roles** | `application`, `document`, `feed`, `main`, `none`, `presentation`, `region` |
| **DOM interface** | `HTMLElement` |

### See also

- Other section-related elements: `<body>`, `<nav>`, `<section>`, `<aside>`, h1, h2, h3, h4, h5, h6, `<hgroup>`, `<header>`, `<footer>`, `<address>`
- Using HTML sections and outlines

## <aside> HTML aside element

The **`<aside>`** HTML element represents a portion of a document whose content is only indirectly related to the document's main content. Asides are frequently presented as sidebars or call-out boxes.

### Try it

```
<p>
  Salamanders are a group of amphibians with a lizard-like appearance, including
  short legs and a tail in both larval and adult forms.
</p>

<aside>
  <p>The Rough-skinned Newt defends itself with a deadly neurotoxin.</p>
</aside>

<p>
  Several species of salamander inhabit the temperate rainforest of the Pacific
  Northwest, including the Ensatina, the Northwestern Salamander and the
  Rough-skinned Newt. Most salamanders are nocturnal, and hunt for insects,
  worms and other small creatures.
</p>
```

```
aside {
  width: 40%;
  padding-left: 0.5rem;
  margin-left: 0.5rem;
  float: right;
  box-shadow: inset 5px 0 5px -5px #29627e;
  font-style: italic;
  color: #29627e;
}

aside > p {
  margin: 0.5rem;
}
```

### Attributes

This element only includes the global attributes.

### Usage notes

- Do not use the `<aside>` element to tag parenthesized text, as this kind of text is considered part of the main flow.
- The `<aside>` element has an implicit role of `complementary`, unless it has no accessible name and is nested in sectioning content, in which case it has the `generic` role.

### Examples

#### Using <aside>

This example uses `<aside>` to mark up a paragraph in an article. The paragraph is only indirectly related to the main article content:

```
<article>
  <p>
    The Disney movie <cite>The Little Mermaid</cite> was first released to
    theatres in 1989.
  </p>
  <aside>
    <p>The movie earned $87 million during its initial release.</p>
  </aside>
  <p>More info about the movie…</p>
</article>
```

#### Result

#### Technical summary

| | |
|---|---|
| **Content categories** | Flow content, sectioning content, palpable content. |
| **Permitted content** | Flow content. |
| **Tag omission** | None, both the starting and ending tag are mandatory. |
| **Permitted parents** | Any element that accepts flow content. Note that an `<aside>` element must not be a descendant of an `<address>` element. |
| **Implicit ARIA role** | `complementary`, or `generic` when it has no accessible name and is nested in sectioning content |
| **Permitted ARIA roles** | `feed`, `none`, `note`, `presentation`, `region`, `search` |
| **DOM interface** | `HTMLElement` |

#### See also

- Other section-related elements: `<body>`, `<article>`, `<section>`, `<nav>`, h1, h2, h3, h4, h5, h6, `<hgroup>`, `<header>`, `<footer>`, `<address>`;
- Using HTML sections and outlines
- ARIA: Complementary role

## <audio> HTML embed audio element

The **`<audio>`** HTML element is used to embed sound content in documents. It may contain one or more audio sources, represented using the `src` attribute or the `<source>` element: the browser will choose the most suitable one. It can also be the destination for streamed media, using a `MediaStream`.

#### Try it

```
<figure>
  <figcaption>Listen to the T-Rex:</figcaption>
  <audio controls src="https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/audio//shared-assets/audio/t-rex-roar.mp3">
</audio>
  <a href="https://developer.mozilla.org/shared-assets/audio/t-rex-roar.mp3"> Download audio </a>
</figure>
```

```
figure {
  margin: 0;
}
```

The above example shows basic usage of the `<audio>` element. In a similar manner to the `<img>` element, we include a path to the media we want to embed inside the `src` attribute; we can include other attributes to specify information such as whether we want it to autoplay and loop, whether we want to show the browser's default audio controls, etc.

The content inside the opening and closing `<audio></audio>` tags is shown as a fallback in browsers that don't support the element.

#### Attributes

This element's attributes include the global attributes.

`autoplay`

> A Boolean attribute: if specified, the audio will automatically begin playback as soon as it can do so, without waiting for the entire audio file to finish downloading.

> **Note:** Sites that automatically play audio (or videos with an audio track) can be an unpleasant experience for users, so should be avoided when possible. If you must offer autoplay functionality, you should make it opt-in (requiring a user to specifically enable it). However, this can be useful when creating media elements whose source will be set at a later time, under user control. See our autoplay guide for additional information about how to properly use autoplay.

> **Note:** Audio with the `loading="lazy"` attribute set will not start downloading and autoplaying until the controls for the media are near or within the viewport. Lazy-loaded audio without the `controls` attribute will not autoplay.

`controls`

If this attribute is present, the browser will offer controls to allow the user to control audio playback, including volume, seeking, and pause/resume playback.

`controlslist`

The `controlslist` attribute, when specified, helps the browser select what controls to show for the `audio` element whenever the browser shows its own set of controls (that is, when the `controls` attribute is specified).

The allowed values are `nodownload`, `nofullscreen` and `noremoteplayback`.

`crossorigin`

This enumerated attribute indicates whether to use CORS to fetch the related audio file. CORS-enabled resources can be reused in the `<canvas>` element without being *tainted*. The allowed values are:

`anonymous`

Sends a cross-origin request without a credential. In other words, it sends the `Origin:` HTTP header without a cookie, X.509 certificate, or performing HTTP Basic authentication. If the server does not give credentials to the origin site (by not setting the `Access-Control-Allow-Origin:` HTTP header), the resource will be *tainted*, and its usage restricted.

`use-credentials`

Sends a cross-origin request with a credential. In other words, it sends the `Origin:` HTTP header with a cookie, a certificate, or performing HTTP Basic authentication. If the server does not give credentials to the origin site (through `Access-Control-Allow-Credentials:` HTTP header), the resource will be *tainted* and its usage restricted.

When not present, the resource is fetched without a CORS request (i.e., without sending the `Origin:` HTTP header), preventing its non-tainted use in `<canvas>` elements. If invalid, it is handled as if the enumerated keyword **anonymous** was used. See CORS settings attributes for additional information.

`disableremoteplayback`

A Boolean attribute used to disable the capability of remote playback in devices that are attached using wired (HDMI, DVI, etc.) and wireless technologies (Miracast, Chromecast, DLNA, AirPlay, etc.). See the proposed Remote Playback API specification for more information.

In Safari, you can use `x-webkit-airplay="deny"` as a fallback.

`loading`

Indicates how the browser should load the audio:

`eager`

Loads the audio immediately, regardless of whether or not the audio is currently within the visible viewport (this is the default value).

`lazy`

Defers loading the audio until the controls reach a calculated distance from the viewport, as defined by the browser.

> **Note:** In order for audio elements to intersect visibly with the viewport, they need to be visible. Browsers use the `controls` attribute to make audio elements visible, so it's needed for lazy loading. Lazy-loaded audio without the `controls` attribute will not load.

Lazy loading avoids the network and storage bandwidth required to handle the audio until it's reasonably certain that it will be needed. This improves the performance in most typical use cases.

Lazy-loaded audio located in the visual viewport may not yet be downloaded when the Window `load` event is fired. This is because the event is fired based on eager-loaded audio only — lazy-loaded audio is not considered even if it is located within the visual viewport upon initial page load.

Loading is only deferred when JavaScript is enabled. This is an anti-tracking measure, because if a user agent supported lazy loading when scripting is disabled, it would still be possible for a site to track a user's approximate scroll position throughout a session, by strategically placing audio in a page's markup such that a server can track how many audio requests are made and when.

> **Note:** The `loading="lazy"` attribute also impacts the `autoplay` attribute as described in that section of this page.

`loop`

A Boolean attribute: if specified, the audio player will automatically seek back to the start upon reaching the end of the audio.

`muted`

A Boolean attribute that indicates the default audio mute setting contained in the audio. If set, the audio will be initially silenced. Its default value is `false`, meaning the audio will be heard when the audio is played.

> **Note:** To unmute, setting `muted="false"` will not work; the audio will be muted if the attribute is present at all. To unmute, the attribute must be removed entirely.

`preload`

This enumerated attribute is intended to provide a hint to the browser about what the author thinks will lead to the best user experience. It may have one of the following values:

- `none`: Indicates that the audio should not be preloaded.
- `metadata`: Indicates that only audio metadata (e.g., length) is fetched.
- `auto`: Indicates that the whole audio file can be downloaded, even if the user is not expected to use it.
- *empty string*: A synonym of the `auto` value.

The default value is different for each browser. The spec advises it to be set to `metadata`.

> **Note:**
>
> - Audio with the `loading="lazy"` attribute set will only apply the `preload` behavior once the audio controls are near or within the viewport.
> - The `autoplay` attribute has precedence over `preload`. If `autoplay` is specified, the browser would obviously need to start downloading the audio for playback.
> - The browser is not forced by the specification to follow the value of this attribute; it is a mere hint.

`src`

The URL of the audio to embed. This is subject to HTTP access controls. This is optional; you may instead use the `<source>` element within the audio block to specify the audio to embed.

### Events

`audioprocess`

The input buffer of a `ScriptProcessorNode` is ready to be processed.

`canplay`

The browser can play the media, but estimates that not enough data has been loaded to play the media up to its end without having to stop for further buffering of content.

`canplaythrough`

The browser estimates it can play the media up to its end without stopping for content buffering.

`complete`

The rendering of an `OfflineAudioContext` is terminated.

`durationchange`

The `duration` attribute has been updated.

`emptied`

The media has become empty; for example, this event is sent if the media has already been loaded (or partially loaded), and the `HTMLMediaElement.load` method is called to reload it.

`ended`

Playback has stopped because the end of the media was reached.

`loadeddata`

The first frame of the media has finished loading.

`loadedmetadata`

The metadata has been loaded.

`loadstart`

Fired when the browser has started to load the resource.

`pause`

Playback has been paused.

`play`

Playback has begun.

`playing`

Playback is ready to start after having been paused or delayed due to lack of data.

`ratechange`

The playback rate has changed.

`seeked`

A *seek* operation completed.

`seeking`

A *seek* operation began.

`stalled`

The user agent is trying to fetch media data, but data is unexpectedly not forthcoming.

`suspend`

Media data loading has been suspended.

`timeupdate`

The time indicated by the `currentTime` attribute has been updated.

`volumechange`

The volume has changed.

`waiting`

Playback has stopped because of a temporary lack of data.

### Usage notes

Browsers don't all support the same file types and audio codecs; you can provide multiple sources inside nested `<source>` elements, and the browser will then use the first one it understands:

```
<audio controls>
  <source src="https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/audio/myAudio.mp3" type="audio/mpeg" />
  <source src="https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/audio/myAudio.ogg" type="audio/ogg" />
  <p>
    Download <a href="myAudio.mp3" download="myAudio.mp3">MP3</a> or
    <a href="myAudio.ogg" download="myAudio.ogg">OGG</a> audio.
  </p>
</audio>
```

The audio source can be set to any valid URL, including HTTP(S) URLs and Data URLs. When using HTTP(S) URLs, be aware that the browser's caching behavior will affect how often the file is requested from the server. Data URLs embed the audio data directly in the HTML, which can be useful for small audio files but isn't recommended for larger files as it increases the HTML file size.

When using `<source>` elements, the browser attempts to load each source sequentially. If a source fails (e.g., due to an invalid URL or unsupported format), the next source is attempted, and so on. An `error` event fires on the `<audio>` element after all sources have failed; `error` events are not fired on each individual `<source>` element.

You can also use the Web Audio API to directly generate and manipulate audio streams from JavaScript code rather than streaming pre-existing audio files. You can set the `srcObject` in JavaScript to a `MediaStream` object. This is commonly used for live audio streams or real-time audio processing.

```
const audioElement = document.querySelector("audio");
navigator.mediaDevices
  .getUserMedia({ audio: true })
  .then((stream) => {
    audioElement.srcObject = stream;
  })
  .catch((error) => {
    console.error("Error accessing the microphone", error);
  });
```

Note that `MediaStream` sources have limitations: they are not seekable and only support a limited set of codecs.

We offer a substantive and thorough guide to media file types and the audio codecs that can be used within them. Also available is a guide to the codecs supported for video.

Other usage notes:

- If you don't specify the `controls` attribute, the audio player won't include the browser's default controls. You can, however, create your own custom controls using JavaScript and the `HTMLMediaElement` API.
- To allow precise control over your audio content, `HTMLMediaElement`s fire many different events. This also provides a way to monitor the audio's fetching process so you can watch for errors or detect when enough is available to begin to play or manipulate it.
- `<audio>` elements can't have subtitles or captions associated with them in the same way that `<video>` elements can. See WebVTT and Audio by Ian Devlin for some useful information and workarounds.
- To test the fallback content on browsers that support the element, you can replace `<audio>` with a non-existing element like `<notanaudio>`.

A good general source of information on using HTML `<audio>` is the HTML video and audio beginner's tutorial.

### Styling with CSS

The `<audio>` element has no intrinsic visual output of its own unless the `controls` attribute is specified, in which case the browser's default controls are shown.

The default controls have a `display` value of `inline` by default, and it is often a good idea to set the value to `block` to improve control over positioning and layout, unless you want it to sit within a text block or similar.

You can style the default controls with properties that affect the block as a single unit, so for example you can give it a `border` and `border-radius`, `padding`, `margin`, etc. You can't however style the individual components inside the audio player (e.g., change the button size or icons, change the font, etc.), and the controls are different across the different browsers.

To get a consistent look and feel across browsers, you'll need to create custom controls; these can be marked up and styled in whatever way you want, and then JavaScript can be used along with the `HTMLMediaElement` API to wire up their functionality.

Video player styling basics provides some useful styling techniques — it is written in the context of `<video>`, but much of it is equally applicable to `<audio>`.

### Detecting addition and removal of tracks

You can detect when tracks are added to and removed from an `<audio>` element using the `addtrack` and `removetrack` events. However, these events aren't sent directly to the `<audio>` element itself. Instead, they're sent to the track list object within the `<audio>` element's `HTMLMediaElement` that corresponds to the type of track that was added to the element:

`HTMLMediaElement.audioTracks`

> An `AudioTrackList` containing all of the media element's audio tracks. You can add a listener for `addtrack` to this object to be alerted when new audio tracks are added to the element.

`HTMLMediaElement.videoTracks`

> Add an `addtrack` listener to this `VideoTrackList` object to be informed when video tracks are added to the element.

`HTMLMediaElement.textTracks`

> Add an `addtrack` event listener to this `TextTrackList` to be notified when new text tracks are added to the element.

> **Note:** Even though it's an `<audio>` element, it still has video and text track lists, and can in fact be used to present video, although the user interface implications can be odd.

For example, to detect when audio tracks are added to or removed from an `<audio>` element, you can use code like this:

```
const elem = document.querySelector("audio");

elem.audioTrackList.onaddtrack = (event) => {
  trackEditor.addTrack(event.track);
};

elem.audioTrackList.onremovetrack = (event) => {
  trackEditor.removeTrack(event.track);
};
```

This code watches for audio tracks to be added to and removed from the element, and calls a hypothetical function on a track editor to register and remove the track from the editor's list of available tracks.

You can also use `addEventListener()` to listen for the `addtrack` and `removetrack` events.

### Accessibility

Audio with spoken dialog should provide both captions and transcripts that accurately describe its content. Captions, which are specified using WebVTT, allow people who are hearing impaired to understand an audio recording's content as the recording is being played, while transcripts allow people who need additional time to be able to review the recording's content at a pace and format that is comfortable for them.

If automatic captioning services are used, it is important to review the generated content to ensure it accurately represents the source audio.

The `<audio>` element doesn't directly support WebVTT. You will have to find a library or framework that provides the capability for you, or write the code to display captions yourself. One option is to play your audio using a `<video>` element, which does support WebVTT.

In addition to spoken dialog, subtitles and transcripts should also identify music and sound effects that communicate important information. This includes emotion and tone. For example, in the WebVTT below, note the use of square brackets to provide tone and emotional insight to the viewer; this can help establish the mood otherwise provided using music, nonverbal sounds and crucial sound effects, and so forth.

```
1
00:00:00 --> 00:00:45
[Energetic techno music]

2
00:00:46 --> 00:00:51
Welcome to the Time Keeper's podcast! In this episode we're discussing which Swisswatch is a wrist switchwatch?

16
00:00:52 --> 00:01:02
[Laughing] Sorry! I mean, which wristwatch is a Swiss wristwatch?
```

Also it's a good practice to provide some content (such as the direct download link) as a fallback for viewers who use a browser in which the `<audio>` element is not supported:

```
<audio controls>
  <source src="https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/audio/myAudio.mp3" type="audio/mpeg" />
  <source src="https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/audio/myAudio.ogg" type="audio/ogg" />
  <p>
    Download <a href="myAudio.mp3">MP3</a> or
    <a href="myAudio.ogg" download="myAudio.ogg">OGG</a> audio.
  </p>
</audio>
```

- Web Video Text Tracks Format (WebVTT)
- WebAIM: Captions, Transcripts, and Audio Descriptions
- MDN Understanding WCAG, Guideline 1.2 explanations
- Understanding Success Criterion 1.2.1 | W3C Understanding WCAG 2.0
- Understanding Success Criterion 1.2.2 | W3C Understanding WCAG 2.0

### Examples

### Basic usage

The following example shows basic usage of the `<audio>` element to play an OGG file. It will autoplay due to the `autoplay` attribute—if the page has permission to do so—and also includes fallback content.

```
<!-- Basic audio playback -->
<audio src="https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/audio/AudioTest.ogg" autoplay>
  <a href="AudioTest.ogg" download="AudioTest.ogg">Download OGG audio</a>.
</audio>
```

For details on when autoplay works, how to get permission to use autoplay, and how and when it's appropriate to use autoplay, see our autoplay guide.

#### <audio> element with <source> element

This example specifies which audio track to embed using the `src` attribute on a nested `<source>` element rather than directly on the `<audio>` element. It is always useful to include the file's MIME type inside the `type` attribute, as the browser is able to instantly tell if it can play that file, and not waste time on it if not.

```
<audio controls>
  <source src="https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/audio/foo.wav" type="audio/wav" />
  <a href="foo.wav" download="foo.wav">Download WAV audio</a>.
</audio>
```

#### <audio> with multiple <source> elements

This example includes multiple `<source>` elements. The browser tries to load the first source element (Opus) if it is able to play it; if not it falls back to the second (Vorbis) and finally back to MP3:

```
<audio controls>
  <source src="https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/audio/foo.opus" type="audio/ogg; codecs=opus" />
  <source src="https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/audio/foo.ogg" type="audio/ogg; codecs=vorbis" />
  <source src="https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/audio/foo.mp3" type="audio/mpeg" />
</audio>
```

### Technical summary

| | |
|---|---|
| **Content categories** | Flow content, phrasing content, embedded content. If it has a `controls` attribute: interactive content and palpable content. |
| **Permitted content** | If the element has a `src` attribute: zero or more `<track>` elements followed by transparent content that contains no `<audio>` or `<video>` media elements.<br>Else: zero or more `<source>` elements followed by zero or more `<track>` elements followed by transparent content that contains no `<audio>` or `<video>` media elements. |
| **Tag omission** | None, both the starting and ending tag are mandatory. |
| **Permitted parents** | Any element that accepts embedded content. |
| **Implicit ARIA role** | No corresponding role |
| **Permitted ARIA roles** | `application` |
| **DOM interface** | `HTMLAudioElement` |

### See also

- Web media technologies
  - Media container formats (file types)
  - Guide to audio codecs used on the web
- Web Audio API
- `HTMLAudioElement`
- `<source>`
- `<video>`
- Learning area: HTML video and audio
- Cross-browser audio basics

## <b> HTML bring attention to element

The **`<b>`** HTML element is used to draw the reader's attention to the element's contents, which are not otherwise granted special importance. This was formerly known as the Boldface element, and most browsers still draw the text in boldface. However, you should not use `<b>` for styling text or

granting importance. If you wish to create boldface text, you should use the CSS `font-weight` property. If you wish to indicate an element is of special importance, you should use the `<strong>` element.

### Try it

```
<p>
  The two most popular science courses offered by the school are
  <b class="term">chemistry</b> (the study of chemicals and the composition of
  substances) and <b class="term">physics</b> (the study of the nature and
  properties of matter and energy).
</p>
```

```
b {
  /* Add your styles here */
}
```

### Attributes

This element only includes the global attributes.

### Usage notes

- Use the `<b>` for cases like keywords in a summary, product names in a review, or other spans of text whose typical presentation would be boldfaced (but not including any special importance).
- Do not confuse the `<b>` element with the `<strong>`, `<em>`, or `<mark>` elements. The `<strong>` element represents text of certain *importance*, `<em>` puts some emphasis on the text and the `<mark>` element represents text of certain *relevance*. The `<b>` element doesn't convey such special semantic information; use it only when no others fit.
- Similarly, do not mark titles and headings using the `<b>` element. For this purpose, use the h1 to h6 tags. Further, stylesheets can change the default style of these elements, with the result that they are not *necessarily* displayed in bold.
- It is a good practice to use the `class` attribute on the `<b>` element in order to convey additional semantic information as needed (for example `<b class="lead">` for the first sentence in a paragraph). This makes it easier to manage multiple use cases of `<b>` if your stylistic needs change, without the need to change all of its uses in the HTML.
- Historically, the `<b>` element was meant to make text boldface. Styling information has been deprecated since HTML4, so the meaning of the `<b>` element has been changed.
- If there is no semantic purpose to using the `<b>` element, you should use the CSS `font-weight` property with the value `"bold"` instead in order to make text bold.

### Examples

```
<p>
  This article describes several <b class="keywords">text-level</b> elements. It
  explains their usage in an <b class="keywords">HTML</b> document.
</p>
Keywords are displayed with the default style of the
<b>element, likely in bold.</b>
```

### Result

### Technical summary

| | |
|---|---|
| **Content categories** | Flow content, phrasing content, palpable content. |
| **Permitted content** | Phrasing content. |
| **Tag omission** | None, both the starting and ending tag are mandatory. |
| **Permitted parents** | Any element that accepts phrasing content. |
| **Implicit ARIA role** | `generic` |
| **Permitted ARIA roles** | Any |
| **DOM interface** | `HTMLElement` |

### See also

- Other elements conveying text-level semantics: `<a>`, `<em>`, `<strong>`, `<small>`, `<cite>`, `<q>`, `<dfn>`, `<abbr>`, `<time>`, `<code>`, `<var>`, `<samp>`, `<kbd>`, `<sub>`, `<sup>`, `<i>`, `<mark>`, `<ruby>`, `<rp>`, `<rt>`, `<bdo>`, `<span>`, `<br>`, `<wbr>`.
- Using <b> and <i> elements (W3C)

## <base> HTML document base URL element

The **`<base>`** HTML element specifies the base URL to use for all *relative* URLs in a document. There can be only one `<base>` element in a document.

A document's used base URL can be accessed by scripts with `Node.baseURI`. If the document has no `<base>` elements, then `baseURI` defaults to `location.href`.

### Attributes

This element's attributes include the global attributes.

> **Warning:** A `<base>` element must have an `href` attribute, a `target` attribute, or both. If at least one of these attributes are specified, the `<base>` element **must** come before other elements with attribute values that are URLs, such as a `<link>`'s `href` attribute.

`href`

The base URL to be used throughout the document for relative URLs. Absolute and relative URLs are allowed. `data:` and `javascript:` URLs are not allowed.

`target`

A **keyword** or **author-defined name** of the default browsing context to show the results of navigation from `<a>`, `<area>`, or `<form>` elements without explicit `target` attributes. The following keywords have special meanings:

- `_self` (default): Show the result in the current browsing context.
- `_blank`: Show the result in a new, unnamed browsing context.
- `_parent`: Show the result in the parent browsing context of the current one, if the current page is inside a frame. If there is no parent, acts the same as `_self`.
- `_top`: Show the result in the topmost browsing context (the browsing context that is an ancestor of the current one and has no parent). If there is no parent, acts the same as `_self`.

### Usage notes

#### Multiple <base> elements

If multiple `<base>` elements are used, only the first `href` and first `target` are obeyed — all others are ignored.

#### In-page anchors

Links pointing to a fragment in the document — e.g., `<a href="#some-id">` — are resolved with the `<base>`, triggering an HTTP request to the base URL with the fragment attached.

For example, given `<base href="https://example.com/">` and this link: `<a href="#anchor">To anchor</a>`. The link points to `https://example.com/#anchor`.

#### target may not contain ASCII newline, tab, or <

If the `target` attribute contains an ASCII newline, tab, or the `<` character, the value is reset to `_blank`. This is to prevent dangling markup injection attacks, a script-less attack in which an unclosed `target` attribute is injected into the page so that any text that follows is captured until the browser reaches a character that closes the attribute.

### Open Graph

Open Graph tags do not acknowledge `<base>`, and should always have full absolute URLs. For example:

```
<meta property="og:image" content="https://example.com/thumbnail.jpg" />
```

### Examples

```
<base href="https://www.example.com/" />
<base target="_blank" />
<base target="_top" href="https://example.com/" />
```

### Technical summary

| | |
|---|---|
| **Content categories** | Metadata content. |
| **Permitted content** | None; it is a void element. |
| **Tag omission** | Must have a start tag and must not have an end tag. |
| **Permitted parents** | A `<head>` that doesn't contain another `<base>` element. |
| **Implicit ARIA role** | No corresponding role |
| **Permitted ARIA roles** | No `role` permitted |
| **DOM interface** | `HTMLBaseElement` |

## <bdi> HTML bidirectional isolate element

The **<bdi>** HTML element tells the browser's bidirectional algorithm to treat the text it contains in isolation from its surrounding text. It's particularly useful when a website dynamically inserts some text and doesn't know the directionality of the text being inserted.

### Try it

```
<h1>World wrestling championships</h1>

<ul>
  <li><bdi class="name">Evil Steven</bdi>: 1st place</li>
  <li><bdi class="name">François fatale</bdi>: 2nd place</li>
  <li><span class="name">سما</span>: 3rd place</li>
  <li><bdi class="name">الرجل القوي إيان</bdi>: 4th place</li>
  <li><span class="name" dir="auto">سما</span>: 5th place</li>
</ul>
```

```
html {
  font-family: sans-serif;
}

bdi {
  /* Add your styles here */
}

.name {
  color: red;
}
```

### Attributes

This element only includes the global attributes.

If the `dir` attribute is unspecified, the element still behaves as if `dir="auto"` is specified.

### Usage notes

Bidirectional text is text that may contain both sequences of characters that are arranged left-to-right (LTR) and sequences of characters that are arranged right-to-left (RTL), such as an Arabic quotation embedded in an English string. Browsers implement the Unicode Bidirectional Algorithm to handle this. In this algorithm, characters are given an implicit directionality: for example, Latin characters are treated as LTR while Arabic characters are treated as RTL. Some other characters (such as spaces and some punctuation) are treated as neutral and are assigned directionality based on that of their surrounding characters.

Usually, the bidirectional algorithm will do the right thing without the author having to provide any special markup but, occasionally, the algorithm needs help. That's where `<bdi>` comes in.

The `<bdi>` element is used to wrap a span of text and instructs the bidirectional algorithm to treat this text in isolation from its surroundings. This works in two ways:

- The directionality of text embedded in `<bdi>` *does not influence* the directionality of the surrounding text.
- The directionality of text embedded in `<bdi>` *is not influenced by* the directionality of the surrounding text.

For example, consider some text like:

```
EMBEDDED-TEXT - 1st place
```

If `EMBEDDED-TEXT` is LTR, this works fine. But if `EMBEDDED-TEXT` is RTL, then `- 1` will be treated as RTL text (because it consists of neutral and weak characters). The result will be garbled:

```
1 - EMBEDDED-TEXTst place
```

If you know the directionality of `EMBEDDED-TEXT` in advance, you can fix this problem by wrapping `EMBEDDED-TEXT` in a `<span>` with the `dir` attribute set to the known directionality. But if you don't know the directionality - for example, because `EMBEDDED-TEXT` is being read from a database or entered by the user - you should use `<bdi>` to prevent the directionality of `EMBEDDED-TEXT` from affecting its surroundings.

Though the same visual effect can be achieved using the CSS rule `unicode-bidi: isolate` on a `<span>` or another text-formatting element, HTML authors should not use this approach because it is not semantic and browsers are allowed to ignore CSS styling.

If a semantic element such as `<cite>` is appropriate, prefer using `<cite dir="auto">` instead of `<bdi>`. If no semantic element is appropriate and the intended direction is `auto` (infer from content), `<bdi>` is shorter than `<span dir="auto">` and conveys the author intent better. When specifying explicit directionalities such as `ltr`, `<bdi dir="ltr">` and `<span dir="ltr">` are exactly equivalent.

### Examples

### No bdi with only LTR

This example lists the winners of a competition using `<span>` elements only. When the names only contain LTR text the results look fine:

```
<ul>
  <li><span class="name">Henrietta Boffin</span> - 1st place</li>
  <li><span class="name">Jerry Cruncher</span> - 2nd place</li>
</ul>
```

### No bdi with RTL text

This example lists the winners of a competition using `<span>` elements only, and one of the winners has a name consisting of RTL text. In this case the `- 1`, which consists of characters with neutral or weak directionality, will adopt the directionality of the RTL text, and the result will be garbled:

```
<ul>
  <li><span class="name">اَلأَعْشَى</span> - 1st place</li>
  <li><span class="name">Jerry Cruncher</span> - 2nd place</li>
</ul>
```

### Using bdi with LTR and RTL text

This example lists the winners of a competition using `<bdi>` elements. These elements instruct the browser to treat the name in isolation from its embedding context, so the example output is properly ordered:

```
<ul>
  <li><bdi class="name">اَلأَعْشَى</bdi> - 1st place</li>
  <li><bdi class="name">Jerry Cruncher</bdi> - 2nd place</li>
</ul>
```

### Technical summary

| | |
|---|---|
| **Content categories** | Flow content, phrasing content, palpable content. |
| **Permitted content** | Phrasing content. |
| **Tag omission** | None, both the starting and ending tag are mandatory. |
| **Permitted parents** | Any element that accepts phrasing content. |
| **Implicit ARIA role** | `generic` |
| **Permitted ARIA roles** | Any |
| **DOM interface** | `HTMLElement` |

### See also

- Inline markup and bidirectional text in HTML
- Unicode Bidirectional Algorithm basics
- Localization
- Related HTML element: `<bdo>`
- Related CSS properties: `direction`, `unicode-bidi`

## <bdo> HTML bidirectional text override element

The **`<bdo>`** HTML element overrides the current directionality of text, so that the text within is rendered in a different direction.

### Try it

```
<h1>Famous seaside songs</h1>

<p>The English song "Oh I do like to be beside the seaside"</p>

<p>
  Looks like this in Hebrew:
  <span dir="rtl">אה, אני אוהב להיות ליד חוף הים</span>
</p>

<p>
  In the computer's memory, this is stored as
  <bdo dir="ltr">אה, אני אוהב להיות ליד חוף הים</bdo>
</p>
```

```
html {
  font-family: sans-serif;
}
```

```
bdo {
  /* Add your styles here */
}
```

### Attributes

This element's attributes include the global attributes.

`dir`

The direction in which text should be rendered in this element's contents. Possible values are:

- `ltr`: Indicates that the text should go in a left-to-right direction.
- `rtl`: Indicates that the text should go in a right-to-left direction.

### Usage notes

The text's characters are drawn from the starting point in the given direction; the individual characters' orientation is not affected (so characters don't get drawn backward, for example).

### Examples

```
<!-- Switch text direction -->
<p>This text will go left to right.</p>
<p><bdo dir="rtl">This text will go right to left.</bdo></p>
```

### Result

### Technical summary

| | |
|---|---|
| **Content categories** | Flow content, phrasing content, palpable content. |
| **Permitted content** | Phrasing content. |
| **Tag omission** | None, both the starting and ending tag are mandatory. |
| **Permitted parents** | Any element that accepts phrasing content. |
| **Implicit ARIA role** | `generic` |
| **Permitted ARIA roles** | Any |
| **DOM interface** | `HTMLElement` Up to Gecko 1.9.2 (Firefox 4) inclusive, Firefox implements the `HTMLSpanElement` interface for this element. |

### See also

- Related HTML element: `<bdi>`

## <big> HTML bigger text element

The **`<big>`** HTML deprecated element renders the enclosed text at a font size one level larger than the surrounding text (`medium` becomes `large`, for example). The size is capped at the browser's maximum permitted font size.

> **Warning:** This element has been removed from the specification and shouldn't be used anymore. Use the CSS `font-size` property to adjust the font size.

### Attributes

This element has no other attributes than the global attributes, common to all elements.

### Examples

Here we see examples showing the use of `<big>` followed by an example showing how to accomplish the same results using modern CSS syntax instead.

### Using big

This example uses the obsolete `<big>` element to increase the size of some text.

### HTML

```
<p>
  This is the first sentence.
  <big>This whole sentence is in bigger letters.</big>
</p>
```

#### Result

#### Using CSS <code>font-size</code>

This example uses the CSS `font-size` property to increase the font size by one level.

#### CSS

```
.bigger {
  font-size: larger;
}
```

#### HTML

```
<p>
  This is the first sentence.
  <span class="bigger">This whole sentence is in bigger letters.</span>
</p>
```

#### Result

#### DOM interface

This element implements the `HTMLElement` interface.

#### See also

- CSS: `font-size`, `font`
- HTML: `<small>`, `<font>`, `<style>`

### <blockquote> HTML block quotation element

The **`<blockquote>`** HTML element indicates that the enclosed text is an extended quotation. Usually, this is rendered visually by indentation (see Notes for how to change it). A URL for the source of the quotation may be given using the `cite` attribute, while a text representation of the source can be given using the `<cite>` element.

#### Try it

```
<div>
  <blockquote cite="https://www.huxley.net/bnw/four.html">
    <p>
      Words can be like X-rays, if you use them properly—they'll go through
      anything. You read and you're pierced.
    </p>
  </blockquote>
  <p>—Aldous Huxley, <cite>Brave New World</cite></p>
</div>
```

```
div:has(> blockquote) {
  background-color: #ededed;
  margin: 10px auto;
  padding: 15px;
  border-radius: 5px;
}

blockquote p::before {
  content: "\201C";
}

blockquote p::after {
  content: "\201D";
}

blockquote + p {
  text-align: right;
}
```

#### Attributes

This element's attributes include the global attributes.

`cite`

> A URL that designates a source document or message for the information quoted. This attribute is intended to point to information explaining the context or the reference for the quote.

### Usage notes

According to the specification, attribution for the quotation, if any, must be placed outside the `<blockquote>` element.

To change the indentation applied to the quoted text, use the CSS `margin-left` and/or `margin-right` properties, or the `margin` shorthand property.

To include shorter quotes inline rather than in a separate block, use the `<q>` (Quotation) element.

### Examples

This example demonstrates the use of the `<blockquote>` element to quote a passage from RFC 1149, *A Standard for the Transmission of IP Datagrams on Avian Carriers*.

```
<blockquote cite="https://datatracker.ietf.org/doc/html/rfc1149">
  <p>
    Avian carriers can provide high delay, low throughput, and low altitude
    service. The connection topology is limited to a single point-to-point path
    for each carrier, used with standard carriers, but many carriers can be used
    without significant interference with each other, outside early spring. This
    is because of the 3D ether space available to the carriers, in contrast to
    the 1D ether used by IEEE802.3. The carriers have an intrinsic collision
    avoidance system, which increases availability.
  </p>
</blockquote>
```

### Result

### Technical summary

| | |
|---|---|
| **Content categories** | Flow content, sectioning root, palpable content. |
| **Permitted content** | Flow content. |
| **Tag omission** | None, both the starting and ending tag are mandatory. |
| **Permitted parents** | Any element that accepts flow content. |
| **Implicit ARIA role** | `blockquote` |
| **Permitted ARIA roles** | Any |
| **DOM interface** | `HTMLQuoteElement` |

### See also

- The `<q>` element for inline quotations.
- The `<cite>` element for source citations.
- The blockquote element via heydonworks.com (2024)

## <body> HTML document body element

The **`<body>`** HTML element represents the content of an HTML document. There can be only one `<body>` element in a document.

### Attributes

This element includes the global attributes, event attributes, and deprecated attributes:

### Event attributes

> **Note:** Each of the below event attribute names is linked to its equivalent `Window` interface event. You can listen to these events using `addEventListener()` instead of adding the `oneventname` attribute to the `<body>` element.

`onafterprint`

> Function to call after the user has printed the document.

`onbeforeprint`

> Function to call when the user requests printing of the document.

`onbeforeunload`

: Function to call when the document is about to be unloaded.

`onblur`

: Function to call when the document loses focus.

`onerror`

: Function to call when the document fails to load properly.

`onfocus`

: Function to call when the document receives focus.

`onhashchange`

: Function to call when the fragment identifier part (starting with the hash (`'#'`) character) of the document's current address has changed.

`onlanguagechange`

: Function to call when the preferred languages changed.

`onload`

: Function to call when the document has finished loading.

`onmessage`

: Function to call when the document has received a message.

`onmessageerror`

: Function to call when the document has received a message that cannot be deserialized.

`onoffline`

: Function to call when network communication has failed.

`ononline`

: Function to call when network communication has been restored.

`onpageswap`

: Function to call when you navigate across documents, when the previous document is about to unload.

`onpagehide`

: Function to call when the browser hides the current page in the process of presenting a different page from the session's history.

`onpagereveal`

: Function to call when a document is first rendered, either when loading a fresh document from the network or activating a document.

`onpageshow`

: Function to call when the browser displays the window's document due to navigation.

`onpopstate`

: Function to call when the user has navigated session history.

`onresize`

: Function to call when the document has been resized.

`onrejectionhandled`

: Function to call when a JavaScript `Promise` is handled late.

`onstorage`

: Function to call when the storage area has changed.

`onunhandledrejection`

Function to call when a JavaScript `Promise` that has no rejection handler is rejected.

`onunload`

Function to call when the document is going away.

### Deprecated attributes

> **Warning:** Do not use these deprecated attributes; opt for the CSS alternatives listed with each deprecated attribute instead.

`alink`

Color of text for hyperlinks when selected. Use the CSS `color` property in conjunction with the `:active` and `:focus` pseudo-classes instead.

`background`

URI of an image to use as a background. Use the CSS `background-image` property instead.

`bgcolor`

Background color for the document. Use the CSS `background-color` property instead.

`bottommargin`

Ignored.

`leftmargin`

The margin of the left and right of the body. Use the CSS `margin-left` and `margin-right` properties (or the logical `margin-inline` property) instead.

`link`

Color of text for unvisited hypertext links. Use the CSS `color` property in conjunction with the `:link` pseudo-class instead.

`rightmargin`

Ignored.

`text`

Foreground color of text. Use the CSS `color` property instead.

`topmargin`

The margin of the top and bottom of the body. Use the CSS `margin-top` and `margin-bottom` properties (or the logical `margin-block` property) instead.

`vlink`

Color of text for visited hypertext links. Use the CSS `color` property in conjunction with the `:visited` pseudo-class instead.

### Examples

```
<html lang="en">
  <head>
    <title>Document title</title>
  </head>
  <body>
    <p>
      The <code>&lt;body&gt;</code> HTML element represents the content of an
      HTML document. There can be only one <code>&lt;body&gt;</code> element in
      a document.
    </p>
  </body>
</html>
```

### Result

### Technical summary

| | |
|---|---|
| **Content categories** | None. |

| | |
|---|---|
| **Permitted content** | Flow content. |
| **Tag omission** | The start tag may be omitted if the element is empty, or if the first thing inside it is not ASCII whitespace or a comment, except if the first thing inside it is a `<meta>`, `<noscript>`, `<link>`, `<script>`, `<style>`, or `<template>` element. The end tag may be omitted if the `<body>` element is not immediately followed by a comment. |
| **Permitted parents** | It must be the second element of an `<html>` element. |
| **Implicit ARIA role** | `generic` |
| **Permitted ARIA roles** | No `role` permitted |
| **DOM interface** | `HTMLBodyElement`<br>• The `<body>` element exposes the `HTMLBodyElement` interface.<br>• You can access the `<body>` element through the `document.body` property. |

### See also

- `<html>`
- `<head>`
- Event handling overview

## <br> HTML line break element

The **`<br>`** HTML element produces a line break in text (carriage-return). It is useful for writing a poem or an address, where the division of lines is significant.

### Try it

```
<p>
  O'er all the hilltops<br />
  Is quiet now,<br />
  In all the treetops<br />
  Hearest thou<br />
  Hardly a breath;<br />
  The birds are asleep in the trees:<br />
  Wait, soon like these<br />
  Thou too shalt rest.
</p>
```

```
p {
  font-size: 1rem;
  font-family: sans-serif;
  margin: 20px;
}
```

As you can see from the above example, a `<br>` element is included at each point where we want the text to break. The text after the `<br>` begins again at the start of the next line of the text block.

> **Note:** Do not use `<br>` to create margins between paragraphs; wrap them in `<p>` elements and use the CSS `margin` property to control their size.

### Attributes

This element's attributes include the global attributes.

### Deprecated attributes

`clear`

Indicates where to begin the next line after the break.

### Styling with CSS

The `<br>` element has a single, well-defined purpose — to create a line break in a block of text. As such, it has no dimensions or visual output of its own, and there is very little you can do to style it.

You can set a `margin` on `<br>` elements themselves to increase the spacing between the lines of text in the block, but this is a bad practice — you should use the `line-height` property that was designed for that purpose.

### Accessibility

Creating separate paragraphs of text using `<br>` is not only bad practice, it is problematic for people who navigate with the aid of screen reading technology. Screen readers may announce the presence of the element, but not any content contained within `<br>`s. This can be a confusing and frustrating experience for the person using the screen reader.

Use `<p>` elements, and use CSS properties like `margin` to control their spacing.

### Examples

#### Basic br

In the following example we use `<br>` elements to create line breaks between the different lines of a postal address:

```
Mozilla<br />
331 E. Evelyn Avenue<br />
Mountain View, CA<br />
94041<br />
USA<br />
```

#### Result

#### Technical summary

| | |
|---|---|
| **Content categories** | Flow content, phrasing content. |
| **Permitted content** | None; it is a void element. |
| **Tag omission** | Must have a start tag and must not have an end tag. In XHTML documents, write this element as `<br />`. |
| **Permitted parents** | Any element that accepts phrasing content. |
| **Implicit ARIA role** | No corresponding role |
| **Permitted ARIA roles** | `none`, `presentation` |
| **DOM interface** | `HTMLBRElement` |

### See also

- `<address>` element
- `<p>` element
- `<wbr>` element

## <button> HTML button element

The **`<button>`** HTML element is an interactive element activated by a user with a mouse, keyboard, finger, voice command, or other assistive technology. Once activated, it then performs an action, such as submitting a form or opening a dialog.

By default, HTML buttons are presented in a style resembling the platform the user agent runs on, but you can change buttons' appearance with CSS.

### Try it

```
<button class="favorite styled" type="button">Add to favorites</button>
```

```
.styled {
  border: 0;
  line-height: 2.5;
  padding: 0 20px;
  font-size: 1rem;
  text-align: center;
  color: white;
  text-shadow: 1px 1px 1px black;
  border-radius: 10px;
  background-color: tomato;
  background-image: linear-gradient(
    to top left,
    rgb(0 0 0 / 0.2),
    rgb(0 0 0 / 0.2) 30%,
    transparent
  );
  box-shadow:
    inset 2px 2px 3px rgb(255 255 255 / 0.6),
    inset -2px -2px 3px rgb(0 0 0 / 0.6);
}

.styled:hover {
```

```
  background-color: red;
}

.styled:active {
  box-shadow:
    inset -2px -2px 3px rgb(255 255 255 / 0.6),
    inset 2px 2px 3px rgb(0 0 0 / 0.6);
}
```

### Attributes

This element's attributes include the global attributes.

`autofocus`

This Boolean attribute specifies that the button should have input focus when the page loads. **Only one element in a document can have this attribute.**

`command`

Specifies the action to be performed on an element being controlled by a control `<button>` specified via the `commandfor` attribute. The possible values are:

`"show-modal"`

The button will show a `<dialog>` as modal. If the dialog is already modal, no action will be taken. This is a declarative equivalent of calling the `HTMLDialogElement.showModal()` method on the `<dialog>` element.

`"close"`

The button will close a `<dialog>` element. If the dialog is already closed, no action will be taken. This is a declarative equivalent of calling the `HTMLDialogElement.close()` method on the `<dialog>` element. When used with the `value` attribute, the button's value will be passed as the dialog's `returnValue` property.

`"request-close"`

The button will trigger a `cancel` event on a `<dialog>` element to request that the browser dismiss it, followed by a `close` event. This differs from the `close` command in that authors can call `Event.preventDefault()` on the `cancel` event to prevent the `<dialog>` from closing. If the dialog is already closed, no action will be taken. This is a declarative equivalent of calling the `HTMLDialogElement.requestClose()` method on the `<dialog>` element. When used with the button's `value` attribute, the value will be passed as the dialog's `returnValue` property.

`"show-popover"`

The button will show a hidden popover. If you try to show an already showing popover, no action will be taken. See Popover API for more details. This is equivalent to setting a value of `show` for the `popovertargetaction` attribute, and also provides a declarative equivalent to calling the `HTMLElement.showPopover()` method on the popover element.

`"hide-popover"`

The button will hide a showing popover. If you try to hide an already hidden popover, no action will be taken. See Popover API for more details. This is equivalent to setting a value of `hide` for the `popovertargetaction` attribute, and also provides a declarative equivalent to calling the `HTMLElement.hidePopover()` method on the popover element.

`"toggle-popover"`

The button will toggle a popover between showing and hidden. If the popover is hidden, it will be shown; if the popover is showing, it will be hidden. See Popover API for more details. This is equivalent to setting a value of `toggle` for the `popovertargetaction` attribute, and also provides a declarative equivalent to calling the `HTMLElement.togglePopover()` method on the popover element.

Custom values

This attribute can represent custom values that are prefixed with a two hyphen characters (`--`). Buttons with a custom value will dispatch the `CommandEvent` on the controlled element.

`commandfor`

Turns a `<button>` element into a command button, controlling a given interactive element by issuing the command specified in the button's `command` attribute. The `commandfor` attribute takes the ID of the element to control as its value. This is a more general version of `popovertarget`.

`disabled`

This Boolean attribute prevents the user from interacting with the button: it cannot be pressed or focused.

`form`

The `<form>` element to associate the button with (its *form owner*). The value of this attribute must be the `id` of a `<form>` in the same document. (If this attribute is not set, the `<button>` is associated with its ancestor `<form>` element, if any.)

This attribute lets you associate `<button>` elements to `<form>`s anywhere in the document, not just inside a `<form>`. It can also override an ancestor `<form>` element.

`formaction`

The URL that processes the information submitted by the button. Overrides the `action` attribute of the button's form owner. Does nothing if there is no form owner.

`formenctype`

If the button is a submit button (it's inside/associated with a `<form>` and doesn't have `type="button"`), specifies how to encode the form data that is submitted. Possible values:

- `application/x-www-form-urlencoded`: The default if the attribute is not used.
- `multipart/form-data`: Used to submit `<input>` elements with their `type` attributes set to `file`.
- `text/plain`: Specified as a debugging aid; shouldn't be used for real form submission.

If this attribute is specified, it overrides the `enctype` attribute of the button's form owner.

`formmethod`

If the button is a submit button (it's inside/associated with a `<form>` and doesn't have `type="button"`), this attribute specifies the HTTP method used to submit the form. Possible values:

- `post`: The data from the form are included in the body of the HTTP request when sent to the server. Use when the form contains information that shouldn't be public, like login credentials.
- `get`: The form data are appended to the form's `action` URL, with a `?` as a separator, and the resulting URL is sent to the server. Use this method when the form has no side effects, like search forms.
- `dialog`: This method is used to indicate that the button closes the dialog with which it is associated, and does not transmit the form data at all.

If specified, this attribute overrides the `method` attribute of the button's form owner.

`formnovalidate`

If the button is a submit button, this Boolean attribute specifies that the form is not to be validated when it is submitted. If this attribute is specified, it overrides the `novalidate` attribute of the button's form owner.

This attribute is also available on `<input type="image">` and `<input type="submit">` elements.

`formtarget`

If the button is a submit button, this attribute is an author-defined name or standardized, underscore-prefixed keyword indicating where to display the response from submitting the form. This is the `name` of, or keyword for, a *browsing context* (a tab, window, or `<iframe>`). If this attribute is specified, it overrides the `target` attribute of the button's form owner. The following keywords have special meanings:

- `_self`: Load the response into the same browsing context as the current one. This is the default if the attribute is not specified.
- `_blank`: Load the response into a new unnamed browsing context — usually a new tab or window, depending on the user's browser settings.
- `_parent`: Load the response into the parent browsing context of the current one. If there is no parent, this option behaves the same way as `_self`.
- `_top`: Load the response into the top-level browsing context (that is, the browsing context that is an ancestor of the current one, and has no parent). If there is no parent, this option behaves the same way as `_self`.

`interestfor`

Defines the `<button>` element as an **interest invoker**. Its value is the `id` of a target element, which will be affected in some way (normally shown or hidden) when interest is shown or lost on the invoker element (for example, by hovering/unhovering or focusing/blurring it). See Using interest invokers for more details and examples.

`name`

The name of the button, submitted as a pair with the button's `value` as part of the form data, when that button is used to submit the form.

`popovertarget`

Turns a `<button>` element into a popover control button; takes the ID of the popover element to control as its value. Establishing a relationship between a popover and its invoker button using the `popovertarget` attribute has two additional useful effects:

- The browser creates an implicit `aria-details` and `aria-expanded` relationship between popover and invoker, and places the popover in a logical position in the keyboard focus navigation order when shown. This makes the popover more accessible to keyboard and assistive

technology (AT) users (see also Popover accessibility features).
- The browser creates an implicit anchor reference between the two, making it very convenient to position popovers relative to their controls using CSS anchor positioning. See Popover anchor positioning for more details.

`popovertargetaction`

Specifies the action to be performed on a popover element being controlled by a control `<button>`. Possible values are:

`"hide"`

The button will hide a shown popover. If you try to hide an already hidden popover, no action will be taken.

`"show"`

The button will show a hidden popover. If you try to show an already showing popover, no action will be taken.

`"toggle"`

The button will toggle a popover between showing and hidden. If the popover is hidden, it will be shown; if the popover is showing, it will be hidden. If `popovertargetaction` is omitted, `"toggle"` is the default action that will be performed by the control button.

`type`

The default behavior of the button. Possible values are:

- `submit`: The button submits the form data to the server. This is the default if the attribute is not specified for buttons associated with a `<form>`, or if the attribute is an empty or invalid value.
- `reset`: The button resets all the controls to their initial values, like `<input type="reset">`. (This behavior tends to annoy users.)
- `button`: The button has no default behavior, and does nothing when pressed by default. It can have client-side scripts listen to the element's events, which are triggered when the events occur.

`value`

Defines the value associated with the button's `name` when it's submitted with the form data. This value is passed to the server in params when the form is submitted using this button. When used with the `close` or `request-close` commands, the `value` attribute sets the `returnValue` of the `<dialog>` element being controlled.

### Notes

A submit button with the attribute `formaction` set, but without an associated form does nothing. You have to set a form owner, either by wrapping it in a `<form>` or set the attribute `form` to the id of the form.

`<button>` elements are much easier to style than `<input>` elements. You can add inner HTML content (think `<i>`, `<br>`, or even `<img>`), and use `::after` and `::before` pseudo-elements for complex rendering.

If your buttons are not for submitting form data to a server, be sure to set their `type` attribute to `button`. Otherwise, they will try to submit form data and to load the (nonexistent) response, possibly destroying the current state of the document.

While `<button type="button">` has no default behavior, event handlers can be scripted to trigger behaviors. An activated button can perform programmable actions using JavaScript, such as removing an item from a list.

By default, user agents style buttons as `display: flow-root`, which establishes a new block formatting context and centers the button's children both horizontally and vertically as long as they do not overflow. If the button is defined as a flex or grid container, the children will behave as flex or grid items. A button set to `display: inline` will be styled as if the value were set to `display: inline-block`.

### Accessibility

### Icon buttons

Buttons that only display an icon do not have an *accessible name*. Accessible names provide information for assistive technology, such as screen readers, to access when they parse the document and generate an accessibility tree. Assistive technology then uses the accessibility tree to navigate and manipulate page content.

To give an icon button an accessible name, put text in the `<button>` element that concisely describes the button's functionality.

### Examples

```
<button name="favorite">
  <svg fill="black" viewBox="0 0 42 42">
    <path
      d="M21,1c1.081,0,5.141,12.315,6.201,13.126s13.461,1.053,13.791,2.137 c0.34,1.087-9.561,8.938-9.961,10.252c-0.409,1.307,
      3.202,13.769,2.331,14.442c-0.879,0.673-11.05-6.79-12.361-6.79 c-1.311,0-11.481,7.463-12.36,6.79c-0.871-0.674,2.739-13.136,
      2.329-14.442c-0.399-1.313-10.3-9.165-9.96-10.252 c0.33-1.084,12.731-1.326,13.791-2.137S19.91,1,21,1z"></path>
  </svg>
  Add to favorites
</button>
```

### Result

If you want to visually hide the button's text, an accessible way to do so is to use a combination of CSS properties to remove it visually from the screen, but keep it parsable by assistive technology.

However, it is worth noting that leaving the button text visible can help people who may not be familiar with the icon's meaning or understand the button's purpose. This is especially important for people who are not technologically sophisticated or who may have different cultural interpretations of the icon the button uses.

- What is an accessible name? | Vispero
- MDN Understanding WCAG, Guideline 4.1 explanations
- Understanding Success Criterion 4.1.2 | W3C Understanding WCAG 2.0

### Size and Proximity

#### Size

Interactive elements such as buttons should have an area large enough to be easy to activate. This helps a variety of people, including people with motor control issues and people using non-precise forms of input such as a stylus or fingers. A minimum interactive size of 44×44 CSS pixels is recommended.

- Understanding Success Criterion 2.5.5: Target Size | W3C Understanding WCAG 2.1
- Target Size and 2.5.5 | Adrian Roselli
- Quick test: Large touch targets - The A11Y Project

#### Proximity

Large amounts of interactive content — including buttons — placed in close visual proximity to each other should have space separating them. This spacing is beneficial for people who are experiencing motor control issues, who may accidentally activate the wrong interactive content.

Spacing may be created using CSS properties such as `margin`.

- Hand tremors and the giant-button-problem - Axess Lab

### ARIA state information

To describe the state of a button the correct ARIA attribute to use is `aria-pressed` and not `aria-checked` or `aria-selected`. To find out more read the information about the ARIA button role.

### Button styles

It is best not to override the default focus ring for elements that have focus. If the button styles are overridden, it is important to **ensure that the focus state has enough contrast** so that people experiencing low vision conditions can perceive it and people with cognitive differences will understand it.

The `:focus-visible` pseudo-class can be used to apply styles to an element that has `:focus` only when the user agent's heuristics determine that the focus should be highlighted, such as when a `<button>` receives keyboard focus. See :focus vs :focus-visible for more information.

Color contrast ratio is determined by comparing the luminosity of the button text and background color values to the background the button is placed on. To meet current Web Content Accessibility Guidelines (WCAG), a ratio of 4.5:1 is required for text content and 3:1 for large text. (Large text is defined as 18.66px and `bold` or larger, or 24px or larger.)

- WebAIM: Color Contrast Checker
- MDN Understanding WCAG, Guideline 1.4 explanations
- Understanding Success Criterion 1.4.3 | W3C Understanding WCAG 2.0

### Clicking and focus

Whether clicking on a `<button>` or `<input>` button types causes it to (by default) become focused varies by browser and OS. Most browsers do give focus to a button being clicked, but Safari does not, by design.

### Examples

#### Creating a basic button

This example creates a clickable button. The `type="button"` attribute ensures the button has no default behavior. You can make this button interactive using JavaScript or attributes such as `command` and `commandfor`.

```
<button type="button" name="button">I'm a button</button>
```

#### Using the <code>request-close</code> value for the <code>command</code> attribute

The dialog in this example has two radio buttons that control whether or not the dialog can be closed. Select **Yes** or **No**, and then click **Request to Close** to try to close the dialog. If **Yes** is selected, the dialog closes; if **No** is selected, the dialog stays open and shows a message instead.

```
<button type="button" commandfor="mydialog" command="show-modal">
  Open Dialog
</button>
<dialog id="mydialog">
  <div class="wrapper">
    <form>
      <fieldset>
        <legend>Allow this dialog to close when requested?</legend>
        <div>
          <input type="radio" id="no" name="close" value="no" checked />
          <label for="no">No</label>
        </div>
        <div>
          <input type="radio" id="yes" name="close" value="yes" />
          <label for="yes">Yes</label>
        </div>
      </fieldset>
    </form>
    <button commandfor="mydialog" command="request-close">
      Request to Close
    </button>
    <p class="warning" hidden>You must choose "Yes" to close this dialog.</p>
  </div>
</dialog>
```

```
const dialog = document.querySelector("dialog");
const radio = document.querySelector("form").elements["close"];
const warning = document.querySelector(".warning");

dialog.addEventListener("cancel", (e) => {
  if (!e.cancelable) return;
  if (radio.value === "no") {
    warning.hidden = false;
    e.preventDefault();
  } else {
    warning.hidden = true;
  }
});
```

The **Open Dialog** button opens the `<dialog>` element using `command="show-modal"`.

The **Request to Close** button has `command="request-close"`, which targets the `<dialog>` element using the `commandfor="mydialog"` attribute. When it's clicked, it asks the `<dialog>` if it can be closed (unlike the `command="close"` attribute, which would close the `<dialog>` immediately). This checks if the `<dialog>` is `cancelable` using a `cancel` event.

When the event is `cancelable`, the value of the radio buttons is checked:

- If set to `yes`, the dialog is closed.
- If set to `no`, the `hidden` attribute is turned off on the warning and the `preventDefault()` method is called, which prevents the default `<dialog>` closing behavior.

### Using the <code>value</code> attribute with dialog <code>close</code> command

This example demonstrates how to use the button `value` attribute with `close` command to populate the value of a dialog's `returnValue` property.

When either the **Cancel** or **Delete** button is clicked, the dialog closes and sets its `returnValue` to the button's `value` attribute. The `close` event listener checks `dialog.returnValue` to determine which action the user chose and logs the result to the screen.

#### HTML

The HTML first defines a **Delete Record** button that uses the `commandfor` attribute to specify the dialog to be opened.

Within the dialog **Cancel** and **Delete** buttons use the `commandfor` attribute to indicate that they apply to the current dialog. They also set the `command` attribute to "close" and set the `value` attribute to "cancel" and "delete" respectively — the value of the selected button is automatically copied to the dialog `returnValue` when the button is clicked.

```
<button commandfor="confirm-dialog" command="show-modal">Delete Record</button>
<dialog id="confirm-dialog">
  <header>
    <h1>Delete Record?</h1>
  </header>
  <p>Are you sure? This action cannot be undone</p>
```

```
  <footer>
    <button commandfor="confirm-dialog" command="close" value="cancel">
      Cancel
    </button>
    <button commandfor="confirm-dialog" command="close" value="delete">
      Delete
    </button>
  </footer>
</dialog>
```

```
<pre id="log"></pre>
```

### JavaScript

The code uses a `close` event listener to log the dialog's `returnValue`.

```
const dialog = document.getElementById("confirm-dialog");

dialog.addEventListener("close", () => {
  switch (dialog.returnValue) {
    case "cancel":
      log("Cancel was clicked");
      break;
    case "delete":
      log("Delete was clicked");
      break;
    default:
      log("Closed with value:", dialog.returnValue);
  }
});
```

### Results

### Technical summary

| | |
|---|---|
| **Content categories** | Flow content, phrasing content, Interactive content, listed, labelable, and submittable form-associated element, palpable content. |
| **Permitted content** | Phrasing content but there must be no Interactive content. If the `<button>` is the first child of a customizable select element, then it may also contain zero or one `<selectedcontent>` element. |
| **Tag omission** | None, both the starting and ending tag are mandatory. |
| **Permitted parents** | Any element that accepts phrasing content. |
| **Implicit ARIA role** | `button` |
| **Permitted ARIA roles** | `checkbox`, `combobox`, `link`, `menuitem`, `menuitemcheckbox`, `menuitemradio`, `option`, `radio`, `switch`, `tab` |
| **DOM interface** | `HTMLButtonElement` |

## <canvas> HTML graphics canvas element

Use the **HTML `<canvas>` element** with either the canvas scripting API or the WebGL API to draw graphics and animations.

### Attributes

This element's attributes include the global attributes.

`height`

The height of the coordinate space in CSS pixels. Defaults to 150.

`moz-opaque`

Lets the canvas know whether translucency will be a factor. If the canvas knows there's no translucency, painting performance can be optimized. This is only supported by Mozilla-based browsers; use the standardized `canvas.getContext('2d', { alpha: false })` instead.

`width`

The width of the coordinate space in CSS pixels. Defaults to 300.

### Usage notes

### Alternative content

You should provide alternate content inside the `<canvas>` block. That content will be rendered both on older browsers that don't support canvas and in browsers with JavaScript disabled.

### Closing <code></canvas></code> tag

Unlike the `<img>` element, the `<canvas>` element **requires** the closing tag (`</canvas>`).

### Sizing the canvas using CSS versus HTML

The displayed size of the canvas can be changed using CSS, but if you do this the image is scaled during rendering to fit the styled size, which can make the final graphics rendering end up being distorted.

It is better to specify your canvas dimensions by setting the `width` and `height` attributes directly on the `<canvas>` elements, either directly in the HTML or by using JavaScript.

### Maximum canvas size

The exact maximum size of a `<canvas>` element depends on the browser and environment. While in most cases the maximum dimensions exceed 10,000 x 10,000 pixels, notably iOS devices limit the canvas size to only 4,096 x 4,096 pixels. See canvas size limits in different browsers and devices.

> **Note:** Exceeding the maximum dimensions or area renders the canvas unusable — drawing commands will not work.

### Using an offscreen canvas

A canvas can be rendered using the `OffscreenCanvas` API where the document and canvas are decoupled. The benefit is that a worker thread can handle canvas rendering and the main thread of your web application is not blocked by canvas operations. By parallelizing work, other UI elements of your web application will remain responsive even if you are running complex graphics on an offscreen canvas. For more information, see the `OffscreenCanvas` API documentation.

### Accessibility

### Alternative content

The `<canvas>` element on its own is just a bitmap and does not provide information about any drawn objects. Canvas content is not exposed to accessibility tools as semantic HTML is. In general, you should avoid using canvas in an accessible website or app. The following guides can help to make it more accessible.

- Canvas accessibility use cases
- Canvas element accessibility issues
- Best practices for interactive canvas elements

### Examples

### HTML

This code snippet adds a canvas element to your HTML document. A fallback text is provided if a browser is unable to read or render the canvas.

```
<canvas width="120" height="120">
  An alternative text describing what your canvas displays.
</canvas>
```

### JavaScript

Then in the JavaScript code, call `HTMLCanvasElement.getContext()` to get a drawing context and start drawing onto the canvas:

```
const canvas = document.querySelector("canvas");
const ctx = canvas.getContext("2d");
ctx.fillStyle = "green";
// Add a rectangle at (10, 10) with size 100x100 pixels
ctx.fillRect(10, 10, 100, 100);
```

### Result

### Technical summary

| | |
|---|---|
| **Content categories** | Flow content, phrasing content, embedded content, palpable content. |

| | |
|---|---|
| **Permitted content** | Transparent but with no interactive content descendants except for `<a>` elements, `<button>` elements, `<input>` elements whose `type` attribute is `checkbox`, `radio`, or `button`. |
| **Tag omission** | None, both the starting and ending tag are mandatory. |
| **Permitted parents** | Any element that accepts phrasing content. |
| **Implicit ARIA role** | No corresponding role |
| **Permitted ARIA roles** | Any |
| **DOM interface** | `HTMLCanvasElement` |

See also

- Canvas API
- Canvas tutorial
- OffscreenCanvas
- Canvas cheat sheet (2009)
- Canvas cheat sheet (pdf) (2015)
- Safari HTML canvas guide via Apple (2013)
- `CanvasRenderingContext2D` 2D drawing context for a canvas element via Apple.com
- WebGL API
- `<img>`
- SVG
- HTML video and audio

## <caption> HTML table caption element

The **`<caption>`** HTML element specifies the caption (or title) of a table, providing the table an accessible name or accessible description.

Try it

```
<table>
  <caption>
    He-Man and Skeletor facts
  </caption>
  <tbody>
    <tr>
      <td></td>
      <th scope="col" class="heman">He-Man</th>
      <th scope="col" class="skeletor">Skeletor</th>
    </tr>
    <tr>
      <th scope="row">Role</th>
      <td>Hero</td>
      <td>Villain</td>
    </tr>
    <tr>
      <th scope="row">Weapon</th>
      <td>Power Sword</td>
      <td>Havoc Staff</td>
    </tr>
    <tr>
      <th scope="row">Dark secret</th>
      <td>Expert florist</td>
      <td>Cries at romcoms</td>
    </tr>
  </tbody>
</table>
```

```
caption {
  caption-side: bottom;
  padding: 10px;
  font-weight: bold;
}

table {
  border-collapse: collapse;
  border: 2px solid rgb(140 140 140);
  font-family: sans-serif;
  font-size: 0.8rem;
  letter-spacing: 1px;
```

```
}

th,
td {
  border: 1px solid rgb(160 160 160);
  padding: 8px 10px;
}

th {
  background-color: rgb(230 230 230);
}

td {
  text-align: center;
}

tr:nth-child(even) td {
  background-color: rgb(250 250 250);
}

tr:nth-child(odd) td {
  background-color: rgb(240 240 240);
}

.heman {
  font:
    1.4rem "molot",
    sans-serif;
  text-shadow:
    1px 1px 1px white,
    2px 2px 1px black;
}

.skeletor {
  font:
    1.7rem "rapscallion",
    fantasy;
  letter-spacing: 3px;
  text-shadow:
    1px 1px 0 white,
    0 0 9px black;
}
```

### Attributes

This element includes the global attributes.

### Deprecated attributes

The following attributes are deprecated and should not be used. They are documented below for reference when updating existing code and for historical interest only.

`align`

> Specifies on which side of the table the caption should be displayed. The possible enumerated values are `left`, `top`, `right`, or `bottom`. Use the `caption-side` and `text-align` CSS properties instead, as this attribute is deprecated.

### Usage notes

- If included, the `<caption>` element must be the first child of its parent `<table>` element.
- When a `<table>` is nested within a `<figure>` as the figure's only content, it should be captioned via a `<figcaption>` for the `<figure>` instead of as a `<caption>` nested within the `<table>`.
- Any `background-color` applied to a table will not be applied to its caption. Add a `background-color` to the `<caption>` element as well if you want the same color to be behind both.

### Example

See `<table>` for a complete table example introducing common standards and best practices.

### Table with caption

This example demonstrates a basic table that includes a caption describing the data presented.

Such a "title" is helpful for users who are quickly scanning the page, and it is especially beneficial for visually impaired users, allowing them to determine the table's relevance quickly without the need to have a screen reader read the contents of many cells just to find out what the table is about.

### HTML

A `<caption>` element is used as the first child of the `<table>`, with text content similar to a title to describe the table data. Three rows, the first being a header row, with two columns are created using the `<tr>`, `<th>` and `<td>` elements after the `<caption>`.

```
<table>
  <caption>
    User login email addresses
  </caption>
  <thead>
    <tr>
      <th>Login</th>
      <th>Email</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>user1</td>
      <td>user1@example.com</td>
    </tr>
    <tr>
      <td>user2</td>
      <td>user2@example.com</td>
    </tr>
  </tbody>
</table>
```

### CSS

Some basic CSS is used to align and highlight the `<caption>`.

```
caption {
  caption-side: top;
  text-align: left;
  padding-bottom: 10px;
  font-weight: bold;
}
```

### Result

### Technical summary

| | |
|---|---|
| **Content categories** | None. |
| **Permitted content** | Flow content. |
| **Tag omission** | The end tag can be omitted if the element is not immediately followed by ASCII whitespace or a comment. |
| **Permitted parents** | A `<table>` element, as its first descendant. |
| **Implicit ARIA role** | `caption` |
| **Permitted ARIA roles** | No `role` permitted |
| **DOM interface** | `HTMLTableCaptionElement` |

### See also

- Learn: HTML table basics
- `<col>`, `<colgroup>`, `<table>`, `<tbody>`, `<td>`, `<tfoot>`, `<th>`, `<thead>`, `<tr>`: Other table-related elements
- `caption-side`: CSS property to position the `<caption>` relative to its parent `<table>`
- `text-align`: CSS property to horizontally align the text content of the `<caption>`

##  HTML centered text element

The **`<center>`** HTML element is a block-level element that displays its block-level or inline contents centered horizontally within its containing element. The container is usually, but isn't required to be, `<body>`.

This tag has been deprecated in HTML 4 (and XHTML 1) in favor of the CSS `text-align` property, which can be applied to the `<div>` element or to an individual `<p>`. For centering blocks, use other CSS properties like `margin-left` and `margin-right` and set them to `auto` (or set `margin` to `0 auto`).

### DOM interface

This element implements the `HTMLElement` interface.

### Example 1

```
<center>
  This text will be centered.
  <p>So will this paragraph.</p>
</center>
```

### Result

### Example 2 (CSS alternative)

```
<div class="center">
  This text will be centered.
  <p>So will this paragraph.</p>
</div>
```

```
.center {
  text-align: center;
}
```

### Result

### Example 3 (CSS alternative)

```
<p class="center">
  This line will be centered.<br />
  And so will this line.
</p>
```

```
.center {
  text-align: center;
}
```

### Result

> **Note:** Applying `text-align: center` to a `<div>` or `<p>` element centers the *contents* of those elements while leaving their overall dimensions unchanged.

### See also

- `text-align`
- `display`

## <cite> HTML citation element

The **`<cite>`** HTML element is used to mark up the title of a creative work. The reference may be in an abbreviated form according to context-appropriate conventions related to citation metadata.

### Try it

```
<figure>
  <blockquote>
    <p>
      It was a bright cold day in April, and the clocks were striking thirteen.
    </p>
  </blockquote>
  <figcaption>
    First sentence in
    <cite
      ><a href="http://www.george-orwell.org/1984/0.html"
        >Nineteen Eighty-Four</a
      ></cite
    >
    by George Orwell (Part 1, Chapter 1).
  </figcaption>
</figure>
```

```
cite {
  /* Add your styles here */
}
```

### Attributes

This element only includes the global attributes.

### Usage notes

In the context of the `<cite>` element, a creative work could be, for example, one of the following:

- A book
- A research paper
- An essay
- A poem
- A musical score
- A song
- A play or film script
- A film
- A television show
- A game
- A sculpture
- A painting
- A theatrical production
- A play
- An opera
- A musical
- An exhibition
- A legal case report
- A computer program
- A website
- A web page
- A blog post or comment
- A forum post or comment
- A tweet
- A Facebook post
- A written or oral statement
- And so forth.

To include a reference to the source of quoted material which is contained within a `<blockquote>` or `<q>` element, use the `cite` attribute on the element.

Typically, browsers style the contents of a `<cite>` element in italics by default. To avoid this, apply the CSS `font-style` property to the `<cite>` element.

### Examples

```
<p>More information can be found in <cite>[ISO-0000]</cite>.</p>
```

### Result

### Technical summary

| | |
|---|---|
| **Content categories** | Flow content, phrasing content, palpable content. |
| **Permitted content** | Phrasing content. |
| **Tag omission** | None, both the starting and ending tag are mandatory. |
| **Permitted parents** | Any element that accepts phrasing content. |
| **Implicit ARIA role** | No corresponding role |
| **Permitted ARIA roles** | Any |
| **DOM interface** | `HTMLElement` Up to Gecko 1.9.2 (Firefox 4) inclusive, Firefox implements the `HTMLSpanElement` interface for this element. |

### See also

- The element `<blockquote>` for long quotations.
- The element `<q>` for inline quotations and the `cite` attribute.

## <code> HTML inline code element

The **`<code>`** HTML element displays its contents styled in a fashion intended to indicate that the text is a short fragment of computer code. By default, the content text is displayed using the user agent's default monospace font.

### Try it

```
<p>
  The <code>push()</code> method adds one or more elements to the end of an
  array and returns the new length of the array.
</p>
```

```
code {
  background-color: #eeeeee;
  border-radius: 3px;
  font-family: "Courier New", monospace;
  padding: 0 3px;
}
```

### Attributes

This element only includes the global attributes.

### Examples

A paragraph of text that includes `<code>`:

```
<p>
  The function <code>selectAll()</code> highlights all the text in the input
  field so the user can, for example, copy or delete the text.
</p>
```

### Result

### Notes

To represent multiple lines of code, wrap the `<code>` element within a `<pre>` element. The `<code>` element by itself only represents a single phrase of code or line of code.

A CSS rule can be defined for the `code` selector to override the browser's default font face. Preferences set by the user might take precedence over the specified CSS.

### Technical summary

| | |
|---|---|
| **Content categories** | Flow content, phrasing content, palpable content. |
| **Permitted content** | Phrasing content. |
| **Tag omission** | None, both the starting and ending tag are mandatory. |
| **Permitted parents** | Any element that accepts phrasing content. |
| **Implicit ARIA role** | `code` |
| **Permitted ARIA roles** | Any |
| **DOM interface** | `HTMLElement` Up to Gecko 1.9.2 (Firefox 4) inclusive, Firefox implements the `HTMLSpanElement` interface for this element. |

### See also

- `<samp>`
- `<kbd>`
- `<var>`
- `<pre>`

## <col> HTML table column element

The **`<col>`** HTML element defines one or more columns in a column group represented by its parent `<colgroup>` element. The `<col>` element is only valid as a child of a `<colgroup>` element that has no `span` attribute defined.

### Try it

```
<table>
  <caption>
    Superheros and sidekicks
  </caption>
  <colgroup>
    <col />
    <col span="2" class="batman" />
```

```
    <col span="2" class="flash" />
  </colgroup>
  <thead>
    <tr>
      <td></td>
      <th scope="col">Batman</th>
      <th scope="col">Robin</th>
      <th scope="col">The Flash</th>
      <th scope="col">Kid Flash</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <th scope="row">Skill</th>
      <td>Smarts, strong</td>
      <td>Dex, acrobat</td>
      <td>Super speed</td>
      <td>Super speed</td>
    </tr>
  </tbody>
</table>
```

```
.batman {
  background-color: #d7d9f2;
}

.flash {
  background-color: #ffe8d4;
}

table {
  border-collapse: collapse;
  border: 2px solid rgb(140 140 140);
  font-family: sans-serif;
  font-size: 0.8rem;
  letter-spacing: 1px;
}

caption {
  caption-side: bottom;
  padding: 10px;
}

th,
td {
  border: 1px solid rgb(160 160 160);
  padding: 8px 6px;
}

td {
  text-align: center;
}
```

### Attributes

This element includes the global attributes.

`span`

> Specifies the number of consecutive columns the `<col>` element spans. The value must be a positive integer greater than zero. If not present, its default value is `1`.

### Deprecated attributes

The following attributes are deprecated and should not be used. They are documented below for reference when updating existing code and for historical interest only.

`align`

> Specifies the horizontal alignment of each column cell. The possible enumerated values are `left`, `center`, `right`, `justify`, and `char`. When supported, the `char` value aligns the textual content on the character defined in the `char` attribute and on offset defined by the `charoff` attribute. Note that this attribute overrides the specified `align` of its `<colgroup>` parent element. Use the `text-align` CSS property on the `<td>` and `<th>` elements instead, as this attribute is deprecated.
>
> > **Note:** Setting `text-align` on the `<col>` element has no effect as `<col>` has no descendants, and therefore no elements inherit from it.

> If the table does not use a `colspan` attribute, use the `td:nth-of-type(an+b)` CSS selector. Set `a` to zero and `b` to the position of the column in the table, e.g., `td:nth-of-type(2) { text-align: right; }` to right-align the second column cells.
>
> If the table does use a `colspan` attribute, the effect can be achieved by combining adequate CSS attribute selectors like `[colspan=n]`, though this is not trivial.

`bgcolor`

Defines the background color of each column cell. The value is an HTML color; either a 6-digit hexadecimal RGB code, prefixed by a `#`, or a color keyword. Other CSS `<color>` values are not supported. Use the `background-color` CSS property instead, as this attribute is deprecated.

`char`

Does nothing. It was originally intended to specify the alignment of the content to a character of each column cell. Typical values for this include a period (`.`) when attempting to align numbers or monetary values. If `align` is not set to `char`, this attribute is ignored, though it will still override the specified `char` of its `<colgroup>` parent element.

`charoff`

Does nothing. It was originally intended to specify the number of characters to offset the column cell content from the alignment character specified by the `char` attribute.

`valign`

Specifies the vertical alignment of each column cell. The possible enumerated values are `baseline`, `bottom`, `middle`, and `top`. Note that this attribute overrides the specified `valign` of its `<colgroup>` parent element. Use the `vertical-align` CSS property on the `<td>` and `<th>` elements instead, as this attribute is deprecated.

> **Note:** Setting `vertical-align` on the `<col>` element has no effect as `<col>` has no descendants, and therefore no elements inherit from it.
>
> If the table does not use a `colspan` attribute, use the `td:nth-of-type(an+b)` CSS selector. Set `a` to zero and `b` to the position of the column in the table, e.g., `td:nth-of-type(2) { vertical-align: middle; }` to center the second column cells vertically.
>
> If the table does use a `colspan` attribute, the effect can be achieved by combining adequate CSS attribute selectors like `[colspan=n]`, though this is not trivial.

`width`

Specifies a default width for each column. In addition to the standard pixel and percentage values, this attribute can take the special form `0*`, which means that the width of each column spanned should be the minimum width necessary to hold the column's contents. Relative widths such as `5*` can also be used. Note that this attribute overrides the specified `width` of its `<colgroup>` parent element. Use the `width` CSS property instead, as this attribute is deprecated.

### Usage notes

- The `<col>` element is used within a `<colgroup>` element that doesn't have a `span` attribute.
- The `<col>` elements do not structurally group columns together. This is the role of the `<colgroup>` element.
- Only a limited number of CSS properties affect `<col>`:
  - `background` : The various `background` properties will set the background for cells within the column. As the column background color is painted on top of the table and column groups (`<colgroup>`), but behind background colors applied to the row groups (`<thead>`, `<tbody>`, and `<tfoot>`), the rows (`<tr>`), and the individual cells (`<th>` and `<td>`), backgrounds applied to table columns are only visible if every layer painted on top of them has a transparent background.
  - `border`: The various `border` properties apply, but only if the `<table>` has `border-collapse: collapse` set.
  - `visibility`: The value `collapse` for a column results in all cells of that column not being rendered, and cells spanning into other columns being clipped. The space these columns would have occupied is removed. However, the size of other columns is still calculated as though the cells in the collapsed column(s) are present. Other values for `visibility` have no effect.
  - `width`: The `width` property defines a minimum width for the column, as if `min-width` were set.

### Example

See `<table>` for a complete table example introducing common standards and best practices.

This example demonstrates an eight-column table divided into three `<col>` elements.

#### HTML

A `<colgroup>` element provides structures to a basic table, creating a single implicit column group. Three `<col>` elements are included within the `<colgroup>`, creating three stylable columns. The `span` attribute specifies the number of table columns each `<col>` should span (defaulting to `1` when omitted), enabling attributes to be shared across the columns in each `<col>`.

```
<table>
  <caption>
    Personal weekly activities
  </caption>
  <colgroup>
    <col />
    <col span="5" class="weekdays" />
    <col span="2" class="weekend" />
  </colgroup>
  <thead>
    <tr>
      <th>Period</th>
      <th>Mon</th>
      <th>Tue</th>
      <th>Wed</th>
      <th>Thu</th>
      <th>Fri</th>
      <th>Sat</th>
      <th>Sun</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <th>a.m.</th>
      <td>Clean room</td>
      <td>Football training</td>
      <td>Dance Course</td>
      <td>History Class</td>
      <td>Buy drinks</td>
      <td>Study hour</td>
      <td>Free time</td>
    </tr>
    <tr>
      <th>p.m.</th>
      <td>Yoga</td>
      <td>Chess Club</td>
      <td>Meet friends</td>
      <td>Gymnastics</td>
      <td>Birthday party</td>
      <td>Fishing trip</td>
      <td>Free time</td>
    </tr>
  </tbody>
</table>
```

### CSS

We use CSS, instead of deprecated HTML attributes, to provide a background color to the columns and align the cell content:

```
table {
  border-collapse: collapse;
  border: 2px solid rgb(140 140 140);
}

caption {
  caption-side: bottom;
  padding: 10px;
}

th,
td {
  border: 1px solid rgb(160 160 160);
  padding: 8px 6px;
  text-align: center;
}

.weekdays {
  background-color: #d7d9f2;
}

.weekend {
  background-color: #ffe8d4;
}
```

### Result

### Technical summary

| | |
|---|---|
| **Content categories** | None. |
| **Permitted content** | None; it is a void element. |
| **Tag omission** | Must have a start tag and must not have an end tag. |
| **Permitted parents** | `<colgroup>` only, though it can be implicitly defined as its start tag is not mandatory. The `<colgroup>` must not have a `span` attribute. |
| **Implicit ARIA role** | No corresponding role |
| **Permitted ARIA roles** | No `role` permitted |
| **DOM interface** | `HTMLTableColElement` |

### See also

- `<caption>`, `<colgroup>`, `<table>`, `<tbody>`, `<td>`, `<tfoot>`, `<th>`, `<thead>`, `<tr>`: Other table-related elements
- `background-color`: CSS property to set the background color of each column cell
- `border`: CSS property to control borders of column cells
- `text-align`: CSS property to horizontally align each column cell content
- `vertical-align`: CSS property to vertically align each column cell content
- `visibility`: CSS property to hide cells of a column
- `width`: CSS property to control the default width for each column
- `:nth-of-type`, `:first-of-type`, `:last-of-type`: CSS pseudo-classes to select the desired column cells

## <colgroup> HTML table column group element

The **`<colgroup>`** HTML element defines a group of columns within a table.

### Try it

```
<table>
  <caption>
    Superheros and sidekicks
  </caption>
  <colgroup>
    <col />
    <col span="2" class="batman" />
    <col span="2" class="flash" />
  </colgroup>
  <thead>
    <tr>
      <td></td>
      <th scope="col">Batman</th>
      <th scope="col">Robin</th>
      <th scope="col">The Flash</th>
      <th scope="col">Kid Flash</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <th scope="row">Skill</th>
      <td>Smarts, strong</td>
      <td>Dex, acrobat</td>
      <td>Super speed</td>
      <td>Super speed</td>
    </tr>
  </tbody>
</table>
```

```
.batman {
  background-color: #d7d9f2;
}

.flash {
  background-color: #ffe8d4;
}

table {
  border-collapse: collapse;
  border: 2px solid rgb(140 140 140);
  font-family: sans-serif;
  font-size: 0.8rem;
```

```
  letter-spacing: 1px;
}

caption {
  caption-side: bottom;
  padding: 10px;
}

th,
td {
  border: 1px solid rgb(160 160 160);
  padding: 8px 6px;
}

td {
  text-align: center;
}
```

### Attributes

This element includes the global attributes.

`span`

Specifies the number of consecutive columns the `<colgroup>` element spans. The value must be a positive integer greater than zero. If not present, its default value is `1`.

> **Note:** The `span` attribute is not permitted if there are one or more `<col>` elements within the `<colgroup>`.

### Deprecated attributes

The following attributes are deprecated and should not be used. They are documented below for reference when updating existing code and for historical interest only.

`align`

Specifies the horizontal alignment of each column group cell. The possible enumerated values are `left`, `center`, `right`, `justify`, and `char`. When supported, the `char` value aligns the textual content on the character defined in the `char` attribute and the offset defined by the `charoff` attribute. Note that the descendant `<col>` elements may override this value using their own `align` attribute. Use the `text-align` CSS property on the `<td>` and `<th>` elements instead, as this attribute is deprecated.

> **Note:** Setting `text-align` on the `<colgroup>` element has no effect as `<td>` and `<th>` elements are not descendants of the `<colgroup>` element, and therefore they do not inherit from it.
>
> If the table does not use a `colspan` attribute, use the `td:nth-of-type(an+b)` CSS selector per column, where `a` is the total number of the columns in the table and `b` is the ordinal position of the column in the table, e.g., `td:nth-of-type(7n+2) { text-align: right; }` to right-align the second column cells.
>
> If the table does use a `colspan` attribute, the effect can be achieved by combining adequate CSS attribute selectors like `[colspan=n]`, though this is not trivial.

`bgcolor`

Defines the background color of each column group cell. The value is an HTML color; either a 6-digit hexadecimal RGB code, prefixed by a `#`, or a color keyword. Other CSS `<color>` values are not supported. Use the `background-color` CSS property instead, as this attribute is deprecated.

`char`

Does nothing. It was originally intended to specify the alignment of the content to a character of each column group cell. Typical values for this include a period (`.`) when attempting to align numbers or monetary values. If `align` is not set to `char`, this attribute is ignored, though it will still be used as the default value for the `align` of the `<col>` elements which are members of this column group.

`charoff`

Does nothing. It was originally intended to specify the number of characters to offset the column group cell content from the alignment character specified by the `char` attribute.

`valign`

Specifies the vertical alignment of each column group cell. The possible enumerated values are `baseline`, `bottom`, `middle`, and `top`. Note that the descendant `<col>` elements may override this value using their own `valign` attribute. Use the `vertical-align` CSS property on the `<td>` and `<th>` elements instead, as this attribute is deprecated.

> **Note:** Setting `vertical-align` on the `<colgroup>` element has no effect as `<td>` and `<th>` elements are not descendants of the `<colgroup>` element, and therefore they do not inherit from it.
>
> If the table does not use a `colspan` attribute, use the `td:nth-of-type()` CSS selector per column, e.g., `td:nth-of-type(2) { vertical-align: middle; }` to center the second column cells vertically.
>
> If the table does use a `colspan` attribute, the effect can be achieved by combining adequate CSS attribute selectors like `[colspan=n]`, though this is not trivial.

`width`

Specifies a default width for each column in the current column group. In addition to the standard pixel and percentage values, this attribute can take the special form `0*`, which means that the width of each column spanned should be the minimum width necessary to hold the column's contents. Relative widths such as `5*` can also be used. Note that the descendant `<col>` elements may override this value using their own `width` attribute. Use the `width` CSS property instead, as this attribute is deprecated.

### Usage notes

- The `<colgroup>` should appear within a `<table>`, after any `<caption>` element (if used), but before any `<thead>`, `<tbody>`, `<tfoot>`, and `<tr>` elements.
- Only a limited number of CSS properties affect `<colgroup>`:
  - `background` : The various `background` properties will set the background for cells within the column group. As the column group background color is painted on top of the table, but behind background colors applied to the columns (`<col>`), the row groups (`<thead>`, `<tbody>`, and `<tfoot>`), the rows (`<tr>`), and the individual cells (`<th>` and `<td>`), backgrounds applied to table column groups are only visible if every layer painted on top of them has a transparent background.
  - `border`: The various `border` properties apply, but only if the `<table>` has `border-collapse: collapse` set.
  - `visibility`: The value `collapse` for a column group results in all cells of the columns in that column group not being rendered, and cells spanning into other columns being clipped. The space these columns in the column group would have occupied is removed. However, the size of other columns is still calculated as though the cells in the collapsed column(s) in the column group are present. Other values for `visibility` have no effect.
  - `width`: The `width` property defines a minimum width for the columns within the column group, as if `min-width` were set.

### Example

See `<table>` for a complete table example introducing common standards and best practices.

This example demonstrates a seven-column table divided into two `<colgroup>` elements that span multiple columns.

### HTML

Two `<colgroup>` elements are used to structure a basic table by creating column groups. The number of columns in each column group is specified by the `span` attribute.

```
<table>
  <caption>
    Personal weekly activities
  </caption>
  <colgroup span="5" class="weekdays"></colgroup>
  <colgroup span="2" class="weekend"></colgroup>
  <thead>
    <tr>
      <th>Mon</th>
      <th>Tue</th>
      <th>Wed</th>
      <th>Thu</th>
      <th>Fri</th>
      <th>Sat</th>
      <th>Sun</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>Clean room</td>
      <td>Football training</td>
      <td>Dance Course</td>
      <td>History Class</td>
      <td>Buy drinks</td>
      <td>Study hour</td>
```

```
      <td>Free time</td>
    </tr>
    <tr>
      <td>Yoga</td>
      <td>Chess Club</td>
      <td>Meet friends</td>
      <td>Gymnastics</td>
      <td>Birthday party</td>
      <td>Fishing trip</td>
      <td>Free time</td>
    </tr>
  </tbody>
</table>
```

### CSS

Grouped columns can be used to visually highlight the structure using CSS:

```
table {
  border-collapse: collapse;
  border: 2px solid rgb(140 140 140);
}

caption {
  caption-side: bottom;
  padding: 10px;
}

th,
td {
  border: 1px solid rgb(160 160 160);
  padding: 8px 6px;
  text-align: center;
}

.weekdays {
  background-color: #d7d9f2;
}

.weekend {
  background-color: #ffe8d4;
}
```

### Result

### Technical summary

| | |
|---|---|
| **Content categories** | None. |
| **Permitted content** | If the `span` attribute is present: none.<br>If the attribute is not present: zero or more `<col>` element |
| **Tag omission** | The start tag may be omitted, if it has a `<col>` element as its first child and if it is not preceded by a `<colgroup>` whose end tag has been omitted.<br>The end tag may be omitted, if it is not followed by a space or a comment. |
| **Permitted parents** | A `<table>` element. The `<colgroup>` must appear after any `<caption>` element, but before any `<thead>`, `<tbody>`, `<tfoot>`, and `<tr>` elements. |
| **Implicit ARIA role** | No corresponding role |
| **Permitted ARIA roles** | No `role` permitted |
| **DOM interface** | `HTMLTableColElement` |

### See also

- Learn: HTML table basics
- `<caption>`, `<col>`, `<table>`, `<tbody>`, `<td>`, `<tfoot>`, `<th>`, `<thead>`, `<tr>`: Other table-related elements
- `background-color`: CSS property to set the background color of each column group cell
- `border`: CSS property to control borders of column group cells
- `text-align`: CSS property to horizontally align each column group cell content
- `vertical-align`: CSS property to vertically align each column group cell content
- `visibility`: CSS property to hide (or show) cells of a column group

- `width`: CSS property to control the default width for each column in a column group
- `:nth-of-type`, `:first-of-type`, `:last-of-type`: CSS pseudo-classes to select the desired column cells

## <data> HTML data element

The **`<data>`** HTML element links a given piece of content with a machine-readable translation. If the content is time- or date-related, the `<time>` element must be used.

### Try it

```
<p>New Products:</p>
<ul>
  <li><data value="398">Mini Ketchup</data></li>
  <li><data value="399">Jumbo Ketchup</data></li>
  <li><data value="400">Mega Jumbo Ketchup</data></li>
</ul>
```

```
data:hover::after {
  content: " (ID " attr(value) ")";
  font-size: 0.7em;
}
```

### Attributes

This element's attributes include the global attributes.

`value`

This attribute specifies the machine-readable translation of the content of the element.

### Examples

The following example displays product names but also associates each name with a product number.

```
<p>New Products</p>
<ul>
  <li><data value="398">Mini Ketchup</data></li>
  <li><data value="399">Jumbo Ketchup</data></li>
  <li><data value="400">Mega Jumbo Ketchup</data></li>
</ul>
```

### Result

### Technical summary

| | |
|---|---|
| **Content categories** | Flow content, phrasing content, palpable content. |
| **Permitted content** | Phrasing content. |
| **Tag omission** | None, both the starting and ending tag are mandatory. |
| **Permitted parents** | Any element that accepts phrasing content. |
| **Implicit ARIA role** | `generic` |
| **Permitted ARIA roles** | Any |
| **DOM interface** | `HTMLDataElement` |

### See also

- The HTML `<time>` element.

## <datalist> HTML data list element

The **`<datalist>`** HTML element contains a set of `<option>` elements that represent the permissible or recommended options available to choose from within other controls.

### Try it

```
<label for="ice-cream-choice">Choose a flavor:</label>
<input list="ice-cream-flavors" id="ice-cream-choice" name="ice-cream-choice" />

<datalist id="ice-cream-flavors">
  <option value="Chocolate"></option>
  <option value="Coconut"></option>
  <option value="Mint"></option>
```

```
  <option value="Strawberry"></option>
  <option value="Vanilla"></option>
</datalist>
```

```
label {
  display: block;
  margin-bottom: 10px;
}
```

### Attributes

This element has no other attributes than the global attributes, common to all elements.

### Usage notes

To bind the `<datalist>` element to the control, we give it a unique identifier in the `id` attribute, and then add the `list` attribute to the `<input>` element with the same identifier as value. Only certain types of `<input>` support this behavior, and it can also vary from browser to browser.

Each `<option>` element should have a `value` attribute, which represents a suggestion to be entered into the input. It can also have a `label` attribute, or, missing that, some text content, which may be displayed by the browser instead of `value` (Firefox), or in addition to `value` (Chrome and Safari, as supplemental text). The exact content of the drop-down menu depends on the browser, but when clicked, content entered into control field will always come from the `value` attribute.

> **Note:** `<datalist>` is not a replacement for `<select>`. A `<datalist>` does not represent an input itself; it is a list of suggested values for an associated control. The control can still accept any value that passes validation, even if it is not in this suggestion list.

### Accessibility

When deciding to use the `<datalist>` element, here are some accessibility issues to be mindful of:

- The font size of the data list's options does not zoom, always remaining the same size. The contents of the autosuggest do not grow or shrink when the rest of the contents are zoomed in or out.
- As targeting the list of options with CSS is very limited to non-existent, rendering cannot be styled for high-contrast mode.
- Some screen reader/browser combinations, including NVDA and Firefox, do not announce the contents of the autosuggest popup.

### Examples

### Textual types

Recommended values in types text, search, url, tel, email and number, are displayed in a drop-down menu when user clicks or double-clicks on the control. Typically the right side of a control will also have an arrow pointing to the presence of predefined values.

```
<label for="myBrowser">Choose a browser from this list:</label>
<input list="browsers" id="myBrowser" name="myBrowser" />
<datalist id="browsers">
  <option value="Chrome"></option>
  <option value="Firefox"></option>
  <option value="Opera"></option>
  <option value="Safari"></option>
  <option value="Microsoft Edge"></option>
</datalist>
```

### Date and Time types

The types month, week, date, time and datetime-local can show an interface that allows a convenient selection of a date and time. Predefined values can be shown there, allowing the user to quickly fill the control value.

> **Note:** When these types are not supported, a basic `text` type will be rendered instead, creating a text field. That field will correctly recognize recommended values and display them to the user in a drop-down menu.

```
<input type="time" list="popularHours" />
<datalist id="popularHours">
  <option value="12:00"></option>
  <option value="13:00"></option>
  <option value="14:00"></option>
</datalist>
```

### Range type

When `value` attributes are included on `<option>` elements provided for a datalist associated with a range input type, they will be shown as a series of tick marks that the user can easily select.

```
<label for="tick">Tip amount:</label>
<input type="range" list="tickmarks" min="0" max="100" id="tick" name="tick" />
<datalist id="tickmarks">
  <option value="0" label="0%"></option>
  <option value="10" label="Minimum Tip"></option>
  <option value="20" label="Standard"></option>
  <option value="30" label="Generous"></option>
  <option value="50" label="Very Generous"></option>
</datalist>
```

> **Note:** The `label` attribute is intended to provide labels for tick marks, as defined in the HTML Standard. However, current browser support varies; labels might not be displayed visually or as tooltips.

### Color type

The color type can show predefined colors in a browser-provided interface.

```
<label for="colors">Pick a color (preferably a red tone):</label>
<input type="color" list="redColors" id="colors" />
<datalist id="redColors">
  <option value="#800000"></option>
  <option value="#8B0000"></option>
  <option value="#A52A2A"></option>
  <option value="#DC143C"></option>
</datalist>
```

### Technical summary

| | |
|---|---|
| **Content categories** | Flow content, phrasing content. |
| **Permitted content** | Either phrasing content or zero or more `<option>` elements. |
| **Tag omission** | None, both the starting and ending tag are mandatory. |
| **Permitted parents** | Any element that accepts phrasing content. |
| **Implicit ARIA role** | listbox |
| **Permitted ARIA roles** | No `role` permitted |
| **DOM interface** | `HTMLDataListElement` |

### See also

- The `<input>` element, and more specifically its `list` attribute;
- The `<option>` element.

## <dd> HTML description details element

The **`<dd>`** HTML element provides the description, definition, or value for the preceding term (`<dt>`) in a description list (`<dl>`).

### Try it

```
<p>Cryptids of Cornwall:</p>

<dl>
  <dt>Beast of Bodmin</dt>
  <dd>A large feline inhabiting Bodmin Moor.</dd>

  <dt>Morgawr</dt>
  <dd>A sea serpent.</dd>

  <dt>Owlman</dt>
  <dd>A giant owl-like creature.</dd>
</dl>
```

```
p,
dt {
  font-weight: bold;
}

dl,
```

```
dd {
  font-size: 0.9rem;
}

dd {
  margin-bottom: 1em;
}
```

### Attributes

This element only includes the global attributes.

### Examples

For examples, see the examples provided for the `<dl>` element.

### Technical summary

| | |
|---|---|
| **Content categories** | None. |
| **Permitted content** | Flow content. |
| **Tag omission** | The start tag is required. The end tag may be omitted if this element is immediately followed by another `<dd>` element or a `<dt>` element, or if there is no more content in the parent element. |
| **Permitted parents** | A `<dl>` or a `<div>` that is a child of a `<dl>`.<br>This element can be used after a `<dt>` or another `<dd>` element. |
| **Implicit ARIA role** | No corresponding role |
| **Permitted ARIA roles** | No `role` permitted |
| **DOM interface** | `HTMLElement` |

### See also

- `<dl>`
- `<dt>`

## <del> HTML deleted text element

The **`<del>`** HTML element represents a range of text that has been deleted from a document. This can be used when rendering "track changes" or source code diff information, for example. The `<ins>` element can be used for the opposite purpose: to indicate text that has been added to the document.

This element is often (but need not be) rendered by applying a strike-through style to the text.

### Try it

```
<blockquote>
  There is <del>nothing</del> <ins>no code</ins> either good or bad, but
  <del>thinking</del> <ins>running it</ins> makes it so.
</blockquote>
```

```
del {
  text-decoration: line-through;
  background-color: #ffbbbb;
  color: #555555;
}

ins {
  text-decoration: none;
  background-color: #d4fcbc;
}

blockquote {
  padding-left: 15px;
  border-left: 3px solid #d7d7db;
  font-size: 1rem;
}
```

### Attributes

This element's attributes include the global attributes.

`cite`

A URI for a resource that explains the change (for example, meeting minutes).

`datetime`

This attribute indicates the time and date of the change and must be a valid date string with an optional time. If the value cannot be parsed as a date with an optional time string, the element does not have an associated timestamp. For the format of the string without a time, see Date strings. The format of the string if it includes both date and time is covered in Local date and time strings.

### Accessibility

The presence of the `del` element is not announced by most screen reading technology in its default configuration. It can be made to be announced by using the CSS `content` property, along with the `::before` and `::after` pseudo-elements.

```
del::before,
del::after {
  clip-path: inset(100%);
  clip: rect(1px, 1px, 1px, 1px);
  height: 1px;
  overflow: hidden;
  position: absolute;
  white-space: nowrap;
  width: 1px;
}

del::before {
  content: " [deletion start] ";
}

del::after {
  content: " [deletion end] ";
}
```

Some people who use screen readers deliberately disable announcing content that creates extra verbosity. Because of this, it is important to not abuse this technique and only apply it in situations where not knowing content has been deleted would adversely affect understanding.

- Tweaking Text Level Styles | Adrian Roselli

### Examples

```
<p><del>This text has been deleted</del>, here is the rest of the paragraph.</p>
<del><p>This paragraph has been deleted.</p></del>
```

### Result

### Technical summary

| | |
|---|---|
| **Content categories** | Phrasing content, flow content. |
| **Permitted content** | Transparent. |
| **Tag omission** | None, both the starting and ending tag are mandatory. |
| **Permitted parents** | Any element that accepts phrasing content. |
| **Implicit ARIA role** | `deletion` |
| **Permitted ARIA roles** | Any |
| **DOM interface** | `HTMLModElement` |

### See also

- `<ins>` element for insertions into a text
- `<s>` element for strikethrough separate from representing deletion of text

## <details> HTML details disclosure element

The **`<details>`** HTML element creates a disclosure widget in which information is visible only when the widget is toggled into an open state. A summary or label must be provided using the `<summary>` element.

A disclosure widget is typically presented onscreen using a small triangle that rotates (or twists) to indicate open/closed state, with a label next to the triangle. The contents of the `<summary>` element are used as the label for the disclosure widget. The contents of the `<details>` provide the accessible description for the `<summary>`.

### Try it

```
<details>
  <summary>Details</summary>
  Something small enough to escape casual notice.
</details>
```

```
details {
  border: 1px solid #aaaaaa;
  border-radius: 4px;
  padding: 0.5em 0.5em 0;
}

summary {
  font-weight: bold;
  margin: -0.5em -0.5em 0;
  padding: 0.5em;
}

details[open] {
  padding: 0.5em;
}

details[open] summary {
  border-bottom: 1px solid #aaaaaa;
  margin-bottom: 0.5em;
}
```

### Attributes

This element includes the global attributes.

`open`

This Boolean attribute indicates whether the details — that is, the contents of the `<details>` element — are currently visible. The details are shown when this attribute exists, or hidden when this attribute is absent. By default this attribute is absent which means the details are not visible.

> **Note:** You have to remove this attribute entirely to make the details hidden. `open="false"` makes the details visible because this attribute is Boolean.

`name`

This attribute enables multiple `<details>` elements to be connected, with only one open at a time. This allows developers to easily create UI features such as accordions without scripting.

The `name` attribute specifies a group name — give multiple `<details>` elements the same `name` value to group them. Only one of the grouped `<details>` elements can be open at a time — opening one will cause another to close. If multiple grouped `<details>` elements are given the `open` attribute, only the first one in the source order will be rendered open.

> **Note:** `<details>` elements don't have to be adjacent to one another in the source to be part of the same group.

### Usage notes

A `<details>` widget can be in one of two states. The default *closed* state displays only the triangle and the label inside `<summary>` (or a user agent-defined default string if no `<summary>`).

When the user clicks on the widget or focuses it then presses the space bar, it "twists" open, revealing its contents. The common use of a triangle which rotates or twists around to represent opening or closing the widget is why these are sometimes called "twisty".

You can use CSS to style the disclosure widget, and you can programmatically open and close the widget by setting/removing its `open` attribute. Unfortunately, at this time, there's no built-in way to animate the transition between open and closed.

By default when closed, the widget is only tall enough to display the disclosure triangle and summary. When open, it expands to display the details contained within.

Fully standards-compliant implementations automatically apply the CSS `display: list-item` to the `<summary>` element. You can use this or the `::marker` pseudo-element to customize the disclosure widget.

### Events

In addition to the usual events supported by HTML elements, the `<details>` element supports the `toggle` event, which is dispatched to the `<details>` element whenever its state changes between open and closed. It is sent *after* the state is changed, although if the state changes multiple times before the browser can dispatch the event, the events are coalesced so that only one is sent.

You can use an event listener for the `toggle` event to detect when the widget changes state:

```
details.addEventListener("toggle", (event) => {
  if (details.open) {
    /* the element was toggled open */
  } else {
    /* the element was toggled closed */
  }
});
```

### Examples

#### A basic disclosure example

This example shows a basic `<details>` element with a `<summary>`.

```
<details>
  <summary>System Requirements</summary>
  <p>
    Requires a computer running an operating system. The computer must have some
    memory and ideally some kind of long-term storage. An input device as well
    as some form of output device is recommended.
  </p>
</details>
```

#### Result

#### Creating an open disclosure box

To start the `<details>` box in its open state, add the Boolean `open` attribute:

```
<details open>
  <summary>System Requirements</summary>
  <p>
    Requires a computer running an operating system. The computer must have some
    memory and ideally some kind of long-term storage. An input device as well
    as some form of output device is recommended.
  </p>
</details>
```

#### Result

#### Multiple named disclosure boxes

We include several `<details>` boxes, all with the same name so only one can be open at a time:

```
<details name="requirements">
  <summary>Graduation Requirements</summary>
  <p>
    Requires 40 credits, including a passing grade in health, geography,
    history, economics, and wood shop.
  </p>
</details>
<details name="requirements">
  <summary>System Requirements</summary>
  <p>
    Requires a computer running an operating system. The computer must have some
    memory and ideally some kind of long-term storage. An input device as well
    as some form of output device is recommended.
  </p>
</details>
<details name="requirements">
  <summary>Job Requirements</summary>
  <p>
    Requires knowledge of HTML, CSS, JavaScript, accessibility, web performance,
    privacy, security, and internationalization, as well as a dislike of
    broccoli.
  </p>
</details>
```

### Result

Try opening all the disclosure widgets. When you open one, all the others automatically close.

### Customizing the appearance

Now let's apply some CSS to customize the appearance of the disclosure box.

### CSS

```
details {
  font:
    16px "Open Sans",
    "Calibri",
    sans-serif;
  width: 620px;
}

details > summary {
  padding: 2px 6px;
  width: 15em;
  background-color: #dddddd;
  border: none;
  box-shadow: 3px 3px 4px black;
  cursor: pointer;
}

details > p {
  border-radius: 0 0 10px 10px;
  background-color: #dddddd;
  padding: 2px 6px;
  margin: 0;
  box-shadow: 3px 3px 4px black;
}

details:open > summary {
  background-color: #ccccff;
}
```

This CSS creates a look similar to a tabbed interface, where clicking the tab opens it to reveal its contents.

> **Note:** In browsers that don't support the `:open` pseudo-class, you can use the attribute selector `details[open]` to style the `<details>` element when it is in the open state.

### HTML

```
<details>
  <summary>System Requirements</summary>
  <p>
    Requires a computer running an operating system. The computer must have some
    memory and ideally some kind of long-term storage. An input device as well
    as some form of output device is recommended.
  </p>
</details>
```

### Result

See the `<summary>` page for an example of customizing the disclosure widget.

### Technical summary

| | |
|---|---|
| **Content categories** | Flow content, sectioning root, interactive content, palpable content. |
| **Permitted content** | One `<summary>` element followed by flow content. |
| **Tag omission** | None, both the starting and ending tag are mandatory. |
| **Permitted parents** | Any element that accepts flow content. |
| **Implicit ARIA role** | `group` |
| **Permitted ARIA roles** | No `role` permitted |
| **DOM interface** | `HTMLDetailsElement` |

### See also

- `<summary>`
- `::details-content`

## <dfn> HTML definition element

The **`<dfn>`** HTML element indicates a term to be defined. The `<dfn>` element should be used in a complete definition statement, where the full definition of the term can be one of the following:

- The ancestor paragraph (a block of text, sometimes marked by a `<p>` element)
- The `<dt>`/`<dd>` pairing
- The nearest section ancestor of the `<dfn>` element,

### Try it

```
<p>
  A <dfn id="def-validator">validator</dfn> is a program that checks for syntax
  errors in code or documents.
</p>
```

```
dfn {
  /* Add your styles here */
}
```

### Attributes

This element's attributes include the global attributes.

The `title` attribute has special meaning, as noted below.

### Usage notes

There are some not-entirely-obvious aspects to using the `<dfn>` element. We examine those here.

### Specifying the term being defined

The term being defined is identified following these rules:

1. If the `<dfn>` element has a `title` attribute, the value of the `title` attribute is considered to be the term being defined. The element must still have text within it, but that text may be an abbreviation (perhaps using `<abbr>`) or another form of the term.
2. If the `<dfn>` contains a single child element and does not have any text content of its own, and the child element is an `<abbr>` element with a `title` attribute itself, then the exact value of the `<abbr>` element's `title` is the term being defined.
3. Otherwise, the text content of the `<dfn>` element is the term being defined. This is shown in the first example below.

> **Note:** If the `<dfn>` element has a `title` attribute, it *must* contain the term being defined and no other text.

### Links to <code><dfn></code> elements

If you include an `id` attribute on the `<dfn>` element, you can then link to it using `<a>` elements. Such links should be uses of the term, with the intent being that the reader can quickly navigate to the term's definition if they're not already aware of it, by clicking on the term's link.

This is shown in the example under Links to definitions below.

### Examples

Let's take a look at some examples of various usage scenarios.

### Basic identification of a term

This example uses a plain `<dfn>` element to identify the location of a term within the definition.

#### HTML

```
<p>
  The <strong>HTML Definition element (<dfn>&lt;dfn&gt;</dfn>)</strong> is used
  to indicate the term being defined within the context of a definition phrase
  or sentence.
</p>
```

Since the `<dfn>` element has no `title`, the text contents of the `<dfn>` element itself are used as the term being defined.

#### Result

### Links to definitions

To add links to the definitions, you create the link the same way you always do, with the `<a>` element.

#### HTML

```
<p>
  The
  <strong>HTML Definition element (<dfn id="definition-dfn">&lt;dfn&gt;</dfn>)</strong>
  is used to indicate the term being defined within the context of a definition
  phrase or sentence.
</p>

<p>
  Lorem ipsum dolor sit amet, consectetur adipiscing elit. Graece donan, Latine
  voluptatem vocant. Confecta res esset. Duo Reges: constructio interrete.
  Scrupulum, inquam, abeunti;
</p>

<p>
  Because of all of that, we decided to use the
  <code><a href="#definition-dfn">&lt;dfn&gt;</a></code> element for this
  project.
</p>
```

Here we see the definition — now with an `id` attribute, `"definition-dfn"`, which can be used as the target of a link. Later on, a link is created using `<a>` with the `href` attribute set to `"#definition-dfn"` to set up the link back to the definition.

#### Result

### Using abbreviations and definitions together

In some cases, you may wish to use an abbreviation for a term when defining it. This can be done by using the `<dfn>` and `<abbr>` elements in tandem, like this:

#### HTML

```
<p>
  The <dfn><abbr title="Hubble Space Telescope">HST</abbr></dfn> is among the
  most productive scientific instruments ever constructed. It has been in orbit
  for over 20 years, scanning the sky and returning data and photographs of
  unprecedented quality and detail.
</p>

<p>
  Indeed, the <abbr title="Hubble Space Telescope">HST</abbr> has arguably done
  more to advance science than any device ever built.
</p>
```

Note the `<abbr>` element nested inside the `<dfn>`. The former establishes that the term is an abbreviation ("HST") and specifies the full term ("Hubble Space Telescope") in its `title` attribute. The latter indicates that the abbreviated term represents a term being defined.

#### Result

### Technical summary

| | |
|---|---|
| **Content categories** | Flow content, phrasing content, palpable content. |
| **Permitted content** | Phrasing content, but no `<dfn>` element must be a descendant. |
| **Tag omission** | None, both the starting and ending tag are mandatory. |
| **Permitted parents** | Any element that accepts phrasing content. |
| **Implicit ARIA role** | `term` |
| **Permitted ARIA roles** | Any |
| **DOM interface** | `HTMLElement` |

### See also

- Elements related to definition lists: `<dl>`, `<dt>`, `<dd>`
- `<abbr>`

## <dialog> HTML dialog element

The **`<dialog>`** HTML element represents a modal or non-modal dialog box or other interactive component, such as a dismissible alert, inspector, or subwindow.

### Attributes

This element includes the global attributes.

> **Warning:** The `tabindex` attribute must not be used on the `<dialog>` element. See Additional notes.

`closedby`

Specifies the types of user actions that can be used to close the `<dialog>` element. This attribute distinguishes three methods by which a dialog might be closed:

- A *light dismiss user action*, in which the `<dialog>` is closed when the user clicks or taps outside it. This is equivalent to the "light dismiss" behavior of "auto" state popovers.
- A *platform-specific user action*, such as pressing the `Esc` key on desktop platforms, or a "back" or "dismiss" gesture on mobile platforms.
- A developer-specified mechanism such as a `<button>` with a `click` handler that invokes `HTMLDialogElement.close()` or a `<form>` submission.

Possible values are:

`any`

The dialog can be dismissed using any of the three methods.

`closerequest`

The dialog can be dismissed with a platform-specific user action or a developer-specified mechanism.

`none`

The dialog can only be dismissed with a developer-specified mechanism.

If the `<dialog>` element does not have a valid `closedby` value specified, then

- if it was opened using `showModal()`, it behaves as if the value was `"closerequest"`
- otherwise, it behaves as if the value was `"none"`.

`open`

Indicates that the dialog box is active and is available for interaction. If the `open` attribute is not set, the dialog box will not be visible to the user. It is recommended to use the `.show()` or `.showModal()` method to render dialogs, rather than the `open` attribute. If a `<dialog>` is opened using the `open` attribute, it is non-modal.

> **Note:** While you can toggle between the open and closed states of non-modal dialog boxes by toggling the presence of the `open` attribute, this approach is not recommended. See `open` for more information.

### Description

The HTML `<dialog>` element is used to create both modal and non-modal dialog boxes. Modal dialog boxes block interaction with other UI elements, making the rest of the page inert, while non-modal dialog boxes allow interaction with the rest of the page.

### Controlling dialogs using JavaScript

JavaScript can be used to display and close the `<dialog>` element. You can use the `showModal()` method to display a modal dialog and the `show()` method to display a non-modal dialog. The dialog box can be closed using the `close()` method or using the `dialog` method when submitting a `<form>` that is nested within the `<dialog>` element. Modal dialogs can also be closed by pressing the `Esc` key.

### Modal dialogs using invoker commands

Modal dialogs can be declaratively opened and closed using the Invoker Commands API HTML attributes `commandfor` and `command`, which can be set on `<button>` elements.

The `command` attribute sets the particular command that is to be sent when the `<button>` element is clicked, while `commandfor` sets the `id` of the target dialog. The commands that can be sent for dialogs are `"show-modal"`, `"close"`, and `"request-close"`.

The HTML below demonstrates how to apply the attributes to a `<button>` element so it can be pressed to open a modal `<dialog>` with an `id` of "my-dialog".

```
<button command="show-modal" commandfor="my-dialog">Open dialog</button>

<dialog id="my-dialog">
  <p>This dialog was opened using an invoker command.</p>
  <button commandfor="my-dialog" command="close">Close</button>
</dialog>
```

### Non-modal dialogs using popover commands

Non-modal dialogs can be declaratively opened, closed, and toggled using the Popover API HTML attributes `popovertarget` and `popovertargetaction`, which can be defined on `<button>` and `<input>` elements.

The `<dialog>` must be turned into a popover by adding the `popover` attribute. You can then use `popovertarget` on a button/input to indicate the target popover, and `popovertargetaction` to specify the action to occur on the popover when the button is clicked. Note that, because the dialog is a popover, it will be non-modal, so you can close it by clicking outside the dialog.

The HTML below shows how to apply the attributes to a `<button>` element so it can be pressed to show and hide a non-modal `<dialog>` with an `id` of "my-dialog".

```
<button popovertarget="my-dialog">Open dialog</button>

<dialog id="my-dialog" popover>
  <p>This dialog was opened using a popovertargetaction attribute.</p>
  <button popovertarget="my-dialog" popovertargetaction="hide">Close</button>
</dialog>
```

The Popover API also provides properties that can be used to get and set the state in JavaScript.

### Closing dialogs

It is important to provide a closing mechanism for every `<dialog>` element, and to ensure that this works on devices that might not have a physical keyboard.

There are numerous ways to close a dialog:

- Submitting the form within the `<dialog>` element with `method="dialog"` set on the `<form>` element (see the Using the dialog open attribute example).
- Clicking outside the dialog area when "light dismiss" is enabled (see the Popover API HTML attributes example).
- Pressing the `Esc` key, in dialogs where it is enabled (see the Popover API HTML attributes example).
- Calling the `HTMLDialogElement.close()` method (see the modal example).

### CSS Styling

A `<dialog>` can be selected using its element name (like any other element), and you can also match its state using pseudo-classes such as `:modal` and `:open`.

The CSS `::backdrop` pseudo-element can be used to style the backdrop of a modal dialog, which is displayed behind the `<dialog>` element when the dialog is displayed using the `HTMLDialogElement.showModal()` method. This pseudo-element could be used, for example, to blur, darken, or otherwise obfuscate the inert content behind the modal dialog.

### Additional notes

- HTML `<form>` elements can be used to close a dialog box if they have the attribute `method="dialog"` or if the button used to submit the form has `formmethod="dialog"` set. When a `<form>` within a `<dialog>` is submitted via the `dialog` method, the dialog box closes, the states of the form controls are saved but not submitted, and the `returnValue` property gets set to the value of the button that was activated.
- The `autofocus` attribute should be added to the element the user is expected to interact with immediately upon opening a modal dialog. If no other element involves more immediate interaction, it is recommended to add `autofocus` to the close button inside the dialog, or the dialog itself if the user is expected to click/activate it to dismiss.
- Do not add the `tabindex` property to the `<dialog>` element as it is not interactive and does not receive focus. The dialog's contents, including the close button contained in the dialog, can receive focus and be interactive.

### Accessibility

When implementing a dialog, it is important to consider the most appropriate place to set user focus. When using `HTMLDialogElement.showModal()` to open a `<dialog>`, focus is set on the first nested focusable element. Explicitly indicating the initial focus placement by using the `autofocus` attribute will help ensure initial focus is set on the element deemed the best initial focus placement for any particular dialog. When in doubt, as it may not always be known where initial focus could be set within a dialog, particularly for instances where a dialog's content is dynamically rendered when invoked, the `<dialog>` element itself may provide the best initial focus placement.

Ensure a mechanism is provided to allow users to close the dialog. The most robust way to ensure that all users can close the dialog is to include an explicit button to do so, such as a confirmation, cancellation, or close button.

By default, a dialog invoked by the `showModal()` method can be dismissed by pressing the `Esc` key. A non-modal dialog does not dismiss via the `Esc` key by default, and depending on what the non-modal dialog represents, it may not be desired for this behavior. Keyboard users expect the `Esc` key to close modal dialogs; ensure that this behavior is implemented and maintained. If multiple modal dialogs are open, pressing the `Esc` key should close only the last shown dialog. When using `<dialog>`, this behavior is provided by the browser.

While dialogs can be created using other elements, the native `<dialog>` element provides usability and accessibility features that must be replicated if you use other elements for a similar purpose. If you're creating a custom dialog implementation, ensure that all expected default behaviors are supported and proper labeling recommendations are followed.

The `<dialog>` element is exposed by browsers in a manner similar to custom dialogs that use the ARIA role="dialog" attribute. `<dialog>` elements invoked by the `showModal()` method implicitly have aria-modal="true", whereas `<dialog>` elements invoked by the `show()` method or displayed using the `open` attribute or by changing the default `display` of a `<dialog>` are exposed as `[aria-modal="false"]`. When implementing modal dialogs, everything other than the `<dialog>` and its contents should be rendered inert using the `inert` attribute. When using `<dialog>` along with the `HTMLDialogElement.showModal()` method, this behavior is provided by the browser.

### Examples

#### Invoker Command API HTML attributes

This example demonstrates how you can use open and close a modal dialog using the `commandfor` and `command` HTML attributes of the Invoker Commands API.

First, we declare a `<button>` element, setting the `command` attribute to `"show-modal"`, and the `commandfor` attribute to the `id` of the dialog to open (`my-dialog`). Then we declare a `<dialog>` element that contains a "Close" `<button>`. This button sends the `"close"` command to the (same) dialog id.

```
<button command="show-modal" commandfor="my-dialog">Open dialog</button>

<dialog id="my-dialog">
  <p>This dialog was opened using an invoker command.</p>
  <button commandfor="my-dialog" command="close">Close</button>
</dialog>
```

#### Result

Open the dialog by pressing the "Open dialog" button. You can close the dialog by selecting the "Close" button or pressing the `Esc` key.

#### Popover API HTML attributes

This example demonstrates how you can open and close a non-modal dialog using the `popover`, `popovertarget`, and `popovertargetaction` HTML attributes of the Popover API.

The `<dialog>` is turned into a popover by adding the `popover` attribute. Since we haven't specified a value for the attribute, the default value of `"auto"` is used. This enables "light dismiss" behavior, allowing the dialog to be closed by clicking outside the dialog or by pressing `Esc`. We could instead have set `popover="manual"` to disable "light dismiss" behavior, in which case the dialog would have to be closed using the "Close" button.

Note that we haven't specified the `popovertargetaction` attribute for the `<button>` that opens the dialog. It isn't needed in this case, because its default value is `toggle`, which will toggle the dialog between its open and closed states when the button is clicked.

```
<button popovertarget="my-dialog">Open dialog</button>

<dialog id="my-dialog" popover>
  <p>This dialog was opened using a popovertargetaction attribute.</p>
  <button popovertarget="my-dialog" popovertargetaction="hide">Close</button>
</dialog>
```

#### Result

Open the dialog by pressing the "Open dialog" button. You can close the dialog by selecting the "Close" button or pressing the `Esc` key. You can also close it by selecting outside the dialog, as it is non-modal.

#### Using the dialog <code>open</code> attribute

This example demonstrates how you can set the boolean `open` attribute on a `<dialog>` element, in order to create an HTML-only non-modal dialog that is already open when the page loads.

The dialog can be closed by clicking the "OK" button because the `method` attribute in the `<form>` element is set to `"dialog"`. In this case, no JavaScript is needed to close the form.

```
<dialog open>
  <p>Greetings, one and all!</p>
  <form method="dialog">
    <button>OK</button>
```

```
  </form>
</dialog>
```

### Result

This dialog is initially open and non-modal because of the presence of the `open` attribute. After clicking "OK", the dialog gets dismissed, leaving the Result frame empty.

> **Note:** Reload the page to reset the output.

When the dialog is dismissed, there is no method provided to reopen it. The preferred method to display non-modal dialogs is to use the `HTMLDialogElement.show()` method. It is possible to toggle the display of the dialog by adding or removing the boolean `open` attribute, but this is not the recommended practice.

### Creating a modal dialog

This example demonstrates a modal dialog with a gradient backdrop. The `.showModal()` method opens the modal dialog when the "Show the dialog" button is activated. The dialog can be closed by pressing the `Esc` key or via the `close()` method when the "Close" button within the dialog is activated.

When a dialog opens, the browser, by default, gives focus to the first element that can be focused within the dialog. In this example, the `autofocus` attribute is applied to the "Close" button, giving it focus when the dialog opens, as this is the element we expect the user will interact with immediately after the dialog opens.

### HTML

```
<dialog>
  <button autofocus>Close</button>
  <p>This modal dialog has a groovy backdrop!</p>
</dialog>
<button>Show the dialog</button>
```

### CSS

We can style the backdrop of the dialog by using the `::backdrop` pseudo-element.

```
::backdrop {
  background-image: linear-gradient(
    45deg,
    magenta,
    rebeccapurple,
    dodgerblue,
    green
  );
  opacity: 0.75;
}
```

### JavaScript

The dialog is opened modally using the `.showModal()` method and closed using the `.close()` or `.requestClose()` methods.

```
const dialog = document.querySelector("dialog");
const showButton = document.querySelector("dialog + button");
const closeButton = document.querySelector("dialog button");

// "Show the dialog" button opens the dialog modally
showButton.addEventListener("click", () => {
  dialog.showModal();
});

// "Close" button closes the dialog
closeButton.addEventListener("click", () => {
  dialog.close();
});
```

### Result

When the modal dialog is displayed, it appears above any other dialogs that might be present. Everything outside the modal dialog is inert and interactions outside the dialog are blocked. Notice that when the dialog is open, with the exception of the dialog itself, interaction with the document is not possible; the "Show the dialog" button is mostly obfuscated by the almost opaque backdrop of the dialog and is inert.

### Handling the return value from the dialog

This example demonstrates the `returnValue` of the `<dialog>` element and how to close a modal dialog by using a form. By default, the `returnValue` is the empty string or the value of the button that submits the form within the `<dialog>` element, if there is one.

This example opens a modal dialog when the "Show the dialog" button is activated. The dialog contains a form with a `<select>` and three `<button>` elements, which default to `type="submit"`. Only the "Cancel" button has an explicit `value` attribute, which will be automatically used to set the dialog's `returnValue`.

### HTML

```
<!-- A modal dialog containing a form -->
<dialog id="favDialog">
  <form>
    <p>
      <label>
        Favorite animal:
        <select>
          <option value="nothing">Choose…</option>
          <option>Brine shrimp</option>
          <option>Red panda</option>
          <option>Spider monkey</option>
        </select>
      </label>
    </p>
    <div>
      <button value="cancel" formmethod="dialog">Cancel</button>
      <button id="requestCloseBtn">Cancel with requestClose</button>
      <button id="confirmBtn">Confirm</button>
    </div>
  </form>
</dialog>
<p>
  <button id="showDialog">Show the dialog</button>
</p>
<output></output>
```

### JavaScript

The dialog is opened using an event listener on the "Show the dialog" button, which calls `HTMLDialogElement.showModal()` when the button is clicked.

The dialog is closed when the "Cancel" button is clicked, because the `<button>` includes the `formmethod="dialog"` attribute. When a form's method is `dialog`, the state of the form is saved but not submitted, and the dialog gets closed (the attribute overrides the `<form>`'s default `GET` method). Without an `action`, submitting the form via the default `GET` method causes a page to reload. For the other two buttons, we use JavaScript to prevent the submission with `event.preventDefault()` and close the dialog with `HTMLDialogElement.close()` or `HTMLDialogElement.requestClose()`.

If the dialog is closed by pressing the `Esc` key or the `requestClose()` button, a `cancel` event gets triggered first, giving code a chance to prevent closure in some cases. In this example, the `cancel` event listener doesn't cancel the event; it just sets `returnValue` to `"cancelEvent"`. This value is only observable when pressing the `Esc` key, because the argument passed to `requestClose()`, `"requestClose"`, overwrites `returnValue` just before the `close` event listener gets called.

However the dialog is closed, the `close` event listener updates the text in the `<output>` with the final `returnValue`.

```
const showButton = document.getElementById("showDialog");
const favDialog = document.getElementById("favDialog");
const outputBox = document.querySelector("output");
const selectEl = favDialog.querySelector("select");
const requestCloseBtn = favDialog.querySelector("#requestCloseBtn");
const confirmBtn = favDialog.querySelector("#confirmBtn");

// "Show the dialog" button opens the <dialog> modally
showButton.addEventListener("click", () => {
  favDialog.showModal();
});

requestCloseBtn.addEventListener("click", (event) => {
  event.preventDefault();
  favDialog.requestClose("requestClose");
});

// Close the dialog with the selected animal
confirmBtn.addEventListener("click", (event) => {
  event.preventDefault();
  favDialog.close(selectEl.value);
});

// From Escape key or requestClose()
```

```
favDialog.addEventListener("cancel", () => {
  favDialog.returnValue = "cancelEvent";
});

// Display the return value whenever the dialog closes
favDialog.addEventListener("close", () => {
  outputBox.value = `ReturnValue: ${favDialog.returnValue}.`;
});
```

#### Result

### Closing a dialog with a required form input

When a form inside a dialog has a required input, the user agent will only let you close the dialog once you provide a value for the required input. To close such dialog, either use the `formnovalidate` attribute on the close button or call the `close()` method on the dialog object when the close button is clicked.

```
<dialog id="dialog">
  <form method="dialog">
    <p>
      <label>
        Favorite animal:
        <input type="text" required />
      </label>
    </p>
    <div>
      <input type="submit" id="normal-close" value="Normal close" />
      <input
        type="submit"
        id="novalidate-close"
        value="Novalidate close"
        formnovalidate />
      <input type="submit" id="js-close" value="JS close" />
    </div>
  </form>
</dialog>
<p>
  <button id="show-dialog">Show the dialog</button>
</p>
<output></output>
```

#### JavaScript

```
const showBtn = document.getElementById("show-dialog");
const dialog = document.getElementById("dialog");
const jsCloseBtn = dialog.querySelector("#js-close");

showBtn.addEventListener("click", () => {
  dialog.showModal();
});

jsCloseBtn.addEventListener("click", (e) => {
  e.preventDefault();
  dialog.close();
});
```

#### Result

From the output, we see it is impossible to close the dialog using the *Normal close* button. But the dialog can be closed if we bypass the form validation using the `formnovalidate` attribute on the *Cancel* button. Programmatically, `dialog.close()` will also close such dialog.

### Comparison of different closedby behaviors

This example demonstrates the difference in behavior between different values of the `closedby` attribute.

#### HTML

We provide three `<button>` elements and three `<dialog>` elements. Each button will be programmed to open a different dialog that demonstrates the behavior of one of the three values of the `closedby` attribute — `none`, `closerequest`, and `any`. Note that each `<dialog>` element contains a `<button>` element that will be used to close it.

```
<p>Choose a <code>&lt;dialog&gt;</code> type to show:</p>
<div id="controls">
  <button id="none-btn"><code>closedby="none"</code></button>
  <button id="closerequest-btn">
```

```
    <code>closedby="closerequest"</code>
  </button>
  <button id="any-btn"><code>closedby="any"</code></button>
</div>

<dialog closedby="none">
  <h2><code>closedby="none"</code></h2>
  <p>
    Only closable using a specific provided mechanism, which in this case is
    pressing the "Close" button below.
  </p>
  <button class="close">Close</button>
</dialog>

<dialog closedby="closerequest">
  <h2><code>closedby="closerequest"</code></h2>
  <p>Closable using the "Close" button or the Esc key.</p>
  <button class="close">Close</button>
</dialog>

<dialog closedby="any">
  <h2><code>closedby="any"</code></h2>
  <p>
    Closable using the "Close" button, the Esc key, or by clicking outside the
    dialog. "Light dismiss" behavior.
  </p>
  <button class="close">Close</button>
</dialog>
```

### JavaScript

Here we assign different variables to reference the main control `<button>` elements, the `<dialog>` elements, and the "Close" `<button>` elements inside the dialogs. First we assign a `click` event listener to each control button using `addEventListener`, the event handler function of which opens the associated `<dialog>` element via `showModal()`. We then loop through the "Close" `<button>` references, assigning each one a `click` event handler function that closes its `<dialog>` element via `close()`.

```
const noneBtn = document.getElementById("none-btn");
const closerequestBtn = document.getElementById("closerequest-btn");
const anyBtn = document.getElementById("any-btn");

const noneDialog = document.querySelector("[closedby='none']");
const closerequestDialog = document.querySelector("[closedby='closerequest']");
const anyDialog = document.querySelector("[closedby='any']");

const closeBtns = document.querySelectorAll(".close");

noneBtn.addEventListener("click", () => {
  noneDialog.showModal();
});

closerequestBtn.addEventListener("click", () => {
  closerequestDialog.showModal();
});

anyBtn.addEventListener("click", () => {
  anyDialog.showModal();
});

closeBtns.forEach((btn) => {
  btn.addEventListener("click", () => {
    btn.parentElement.close();
  });
});
```

### Result

The rendered result is as follows:

Try clicking each button to open a dialog. The first one can only be closed by clicking its "Close" button. The second one can also be closed via a device-specific user action such as pressing the `Esc` key. The third one has full "light-dismiss" behavior, so it can also be closed by clicking or tapping outside the dialog.

### Animating dialogs

`<dialog>`s are set to `display: none;` when hidden and `display: block;` when shown, as well as being removed from / added to the top layer and the accessibility tree. Therefore, for `<dialog>` elements to be animated the `display` property needs to be animatable. Supporting browsers animate `display` with a variation on the discrete animation type. Specifically, the browser will flip between `none` and another value of `display` so that the animated content is shown for the entire animation duration.

So for example:

- When animating `display` from `none` to `block` (or another visible `display` value), the value will flip to `block` at `0%` of the animation duration so it is visible throughout.
- When animating `display` from `block` (or another visible `display` value) to `none`, the value will flip to `none` at `100%` of the animation duration so it is visible throughout.

> **Note:** When animating using CSS transitions, `transition-behavior: allow-discrete` needs to be set to enable the above behavior. This behavior is available by default when animating with CSS animations; an equivalent step is not required.

### Transitioning dialog elements

When animating `<dialog>`s with CSS transitions, the following features are required:

`@starting-style` at-rule

> Provides a set of starting values for properties set on the `<dialog>` that you want to transition from every time it is opened. This is needed to avoid unexpected behavior. By default, CSS transitions only occur when a property changes from one value to another on a visible element; they are not triggered on elements' first style updates, or when the `display` type changes from `none` to another type.

`display` property

> Add `display` to the transitions list so that the `<dialog>` will remain as `display: block` (or another visible `display` value set on the dialog's open state) for the duration of the transition, ensuring the other transitions are visible.

`overlay` property

> Include `overlay` in the transitions list to ensure the removal of the `<dialog>` from the top layer is deferred until the transition completes, again ensuring the transition is visible.

`transition-behavior` property

> Set `transition-behavior: allow-discrete` on the `display` and `overlay` transitions (or on the `transition` shorthand) to enable discrete transitions on these two properties that are not by default animatable.

Here is a quick example to show what this might look like.

### HTML

The HTML contains a `<dialog>` element, plus a button to show the dialog. Additionally, the `<dialog>` element contains another button to close itself.

```
<dialog id="dialog">
  Content here
  <button class="close">close</button>
</dialog>

<button class="show">Show Modal</button>
```

### CSS

In the CSS, we include a `@starting-style` block that defines the transition starting styles for the `opacity` and `transform` properties, transition end styles on the `dialog:open` state, and default styles on the default `dialog` state to transition back to once the `<dialog>` has appeared. Note how the `<dialog>`'s `transition` list includes not only these properties, but also the `display` and `overlay` properties, each with `allow-discrete` set on them.

We also set a starting style value for the `background-color` property on the `::backdrop` that appears behind the `<dialog>` when it opens, to provide a nice darkening animation. The `dialog:open::backdrop` selector selects only the backdrops of `<dialog>` elements when the dialog is open.

```
/* Open state of the dialog  */
dialog:open {
  opacity: 1;
  transform: scaleY(1);
}

/* Closed state of the dialog   */
dialog {
  opacity: 0;
  transform: scaleY(0);
```

```
  transition:
    opacity 0.7s ease-out,
    transform 0.7s ease-out,
    overlay 0.7s ease-out allow-discrete,
    display 0.7s ease-out allow-discrete;
  /* Equivalent to
  transition: all 0.7s allow-discrete; */
}

/* Before open state  */
/* Needs to be after the previous dialog:open rule to take effect,
    as the specificity is the same */
@starting-style {
  dialog:open {
    opacity: 0;
    transform: scaleY(0);
  }
}

/* Transition the :backdrop when the dialog modal is promoted to the top layer */
dialog::backdrop {
  background-color: transparent;
  transition:
    display 0.7s allow-discrete,
    overlay 0.7s allow-discrete,
    background-color 0.7s;
  /* Equivalent to
  transition: all 0.7s allow-discrete; */
}

dialog:open::backdrop {
  background-color: rgb(0 0 0 / 25%);
}

/* This starting-style rule cannot be nested inside the above selector
because the nesting selector cannot represent pseudo-elements. */

@starting-style {
  dialog:open::backdrop {
    background-color: transparent;
  }
}
```

> **Note:** In browsers that don't support the `:open` pseudo-class, you can use the attribute selector `dialog[open]` to style the `<dialog>` element when it is in the open state.

### JavaScript

The JavaScript adds event handlers to the show and close buttons causing them to show and close the `<dialog>` when they are clicked:

```
const dialogElem = document.getElementById("dialog");
const showBtn = document.querySelector(".show");
const closeBtn = document.querySelector(".close");

showBtn.addEventListener("click", () => {
  dialogElem.showModal();
});

closeBtn.addEventListener("click", () => {
  dialogElem.close();
});
```

### Result

The code renders as follows:

> **Note:** Because `<dialog>`s change from `display: none` to `display: block` each time they are shown, the `<dialog>` transitions from its `@starting-style` styles to its `dialog:open` styles every time the entry transition occurs. When the `<dialog>` closes, it transitions from its `dialog:open` state to the default `dialog` state.
>
> It is possible for the style transition on entry and exit to be different in such cases. See our Demonstration of when starting styles are used example for a proof of this.

### dialog keyframe animations

When animating a `<dialog>` with CSS keyframe animations, there are some differences to note from transitions:

- You don't provide a `@starting-style`.
- You include the `display` value in a keyframe; this will be the `display` value for the entirety of the animation, or until another non-`none` display value is encountered.
- You don't need to explicitly enable discrete animations; there is no equivalent to `allow-discrete` inside keyframes.
- You don't need to set `overlay` inside keyframes either; the `display` animation handles the animation of the `<dialog>` from shown to hidden.

Let's have a look at an example so you can see what this looks like.

#### HTML

First, the HTML contains a `<dialog>` element, plus a button to show the dialog. Additionally, the `<dialog>` element contains another button to close itself.

```
<dialog id="dialog">
  Content here
  <button class="close">close</button>
</dialog>

<button class="show">Show Modal</button>
```

#### CSS

The CSS defines keyframes to animate between the closed and shown states of the `<dialog>`, plus the fade-in animation for the `<dialog>`'s backdrop. The `<dialog>` animations include animating `display` to make sure the actual visible animation effects remain visible for the whole duration. Note that it wasn't possible to animate the backdrop fade out — the backdrop is immediately removed from the DOM when the `<dialog>` is closed, so there is nothing to animate.

```
dialog {
  animation: fade-out 0.7s ease-out;
}

dialog:open {
  animation: fade-in 0.7s ease-out;
}

dialog:open::backdrop {
  background-color: black;
  animation: backdrop-fade-in 0.7s ease-out forwards;
}

/* Animation keyframes */

@keyframes fade-in {
  0% {
    opacity: 0;
    transform: scaleY(0);
    display: none;
  }

  100% {
    opacity: 1;
    transform: scaleY(1);
    display: block;
  }
}

@keyframes fade-out {
  0% {
    opacity: 1;
    transform: scaleY(1);
    display: block;
  }

  100% {
    opacity: 0;
    transform: scaleY(0);
    display: none;
  }
}

@keyframes backdrop-fade-in {
```

```
  0% {
    opacity: 0;
  }

  100% {
    opacity: 0.25;
  }
}

body,
button {
  font-family: system-ui;
}
```

### JavaScript

Finally, the JavaScript adds event handlers to the buttons to enable showing and closing the `<dialog>`:

```
const dialogElem = document.getElementById("dialog");
const showBtn = document.querySelector(".show");
const closeBtn = document.querySelector(".close");

showBtn.addEventListener("click", () => {
  dialogElem.showModal();
});

closeBtn.addEventListener("click", () => {
  dialogElem.close();
});
```

### Result

The code renders as follows:

### Technical summary

| | |
|---|---|
| **Content categories** | Flow content, sectioning root |
| **Permitted content** | Flow content |
| **Tag omission** | None, both the starting and ending tag are mandatory. |
| **Permitted parents** | Any element that accepts flow content |
| **Implicit ARIA role** | dialog |
| **Permitted ARIA roles** | `alertdialog` |
| **DOM interface** | `HTMLDialogElement` |

### See also

- `HTMLDialogElement` interface
- `close` event of the `HTMLDialogElement` interface
- `cancel` event of the `HTMLDialogElement` interface
- `open` property of the `HTMLDialogElement` interface
- `inert` global attribute for HTML elements
- `::backdrop` CSS pseudo-element
- Web forms in the Learn area

## <dir> HTML directory element

The **`<dir>`** HTML element is used as a container for a directory of files and/or folders, potentially with styles and icons applied by the user agent. Do not use this obsolete element; instead, you should use the `<ul>` element for lists, including lists of files.

> **Warning:** Do not use this element. Though present in early HTML specifications, it has been deprecated in HTML 4, and has since been removed entirely.

### DOM interface

This element implements the `HTMLDirectoryElement` interface.

### Attributes

Like all other HTML elements, this element supports the global attributes.

`compact`

> This Boolean attribute hints that the list should be rendered in a compact style. The interpretation of this attribute depends on the user agent and it doesn't work in all browsers.

### See also

- Other list-related HTML Elements: `<ol>`, `<ul>`, `<li>`, and `<menu>`;
- CSS properties that may be specially useful to style the `<dir>` element:
  - The `list-style` property, useful to choose the way the ordinal is displayed.
  - CSS counters, useful to handle complex nested lists.
  - The `line-height` property, useful to simulate the deprecated `compact` attribute.
  - The `margin` property, useful to control the indent of the list.

##  HTML content division element

The **`<div>`** HTML element is the generic container for flow content. It has no effect on the content or layout until styled in some way using CSS (e.g., styling is directly applied to it, or some kind of layout model like Flexbox is applied to its parent element).

### Try it

```
<div class="warning">
  <img
    src="https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/div//shared-assets/images/examples/leopard.jpg"
    alt="An intimidating leopard." />
  <p>Beware of the leopard</p>
</div>
```

```
.warning {
  border: 10px ridge red;
  background-color: yellow;
  padding: 0.5rem;
  display: flex;
  flex-direction: column;
}

.warning img {
  width: 100%;
}

.warning p {
  font: small-caps bold 1.2rem sans-serif;
  text-align: center;
}
```

### Attributes

This element includes the global attributes.

> **Note:** The `align` attribute is obsolete; do not use it anymore. Instead, you should use CSS properties or techniques such as CSS Grid or CSS Flexbox to align and position `<div>` elements on the page.

### Usage notes

As a "pure" container, the `<div>` element does not inherently represent anything. Instead, it's used to group content so it can be easily styled using the `class` or `id` attributes, marking a section of a document as being written in a different language (using the `lang` attribute), and so on.

The `<div>` element should be used only when no other semantic element (such as `<article>` or `<nav>`) is appropriate.

### Accessibility

The `<div>` element has an implicit role of `generic`, and not none. This may affect certain ARIA combination declarations that expect a direct descendant element with a certain role to function properly.

### Examples

### A basic example

```
<div>
  <p>
    Any kind of content here. Such as &lt;p&gt;, &lt;table&gt;. You name it!
```

```
  </p>
</div>
```

### Result

### A styled example

This example creates a shadowed box by applying a style to the `<div>` using CSS. Note the use of the `class` attribute on the `<div>` to apply the style named `"shadowbox"` to the element.

### HTML

```
<div class="shadowbox">
  <p>Here's a very interesting note displayed in a lovely shadowed box.</p>
</div>
```

### CSS

```
.shadowbox {
  width: 15em;
  border: 1px solid #333333;
  box-shadow: 8px 8px 5px #444444;
  padding: 8px 12px;
  background-image: linear-gradient(180deg, white, #dddddd 40%, #cccccc);
}
```

### Result

### Technical summary

| | |
|---|---|
| **Content categories** | Flow content, palpable content. |
| **Permitted content** | Flow content.<br>Or (in WHATWG HTML): If the parent is a `<dl>` element: one or more `<dt>` elements followed by one or more `<dd>` elements, optionally intermixed with `<script>` and `<template>` elements. |
| **Tag omission** | None, both the starting and ending tag are mandatory. |
| **Permitted parents** | Any element that accepts flow content.<br>Or (in WHATWG HTML): `<dl>` element. |
| **Implicit ARIA role** | `generic` |
| **Permitted ARIA roles** | Any |
| **DOM interface** | `HTMLDivElement` |

### See also

- Semantic sectioning elements: `<section>`, `<article>`, `<nav>`, `<header>`, `<footer>`
- `<span>` element for styling of phrasing content

## <dl> HTML description list element

The **`<dl>`** HTML element represents a description list. The element encloses a list of groups of terms (specified using the `<dt>` element) and descriptions (provided by `<dd>` elements). Common uses for this element are to implement a glossary or to display metadata (a list of key-value pairs).

### Try it

```
<p>Cryptids of Cornwall:</p>

<dl>
  <dt>Beast of Bodmin</dt>
  <dd>A large feline inhabiting Bodmin Moor.</dd>

  <dt>Morgawr</dt>
  <dd>A sea serpent.</dd>

  <dt>Owlman</dt>
  <dd>A giant owl-like creature.</dd>
</dl>
```

```
p,
dt {
  font-weight: bold;
}

dl,
dd {
  font-size: 0.9rem;
}

dd {
  margin-bottom: 1em;
}
```

### Attributes

This element also accepts the global attributes.

`compact`

> This Boolean attribute hints that the list should be rendered in a compact style. The interpretation of this attribute is browser-specific. Use CSS instead: to give a similar effect as the `compact` attribute, the CSS property `line-height` can be used with a value of `80%`.

### Accessibility

Each screen reader exposes `<dl>` content differently, including total count, terms/definitions context, and navigation methods. These differences are not necessarily bugs. As of iOS 14, VoiceOver will announce that `<dl>` content is a list when navigating with the virtual cursor (not via the read-all command). VoiceOver does not support list navigation commands with `<dl>`. Be careful applying ARIA `term` and `definition` roles to `<dl>` constructs as VoiceOver (macOS and iOS) will adjust how they are announced.

- VoiceOver on iOS 14 Supports Description Lists
- Brief Note on Description List Support

### Examples

#### Single term and description

```
<dl>
  <dt>Firefox</dt>
  <dd>
    A free, open source, cross-platform, graphical web browser developed by the
    Mozilla Corporation and hundreds of volunteers.
  </dd>

  <!-- Other terms and descriptions -->
</dl>
```

#### Result

#### Multiple terms, single description

```
<dl>
  <dt>Firefox</dt>
  <dt>Mozilla Firefox</dt>
  <dt>Fx</dt>
  <dd>
    A free, open source, cross-platform, graphical web browser developed by the
    Mozilla Corporation and hundreds of volunteers.
  </dd>

  <!-- Other terms and descriptions -->
</dl>
```

#### Result

#### Single term, multiple descriptions

```
<dl>
  <dt>Firefox</dt>
  <dd>
    A free, open source, cross-platform, graphical web browser developed by the
    Mozilla Corporation and hundreds of volunteers.
  </dd>
  <dd>
```

```
    The Red Panda also known as the Lesser Panda, Wah, Bear Cat or Firefox, is a
    mostly herbivorous mammal, slightly larger than a domestic cat (60 cm long).
  </dd>

  <!-- Other terms and descriptions -->
</dl>
```

### Result

### Multiple terms and descriptions

It is also possible to define multiple terms with multiple corresponding descriptions, by combining the examples above.

### Metadata

Description lists are useful for displaying metadata as a list of key-value pairs.

```
<dl>
  <dt>Name</dt>
  <dd>Godzilla</dd>
  <dt>Born</dt>
  <dd>1952</dd>
  <dt>Birthplace</dt>
  <dd>Japan</dd>
  <dt>Color</dt>
  <dd>Green</dd>
</dl>
```

### Result

Tip: It can be handy to define a key-value separator in the CSS, such as:

```
dt::after {
  content: ": ";
}
```

### Wrapping name-value groups in <code>div</code> elements

HTML allows wrapping each name-value group in a `<dl>` element in a `<div>` element. This can be useful when using microdata, or when global attributes apply to a whole group, or for styling purposes.

```
<dl>
  <div>
    <dt>Name</dt>
    <dd>Godzilla</dd>
  </div>
  <div>
    <dt>Born</dt>
    <dd>1952</dd>
  </div>
  <div>
    <dt>Birthplace</dt>
    <dd>Japan</dd>
  </div>
  <div>
    <dt>Color</dt>
    <dd>Green</dd>
  </div>
</dl>
```

### Result

### Notes

Do not use this element (nor `<ul>` elements) to merely create indentation on a page. Although it works, this is a bad practice and obscures the meaning of description lists.

To change the indentation of a description term, use the CSS `margin` property.

### Technical summary

| | |
|---|---|
| **Content categories** | Flow content, and if the `<dl>` element's children include one name-value group, palpable content. |

| | |
|---|---|
| **Permitted content** | Either: Zero or more groups each consisting of one or more `<dt>` elements followed by one or more `<dd>` elements, optionally intermixed with `<script>` and `<template>` elements.<br>Or: (in WHATWG HTML, W3C HTML 5.2 and later) One or more `<div>` elements, optionally intermixed with `<script>` and `<template>` elements. |
| **Tag omission** | None, both the starting and ending tag are mandatory. |
| **Permitted parents** | Any element that accepts flow content. |
| **Implicit ARIA role** | No corresponding role |
| **Permitted ARIA roles** | `group`, `list`, `none`, `presentation` |
| **DOM interface** | `HTMLDListElement` |

### See also

- `<dt>`
- `<dd>`

## <dt> HTML description term element

The **`<dt>`** HTML element specifies a term in a description or definition list, and as such must be used inside a `<dl>` element. It is usually followed by a `<dd>` element; however, multiple `<dt>` elements in a row indicate several terms that are all defined by the immediate next `<dd>` element.

The subsequent `<dd>` (**Description Details**) element provides the definition or other related text associated with the term specified using `<dt>`.

### Try it

```
<p>Please use the following paint colors for the new house:</p>

<dl>
  <dt>Denim (semigloss finish)</dt>
  <dd>Ceiling</dd>

  <dt>Denim (eggshell finish)</dt>
  <dt>Evening Sky (eggshell finish)</dt>
  <dd>Layered on the walls</dd>
</dl>
```

```
p,
dl {
  font:
    1rem "Fira Sans",
    sans-serif;
}

dl > dt {
  font-weight: normal;
  font-style: oblique;
}

dd {
  margin-bottom: 1rem;
}
```

### Attributes

This element only includes the global attributes.

### Examples

For examples, see the examples provided for the `<dl>` element.

### Technical summary

| | |
|---|---|
| **Content categories** | None. |
| **Permitted content** | Flow content, but with no `<header>`, `<footer>`, sectioning content or heading content descendants. |

| | |
|---|---|
| **Tag omission** | The start tag is required. The end tag may be omitted if this element is immediately followed by another `<dt>` element or a `<dd>` element, or if there is no more content in the parent element. |
| **Permitted parents** | A `<dl>` or (in WHATWG HTML, W3C HTML 5.2 and later) a `<div>` that is a child of a `<dl>`.<br>This element can be used before a `<dd>` or another `<dt>` element. |
| **Implicit ARIA role** | No corresponding role |
| **Permitted ARIA roles** | `listitem` |
| **DOM interface** | `HTMLElement` Up to Gecko 1.9.2 (Firefox 4) inclusive, Firefox implements the `HTMLSpanElement` interface for this element. |

### See also

- `<dl>`
- `<dd>`

##  HTML emphasis element

The **** HTML element marks text that has stress emphasis. The `<em>` element can be nested, with each level of nesting indicating a greater degree of emphasis.

### Try it

```
<p>Get out of bed <em>now</em>!</p>

<p>We <em>had</em> to do something about it.</p>

<p>This is <em>not</em> a drill!</p>
```

```
em {
  /* Add your styles here */
}
```

### Attributes

This element only includes the global attributes.

### Usage notes

The `<em>` element is for words that have a stressed emphasis compared to surrounding text, which is often limited to a word or words of a sentence and affects the meaning of the sentence itself.

Typically this element is displayed in italic type. However, it should not be used to apply italic styling; use the CSS `font-style` property for that purpose. Use the `<cite>` element to mark the title of a work (book, play, song, etc.). Use the `<i>` element to mark text that is in an alternate tone or mood, which covers many common situations for italics such as scientific names or words in other languages. Use the `<strong>` element to mark text that has greater importance than surrounding text.

### <i> vs. <em>

Some developers may be confused by how multiple elements seemingly produce similar visual results. `<em>` and `<i>` are a common example, since they both italicize text. What's the difference? Which should you use?

By default, the visual result is the same. However, the semantic meaning is different. The `<em>` element represents stress emphasis of its contents, while the `<i>` element represents text that is set off from the normal prose, such as a foreign word, fictional character thoughts, or when the text refers to the definition of a word instead of representing its semantic meaning. (The title of a work, such as the name of a book or movie, should use `<cite>`.)

This means the right one to use depends on the situation. Neither is for purely decorative purposes, that's what CSS styling is for.

Examples for `<em>` could be:

```
<p>Just <em>do</em> it already!</p>
<p>We <em>had</em> to do something about it.</p>
```

A person or software reading the text would pronounce the words in italics with an emphasis, using verbal stress.

Examples for `<i>` could be:

```
<p>The word <i>the</i> is an article.</p>
<p>The <i>Queen Mary</i> sailed last night.</p>
```

Here, there is no added emphasis or importance on the word "Queen Mary". It is merely indicated that the object in question is not a queen named Mary but a ship named "Queen Mary".

### Examples

In this example, the `<em>` element is used to highlight an implicit or explicit contrast between two ingredient lists:

```
<p>
  Ice cream is made with milk, sweetener, and cream. Frozen custard, on the
  other hand, is made of milk, cream, sweetener, and <em>egg yolks</em>.
</p>
```

### Result

### Technical summary

| | |
|---|---|
| **Content categories** | Flow content, phrasing content, palpable content. |
| **Permitted content** | Phrasing content. |
| **Tag omission** | None, both the starting and ending tag are mandatory. |
| **Permitted parents** | Any element that accepts phrasing content. |
| **Implicit ARIA role** | `emphasis` |
| **Permitted ARIA roles** | Any |
| **DOM interface** | `HTMLElement` Up to Gecko 1.9.2 (Firefox 4) inclusive, Firefox implements the `HTMLSpanElement` interface for this element. |

### See also

- `<i>`

## <embed> HTML embed external content element

The **`<embed>`** HTML element embeds external content at the specified point in the document. This content is handled by the browser or an external application.

### Try it

```
<embed
  type="image/jpeg"
  src="https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/embed//shared-assets/images/examples/flowers.jpg"
  width="250"
  height="200" />
```

### Attributes

This element's attributes include the global attributes.

`height`

The displayed height of the resource, in CSS pixels. This must be an absolute value; percentages are *not* allowed.

`src`

The URL of the resource being embedded.

`type`

The MIME type of the embedded resource.

`width`

The displayed width of the resource, in CSS pixels. This must be an absolute value; percentages are *not* allowed.

### Usage notes

Modern browsers no longer support browser plug-ins such as Flash, but `<embed>` can still display content handled by the browser itself, such as PDFs in a built-in PDF viewer.

For PDF previews, consider `<object>` or `<iframe>`. Unlike `<object>`, `<embed>` cannot contain fallback content. Unlike `<iframe>`, it has no attributes that control loading.

You can use the `object-position` property to adjust the positioning of the embedded object within the element's frame.

> **Note:** The `object-fit` property has no effect on `<embed>` elements.

### Accessibility

Use the `title` attribute on an `embed` element to label its content so that people navigating with assistive technology such as a screen reader can understand what it contains. The title's value should concisely describe the embedded content. Without a title, they may not be able to determine what its embedded content is. This context shift can be confusing and time-consuming, especially if the `embed` element contains interactive content like video or audio.

### Examples

```
<embed
  type="video/quicktime"
  src="https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/embed/movie.mov"
  width="640"
  height="480"
  title="Title of my video" />
```

### Technical summary

| | |
|---|---|
| **Content categories** | Flow content, phrasing content, embedded content, interactive content, palpable content. |
| **Permitted content** | None; it is a void element. |
| **Tag omission** | Must have a start tag and must not have an end tag. |
| **Permitted parents** | Any element that accepts embedded content. |
| **Implicit ARIA role** | No corresponding role |
| **Permitted ARIA roles** | `application`, `document`, `img`, `none`, `presentation` |
| **DOM interface** | `HTMLEmbedElement` |

### See also

- Other elements that are used for embedding content of various types include `<audio>`, `<canvas>`, `<iframe>`, `<img>`, `<math>`, `<object>`, `<svg>`, and `<video>`.
- Positioning and sizing the embedded content within its frame: `object-position` and `object-fit`

## <fencedframe> HTML fenced frame element

The **`<fencedframe>`** HTML element represents a nested browsing context, embedding another HTML page into the current one. `<fencedframe>`s are very similar to `<iframe>` elements in form and function, except that:

- Communication is restricted between the `<fencedframe>` content and its embedding site.
- A `<fencedframe>` can access cross-site data, but only in a very specific set of controlled circumstances that preserve user privacy.
- A `<fencedframe>` cannot be manipulated or have its data accessed via regular scripting (for example reading or setting the source URL). `<fencedframe>` content can only be embedded via specific APIs.
- A `<fencedframe>` cannot access the embedding context's DOM, nor can the embedding context access the `<fencedframe>`'s DOM.

The `<fencedframe>` element is a type of `<iframe>` with more native privacy features built in. It addresses shortcomings of `<iframe>`s such as reliance on third-party cookies and other privacy risks. See Fenced frame API for more details.

### Attributes

This element includes the global attributes.

`allow`

> Specifies a Permissions Policy for the `<fencedframe>`, which defines what features are available to the `<fencedframe>` based on the origin of the request. See Permissions policies available to fenced frames for more details of which features can be controlled via a policy set on a fenced frame.

`height`

> A unitless integer representing the height of the fenced frame in CSS pixels. The default is `150`.

`width`

> A unitless integer representing the width of the fenced frame in CSS pixels. The default is `300`.

### Permissions policies available to fenced frames

Permissions delegated from the top-level context to a fenced frame for allowing and denying features could be used as a communication channel, so constitute a privacy threat. As a result, standard web features that can have their availability controlled via Permissions Policy (for example, `camera` or `geolocation`) are **not available** within fenced frames.

The only features that can be enabled by a policy inside fenced frames are the specific features designed to be used inside fenced frames:

- Protected Audience API
  - `attribution-reporting`
  - `private-aggregation`
  - `shared-storage`
  - `shared-storage-select-url`
- Shared Storage API
  - `attribution-reporting`
  - `private-aggregation`
  - `shared-storage`
  - `shared-storage-select-url`

Currently these are always enabled inside fenced frames. In the future, which ones are enabled will be controllable using the `<fencedframe> allow` attribute. Blocking privacy sandbox features in this manner will also block the fenced frame from loading — there will be no communication channel at all.

### Focusing across fenced frame boundaries

The ability of the document's active focus to be moved across fenced frame boundaries (i.e., from an element outside the fenced frame to one inside, or vice versa) is limited. User-initiated actions such as a click or a tab can do so, as there is no fingerprinting risk there.

However, trying to traverse the boundary via an API call such as `HTMLElement.focus()` is prohibited — a malicious script could use a series of such calls to leak inferred information across the boundary.

### Positioning and scaling

Being a replaced element, the `<fencedframe>` allows the position of the embedded document within its box to be adjusted using the `object-position` property.

> **Note:** The `object-fit` property has no effect on `<fencedframe>` elements.

The size of the embedded content may be set by internal `contentWidth` and `contentHeight` properties of the `<fencedframe>`'s `config` object. In such cases, changing the `width` or `height` of the `<fencedframe>` will change the size of the embedded container on the page, but the document inside the container will be visually scaled to fit. The reported width and height of the embedded document (i.e., `Window.innerWidth` and `Window.innerHeight`) will be unchanged.

### Accessibility

People navigating with assistive technology, such as a screen reader, can use the `title` attribute on a `<fencedframe>` to label its content. The title's value should concisely describe the embedded content:

```
<fencedframe
  title="Advertisement for new Log. From Blammo!"
  width="640"
  height="320"></fencedframe>
```

Without this title, they have to navigate into the `<fencedframe>` to determine what its embedded content is. This context shift can be confusing and time-consuming, especially for pages with multiple `<fencedframe>`s and/or if embeds contain interactive content like video or audio.

### Examples

To set what content will be shown in a `<fencedframe>`, a utilizing API (such as Protected Audience or Shared Storage) generates a `FencedFrameConfig` object, which is then set as the value of the `<fencedframe>`'s `config` property.

The following example gets a `FencedFrameConfig` from a Protected Audience API's ad auction, which is then used to display the winning ad in a `<fencedframe>`:

```
<fencedframe width="640" height="320"></fencedframe>
```

```
const frameConfig = await navigator.runAdAuction({
  // … auction configuration
  resolveToConfig: true,
});
```

```
const frame = document.querySelector("fencedframe");
frame.config = frameConfig;
```

> **Note:** `resolveToConfig: true` must be passed in to the `runAdAuction()` call to obtain a `FencedFrameConfig` object. If it is not set, the resulting `Promise` will resolve to a URN that can only be used in an `<iframe>`.

### Technical summary

| | |
|---|---|
| **Content categories** | Flow content, phrasing content, embedded content, interactive content, palpable content. |
| **Permitted content** | None. |
| **Tag omission** | None, both the starting and ending tag are mandatory. |
| **Permitted parents** | Any element that accepts embedded content. |
| **Implicit ARIA role** | No corresponding role |
| **Permitted ARIA roles** | `application`, `document`, `img`, `none`, `presentation` |
| **DOM interface** | `HTMLFencedFrameElement` |

### See also

- Fenced Frame API
- Fenced frames on privacysandbox.google.com
- The Privacy Sandbox on privacysandbox.google.com

## <fieldset> HTML field set element

The **`<fieldset>`** HTML element is used to group several controls as well as labels (`<label>`) within a web form.

### Try it

```
<form>
  <fieldset>
    <legend>Choose your favorite monster</legend>

    <input type="radio" id="kraken" name="monster" value="K" />
    <label for="kraken">Kraken</label><br />

    <input type="radio" id="sasquatch" name="monster" value="S" />
    <label for="sasquatch">Sasquatch</label><br />

    <input type="radio" id="mothman" name="monster" value="M" />
    <label for="mothman">Mothman</label>
  </fieldset>
</form>
```

```
legend {
  background-color: black;
  color: white;
  padding: 3px 6px;
}

input {
  margin: 0.4rem;
}
```

As the example above shows, the `<fieldset>` element provides a grouping for a part of an HTML form, with a nested `<legend>` element providing a caption for the `<fieldset>`. It takes few attributes, the most notable of which are `form`, which can contain the `id` of a `<form>` on the same page, allowing you to make the `<fieldset>` part of that `<form>` even if it is not nested inside it, and `disabled`, which allows you to disable the `<fieldset>` and all its contents in one go.

### Attributes

This element includes the global attributes.

`disabled`

> If this Boolean attribute is set, all form controls that are descendants of the `<fieldset>`, are disabled, meaning they are not editable and won't be submitted along with the `<form>`. They won't receive any browsing events, like mouse clicks or focus-related events. By default browsers display such controls grayed out. Note that form elements inside the `<legend>` element won't be disabled.

`form`

This attribute takes the value of the `id` attribute of a `<form>` element you want the `<fieldset>` to be part of, even if it is not inside the form. Please note that usage of this is confusing — if you want the `<input>` elements inside the `<fieldset>` to be associated with the form, you need to use the `form` attribute directly on those elements. You can check which elements are associated with a form via JavaScript, using `HTMLFormElement.elements`.

`name`

The name associated with the group.

> **Note:** The caption for the fieldset is given by the first `<legend>` element nested inside it.

### Styling with CSS

There are several special styling considerations for `<fieldset>`.

Its `display` value is `block` by default, and it establishes a block formatting context. If the `<fieldset>` is styled with an inline-level `display` value, it will behave as `inline-block`, otherwise it will behave as `block`. By default there is a `2px groove` border surrounding the contents, and a small amount of default padding. The element has `min-inline-size: min-content` by default.

If a `<legend>` is present, it is placed over the `block-start` border. The `<legend>` shrink-wraps, and also establishes a formatting context. The `display` value is blockified. (For example, `display: inline` behaves as `block`.)

There will be an anonymous box holding the contents of the `<fieldset>`, which inherits certain properties from the `<fieldset>`. If the `<fieldset>` is styled with `display: grid` or `display: inline-grid`, then the anonymous box will be a grid formatting context. If the `<fieldset>` is styled with `display: flex` or `display: inline-flex`, then the anonymous box will be a flex formatting context. Otherwise, it establishes a block formatting context.

You can feel free to style the `<fieldset>` and `<legend>` in any way you want to suit your page design.

### Examples

#### Basic fieldset

This example includes a `<fieldset>` with a `<legend>`, with a single control inside it.

```
<form action="#">
  <fieldset>
    <legend>Do you agree?</legend>
    <input type="checkbox" id="chbx" name="agree" value="Yes!" />
    <label for="chbx">I agree</label>
  </fieldset>
</form>
```

#### Result

#### Disabled fieldset

This example shows a disabled `<fieldset>` with two controls inside it. Note how both the controls are disabled due to being inside a disabled `<fieldset>`.

```
<form action="#">
  <fieldset disabled>
    <legend>Disabled login fieldset</legend>
    <div>
      <label for="name">Name: </label>
      <input type="text" id="name" value="Chris" />
    </div>
    <div>
      <label for="pwd">Archetype: </label>
      <input type="password" id="pwd" value="Wookie" />
    </div>
  </fieldset>
</form>
```

#### Result

### Technical summary

| | |
|---|---|
| **Content categories** | Flow content, sectioning root, listed, form-associated element, palpable content. |

| | |
|---|---|
| **Permitted content** | An optional `<legend>` element, followed by flow content. |
| **Tag omission** | None, both the starting and ending tag are mandatory. |
| **Permitted parents** | Any element that accepts flow content. |
| **Implicit ARIA role** | `group` |
| **Permitted ARIA roles** | `radiogroup`, `presentation`, `none` |
| **DOM interface** | `HTMLFieldSetElement` |

### See also

- The `<legend>` element
- The `<input>` element
- The `<label>` element
- The `<form>` element

## <figcaption> HTML figure caption element

The **`<figcaption>`** HTML element represents a caption or legend describing the rest of the contents of its parent `<figure>` element, providing the `<figure>` an accessible name.

### Try it

```
<figure>
  <img
    src="https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/figcaption//shared-assets/images/examples/elephant.jpg"
    alt="Elephant at sunset" />
  <figcaption>An elephant at sunset</figcaption>
</figure>
```

```
figure {
  border: thin silver solid;
  display: flex;
  flex-flow: column;
  padding: 5px;
  max-width: 220px;
  margin: auto;
}

img {
  max-width: 220px;
  max-height: 150px;
}

figcaption {
  background-color: #222222;
  color: white;
  font: italic smaller sans-serif;
  padding: 3px;
  text-align: center;
}
```

### Attributes

This element only includes the global attributes.

### Examples

Please see the `<figure>` page for examples on `<figcaption>`.

### Technical summary

| | |
|---|---|
| **Content categories** | None. |
| **Permitted content** | Flow content. |
| **Tag omission** | None, both the starting and ending tag are mandatory. |
| **Permitted parents** | A `<figure>` element; the `<figcaption>` element must be its first or last child. |
| **Implicit ARIA role** | No corresponding role |
| **Permitted ARIA roles** | `group`, `none`, `presentation` |
| **DOM interface** | `HTMLElement` |

### See also

- The `<figure>` element.

## <figure> HTML figure with optional caption element

The **`<figure>`** HTML element represents self-contained content, potentially with an optional caption, which is specified using the `<figcaption>` element. The figure, its caption, and its contents are referenced as a single unit.

### Try it

```
<figure>
  <img
    src="https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/figure//shared-assets/images/examples/elephant.jpg"
    alt="Elephant at sunset" />
  <figcaption>An elephant at sunset</figcaption>
</figure>
```

```
figure {
  border: thin silver solid;
  display: flex;
  flex-flow: column;
  padding: 5px;
  max-width: 220px;
  margin: auto;
}

img {
  max-width: 220px;
  max-height: 150px;
}

figcaption {
  background-color: #222222;
  color: white;
  font: italic smaller sans-serif;
  padding: 3px;
  text-align: center;
}
```

### Attributes

This element only includes the global attributes.

### Usage notes

- Usually a `<figure>` is an image, illustration, diagram, code snippet, etc., that is referenced in the main flow of a document, but that can be moved to another part of the document or to an appendix without affecting the main flow.
- A caption can be associated with the `<figure>` element by inserting a `<figcaption>` inside it (as the first or the last child). The first `<figcaption>` element found in the figure is presented as the figure's caption.
- The `<figcaption>` provides the accessible name for the parent `<figure>`.

### Examples

### Images

```
<!-- Just an image -->
<figure>
  <img src="https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/figure/favicon-192x192.png" alt="The beautiful MDN
logo." />
</figure>

<!-- Image with a caption -->
<figure>
  <img src="https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/figure/favicon-192x192.png" alt="The beautiful MDN
logo." />
  <figcaption>MDN Logo</figcaption>
</figure>
```

### Result

### Code snippets

```
<figure>
  <figcaption>Get browser details using <code>navigator</code>.</figcaption>
  <pre>
function NavigatorExample() {
  let txt = `Browser CodeName: ${navigator.appCodeName};\n`;
  txt += `Browser Name: ${navigator.appName};\n`;
  txt += `Browser Version: ${navigator.appVersion};\n`;
  txt += `Cookies Enabled: ${navigator.cookieEnabled};\n`;
  txt += `Platform: ${navigator.platform};\n`;
  txt += `User-agent header: ${navigator.userAgent};`;
  console.log("NavigatorExample", txt);
}
  </pre>
</figure>
```

### Result

### Quotations

```
<figure>
  <figcaption><b>Edsger Dijkstra:</b></figcaption>
  <blockquote>
    If debugging is the process of removing software bugs, then programming must
    be the process of putting them in.
  </blockquote>
</figure>
```

### Result

### Poems

```
<figure>
  <p>
    Bid me discourse, I will enchant thine ear,<br />
    Or like a fairy trip upon the green,<br />
    Or, like a nymph, with long dishevelled hair,<br />
    Dance on the sands, and yet no footing seen:<br />
    Love is a spirit all compact of fire,<br />
    Not gross to sink, but light, and will aspire.<br />
  </p>
  <figcaption><cite>Venus and Adonis</cite>, by William Shakespeare</figcaption>
</figure>
```

### Result

### Technical summary

| | |
|---|---|
| **Content categories** | Flow content, palpable content. |
| **Permitted content** | A `<figcaption>` element, followed by flow content; or flow content followed by a `<figcaption>` element; or flow content. |
| **Tag omission** | None, both the starting and ending tag are mandatory. |
| **Permitted parents** | Any element that accepts Flow content. |
| **Implicit ARIA role** | figure |
| **Permitted ARIA roles** | With no figcaption descendant: any, otherwise no permitted roles |
| **DOM interface** | `HTMLElement` |

### See also

- The `<figcaption>` element.

## <font> HTML font element

The **`<font>`** HTML element defines the font size, color and face for its content.

> **Warning:** Do not use this element. Use the CSS Fonts properties to style text.

### Attributes

Like all other HTML elements, this element supports the global attributes.

`color`

This attribute sets the text color using either a named color or a color specified in the hexadecimal #RRGGBB format.

`face`

This attribute contains a comma-separated list of one or more font names. The document text in the default style is rendered in the first font face that the client's browser supports. If no font listed is installed on the local system, the browser typically defaults to the proportional or fixed-width font for that system.

`size`

This attribute specifies the font size as either a numeric or relative value. Numeric values range from `1` to `7` with `1` being the smallest and `3` the default. It can be defined using a relative value, like `+2` or `-3`, which sets it relative to `3`, the default value.

### DOM interface

This element implements the `HTMLFontElement` interface.

## <footer> HTML footer element

The **`<footer>`** HTML element represents a footer for its nearest ancestor sectioning content or sectioning root element. A `<footer>` typically contains information about the author of the section, copyright data or links to related documents.

### Try it

```
<article>
  <h1>How to be a wizard</h1>
  <ol>
    <li>Grow a long, majestic beard.</li>
    <li>Wear a tall, pointed hat.</li>
    <li>Have I mentioned the beard?</li>
  </ol>
  <footer>
    <p>© 2018 Gandalf</p>
  </footer>
</article>
```

```
article {
  min-height: 100%;
  display: grid;
  grid-template-rows: auto 1fr auto;
}

footer {
  display: flex;
  justify-content: center;
  padding: 5px;
  background-color: #45a1ff;
  color: white;
}
```

### Attributes

This element only includes the global attributes.

### Usage notes

- Enclose information about the author in an `<address>` element that can be included into the `<footer>` element.
- When the nearest ancestor sectioning content or sectioning root element is the body element the footer applies to the whole page.
- The `<footer>` element is not sectioning content and therefore doesn't introduce a new section in the outline.

### Accessibility

Prior to the release of Safari 13, the `contentinfo` landmark role was not properly exposed by VoiceOver. If needing to support legacy Safari browsers, add `role="contentinfo"` to the `footer` element to ensure the landmark will be properly exposed.

- Related: WebKit Bugzilla: 146930 – AX: HTML native elements (header, footer, main, aside, nav) should work the same as ARIA landmarks, sometimes they don't

### Examples

```
<body>
  <h3>FIFA World Cup top goalscorers</h3>
  <ol>
    <li>Miroslav Klose, 16</li>
    <li>Ronaldo Nazário, 15</li>
    <li>Gerd Müller, 14</li>
  </ol>

  <footer>
    <small>
      Copyright © 2023 Football History Archives. All Rights Reserved.
    </small>
  </footer>
</body>
```

```
footer {
  text-align: center;
  padding: 5px;
  background-color: #abbaba;
  color: black;
}
```

### Technical summary

| | |
|---|---|
| **Content categories** | Flow content, palpable content. |
| **Permitted content** | Flow content, but with no `<footer>` or `<header>` descendants. |
| **Tag omission** | None, both the starting and ending tag are mandatory. |
| **Permitted parents** | Any element that accepts flow content. Note that a `<footer>` element must not be a descendant of an `<address>`, `<header>` or another `<footer>` element. |
| **Implicit ARIA role** | contentinfo, or generic if a descendant of an article, aside, main, nav or section element, or an element with `article`, `complementary`, `main`, `navigation` or `region` role |
| **Permitted ARIA roles** | `group`, `presentation` or `none` |
| **DOM interface** | `HTMLElement` |

### See also

- Other section-related elements: `<body>`, `<nav>`, `<article>`, `<aside>`, h1, h2, h3, h4, h5, h6, `<hgroup>`, `<header>`, `<section>`, `<address>`;
- Using HTML sections and outlines
- ARIA: Contentinfo role

## <form> HTML form element

The **`<form>`** HTML element represents a document section containing interactive controls for submitting information.

It is possible to use the `:valid` and `:invalid` CSS pseudo-classes to style a `<form>` element based on whether the `elements` inside the form are valid.

### Try it

```
<form action="" method="get" class="form-example">
  <div class="form-example">
    <label for="name">Enter your name: </label>
    <input type="text" name="name" id="name" required />
  </div>
  <div class="form-example">
    <label for="email">Enter your email: </label>
    <input type="email" name="email" id="email" required />
  </div>
  <div class="form-example">
    <input type="submit" value="Subscribe!" />
  </div>
</form>
```

```
document.querySelector("form").addEventListener("submit", (event) => {
  event.preventDefault();
});
```

```
form.form-example {
  display: table;
}

div.form-example {
  display: table-row;
}

label,
input {
  display: table-cell;
  margin-bottom: 10px;
}

label {
  padding-right: 10px;
}
```

### Attributes

This element includes the global attributes.

`accept`

Comma-separated content types the server accepts.

> **Note: This attribute has been deprecated and should not be used.** Instead, use the `accept` attribute on `<input type=file>` elements.

`accept-charset`

The character encoding accepted by the server. The specification allows a single case-insensitive value of `"UTF-8"`, reflecting the ubiquity of this encoding (historically multiple character encodings could be specified as a comma-separated or space-separated list).

`autocapitalize`

Controls whether inputted text is automatically capitalized and, if so, in what manner. See the `autocapitalize` global attribute page for more information.

`autocomplete`

Indicates whether input elements can by default have their values automatically completed by the browser. `autocomplete` attributes on form elements override it on `<form>`. Possible values:

- `off`: The browser may not automatically complete entries. (Browsers tend to ignore this for suspected login forms; see Managing autofill for login fields.)
- `on`: The browser may automatically complete entries.

`name`

The name of the form. The value must not be the empty string, and must be unique among the `form` elements in the forms collection that it is in, if any. The name becomes a property of the `Window`, `Document`, and `document.forms` objects, containing a reference to the form element.

`rel`

Controls the annotations and what kinds of links the form creates. Annotations include `external`, `nofollow`, `opener`, `noopener`, and `noreferrer`. Link types include `help`, `prev`, `next`, `search`, and `license`. The `rel` value is a space-separated list of these enumerated values.

### Attributes for form submission

The following attributes control behavior during form submission.

`action`

The URL that processes the form submission. This value can be overridden by a `formaction` attribute on a `<button>`, `<input type="submit">`, or `<input type="image">` element. This attribute is ignored when `method="dialog"` is set.

`enctype`

If the value of the `method` attribute is `post`, `enctype` is the MIME type of the form submission. Possible values:

- `application/x-www-form-urlencoded`: The default value.
- `multipart/form-data`: Use this if the form contains `<input>` elements with `type=file`.
- `text/plain`: Useful for debugging purposes.

This value can be overridden by `formenctype` attributes on `<button>`, `<input type="submit">`, or `<input type="image">` elements.

`method`

The HTTP method to submit the form with. The only allowed methods/values are (case insensitive):

- `post`: The `POST` method; form data sent as the request body.
- `get` (default): The `GET`; form data appended to the `action` URL with a `?` separator. Use this method when the form has no side effects.
- `dialog`: When the form is inside a `<dialog>`, closes the dialog and causes a `submit` event to be fired on submission, without submitting data or clearing the form.

This value is overridden by `formmethod` attributes on `<button>`, `<input type="submit">`, or `<input type="image">` elements.

`novalidate`

This Boolean attribute indicates that the form shouldn't be validated when submitted. If this attribute is not set (and therefore the form ***is*** validated), it can be overridden by a `formnovalidate` attribute on a `<button>`, `<input type="submit">`, or `<input type="image">` element belonging to the form.

`target`

Indicates where to display the response after submitting the form. It is a name/keyword for a *browsing context* (for example, tab, window, or iframe). The following keywords have special meanings:

- `_self` (default): Load into the same browsing context as the current one.
- `_blank`: Load into a new unnamed browsing context. This provides the same behavior as setting `rel="noopener"` which does not set `window.opener`.
- `_parent`: Load into the parent browsing context of the current one. If no parent, behaves the same as `_self`.
- `_top`: Load into the top-level browsing context (i.e., the browsing context that is an ancestor of the current one and has no parent). If no parent, behaves the same as `_self`.
- `_unfencedTop`: Load the response from a form inside an embedded fenced frame into the top-level frame (i.e., traversing beyond the root of the fenced frame, unlike other reserved destinations). Only available inside fenced frames.

This value can be overridden by a `formtarget` attribute on a `<button>`, `<input type="submit">`, or `<input type="image">` element.

### Examples

```
<!-- Form which will send a GET request to the current URL -->
<form method="get">
  <label>
    Name:
    <input name="submitted-name" autocomplete="name" />
  </label>
  <button>Save</button>
</form>

<!-- Form which will send a POST request to the current URL -->
<form method="post">
  <label>
    Name:
    <input name="submitted-name" autocomplete="name" />
  </label>
  <button>Save</button>
</form>

<!-- Form with fieldset, legend, and label -->
<form method="post">
  <fieldset>
    <legend>Do you agree to the terms?</legend>
    <label><input type="radio" name="radio" value="yes" /> Yes</label>
    <label><input type="radio" name="radio" value="no" /> No</label>
  </fieldset>
</form>
```

### Result

### Technical summary

| | |
|---|---|
| **Content categories** | Flow content, palpable content |
| **Permitted content** | Flow content, but not containing `<form>` elements |
| **Tag omission** | None, both the starting and ending tag are mandatory. |
| **Permitted parents** | Any element that accepts flow content |
| **Implicit ARIA role** | `form` |

| Permitted ARIA roles | `search`, `none` or `presentation` |
| --- | --- |
| DOM interface | `HTMLFormElement` |

### See also

- HTML forms guide
- Other elements that are used when creating forms: `<button>`, `<datalist>`, `<fieldset>`, `<input>`, `<label>`, `<legend>`, `<meter>`, `<optgroup>`, `<option>`, `<output>`, `<progress>`, `<select>`, `<textarea>`.
- Getting a list of the elements in the form: `HTMLFormElement.elements`
- ARIA: Form role
- ARIA: Search role

## <frame> HTML frame element

The **`<frame>`** HTML element defines a particular area in which another HTML document can be displayed. A frame should be used within a `<frameset>`.

Using the `<frame>` element is not encouraged because of certain disadvantages such as performance problems and lack of accessibility for users with screen readers. Instead of the `<frame>` element, `<iframe>` may be preferred.

### Attributes

Like all other HTML elements, this element supports the global attributes.

`src`

This attribute specifies the document that will be displayed by the frame.

`name`

This attribute is used for labeling frames. Without labeling, every link will open in the frame that it's in – the closest parent frame. See the `target` attribute for more information.

`noresize`

This attribute prevents resizing of frames by users.

`scrolling`

This attribute defines the existence of a scrollbar. If this attribute is not used, the browser adds a scrollbar when necessary. There are two choices: "yes" for forcing a scrollbar even when it is not necessary and "no" for forcing no scrollbar even when it *is* necessary.

`marginheight`

This attribute defines the height of the margin between frames.

`marginwidth`

This attribute defines the width of the margin between frames.

`frameborder`

This attribute allows you to specify a frame's border.

### Example

#### A frameset document

A frameset document has a `<frameset>` element instead of a `<body>` element. The `<frame>` elements are placed within the `<frameset>`.

```
<!doctype html>
<html lang="en-US">
  <head>
    <!-- Document metadata goes here -->
  </head>
  <frameset cols="400, 500">
    <frame src="https://developer.mozilla.org/en/HTML/Element/iframe" />
    <frame src="https://developer.mozilla.org/en/HTML/Element/frame" />
  </frameset>
</html>
```

If you want to embed another HTML page into the `<body>` of a document, use an `<iframe>` element.

### See also

- `<frameset>`
- `<iframe>`

## <frameset> HTML frameset element

The **`<frameset>`** HTML element is used to contain `<frame>` elements.

> **Note:** Because the use of frames is now discouraged in favor of using `<iframe>`, this element is not typically used by modern websites.

### Attributes

Like all other HTML elements, this element supports the global attributes.

`cols`

This attribute specifies the number and size of horizontal spaces in a frameset.

`rows`

This attribute specifies the number and size of vertical spaces in a frameset.

### Example

#### A frameset document

A frameset document has a `<frameset>` element instead of a `<body>` element. The `<frame>` elements are placed within the `<frameset>`.

```
<!doctype html>
<html lang="en-US">
  <head>
    <!-- Document metadata goes here -->
  </head>
  <frameset cols="50%, 50%">
    <frame
      src="https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/iframe" />
    <frame
      src="https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/frame" />
  </frameset>
</html>
```

If you want to embed another HTML page into the `<body>` of a document, use an `<iframe>` element.

### See also

- `<frame>`
- `<iframe>`

## <geolocation> HTML geolocation element

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`<geolocation>`** HTML element creates an interactive control for the user to share their location data with the page.

It provides:

- An intuitive browser-defined UI.
- A process for handling the necessary permissions for the `geolocation` feature.
- API features for accessing location data and responding to received location data and permission changes.

### Attributes

This element includes the global attributes.

`autolocate`

A boolean attribute that, when set to `true`, specifies that the browser should immediately retrieve location data when the `<geolocation>` element is rendered, provided permission was previously granted. If set to `false`, location data is not retrieved until the user activates the control. Defaults to `false`.

If permission was not previously granted, this attribute has no effect.

`watch`

A boolean attribute that, when set to `true`, specifies that the browser should retrieve location data whenever the position of the user's device changes. If set to `false`, location data is only retrieved once. Defaults to `false`.

### Description

The `<geolocation>` element provides a declarative browser-defined control for sharing location data. In Chrome, for example, the button features a "map pin" icon and intuitive text ("Use location" in English content).

It also allows for intuitive management of user permissions. For example, in Chrome, if the user previously denied permission to access location data, or dismissed the permission dialog without making a choice, they are free to press the button again to update their choice. In cases where they previously denied permission, subsequent dialogs will inform them that they previously didn't allow location data to be shared, and ask them whether they want to continue not allowing it, or to allow it.

A key aspect of the `<geolocation>` element is that it reflects the user's conscious choice, and blocks possible usage that might trick the user into providing their location data unwittingly (see `<geolocation>` blocking for more information).

The element's DOM API interface, `HTMLGeolocationElement`, provides features to access returned position data, current permission status, and errors if the data retrieval was unsuccessful, reducing the amount of JavaScript logic that needs to be written. It also has events available to run code in response to location data being received, changes in permission status, and user interactions with the permission dialog.

> **Note:** For performance reasons, a maximum of three `<geolocation>` elements are allowed on any one page. If this quota is exceeded, all `<geolocation>` elements have their functionality disabled.

### Relationship with the Geolocation API

The Geolocation API provides an older alternative for handling location data. This API has some shortcomings that the `<geolocation>` element aims to solve, most notably that the UI and underlying logic for requesting the data needs to be implemented from scratch each time, and the handling of permissions can be unintuitive.

The `<geolocation>` element uses features of the Geolocation API in the background. By default, the browser requests location data once, as if the `Geolocation.getCurrentPosition()` method was called. However, if the `watch` attribute is set to `true`, the browser updates the data whenever the device position changes, as if `Geolocation.watchPosition()` was called.

If data is successfully retrieved, it is available in the `HTMLGeolocationElement.position` property, which contains a `GeolocationPosition` object. If data retrieval is unsuccessful, error information is available in the `HTMLGeolocationElement.error` property, which contains a `GeolocationPositionError` object.

### Setting the button language

The global `lang` attribute is used by the `<geolocation>` element to select a language for its rendered text. This means that you can set a `lang` attribute directly on the `<geolocation>` element or on one of its ancestors to tell the browser what language to use for the button label.

If no suitable `lang` attribute is set, the browser's preferred language setting is used.

### Including fallback content

You can include fallback content between the `<geolocation>` element's opening and closing tags that will be displayed if it isn't supported. For example, you might include a "Not supported" message:

```
<geolocation>
  <p>Your browser doesn't support the Geolocation element.</p>
</geolocation>
```

However, a better real-world solution might be to include a regular `<button>` element that uses the Geolocation API to retrieve location data:

```
<geolocation>
  <button id="fallback">Use location</button>
</geolocation>
```

### <code><geolocation></code> blocking

One key idea behind the design of the `<geolocation>` element is that it should reflect a user's conscious choice to expose position information, and prevent bad actors from tricking users into activating it, for example via clickjacking. Because of this, the browser keeps a record of so-called **blocker reasons** for each rendered element.

When a blocker is active on a `<geolocation>` element, it is prevented from functioning (blocked), either temporarily or permanently, depending on the reason. When a `<geolocation>` element is blocked, it is said to be invalid. You can check whether it is invalid by querying the

`HTMLGeolocationElement.isValid` property. You can also return the reason why it is invalid via the `HTMLGeolocationElement.invalidReason` property — see that page for a full list of possible reasons.

### Styling restrictions

The `<geolocation>` element has several constraints on the CSS styles that can be applied to it. Some of these constraints are designed to enforce fundamental accessibility, and will result in the button being deactivated if they are not adhered to. Some enforce certain values or value ranges for various properties.

Any properties that are not listed in the following sub-sections, or logically equivalent to a physical property listed in the following sub-sections, are ignored when set on the `<geolocation>` element.

### Accessibility restrictions

The rendered `<geolocation>` button is deactivated (meaning that pressing it will have no effect) if the following constraints are not adhered to:

- The color contrast ratio between `color` and `background-color` must be at least 3:1.
- The `font-size` must not be smaller than the `small` value (in the case of keyword values), or its computed value (in the case of other value types).

### Value constraints

The following CSS property value constraints are applied to the `<geolocation>` element. If an attempt is made to set these properties to values outside the listed constraints on the `<geolocation>` element, the value is adjusted to equal the constraint (in the case of an exact value constraint) or to equal to nearest computed value upper or lower bound (in the case of a range constraint).

`opacity`

: `1.0`

`line-height`

: `normal`

`white-space`

: `nowrap`

`user-select`

: `none`

`appearance`

: `auto`

`box-sizing`

: `content-box`

`vertical-align`

: `middle`

`text-emphasis`

: `initial`

`text-shadow`

: `initial`

`outline-offset`

: `0` or greater.

`font-weight`

: `200` or greater.

`word-spacing`

: Between `0` and `0.5em`, inclusive.

`letter-spacing`

: Between `-0.05em` and `0.2em`, inclusive.

`letter-spacing`

: Between `-0.05em` and `0.2em`, inclusive.

`min-height`

: `1em` or greater.

`max-height`

: `3em` or less. `none` is an accepted value.

`min-width`

: The computed value of `fit-content` or less.

`border-width`

: `1em` or less.

### Complex constraints

The following constraints are more complex than simple value constraints:

Block direction padding

: If the `block-size` is set to `auto`, the `padding-block-start` and `padding-block-end` (and equivalent physical properties for the current writing mode) are constrained to a maximum of `1em` and must be equal.

Inline direction padding

: If the `inline-size` is set to `auto`, the `padding-inline-start` and `padding-inline-end` (and equivalent physical properties for the current writing mode) are constrained to a maximum of `5em` and must be equal.

### Properties that can be set normally

The following CSS properties can be used normally:

- `font-kerning`
- `font-optical-sizing`
- `font-stretch`
- `font-synthesis-weight`
- `font-synthesis-style`
- `font-synthesis-small-caps`
- `font-feature-settings`
- `forced-color-adjust`
- `text-rendering`
- `align-self`
- `anchor-name`
- `aspect-ratio`
- `border`, `border-top`, `border-right`, `border-bottom`, and `border-left`
- `clear`
- `color-scheme`
- `contain-intrinsic-width`
- `contain-intrinsic-height`
- `container-name`
- `container-type`
- `counter-reset`, `counter-increment`, and `counter-set`
- `flex`, `flex-grow`, `flex-shrink`, and `flex-basis`
- `float`
- `height`
- `isolation`
- `justify-self`
- `left`
- `order`
- `orphans`
- `outline`, `outline-color`, and `outline-style`
- `overflow-anchor`
- `overscroll-behavior`, `overscroll-behavior-inline`, `overscroll-behavior-block`, `overscroll-behavior-x`, and `overscroll-behavior-y`
- `page`
- `position`
- `position-anchor`

- `right`
- `scroll-margin`, `scroll-margin-top`, `scroll-margin-right`, `scroll-margin-bottom`, and `scroll-margin-left`
- `scroll-padding`, `scroll-padding-top`, `scroll-padding-right`, `scroll-padding-bottom`, `scroll-padding-left`, `scroll-padding-inline-start`, `scroll-padding-block-start`, `scroll-padding-block-start`, `scroll-padding-inline-end`, and `scroll-padding-block-end`
- `text-spacing-trim`
- `text-transform`
- `top`
- `visibility`
- `x`
- `y`
- `ruby-position`
- `user-select`
- `width`
- `will-change`
- `z-index`

### Accessibility

The `<geolocation>` element has an accessible name written in the language it is set to. It also has a `role` of `button` so that it is recognized as a button by screen readers.

In addition, the `<geolocation>` element has a default `tabindex` value of `0`, so it behaves like a real `<button>` with respect to keyboard focus.

Finally, refer back to the Accessibility restrictions section for information on styling constraints applied to the `<geolocation>` element to enforce fundamental accessibility requirements.

### Examples

### Basic usage example

This example uses the `<geolocation>` element to retrieve your current location, which is printed out below the button in a `<p>` element. The example also uses a regular `<button>` fallback to retrieve the location data in non-supporting browsers.

### HTML

We include a `<geolocation>` element with a `<button>` fallback nested inside it, which will be rendered in browsers that do not support `<geolocation>`. We also include a `<p>` to output location data and errors into.

```
<geolocation>
  <button id="fallback">Use location</button>
</geolocation>
<p id="output"></p>
```

### JavaScript

In our script, we start off by grabbing a reference to the output `<p>` element. We then detect whether the `<geolocation>` element is supported by testing `typeof HTMLGeolocationElement === "function"`:

- If it is supported, we first grab a reference to the `<geolocation>` element and then add a `location` event listener. When the button is pressed and the data is retrieved, the listener prints the (lat, long) coordinates to the output `<p>` (retrieved via the `position` property), or an error message if the data retrieval was unsuccessful (retrieved via the `error` property).
- If it isn't supported, we grab a reference to the fallback `<button>` element and retrieve and print the same data, except that this time we are using a `click` event listener on the button, and a `Geolocation.getCurrentPosition()` call to retrieve the data.

```
const outputElem = document.querySelector("#output");

if (typeof HTMLGeolocationElement === "function") {
  const geo = document.querySelector("geolocation");
  geo.addEventListener("location", () => {
    if (geo.position) {
      outputElem.textContent += `(${geo.position.coords.latitude},${geo.position.coords.longitude}), `;
    } else if (geo.error) {
      outputElem.textContent += `${geo.error.message}, `;
    }
  });
} else {
  const fallback = document.querySelector("#fallback");
  fallback.addEventListener("click", () => {
    navigator.geolocation.getCurrentPosition(
      (position) => {
        outputElem.textContent += `(${position.coords.latitude}, ${position.coords.longitude}), `;
      },
```

```
      (error) => {
        outputElem.textContent += `${error.message}, `;
      },
    );
  });
}
```

### Result

See this code running live (source code). You can also find a version of this example that includes the `watch` attribute on the `<geolocation>` element and therefore fetches location data each time the user's device position changes (see it running live, and the source code).

Try viewing the demos in a supported browser and an unsupported browser if possible, and note the difference in permissions dialog flow when you choose to allow or deny permission to use `geolocation`.

For a walkthrough of a more complete example that uses location data to create a map of your local area, see the `HTMLGeolocationElement` reference page.

### Technical summary

| | |
|---|---|
| **Content categories** | Flow content, phrasing content, interactive content, palpable content. |
| **Permitted content** | Any suitable transparent fallback content. |
| **Tag omission** | None, both the starting and ending tag are mandatory. |
| **Permitted parents** | Any element that accepts phrasing content. |
| **Implicit ARIA role** | No corresponding role |
| **Permitted ARIA roles** | `button` |
| **DOM interface** | `HTMLGeolocationElement` |

### See also

- `HTMLGeolocationElement`
- The `geolocation` Permissions Policy
- Geolocation API
- Permissions API
- Introducing the `<geolocation>` HTML element on developer.chrome.com (2026)

## <head> HTML document metadata (header) element

The **`<head>`** HTML element contains machine-readable information (metadata) about the document, like its title, scripts, and style sheets. There can be only one `<head>` element in an HTML document.

> **Note:** `<head>` primarily holds information for machine processing, not human-readability. For human-visible information, like top-level headings and listed authors, see the `<header>` element.

### Attributes

This element includes the global attributes.

`profile`

The URIs of one or more metadata profiles, separated by white space.

### Examples

```
<!doctype html>
<html lang="en-US">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width" />
    <title>Document title</title>
  </head>
</html>
```

### Technical summary

| | |
|---|---|
| **Content categories** | None. |

| | |
|---|---|
| **Permitted content** | If the document is an `<iframe>` `srcdoc` document, or if title information is available from a higher level protocol (like the subject line in HTML email), zero or more elements of metadata content.<br><br>Otherwise, one or more elements of metadata content where exactly one is a `<title>` element. |
| **Tag omission** | The start tag may be omitted if the first thing inside the `<head>` element is an element.<br>The end tag may be omitted if the first thing following the `<head>` element is not a space character or a comment. |
| **Permitted parents** | An `<html>` element, as its first child. |
| **Implicit ARIA role** | No corresponding role |
| **Permitted ARIA roles** | No `role` permitted |
| **DOM interface** | `HTMLHeadElement` |

### See also

- Elements that can be used inside the `<head>`:
  - `<title>`
  - `<base>`
  - `<link>`
  - `<style>`
  - `<meta>`
  - `<script>`
  - `<noscript>`
  - `<template>`

## <header> HTML header element

The **`<header>`** HTML element represents introductory content, typically a group of introductory or navigational aids. It may contain some heading elements but also a logo, a search form, an author name, and other elements.

### Try it

```
<header>
  <a class="logo" href="#">Cute Puppies Express!</a>
</header>

<article>
  <header>
    <h1>Beagles</h1>
    <time>08.12.2014</time>
  </header>
  <p>
    I love beagles <em>so</em> much! Like, really, a lot. They're adorable and
    their ears are so, so snugly soft!
  </p>
</article>
```

```
.logo {
  background: left / cover
    url("/shared-assets/images/examples/puppy-header.jpg");
  display: flex;
  height: 120px;
  align-items: center;
  justify-content: center;
  font:
    bold calc(1em + 2 * (100vw - 120px) / 100) "Dancing Script",
    fantasy;
  color: #ff0083;
  text-shadow: black 2px 2px 0.2rem;
}

header > h1 {
  margin-bottom: 0;
}

header > time {
  font: italic 0.7rem sans-serif;
}
```

### Usage notes

When not nested within sectioning content, `<main>`, or an element with the same ARIA role as these elements' implicit ARIA role, then the `<header>` element has an identical meaning to the site-wide `banner` landmark role. It defines a global site header, which usually includes a logo, company name, search feature, and possibly the global navigation or a slogan. It is generally located at the top of the page.

Otherwise, when nested within said elements, it loses its landmark status and represents a group of introductory or navigational aids for the surrounding section. It usually contains the surrounding section's heading (an `h1` – `h6` element) and optional subheading, but this is **not** required.

### Historical Usage

The `<header>` element originally existed at the very beginning of HTML for headings. It is seen in the very first website. At some point, headings became `<h1>` through `<h6>`, allowing `<header>` to be free to fill a different role.

### Attributes

This element only includes the global attributes.

### Accessibility

The `<header>` element defines a `banner` landmark when its context is the `<body>` element.

When placed inside an `<article>`, `<main>`, `<section>`, `<nav>`, `<aside>`, or an element with the same ARIA role as these elements' implicit ARIA role, the `<header>` element has the `generic` role instead, and is no longer considered a landmark. In this case, it cannot be labeled with `aria-label` or `aria-labelledby`.

### Examples

#### Page Header

```
<header>
  <h1>Main Page Title</h1>
  <img src="https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/header/mdn-logo-sm.png" alt="MDN logo" />
</header>
```

#### Result

#### Article Header

```
<article>
  <header>
    <h2>The Planet Earth</h2>
    <p>
      Posted on Wednesday, <time datetime="2017-10-04">4 October 2017</time> by
      Jane Smith
    </p>
  </header>
  <p>
    We live on a planet that's blue and green, with so many things still unseen.
  </p>
  <p><a href="https://example.com/the-planet-earth/">Continue reading…</a></p>
</article>
```

#### Result

### Technical summary

| | |
|---|---|
| **Content categories** | Flow content, palpable content. |
| **Permitted content** | Flow content, but with no `<header>` or `<footer>` descendant. |
| **Tag omission** | None, both the starting and ending tag are mandatory. |
| **Permitted parents** | Any element that accepts flow content. Note that a `<header>` element must not be a descendant of an `<address>`, `<footer>` or another `<header>` element. |
| **Implicit ARIA role** | banner, or generic if a descendant of an `article`, `aside`, `main`, `nav` or `section` element, or an element with `article`, `complementary`, `main`, `navigation` or `region` role |
| **Permitted ARIA roles** | `group`, `presentation` or `none` |
| **DOM interface** | `HTMLElement` |

### See also

- Other section-related elements: `<body>`, `<nav>`, `<article>`, `<aside>`, h1, h2, h3, h4, h5, h6, `<footer>`, `<section>`, `<address>`.

## <h1>–<h6> HTML section heading elements

The **`<h1>`** to **`<h6>`** HTML elements represent six levels of section headings. `<h1>` is the highest section level and `<h6>` is the lowest. By default, all heading elements create a block-level box in the layout, starting on a new line and taking up the full width available in their containing block.

### Try it

```
<h1>Beetles</h1>
<h2>External morphology</h2>
<h3>Head</h3>
<h4>Mouthparts</h4>
<h3>Thorax</h3>
<h4>Prothorax</h4>
<h4>Pterothorax</h4>
```

```
h1,
h2,
h3,
h4 {
  margin: 0.1rem 0;
}

h1 {
  font-size: 2rem;
}

h2 {
  font-size: 1.5rem;
  padding-left: 20px;
}

h3 {
  font-size: 1.2rem;
  padding-left: 40px;
}

h4 {
  font-size: 1rem;
  font-style: italic;
  padding-left: 60px;
}
```

### Attributes

These elements only include the global attributes.

The `headingoffset` and `headingreset` global attributes can be used to adjust the computed heading level of these elements.

### Usage notes

- Heading information can be used by user agents to construct a table of contents for a document automatically.
- Do not use heading elements to resize text. Instead, use the CSS `font-size` property.
- Do not skip heading levels: always start from `<h1>`, followed by `<h2>` and so on.

### Avoid using multiple <code><h1></code> elements on one page

While using multiple `<h1>` elements on one page is allowed by the HTML standard (as long as they are not nested), this is not considered a best practice. A page should generally have a single `<h1>` element that describes the content of the page (similar to the document's `<title>` element).

> **Note:** Nesting multiple `<h1>` elements in nested sectioning elements was allowed in older versions of the HTML standard. However, this was never considered a best practice and is now non-conforming. Read more in There Is No Document Outline Algorithm.

Prefer using only one `<h1>` per page and nest headings without skipping levels.

### Specifying a uniform font size for <code><h1></code>

Before May 2025, the HTML standard specified that `<h1>` elements in a `<section>`, `<article>`, `<aside>`, or `<nav>` element should render as an `<h2>` (smaller `font-size` with an adjusted `margin-block`), or as an `<h3>` if nested another level, and so on. This special context-dependent default style has now been removed.

To ensure consistent `<h1>` rendering for browsers that implement the old context-dependent default style, use the following style rule:

```
h1 {
  margin-block: 0.67em;
  font-size: 2em;
}
```

Alternatively, to avoid overwriting other style rules that target `<h1>` you can use `:where()`, which has zero specificity:

```
:where(h1) {
  margin-block: 0.67em;
  font-size: 2em;
}
```

### Accessibility

### Navigation

A common navigation technique for users of screen reading software is to quickly jump from heading to heading in order to determine the content of the page. Because of this, it is important to not skip one or more heading levels. Doing so may create confusion, as the person navigating this way may be left wondering where the missing heading is.

**Don't do this:**

```
<h1>Heading level 1</h1>
<h3>Heading level 3</h3>
<h4>Heading level 4</h4>
```

**Prefer this:**

```
<h1>Heading level 1</h1>
<h2>Heading level 2</h2>
<h3>Heading level 3</h3>
```

### Nesting

Headings may be nested as subsections to reflect the organization of the content of the page. Most screen readers can also generate an ordered list of all the headings on a page, which can help a person quickly determine the content hierarchy and navigate to different headings.

Given the following page structure:

```
<h1>Beetles</h1>

<h2>Etymology</h2>

<h2>Distribution and Diversity</h2>

<h2>Evolution</h2>
<h3>Late Paleozoic</h3>
<h3>Jurassic</h3>
<h3>Cretaceous</h3>
<h3>Cenozoic</h3>

<h2>External Morphology</h2>
<h3>Head</h3>
<h4>Mouthparts</h4>
<h3>Thorax</h3>
<h4>Prothorax</h4>
<h4>Pterothorax</h4>
<h3>Legs</h3>
<h3>Wings</h3>
<h3>Abdomen</h3>
```

Screen readers would generate a list like this:

1. `h1` Beetles
    1. `h2` Etymology
    2. `h2` Distribution and Diversity
    3. `h2` Evolution
        1. `h3` Late Paleozoic

        2. h3 Jurassic
        3. h3 Cretaceous
        4. h3 Cenozoic
    4. h2 External Morphology
        1. h3 Head
            1. h4 Mouthparts
        2. h3 Thorax
            1. h4 Prothorax
            2. h4 Pterothorax
        3. h3 Legs
        4. h3 Wings
        5. h3 Abdomen

When headings are nested, heading levels may be "skipped" when closing a subsection.

- Headings • Page Structure • WAI Web Accessibility Tutorials
- MDN Understanding WCAG, Guideline 1.3 explanations
- Understanding Success Criterion 1.3.1 | W3C Understanding WCAG 2.0
- MDN Understanding WCAG, Guideline 2.4 explanations
- Understanding Success Criterion 2.4.1 | W3C Understanding WCAG 2.0
- Understanding Success Criterion 2.4.6 | W3C Understanding WCAG 2.0
- Understanding Success Criterion 2.4.10 | W3C Understanding WCAG 2.0

### Labeling section content

Another common navigation technique for users of screen reading software is to generate a list of sectioning content and use it to determine the page's layout.

Sectioning content can be labeled using a combination of the `aria-labelledby` and `id` attributes, with the label concisely describing the purpose of the section. This technique is useful for situations where there is more than one sectioning element on the same page.

### Sectioning content examples

```
<header>
  <nav aria-labelledby="primary-navigation">
    <h2 id="primary-navigation">Primary navigation</h2>
    <!-- navigation items -->
  </nav>
</header>

<!-- page content -->

<footer>
  <nav aria-labelledby="footer-navigation">
    <h2 id="footer-navigation">Footer navigation</h2>
    <!-- navigation items -->
  </nav>
</footer>
```

In this example, screen reading technology would announce that there are two `<nav>` sections, one called "Primary navigation" and one called "Footer navigation". If labels were not provided, the person using screen reading software may have to investigate each `nav` element's contents to determine their purpose.

- Using the aria-labelledby attribute
- Labeling Regions • Page Structure • W3C WAI Web Accessibility Tutorials

### Examples

### All headings

The following code shows all the heading levels, in use.

```
<h1>Heading level 1</h1>
<h2>Heading level 2</h2>
<h3>Heading level 3</h3>
<h4>Heading level 4</h4>
```

```
<h5>Heading level 5</h5>
<h6>Heading level 6</h6>
```

### Example page

The following code shows a few headings with some content under them.

```
<h1>Heading elements</h1>
<h2>Summary</h2>
<p>Some text here…</p>

<h2>Examples</h2>
<h3>Example 1</h3>
<p>Some text here…</p>

<h3>Example 2</h3>
<p>Some text here…</p>

<h2>See also</h2>
<p>Some text here…</p>
```

### Technical summary

| | |
|---|---|
| **Content categories** | Flow content, heading content, palpable content. |
| **Permitted content** | Phrasing content. |
| **Tag omission** | None, both the starting and ending tag are mandatory. |
| **Permitted parents** | Any element that accepts flow content. |
| **Implicit ARIA role** | heading |
| **Permitted ARIA roles** | `tab`, `presentation` or `none` |
| **DOM interface** | `HTMLHeadingElement` |

### Browser compatibility

### See also

- `<p>`
- `<div>`
- `<section>`

## <hgroup> HTML heading group element

The **`<hgroup>`** HTML element represents a heading and related content. It groups a single `<h1>`–`<h6>` element with one or more `<p>`.

### Try it

```
<hgroup>
  <h1>Frankenstein</h1>
  <p>Or: The Modern Prometheus</p>
</hgroup>
<p>
  Victor Frankenstein, a Swiss scientist, has a great ambition: to create
  intelligent life. But when his creature first stirs, he realizes he has made a
  monster. A monster which, abandoned by his master and shunned by everyone who
  sees it, follows Dr Frankenstein to the very ends of the earth.
</p>
```

```
hgroup {
  text-align: right;
  padding-right: 16px;
  border-right: 10px solid #00c8d7;
}

hgroup h1 {
  margin-bottom: 0;
}

hgroup p {
  margin: 0;
  font-weight: bold;
}
```

### Attributes

This element only includes the global attributes.

### Usage notes

The `<hgroup>` element allows the grouping of a heading with any secondary content, such as subheadings, an alternative title, or tagline. Each of these types of content represented as a `<p>` element within the `<hgroup>`.

The `<hgroup>` itself has no impact on the document outline of a web page. Rather, the single allowed heading within the `<hgroup>` contributes to the document outline.

### Examples

```
<!doctype html>
<title>HTML Standard</title>
<body>
  <hgroup id="document-title">
    <h1>HTML: Living Standard</h1>
    <p>Last Updated 12 July 2022</p>
  </hgroup>
  <p>Some intro to the document.</p>
  <h2>Table of contents</h2>
  <ol id="toc">
    …
  </ol>
  <h2>First section</h2>
  <p>Some intro to the first section.</p>
</body>
```

### Result

### Technical summary

| | |
|---|---|
| **Content categories** | Flow content, heading content, palpable content. |
| **Permitted content** | Zero or more `<p>` elements, followed by one h1, h2, h3, h4, h5, or h6 element, followed by zero or more `<p>` elements. |
| **Tag omission** | None, both the starting and ending tag are mandatory. |
| **Permitted parents** | Any element that accepts flow content. |
| **Implicit ARIA role** | `group` |
| **Permitted ARIA roles** | Any |
| **DOM interface** | `HTMLElement` |

### See also

- Others section-related elements: `<body>`, `<article>`, `<section>`, `<aside>`, h1, h2, h3, h4, h5, h6, `<nav>`, `<header>`, `<footer>`, `<address>`;
- Sections and outlines of an HTML document.

## <hr> HTML thematic break (horizontal rule) element

The **`<hr>`** HTML element represents a thematic break between elements: for example, a change of scene in a story, or a shift of topic within a section.

### Try it

```
<p>§1: The first rule of Fight Club is: You do not talk about Fight Club.</p>

<hr />

<p>§2: The second rule of Fight Club is: Always bring cupcakes.</p>
```

```
hr {
  border: none;
  border-top: 3px double #333333;
  color: #333333;
  overflow: visible;
  text-align: center;
  height: 5px;
}

hr::after {
```

```
  background: white;
  content: "§";
  padding: 0 4px;
  position: relative;
  top: -13px;
}
```

### Attributes

This element's attributes include the global attributes.

`align`

Sets the alignment of the rule on the page. If no value is specified, the default value is `left`.

`color`

Sets the color of the rule through color name or hexadecimal value.

`noshade`

Sets the rule to have no shading.

`size`

Sets the height, in pixels, of the rule.

`width`

Sets the length of the rule on the page through a pixel or percentage value.

### Usage notes

Historically, the `<hr>` element has always been presented as a horizontal rule or line. While it may still be displayed as a horizontal rule in visual browsers, this element is now defined in semantic terms, rather than presentational terms. Therefore, if you wish to draw a horizontal line, you should do so by adding a border to an existing element using CSS.

The `border-*` properties (for example, `border-style` and `border-color`) allow you to significantly customize a line's appearance, whether you are customizing an `<hr>` element or a border drawn on a different element.

### Example

#### Thematic break between paragraphs

The following example adds a thematic break between paragraph-level elements.

#### HTML

```
<article>
  <p>
    This is the first paragraph of text. This is the first paragraph of text.
    This is the first paragraph of text. This is the first paragraph of text.
  </p>
  <hr />
  <p>
    This is the second paragraph of text. This is the second paragraph of text.
    This is the second paragraph of text. This is the second paragraph of text.
  </p>
</article>
```

#### Result

#### Thematic break between list items

The `<hr>` tag can be placed within a list item for visual separation, to create a separator between sections of a list.

#### HTML

```
<ul>
  <li>Cut</li>
  <li>Copy</li>
  <li>Paste</li>
  <li role="presentation"><hr /></li>
  <li>Delete</li>
</ul>
```

### Result

### Thematic break between select options

The `<hr>` element is allowed inside a `<select>` element to create a visual separator between `<option>` elements.

### HTML

```
<select>
  <option value="">--Choose an option--</option>
  <hr />
  <option value="option1">Option 1</option>
  <option value="option2">Option 2</option>
  <hr />
  <option value="option3">Option 3</option>
  <option value="option4">Option 4</option>
</select>
```

### Result

### Technical summary

| | |
|---|---|
| **Content categories** | Flow content. |
| **Permitted content** | None; it is a void element. |
| **Tag omission** | Must have a start tag and must not have an end tag. |
| **Permitted parents** | • Any element that accepts flow content<br>• `<select>` element |
| **Implicit ARIA role** | `separator` |
| **Permitted ARIA roles** | `presentation` or `none` |
| **DOM interface** | `HTMLHRElement` |

### See also

- `<p>`
- `<hr>` in `<select>`

## <html> HTML document root element

The **`<html>`** HTML element represents the root (top-level element) of an HTML document, so it is also referred to as the *root element*. All other elements must be descendants of this element. There can be only one `<html>` element in a document.

### Attributes

This element includes the global attributes.

`version`

> Specifies the version of the HTML Document Type Definition that governs the current document. This attribute is not needed, because it is redundant with the version information in the document type declaration.

`xmlns`

> Specifies the XML Namespace of the document. Default value is `"http://www.w3.org/1999/xhtml"`. This is required in documents parsed with XML parsers, and optional in text/html documents.

### Accessibility

While HTML does not require authors to specify `<html>` element start and ending tags, it is important for authors to do so as it will allow them to specify the `lang` for the webpage. Providing a `lang` attribute with a valid BCP 47 language tag on the `<html>` element will help screen reading technology determine the proper language to announce. The identifying language tag should describe the language used by the majority of the content of the page. Without it, screen readers will typically default to the operating system's set language, which may cause mispronunciations.

Including a valid `lang` declaration on the `<html>` element also ensures that important metadata contained in the page's `<head>`, such as the page's `<title>`, are also announced properly.

- MDN Understanding WCAG, Guideline 3.1 explanations
- Understanding Success Criterion 3.1.1 | W3C Understanding WCAG 2.1

### Example

```
<!doctype html>
<html lang="en">
  <head>
    <!-- … -->
  </head>
  <body>
    <!-- … -->
  </body>
</html>
```

### Technical summary

| | |
|---|---|
| **Content categories** | None. |
| **Permitted content** | One `<head>` element, followed by one `<body>` element. |
| **Tag omission** | The start tag may be omitted if the first thing inside the `<html>` element is not a comment.<br>The end tag may be omitted if the `<html>` element is not immediately followed by a comment. |
| **Permitted parents** | None. This is the root element of a document. |
| **Implicit ARIA role** | document |
| **Permitted ARIA roles** | No `role` permitted |
| **DOM interface** | `HTMLHtmlElement` |

### See also

- MathML top-level element: `<math>`
- SVG top-level element: `<svg>`

## <i> HTML idiomatic text element

The **`<i>`** HTML element represents a range of text that is set off from the normal text for some reason, such as idiomatic text, technical terms, taxonomical designations, among others. Historically, these have been presented using italicized type, which is the original source of the `<i>` naming of this element.

### Try it

```
<p>I looked at it and thought <i>This can't be real!</i></p>

<p>
  <i>Musa</i> is one of two or three genera in the family <i>Musaceae</i>; it
  includes bananas and plantains.
</p>

<p>
  The term <i>bandwidth</i> describes the measure of how much information can
  pass through a data connection in a given amount of time.
</p>
```

```
i {
  /* Add your styles here */
}
```

### Attributes

This element only includes the global attributes.

### Usage notes

- Use the `<i>` element for text that is set off from the normal prose for readability reasons. This would be a range of text with different semantic meaning than the surrounding text. Among the use cases for the `<i>` element are spans of text representing a different quality or mode of text, such as:
    - Alternative voice or mood
    - Taxonomic designations (such as the genus and species "*Homo sapiens*")
    - Idiomatic terms from another language (such as "*et cetera*"); these should include the `lang` attribute to identify the language
    - Technical terms
    - Transliterations
    - Thoughts (such as "She wondered, *What is this writer talking about, anyway?*")
    - Ship or vessel names in Western writing systems (such as "They searched the docks for the *Empress of the Galaxy*, the ship to which they were assigned.")

- In earlier versions of the HTML specification, the `<i>` element was merely a presentational element used to display text in italics, much like the `<b>` element was used to display text in bold letters. This is no longer true, as these tags now define semantics rather than typographic appearance. A browser will typically still display the contents of the `<i>` element in italic type, but is, by definition, no longer required to do so. To display text in italic type, authors should use the CSS `font-style` property.
- Be sure the text in question is not actually more appropriately marked up with another element.
  - Use `<em>` to indicate stress emphasis.
  - Use `<strong>` to indicate importance, seriousness, or urgency.
  - Use `<mark>` to indicate relevance.
  - Use `<cite>` to mark up the name of a work, such as a book, play, or song.
  - Use `<dfn>` to mark up the defining instance of a term.

### Examples

This example demonstrates using the `<i>` element to mark text that is in another language.

```
<p>
  The Latin phrase <i lang="la">Veni, vidi, vici</i> is often mentioned in
  music, art, and literature.
</p>
```

### Result

### Technical summary

| | |
|---|---|
| **Content categories** | Flow content, phrasing content, palpable content. |
| **Permitted content** | Phrasing content. |
| **Tag omission** | None, both the starting and ending tag are mandatory. |
| **Permitted parents** | Any element that accepts phrasing content. |
| **Implicit ARIA role** | `generic` |
| **Permitted ARIA roles** | Any |
| **DOM interface** | `HTMLElement` |

### See also

- `<em>`
- Other italicized elements: `<var>`, `<dfn>`, `<cite>`, `<address>`

## <iframe> HTML inline frame element

The **`<iframe>`** HTML element represents a nested browsing context, embedding another document into the current one.

### Try it

```
<iframe
  id="inlineFrameExample"
  title="Inline Frame Example"
  width="300"
  height="200"
  src="https://www.openstreetmap.org/export/embed.html?
bbox=-0.004017949104309083%2C51.47612752641776%2C0.0003057718276977539 6%2C51.478569861898606&amp;layer=mapnik">
</iframe>
```

```
iframe {
  border: 1px solid black;
  width: 100%; /* takes precedence over the width set with the HTML width attribute */
}
```

### Attributes

This element includes the global attributes.

`allow`

Specifies a Permissions Policy for the `<iframe>`. The policy defines what features are available to the `<iframe>` (for example, access to the microphone, camera, battery, web-share, etc.) based on the origin of the request.

See iframes in the `Permissions-Policy` topic for examples.

> **Note:** A Permissions Policy specified by the `allow` attribute implements a further restriction on top of the policy specified in the `Permissions-Policy` header. It doesn't replace it.

`allowfullscreen`

Set to `true` if the `<iframe>` can activate fullscreen mode by calling the `requestFullscreen()` method.

> **Note:** This attribute is considered a legacy attribute and redefined as `allow="fullscreen *"`.

`allowpaymentrequest`

Set to `true` if a cross-origin `<iframe>` should be allowed to invoke the Payment Request API.

> **Note:** This attribute is considered a legacy attribute and redefined as `allow="payment *"`.

`browsingtopics`

A boolean attribute that, if present, specifies that the selected topics for the current user should be sent with the request for the `<iframe>`'s source.

`credentialless`

Set to `true` to make the `<iframe>` credentialless, meaning that its content will be loaded in a new, ephemeral context. It doesn't have access to the network, cookies, and storage data associated with its origin. It uses a new context local to the top-level document lifetime. In return, the `Cross-Origin-Embedder-Policy` (COEP) embedding rules can be lifted, so documents with COEP set can embed third-party documents that do not. See IFrame credentialless for more details.

`csp`

A Content Security Policy enforced for the embedded resource. See `HTMLIFrameElement.csp` for details.

`height`

The height of the frame in CSS pixels. Default is `150`.

`loading`

Indicates when the browser should load the iframe:

`eager`

Load the iframe immediately on page load (this is the default value).

`lazy`

Defer loading of the iframe until it reaches a calculated distance from the visual viewport, as defined by the browser. The intent is to avoid using the network and storage bandwidth required to fetch the frame until the browser is reasonably certain that it will be needed. This improves the performance and cost in most typical use cases, in particular by reducing initial page load times.

Loading is only deferred when JavaScript is enabled. This is an anti-tracking measure, because if a user agent supported lazy loading when scripting is disabled, it would still be possible for a site to track a user's approximate scroll position throughout a session, by strategically placing iframes in a page's markup such that a server can track how many iframes are requested and when.

`name`

A targetable name for the embedded browsing context. This can be used in the `target` attribute of the `<a>`, `<form>`, or `<base>` elements; the `formtarget` attribute of the `<input>` or `<button>` elements; or the `windowName` parameter in the `window.open()` method. In addition, the name becomes a property of the `Window` and `Document` objects, containing a reference to the embedded window or the element itself.

`privateToken`

Contains a string representation of an options object representing a private state token operation; this object has the same structure as the `RequestInit` dictionary's `privateToken` property. IFrames containing this attribute can initiate operations such as issuing or redeeming tokens when their embedded content is loaded.

`referrerpolicy`

Indicates which referrer to send when fetching the frame's resource:

`no-referrer`

The `Referer` header will not be sent.

`no-referrer-when-downgrade`

The `Referer` header will not be sent to origins without TLS (HTTPS).

`origin`

The sent referrer will be limited to the origin of the referring page: its scheme, host, and port.

`origin-when-cross-origin`

The referrer sent to other origins will be limited to the scheme, the host, and the port. Navigations on the same origin will still include the path.

`same-origin`

A referrer will be sent for same origin, but cross-origin requests will contain no referrer information.

`strict-origin`

Only send the origin of the document as the referrer when the protocol security level stays the same (HTTPS→HTTPS), but don't send it to a less secure destination (HTTPS→HTTP).

`strict-origin-when-cross-origin` (default)

Send a full URL when performing a same-origin request, only send the origin when the protocol security level stays the same (HTTPS→HTTPS), and send no header to a less secure destination (HTTPS→HTTP).

`unsafe-url`

The referrer will include the origin *and* the path (but not the fragment, password, or username). **This value is unsafe**, because it leaks origins and paths from TLS-protected resources to insecure origins.

`sandbox`

Controls the restrictions applied to the content embedded in the `<iframe>`. The value of the attribute can either be empty to apply all restrictions, or space-separated tokens to lift particular restrictions:

`allow-downloads`

Allows downloading files through an `<a>` or `<area>` element with the download attribute, as well as through the navigation that leads to a download of a file. This works regardless of whether the user clicked on the link, or JS code initiated it without user interaction.

`allow-forms`

Allows the page to submit forms. If this keyword is not used, a form will be displayed as normal, but submitting it will not trigger input validation, send data to a web server, or close a dialog.

`allow-modals`

Allows the page to open modal windows by `Window.alert()`, `Window.confirm()`, `Window.print()` and `Window.prompt()`, while opening a `<dialog>` is allowed regardless of this keyword. It also allows the page to receive `BeforeUnloadEvent` event.

`allow-orientation-lock`

Lets the resource lock the screen orientation.

`allow-pointer-lock`

Allows the page to use the Pointer Lock API.

`allow-popups`

Allows popups (created, for example, by `Window.open()` or `target="_blank"`). If this keyword is not used, such functionality will silently fail.

`allow-popups-to-escape-sandbox`

Allows a sandboxed document to open a new browsing context without forcing the sandboxing flags upon it. This will allow, for example, a third-party advertisement to be safely sandboxed without forcing the same restrictions upon the page the ad links to. If this flag is not included, a redirected page, popup window, or new tab will be subject to the same sandbox restrictions as the originating `<iframe>`.

`allow-presentation`

Allows embedders to have control over whether an iframe can start a presentation session.

`allow-same-origin`

If this token is not used, the resource is treated as being from a special origin that always fails the same-origin policy (potentially preventing access to data storage/cookies and some JavaScript APIs).

> **Note:** When `allow-same-origin` is present, a same-origin parent document can still access and interact with the iframe's DOM even if `allow-scripts` is not set. The `allow-scripts` token only controls script execution within the embedded browsing context and does not affect DOM access from the parent.

`allow-scripts`

Allows the page to run scripts (but not create pop-up windows). If this keyword is not used, this operation is not allowed.

`allow-storage-access-by-user-activation`

Allows a document loaded in the `<iframe>` to use the Storage Access API to request access to unpartitioned cookies.

`allow-top-navigation`

Lets the resource navigate the top-level browsing context (the one named `_top`).

`allow-top-navigation-by-user-activation`

Lets the resource navigate the top-level browsing context, but only if initiated by a user gesture.

`allow-top-navigation-to-custom-protocols`

Allows navigations to non-`http` protocols built into browser or registered by a website. This feature is also activated by `allow-popups` or `allow-top-navigation` keyword.

> **Note:**
>
> - The `sandbox` attribute can block the browser's built-in PDF viewer. See Embedding PDFs.
> - When the embedded document has the same origin as the embedding page, it is **strongly discouraged** to use both `allow-scripts` and `allow-same-origin`, as that lets the embedded document remove the `sandbox` attribute — making it no more secure than not using the `sandbox` attribute at all.
> - Sandboxing is useless if the attacker can display content outside a sandboxed `iframe` — such as if the viewer opens the frame in a new tab. Such content should be also served from a *separate origin* to limit potential damage.

> **Note:** When redirecting the user, opening a popup window, or opening a new tab from an embedded page within an `<iframe>` with the `sandbox` attribute, the new browsing context is subject to the same `sandbox` restrictions. This can create issues — for example, if a page embedded within an `<iframe>` without a `sandbox="allow-forms"` or `sandbox="allow-popups-to-escape-sandbox"` attribute set on it opens a new site in a separate tab, form submission in that new browsing context will silently fail.

`src`

The URL of the page to embed. Use a value of `about:blank` to embed an empty page that conforms to the same-origin policy. Also note that programmatically removing an `<iframe>`'s src attribute (e.g., via `Element.removeAttribute()`) causes `about:blank` to be loaded in the frame in Firefox (from version 65), Chromium-based browsers, and Safari/iOS.

> **Note:** The `about:blank` page uses the embedding document's URL as its base URL when resolving any relative URLs, such as anchor links.

`srcdoc`

Inline HTML to embed, overriding the `src` attribute. Its content should follow the syntax of a full HTML document, which includes the doctype directive, `<html>`, `<body>` tags, etc., although most of them can be omitted, leaving only the body content. This doc will have `about:srcdoc` as its location. If a browser does not support the `srcdoc` attribute, it will fall back to the URL in the `src` attribute.

> **Note:** The `about:srcdoc` page uses the embedding document's URL as its base URL when resolving any relative URLs, such as anchor links.

`width`

The width of the frame in CSS pixels. Default is `300`.

### Deprecated attributes

These attributes are deprecated and may no longer be supported by all user agents. You should not use them in new content, and try to remove them from existing content.

`align`

The alignment of this element with respect to the surrounding context.

`frameborder`

The value `1` (the default) draws a border around this frame. The value `0` removes the border around this frame, but you should instead use the CSS property `border` to control `<iframe>` borders.

`longdesc`

A URL of a long description of the frame's content. Due to widespread misuse, this is not helpful for non-visual browsers.

`marginheight`

The amount of space in pixels between the frame's content and its top and bottom borders.

`marginwidth`

The amount of space in pixels between the frame's content and its left and right borders.

`scrolling`

Indicates when the browser should provide a scrollbar for the frame:

`auto`

Only when the frame's content is larger than its dimensions.

`yes`

Always show a scrollbar.

`no`

Never show a scrollbar.

### Usage notes

Each embedded browsing context has its own document and allows URL navigations. The navigations of each embedded browsing context are linearized into the session history of the *topmost* browsing context. The browsing context that embeds the others is called the *parent browsing context*. The *topmost* browsing context — the one with no parent — is usually the browser window, represented by the `Window` object.

> **Warning:** Because each browsing context is a complete document environment, every `<iframe>` in a page requires increased memory and other computing resources. While theoretically you can use as many `<iframe>`s as you like, check for performance problems.

### Scripting

Inline frames, like `<frame>` elements, are included in the `window.frames` pseudo-array.

With the DOM `HTMLIFrameElement` object, scripts can access the `window` object of the framed resource via the `contentWindow` property. The `contentDocument` property refers to the `document` inside the `<iframe>`, same as `contentWindow.document`.

From the inside of a frame, a script can get a reference to its parent window with `window.parent`.

Script access to a frame's content is subject to the same-origin policy. Scripts cannot access most properties in other `window` objects if the script was loaded from a different origin, including scripts inside a frame accessing the frame's parent. Cross-origin communication can be achieved using `Window.postMessage()`.

### Top navigation in cross-origin frames

Scripts running in a same-origin frame can access the `Window.top` property and set `window.top.location` to redirect the top-level page to a new location. This behavior is referred to as "top navigation".

A cross-origin frame is allowed to redirect the top-level page using `top` only if the frame has sticky activation. If top navigation is blocked, browsers may either prompt for user permission to redirect or report the error in the developer console (or both). This restriction by browsers is called *framebusting intervention*. What this means is that a cross-origin frame can't immediately redirect the top-level page — the user must have previously interacted with the frame or granted permission to redirect.

A sandboxed frame blocks all top navigation unless the `sandbox` attribute values are set to `allow-top-navigation` or `allow-top-navigation-by-user-activation`. Note that top-navigation permissions are inherited, so a nested frame can perform a top navigation only if its parent frames are also

allowed to.

### Embedding PDFs

An `<iframe>` can display a PDF using the browser's built-in PDF viewer. Unlike `<object>`, it does not support child content as a fallback if the PDF cannot be displayed. Provide a link outside the `<iframe>` so users can open the PDF separately.

The `sandbox` attribute can prevent the built-in PDF viewer from loading, even with `allow-scripts` or `allow-downloads`. It is not a portable way to add restrictions to a native PDF preview. The browser's PDF viewer already sandboxes all executable content.

### Positioning and scaling

Being a replaced element, the `<iframe>` allows the position of the embedded document within its box to be adjusted using the `object-position` property.

> **Note:** The `object-fit` property has no effect on `<iframe>` elements.

### <code>error</code> and <code>load</code> event behavior

The `error` and `load` events fired on `<iframe>`s could be used to probe the URL space of the local network's HTTP servers. Therefore, as a security precaution user agents do not fire the error event on `<iframe>`s, and the load event is always triggered even if the `<iframe>` content fails to load.

### Responsive <code><iframe></code> sizing

For security and privacy reasons, `<iframe>` elements do not by default expose any information to the parent document about the size of the content in the document they are embedding.

To enable responsive sizing of `<iframe>` elements based on their content, the `<meta name="responsive-embedded-sizing">` tag can be included in an embedded document to opt it in to sharing its size information with the parent document. The `frame-sizing` CSS property can then be set on the `<iframe>` to cause it to adopt the same horizontal or vertical size as the embedded document's actual content size. This ensures that `<iframe>` content fits seamlessly into its embedder, avoiding unnecessary scrollbars.

To resize the `<iframe>` dynamically as the embedded document changes layout size, you can call the `Window.requestResize()` method from the embedded document to make it report an updated size.

### Accessibility

People navigating with assistive technology such as a screen reader can use the `title` attribute on an `<iframe>` to label its content. The title's value should concisely describe the embedded content:

```
<iframe
  title="Wikipedia page for Avocados"
  src="https://en.wikipedia.org/wiki/Avocado"></iframe>
```

Without this title, they have to navigate into the `<iframe>` to determine what its embedded content is. This context shift can be confusing and time-consuming, especially for pages with multiple `<iframe>`s and/or if embeds contain interactive content like video or audio.

### Examples

### A basic <iframe>

This example embeds the page at https://example.org in an iframe. This is a common use case of iframes: to embed content from another site. For example, the live sample itself, and the try it example at the top, are both `<iframe>` embeds of content from another MDN site.

### HTML

```
<iframe
  src="https://example.org"
  title="iframe Example 1"
  width="400"
  height="300">
</iframe>
```

### Result

### Embedding source code in an <iframe>

This example directly renders source code in an iframe. This can be used as a technique to prevent script injection when displaying user-generated content, when combined with the `sandbox` attribute.

Note that when using `srcdoc`, any relative URLs in the embedded content will be resolved relative to the URL of the embedding page. If you want to use anchor links that point to places in the embedded content, you need to explicitly specify `about:srcdoc` as the base URL.

### HTML

```
<article>
  <footer>Nine minutes ago, <i>jc</i> wrote:</footer>
  <iframe
    sandbox
    srcdoc="<p>There are two ways to use the <code>iframe</code> element:</p>
<ol>
<li><a href=&quot;about:srcdoc#embed_another&quot;>To embed content from another page</a></li>
<li><a href=&quot;about:srcdoc#embed_user&quot;>To embed user-generated content</a></li>
</ol>
<h2 id=&quot;embed_another&quot;>Embedding content from another page</h2>
<p>Use the <code>src</code> attribute to specify the URL of the page to embed:</p>
<pre><code>&amp;lt;iframe src=&quot;https://example.org&quot;&amp;gt;&amp;lt;/iframe&amp;gt;</code></pre>
<h2 id=&quot;embed_user&quot;>Embedding user-generated content</h2>
<p>Use the <code>srcdoc</code> attribute to specify the content to embed. This post is already an example!</p>
"
    width="500"
    height="250"
></iframe>
</article>
```

Here's how to write escape sequences when using `srcdoc`:

- First, write the HTML out, escaping anything you would escape in a normal HTML document (such as `<`, `>`, `&`, etc.).
- `<` and `<` represent the exact same character in the `srcdoc` attribute. Therefore, to make it an actual escape sequence in the HTML document, replace any ampersands (`&`) with `&`. For example, `<` becomes `<`, and `&` becomes `&`.
- Replace any double quotes (`"`) with `"` to prevent the `srcdoc` attribute from being prematurely terminated (if you use `'` instead, then you should replace `'` with `'` instead). This step happens after the previous one, so `"` generated in this step doesn't become `"`.

### Result

### Technical summary

| | |
|---|---|
| **Content categories** | Flow content, phrasing content, embedded content, interactive content, palpable content. |
| **Permitted content** | None. |
| **Tag omission** | None, both the starting and ending tags are mandatory. |
| **Permitted parents** | Any element that accepts embedded content. |
| **Implicit ARIA role** | No corresponding role |
| **Permitted ARIA roles** | `application`, `document`, `img`, `none`, `presentation` |
| **DOM interface** | `HTMLIFrameElement` |

### See also

- CSP: frame-ancestors
- Privacy, permissions, and information security
- Local network access

## <img> HTML image embed element

The **`<img>`** HTML element embeds an image into the document.

### Try it

```
<img
  class="fit-picture"
  src="https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/img//shared-assets/images/examples/grapefruit-slice.jpg"
  alt="Grapefruit slice atop a pile of other slices" />
```

```
.fit-picture {
  width: 250px;
}
```

### Attributes

This element includes the global attributes.

`alt`

Defines text that can replace the image in the page.

> **Note:** Browsers do not always display images. There are a number of situations in which a browser might not display images, such as:
>
> - Non-visual browsers (such as those used by people with visual impairments)
> - The user chooses not to display images (saving bandwidth, privacy reasons)
> - The image is invalid or an unsupported type
>
> In these cases, the browser may replace the image with the text in the element's `alt` attribute. For these reasons and others, provide a useful value for `alt` whenever possible.

Setting this attribute to an empty string (`alt=""`) indicates that this image is *not* a key part of the content (it's decoration or a tracking pixel), and that non-visual browsers may omit it from rendering. Visual browsers will also hide the broken image icon if the `alt` attribute is empty and the image failed to display.

This attribute is also used when copying and pasting the image to text, or saving a linked image to a bookmark.

`attributionsrc`

Specifies that you want the browser to send an `Attribution-Reporting-Eligible` header along with the image request.

On the server-side this is used to trigger sending an `Attribution-Reporting-Register-Source` or `Attribution-Reporting-Register-Trigger` header in the response, to register an image-based attribution source or attribution trigger, respectively. Which response header should be sent back depends on the value of the `Attribution-Reporting-Eligible` header that triggered the registration.

The corresponding source or trigger event is set off once the browser receives the response containing the image file.

> **Note:** See the Attribution Reporting API for more details.

There are two versions of this attribute that you can set:

- Boolean, i.e., just the `attributionsrc` name. This specifies that you want the `Attribution-Reporting-Eligible` header sent to the same server as the `src` attribute points to. This is fine when you are handling the attribution source or trigger registration on the same server. When registering an attribution trigger this property is optional, and a boolean value will be used if it is omitted.
- Value containing one or more URLs, for example:

```
<img
  src="https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/img/image-file.png"
  alt="My image file description"
  attributionsrc="https://a.example/register-source
                  https://b.example/register-source" />
```

This is useful in cases where the requested resource is not on a server you control, or you just want to handle registering the attribution source on a different server. In this case, you can specify one or more URLs as the value of `attributionsrc`. When the resource request occurs the `Attribution-Reporting-Eligible` header will be sent to the URL(s) specified in `attributionSrc` in addition to the resource origin. These URLs can then respond with an `Attribution-Reporting-Register-Source` or `Attribution-Reporting-Register-Trigger` header as appropriate to complete registration.

> **Note:** Specifying multiple URLs means that multiple attribution sources can be registered on the same feature. You might for example have different campaigns that you are trying to measure the success of, which involve generating different reports on different data.

`crossorigin`

Indicates if the fetching of the image must be done using a CORS request. Image data from a CORS-enabled image returned from a CORS request can be reused in the `<canvas>` element without being marked "tainted".

If the `crossorigin` attribute is *not* specified, then a non-CORS request is sent (without the `Origin` request header), and the browser marks the image as tainted and restricts access to its image data, preventing its usage in `<canvas>` elements.

If the `crossorigin` attribute *is* specified, then a CORS request is sent (with the `Origin` request header); but if the server does not opt into allowing cross-origin access to the image data by the origin site (by not sending any `Access-Control-Allow-Origin` response header, or by not including the site's origin in any `Access-Control-Allow-Origin` response header it does send), then the browser blocks the image from loading, and logs a CORS error to the devtools console.

Allowed values:

`anonymous`

A CORS request is sent with credentials omitted (that is, no cookies, X.509 certificates, or `Authorization` request header).

`use-credentials`

The CORS request is sent with any credentials included (that is, cookies, X.509 certificates, and the `Authorization` request header). If the server does not opt into sharing credentials with the origin site (by sending back the `Access-Control-Allow-Credentials: true` response header), then the browser marks the image as tainted and restricts access to its image data.

If the attribute has an invalid value, browsers handle it as if the `anonymous` value was used. See CORS settings attributes for additional information.

`decoding`

This attribute provides a hint to the browser as to whether it should perform image decoding along with rendering the other DOM content in a single presentation step that looks more "correct" (`sync`), or render and present the other DOM content first and then decode the image and present it later (`async`). In practice, `async` means that the next paint does not wait for the image to decode.

It is often difficult to perceive any noticeable effect when using `decoding` on static `<img>` elements. They'll likely be initially rendered as empty images while the image files are fetched (either from the network or from the cache) and then handled independently anyway, so the "syncing" of content updates is less apparent. However, the blocking of rendering while decoding happens, while often quite small, *can* be measured — even if it is difficult to observe with the human eye. See What does the image decoding attribute actually do? for a more detailed analysis (tunetheweb.com, 2023).

Using different `decoding` types can result in more noticeable differences when dynamically inserting `<img>` elements into the DOM via JavaScript — see `HTMLImageElement.decoding` for more details.

Allowed values:

`sync`

Decode the image synchronously along with rendering the other DOM content, and present everything together.

`async`

Decode the image asynchronously, after rendering and presenting the other DOM content.

`auto`

No preference for the decoding mode; the browser decides what is best for the user. This is the default value.

`elementtiming`

Marks the image for observation by the `PerformanceElementTiming` API. The value given becomes an identifier for the observed image element. See also the `elementtiming` attribute page.

`fetchpriority`

Provides a hint of the relative priority to use when fetching the image. Allowed values:

`high`

Fetch the image at a high priority relative to other images.

`low`

Fetch the image at a low priority relative to other images.

`auto`

Don't set a preference for the fetch priority. This is the default. It is used if no value or an invalid value is set.

`height`

The intrinsic height of the image, in pixels. Must be an integer without a unit.

> **Note:** Including `height` and `width` enables the aspect ratio of the image to be calculated by the browser prior to the image being loaded. This aspect ratio is used to reserve the space needed to display the image, reducing or even preventing a layout shift when the image is downloaded and painted to the screen. Reducing layout shift is a major component of good user experience and web performance.

`ismap`

This Boolean attribute indicates that the image is part of a server-side map. If so, the coordinates where the user clicked on the image are sent to the server.

> **Note:** This attribute is allowed only if the `<img>` element is a descendant of an `<a>` element with a valid `href` attribute. This gives users without pointing devices a fallback destination.

`loading`

Indicates how the browser should load the image:

`eager`

Loads the image immediately, regardless of whether or not the image is currently within the visual viewport (this is the default value).

`lazy`

Defers loading the image until it reaches a calculated distance from the viewport, as defined by the browser.

Lazy loading avoids the network and storage bandwidth required to handle the image until it's reasonably certain that it will be needed. This improves the performance in most typical use cases.

While explicit `width` and `height` attributes are recommended for all images to avoid layout shift, they are especially important for lazy-loaded ones. Lazy-loaded images will never be loaded if they do not intersect a visible part of an element, even if loading them would change that, because unloaded images have a `width` and `height` of `0`. It creates an even more disruptive user experience when the content visible in the viewport reflows in the middle of reading it.

Lazy-loaded images located in the visual viewport may not yet be visible when the Window `load` event is fired. This is because the event is fired based on eager-loaded images — lazy-loaded images are not considered even if they are located within the visual viewport upon initial page load.

Loading is only deferred when JavaScript is enabled. This is an anti-tracking measure, because if a user agent supported lazy loading when scripting is disabled, it would still be possible for a site to track a user's approximate scroll position throughout a session, by strategically placing images in a page's markup such that a server can track how many images are requested and when.

`referrerpolicy`

A string indicating which referrer to use when fetching the resource:

- `no-referrer`: The `Referer` header will not be sent.
- `no-referrer-when-downgrade`: The `Referer` header will not be sent to origins without TLS (HTTPS).
- `origin`: The sent referrer will be limited to the origin of the referring page: its scheme, host, and port.
- `origin-when-cross-origin`: The referrer sent to other origins will be limited to the scheme, the host, and the port. Navigations on the same origin will still include the path.
- `same-origin`: A referrer will be sent for same origin, but cross-origin requests will contain no referrer information.
- `strict-origin`: Only send the origin of the document as the referrer when the protocol security level stays the same (HTTPS→HTTPS), but don't send it to a less secure destination (HTTPS→HTTP).
- `strict-origin-when-cross-origin` (default): Send a full URL when performing a same-origin request, only send the origin when the protocol security level stays the same (HTTPS→HTTPS), and send no header to a less secure destination (HTTPS→HTTP).
- `unsafe-url`: The referrer will include the origin *and* the path (but not the fragment, password, or username). **This value is unsafe**, because it leaks origins and paths from TLS-protected resources to insecure origins.

`sizes`

One or more values separated by commas, which can be source sizes or the `auto` keyword. The spec requires that the `sizes` attribute only be present when `srcset` uses width descriptors.

#### source size

A **source size** consists of:

1. A media condition, omitted for the last item in the list.
2. A source size value.

For example, the following source size proposes using a `1000px`-wide image source if the *viewport* width is 500px or less.

```
(width <= 500px) 1000px
```

Media conditions describe properties of the *viewport*, not the *image*. Because a source size descriptor specifies the width to use for the image during layout, the media condition is typically (but not necessarily) based on the `width`.

Source size values specify the intended display size of the image. User agents use the current source size to select one of the sources supplied by the `srcset` attribute, when those sources are described using width (`w`) descriptors. The `w` value defined in sizes determines

the image's default layout width. In the absence of CSS, the browser will render the image at this size, regardless of the physical pixel dimensions of the downloaded file.

A source size value can be any non-negative length. It must not use CSS functions other than the math functions. Units are interpreted in the same way as media queries, meaning that all relative length units are relative to the document root rather than the `<img>` element. For example, an `em` value is relative to the root font size, not the font size of the image. Percentage values are not allowed. If the `sizes` attribute is not provided, it has a default value of `100vw` (the viewport width).

`auto`

The `auto` keyword indicates that the browser should use the expected layout width of the element to select the image to display. That is, it should use the concrete size of the image, calculated after layout from HTML and CSS has been applied. This is only valid when combined with `loading="lazy"`, as the page is expected to already have CSS and other layout information by the time the image loads.

Using `auto` saves you having to specify your layout media conditions twice: once for layout, and once for selection of an appropriate image to fetch and display.

If `auto` cannot resolve — either because the browser does not support it, or because the image has no layout size yet — the browser falls back to the *source sizes* in the list to determine the width, then to `width`/`height` attributes defined on the element, and finally to the default intrinsic size for `<img>` elements defined in the user agent stylesheet (300px by 150px).

For better backward compatibility with browsers that do not support `auto`, you can include fallback sizes after `auto` in the `sizes` attribute. You should also set the element's `width` and `height` attributes to the intrinsic dimensions of the largest image in your `srcset`, so the browser can reserve space using the correct aspect ratio:

```
<img
  loading="lazy"
  width="200"
  height="200"
  sizes="auto, (max-width: 30em) 100vw, (max-width: 50em) 50vw, calc(33vw - 100px)"
  srcset="
    swing-200.jpg   200w,
    swing-400.jpg   400w,
    swing-800.jpg   800w,
    swing-1600.jpg 1600w
  "
  src="https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/img/swing-400.jpg"
  alt="Kettlebell Swing" />
```

`src`

The image URL. At least one of `src` and `srcset` is required for an `<img>` element. If `srcset` is specified, `src` is used in one of two ways:

- as a fallback for browsers that don't support `srcset`.
- if `srcset` uses the "x" descriptor, then `src` is equivalent to a source with the density descriptor `1x`; that is, the image specified by `src` is used on low-density screens (such as typical 72 DPI or 96 DPI displays).

`srcset`

One or more strings separated by commas, indicating possible image sources for the user agent to use.

Each string is composed of:

1. A URL to an image
2. Optionally, whitespace followed by one of:
   - A width descriptor (a positive integer directly followed by `w`). It *must* match the intrinsic width of the referenced image. The width descriptor is divided by the source size given in the `sizes` attribute to calculate the effective pixel density. For example, to provide an image resource to be used when the renderer needs a 450 pixel wide image, use the width descriptor `450w`. When a `srcset` contains "w" descriptors, the browser uses those descriptors together with the `sizes` attribute to pick a resource.
   - A pixel density descriptor (a positive floating point number directly followed by `x`). It specifies the condition in which the corresponding image resource should be used as the display's pixel density. For example, to provide an image resource to be used when the pixel density is double the standard density, use the pixel density descriptor `2x` or `2.0x`.

If no descriptor is specified, the source is assigned the default descriptor of `1x`. It is incorrect to mix width descriptors and pixel density descriptors in the same `srcset` attribute. Duplicate descriptors (for instance, two sources in the same `srcset` which are both described with `2x`) are also invalid.

Space characters, other than the whitespace separating the URL and the corresponding condition descriptor, are ignored; this includes both leading and trailing space, as well as space before or after each comma. However, if an image candidate string contains no descriptors and no whitespace after the URL, the following image candidate string, if there is one, must begin with one or more whitespace, or the comma will be considered part of the URL.

When the `<img>` element's `srcset` uses `x` descriptors, browsers also consider the URL in the `src` attribute (if present) as a candidate, and assign it a default descriptor of `1x`. On the other hand, if the `srcset` attribute uses width descriptors, `src` is not considered, and the `sizes` attribute is

used instead.

The user agent selects any of the available sources at its discretion. This provides them with significant leeway to tailor their selection based on things like user preferences or bandwidth conditions. See our Responsive images tutorial for an example.

`width`

The intrinsic width of the image in pixels. Must be an integer without a unit.

`usemap`

The partial URL (starting with `#`) of an image map associated with the element.

> **Note:** You cannot use this attribute if the `<img>` element is inside an `<a>` or `<button>` element.

### Deprecated attributes

`align`

Aligns the image with its surrounding context. Use the `float` and/or `vertical-align` CSS properties instead of this attribute. Allowed values:

`top`

Equivalent to `vertical-align: top` or `vertical-align: text-top`

`middle`

Equivalent to `vertical-align: -moz-middle-with-baseline`

`bottom`

The default, equivalent to `vertical-align: unset` or `vertical-align: initial`

`left`

Equivalent to `float: left`

`right`

Equivalent to `float: right`

`border`

The width of a border around the image. Use the `border` CSS property instead.

`hspace`

The number of pixels of white space on the left and right of the image. Use the `margin` CSS property instead.

`longdesc`

A link to a more detailed description of the image. Possible values are a URL or an element `id`.

> **Note:** This attribute is considered obsolete in the HTML spec. It has an uncertain future; authors should use a WAI-ARIA alternative such as `aria-describedby` or `aria-details`.

`name`

A name for the element. Use the `id` attribute instead.

`vspace`

The number of pixels of white space above and below the image. Use the `margin` CSS property instead.

### Usage notes

You need at least two attributes for each `<img>` element. Most commonly, they are `src` and `alt`.

- The `src` attribute holds the path to the image you want to embed. It is not mandatory if the srcset attribute is available. However, at least one of the `src` or `srcset` attributes must be provided.
- The `alt` attribute holds a textual replacement for the image, which is mandatory and **incredibly useful** for accessibility — screen readers read the attribute value out to their users so they know what the image means. Alt text is also displayed on the page if the image can't be loaded for some reason: for example, network errors, content blocking, or link rot.

There are many other attributes to achieve various purposes:

- Referrer/CORS control for security and privacy: see `crossorigin` and `referrerpolicy`.
- Use both `width` and `height` to set the intrinsic size of the image, allowing it to take up space before it loads, to mitigate content layout shifts.
- Responsive image hints with `sizes` and `srcset` (see also the `<picture>` element and our Responsive images tutorial).

The Image file type and format guide provides information about the supported image formats and broad recommendations about where they should be used.

### Image loading errors

If an error occurs while loading or rendering an image, and an `onerror` event handler has been set for the `error` event, that event handler will get called. This can happen in several situations, including:

- The `src` or `srcset` attributes are empty (`""`) or `null`.
- The `src` URL is the same as the URL of the page the user is currently on.
- The image is corrupted in some way that prevents it from being loaded.
- The image's metadata is corrupted in such a way that it's impossible to retrieve its dimensions, and no dimensions were specified in the `<img>` element's attributes.
- The image is in a format not supported by the user agent.

### Styling with CSS

`<img>` is a replaced element; it has a `display` value of `inline` by default, but its default dimensions are defined by the embedded image's intrinsic values, like it were `inline-block`. You can set properties like `border`/`border-radius`, `padding`/`margin`, `width`, `height`, etc. on an image.

`<img>` has no baseline, so when images are used in an inline formatting context with `vertical-align: baseline`, the bottom of the image will be placed on the text baseline.

You can use the `object-position` property to position the image within the element's box, and the `object-fit` property to adjust the sizing of the image within the box (for example, whether the image should fit the box or fill it even if clipping is required).

Depending on its type, an image may have an intrinsic width and height. For some image types, however, intrinsic dimensions are unnecessary. SVG images, for instance, have no intrinsic dimensions if their root `<svg>` element doesn't have a `width` or `height` set on it.

### Accessibility

### Authoring meaningful alternate descriptions

An `alt` attribute's value should provide a clear and concise text replacement for the image's content. It should not describe the presence of the image itself or the file name of the image. If the `alt` attribute is purposefully left off because the image has no textual equivalent, consider alternate methods to present what the image is trying to communicate.

### Don't

```
<img alt="image" src="https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/img/penguin.jpg" />
```

### Do

```
<img alt="A Penguin on a beach." src="https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/img/penguin.jpg" />
```

An important accessibility test is to read the `alt` attribute content together with preceding textual content to see if it conveys the same meaning as the image. For example, if the image was preceded by the sentence "On my travels, I saw a cute little animal:", the *Don't* example could be read by a screen reader as "On my travels, I saw a cute little animal: image", which doesn't make sense. The *Do* example could be read by a screen reader as "On my travels, I saw a cute little animal: A Penguin on a beach.", which does make sense.

For images used to trigger an action, for example, images nested inside an `<a>` or `<button>` element, consider describing the triggered action inside the `alt` attribute value. For example, you could write `alt="next page"` instead of `alt="arrow right"`. You could also consider adding an optional further description inside a `title` attribute; this may be read by screen readers if requested by the user.

When an `alt` attribute is not present on an image, some screen readers may announce the image's file name instead. This can be a confusing experience if the file name isn't representative of the image's contents.

- An alt Decision Tree • Images • WAI Web Accessibility Tutorials
- Alt-texts: The Ultimate Guide — Axess Lab
- How to Design Great Alt Text: An Introduction | Deque
- MDN Understanding WCAG, Guideline 1.1 explanations
- Understanding Success Criterion 1.1.1 | W3C Understanding WCAG 2.0

### Identifying SVG as an image

Due to a VoiceOver bug, VoiceOver does not correctly announce SVG images as images. Include `role="img"` to all `<img>` elements with SVG source files to ensure assistive technologies correctly announce the SVG as image content.

```
<img src="https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/img/mdn.svg" alt="MDN" role="img" />
```

### The title attribute

The `title` attribute is not an acceptable substitute for the `alt` attribute. Additionally, avoid duplicating the `alt` attribute's value in a `title` attribute declared on the same image. Doing so may cause some screen readers to announce the same text twice, creating a confusing experience.

The `title` attribute should also not be used as supplemental captioning information to accompany an image's `alt` description. If an image needs a caption, use the `figure` and `figcaption` elements.

The value of the `title` attribute is usually presented to the user as a tooltip, which appears shortly after the cursor stops moving over the image. While this *can* provide additional information to the user, you should not assume that the user will ever see it: the user may only have keyboard or touchscreen. If you have information that's particularly important or valuable for the user, present it inline using one of the methods mentioned above instead of using `title`.

- Using the HTML title attribute – updated | Vispero

## Examples

### Alternative text

The following example embeds an image into the page and includes alternative text for accessibility.

```
<img src="https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/img//shared-assets/images/examples/favicon144.png"
alt="MDN" />
```

### Image link

This example builds upon the previous one, showing how to turn the image into a link. To do so, nest the `<img>` tag inside the `<a>`. You should make the alternative text describe the resource the link is pointing to, as if you were using a text link instead.

```
<a href="https://developer.mozilla.org">
  <img
    src="https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/img//shared-assets/images/examples/favicon144.png"
    alt="Visit the MDN site" />
</a>
```

### Using the srcset attribute

In this example we include a `srcset` attribute with a reference to a high-resolution version of the logo; this will be loaded instead of the `src` image on high-resolution devices. The image referenced in the `src` attribute is counted as a `1x` candidate in user agents that support `srcset`.

```
<img
  src="https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/img//shared-assets/images/examples/favicon72.png"
  alt="MDN"
  srcset="/shared-assets/images/examples/favicon144.png 2x" />
```

### Using the srcset and sizes attributes

The `src` attribute is ignored in user agents that support `srcset` when `w` descriptors are included. When the `(width <= 600px)` media condition matches, the 200 pixel-wide image will load (it is the one that matches `200px` most closely), otherwise the other image will load.

```
<img
  src="https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/img/clock-demo-200px.png"
  alt="The time is 12:45."
  srcset="clock-demo-200px.png 200w, clock-demo-400px.png 400w"
  sizes="(width <= 600px) 200px, 50vw" />
```

> **Note:** To see the resizing in action, view the example on a separate page, so you can actually resize the content area.

### Security and privacy concerns

Although `<img>` elements have innocent uses, they can have undesirable consequences for user security and privacy. See Referer header: privacy and security concerns for more information and mitigations.

### Technical summary

| | |
|---|---|
| **Content categories** | Flow content, phrasing content, embedded content, palpable content. If the element has a `usemap` attribute, it also is a part of the interactive content category. |
| **Permitted content** | None; it is a void element. |
| **Tag omission** | Must have a start tag and must not have an end tag. |
| **Permitted parents** | Any element that accepts embedded content. |
| **Implicit ARIA role** | • with non-empty `alt` attribute or no `alt` attribute: `img`<br>• with empty `alt` attribute: `presentation` |
| **Permitted ARIA roles** | • with non-empty `alt` attribute:<br>  ◦ `button`<br>  ◦ `checkbox`<br>  ◦ `link`<br>  ◦ `menuitem`<br>  ◦ `menuitemcheckbox`<br>  ◦ `menuitemradio`<br>  ◦ `option`<br>  ◦ `progressbar`<br>  ◦ `scrollbar`<br>  ◦ `separator`<br>  ◦ `slider`<br>  ◦ `switch`<br>  ◦ `tab`<br>  ◦ `treeitem`<br>• with empty `alt` attribute, `none` or `presentation`<br>• with no `alt` attribute, no `role` permitted |
| **DOM interface** | `HTMLImageElement` |

See also

- `<picture>`, `<object>`, and `<embed>` elements
- `object-fit`, `object-position`, `image-orientation`, `image-rendering`, and `image-resolution`: Image-related CSS properties.
- `HTMLImageElement` interface for this element
- HTML images
- Image file type and format guide
- Responsive images

## <input> HTML input element

The **`<input>`** HTML element is used to create interactive controls for web-based forms in order to accept data from the user; a wide variety of types of input data and control widgets are available, depending on the device and user agent. The `<input>` element is one of the most powerful and complex in all of HTML due to the sheer number of combinations of input types and attributes.

Try it

```
<label for="name">Name (4 to 8 characters):</label>

<input
  type="text"
  id="name"
  name="name"
  required
  minlength="4"
  maxlength="8"
  size="10" />
```

```
label {
  display: block;
  font:
    1rem "Fira Sans",
    sans-serif;
}

input,
label {
```

```
  margin: 0.4rem 0;
}
```

### <input> types

How an `<input>` works varies considerably depending on the value of its `type` attribute, hence the different types are covered in their own separate reference pages. If this attribute is not specified, the default type adopted is `text`.

The available types are as follows:

| Type | Description | Basic Examples |
|---|---|---|
| button | A push button with no default behavior displaying the value of the `value` attribute, empty by default. | |
| checkbox | A checkbox allowing single values to be selected/deselected. | |
| color | A control for specifying a color; opening a color picker when active in supporting browsers. | |
| date | A control for entering a date (year, month, and day, with no time). Opens a date picker or numeric wheels for year, month, day when active in supporting browsers. | |
| datetime-local | A control for entering a date and time, with no time zone. Opens a date picker or numeric wheels for date- and time-components when active in supporting browsers. | |
| email | A field for editing an email address. Looks like a `text` input, but has validation parameters and relevant keyboard in supporting browsers and devices with dynamic keyboards. | |
| file | A control that lets the user select a file. Use the `accept` attribute to define the types of files that the control can select. | |
| hidden | A control that is not displayed but whose value is submitted to the server. There is an example in the next column, but it's hidden! | |
| image | A graphical `submit` button. Displays an image defined by the `src` attribute. The `alt` attribute displays if the image `src` is missing. | |
| month | A control for entering a month and year, with no time zone. | |
| number | A control for entering a number. Displays a spinner and adds default validation. Displays a numeric keypad in some devices with dynamic keypads. | |
| password | A single-line text field whose value is obscured. Will alert user if site is not secure. | |
| radio | A radio button, allowing a single value to be selected out of multiple choices with the same `name` value. | |
| range | A control for entering a number whose exact value is not important. Displays as a range widget defaulting to the middle value. Used in conjunction `min` and `max` to define the range of acceptable values. | |
| reset | A button that resets the contents of the form to default values. Not recommended. | |
| search | A single-line text field for entering search strings. Line-breaks are automatically removed from the input value. May include a delete icon in supporting browsers that can be used to clear the field. Displays a search icon instead of enter key on some devices with dynamic keypads. | |
| submit | A button that submits the form. | |
| tel | A control for entering a telephone number. Displays a telephone keypad in some devices with dynamic keypads. | |
| text | The default value. A single-line text field. Line-breaks are automatically removed from the input value. | |
| time | A control for entering a time value with no time zone. | |
| url | A field for entering a URL. Looks like a `text` input, but has validation parameters and relevant keyboard in supporting browsers and devices with dynamic keyboards. | |

| Type | Description | Basic Examples |
| --- | --- | --- |
| week | A control for entering a date consisting of a week-year number and a week number with no time zone. | |
| **Obsolete values** | | |
| `datetime` | A control for entering a date and time (hour, minute, second, and fraction of a second) based on UTC time zone. | |

### Attributes

The `<input>` element is so powerful because of its attributes; the `type` attribute, described with examples above, being the most important. Since every `<input>` element, regardless of type, is based on the `HTMLInputElement` interface, they technically share the exact same set of attributes. However, in reality, most attributes have an effect on only a specific subset of input types. In addition, the way some attributes impact an input depends on the input type, impacting different input types in different ways.

This section provides a table listing all the attributes with a brief description. This table is followed by a list describing each attribute in greater detail, along with which input types they are associated with. Those that are common to most or all input types are defined in greater detail below. Attributes that are unique to particular input types—or attributes which are common to all input types but have special behaviors when used on a given input type—are instead documented on those types' pages.

Attributes for the `<input>` element include the global HTML attributes and additionally:

| Attribute | Type(s) | Description |
| --- | --- | --- |
| `accept` | `file` | Hint for expected file type in file upload controls |
| `alpha` | `color` | Opacity of the color |
| `alt` | `image` | alt attribute for the image type. Required for accessibility |
| `autocapitalize` | all except `url`, `email`, and `password` | Controls automatic capitalization in inputted text. |
| `autocomplete` | all except `checkbox`, `radio`, and buttons | Hint for form autofill feature |
| `capture` | `file` | Media capture input method in file upload controls |
| `checked` | `checkbox`, `radio` | Whether the command or control is checked |
| `colorspace` | `color` | The color space that should be used for selecting the color value |
| `dirname` | `hidden`, `text`, `search`, `url`, `tel`, `email` | Name of form field to use for sending the element's directionality in form submission |
| `disabled` | all | Whether the form control is disabled |
| `form` | all | Associates the control with a form element |
| `formaction` | `image`, `submit` | URL to use for form submission |
| `formenctype` | `image`, `submit` | Form data set encoding type to use for form submission |
| `formmethod` | `image`, `submit` | HTTP method to use for form submission |
| `formnovalidate` | `image`, `submit` | Bypass form control validation for form submission |
| `formtarget` | `image`, `submit` | Browsing context for form submission |
| `height` | `image` | Same as height attribute for `<img>`; vertical dimension |
| `list` | all except `hidden`, `password`, `checkbox`, `radio`, and buttons | Value of the id attribute of the `<datalist>` of autocomplete options |
| `max` | `date`, `month`, `week`, `time`, `datetime-local`, `number`, `range` | Maximum value |
| `maxlength` | `text`, `search`, `url`, `tel`, `email`, `password` | Maximum length (number of characters) of `value` |
| `min` | `date`, `month`, `week`, `time`, `datetime-local`, `number`, `range` | Minimum value |
| `minlength` | `text`, `search`, `url`, `tel`, `email`, `password` | Minimum length (number of characters) of `value` |
| `multiple` | `email`, `file` | Boolean. Whether to allow multiple values |
| `name` | all | Name of the form control. Submitted with the form as part of a name/value pair |
| `pattern` | `text`, `search`, `url`, `tel`, `email`, `password` | Pattern the `value` must match to be valid |
| `placeholder` | `text`, `search`, `url`, `tel`, `email`, `password`, `number` | Text that appears in the form control when it has no value set |
| `popovertarget` | `button` | Designates an `<input type="button">` as a control for a popover element |
| `popovertargetaction` | `button` | Specifies the action that a popover control should perform |

| Attribute | Type(s) | Description |
| --- | --- | --- |
| `readonly` | all except `hidden`, `range`, `color`, `checkbox`, `radio`, and buttons | Boolean. The value is not editable |
| `required` | all except `hidden`, `range`, `color`, and buttons | Boolean. A value is required or must be checked for the form to be submittable |
| `size` | `text`, `search`, `url`, `tel`, `email`, `password` | Size of the control |
| `src` | `image` | Same as `src` attribute for `<img>`; address of image resource |
| `step` | `date`, `month`, `week`, `time`, `datetime-local`, `number`, `range` | Incremental values that are valid |
| `switch` | `checkbox` | Whether the checkbox input should be rendered as a switch |
| `type` | all | Type of form control |
| `value` | all except `image` | The value of the control. When specified in the HTML, corresponds to the initial value |
| `width` | `image` | Same as `width` attribute for `<img>` |

A few additional non-standard attributes are listed following the descriptions of the standard attributes.

### Individual attributes

`accept`

Valid for the `file` input type only, the `accept` attribute defines which file types are selectable in a `file` upload control. See the file input type.

`alpha`

Valid for the `color` input type only, the `alpha` attribute provides the end user with the ability to set the opacity of the color being selected.

`alt`

Valid for the `image` button only, the `alt` attribute provides alternative text for the image, displaying the value of the attribute if the image `src` is missing or otherwise fails to load. See the image input type.

`autocapitalize`

Controls whether inputted text is automatically capitalized and, if so, in what manner. See the `autocapitalize` global attribute page for more information.

`autocomplete`

(**Not** a Boolean attribute!) The `autocomplete` attribute takes as its value a space-separated string that describes what, if any, type of autocomplete functionality the input should provide. A typical implementation of autocomplete recalls previous values entered in the same input field, but more complex forms of autocomplete can exist. For instance, a browser could integrate with a device's contacts list to autocomplete `email` addresses in an email input field. See `autocomplete` for permitted values.

The `autocomplete` attribute is valid on `hidden`, `text`, `search`, `url`, `tel`, `email`, `date`, `month`, `week`, `time`, `datetime-local`, `number`, `range`, `color`, and `password`. This attribute has no effect on input types that do not return numeric or text data, being valid for all input types except `checkbox`, `radio`, `file`, or any of the button types.

See the `autocomplete` attribute for additional information, including information on password security and how `autocomplete` is slightly different for `hidden` than for other input types.

`autofocus`

A Boolean attribute which, if present, indicates that the input should automatically have focus when the page has finished loading (or when the `<dialog>` containing the element has been displayed).

> **Note:** An element with the `autofocus` attribute may gain focus before the `DOMContentLoaded` event is fired.

No more than one element in the document may have the `autofocus` attribute. If put on more than one element, the first one with the attribute receives focus.

The `autofocus` attribute cannot be used on inputs of type `hidden`, since hidden inputs cannot be focused.

> **Warning:** Automatically focusing a form control can confuse visually-impaired people using screen-reading technology and people with cognitive impairments. When `autofocus` is assigned, screen-readers "teleport" their user to the form control without warning them

> beforehand.

Use careful consideration for accessibility when applying the `autofocus` attribute. Automatically focusing on a control can cause the page to scroll on load. The focus can also cause dynamic keyboards to display on some touch devices. While a screen reader will announce the label of the form control receiving focus, the screen reader will not announce anything before the label, and the sighted user on a small device will equally miss the context created by the preceding content.

`capture`

Introduced in the HTML Media Capture specification and valid for the `file` input type only, the `capture` attribute defines which media—microphone, video, or camera—should be used to capture a new file for upload with `file` upload control in supporting scenarios. See the file input type.

`checked`

Valid for both `radio` and `checkbox` types, `checked` is a Boolean attribute. If present on a `radio` type, it indicates that the radio button is the currently selected one in the group of same-named radio buttons. If present on a `checkbox` type, it indicates that the checkbox is checked by default (when the page loads). It does *not* indicate whether this checkbox is currently checked: if the checkbox's state is changed, this content attribute does not reflect the change. (Only the `HTMLInputElement`'s `checked` IDL attribute is updated.)

> **Note:** Unlike other input controls, a checkboxes and radio buttons value are only included in the submitted data if they are currently `checked`. If they are, the name and the value(s) of the checked controls are submitted.
>
> For example, if a checkbox whose `name` is `fruit` has a `value` of `cherry`, and the checkbox is checked, the form data submitted will include `fruit=cherry`. If the checkbox isn't active, it isn't listed in the form data at all. The default `value` for checkboxes and radio buttons is `on`.

`colorspace`

Valid for the `color` input type only, the `colorspace` attribute specifies the color space that is used by the `type="color"` input. Possible enumerated values are:

- `"limited-srgb"`: The color is in the sRGB color space. This includes `rgb()`, `hsl()`, `hwb()`, and `<hex-color>` values. The color value is limited to 8-bits per `r`, `g`, and `b` component. This is the default.
- `"display-p3"`: The Display P3 color space, e.g., `color(display-p3 1.84 -0.19 0.72 / 0.6)`

`dirname`

Valid for `hidden`, `text`, `search`, `url`, `tel`, and `email` input types, the `dirname` attribute enables the submission of the directionality of the element. When included, the form control will submit with two name/value pairs: the first being the `name` and `value`, and the second being the value of the `dirname` attribute as the name, with a value of `ltr` or `rtl` as set by the browser.

```
<form action="page.html" method="post">
  <label>
    Fruit:
    <input type="text" name="fruit" dirname="fruit-dir" value="cherry" />
  </label>
  <input type="submit" />
</form>
<!-- page.html?fruit=cherry&fruit-dir=ltr -->
```

When the form above is submitted, the input cause both the `name` / `value` pair of `fruit=cherry` and the `dirname` / direction pair of `fruit-dir=ltr` to be sent. For more information, see the `dirname` attribute.

`disabled`

A Boolean attribute which, if present, indicates that the user should not be able to interact with the input. Disabled inputs are typically rendered with a dimmer color or using some other form of indication that the field is not available for use.

Specifically, disabled inputs do not receive the `click` event, and disabled inputs are not submitted with the form.

> **Note:** Although not required by the specification, Firefox will by default persist the dynamic disabled state of an `<input>` across page loads. Use the `autocomplete` attribute to control this feature.

`form`

A string specifying the `<form>` element with which the input is associated (that is, its **form owner**). This string's value, if present, must match the `id` of a `<form>` element in the same document. If this attribute isn't specified, the `<input>` element is associated with the nearest containing form, if any.

The `form` attribute lets you place an input anywhere in the document but have it included with a form elsewhere in the document.

> **Note:** An input can only be associated with one form.

`formaction`

Valid for the `image` and `submit` input types only. See the submit input type for more information.

`formenctype`

Valid for the `image` and `submit` input types only. See the submit input type for more information.

`formmethod`

Valid for the `image` and `submit` input types only. See the submit input type for more information.

`formnovalidate`

Valid for the `image` and `submit` input types only. See the submit input type for more information.

`formtarget`

Valid for the `image` and `submit` input types only. See the submit input type for more information.

`height`

Valid for the `image` input button only, the `height` is the height of the image file to display to represent the graphical submit button. See the image input type.

`id`

Global attribute valid for all elements, including all the input types, it defines a unique identifier (ID) which must be unique in the whole document. Its purpose is to identify the element when linking. The value is used as the value of the `<label>`'s `for` attribute to link the label with the form control. See `<label>`.

`inputmode`

Global value valid for all elements, it provides a hint to browsers as to the type of virtual keyboard configuration to use when editing this element or its contents. Values include `none`, `text`, `tel`, `url`, `email`, `numeric`, `decimal`, and `search`.

`list`

The value given to the `list` attribute should be the `id` of a `<datalist>` element located in the same document. The `<datalist>` provides a list of predefined values to suggest to the user for this input. Any values in the list that are not compatible with the `type` are not included in the suggested options. The values provided are suggestions, not requirements: users can select from this predefined list or provide a different value.

It is valid on `text`, `search`, `url`, `tel`, `email`, `date`, `month`, `week`, `time`, `datetime-local`, `number`, `range`, and `color`.

Per the specifications, the `list` attribute is not supported by the `hidden`, `password`, `checkbox`, `radio`, `file`, or any of the button types.

Depending on the browser, the user may see a custom color palette suggested, tic marks along a range, or even an input that opens like a `<select>` but allows for non-listed values. Check out the browser compatibility table for the other input types.

See the `<datalist>` element.

`max`

Valid for `date`, `month`, `week`, `time`, `datetime-local`, `number`, and `range`, it defines the greatest value in the range of permitted values. If the `value` entered into the element exceeds this, the element fails constraint validation. If the value of the `max` attribute isn't a number, then the element has no maximum value.

There is a special case: if the data type is periodic (such as for dates or times), the value of `max` may be lower than the value of `min`, which indicates that the range may wrap around; for example, this allows you to specify a time range from 10 PM to 4 AM.

`maxlength`

Valid for `text`, `search`, `url`, `tel`, `email`, and `password`, it defines the maximum string length (measured in UTF-16 code units) that the user can enter into the field. This must be an integer value of 0 or higher. If no `maxlength` is specified, or an invalid value is specified, the field has no maximum length. This value must also be greater than or equal to the value of `minlength`.

The input will fail constraint validation if the length of the text entered into the field is greater than `maxlength` UTF-16 code units long. By default, browsers prevent users from entering more characters than allowed by the `maxlength` attribute. Constraint validation is only applied when the value is changed by the user. See Client-side validation for more information.

`min`

Valid for `date`, `month`, `week`, `time`, `datetime-local`, `number`, and `range`, it defines the most negative value in the range of permitted values. If the `value` entered into the element is less than this, the element fails constraint validation. If the value of the `min` attribute isn't a number, then the element has no minimum value.

This value must be less than or equal to the value of the `max` attribute. If the `min` attribute is present but is not specified or is invalid, no `min` value is applied. If the `min` attribute is valid and a non-empty value is less than the minimum allowed by the `min` attribute, constraint validation will prevent form submission. See Client-side validation for more information.

There is a special case: if the data type is periodic (such as for dates or times), the value of `max` may be lower than the value of `min`, which indicates that the range may wrap around; for example, this allows you to specify a time range from 10 PM to 4 AM.

`minlength`

Valid for `text`, `search`, `url`, `tel`, `email`, and `password`, it defines the minimum string length (measured in UTF-16 code units) that the user can enter into the entry field. This must be a non-negative integer value smaller than or equal to the value specified by `maxlength`. If no `minlength` is specified, or an invalid value is specified, the input has no minimum length.

The input will fail constraint validation if the length of the text entered into the field is fewer than `minlength` UTF-16 code units long, preventing form submission. Constraint validation is only applied when the value is changed by the user. See Client-side validation for more information.

`multiple`

The Boolean `multiple` attribute, if set, means the user can enter comma separated email addresses in the email widget or can choose more than one file with the `file` input. See the email and file input type.

`name`

A string specifying a name for the input control. This name is submitted along with the control's value when the form data is submitted.

Consider the `name` a required attribute (even though it's not). If an input has no `name` specified, or `name` is empty, the input's value is not submitted with the form! (Disabled controls, unchecked radio buttons, unchecked checkboxes, and reset buttons are also not sent.)

There are two special cases:

1. `_charset_` : If used as the name of an `<input>` element of type hidden, the input's `value` is automatically set by the user agent to the character encoding being used to submit the form.
2. `isindex`: For historical reasons, the name `isindex` is not allowed.

The `name` attribute creates a unique behavior for radio buttons.

Only one radio button in a same-named group of radio buttons can be checked at a time. Selecting any radio button in that group automatically deselects any currently-selected radio button in the same group. The value of that one checked radio button is sent along with the name if the form is submitted,

When tabbing into a series of same-named group of radio buttons, if one is checked, that one will receive focus. If they aren't grouped together in source order, if one of the group is checked, tabbing into the group starts when the first one in the group is encountered, skipping all those that aren't checked. In other words, if one is checked, tabbing skips the unchecked radio buttons in the group. If none are checked, the radio button group receives focus when the first button in the same name group is reached.

Once one of the radio buttons in a group has focus, using the arrow keys will navigate through all the radio buttons of the same name, even if the radio buttons are not grouped together in the source order.

When an input element is given a `name`, that name becomes a property of the owning form element's `HTMLFormElement.elements` property. If you have an input whose `name` is set to `guest` and another whose `name` is `hat-size`, the following code can be used:

```
let form = document.querySelector("form");

let guestName = form.elements.guest;
let hatSize = form.elements["hat-size"];
```

When this code has run, `guestName` will be the `HTMLInputElement` for the `guest` field, and `hatSize` the object for the `hat-size` field.

> **Warning:** Avoid giving form elements a `name` that corresponds to a built-in property of the form, since you would then override the predefined property or method with this reference to the corresponding input.

`pattern`

Valid for `text`, `search`, `url`, `tel`, `email`, and `password`, the `pattern` attribute is used to compile a regular expression that the input's `value` must match in order for the value to pass constraint validation. It must be a valid JavaScript regular expression, as used by the `RegExp` type, and as documented in our guide on regular expressions. No forward slashes should be specified around the pattern text. When compiling the regular expression:

1. the pattern will be implicitly wrapped with `^(?:` and `)$`, such that the match is required against the *entire* input value, i.e., `^(?:<pattern>)$`.
2. the `'v'` flag is specified so that the pattern is treated as a sequence of Unicode code points, instead of as ASCII.

If the `pattern` attribute is present but is not specified or is invalid, no regular expression is applied and this attribute is ignored completely. If the pattern attribute is valid and a non-empty value does not match the pattern, constraint validation will prevent form submission. If the `multiple` is present, the compiled regular expression is matched against each comma separated value.

> **Note:** If using the `pattern` attribute, inform the user about the expected format by including explanatory text nearby. You can also include a `title` attribute to explain what the requirements are to match the pattern; most browsers will display this title as a tooltip. The visible explanation is required for accessibility. The tooltip is an enhancement.

See Client-side validation for more information.

`placeholder`

Valid for `text`, `search`, `url`, `tel`, `email`, `password`, and `number`, the `placeholder` attribute provides a brief hint to the user as to what kind of information is expected in the field. It should be a word or short phrase that provides a hint as to the expected type of data, rather than an explanation or prompt. The text *must not* include carriage returns or line feeds. So for example if a field is expected to capture a user's first name, and its label is "First Name", a suitable placeholder might be "e.g., Mustafa".

> **Note:** The `placeholder` attribute is not as semantically useful as other ways to explain your form, and can cause unexpected technical issues with your content. See Labels for more information.

`popovertarget`

Turns an `<input type="button">` element into a popover control button; takes the ID of the popover element to control as its value. See the Popover API landing page for more details. Establishing a relationship between a popover and its invoker button using the `popovertarget` attribute has two additional useful effects:

- The browser creates an implicit `aria-details` and `aria-expanded` relationship between popover and invoker, and places the popover in a logical position in the keyboard focus navigation order when shown. This makes the popover more accessible to keyboard and assistive technology (AT) users (see also Popover accessibility features).
- The browser creates an implicit anchor reference between the two, making it very convenient to position popovers relative to their controls using CSS anchor positioning. See Popover anchor positioning for more details.

`popovertargetaction`

Specifies the action to be performed on a popover element being controlled by a control `<input type="button">`. Possible values are:

`"hide"`

The button will hide a shown popover. If you try to hide an already hidden popover, no action will be taken.

`"show"`

The button will show a hidden popover. If you try to show an already showing popover, no action will be taken.

`"toggle"`

The button will toggle a popover between showing and hidden. If the popover is hidden, it will be shown; if the popover is showing, it will be hidden. If `popovertargetaction` is omitted, `"toggle"` is the default action that will be performed by the control button.

`readonly`

A Boolean attribute which, if present, indicates that the user should not be able to edit the value of the input. The `readonly` attribute is supported by the `text`, `search`, `url`, `tel`, `email`, `date`, `month`, `week`, `time`, `datetime-local`, `number`, and `password` input types.

See the HTML attribute: `readonly` for more information.

`required`

`required` is a Boolean attribute which, if present, indicates that the user must specify a value for the input before the owning form can be submitted. The `required` attribute is supported by `text`, `search`, `url`, `tel`, `email`, `date`, `month`, `week`, `time`, `datetime-local`, `number`, `password`, `checkbox`, `radio`, and `file` inputs.

See Client-side validation and the HTML attribute: `required` for more information.

`size`

Valid for `email`, `password`, `tel`, `url`, and `text`, the `size` attribute specifies how much of the input is shown. Basically creates same result as setting CSS `width` property with a few specialties. The actual unit of the value depends on the input type. For `password` and `text`, it is a number of characters (or `em` units) with a default value of `20`, and for others, it is pixels (or `px` units). CSS `width` takes precedence over the `size` attribute.

`src`

Valid for the `image` input button only, the `src` is string specifying the URL of the image file to display to represent the graphical submit button. See the image input type.

`step`

Valid for `date`, `month`, `week`, `time`, `datetime-local`, `number`, and `range`, the `step` attribute is a number that specifies the granularity that the value must adhere to. Only values which are a whole number of steps from the step base are valid. The step base is `min` if specified, `value` otherwise, or `0` if neither is provided (except for `week`, which has a default step base of −259,200,000, representing the start of week `1970-W01`).

If not explicitly included:

- `step` defaults to 1 for `number` and `range`.
- Each date/time input type has a default `step` value appropriate for the type; see the individual input pages: `date`, `datetime-local`, `month`, `time`, and `week`.

The value must be a positive number—integer or float—or the special value `any`, which means no stepping is implied, and any value is allowed (barring other constraints, such as `min` and `max`).

For example, if you have `<input type="number" min="10" step="2">`, then any even integer, `10` or greater, is valid. If omitted, `<input type="number">`, any integer is valid, but floats (like `4.2`) are not valid, because `step` defaults to `1`. For `4.2` to be valid, `step` would have had to be set to `any`, 0.1, 0.2, or the `min` value would have had to be a number ending in `.2`, such as `<input type="number" min="-5.2">`.

> **Note:** When the data entered by the user doesn't adhere to the stepping configuration, the value is considered invalid in constraint validation and will match the `:invalid` pseudoclass.

See Client-side validation for more information.

`switch`

Valid for `checkbox` input only, `switch` is a Boolean attribute that indicates whether the checkbox input should be rendered as a switch.

> **Note:** This attribute is still experimental and has limited browser support. The attribute is ignored on unsupported browsers.

`tabindex`

Global attribute valid for all elements, including all the input types, an integer attribute indicating if the element can take input focus (is focusable), if it should participate to sequential keyboard navigation. As all input types except for input of type hidden are focusable, this attribute should not be used on form controls, because doing so would require the management of the focus order for all elements within the document with the risk of harming usability and accessibility if done incorrectly.

`title`

Global attribute valid for all elements, including all input types, containing a text representing advisory information related to the element it belongs to. Such information can typically, but not necessarily, be presented to the user as a tooltip. The title should NOT be used as the primary explanation of the purpose of the form control. Instead, use the `<label>` element with a `for` attribute set to the form control's `id` attribute. See Labels below.

`type`

A string specifying the type of control to render. For example, to create a checkbox, a value of `checkbox` is used. If omitted (or an unknown value is specified), the input type `text` is used, creating a plaintext input field.

Permitted values are listed in Input types above.

`value`

The input control's value. When specified in the HTML, this is the initial value, and from then on it can be altered or retrieved at any time using JavaScript to access the respective `HTMLInputElement` object's `value` property. The `value` attribute is always optional, though should be considered mandatory for `checkbox`, `radio`, and `hidden`.

`webkitdirectory`

The Boolean `webkitdirectory` attribute, if present, indicates that only directories should be available to be selected by the user in the file picker interface. See `HTMLInputElement.webkitdirectory` for additional details and examples.

> **Note:** `webkitdirectory` is defined in the File and Directory Entries API. It's named `webkitdirectory` because of its origins as a Chrome-specific API. It is now available in all browsers.

`width`

Valid for the `image` input button only, the `width` is the width of the image file to display to represent the graphical submit button. See the image input type.

### Non-standard attributes

The following non-standard attributes are also available on some browsers. As a general rule, you should avoid using them unless it can't be helped.

| Attribute | Description |
| --- | --- |
| `incremental` | Whether or not to send repeated `search` events to allow updating live search results while the user is still editing the value of the field. **WebKit and Blink only (Safari, Chrome, Opera, etc.).** |
| `mozactionhint` | A string indicating the type of action that will be taken when the user presses the `Enter` or `Return` key while editing the field; this is used to determine an appropriate label for that key on a virtual keyboard. **Since this attribute is deprecated, use `enterkeyhint` instead.** |
| `orient` | Sets the orientation of the range slider. **Firefox only**. |
| `results` | The maximum number of items that should be displayed in the drop-down list of previous search queries. **Safari only.** |

`incremental`

The Boolean attribute `incremental` is a WebKit and Blink extension (so supported by Safari, Opera, Chrome, etc.) which, if present, tells the user agent to process the input as a live search. As the user edits the value of the field, the user agent sends `search` events to the `HTMLInputElement` object representing the search box. This allows your code to update the search results in real time as the user edits the search.

If `incremental` is not specified, the `search` event is only sent when the user explicitly initiates a search (such as by pressing the `Enter` or `Return` key while editing the field).

The `search` event is rate-limited so that it is not sent more frequently than an implementation-defined interval.

`orient`

Similar to the -moz-orient non-standard CSS property impacting the `<progress>` and `<meter>` elements, the `orient` attribute defines the orientation of the range slider. Values include `horizontal`, meaning the range is rendered horizontally, and `vertical`, where the range is rendered vertically. See Creating vertical form controls for a modern approach to creating vertical form controls.

`results`

The `results` attribute—supported only by Safari—is a numeric value that lets you override the maximum number of entries to be displayed in the `<input>` element's natively-provided drop-down menu of previous search queries.

The value must be a non-negative decimal number. If not provided, or an invalid value is given, the browser's default maximum number of entries is used.

### Methods

The following methods are provided by the `HTMLInputElement` interface which represents `<input>` elements in the DOM. Also available are those methods specified by the parent interfaces, `HTMLElement`, `Element`, `Node`, and `EventTarget`.

`checkValidity()`

Returns `true` if the element's value passes validity checks; otherwise, returns `false` and fires an `invalid` event at the element.

`reportValidity()`

Returns `true` if the element's value passes validity checks; otherwise, returns `false`, fires an `invalid` event at the element, and (if the event isn't canceled) reports the problem to the user.

`select()`

Selects the entire content of the `<input>` element, if the element's content is selectable. For elements with no selectable text content (such as a visual color picker or calendar date input), this method does nothing.

`setCustomValidity()`

Sets a custom message to display if the input element's value isn't valid.

`setRangeText()`

Sets the contents of the specified range of characters in the input element to a given string. A `selectMode` parameter is available to allow controlling how the existing content is affected.

`setSelectionRange()`

Selects the specified range of characters within a textual input element. Does nothing for inputs which aren't presented as text input fields.

`showPicker()`

Displays the browser picker for the input element that would normally be displayed when the element is selected, but triggered from a button press or other user interaction.

`stepDown()`

Decrements the value of a numeric input by one, by default, or by the specified number of units.

`stepUp()`

Increments the value of a numeric input by one or by the specified number of units.

## CSS

Inputs, being replaced elements, have a few features not applicable to non form elements. There are CSS selectors that can specifically target form controls based on their UI features, also known as UI pseudo-classes. The input element can also be targeted by type with attribute selectors. There are some properties that are especially useful as well.

### UI pseudo-classes

Pseudo-classes relevant to the `<input>` element:

| Pseudo-class | Description |
|---|---|
| `:enabled` | Any currently enabled element that can be activated (selected, clicked on, typed into, etc.) or accept focus and also has a disabled state, in which it can't be activated or accept focus. |
| `:disabled` | Any currently disabled element that has an enabled state, meaning it otherwise could be activated (selected, clicked on, typed into, etc.) or accept focus were it not disabled. |
| `:read-only` | Element not editable by the user |
| `:read-write` | Element that is editable by the user. |
| `:placeholder-shown` | Element that is currently displaying `placeholder` text, including `<input>` and `<textarea>` elements with the `placeholder` attribute present that has, as yet, no value. |
| `:default` | Form elements that are the default in a group of related elements. Matches checkbox and radio input types that were checked on page load or render. |
| `:checked` | Matches checkbox and radio input types that are currently checked (and the `<option>` in a `<select>` that is currently selected). |
| `:indeterminate` | checkbox elements whose indeterminate property is set to true by JavaScript, radio elements, when all radio buttons with the same name value in the form are unchecked, and `<progress>` elements in an indeterminate state |
| `:valid` | Form controls that can have constraint validation applied and are currently valid. |
| `:invalid` | Form controls that have constraint validation applied and are currently not valid. Matches a form control whose value doesn't match the constraints set on it by its attributes, such as `required`, `pattern`, `step` and `max`. |
| `:in-range` | A non-empty input whose current value is within the range limits specified by the `min` and `max` attributes and the `step`. |
| `:out-of-range` | A non-empty input whose current value is NOT within the range limits specified by the `min` and `max` attributes or does not adhere to the `step` constraint. |
| `:required` | `<input>`, `<select>`, or `<textarea>` element that has the `required` attribute set on it. Only matches elements that can be required. The attribute included on a non-requirable element will not make for a match. |
| `:optional` | `<input>`, `<select>`, or `<textarea>` element that does NOT have the `required` attribute set on it. Does not match elements that can't be required. |
| `:blank` | `<input>` and `<textarea>` elements that currently have no value. |
| `:user-invalid` | Similar to `:invalid`, but is activated on blur. Matches invalid input but only after the user interaction, such as by focusing on the control, leaving the control, or attempting to submit the form containing the invalid control. |
| `:open` | `<input>` elements that display a picker for the user to choose a value from (for example `<input type="color">`) — but only when the element is in the open state, that is, when the picker is displayed. |

### Pseudo-classes example

We can style a checkbox label based on whether the checkbox is checked or not. In this example, we are styling the `color` and `font-weight` of the `<label>` that comes immediately after a checked input. We haven't applied any styles if the `input` is not checked.

```
input:checked + label {
  color: red;
  font-weight: bold;
}
```

### Attribute selectors

It is possible to target different types of form controls based on their `type` using attribute selectors. CSS attribute selectors match elements based on either just the presence of an attribute or the value of a given attribute.

```
/* matches a password input */
input[type="password"] {
}

/* matches a form control whose valid values are limited to a range of values */
input[min][max] {
}

/* matches a form control with a pattern attribute */
input[pattern] {
}
```

### ::placeholder

By default, the appearance of placeholder text is a translucent or light gray. The `::placeholder` pseudo-element is the input's `placeholder` text. It can be styled with a limited subset of CSS properties.

```
::placeholder {
  color: blue;
}
```

Only the subset of CSS properties that apply to the `::first-line` pseudo-element can be used in a rule using `::placeholder` in its selector.

### caret-color

A property specific to text entry-related elements is the CSS `caret-color` property, which lets you set the color used to draw the text input caret:

### HTML

```
<label for="textInput">Note the red caret:</label>
<input id="textInput" class="custom" size="32" />
```

### CSS

```
input.custom {
  caret-color: red;
  font:
    16px "Helvetica",
    "Arial",
    sans-serif;
}
```

### Result

### field-sizing

The `field-sizing` property enables you to control the sizing behavior of form inputs (i.e., they are given a default preferred size by default.) This property enables you to override the default behavior, allowing form controls to adjust in size to fit their contents.

This property is typically used to create form fields that shrinkwrap their content and grow as more text is entered. This works with input types that accept direct text input (for example, `text` and `url`), input type `file`, and `<textarea>` elements.

### object-position and object-fit

In certain cases (typically involving non-textual inputs and specialized interfaces), the `<input>` element is a replaced element. When it is, the position and size of the element's size and positioning within its frame can be adjusted using the CSS `object-position` and `object-fit` properties.

### Styling

For more information about adding color to elements in HTML, see:

- Applying color to HTML elements using CSS.

Also see:

- Styling HTML forms
- Advanced styling for HTML forms and

### Additional features

#### Labels

Labels are needed to associate assistive text with an `<input>`. The `<label>` element provides explanatory information about a form field that is *always* appropriate (aside from any layout concerns you have). It's never a bad idea to use a `<label>` to explain what should be entered into an `<input>` or `<textarea>`.

#### Associated labels

The semantic pairing of `<input>` and `<label>` elements is useful for assistive technologies such as screen readers. By pairing them using the `<label>`'s `for` attribute, you bond the label to the input in a way that lets screen readers describe inputs to users more precisely.

It does not suffice to have plain text adjacent to the `<input>` element. Rather, usability and accessibility requires the inclusion of either implicit or explicit `<label>`:

```
<!-- inaccessible -->
<p>Enter your name: <input id="name" type="text" size="30" /></p>

<!-- implicit label -->
<p>
  <label>Enter your name: <input id="name" type="text" size="30" /></label>
</p>

<!-- explicit label -->
<p>
  <label for="name">Enter your name: </label>
  <input id="name" type="text" size="30" />
</p>
```

The first example is inaccessible: no relationship exists between the prompt and the `<input>` element.

In addition to an accessible name, the label provides a larger 'hit' area for mouse and touch screen users to click on or touch. By pairing a `<label>` with an `<input>`, clicking on either one will focus the `<input>`. If you use plain text to "label" your input, this won't happen. Having the prompt part of the activation area for the input is helpful for people with motor control conditions.

As web developers, it's important that we never assume that people will know all the things that we know. The diversity of people using the web—and by extension your website—practically guarantees that some of your site's visitors will have some variation in thought processes and/or circumstances that leads them to interpret your forms very differently from you without clear and properly-presented labels.

#### Placeholders are not accessible

The `placeholder` attribute lets you specify text that appears within the `<input>` element's content area itself when it is empty. The placeholder should never be required to understand your forms. It is not a label, and should not be used as a substitute, because it isn't. The placeholder is used to provide a hint as to what an inputted value should look like, not an explanation or prompt.

Not only is the placeholder not accessible to screen readers, but once the user enters any text into the form control, or if the form control already has a value, the placeholder disappears. Browsers with automatic page translation features may skip over attributes when translating, meaning the `placeholder` may not get translated.

> **Note:** Don't use the `placeholder` attribute if you can avoid it. If you need to label an `<input>` element, use the `<label>` element.

#### Client-side validation

> **Warning:** Client-side validation is useful, but it does *not* guarantee that the server will receive valid data. If the data must be in a specific format, *always* verify it also on the server-side, and return a `400` HTTP response if the format is invalid.

In addition to using CSS to style inputs based on the `:valid` or `:invalid` UI states based on the current state of each input, as noted in the UI pseudo-classes section above, the browser provides for client-side validation on (attempted) form submission. On form submission, if there is a form control that fails constraint validation, supporting browsers will display an error message on the first invalid form control; displaying a default message based on the error type, or a message set by you.

Some input types and other attributes place limits on what values are valid for a given input. For example, `<input type="number" min="2" max="10" step="2">` means only the number 2, 4, 6, 8, or 10 are valid. Several errors could occur, including a `rangeUnderflow` error if the value is less than 2, `rangeOverflow` if greater than 10, `stepMismatch` if the value is a number between 2 and 10, but not an even integer (does not match the requirements of the `step` attribute), or `typeMismatch` if the value is not a number.

For the input types whose domain of possible values is periodic (that is, at the highest possible value, the values wrap back around to the beginning rather than ending), it's possible for the values of the `max` and `min` properties to be reversed, which indicates that the range of permitted values starts at `min`, wraps around to the lowest possible value, then continues on until `max` is reached. This is particularly useful for dates and times, such as when you want to allow the range to be from 8 PM to 8 AM:

```
<input type="time" min="20:00" max="08:00" name="overnight" />
```

Specific attributes and their values can lead to a specific error `ValidityState`:

Validity object errors depend on the `<input>` attributes and their values:

| Attribute | Relevant property | Description |
|---|---|---|
| `max` | `validityState.rangeOverflow` | Occurs when the value is greater than the maximum value as defined by the `max` attribute |
| `maxlength` | `validityState.tooLong` | Occurs when the number of characters is greater than the number allowed by the `maxlength` property |
| `min` | `validityState.rangeUnderflow` | Occurs when the value is less than the minimum value as defined by the `min` attribute |
| `minlength` | `validityState.tooShort` | Occurs when the number of characters is less than the number required by the `minlength` property |
| `pattern` | `validityState.patternMismatch` | Occurs when a pattern attribute is included with a valid regular expression and the `value` does not match it. |
| `required` | `validityState.valueMissing` | Occurs when the `required` attribute is present but the value is `null` or radio or checkbox is not checked. |
| `step` | `validityState.stepMismatch` | The value doesn't match the step increment. Increment default is `1`, so only integers are valid on `type="number"` is step is not included. `step="any"` will never throw this error. |
| `type` | `validityState.typeMismatch` | Occurs when the value is not of the correct type, for example an email does not contain an `@` or a url doesn't contain a protocol. |

If a form control doesn't have the `required` attribute, no value, or an empty string, is not invalid. Even if the above attributes are present, with the exception of `required`, an empty string will not lead to an error.

We can set limits on what values we accept, and supporting browsers will natively validate these form values and alert the user if there is a mistake when the form is submitted.

In addition to the errors described in the table above, the `validityState` interface contains the `badInput`, `valid`, and `customError` boolean readonly properties. The validity object includes:

- `validityState.valueMissing`
- `validityState.typeMismatch`
- `validityState.patternMismatch`
- `validityState.tooLong`
- `validityState.tooShort`
- `validityState.rangeUnderflow`
- `validityState.rangeOverflow`
- `validityState.stepMismatch`
- `validityState.badInput`
- `validityState.valid`
- `validityState.customError`

For each of these Boolean properties, a value of `true` indicates that the specified reason validation may have failed is true, with the exception of the `valid` property, which is `true` if the element's value obeys all constraints.

If there is an error, supporting browsers will both alert the user and prevent the form from being submitted. A word of caution: if a custom error is set to a truthy value (anything other than the empty string or `null`), the form will be prevented from being submitted. If there is no custom error message, and none of the other properties return true, `valid` will be true, and the form can be submitted.

```
function validate(input) {
  let validityState = input.validity;
  if (validityState.valueMissing) {
    input.setCustomValidity("A value is required");
  } else if (validityState.rangeUnderflow) {
    input.setCustomValidity("Your value is too low");
  } else if (validityState.rangeOverflow) {
    input.setCustomValidity("Your value is too high");
```

```
  } else {
    input.setCustomValidity("");
  }
}
```

The last line, setting the custom validity message to the empty string is vital. If the user makes an error, and the validity is set, it will fail to submit, even if all the values are valid, until the message is `null`.

### Custom validation error example

If you want to present a custom error message when a field fails to validate, you need to use the Constraint Validation API available on `<input>` (and related) elements. Take the following form:

```
<form>
  <label for="name">Enter username (upper and lowercase letters): </label>
  <input type="text" name="name" id="name" required pattern="[A-Za-z]+" />
  <button>Submit</button>
</form>
```

The basic HTML form validation features will cause this to produce a default error message if you try to submit the form with either no valid filled in, or a value that does not match the `pattern`.

If you wanted to instead display custom error messages, you could use JavaScript like the following:

```
const nameInput = document.querySelector("input");

nameInput.addEventListener("input", () => {
  nameInput.setCustomValidity("");
  nameInput.checkValidity();
});

nameInput.addEventListener("invalid", () => {
  if (nameInput.value === "") {
    nameInput.setCustomValidity("Enter your username!");
  } else {
    nameInput.setCustomValidity(
      "Usernames can only contain upper and lowercase letters. Try again!",
    );
  }
});
```

The example renders like so:

In brief:

- We check the valid state of the input element every time its value is changed by running the `checkValidity()` method via the `input` event handler.
- If the value is invalid, an `invalid` event is raised, and the `invalid` event handler function is run. Inside this function we work out whether the value is invalid because it is empty, or because it doesn't match the pattern, using an `if ()` block, and set a custom validity error message.
- As a result, if the input value is invalid when the submit button is pressed, one of the custom error messages will be shown.
- If it is valid, it will submit as you'd expect. For this to happen, the custom validity has to be canceled, by invoking `setCustomValidity()` with an empty string value. We therefore do this every time the `input` event is raised. If you don't do this, and a custom validity was previously set, the input will register as invalid, even if it currently contains a valid value on submission.

> **Note:** Always validate input constraints both client side and server side. Constraint validation doesn't remove the need for validation on the *server side*. Invalid values can still be sent by older browsers or by bad actors.

> **Note:** Firefox supported a proprietary error attribute — `x-moz-errormessage` — for many versions, which allowed you set custom error messages in a similar way. This has been removed as of version 66 (see Firefox bug 1513890).

### Localization

The allowed inputs for certain `<input>` types depend on the locale. In some locales, 1,000.00 is a valid number, while in other locales the valid way to enter this number is 1.000,00.

Firefox uses the following heuristics to determine the locale to validate the user's input (at least for `type="number"`):

- Try the language specified by a `lang`/`xml:lang` attribute on the element or any of its parents.
- Try the language specified by any `Content-Language` HTTP header. Or,
- If none specified, use the browser's locale.

### Accessibility

#### Labels

When including inputs, it is an accessibility requirement to add labels alongside. This is needed so those who use assistive technologies can tell what the input is for. Also, clicking or touching a label gives focus to the label's associated form control. This improves the accessibility and usability for sighted users, increases the area a user can click or touch to activate the form control. This is especially useful (and even needed) for radio buttons and checkboxes, which are tiny. For more information about labels in general see Labels.

The following is an example of how to associate the `<label>` with an `<input>` element in the above style. You need to give the `<input>` an `id` attribute. The `<label>` then needs a `for` attribute whose value is the same as the input's `id`.

```
<label for="peas">Do you like peas?</label>
<input type="checkbox" name="peas" id="peas" />
```

#### Size

Interactive elements such as form input should provide an area large enough that it is easy to activate them. This helps a variety of people, including people with motor control issues and people using non-precise forms of input such as a stylus or fingers. A minimum interactive size of 44×44 CSS pixels is recommended.

- Understanding Success Criterion 2.5.5: Target Size | W3C Understanding WCAG 2.1
- Target Size and 2.5.5 | Adrian Roselli
- Quick test: Large touch targets - The A11Y Project

### Technical summary

| | |
|---|---|
| **Content categories** | Flow content, listed, submittable, resettable, form-associated element, phrasing content. If the `type` is not `hidden`, then labelable element, palpable content. |
| **Permitted content** | None; it is a void element. |
| **Tag omission** | Must have a start tag and must not have an end tag. |
| **Permitted parents** | Any element that accepts phrasing content. |
| **Implicit ARIA role** | • `type=button`: `button`<br>• `type=checkbox`: `checkbox`<br>• `type=email`<br>  ◦ with no `list` attribute: `textbox`<br>  ◦ with `list` attribute: `combobox`<br>• `type=image`: `button`<br>• `type=number`: `spinbutton`<br>• `type=radio`: `radio`<br>• `type=range`: `slider`<br>• `type=reset`: `button`<br>• `type=search`<br>  ◦ with no `list` attribute: `searchbox`<br>  ◦ with `list` attribute: `combobox`<br>• `type=submit`: `button`<br>• `type=tel`<br>  ◦ with no `list` attribute: `textbox`<br>  ◦ with `list` attribute: `combobox`<br>• `type=text`<br>  ◦ with no `list` attribute: `textbox`<br>  ◦ with `list` attribute: `combobox`<br>• `type=url`<br>  ◦ with no `list` attribute: `textbox`<br>  ◦ with `list` attribute: `combobox`<br>• `type=color\|date\|datetime-local\|file\|hidden\|month\|password\|time\|week`: no corresponding role |
| **Permitted ARIA roles** | • `type=button`: `checkbox`, `combobox`, `link`, `menuitem`, `menuitemcheckbox`, `menuitemradio`, `option`, `radio`, `switch`, `tab`<br>• `type=checkbox`: `button` when used with `aria-pressed`, `menuitemcheckbox`, `option`, `switch`<br>• `type=image`: `link`, `menuitem`, `menuitemcheckbox`, `menuitemradio`, `radio`, `switch`<br>• `type=radio`: `menuitemradio`<br>• `type=text` with no `list` attribute: `combobox`, `searchbox`, `spinbutton` |

| | |
|---|---|
| | • `type=color\|date\|datetime-local\|email\|file\|hidden\|month\|number\|password\|range\|reset\|search\|submit\|tel\|url\|week` or `text` with `list` attribute: no `role` permitted |
| **DOM interface** | `HTMLInputElement` |

### See also

- CSS `appearance` property
- Your first HTML form
- How to structure an HTML form
- The native form widgets
- Sending form data
- Form constraint validation
- Styling HTML forms

## `<input type="button">` HTML attribute value

`<input>` elements of type **`button`** are rendered as push buttons, which can be programmed to control custom functionality anywhere on a webpage as required when assigned an event handler function (typically for the `click` event).

> **Note:** While `<input>` elements of type `button` are still perfectly valid HTML, the `<button>` element is the favored way to create buttons. Given that a `<button>`'s label text is inserted between the opening and closing tags, you can include HTML in the label, even images.

### Try it

```
<input class="styled" type="button" value="Add to favorites" />
```

```
.styled {
  border: 0;
  line-height: 2.5;
  padding: 0 20px;
  font-size: 1rem;
  text-align: center;
  color: white;
  text-shadow: 1px 1px 1px black;
  border-radius: 10px;
  background-color: tomato;
  background-image: linear-gradient(
    to top left,
    rgb(0 0 0 / 20%),
    rgb(0 0 0 / 20%) 30%,
    transparent
  );
  box-shadow:
    inset 2px 2px 3px rgb(255 255 255 / 60%),
    inset -2px -2px 3px rgb(0 0 0 / 60%);
}

.styled:hover {
  background-color: red;
}

.styled:active {
  box-shadow:
    inset -2px -2px 3px rgb(255 255 255 / 60%),
    inset 2px 2px 3px rgb(0 0 0 / 60%);
}
```

### Value

#### Button with a value

An `<input type="button">` elements' `value` attribute contains a string that is used as the button's label. The `value` provides the accessible description for the button.

```
<input type="button" value="Click Me" />
```

#### Button without a value

If you don't specify a `value`, you get an empty button:

```
<input type="button" />
```

### Using buttons

`<input type="button">` elements have no default behavior (their cousins, `<input type="submit">` and `<input type="reset">` are used to submit and reset forms, respectively). To make buttons do anything, you have to write JavaScript code to do the work.

### A basic button

We'll begin by creating a basic button with a `click` event handler that starts our machine (well, it toggles the `value` of the button and the text content of the following paragraph):

```
<form>
  <input type="button" value="Start machine" />
</form>
<p>The machine is stopped.</p>
```

```
const button = document.querySelector("input");
const paragraph = document.querySelector("p");

button.addEventListener("click", updateButton);

function updateButton() {
  if (button.value === "Start machine") {
    button.value = "Stop machine";
    paragraph.textContent = "The machine has started!";
  } else {
    button.value = "Start machine";
    paragraph.textContent = "The machine is stopped.";
  }
}
```

The script gets a reference to the `HTMLInputElement` object representing the `<input>` in the DOM, saving this reference in the variable `button`. `addEventListener()` is then used to establish a function that will be run when `click` events occur on the button.

### Adding keyboard shortcuts to buttons

Keyboard shortcuts, also known as access keys and keyboard equivalents, let the user trigger a button using a key or combination of keys on the keyboard. To add a keyboard shortcut to a button — just as you would with any `<input>` for which it makes sense — you use the `accesskey` global attribute.

In this example, `s` is specified as the access key (you'll need to press `s` plus the particular modifier keys for your browser/OS combination; see accesskey for a useful list of those).

```
<form>
  <input type="button" value="Start machine" accesskey="s" />
</form>
<p>The machine is stopped.</p>
```

> **Note:** The problem with the above example of course is that the user will not know what the access key is! In a real site, you'd have to provide this information in a way that doesn't interfere with the site design (for example by providing an easily accessible link that points to information on what the site access keys are).

### Disabling and enabling a button

To disable a button, specify the `disabled` global attribute on it, like so:

```
<input type="button" value="Disable me" disabled />
```

### Setting the disabled attribute

You can enable and disable buttons at run time by setting `disabled` to `true` or `false`. In this example our button starts off enabled, but if you press it, it is disabled using `button.disabled = true`. A `setTimeout()` function is then used to reset the button back to its enabled state after two seconds.

```
<input type="button" value="Enabled" />
```

```
const button = document.querySelector("input");

button.addEventListener("click", disableButton);
```

```
function disableButton() {
  button.disabled = true;
  button.value = "Disabled";
  setTimeout(() => {
    button.disabled = false;
    button.value = "Enabled";
  }, 2000);
}
```

### Inheriting the disabled state

If the `disabled` attribute isn't specified, the button inherits its `disabled` state from its parent element. This makes it possible to enable and disable groups of elements all at once by enclosing them in a container such as a `<fieldset>` element, and then setting `disabled` on the container.

The example below shows this in action. This is very similar to the previous example, except that the `disabled` attribute is set on the `<fieldset>` when the first button is pressed — this causes all three buttons to be disabled until the two second timeout has passed.

```
<fieldset>
  <legend>Button group</legend>
  <input type="button" value="Button 1" />
  <input type="button" value="Button 2" />
  <input type="button" value="Button 3" />
</fieldset>
```

```
const button = document.querySelector("input");
const fieldset = document.querySelector("fieldset");

button.addEventListener("click", disableButton);

function disableButton() {
  fieldset.disabled = true;
  setTimeout(() => {
    fieldset.disabled = false;
  }, 2000);
}
```

> **Note:** Unlike other browsers, Firefox persists the `disabled` state of an `<input>` element even after the page is reloaded. As a workaround, set the `<input>` element's `autocomplete` attribute to `off`. (See Firefox bug 654072 for more details.)

### Validation

Buttons don't participate in constraint validation; they have no real value to be constrained.

### Examples

The below example shows a very basic drawing app created using a `<canvas>` element and some CSS and JavaScript (we'll hide the CSS for brevity). The top two controls allow you to choose the color and size of the drawing pen. The button, when clicked, invokes a function that clears the canvas.

```
<div class="toolbar">
  <input type="color" aria-label="select pen color" />
  <input
    type="range"
    min="2"
    max="50"
    value="30"
    aria-label="select pen size" /><span class="output">30</span>
  <input type="button" value="Clear canvas" />
</div>

<canvas class="myCanvas">
  <p>Add suitable fallback here.</p>
</canvas>
```

```
const canvas = document.querySelector(".myCanvas");
const width = (canvas.width = window.innerWidth);
const height = (canvas.height = window.innerHeight - 85);
const ctx = canvas.getContext("2d");

ctx.fillStyle = "rgb(0 0 0)";
ctx.fillRect(0, 0, width, height);

const colorPicker = document.querySelector('input[type="color"]');
const sizePicker = document.querySelector('input[type="range"]');
```

```
const output = document.querySelector(".output");
const clearBtn = document.querySelector('input[type="button"]');

// convert degrees to radians
function degToRad(degrees) {
  return (degrees * Math.PI) / 180;
}

// update size picker output value

sizePicker.oninput = () => {
  output.textContent = sizePicker.value;
};

// store mouse pointer coordinates, and whether the button is pressed
let curX;
let curY;
let pressed = false;

// update mouse pointer coordinates
document.onmousemove = (e) => {
  curX = e.pageX;
  curY = e.pageY;
};

canvas.onmousedown = () => {
  pressed = true;
};

canvas.onmouseup = () => {
  pressed = false;
};

clearBtn.onclick = () => {
  ctx.fillStyle = "rgb(0 0 0)";
  ctx.fillRect(0, 0, width, height);
};

function draw() {
  if (pressed) {
    ctx.fillStyle = colorPicker.value;
    ctx.beginPath();
    ctx.arc(
      curX,
      curY - 85,
      sizePicker.value,
      degToRad(0),
      degToRad(360),
      false,
    );
    ctx.fill();
  }

  requestAnimationFrame(draw);
}

draw();
```

### Technical summary

| | |
|---|---|
| **Value** | A string used as the button's label |
| **Events** | `click` |
| **Supported common attributes** | `type` and `value` |
| **IDL attributes** | `value` |
| **DOM interface** | `HTMLInputElement` |
| **Implicit ARIA Role** | `button` |

### See also

- `<input>` and the `HTMLInputElement` interface which implements it.
- The more modern `<button>` element.

## <input type="checkbox"> HTML attribute value

`<input>` elements of type **`checkbox`** are rendered by default as boxes that are checked (ticked) when activated, like you might see in an official government paper form. The exact appearance depends upon the operating system configuration under which the browser is running. Generally this is a square but it may have rounded corners. A checkbox allows you to select single values for submission in a form (or not).

### Try it

```
<fieldset>
  <legend>Choose your monster's features:</legend>

  <div>
    <input type="checkbox" id="scales" name="scales" checked />
    <label for="scales">Scales</label>
  </div>

  <div>
    <input type="checkbox" id="horns" name="horns" />
    <label for="horns">Horns</label>
  </div>
</fieldset>
```

```
p,
label {
  font:
    1rem "Fira Sans",
    sans-serif;
}

input {
  margin: 0.4rem;
}
```

### Value

A string representing the value of the checkbox. This is not displayed on the client-side, but on the server this is the `value` given to the data submitted with the checkbox's `name`. Take the following example:

```
<form>
  <div>
    <input
      type="checkbox"
      id="subscribeNews"
      name="subscribe"
      value="newsletter" />
    <label for="subscribeNews">Subscribe to newsletter?</label>
  </div>
  <div>
    <button type="submit">Subscribe</button>
  </div>
</form>
```

In this example, we've got a name of `subscribe`, and a value of `newsletter`. When the form is submitted, the data name/value pair will be `subscribe=newsletter`.

If the `value` attribute was omitted, the default value for the checkbox is `on`, so the submitted data in that case would be `subscribe=on`.

> **Note:** If a checkbox is unchecked when its form is submitted, neither the name nor the value is submitted to the server. There is no HTML-only method of representing a checkbox's unchecked state (e.g., `value=unchecked`). If you wanted to submit a default value for the checkbox when it is unchecked, you could include JavaScript to create an <input type="hidden"> within the form with a value indicating an unchecked state.

### Additional attributes

In addition to the common attributes shared by all `<input>` elements, `checkbox` inputs support the following attributes.

`checked`

A boolean attribute indicating whether this checkbox is checked by default (when the page loads). It does *not* indicate whether this checkbox is currently checked: if the checkbox's state is changed, this content attribute does not reflect the change. (Only the `HTMLInputElement`'s `checked` IDL attribute is updated.)

> **Note:** Unlike other input controls, a checkbox's value is only included in the submitted data if the checkbox is currently `checked`. If it is, then the value of the checkbox's `value` attribute is reported as the input's value, or `on` if no `value` is set. Unlike other browsers, Firefox by

> default persists the dynamic checked state of an `<input>` across page loads. Use the `autocomplete` attribute to control this feature.

`value`

> The `value` attribute is one which all `<input>`s share; however, it serves a special purpose for inputs of type `checkbox`: when a form is submitted, only checkboxes which are currently checked are submitted to the server, and the reported value is the value of the `value` attribute. If the `value` is not otherwise specified, it is the string `on` by default. This is demonstrated in the section Value above.

`switch`

> A boolean attribute that applies only to `checkbox` inputs. When present, it indicates that the `checkbox` represents an on/off `switch` rather than a normal `checkbox`. It alters the appearance of the `checkbox` control, but the underlying behavior remains the same as that of a normal `checkbox`.
>
> **Note:** This attribute allows user agents to expose `switch` ARIA semantics to assistive technologies — without requiring documents to explicitly specify `role="switch"`. The markup and API are similar to those of checkboxes, except that the `:indeterminate` pseudo-class never matches.
>
> **Warning:** This attribute is still experimental and has limited browser support. The attribute is ignored on unsupported browsers.

### Using checkbox inputs

Checkboxes are similar to radio buttons, but with an important distinction — same-named radio buttons are grouped into a set in which only one radio button can be selected at a time, whereas checkboxes allow you to turn single values on and off. Where multiple same-named controls exist, radio buttons allow one to be selected out of them all, whereas checkboxes allow multiple values to be selected.

### Handling multiple checkboxes

The example we saw above only contained one checkbox; in real-world situations you'll be likely to encounter multiple checkboxes. If they are completely unrelated, then you can just deal with them all separately, as shown above. However, if they're all related, things are not quite so simple.

For example, in the following demo we include multiple checkboxes to allow the user to select their interests (see the full version in the Examples section).

```
<fieldset>
  <legend>Choose your interests</legend>
  <div>
    <input type="checkbox" id="coding" name="interest" value="coding" />
    <label for="coding">Coding</label>
  </div>
  <div>
    <input type="checkbox" id="music" name="interest" value="music" />
    <label for="music">Music</label>
  </div>
</fieldset>
```

In this example you will see that we've given each checkbox the same `name`. If both checkboxes are checked and then the form is submitted, you'll get a string of name/value pairs submitted like this: `interest=coding&interest=music`. When this string reaches the server, you need to parse it other than as an associative array, so all values, not only the last value, of `interest` are captured. For one technique used with Python, see Handle Multiple Checkboxes with a Single Serverside Variable, for example.

### Checking boxes by default

To make a checkbox checked by default, you give it the `checked` attribute. See the below example:

```
<fieldset>
  <legend>Choose your interests</legend>
  <div>
    <input type="checkbox" id="coding" name="interest" value="coding" checked />
    <label for="coding">Coding</label>
  </div>
  <div>
    <input type="checkbox" id="music" name="interest" value="music" />
    <label for="music">Music</label>
  </div>
</fieldset>
```

### Switch as a checkbox

The following example shows how to make a checkbox look and act like an on/off switch.

```
<fieldset>
  <legend>Adjust your setting</legend>
  <div>
    <label for="theme">Dark mode</label>
    <input type="checkbox" name="theme" id="theme" switch checked />
  </div>
  <div>
    <label for="notifications">Notifications</label>
    <input type="checkbox" name="notifications" id="notifications" switch />
  </div>
</fieldset>
```

> **Note:** While only some browsers render the checkbox as a switch, the behavior is the same across all browsers.

### Providing a bigger hit area for your checkboxes

In the above examples, you may have noticed that you can toggle a checkbox by clicking on its associated `<label>` element as well as on the checkbox itself. This is a really useful feature of HTML form labels that makes it easier to click the option you want, especially on small-screen devices like smartphones.

Beyond accessibility, this is another good reason to properly set up `<label>` elements on your forms.

### Indeterminate state checkboxes

A checkbox can be in an **indeterminate** state. This is set using the `HTMLInputElement` object's `indeterminate` property via JavaScript (it cannot be set using an HTML attribute):

```
inputInstance.indeterminate = true;
```

When `indeterminate` is `true`, the checkbox has a horizontal line in the box (it looks somewhat like a hyphen or minus sign) instead of a check/tick in most browsers.

> **Note:** This is purely a visual change. It has no impact on whether the checkbox's `value` is used in a form submission. That is decided by the `checked` state, regardless of the `indeterminate` state.

There are not many use cases for this property. The most common is when a checkbox is available that "owns" a number of sub-options (which are also checkboxes). If all of the sub-options are checked, the owning checkbox is also checked, and if they're all unchecked, the owning checkbox is unchecked. If any one or more of the sub-options have a different state than the others, the owning checkbox is in the indeterminate state.

This can be seen in the below example (thanks to CSS Tricks for the inspiration). In this example we keep track of the ingredients we are collecting for a recipe. When you check or uncheck an ingredient's checkbox, a JavaScript function checks the total number of checked ingredients:

- If none are checked, the recipe name's checkbox is set to unchecked.
- If one or two are checked, the recipe name's checkbox is set to `indeterminate`.
- If all three are checked, the recipe name's checkbox is set to `checked`.

So in this case the `indeterminate` state is used to state that collecting the ingredients has started, but the recipe is not yet complete.

```
const overall = document.querySelector("#enchantment");
const ingredients = document.querySelectorAll("ul input");

overall.addEventListener("click", (e) => {
  e.preventDefault();
});

for (const ingredient of ingredients) {
  ingredient.addEventListener("click", updateDisplay);
}

function updateDisplay() {
  let checkedCount = 0;
  for (const ingredient of ingredients) {
    if (ingredient.checked) {
      checkedCount++;
    }
  }

  if (checkedCount === 0) {
    overall.checked = false;
    overall.indeterminate = false;
```

```
  } else if (checkedCount === ingredients.length) {
    overall.checked = true;
    overall.indeterminate = false;
  } else {
    overall.checked = false;
    overall.indeterminate = true;
  }
}
```

```
<fieldset>
  <legend>Complete the recipe</legend>
  <div>
    <input type="checkbox" id="enchantment" name="enchantment" />
    <label for="enchantment">Enchantment table</label>
    <ul>
      <li>
        <input type="checkbox" id="book" name="ingredient" value="book" />
        <label for="book">Book</label>
      </li>
      <li>
        <input
          type="checkbox"
          id="diamonds"
          name="ingredient"
          value="diamonds" />
        <label for="diamonds">Diamonds (x2)</label>
      </li>
      <li>
        <input
          type="checkbox"
          id="obsidian"
          name="ingredient"
          value="obsidian" />
        <label for="obsidian">Obsidian (x4)</label>
      </li>
    </ul>
  </div>
</fieldset>
```

### Validation

Checkboxes do support validation (offered to all `<input>`s). However, most of the `ValidityState`s will always be `false`. If the checkbox has the `required` attribute, but is not checked, then `ValidityState.valueMissing` will be `true`.

### Examples

The following example is an extended version of the "multiple checkboxes" example we saw above — it has more standard options, plus an "other" checkbox that when checked causes a text field to appear to enter a value for the "other" option. This is achieved with a short block of JavaScript. The example includes implicit labels, with the `<input>` directly inside the `<label>`. The text input, without a visible label, includes the `aria-label` attribute which provides its accessible name. This example also includes some CSS to improve the styling.

#### HTML

```
<fieldset>
  <legend>Choose your interests</legend>
  <div>
    <label>
      <input type="checkbox" id="coding" name="interest" value="coding" />
      Coding
    </label>
  </div>
  <div>
    <label>
      <input type="checkbox" id="music" name="interest" value="music" />
      Music
    </label>
  </div>
  <div>
    <label>
      <input type="checkbox" id="art" name="interest" value="art" />
      Art
    </label>
  </div>
  <div>
    <label>
      <input type="checkbox" id="sports" name="interest" value="sports" />
```

```
      Sports
    </label>
  </div>
  <div>
    <label>
      <input type="checkbox" id="cooking" name="interest" value="cooking" />
      Cooking
    </label>
  </div>
  <div>
    <label>
      <input type="checkbox" id="other" name="interest" value="other" />
      Other
    </label>
    <input
      type="text"
      id="otherValue"
      name="other"
      aria-label="Other interest" />
  </div>
</fieldset>
```

### CSS

```
html {
  font-family: sans-serif;
}

div {
  margin-bottom: 10px;
}

fieldset {
  width: 600px;
  margin: 0 auto;
  background: cyan;
  border: 5px solid blue;
}

legend {
  padding: 10px;
  background: blue;
  color: cyan;
}
```

### JavaScript

```
const otherCheckbox = document.querySelector("#other");
const otherText = document.querySelector("#otherValue");
otherText.style.visibility = "hidden";

otherCheckbox.addEventListener("change", () => {
  if (otherCheckbox.checked) {
    otherText.style.visibility = "visible";
    otherText.value = "";
  } else {
    otherText.style.visibility = "hidden";
  }
});
```

### Technical summary

| | |
|---|---|
| **Value** | A string representing the value of the checkbox. |
| **Events** | `change` and `input` |
| **Supported common attributes** | `checked` and `switch` |
| **IDL attributes** | `checked`, `indeterminate` and `value` |
| **DOM interface** | `HTMLInputElement` |
| **Implicit ARIA Role** | `checkbox` |

### Browser compatibility

### See also

- `:checked`, `:indeterminate`: CSS selectors that let you style checkboxes based on their current state
- `HTMLInputElement`: HTML DOM API that implements the `<input>` element

### `<input type="color">` HTML attribute value

`<input>` elements of type **`color`** provide a user interface element that lets a user specify a color, either by using a visual color picker interface or by entering the color into a text field in a CSS color value format.

The element's presentation may vary substantially from one browser and/or platform to another—it might be a basic textual input that automatically validates to ensure that the color information is entered in the proper format, or a platform-standard color picker, or some kind of custom color picker window.

#### Try it

```
<p>Choose your colors:</p>

<div>
  <input type="color" id="foreground" name="foreground" value="#e66465" />
  <label for="foreground">Foreground color</label>
</div>

<div>
  <input
    type="color"
    id="background"
    name="background"
    value="oklab(50% 0.1 0.1 / 0.5)"
    colorspace="display-p3"
    alpha />
  <label for="background">Background color</label>
</div>
```

```
p,
label {
  font:
    1rem "Fira Sans",
    sans-serif;
}

input {
  margin: 0.4rem;
}
```

#### Value

A CSS color value.

> **Note:** Historically, only basic hexadecimal colors (without alpha channel) were allowed. Now, any CSS color format, including named colors, functional notations, and hexadecimal colors with an alpha channel, can be used. The default value is `#000000` (black) if a `value` is omitted or is invalid.

#### Additional attributes

In addition to the global attribute and the input attributes common to all `<input>` elements, the `color` input also supports the following attributes:

`alpha`

> A boolean attribute, if present, indicates the color's alpha component can be manipulated by the end user and does not have to be fully opaque.

`colorspace`

> Defines the color space for the color and hints at the desired user interface for the color picker widget. Possible enumerated values are:
>
> - `"limited-srgb"`: The color is in the sRGB color space. This includes `rgb()`, `hsl()`, `hwb()`, and `<hex-color>` values. The color value is limited to 8-bits per `r`, `g`, and `b` component. This is the default.
> - `"display-p3"`: The Display P3 color space, e.g., `color(display-p3 1.84 -0.19 0.72 / 0.6)`

#### Using color inputs

Inputs of type `color` are simple, due to the limited number of attributes they support.

#### Providing a default color

You can update the example above to set a default value, so that the color picker is pre-filled with the default color and the color picker (if any) will also default to that color.

```
<input type="color" value="#ff0000" />
<input
  type="color"
  id="body"
  name="body"
  value="oklab(50% 0.1 0.1 / 0.5)"
  colorspace="display-p3"
  alpha />
```

If you don't specify a value or if the value is invalid or otherwise not supported by the browser, the value defaults to `#000000`, which is opaque black.

### Tracking color changes

As is the case with other `<input>` types, there are two events that can be used to detect changes to the color value: `input` and `change`. `input` is fired on the `<input>` element every time the color changes. The `change` event is fired when the user dismisses the color picker. In both cases, you can determine the new value of the element by looking at its `value`.

Here's an example that watches changes over time to the color value:

```
colorPicker.addEventListener("input", updateFirst);
colorPicker.addEventListener("change", watchColorPicker);

function watchColorPicker(event) {
  document.querySelectorAll("p").forEach((p) => {
    p.style.color = event.target.value;
  });
}
```

### Selecting the value

When a browser doesn't support a color picker interface, its implementation of color inputs will be a text box that validates the contents automatically to ensure that the value is in the correct format. In this case you can use the `select()` method to select the text currently in the edit field.

If the browser instead uses a color picker, `select()` does nothing. You should be aware of this behavior so your code can respond appropriately in either case.

```
colorPicker.select();
```

### Validation

A color input's value is considered to be invalid if the user agent is unable to convert the user's input into seven-character lower-case hexadecimal notation. If and when this is the case, the `:invalid` pseudo-class is applied to the element.

### Example

Let's create an example which does a little more with the color input by tracking the `change` and `input` events to take the new color and apply it to every `<p>` element in the document.

#### HTML

The HTML is fairly straightforward — a couple of paragraphs of descriptive material with an `<input>` of type `color` with the ID `color-picker`, which we'll use to change the color of the paragraphs' text.

```
<p>
  An example demonstrating the use of the
  <code>&amp;lt;input type="color"&amp;gt;</code> control.
</p>

<label for="color-picker">Color:</label>
<input type="color" value="#ff0000" id="color-picker" />

<p>
  Watch the paragraph colors change when you adjust the color picker. As you
  make changes in the color picker, the first paragraph's color changes, as a
  preview (this uses the <code>input</code> event). When you close the color
  picker, the <code>change</code> event fires, and we detect that to change
  every paragraph to the selected color.
</p>
```

### JavaScript

#### Initialization

The following code initializes the color input:

```
const defaultColor = "#0000ff";
const colorPicker = document.querySelector("#color-picker");
colorPicker.value = defaultColor;
colorPicker.addEventListener("input", updateFirst);
colorPicker.addEventListener("change", updateAll);
colorPicker.select();
```

This gets a reference to the color `<input>` element in a variable called `colorPicker`, then sets the color input's value to the value in `defaultColor`. Then the color input's `input` event is set up to call our `updateFirst()` function, and the `change` event is set to call `updateAll()`. These are both seen below.

Finally, we call `select()` to select the text content of the color input if the control is implemented as a text field (this has no effect if a color picker interface is provided instead).

#### Reacting to color changes

We provide two functions that deal with color changes. The `updateFirst()` function is called in response to the `input` event. It changes the color of the first paragraph element in the document to match the new value of the color input. Since `input` events are fired every time an adjustment is made to the value (for example, if the brightness of the color is increased), these will happen repeatedly as the color picker is used.

```
function updateFirst(event) {
  const p = document.querySelector("p");
  if (p) {
    p.style.color = event.target.value;
  }
}
```

When the color picker is dismissed, indicating that the value will not change again (unless the user re-opens the color picker), a `change` event is sent to the element. We handle that event using the `updateAll()` function, using `Event.target.value` to obtain the final selected color:

```
function updateAll(event) {
  document.querySelectorAll("p").forEach((p) => {
    p.style.color = event.target.value;
  });
}
```

This sets the color of every `<p>` block so that its `color` attribute matches the current value of the color input, which is referred to using `event.target`.

#### Result

The final result looks like this:

### Technical summary

| | |
|---|---|
| **Value** | Any CSS `<color>` value in any notation. |
| **Events** | `change` and `input` |
| **Supported common attributes** | `autocomplete` and `list` |
| **IDL attributes** | `alpha`, `colorSpace`, `list`, and `value` |
| **DOM interface** | `HTMLInputElement` |
| **Implicit ARIA Role** | no corresponding role |

### Browser compatibility

### See also

- `HTMLInputElement.alpha`
- `HTMLInputElement.colorspace`

## <input type="date"> HTML attribute value

`<input>` elements of **`type="date"`** create input fields that let the user enter a date. The appearance of the date picker input UI varies based on the browser and operating system. The value is normalized to the format `yyyy-mm-dd`.

The resulting value includes the year, month, and day, but *not* the time. The time and datetime-local input types support time and date+time input.

### Try it

```
<label for="start">Start date:</label>

<input
  type="date"
  id="start"
  name="trip-start"
  value="2018-07-22"
  min="2018-01-01"
  max="2018-12-31" />
```

```
label {
  display: block;
  font:
    1rem "Fira Sans",
    sans-serif;
}

input,
label {
  margin: 0.4rem 0;
}
```

### Value

A string representing the date entered in the input. The date is formatted according to Date strings format.

You can set a default value for the input with a date inside the `value` attribute, like so:

```
<input type="date" value="2017-06-01" />
```

> **Note:** The displayed date format will differ from the actual `value` — the displayed date is formatted *based on the locale of the user's browser*, but the parsed `value` is always formatted `yyyy-mm-dd`.

You can get and set the date value in JavaScript with the `HTMLInputElement` `value` and `valueAsNumber` properties. For example:

```
const dateControl = document.querySelector('input[type="date"]');
dateControl.value = "2017-06-01";
console.log(dateControl.value); // prints "2017-06-01"
console.log(dateControl.valueAsNumber); // prints 1496275200000, a JavaScript timestamp (ms)
```

This code finds the first `<input>` element whose `type` is `date`, and sets its value to `2017-06-01` (June 1st, 2017). It then reads that value back in string and number formats.

### Additional attributes

In addition to global attributes and the input attributes common to all `<input>` elements, the `date` input supports the following attributes:

#### max

The latest date to accept. If the `value` entered into the element occurs afterward, the element fails constraint validation. If the value of the `max` attribute isn't a possible date string in the format `yyyy-mm-dd`, then the element has no maximum date value.

If both the `max` and `min` attributes are set, this value must be a date string **later than or equal to** the one in the `min` attribute.

#### min

The earliest date to accept. If the `value` entered into the element occurs beforehand, the element fails constraint validation. If the value of the `min` attribute isn't a possible date string in the format `yyyy-mm-dd`, then the element has no minimum date value.

If both the `max` and `min` attributes are set, this value must be a date string **earlier than or equal to** the one in the `max` attribute.

#### step

The `step` attribute is a number that specifies the granularity that the value must adhere to, or the special value `any`, which is described below. Only values which are a whole number of steps from the step base are valid. The step base is `min` if specified, `value` otherwise, or `0` (the Unix epoch, `1970-01-01`) if neither is provided.

For `date` inputs, the value of `step` is given in days and is treated as a number of milliseconds equal to 86,400,000 times the `step` value (the underlying numeric value is in milliseconds). The default value is 1, indicating 1 day.

A string value of `any` means that no stepping is implied, and any value is allowed (barring other constraints, such as `min` and `max`). In reality, it has the same effect as `1` for `date` inputs because the picker UI only allows selecting whole days.

> **Note:** When the data entered by the user doesn't adhere to the stepping configuration, the user agent may round to the nearest valid value, preferring numbers in the positive direction when there are two equally close options.

### Using date inputs

Date inputs provide an easy interface for choosing dates, and they normalize the data format sent to the server regardless of the user's locale.

In this section, we'll look at basic and then more complex uses of `<input type="date">`.

### Basic uses of date

The most basic use of `<input type="date">` involves one `<input>` combined with its `<label>`, as seen below:

```
<label>
  Enter your birthday:
  <input type="date" name="bday" />
</label>
```

If this input is included in a form submitted to `https://example.com` using the `GET` method, the entered date is sent under the key `bday` — resulting in a URL like `https://example.com/?bday=1955-06-08`.

### Setting maximum and minimum dates

You can use the `min` and `max` attributes to restrict the dates that can be chosen by the user. In the following example, we set a minimum date of `2017-04-01` and a maximum date of `2017-04-30`:

```
<label>
  Choose your preferred party date:
  <input type="date" name="party" min="2017-04-01" max="2017-04-30" />
</label>
```

The result is that only days in April 2017 can be selected — the month and year parts of the textbox will be uneditable, and dates outside April 2017 can't be selected in the picker widget.

You can use the `step` attribute to vary the number of days jumped each time the date is incremented (e.g., to only make Saturdays selectable).

### Controlling input size

`<input type="date">` doesn't support form sizing attributes such as `size`. Prefer CSS for sizing it.

### Validation

By default, `<input type="date">` doesn't validate the entered value beyond its format. The interfaces generally don't let you enter anything that isn't a date — which is helpful.

If you use `min` and `max` to restrict the available dates (see Setting maximum and minimum dates), the form control disables invalid dates, and will display an error if you try to submit a date that is out of bounds.

You can also use the `required` attribute to make filling in the date mandatory — an error will be displayed if you try to submit an empty date field.

Let's look at an example of minimum and maximum dates, and also make a field required:

```
<form>
  <label>
    Choose your preferred party date (required, April 1st to 20th):
    <input
      type="date"
      name="party"
      min="2017-04-01"
      max="2017-04-20"
      required />
    <span class="validity"></span>
  </label>

  <p>
    <button>Submit</button>
  </p>
</form>
```

If you try to submit the form with an incomplete date (or with a date outside the set bounds), the browser displays an error. Try playing with the example now:

Here's the CSS used in the above example. We make use of the `:valid` and `:invalid` pseudo-elements to add an icon next to the input, based on whether the current value is valid. We had to put the icon on a `<span>` next to the input, not on the input itself, because in Chrome at least the input's generated content is placed inside the form control, and can't be styled or shown effectively.

```
label {
  display: flex;
  align-items: center;
}

span::after {
  padding-left: 5px;
}

input:invalid + span::after {
  content: "✖";
}

input:valid + span::after {
  content: "✓";
}
```

> **Warning:** Client-side form validation *is not a substitute* for validating on the server. It's easy for someone to modify the HTML, or bypass your HTML entirely and submit the data directly to your server. If your server fails to validate the received data, disaster could strike with data that is badly-formatted, too large, of the wrong type, etc.

### Examples

In this example, we create a date picker using the native `<input type="date">` picker.

### HTML

The HTML looks like so:

```
<div class="nativeDatePicker">
  <label for="bday">Enter your birthday:</label>
  <input type="date" id="bday" name="bday" />
  <span class="validity"></span>
</div>
```

### CSS

The CSS looks like so:

```
input:invalid + span::after {
  content: " ✖";
}

input:valid + span::after {
  content: " ✓";
}
```

### Results

### Technical summary

| | |
|---|---|
| **Value** | A string representing a date in YYYY-MM-DD format, or empty |
| **Events** | `change` and `input` |
| **Supported common attributes** | `autocomplete`, `list`, `readonly`, `step` |
| **IDL attributes** | `value`, `valueAsDate`, `valueAsNumber` |
| **DOM interface** | `HTMLInputElement` |
| **Implicit ARIA Role** | no corresponding role |

### See also

- The generic `<input>` element and the interface used to manipulate it, `HTMLInputElement`
- Date and Time picker tutorial
- Date and time formats used in HTML

## `<input type="datetime-local">` HTML attribute value

`<input>` elements of type **`datetime-local`** create input controls that let the user easily enter both a date and a time, including the year, month, and day as well as the time in hours and minutes.

The control's UI varies in general from browser to browser. The control is intended to represent *a local date and time*, not necessarily *the user's local date and time*. In other words, the input allows any valid combination of year, month, day, hour, and minute—even if such a combination is invalid in the user's local time zone (such as the one hour within a daylight saving time spring-forward transition gap).

### Try it

```
<label for="meeting-time">Choose a time for your appointment:</label>

<input
  type="datetime-local"
  id="meeting-time"
  name="meeting-time"
  value="2018-06-12T19:30"
  min="2018-06-07T00:00"
  max="2018-06-14T00:00" />
```

```
label {
  display: block;
  font:
    1rem "Fira Sans",
    sans-serif;
}

input,
label {
  margin: 0.4rem 0;
}
```

### Value

A string representing the value of the date entered into the input. The format of the date and time value used by this input type is described in Local date and time strings.

You can set a default value for the input by including a date and time inside the `value` attribute, like so:

```
<label for="party">Enter a date and time for your party booking:</label>
<input
  id="party"
  type="datetime-local"
  name="party-date"
  value="2017-06-01T08:30" />
```

One thing to note is that the displayed date and time formats differ from the actual `value`; the displayed date and time are formatted according to the user's locale as reported by their operating system, whereas the date/time `value` is always formatted `YYYY-MM-DDTHH:mm`. When the above value is submitted to the server, for example, it will look like `party-date=2024-06-01T08:30`.

> **Note:** Also bear in mind that if such data is submitted via HTTP `GET`, the colon character will need to be escaped for inclusion in the URL parameters, e.g., `party-date=2024-06-01T08%3A30`. See `encodeURI()` for one way to do this.

You can also get and set the date value in JavaScript using the `HTMLInputElement` `value` property, for example:

```
const dateControl = document.querySelector('input[type="datetime-local"]');
dateControl.value = "2017-06-01T08:30";
```

### Additional attributes

In addition to the attributes common to all `<input>` elements, `datetime-local` inputs offer the following attributes.

#### max

The latest date and time to accept. If the `value` entered into the element is later than this timestamp, the element fails constraint validation. If the value of the `max` attribute isn't a valid string that follows the format `YYYY-MM-DDTHH:mm`, then the element has no maximum value.

This value must specify a date string later than or equal to the one specified by the `min` attribute.

#### min

The earliest date and time to accept; timestamps earlier than this will cause the element to fail constraint validation. If the value of the `min` attribute isn't a valid string that follows the format `YYYY-MM-DDTHH:mm`, then the element has no minimum value.

This value must specify a date string earlier than or equal to the one specified by the `max` attribute.

#### step

The `step` attribute is a number that specifies the granularity that the value must adhere to, or the special value `any`, which is described below. Only values which are a whole number of steps from the step base are valid. The step base is `min` if specified, `value` otherwise, or `0` (the Unix epoch, `1970-01-01T00:00`) if neither is provided.

For `datetime-local` inputs, the value of `step` is given in seconds and is treated as a number of milliseconds equal to 1000 times the `step` value (the underlying numeric value is in milliseconds). The default value is 60, indicating 1 minute.

A string value of `any` means that no stepping is implied, and any value is allowed (barring other constraints, such as `min` and `max`). In reality, it has the same effect as `60` for `datetime-local` inputs because the picker UI in this case only allows selecting whole minutes.

> **Note:** When the data entered by the user doesn't adhere to the stepping configuration, the user agent may round to the nearest valid value, preferring numbers in the positive direction when there are two equally close options.

### Using datetime-local inputs

Date/time inputs are convenient for the developer; they provide an easy UI for choosing dates and times, and they normalize the data format sent to the server, regardless of the user's locale. However, it is important to consider your users. Don't require your users to enter data that is not needed for your app to function.

### Controlling input size

`<input type="datetime-local">` doesn't support form control sizing attributes such as `size`. You'll have to resort to CSS for customizing the sizes of these elements.

### Setting timezones

One thing the `datetime-local` input type doesn't provide is a way to set the time zone and/or locale of the date/time control. This was available in the `datetime` input type, but this type is now obsolete, having been removed from the spec. The main reasons why this was removed are a lack of implementation in browsers and concerns over the user interface/experience. It is easier to just have a control (or controls) for setting the date/time and then deal with the locale in a separate control.

For example, if you are creating a system where the user is likely to already be logged in, with their locale already set, you could provide the timezone in a `hidden` input type. For example:

```
<input type="hidden" id="timezone" name="timezone" value="-08:00" />
```

On the other hand, if you were required to allow the user to enter a time zone along with a date/time input, you could have a `<select>` element to enable the user to set the right time zone by choosing a particular location from among a set of locations:

```
<select name="timezone" id="timezone">
  <option value="Pacific/Kwajalein">Eniwetok, Kwajalein</option>
  <option value="Pacific/Midway">Midway Island, Samoa</option>
  <option value="Pacific/Honolulu">Hawaii</option>
  <option value="Pacific/Marquesas">Taiohae</option>
  <!-- and so on -->
</select>
```

In either case, the date/time and time zone values would be submitted to the server as separate data points, and then you'd need to store them appropriately in the database on the server-side.

### Validation

By default, `<input type="datetime-local">` does not apply any validation to entered values. The UI implementations generally don't let you enter anything that isn't a date/time — which is helpful — but a user might still fill in no value and submit, or enter an invalid date and/or time (e.g., the 32nd of April).

You can use `min` and `max` to restrict the available dates (see Setting maximum and minimum dates), and you can use the `required` attribute to make filling in the date/time mandatory. As a result, browsers will display an error if you try to submit a date that is outside the set bounds or an empty date field.

Let's look at an example; here we've set minimum and maximum date/time values, and also made the field required:

```
<form>
  <div>
    <label for="party">
      Choose your preferred party date and time (required, June 1st 8.30am to
      June 30th 4.30pm):
```

```
    </label>
    <input
      id="party"
      type="datetime-local"
      name="party-date"
      min="2017-06-01T08:30"
      max="2017-06-30T16:30"
      required />
    <span class="validity"></span>
  </div>
  <div>
    <input type="submit" value="Book party!" />
  </div>
</form>
```

If you try to submit the form with an incomplete date (or with a date outside the set bounds), the browser displays an error. Try playing with the example now:

Here's the CSS used in the above example. Here we make use of the `:valid` and `:invalid` CSS properties to style the input based on whether the current value is valid. We put the icons on a `<span>` next to the input.

```
div {
  margin-bottom: 10px;
  display: flex;
  align-items: center;
}

label {
  display: inline-block;
  width: 300px;
}

input:invalid + span::after {
  content: "✖";
  padding-left: 5px;
}

input:valid + span::after {
  content: "✓";
  padding-left: 5px;
}
```

> **Warning:** HTML form validation is *not* a substitute for scripts that ensure that the entered data is in the proper format. It's far too easy for someone to make adjustments to the HTML that allow them to bypass the validation, or to remove it entirely. It's also possible for someone to bypass your HTML entirely and submit the data directly to your server. If your server-side code fails to validate the data it receives, problems can arise when improperly-formatted data is submitted (or data that is too large, is of the wrong type, and so forth).

> **Note:** With a `datetime-local` input, the date value is always normalized to the format `YYYY-MM-DDTHH:mm`.

### Examples

#### Basic uses of datetime-local

The most basic use of `<input type="datetime-local">` involves a basic `<input>` and `<label>` element combination, as seen below:

```
<label for="party">Enter a date and time for your party booking:</label>
<input id="party" type="datetime-local" name="party-date" />
```

#### Setting maximum and minimum dates and times

You can use the `min` and `max` attributes to restrict the dates/times that can be chosen by the user. In the following example, we are setting a minimum datetime of `2025-06-01T08:30` and a maximum datetime of `2025-06-30T16:30`:

```
<label for="party">Enter a date and time for your party booking:</label>
<input
  id="party"
  type="datetime-local"
  name="party-date"
  min="2025-06-01T08:30"
  max="2025-06-30T16:30" />
```

Only days in June 2025 can be selected. Depending on what browser you are using, times outside the specified values might not be selectable. In other browsers, invalid dates and times are selectable but will match `:invalid` and `:out-of-range` and will fail validation.

In some browsers (Safari), the date picker will appear to allow any date, but the value will be clamped to the valid range when a date is selected.

The valid range included all times between the `min` and `max` values; the time of day is only constrained on the first and last dates in the range.

> **Note:** You should be able to use the `step` attribute to vary the number of days jumped each time the date is incremented (e.g., maybe you only want to make Saturdays selectable). However, this does not seem to work effectively in any implementation at the time of writing.

### Technical summary

| | |
|---|---|
| **Value** | A string representing a date and time (in the local time zone), or empty. |
| **Events** | `change` and `input` |
| **Supported common attributes** | `autocomplete`, `list`, `readonly`, `step` |
| **IDL attributes** | `list`, `value`, `valueAsDate`, `valueAsNumber` |
| **DOM interface** | `HTMLInputElement` |
| **Implicit ARIA Role** | no corresponding role |

### See also

- The generic `<input>` element and the interface used to manipulate it, `HTMLInputElement`
- `<input type="date">` and `<input type="time">`
- Date and time formats used in HTML
- Date and Time picker tutorial

## `<input type="email">` HTML attribute value

`<input>` elements of type **`email`** are used to let the user enter and edit an email address, or, if the `multiple` attribute is specified, a list of email addresses.

The input value is automatically validated to ensure that it's either empty or a properly-formatted email address (or list of addresses) before the form can be submitted. The `:valid` and `:invalid` CSS pseudo-classes are automatically applied as appropriate to visually denote whether the current value of the field is a valid email address or not.

### Try it

```
<label for="email">Enter your example.com email:</label>

<input type="email" id="email" pattern=".+@example\.com" size="30" required />
```

```
label {
  display: block;
  font:
    1rem "Fira Sans",
    sans-serif;
}

input,
label {
  margin: 0.4rem 0;
}
```

### Value

The `<input>` element's `value` attribute contains a string which is automatically validated as conforming to email syntax. More specifically, there are three possible value formats that will pass validation:

1. An empty string ("") indicating that the user did not enter a value or that the value was removed.
2. A single properly-formed email address. This doesn't necessarily mean the email address exists, but it is at least formatted correctly. This means `username@domain` or `username@domain.tld`. There's more to it than that, of course; see Validation for a regular expression that matches the email address validation algorithm.
3. If and only if the `multiple` attribute is specified, the value can be a list of properly-formed comma-separated email addresses. Any trailing and leading whitespace is removed from each address in the list.

See Validation for details on how email addresses are validated to ensure that they're formatted properly.

### Additional attributes

In addition to the global attributes, and the attributes that operate on all `<input>` elements regardless of their type, `email` inputs support the following attributes.

> **Note:** The `autocorrect` global attribute can be added to email inputs, but the stored state is always `off`.

#### list

The values of the list attribute is the `id` of a `<datalist>` element located in the same document. The `<datalist>` provides a list of predefined values to suggest to the user for this input. Any values in the list that are not compatible with the `type` are not included in the suggested options. The values provided are suggestions, not requirements: users can select from this predefined list or provide a different value.

#### maxlength

The maximum string length (measured in UTF-16 code units) that the user can enter into the `email` input. This must be an integer value of 0 or higher. If no `maxlength` is specified, or an invalid value is specified, the `email` input has no maximum length. This value must also be greater than or equal to the value of `minlength`.

The input will fail constraint validation if the length of the text value of the field is greater than `maxlength` UTF-16 code units long. Constraint validation is only applied when the value is changed by the user.

#### minlength

The minimum string length (measured in UTF-16 code units) that the user can enter into the `email` input. This must be a non-negative integer value smaller than or equal to the value specified by `maxlength`. If no `minlength` is specified, or an invalid value is specified, the `email` input has no minimum length.

The input will fail constraint validation if the length of the text entered into the field is fewer than `minlength` UTF-16 code units long. Constraint validation is only applied when the value is changed by the user.

#### multiple

A Boolean attribute which, if present, indicates that the user can enter a list of multiple email addresses, separated by commas and, optionally, whitespace characters. See Allowing multiple email addresses for an example, or HTML attribute: multiple for more details.

> **Note:** Normally, if you specify the `required` attribute, the user must enter a valid email address for the field to be considered valid. However, if you add the `multiple` attribute, a list of zero email addresses (an empty string, or one which is entirely whitespace) is a valid value. In other words, the user does not have to enter even one email address when `multiple` is specified, regardless of the value of `required`.

#### pattern

The `pattern` attribute, when specified, is a regular expression that the input's `value` must match for the value to pass constraint validation. It must be a valid JavaScript regular expression, as used by the `RegExp` type, and as documented in our guide on regular expressions; the `'u'` flag is specified when compiling the regular expression so that the pattern is treated as a sequence of Unicode code points, instead of as ASCII. No forward slashes should be specified around the pattern text.

If the specified pattern is not specified or is invalid, no regular expression is applied and this attribute is ignored completely.

> **Note:** Use the `title` attribute to specify text that most browsers will display as a tooltip to explain what the requirements are to match the pattern. You should also include other explanatory text nearby.

See the section Pattern validation for details and an example.

#### <code>placeholder</code>

The `placeholder` attribute is a string that provides a brief hint to the user as to what kind of information is expected in the field. It should be a word or short phrase that demonstrates the expected type of data, rather than an explanatory message. The text *must not* include carriage returns or line feeds.

If the control's content has one directionality (LTR or RTL) but needs to present the placeholder in the opposite directionality, you can use Unicode bidirectional algorithm formatting characters to override directionality within the placeholder; see How to use Unicode controls for bidi text for more information.

> **Note:** Avoid using the `placeholder` attribute if you can. It is not as semantically useful as other ways to explain your form, and can cause unexpected technical issues with your content. See `<input>` labels for more information.

### <code>readonly</code>

A Boolean attribute which, if present, means this field cannot be edited by the user. Its `value` can, however, still be changed by JavaScript code directly setting the `HTMLInputElement` `value` property.

> **Note:** Because a read-only field cannot have a value, `required` does not have any effect on inputs with the `readonly` attribute also specified.

### <code>size</code>

The `size` attribute is a numeric value indicating how many characters wide the input field should be. The value must be a number greater than zero, and the default value is 20. Since character widths vary, this may or may not be exact and should not be relied upon to be so; the resulting input may be narrower or wider than the specified number of characters, depending on the characters and the font (`font` settings in use).

This does *not* set a limit on how many characters the user can enter into the field. It only specifies approximately how many can be seen at a time. To set an upper limit on the length of the input data, use the `maxlength` attribute.

### Using email inputs

Email addresses are among the most frequently-inputted textual data forms on the web; they're used when logging into websites, when requesting information, to allow order confirmation, for webmail, and so forth. As such, the `email` input type can make your job as a web developer much easier since it can help simplify your work when building the user interface and logic for email addresses. When you create an email input with the proper `type` value, `email`, you get automatic validation that the entered text is at least in the correct form to potentially be a legitimate email address. This can help avoid cases in which the user mistypes their address, or provides an invalid address.

It's important, however, to note that this is not enough to ensure that the specified text is an email address which actually exists, corresponds to the user of the site, or is acceptable in any other way. It ensures that the value of the field is properly formatted to be an email address.

> **Note:** It's also crucial to remember that a user can tinker with your HTML behind the scenes, so your site *must not* use this validation for any security purposes. You *must* verify the email address on the server side of any transaction in which the provided text may have any security implications of any kind.

### A basic email input

Currently, all browsers which implement this element implement it as a standard text input field with basic validation features. The specification does, however, allow browsers latitude on this. For example, the element could be integrated with the user's device's built-in address book to allow picking email addresses from that list. In its most basic form, an `email` input can be implemented like this:

```
<input id="emailAddress" type="email" />
```

Notice that it's considered valid when empty and when a single validly-formatted email address is entered, but is otherwise not considered valid. By adding the `required` attribute, only validly-formed email addresses are allowed; the input is no longer considered valid when empty.

### Allowing multiple email addresses

By adding the `multiple` Boolean attribute, the input can be configured to accept multiple email addresses.

```
<input id="emailAddress" type="email" multiple />
```

The input is now considered valid when a single email address is entered, or when any number of email addresses separated by commas and, optionally, some number of whitespace characters are present.

> **Note:** When `multiple` is used, the value *is* allowed to be empty.

Some examples of valid strings when `multiple` is specified:

- `""`
- `"me@example"`
- `"me@example.org"`
- `"me@example.org,you@example.org"`
- `"me@example.org, you@example.org"`
- `"me@example.org,you@example.org, us@example.org"`

Some examples of invalid strings:

- `","`
- `"me"`

- `"me@example.org you@example.org"`

### Placeholders

Sometimes it's helpful to offer an in-context hint as to what form the input data should take. This can be especially important if the page design doesn't offer descriptive labels for each `<input>`. This is where **placeholders** come in. A placeholder is a value that demonstrates the form the `value` should take by presenting an example of a valid value, which is displayed inside the edit box when the element's `value` is "". Once data is entered into the box, the placeholder disappears; if the box is emptied, the placeholder reappears.

Here, we have an `email` input with the placeholder `sophie@example.com`. Note how the placeholder disappears and reappears as you manipulate the contents of the edit field.

```
<input type="email" placeholder="sophie@example.com" />
```

### Controlling the input size

You can control not only the physical length of the input box, but also the minimum and maximum lengths allowed for the input text itself.

### Physical input element size

The physical size of the input box can be controlled using the `size` attribute. With it, you can specify the number of characters the input box can display at a time. In this example the `email` edit box is 15 characters wide:

```
<input type="email" size="15" />
```

### Element value length

The `size` is separate from the length limitation on the entered email address itself so that you can have fields fit in a small space while still allowing longer email address strings to be entered. You can specify a minimum length, in characters, for the entered email address using the `minlength` attribute; similarly, use `maxlength` to set the maximum length of the entered email address.

The example below creates a 32 character-wide email address entry box, requiring that the contents be no shorter than 3 characters and no longer than 64 characters.

```
<input type="email" size="32" minlength="3" maxlength="64" />
```

### Providing default options

### Providing a single default using the value attribute

As always, you can provide a default value for an `email` input box by setting its `value` attribute:

```
<input type="email" value="default@example.com" />
```

### Offering suggested values

Taking it a step further, you can provide a list of default options from which the user can select by specifying the `list` attribute. This doesn't limit the user to those options, but does allow them to select commonly-used email addresses more quickly. This also offers hints to `autocomplete`. The `list` attribute specifies the ID of a `<datalist>`, which in turn contains one `<option>` element per suggested value; each `option`'s `value` is the corresponding suggested value for the email entry box.

```
<input type="email" size="40" list="defaultEmails" />

<datalist id="defaultEmails">
  <option value="jbond007@mi6.defence.gov.uk"></option>
  <option value="jbourne@unknown.net"></option>
  <option value="nfury@shield.org"></option>
  <option value="tony@starkindustries.com"></option>
  <option value="hulk@grrrrrrrr.arg"></option>
</datalist>
```

With the `<datalist>` element and its `<option>`s in place, the browser will offer the specified values as potential values for the email address; this is typically presented as a popup or drop-down menu containing the suggestions. While the specific user experience may vary from one browser to another, typically clicking in the edit box presents a drop-down of the suggested email addresses. Then, as the user types, the list is filtered to show only matching values. Each typed character narrows down the list until the user makes a selection or types a custom value.

### Validation

There are two levels of content validation available for `email` inputs. First, there's the standard level of validation offered to all `<input>`s, which automatically ensures that the contents meet the requirements to be a valid email address. But there's also the option to add additional filtering to ensure that your own specialized needs are met, if you have any.

> **Warning:** HTML form validation is *not* a substitute for scripts that ensure that the entered data is in the proper format. It's far too easy for someone to make adjustments to the HTML that allow them to bypass the validation, or to remove it completely. It's also possible for someone to bypass your HTML entirely and submit the data directly to your server. If your server-side code fails to validate the data it receives, disaster could strike when improperly-formatted data (or data which is too large, is of the wrong type, and so forth) is entered into your database.

### Basic validation

Browsers automatically provide validation to ensure that only text that matches the standard format for Internet email addresses is entered into the input box. Browsers use an algorithm equivalent to the following regular expression:

```
/^[\w.!#$%&'*+/=?^`{|}~-]+@[a-z\d](?:[a-z\d-]{0,61}[a-z\d])?(?:\.[a-z\d](?:[a-z\d-]{0,61}[a-z\d])?)*$/i;
```

To learn more about how form validation works and how to take advantage of the `:valid` and `:invalid` CSS properties to style the input based on whether the current value is valid, see Form data validation.

> **Note:** There are known specification issues related to international domain names and the validation of email addresses in HTML. See W3C bug 15489 and whatwg/html#4562 for details.

### Pattern validation

If you need the entered email address to be restricted further than just "any string that looks like an email address," you can use the `pattern` attribute to specify a regular expression the value must match for it to be valid. If the `multiple` attribute is specified, each individual item in the comma-delineated list of values must match the regular expression.

For example, let's say you're building a page for employees of Best Startup Ever, Inc. which will let them contact their IT department for help. In our simplified form, the user needs to enter their email address and a message describing the problem they need help with. We want to ensure that not only does the user provide a valid email address, but for security purposes, we require that the address be an internal corporate email address.

Since inputs of type `email` validate against both the standard email address validation *and* the specified `pattern`, you can implement this easily. Let's see how:

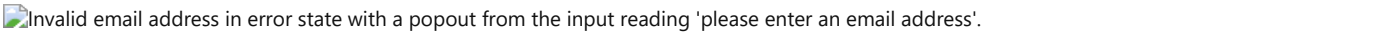
```
<form>
  <div class="emailBox">
    <label for="emailAddress">Your email address</label><br />
    <input
      id="emailAddress"
      type="email"
      size="64"
      maxlength="64"
      required
      placeholder="username@beststartupever.com"
      pattern=".+@beststartupever\.com"
      title="Please provide only a Best Startup Ever corporate email address" />
  </div>

  <div class="messageBox">
    <label for="message">Request</label><br />
    <textarea
      id="message"
      cols="80"
      rows="8"
      required
      placeholder="My shoes are too tight, and I have forgotten how to dance."></textarea>
  </div>
  <input type="submit" value="Send Request" />
</form>
```

Our `<form>` contains one `<input>` of type `email` for the user's email address, a `<textarea>` to enter their message for IT into, and an `<input>` of type `"submit"`, which creates a button to submit the form. Each text entry box has a `<label>` associated with it to let the user know what's expected of them.

Let's take a closer look at the email address entry box. Its `size` and `maxlength` attributes are both set to 64 in order to show room for 64 characters worth of email address, and to limit the number of characters actually entered to a maximum of 64. The `required` attribute is specified, making it mandatory that a valid email address be provided.

An appropriate `placeholder` is provided—`username@beststartupever.com`—to demonstrate what constitutes a valid entry. This string demonstrates both that an email address should be entered, and suggests that it should be a corporate beststartupever.com account. This is in addition to the fact that using type `email` will validate the text to ensure that it's formatted like an email address. If the text in the input box isn't an email address, you'll get an error message that looks something like this:

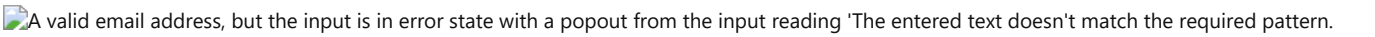

Invalid email address in error state with a popout from the input reading 'please enter an email address'.

If we left things at that, we would at least be validating on legitimate email addresses. But we want to go one step farther: we want to make sure that the email address is in fact in the form `[username]@beststartupever.com`. This is where we'll use `pattern`. We set `pattern` to `.+@beststartupever.com`. This regular expression requests a string that consists of at least one character of any kind, then an "@" followed by the domain name "beststartupever.com".

Note that this is not even close to an adequate filter for valid email addresses; it would allow things such as " @beststartupever.com" (note the leading space) or "@@beststartupever.com", neither of which is valid. However, the browser runs both the standard email address filter *and* our custom pattern against the specified text. As a result, we wind up with a validation which says "make sure this resembles a valid email address, and if it is, make sure it's also a beststartupever.com address."

It's advisable to use the `title` attribute along with `pattern`. If you do, the `title` *must* describe the pattern. That is, it should explain what format the data should take on, rather than any other information. That's because the `title` may be displayed or spoken as part of a validation error message. For example, the browser might present the message "The entered text doesn't match the required pattern." followed by your specified `title`. If your `title` is something like "Email address", the result would be the message "The entered text doesn't match the required pattern. Email address", which isn't very good.

That's why, instead, we specify the string "Please provide only a Best Startup Ever corporate email address" By doing that, the resulting full error message might be something like "The entered text doesn't match the required pattern. Please provide only a Best Startup Ever corporate email address."


A valid email address, but the input is in error state with a popout from the input reading 'The entered text doesn't match the required pattern. Please provide only a Best Startup Ever corporate email address.'

> **Note:** If you run into trouble while writing your validation regular expressions and they're not working properly, check your browser's console; there may be helpful error messages there to aid you in solving the problem.

### Examples

Here we have an email input with the ID `emailAddress` which is allowed to be up to a maximum of 256 characters long. The input box itself is physically 64 characters wide, and displays the text `user@example.gov` as a placeholder anytime the field is empty. In addition, by using the `multiple` attribute, the box is configured to allow the user to enter zero or more email addresses, separated by commas, as described in Allowing multiple email addresses. As a final touch, the `list` attribute contains the ID of a `<datalist>` whose `<option>`s specify a set of suggested values the user can choose from.

As an added touch, the `<label>` element is used to establish a label for the email entry box, with its `for` attribute referencing the `emailAddress` ID of the `<input>` element. By associating the two elements in this way, clicking on the label will focus the input element.

```
<label for="emailAddress">Email</label><br />
<input
  id="emailAddress"
  type="email"
  placeholder="user@example.gov"
  list="defaultEmails"
  size="64"
  maxlength="256"
  multiple />

<datalist id="defaultEmails">
  <option value="jbond007@mi6.defence.gov.uk"></option>
  <option value="jbourne@unknown.net"></option>
  <option value="nfury@shield.org"></option>
  <option value="tony@starkindustries.com"></option>
  <option value="hulk@grrrrrrrr.arg"></option>
</datalist>
```

### Technical summary

| | |
|---|---|
| **Value** | A string representing an email address, or empty |
| **Events** | `change` and `input` |
| **Supported Common Attributes** | `autocomplete`, `list`, `maxlength`, `minlength`, `multiple`, `name`, `pattern`, `placeholder`, `readonly`, `required`, `size`, and `type` |
| **IDL attributes** | `list` and `value` |
| **DOM interface** | `HTMLInputElement` |
| **Implicit ARIA Role** | with no `list` attribute: `textbox`<br>with `list` attribute: `combobox` |

### See also

- HTML forms guide
- `<input>`
- `<input type="tel">`
- `<input type="url">`
- Attributes:
  - `list`
  - `minlength`
  - `maxlength`
  - `multiple`
  - `pattern`
  - `placeholder`
  - `readonly`
  - `size`

## <input type="file"> HTML attribute value

`<input>` elements with **`type="file"`** let the user choose one or more files from their device storage. Once chosen, the files can be uploaded to a server using form submission, or manipulated using JavaScript code and the File API.

### Try it

```
<label for="avatar">Choose a profile picture:</label>

<input type="file" id="avatar" name="avatar" accept="image/png, image/jpeg" />
```

```
label {
  display: block;
  font:
    1rem "Fira Sans",
    sans-serif;
}

input,
label {
  margin: 0.4rem 0;
}
```

### Value

A file input's `value` attribute contains a string that represents the path to the selected file(s). If no file is selected yet, the value is an empty string (`""`). When the user selected multiple files, the `value` represents the first file in the list of files they selected. The other files can be identified using the input's `HTMLInputElement.files` property.

> **Note:** The value is always the file's name prefixed with `C:\fakepath\`, which isn't the real path of the file. This is to prevent malicious software from guessing the user's file structure.

### Additional attributes

In addition to the common attributes shared by all `<input>` elements, inputs of type `file` also support the following attributes.

#### accept

The `accept` attribute value is a string that defines the file types the file input should accept. This string is a comma-separated list of **unique file type specifiers**. Because a given file type may be identified in more than one manner, it's useful to provide a thorough set of type specifiers when you need files of a given format.

For instance, there are a number of ways Microsoft Word files can be identified, so a site that accepts Word files might use an `<input>` like this:

```
<input
  type="file"
  id="docpicker"
  accept=".doc,.docx,.xml,application/msword,application/vnd.openxmlformats-officedocument.wordprocessingml.document" />
```

#### capture

The `capture` attribute value is a string that specifies which camera to use for capture of image or video data, if the `accept` attribute indicates that the input should be of one of those types. A value of `user` indicates that the user-facing camera and/or microphone should be used. A value of

`environment` specifies that the outward-facing camera and/or microphone should be used. If this attribute is missing, the user agent is free to decide on its own what to do. If the requested facing mode isn't available, the user agent may fall back to its preferred default mode.

> **Note:** `capture` was previously a Boolean attribute which, if present, requested that the device's media capture device(s) such as camera or microphone be used instead of requesting a file input.

### multiple

When the `multiple` Boolean attribute is specified, the file input allows the user to select more than one file.

### webkitdirectory

The Boolean `webkitdirectory` attribute, if present, indicates that only directories should be available to be selected by the user in the file picker interface. See `HTMLInputElement.webkitdirectory` for additional details and examples.

> **Note:** `webkitdirectory` is defined in the File and Directory Entries API. It's named `webkitdirectory` because of its origins as a Chrome-specific API. It is now available in all browsers.

## Unique file type specifiers

A **unique file type specifier** is a string that describes a type of file that may be selected by the user in an `<input>` element of type `file`. Each unique file type specifier may take one of the following forms:

- A valid case-insensitive filename extension, starting with a period (".") character. For example: `.jpg`, `.pdf`, or `.doc`.
- A valid MIME type string, with no extensions.
- The string `audio/*` meaning "any audio file".
- The string `video/*` meaning "any video file".
- The string `image/*` meaning "any image file".

The `accept` attribute takes a string containing one or more of these unique file type specifiers as its value, separated by commas. For example, a file picker that needs content that can be presented as an image, including both standard image formats and PDF files, might look like this:

```
<input type="file" accept="image/*,.pdf" />
```

## Using file inputs

### A basic example

```
<div>
  <label for="file">Choose file to upload</label>
  <input type="file" id="file" name="file" multiple />
</div>
```

This produces the following output:

Regardless of the user's device or operating system, the file input provides a button that opens up a file picker dialog that allows the user to choose a file.

Including the `multiple` attribute, as shown above, specifies that multiple files can be chosen at once. The user can choose multiple files from the file picker in any way that their chosen platform allows (e.g., by holding down `Shift` or `Control` and then clicking). If you only want the user to choose a single file per `<input>`, omit the `multiple` attribute.

### Getting information on selected files

The selected files are returned by the element's `HTMLInputElement.files` property, which is a `FileList` object containing a list of `File` objects. The `FileList` behaves like an array, so you can check its `length` property to get the number of selected files.

Each `File` object contains the following information:

`name`

: The file's name.

`lastModified`

: A number specifying the date and time at which the file was last modified, in milliseconds since the UNIX epoch (January 1, 1970, at midnight).

`lastModifiedDate`

A `Date` object representing the date and time at which the file was last modified. *This is deprecated and should not be used. Use `lastModified` instead.*

`size`

The size of the file in bytes.

`type`

The file's MIME type.

`webkitRelativePath`

A string specifying the file's path relative to the base directory selected in a directory picker (that is, a `file` picker in which the `webkitdirectory` attribute is set). *This is non-standard and should be used with caution.*

### Limiting accepted file types

Often you won't want the user to be able to pick any arbitrary type of file; instead, you often want them to select files of a specific type or types. For example, if your file input lets users upload a profile picture, you probably want them to select web-compatible image formats, such as JPEG or PNG.

Acceptable file types can be specified with the `accept` attribute, which takes a comma-separated list of allowed file extensions or MIME types. Some examples:

- `accept="image/png"` or `accept=".png"` — Accepts PNG files.
- `accept="image/png, image/jpeg"` or `accept=".png, .jpg, .jpeg"` — Accept PNG or JPEG files.
- `accept="image/*"` — Accept any file with an `image/*` MIME type. (Many mobile devices also let the user take a picture with the camera when this is used.)
- `accept=".doc,.docx,.xml,application/msword,application/vnd.openxmlformats-officedocument.wordprocessingml.document"` — accept anything that smells like an MS Word document.

Let's look at a more complete example:

```
<div>
  <label for="profile_pic">Choose file to upload</label>
  <input
    type="file"
    id="profile_pic"
    name="profile_pic"
    accept=".jpg, .jpeg, .png" />
</div>
```

This produces a similar-looking output to the previous example:

It may look similar, but if you try selecting a file with this input, you'll see that the file picker only lets you select the file types specified in the `accept` value (the exact interface differs across browsers and operating systems).

The `accept` attribute doesn't validate the types of the selected files; it provides hints for browsers to guide users towards selecting the correct file types. It is still possible (in most cases) for users to toggle an option in the file chooser that makes it possible to override this and select any file they wish, and then choose incorrect file types.

Because of this, you should make sure that the `accept` attribute is backed up by appropriate server-side validation.

### Detecting cancellations

The `cancel` event is fired when the user does not change their selection, reselecting the previously selected files. The `cancel` event is also fired when the file picker dialog gets closed, or canceled, via the "cancel" button or the `escape` key.

For example, the following code will log to the console if the user closes the popup without selecting a file:

```
const elem = document.createElement("input");
elem.type = "file";
elem.addEventListener("cancel", () => {
  console.log("Canceled.");
});
elem.addEventListener("change", () => {
  if (elem.files.length === 1) {
    console.log("File selected: ", elem.files[0]);
  }
});
elem.click();
```

### Notes

1. You cannot set the value of a file picker from a script — doing something like the following has no effect:

   ```
   const input = document.querySelector("input[type=file]");
   input.value = "foo";
   ```

2. When a file is chosen using an `<input type="file">`, the real path to the source file is not shown in the input's `value` attribute for obvious security reasons. Instead, the filename is shown, with `C:\fakepath\` prepended to it. There are some historical reasons for this quirk, but it is supported across all modern browsers, and in fact is defined in the spec.

### Examples

#### Complete file example

In this example, we'll present a slightly more advanced file chooser that takes advantage of the file information available in the `HTMLInputElement.files` property, as well as showing off a few clever tricks.

#### HTML

The HTML looks like so:

```
<div class="file-picker">
  <div>
    <label for="image_uploads">Choose images to upload (PNG, JPG)</label>
    <input
      type="file"
      id="image_uploads"
      name="image_uploads"
      accept=".jpg, .jpeg, .png"
      multiple />
  </div>
  <div class="preview">
    <p>No files currently selected for upload</p>
  </div>
  <div>
    <button type="button">Submit</button>
  </div>
</div>
```

This is similar to what we've seen earlier, so we won't spend time describing it. Also, we've hidden the CSS applied to the example, as it isn't relevant to understanding the file input or the JavaScript that powers it.

#### JavaScript

In the first lines of script, we get references to the form input itself, and the `<div>` element with the class of `.preview`. Next, we hide the `<input>` element — we do this because file inputs tend to be ugly, difficult to style, and inconsistent in their design across browsers. You can activate the `input` element by clicking its `<label>`, so it is better to visually hide the `input` and style the label like a button, so the user will know to interact with it if they want to upload files.

```
const input = document.querySelector("input");
const preview = document.querySelector(".preview");

input.style.opacity = 0;
```

> **Note:** `opacity` is used to hide the file input instead of `visibility: hidden` or `display: none`, because assistive technology interprets the latter two styles to mean the file input isn't interactive.

Next, we add an event listener to the input to listen for changes to its selected value (in this case, when files are selected). The event listener invokes our custom `updateImageDisplay()` function.

```
input.addEventListener("change", updateImageDisplay);
```

Whenever the `updateImageDisplay()` function is invoked, we:

- Use a `while` loop to empty the previous contents of the preview `<div>`.
- Grab the `FileList` object that contains the information on all the selected files, and store it in a variable called `curFiles`.
- Check to see if no files were selected, by checking if `curFiles.length` is equal to 0. If so, print a message into the preview `<div>` stating that no files have been selected.
- If files *have* been selected, we loop through each one, printing information about it into the preview `<div>`. Things to note here:

- We use the custom `validFileType()` function to check whether the file is of the correct type (e.g., the image types specified in the `accept` attribute).
- If it is, we:
  - Print out its name and file size into a list item inside the previous `<div>` (obtained from `file.name` and `file.size`). The custom `returnFileSize()` function returns a nicely-formatted version of the size in bytes/KB/MB (by default the browser reports the size in absolute bytes).
  - Generate a thumbnail preview of the image by calling `URL.createObjectURL(file)`. Then, insert the image into the list item too by creating a new `<img>` and setting its `src` to the thumbnail.
- If the file type is invalid, we display a message inside a list item telling the user that they need to select a different file type.

```
function updateImageDisplay() {
  while (preview.firstChild) {
    preview.removeChild(preview.firstChild);
  }

  const curFiles = input.files;
  if (curFiles.length === 0) {
    const para = document.createElement("p");
    para.textContent = "No files currently selected for upload";
    preview.appendChild(para);
  } else {
    const list = document.createElement("ol");
    preview.appendChild(list);

    for (const file of curFiles) {
      const listItem = document.createElement("li");
      const para = document.createElement("p");
      if (validFileType(file)) {
        para.textContent = `File name ${file.name}, file size ${returnFileSize(
          file.size,
        )}.`;
        const image = document.createElement("img");
        image.src = URL.createObjectURL(file);
        image.alt = image.title = file.name;

        listItem.appendChild(image);
        listItem.appendChild(para);
      } else {
        para.textContent = `File name ${file.name}: Not a valid file type. Update your selection.`;
        listItem.appendChild(para);
      }

      list.appendChild(listItem);
    }
  }
}
```

The custom `validFileType()` function takes a `File` object as a parameter, then uses `Array.prototype.includes()` to check if any value in the `fileTypes` matches the file's `type` property. If a match is found, the function returns `true`. If no match is found, it returns `false`.

```
// https://developer.mozilla.org/en-US/docs/Web/Media/Guides/Formats/Image_types
const fileTypes = [
  "image/apng",
  "image/bmp",
  "image/gif",
  "image/jpeg",
  "image/pjpeg",
  "image/png",
  "image/svg+xml",
  "image/tiff",
  "image/webp",
  "image/x-icon",
];

function validFileType(file) {
  return fileTypes.includes(file.type);
}
```

The `returnFileSize()` function takes a number (of bytes, taken from the current file's `size` property), and turns it into a nicely formatted size in bytes/KB/MB.

```
function returnFileSize(number) {
  if (number < 1e3) {
    return `${number} bytes`;
  } else if (number >= 1e3 && number < 1e6) {
    return `${(number / 1e3).toFixed(1)} KB`;
  }
  return `${(number / 1e6).toFixed(1)} MB`;
}
```

> **Note:** The "KB" and "MB" units here use the SI prefix convention of 1KB = 1000B, similar to macOS. Different systems represent file sizes differently—for example, Ubuntu uses IEC prefixes where 1KiB = 1024B, while RAM specifications often use SI prefixes to represent powers of two (1KB = 1024B). For this reason, we used `1e3` (`1000`) and `1e6` (`100000`) instead of `1024` and `1048576`. In your application, you should communicate the unit system clearly to your users if the exact size is important.

### Result

### Technical summary

| | |
|---|---|
| **Value** | A string representing the path to the selected file. |
| **Events** | `change`, `input` and `cancel` |
| **Supported common attributes** | `required` |
| **Additional Attributes** | `accept`, `capture`, `multiple` |
| **IDL attributes** | `files` and `value` |
| **DOM interface** | `HTMLInputElement` |
| **Implicit ARIA Role** | no corresponding role |

### See also

- Using files from web applications — contains a number of other useful examples related to `<input type="file">` and the File API.

## <input type="hidden"> HTML attribute value

`<input>` elements of type **`hidden`** allow web developers to include data in a form that is not visually accessible to users when a form is submitted. For example, the ID of the content that is currently being ordered or edited, or a unique security token. Hidden inputs are completely invisible in the rendered page, but are still submitted as any other form input. While the data is not presented to users in the rendered content, it is still accessible in the HTML output and is editable to users using tools such as their browser's developer tools.

> **Note:** The `input` and `change` events do not apply to this input type. Hidden inputs cannot be focused even using JavaScript (e.g., `hiddenInput.focus()`).

### Value

The `<input>` element's `value` attribute holds a string that contains the hidden data you want to include when the form is submitted to the server. This value is not presented to the user via the user interface.

> **Warning:** While the value isn't displayed to the user in the page's content, it is visible—and can be edited—using any browser's developer tools or "View Source" functionality. Do not rely on `hidden` inputs as a form of security.

### Additional attributes

In addition to the attributes common to all `<input>` elements, `hidden` inputs offer the following attributes.

#### name

This is actually one of the common attributes, but it has a special meaning available for hidden inputs. Normally, the `name` attribute functions on hidden inputs just like on any other input. However, when the form is submitted, a hidden input whose `name` is set to `_charset_` will automatically be reported with the value set to the character encoding used to submit the form.

### Using hidden inputs

As mentioned above, hidden inputs can be used anywhere that you want to include data the user can't see along with the form when it's submitted to the server. Let's look at some examples that illustrate its use.

#### Tracking edited content

One of the most common uses for hidden inputs is to keep track of what database record needs to be updated when an edit form is submitted. A typical workflow looks like this:

1. User decides to edit some content they have control over, such as a blog post, or a product entry. They get started by pressing the edit button.
2. The content to be edited is taken from the database and loaded into an HTML form to allow the user to make changes.
3. After editing, the user submits the form, and the updated data is sent back to the server to be updated in the database.

The idea here is that during step 2, the ID of the record being updated is kept in a hidden input. When the form is submitted in step 3, the ID is automatically sent back to the server with the record content. The ID lets the site's server-side component know exactly which record needs to be updated with the submitted data.

You can see a full example of what this might look like in the Examples section below.

### Improving website security

One common use of hidden input fields is to store Cross-Site Request Forgery (CSRF) tokens, which help protect websites against CSRF attacks.

> **Note:** Data placed in a hidden input is not inherently secured. Its value is still observable by the end user. You need to use well-designed secrets to actually secure your website.

### Validation

Hidden inputs don't participate in constraint validation; they have no real value to be constrained.

### Examples

### Using a hidden record ID

Let's look at how we might implement a version of the edit form we described earlier (see Tracking edited content), using a hidden input to remember the ID of the record being edited.

### HTML

The edit form's HTML looks like this:

```
<form>
  <div>
    <label for="title">Post title:</label>
    <input type="text" id="title" name="title" value="My excellent blog post" />
  </div>
  <div>
    <label for="content">Post content:</label>
    <textarea id="content" name="content" cols="60" rows="5">
This is the content of my excellent blog post. I hope you enjoy it!
    </textarea>
  </div>
  <div>
    <button type="submit">Update post</button>
  </div>
  <input type="hidden" id="postId" name="postId" value="34657" />
</form>
```

The server would set the value of the hidden input with the ID `postID` to the ID of the post in its database before sending the form to the user's browser and would use that information when the form is returned to know which database record to update with modified information. No scripting is needed in the content to handle this.

### Result

When submitted, the form data sent to the server would look something like this:

```
title=My+excellent+blog+post&content=This+is+the+content+of+my+excellent+blog+post.+I+hope+you+enjoy+it!&postId=34657
```

Even though the hidden input is not visible in the form, its data is still submitted.

### Technical summary

| | |
|---|---|
| **Value** | A string representing the value of the hidden data you want to pass back to the server. |
| **Events** | None. |
| **Supported Common Attributes** | `autocomplete` |
| **IDL attributes** | `value` |

| | |
|---|---|
| **DOM interface** | `HTMLInputElement` |
| **Implicit ARIA Role** | no corresponding role |

### See also

- HTML forms guide
- `<input>` and the `HTMLInputElement` interface it's based upon

## `<input type="image">` HTML attribute value

`<input>` elements of type **`image`** are used to create graphical submit buttons, i.e., submit buttons that take the form of an image rather than text.

### Try it

```
<p>Sign in to your account:</p>

<div>
  <label for="userId">User ID</label>
  <input type="text" id="userId" name="userId" />
</div>

<input
  type="image"
  id="image"
  alt="Login"
  src="https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/input/image//shared-assets/images/examples/login-button.png"
/>
```

```
label {
  font-size: 0.8rem;
}

label,
input[type="image"] {
  margin-top: 1rem;
}

input[type="image"] {
  width: 80px;
}
```

### Value

`<input type="image">` elements do not accept `value` attributes. The path to the image to be displayed is specified in the `src` attribute.

### Additional attributes

In addition to the attributes shared by all `<input>` elements, `image` button inputs support the following attributes.

#### alt

The `alt` attribute provides an alternate string to use as the button's label if the image cannot be shown (due to error, a user agent that cannot or is configured not to show images, or if the user is using a screen reading device). If provided, it must be a non-empty string appropriate as a label for the button.

For example, if you have a graphical button that shows an image with an icon and/or image text "Login Now", you should also set the `alt` attribute to something like `Login Now`.

> **Note:** While the `alt` attribute is technically optional, you should always include one to maximize the usability of your content.

Functionally, the `alt` attribute of the `<input type="image">` element works just like the `alt` attribute on `<img>` elements.

#### formaction

A string indicating the URL to which to submit the data. This takes precedence over the `action` attribute on the `<form>` element that owns the `<input>`.

This attribute is also available on `<input type="submit">` and `<button>` elements.

#### formenctype

A string that identifies the encoding method to use when submitting the form data to the server. There are three permitted values:

`application/x-www-form-urlencoded`

This, the default value, sends the form data as a string after percent-encoding the text using an algorithm such as `encodeURI()`.

`multipart/form-data`

Uses the `FormData` API to manage the data, allowing for files to be submitted to the server. You *must* use this encoding type if your form includes any `<input>` elements of `type` `file` (`<input type="file">`).

`text/plain`

Plain text; mostly useful only for debugging, so you can easily see the data that's to be submitted.

If specified, the value of the `formenctype` attribute overrides the owning form's `enctype` attribute.

This attribute is also available on `<input type="submit">` and `<button>` elements.

### formmethod

A string indicating the HTTP method to use when submitting the form's data; this value overrides any `method` attribute given on the owning form. Permitted values are:

`get`

A URL is constructed by starting with the URL given by the `formaction` or `action` attribute, appending a question mark ("?") character, then appending the form's data, encoded as described by `formenctype` or the form's `enctype` attribute. This URL is then sent to the server using an HTTP `get` request. This method works well for forms that contain only ASCII characters and have no side effects. This is the default value.

`post`

The form's data is included in the body of the request that is sent to the URL given by the `formaction` or `action` attribute using an HTTP `post` request. This method supports complex data and file attachments.

`dialog`

This method is used to indicate that the button closes the dialog with which the input is associated, and does not transmit the form data at all.

This attribute is also available on `<input type="submit">` and `<button>` elements.

### formnovalidate

A Boolean attribute which, if present, specifies that the form should not be validated before submission to the server. This overrides the value of the `novalidate` attribute on the element's owning form.

This attribute is also available on `<input type="submit">` and `<button>` elements.

### formtarget

A string which specifies a name or keyword that indicates where to display the response received after submitting the form. The string must be the name of a **browsing context** (that is, a tab, window, or `<iframe>`. A value specified here overrides any target given by the `target` attribute on the `<form>` that owns this input.

In addition to the actual names of tabs, windows, or inline frames, there are a few special keywords that can be used:

`_self`

Loads the response into the same browsing context as the one that contains the form. This will replace the current document with the received data. This is the default value used if none is specified.

`_blank`

Loads the response into a new, unnamed, browsing context. This is typically a new tab in the same window as the current document, but may differ depending on the configuration of the user agent.

`_parent`

Loads the response into the parent browsing context of the current one. If there is no parent context, this behaves the same as `_self`.

`_top`

Loads the response into the top-level browsing context; this is the browsing context that is the topmost ancestor of the current context. If the current context is the topmost context, this behaves the same as `_self`.

This attribute is also available on `<input type="submit">` and `<button>` elements.

### height

A number specifying the height, in CSS pixels, at which to draw the image specified by the `src` attribute.

#### src

A string specifying the URL of the image file to display to represent the graphical submit button. When the user interacts with the image, the input is handled like any other button input.

#### width

A number indicating the width at which to draw the image, in CSS pixels.

### Obsolete attributes

The following attribute was defined by HTML 4 for `image` inputs, but was not implemented by all browsers and has since been deprecated.

#### usemap

If `usemap` is specified, it must be the name of an image map element, `<map>`, that defines an image map to use with the image. This usage is obsolete; you should switch to using the `<img>` element when you want to use image maps.

## Using image inputs

The `<input type="image">` element is a replaced element (an element whose content isn't generated or directly managed by the CSS layer), behaving in much the same way as a regular `<img>` element, but with the capabilities of a submit button.

### Essential image input features

Let's look at a basic example that includes all the essential features you'd need to use (These work exactly the same as they do on the `<img>` element.):

```
<input
  id="image"
  type="image"
  width="100"
  height="30"
  alt="Login"
  src="https://mdn.github.io/shared-assets/images/examples/login-button.png" />
```

- The `src` attribute is used to specify the path to the image you want to display in the button.
- The `alt` attribute provides alt text for the image, so screen reader users can get a better idea of what the button is used for. It will also display if the image can't be shown for any reason (for example if the path is misspelled). If possible, use text which matches the label you'd use if you were using a standard submit button.
- The `width` and `height` attributes are used to specify the width and height the image should be shown at, in pixels. The button is the same size as the image; if you need the button's hit area to be bigger than the image, you will need to use CSS (e.g., `padding`). Also, if you specify only one dimension, the other is automatically adjusted so that the image maintains its original aspect ratio.

### Overriding default form behaviors

`<input type="image">` elements — like regular submit buttons — can accept a number of attributes that override the default form behavior:

`formaction`

> The URI of a program that processes the information submitted by the input element; overrides the `action` attribute of the element's form owner.

`formenctype`

> Specifies the type of content that is used to submit the form to the server. Possible values are:
>
> - `application/x-www-form-urlencoded`: The default value if the attribute is not specified.
> - `text/plain`.
>
> If this attribute is specified, it overrides the `enctype` attribute of the element's form owner.

`formmethod`

> Specifies the HTTP method that the browser uses to submit the form. Possible values are:
>
> - `post`: The data from the form is included in the body of the form and is sent to the server.
> - `get`: The data from the form is appended to the **`form`** attribute URI, with a '?' as a separator, and the resulting URI is sent to the server. Use this method when the form has no side effects and contains only ASCII characters.
>
> If specified, this attribute overrides the `method` attribute of the element's form owner.

`formnovalidate`

A Boolean attribute specifying that the form is not to be validated when it is submitted. If this attribute is specified, it overrides the `novalidate` attribute of the element's form owner.

`formtarget`

A name or keyword indicating where to display the response that is received after submitting the form. This is a name of, or keyword for, a *browsing context* (for example, tab, window, or inline frame). If this attribute is specified, it overrides the `target` attribute of the element's form owner. The following keywords have special meanings:

- `_self`: Load the response into the same browsing context as the current one. This value is the default if the attribute is not specified.
- `_blank`: Load the response into a new unnamed browsing context.
- `_parent`: Load the response into the parent browsing context of the current one. If there is no parent, this option behaves the same way as `_self`.
- `_top`: Load the response into the top-level browsing context (that is, the browsing context that is an ancestor of the current one, and has no parent). If there is no parent, this option behaves the same as `_self`.

### Using the x and y data points

When you submit a form using a button created with `<input type="image">`, two extra data points are submitted to the server automatically by the browser — `x` and `y`. You can see this in action in our X Y coordinates example.

When you click on the image to submit the form, you'll see the data appended to the URL as parameters, such as `?x=52&y=55`. If the image input has a `name` attribute, then the specified name is prefixed to each coordinate entry's name with a dot separator. For example, if the `name` were `position`, then the returned coordinates would be formatted in the URL as `?position.x=52&position.y=55`.

These are the X and Y coordinates of the image that the mouse clicked on to submit the form, where (0,0) is the top-left of the image and the default in case submission happens without a click on the image. These can be used when the position the image was clicked on is significant, for example you might have a map that when clicked, sends the coordinates that were clicked to the server. The server-side code then works out what location was clicked on, and returns information about places nearby.

In our above example, we could write server-side code that works out what color was clicked on by the coordinates submitted, and keeps a tally of the favorite colors people voted for.

### Adjusting the image's position and scaling algorithm

You can use the `object-position` property to adjust the positioning of the image within the `<input>` element's frame, and the `object-fit` property to control how the image's size is adjusted to fit within the frame. This allows you to specify a frame for the image using the `width` and `height` attributes to set aside space in the layout, then adjust where within that space the image is located and how (or if) it is scaled to occupy that space.

## Examples

### A login form

The following example shows the same button as before, but included in the context of a typical login form.

#### HTML

```
<form>
  <p>Login to your account</p>
  <div>
    <label for="userId">User ID</label>
    <input type="text" id="userId" name="userId" />
  </div>
  <div>
    <label for="pwd">Password</label>
    <input type="password" id="pwd" name="pwd" />
  </div>
  <div>
    <input
      id="image"
      type="image"
      src="https://mdn.github.io/shared-assets/images/examples/login-button.png"
      alt="Login"
      width="100" />
  </div>
</form>
```

#### CSS

And now some CSS to make the basic elements sit more neatly:

```
div {
  margin-bottom: 10px;
}

label {
  display: inline-block;
  width: 70px;
  text-align: right;
  padding-right: 10px;
}
```

#### Result

### Adjusting the image position and scaling

In this example, we adapt the previous example to set aside more space for the image and then adjust the actual image's size and positioning using `object-fit` and `object-position`.

#### HTML

```
<form>
  <p>Login to your account</p>
  <div>
    <label for="userId">User ID</label>
    <input type="text" id="userId" name="userId" />
  </div>
  <div>
    <label for="pwd">Password</label>
    <input type="password" id="pwd" name="pwd" />
  </div>
  <div>
    <input
      id="image"
      type="image"
      src="https://mdn.github.io/shared-assets/images/examples/login-button.png"
      alt="Login"
      width="100" />
  </div>
</form>
```

#### CSS

```
div {
  margin-bottom: 10px;
}

label {
  display: inline-block;
  width: 70px;
  text-align: right;
  padding-right: 10px;
}

#image {
  object-position: right top;
  object-fit: contain;
  background-color: #dddddd;
}
```

#### Result

Here, `object-position` is configured to draw the image in the top-right corner of the element, while `object-fit` is set to `contain`, which indicates that the image should be drawn at the largest size that will fit within the element's box without altering its aspect ratio. Note the visible gray background of the element still visible in the area not covered by the image.

### Technical summary

| | |
|---|---|
| **Value** | None — the `value` attribute should not be specified. |
| **Events** | None. |
| **Supported common attributes** | `alt`, `src`, `width`, `height`, `formaction`, `formenctype`, `formmethod`, `formnovalidate`, `formtarget` |
| **IDL attributes** | None. |
| **DOM interface** | `HTMLInputElement` |

| Implicit ARIA Role | `button` |
| --- | --- |

### See also

- `<input>` and the `HTMLInputElement` interface which implements it.
- The HTML `<img>` element
- Positioning and sizing the image within the `<input>` element's frame: `object-position` and `object-fit`

## `<input type="month">` HTML attribute value

`<input>` elements of type **`month`** create input fields that let the user enter a month and year allowing a month and year to be easily entered. The value is a string whose value is in the format `YYYY-MM`, where `YYYY` is the four-digit year and `MM` is the month number.

### Try it

```
<label for="start">Start month:</label>

<input type="month" id="start" name="start" min="2018-03" value="2018-05" />
```

```
label {
  display: block;
  font:
    1rem "Fira Sans",
    sans-serif;
}

input,
label {
  margin: 0.4rem 0;
}
```

### Value

A string representing the value of the month and year entered into the input, in the form YYYY-MM (four or more digit year, then a hyphen (`-`), followed by the two-digit month). The format of the month string used by this input type is described in Month strings.

### Setting a default value

You can set a default value for the input control by including a month and year inside the `value` attribute, like so:

```
<label for="bday-month">What month were you born in?</label>
<input id="bday-month" type="month" name="bday-month" value="2001-06" />
```

One thing to note is that the displayed date format differs from the actual `value`; most user agents display the month and year in a locale-appropriate form, based on the set locale of the user's operating system, whereas the date `value` is always formatted `yyyy-MM`.

When the above value is submitted to the server, for example, it will look like `bday-month=1978-06`.

### Setting the value using JavaScript

You can also get and set the date value in JavaScript using the `HTMLInputElement.value` property, for example:

```
<label for="bday-month">What month were you born in?</label>
<input id="bday-month" type="month" name="bday-month" />
```

```
const monthControl = document.querySelector('input[type="month"]');
monthControl.value = "2001-06";
```

### Additional attributes

In addition to the attributes common to `<input>` elements, month inputs offer the following attributes.

#### list

The values of the list attribute is the `id` of a `<datalist>` element located in the same document. The `<datalist>` provides a list of predefined values to suggest to the user for this input. Any values in the list that are not compatible with the `type` are not included in the suggested options. The values provided are suggestions, not requirements: users can select from this predefined list or provide a different value.

#### max

The latest year and month, in the string format discussed in the Value section above, to accept. If the `value` entered into the element exceeds this, the element fails constraint validation. If the value of the `max` attribute isn't a valid string in `yyyy-MM` format, then the element has no maximum value.

This value must specify a year-month pairing later than or equal to the one specified by the `min` attribute.

### min

The earliest year and month to accept, in the same `yyyy-MM` format described above. If the `value` of the element is less than this, the element fails constraint validation. If a value is specified for `min` that isn't a valid year and month string, the input has no minimum value.

This value must be a year-month pairing which is earlier than or equal to the one specified by the `max` attribute.

### readonly

A Boolean attribute which, if present, means this field cannot be edited by the user. Its `value` can, however, still be changed from JavaScript code that directly sets the value of the `HTMLInputElement.value` property.

> **Note:** Because a read-only field cannot have a value, `required` does not have any effect on inputs with the `readonly` attribute also specified.

### step

The `step` attribute is a number that specifies the granularity that the value must adhere to, or the special value `any`, which is described below. Only values which are a whole number of steps from the step base are valid. The step base is `min` if specified, `value` otherwise, or `0` (the Unix epoch, `1970-01`) if neither is provided.

For `month` inputs, the value of `step` is given in months. The default value of `step` is 1, indicating 1 month.

A string value of `any` means that no stepping is implied, and any value is allowed (barring other constraints, such as `min` and `max`). In reality, it has the same effect as `1` for `month` inputs because the picker UI only allows selecting whole months.

> **Note:** When the data entered by the user doesn't adhere to the stepping configuration, the user agent may round to the nearest valid value, preferring numbers in the positive direction when there are two equally close options.

## Using month inputs

Date-related inputs (including `month`) sound convenient at first glance; they promise an easy UI for choosing dates, and they normalize the data format sent to the server, regardless of the user's locale. However, there are issues with `<input type="month">` because at this time, many major browsers don't yet support it.

We'll look at basic and more complex uses of `<input type="month">`, then offer advice on mitigating the browser support issue in the section Handling browser support.

### Basic uses of month

The most basic use of `<input type="month">` involves a basic `<input>` and `<label>` element combination, as seen below:

```
<label for="bday-month">What month were you born in?</label>
<input id="bday-month" type="month" name="bday-month" />
```

### Setting maximum and minimum dates

You can use the `min` and `max` attributes to restrict the range of dates that the user can choose. In the following example we specify a minimum month of `1900-01` and a maximum month of `2013-12`:

```
<label for="bday-month">What month were you born in?</label>
<input
  id="bday-month"
  type="month"
  name="bday-month"
  min="1900-01"
  max="2013-12" />
```

The result here is that:

- Only months between in January 1900 and December 2013 can be selected; months outside that range can't be scrolled to in the control.
- Depending on what browser you are using, you might find that months outside the specified range might not be selectable in the month picker (e.g., Edge), or invalid (see Validation) but still available (e.g., Chrome).

### Controlling input size

`<input type="month">` doesn't support form sizing attributes such as `size`. You'll have to resort to CSS for sizing needs.

### Validation

By default, `<input type="month">` does not apply any validation to entered values. The UI implementations generally don't let you enter anything that isn't a date — which is helpful — but you can still submit the form with the `month` input empty, or enter an invalid date (e.g., the 32nd of April).

To help avoid this, you can use `min` and `max` to restrict the available dates (see Setting maximum and minimum dates), and in addition use the `required` attribute to make filling in the date mandatory. As a result, supporting browsers will display an error if you try to submit a date that is outside the set bounds, or an empty date field.

Let's look at an example; here we've set minimum and maximum dates, and also made the field required:

```
<form>
  <div>
    <label for="month">
      What month would you like to visit (June to Sept.)?
    </label>
    <input
      id="month"
      type="month"
      name="month"
      min="2022-06"
      max="2022-09"
      required />
    <span class="validity"></span>
  </div>
  <div>
    <input type="submit" value="Submit form" />
  </div>
</form>
```

If you try to submit the form without both the month and year specified (or with a date outside the set bounds), the browser displays an error. Try playing with the example now:

Here's the CSS used in the above example. Here we make use of the `:valid` and `:invalid` CSS properties to style the input based on whether the current value is valid. We had to put the icons on a `<span>` next to the input, not on the input itself, because in Chrome the generated content is placed inside the form control, and can't be styled or shown effectively.

```
div {
  margin-bottom: 10px;
  position: relative;
}

input[type="number"] {
  width: 100px;
}

input + span {
  padding-right: 30px;
}

input:invalid + span::after {
  position: absolute;
  content: "✖";
  padding-left: 5px;
}

input:valid + span::after {
  position: absolute;
  content: "✓";
  padding-left: 5px;
}
```

> **Warning:** HTML form validation is *not* a substitute for scripts that ensure that the entered data is in the proper format. It's far too easy for someone to make adjustments to the HTML that allow them to bypass the validation, or to remove it entirely. It's also possible for someone to bypass your HTML entirely and submit the data directly to your server. If your server-side code fails to validate the data it receives, disaster could strike when improperly-formatted data is submitted (or data which is too large, of the wrong type, and so forth).

### Handling browser support

The control's UI varies in general from browser to browser; at the moment support is patchy, with only Chromium browsers on desktop and mobile browsers having usable implementations. The control is usually presented as either a calendar-like grid or two wheel pickers.

In browsers that don't support `month` inputs, the control degrades gracefully to `<input type="text">`, although there may be automatic validation of the entered text to ensure it's formatted as expected. But this creates problems both in terms of consistency of user interface (the presented control

will be different), and data handling.

The second problem is the more serious of the two. As mentioned earlier, with a `month` input the actual value is always normalized to the format `yyyy-mm`. On the other hand, in its default configuration, a `text` input has no idea what format the date should be in, and this is an issue because of the number of different ways in which people write dates. For example:

- `mmyyyy` (072022)
- `mm/yyyy` (07/2022)
- `mm-yyyy` (07-2022)
- `yyyy-mm` (2022-07)
- `Month yyyy` (July 2022)
- and so forth…

One way around this is to put a `pattern` attribute on your `month` input. Even though the `month` input doesn't use it, if the browser falls back to treating it like a `text` input, the pattern will be used. For example, try viewing the following demo in a browser that doesn't support `month` inputs:

```
<form>
  <div>
    <label for="month">
      What month would you like to visit (June to Sept.)?
    </label>
    <input
      id="month"
      type="month"
      name="month"
      min="2022-06"
      max="2022-09"
      required
      pattern="[0-9]{4}-[0-9]{2}" />
    <span class="validity"></span>
  </div>
  <div>
    <input type="submit" value="Submit form" />
  </div>
</form>
```

If you try submitting it, you'll see that the browser now displays an error message (and highlights the input as invalid) if your entry doesn't match the pattern `nnnn-nn`, where `n` is a number from 0 to 9. Of course, this doesn't stop people from entering invalid dates (such as `0000-42`), or incorrectly formatted dates that follow the pattern.

There's also the problem that the user won't necessarily know which of the many date formats is expected. We have work left to do.

The best way to deal with dates in forms in a cross-browser way (until all major browsers have supported them for a while) is to get the user to enter the month and year in separate controls (`<select>` elements being popular; see below for an implementation), or use JavaScript libraries such as the jQuery date picker plugin.

### Examples

In this example, we create two sets of UI elements, each designed to let the user select a month and year. The first is a native `month` input, and the other is a pair of `<select>` elements that allow choosing a month and year independently, for compatibility with browsers that don't yet support `<input type="month">`.

### HTML

The HTML that requests the month and year looks like this:

```
<div class="nativeDatePicker">
  <label for="month-visit">What month would you like to visit us?</label>
  <input type="month" id="month-visit" name="month-visit" />
  <span class="validity"></span>
</div>
<p class="fallbackLabel">What month would you like to visit us?</p>
<div class="fallbackDatePicker">
  <div>
    <span>
      <label for="month">Month:</label>
      <select id="month" name="month">
        <option selected>January</option>
        <option>February</option>
        <option>March</option>
        <option>April</option>
        <option>May</option>
        <option>June</option>
        <option>July</option>
```

```
        <option>August</option>
        <option>September</option>
        <option>October</option>
        <option>November</option>
        <option>December</option>
      </select>
    </span>
    <span>
      <label for="year">Year:</label>
      <select id="year" name="year"></select>
    </span>
  </div>
</div>
```

The `<div>` with the ID `nativeDatePicker` uses the `month` input type to request the month and year, while the `<div>` with the ID `fallbackDatePicker` instead uses a pair of `<select>` elements. The first requests the month, and the second the year.

The `<select>` for choosing the month is hardcoded with the names of the months, as they don't change (leaving localization out of things). The list of available year values is dynamically generated depending on the current year (see the code comments below for detailed explanations of how these functions work).

### JavaScript

The JavaScript code that handles selecting which approach to use and to set up the list of years to include in the non-native year `<select>` follows.

The part of the example that may be of most interest is the feature detection code. In order to detect whether the browser supports `<input type="month">`, we create a new `<input>` element, try setting its `type` to `month`, then immediately check what its type is set to. Browsers that don't support type `month` will return `text`, since that's What month falls back to when not supported. If `<input type="month">` is not supported, we hide the native picker and show the fallback picker UI instead.

```
// Get UI elements
const nativePicker = document.querySelector(".nativeDatePicker");
const fallbackPicker = document.querySelector(".fallbackDatePicker");
const fallbackLabel = document.querySelector(".fallbackLabel");

const yearSelect = document.querySelector("#year");
const monthSelect = document.querySelector("#month");

// Hide fallback initially
fallbackPicker.style.display = "none";
fallbackLabel.style.display = "none";

// Test whether a new date input falls back to a text input or not
const test = document.createElement("input");

try {
  test.type = "month";
} catch (e) {
  console.log(e.description);
}

// If it does, run the code inside the if () {} block
if (test.type === "text") {
  // Hide the native picker and show the fallback
  nativePicker.style.display = "none";
  fallbackPicker.style.display = "block";
  fallbackLabel.style.display = "block";

  // Populate the years dynamically
  // (the months are always the same, therefore hardcoded)
  populateYears();
}

function populateYears() {
  // Get the current year as a number
  const date = new Date();
  const year = date.getFullYear();

  // Make this year, and the 100 years before it available in the year <select>
  for (let i = 0; i <= 100; i++) {
    const option = document.createElement("option");
    option.textContent = year - i;
    yearSelect.appendChild(option);
  }
}
```

> **Note:** Remember that some years have 53 weeks in them (see Weeks per year)! You'll need to take this into consideration when developing production apps.

### Technical summary

| | |
|---|---|
| **Value** | A string representing a month and year, or empty. |
| **Events** | `change` and `input` |
| **Supported common attributes** | `autocomplete`, `list`, `readonly`, `step` |
| **IDL attributes** | `list`, `value`, `valueAsDate`, `valueAsNumber` |
| **DOM interface** | `HTMLInputElement` |
| **Implicit ARIA Role** | no corresponding role |

### See also

- The generic `<input>` element and the interface used to manipulate it, `HTMLInputElement`
- Date and time formats used in HTML
- Date and Time picker tutorial
- `<input type="datetime-local">`, `<input type="date">`, `<input type="time">`, and `<input type="week">`

## <input type="number"> HTML attribute value

`<input>` elements of type **`number`** are used to let the user enter a number. They include built-in validation to reject non-numerical entries.

The browser may opt to provide stepper arrows to let the user increase and decrease the value using their mouse or by tapping with a fingertip.

On browsers that don't support inputs of type `number`, a `number` input falls back to type `text`.

### Try it

```
<label for="tentacles">Number of tentacles (10-100):</label>

<input type="number" id="tentacles" name="tentacles" min="10" max="100" />
```

```
label {
  display: block;
  font:
    1rem "Fira Sans",
    sans-serif;
}

input,
label {
  margin: 0.4rem 0;
}
```

### Value

A number representing the value of the number entered into the input. You can set a default value for the input by including a number inside the `value` attribute, like so:

```
<input id="number" type="number" value="42" />
```

### Additional attributes

In addition to the attributes commonly supported by all `<input>` types, inputs of type `number` support these attributes.

#### <code>list</code>

The values of the list attribute is the `id` of a `<datalist>` element located in the same document. The `<datalist>` provides a list of predefined values to suggest to the user for this input. Any values in the list that are not compatible with the `type` are not included in the suggested options. The values provided are suggestions, not requirements: users can select from this predefined list or provide a different value.

#### <code>max</code>

The maximum value to accept for this input. If the `value` entered into the element exceeds this, the element fails constraint validation. If the value of the `max` attribute isn't a number, then the element has no maximum value.

This value must be greater than or equal to the value of the `min` attribute.

### <code>min</code>

The minimum value to accept for this input. If the `value` of the element is less than this, the element fails constraint validation. If a value is specified for `min` that isn't a valid number, the input has no minimum value.

This value must be less than or equal to the value of the `max` attribute.

### <code>placeholder</code>

The `placeholder` attribute is a string that provides a brief hint to the user as to what kind of information is expected in the field. It should be a word or short phrase that demonstrates the expected type of data, rather than an explanatory message. The text *must not* include carriage returns or line feeds.

If the control's content has one directionality (LTR or RTL) but needs to present the placeholder in the opposite directionality, you can use Unicode bidirectional algorithm formatting characters to override directionality within the placeholder; see How to use Unicode controls for bidi text for more information.

> **Note:** Avoid using the `placeholder` attribute if you can. It is not as semantically useful as other ways to explain your form, and can cause unexpected technical issues with your content. See `<input>` labels for more information.

### <code>readonly</code>

A Boolean attribute which, if present, means this field cannot be edited by the user. Its `value` can, however, still be changed by JavaScript code directly setting the `HTMLInputElement` `value` property.

> **Note:** Because a read-only field cannot have a value, `required` does not have any effect on inputs with the `readonly` attribute also specified.

### <code>step</code>

The `step` attribute is a number that specifies the granularity that the value must adhere to, or the special value `any`, which is described below. Only values which are a whole number of steps from the step base are valid. The step base is `min` if specified, `value` otherwise, or `0` if neither is provided.

The default stepping value for `number` inputs is `1`, allowing only integers to be entered—*unless* the stepping base is not an integer.

A string value of `any` means that no stepping is implied, and any value is allowed (barring other constraints, such as `min` and `max`).

> **Note:** When the data entered by the user doesn't adhere to the stepping configuration, the user agent may round to the nearest valid value, preferring numbers in the positive direction when there are two equally close options.

## Using number inputs

The `number` input type should only be used for incremental numbers, especially when spinbutton incrementing and decrementing are helpful to user experience. The `number` input type is not appropriate for values that happen to only consist of numbers but aren't strictly speaking a number, such as postal codes in many countries or credit card numbers. For non-numeric inputs, consider using a different input type, such as `<input type="tel">` or other `<input>` type with the `inputmode` attribute:

```
<input type="text" inputmode="numeric" pattern="\d*" />
```

`<input type="number">` elements can help simplify your work when building the user interface and logic for entering numbers into a form. When you create a number input with the proper `type` value, `number`, you get automatic validation that the entered text is a number, and usually a set of up and down buttons to step the value up and down.

> **Warning:** Logically, you should not be able to enter characters inside a number input other than numbers. Some browsers allow invalid characters, others do not; see Firefox bug 1398528.

> **Note:** A user can tinker with your HTML behind the scenes, so your site *must not* use client-side validation for security purposes. You *must* verify on the server side any transaction in which the provided value may have security implications of any kind.

Mobile browsers further help with the user experience by showing a special keyboard more suited for entering numbers when the user tries to enter a value.

## A basic number input

In its most basic form, a number input can be implemented like this:

```
<label for="ticketNum">Number of tickets you would like to buy:</label>
<input id="ticketNum" type="number" name="ticketNum" value="0" />
```

A number input is considered valid when empty and when a single number is entered, but is otherwise invalid. If the `required` attribute is used, the input is no longer considered valid when empty.

> **Note:** Any number is an acceptable value, as long as it is a valid floating point number (that is, not NaN or Infinity).

### Placeholders

Sometimes it's helpful to offer an in-context hint as to what form the input data should take. This can be especially important if the page design doesn't offer descriptive labels for each `<input>`. This is where **placeholders** come in. A placeholder is a value most commonly used to provide a hint as to the format the input should take `value`. It is displayed inside the edit box when the element's `value` is `""`. Once data is entered into the box, the placeholder disappears; if the box is emptied, the placeholder reappears.

Here, we have a `number` input with the placeholder "Multiple of 10". Note how the placeholder disappears and reappears as you manipulate the contents of the edit field.

```
<input type="number" placeholder="Multiple of 10" />
```

### Controlling step size

By default, the up and down buttons provided for you to step the number up and down will step the value up and down by 1. You can change this by providing a `step` attribute, which takes as its value a number specifying the step amount. Our above example contains a placeholder saying that the value should be a multiple of 10, so it makes sense to add a `step` value of `10`:

```
<input type="number" placeholder="multiple of 10" step="10" />
```

In this example, you should find that the up and down step arrows will increase and decrease the value by 10 each time, not 1. You can still manually enter a number that's not a multiple of 10, but it will be considered invalid.

### Specifying minimum and maximum values

You can use the `min` and `max` attributes to specify a minimum and maximum value that the field can have. For example, let's give our example a minimum of `0`, and a maximum of `100`:

```
<input type="number" placeholder="multiple of 10" step="10" min="0" max="100" />
```

In this updated version, you should find that the up and down step buttons will not allow you to go below 0 or above 100. You can still manually enter a number outside these bounds, but it will be considered invalid.

### Allowing decimal values

One issue with number inputs is that their step size is 1 by default. If you try to enter a number with a decimal value that's not a whole number (such as "1.1"), it will be considered invalid. Note that values like "1.0" are considered valid because they are numerically equivalent to whole numbers. If you want to enter values with fractions, you'll need to reflect this in the `step` value (e.g., `step="0.01"` to allow decimals to two decimal places). Here's a basic example:

```
<input type="number" placeholder="1.0" step="0.01" min="0" max="10" />
```

See that this example allows any value between `0.0` and `10.0`, with decimals to two places. For example, "9.52" is valid, but "9.521" is not.

If you want to allow arbitrary decimal values, you can set the `step` value to `"any"`.

### Controlling input size

`<input>` elements of type `number` don't support form sizing attributes such as `size`. You'll have to resort to CSS to change the size of these controls.

For example, to adjust the width of the input to be only as wide as is needed to enter a three-digit number, we can change our HTML to include an `id` and to shorten our placeholder since the field will be too narrow for the text we have been using so far:

```
<input
  type="number"
  placeholder="x10"
  step="10"
  min="0"
  max="100"
  id="number" />
```

Then we add some CSS to narrow the width of the element with the `id` selector `#number`:

```
#number {
  width: 3em;
}
```

The result looks like this:

### Offering suggested values

You can provide a list of default options from which the user can select by specifying the `list` attribute, which contains as its value the `id` of a `<datalist>`, which in turn contains one `<option>` element per suggested value. Each `option`'s `value` is the corresponding suggested value for the number entry box.

```
<input id="ticketNum" type="number" name="ticketNum" list="defaultNumbers" />
<span class="validity"></span>

<datalist id="defaultNumbers">
  <option value="10045678"></option>
  <option value="103421"></option>
  <option value="11111111"></option>
  <option value="12345678"></option>
  <option value="12999922"></option>
</datalist>
```

### Validation

We have already mentioned a number of validation features of `number` inputs, but let's review them now:

- `<input type="number">` elements automatically invalidate any entry that isn't a number (or empty, unless `required` is specified).
- You can use the `required` attribute to make an empty entry invalid. (In other words, the input *must* be filled in.)
- You can use the `step` attribute to constrain valid values to a certain set of steps (e.g., multiples of 10).
- You can use the `min` and `max` attributes to constrain valid values to lower and upper bounds.

The following example exhibits all of the above features, as well as using some CSS to display valid and invalid icons, depending on the `input`'s value:

```
<form>
  <div>
    <label for="balloons">Number of balloons to order (multiples of 10):</label>
    <input
      id="balloons"
      type="number"
      name="balloons"
      step="10"
      min="0"
      max="100"
      required />
    <span class="validity"></span>
  </div>
  <div>
    <input type="submit" />
  </div>
</form>
```

Try submitting the form with different invalid values entered — e.g., no value; a value below 0 or above 100; a value that is not a multiple of 10; or a non-numerical value — and see how the error messages the browser gives you differ with different ones.

The CSS applied to this example is as follows:

```
div {
  margin-bottom: 10px;
}

input:invalid + span::after {
  content: "✖";
  padding-left: 5px;
}

input:valid + span::after {
  content: "✓";
  padding-left: 5px;
}
```

Here we use the `:invalid` and `:valid` pseudo classes to display an appropriate invalid or valid icon as generated content on the adjacent `<span>` element, as a visual indicator of validity.

We put it on a separate `<span>` element for added flexibility. Some browsers don't display generated content very effectively on some types of form inputs. (Read, for example, the section on `<input type="date">` validation.)

> **Warning:** HTML form validation is *not* a substitute for server-side scripts that ensure that the entered data is in the proper format!
>
> It's far too easy for someone to make adjustments to the HTML that allow them to bypass the validation, or to remove it entirely. It's also possible for someone to bypass your HTML and submit the data directly to your server.
>
> If your server-side code fails to validate the data it receives, disaster could strike when improperly-formatted data is submitted (or data which is too large, is of the wrong type, and so forth).

### Pattern validation

`<input type="number">` elements do not support use of the `pattern` attribute for making entered values conform to a specific regex pattern.

The rationale for this is that number inputs won't be valid if they contain anything except numbers, and you can constrain the minimum and maximum number of valid digits using the `min` and `max` attributes (as explained above).

### Accessibility

The implicit role for the `<input type="number">` element is `spinbutton`. If spinbutton is not an important feature for your form control, consider *not* using `type="number"`. Instead, use `inputmode="numeric"` along with a `pattern` attribute that limits the characters to numbers and associated characters. With `<input type="number">`, there is a risk of users accidentally incrementing a number when they're trying to do something else. Additionally, if users try to enter something that's not a number, there's no explicit feedback about what they're doing wrong.

Also consider using the `autocomplete` attribute to help users complete forms more quickly and with fewer chances of errors. For example, to enable autofill on a zip code field, set `autocomplete="postal-code"`.

### Examples

We've already covered the fact that by default, the increment is `1`, and you can use the `step` attribute to allow decimal inputs. Let's take a closer look.

In the following example is a form for entering the user's height. It defaults to accepting a height in meters, but you can click the relevant button to change the form to accept feet and inches instead. The input for the height in meters accepts decimals to two places.

The HTML looks like this:

```
<form>
  <div class="metersInputGroup">
    <label for="meters">Enter your height — meters:</label>
    <input
      id="meters"
      type="number"
      name="meters"
      step="0.01"
      min="0"
      placeholder="e.g. 1.78"
      required />
    <span class="validity"></span>
  </div>
  <div class="feetInputGroup">
    <span>Enter your height — </span>
    <label for="feet">feet:</label>
    <input id="feet" type="number" name="feet" min="0" step="1" />
    <span class="validity"></span>
    <label for="inches">inches:</label>
    <input id="inches" type="number" name="inches" min="0" max="11" step="1" />
    <span class="validity"></span>
  </div>
  <div>
    <input
      type="button"
      class="meters"
      value="Enter height in feet and inches" />
  </div>
  <div>
    <input type="submit" value="Submit form" />
  </div>
</form>
```

You'll see that we are using many of the attributes we've already looked at in the article earlier on. Since we want to accept a meter value in centimeters, we've set the `step` value to `0.01`, so that values like *1.78* are not seen as invalid. We've also provided a placeholder for that input.

We've hidden the feet and inches inputs initially using `style="display: none;"`, so that meters is the default entry type.

Now, onto the CSS. This looks very similar to the validation styling we saw before; nothing remarkable here.

```
div {
  margin-bottom: 10px;
  position: relative;
}

input[type="number"] {
  width: 100px;
}

input + span {
  padding-right: 30px;
}

input:invalid + span::after {
  position: absolute;
  content: "✖";
  padding-left: 5px;
}

input:valid + span::after {
  position: absolute;
  content: "✓";
  padding-left: 5px;
}
```

And finally, the JavaScript:

```
const metersInputGroup = document.querySelector(".metersInputGroup");
const feetInputGroup = document.querySelector(".feetInputGroup");
const metersInput = document.querySelector("#meters");
const feetInput = document.querySelector("#feet");
const inchesInput = document.querySelector("#inches");
const switchBtn = document.querySelector('input[type="button"]');

feetInputGroup.style.display = "none"; // Hide feet/inches inputs initially

switchBtn.addEventListener("click", () => {
  if (switchBtn.getAttribute("class") === "meters") {
    switchBtn.setAttribute("class", "feet");
    switchBtn.value = "Enter height in meters";

    metersInputGroup.style.display = "none";
    feetInputGroup.style.display = "block";

    feetInput.setAttribute("required", "");
    inchesInput.setAttribute("required", "");
    metersInput.removeAttribute("required");

    metersInput.value = "";
  } else {
    switchBtn.setAttribute("class", "meters");
    switchBtn.value = "Enter height in feet and inches";

    metersInputGroup.style.display = "block";
    feetInputGroup.style.display = "none";

    feetInput.removeAttribute("required");
    inchesInput.removeAttribute("required");
    metersInput.setAttribute("required", "");

    feetInput.value = "";
    inchesInput.value = "";
  }
});
```

After declaring a few variables, an event listener is added to the `button` to control the switching mechanism. This involves changing over the button's `class` and `<label>`, and updating the display values of the two sets of inputs when the button is pressed.

(Note that we're not converting back and forth between meters and feet/inches here, which a real-life web application would probably do.)

> **Note:** When the user clicks the button, the `required` attribute(s) are removed from the input(s) we are hiding, and empty the `value` attribute(s). This is so the form can be submitted if both input sets aren't filled in. It also ensures that the form won't submit data that the user didn't mean to.
>
> If you didn't do this, you'd have to fill in both feet/inches **and** meters to submit the form!

### Technical summary

| | |
|---|---|
| **Value** | A `Number` representing a number, or empty |
| **Events** | `change` and `input` |
| **Supported common attributes** | `autocomplete`, `list`, `placeholder`, `readonly` |
| **IDL attributes** | `list`, `value`, `valueAsNumber` |
| **DOM interface** | `HTMLInputElement` |
| **Implicit ARIA Role** | `spinbutton` |

### See also

- HTML forms guide
- `<input>`
- `<input type="tel">`

## <input type="password"> HTML attribute value

`<input>` elements of type **`password`** provide a way for the user to securely enter a password.

The element is presented as a one-line plain text editor control in which the text is obscured so that it cannot be read, usually by replacing each character with a symbol such as the asterisk ("*") or a dot ("•"). This character will vary depending on the user agent and operating system.

### Try it

```
<div>
  <label for="username">Username:</label>
  <input type="text" id="username" name="username" />
</div>

<div>
  <label for="pass">Password (8 characters minimum):</label>
  <input type="password" id="pass" name="password" minlength="8" required />
</div>

<input type="submit" value="Sign in" />
```

```
label {
  display: block;
}

input[type="submit"],
label {
  margin-top: 1rem;
}
```

### Value

The `value` attribute contains a string whose value is the current contents of the text editing control being used to enter the password. If the user hasn't entered anything yet, this value is an empty string (`""`). If the `required` property is specified, then the password edit box must contain a value other than an empty string to be valid.

If the `pattern` attribute is specified, the content of a `password` control is only considered valid if the value passes validation; see Validation for more information.

> **Note:** The line feed (U+000A) and carriage return (U+000D) characters are not permitted in a `password` value. When setting the value of a password control, line feed and carriage return characters are stripped out of the value.

### Additional attributes

In addition to the global attributes, and the attributes that operate on all `<input>` elements regardless of their type, password field inputs support the following attributes.

> **Note:** The `autocorrect` global attribute can be added to password inputs, but the stored state is always `off`.

### maxlength

The maximum string length (measured in UTF-16 code units) that the user can enter into the password field. This must be an integer value of 0 or higher. If no `maxlength` is specified, or an invalid value is specified, the password field has no maximum length. This value must also be greater than or equal to the value of `minlength`.

The input will fail constraint validation if the length of the text entered into the field is greater than `maxlength` UTF-16 code units long. Constraint validation is only applied when the value is changed by the user.

### minlength

The minimum string length (measured in UTF-16 code units) that the user can enter into the password entry field. This must be a non-negative integer value smaller than or equal to the value specified by `maxlength`. If no `minlength` is specified, or an invalid value is specified, the password input has no minimum length.

The input will fail constraint validation if the length of the text entered into the field is fewer than `minlength` UTF-16 code units long. Constraint validation is only applied when the value is changed by the user.

### pattern

The `pattern` attribute, when specified, is a regular expression that the input's `value` must match for the value to pass constraint validation. It must be a valid JavaScript regular expression, as used by the `RegExp` type, and as documented in our guide on regular expressions; the `'u'` flag is specified when compiling the regular expression, so that the pattern is treated as a sequence of Unicode code points, instead of as ASCII. No forward slashes should be specified around the pattern text.

If the specified pattern is not specified or is invalid, no regular expression is applied and this attribute is ignored completely.

> **Note:** Use the `title` attribute to specify text that most browsers will display as a tooltip to explain what the requirements are to match the pattern. You should also include other explanatory text nearby.

Use of a pattern is strongly recommended for password inputs, in order to help ensure that valid passwords using a wide assortment of character classes are selected and used by your users. With a pattern, you can mandate case rules, require the use of some number of digits and/or punctuation characters, and so forth. See the section Validation for details and an example.

### placeholder

The `placeholder` attribute is a string that provides a brief hint to the user as to what kind of information is expected in the field. It should be a word or short phrase that demonstrates the expected type of data, rather than an explanatory message. The text *must not* include carriage returns or line feeds.

If the control's content has one directionality (LTR or RTL) but needs to present the placeholder in the opposite directionality, you can use Unicode bidirectional algorithm formatting characters to override directionality within the placeholder; see How to use Unicode controls for bidi text for more information.

> **Note:** Avoid using the `placeholder` attribute if you can. It is not as semantically useful as other ways to explain your form, and can cause unexpected technical issues with your content. See `<input>` labels for more information.

### readonly

A Boolean attribute which, if present, means this field cannot be edited by the user. Its `value` can, however, still be changed from JavaScript code that directly sets the value of the `HTMLInputElement.value` property.

> **Note:** Because a read-only field cannot have a value, `required` does not have any effect on inputs with the `readonly` attribute also specified.

### size

The `size` attribute is a numeric value indicating how many characters wide the input field should be. The value must be a number greater than zero, and the default value is 20. Since character widths vary, this may or may not be exact and should not be relied upon to be so; the resulting input may be narrower or wider than the specified number of characters, depending on the characters and the font (`font` settings in use).

This does *not* set a limit on how many characters the user can enter into the field. It only specifies approximately how many can be seen at a time. To set an upper limit on the length of the input data, use the `maxlength` attribute.

### Using password inputs

Password input boxes generally work just like other textual input boxes; the main difference is the obscuring of the content to prevent people near the user from reading the password.

The precise behavior of the entry process may vary from browser to browser. Some browsers display the typed character for a moment before obscuring it, while others allow the user to toggle the display of plain-text on and off. Both approaches help a user check that they entered the intended password, which can be particularly difficult on mobile devices.

> **Note:** Any forms involving sensitive information like passwords (such as login forms) should be served over HTTPS. Many browsers now implement mechanisms to warn against insecure login forms.

### A basic password input

Here we see the most basic password input, with a label established using the `<label>` element.

```
<label for="userPassword">Password: </label>
<input id="userPassword" type="password" />
```

### Allowing autocomplete

To allow the user's password manager to automatically enter the password, specify the `autocomplete` attribute. For passwords, this should typically be one of the following:

`on`

: Allow the browser or a password manager to automatically fill out the password field. This isn't as informative as using either `current-password` or `new-password`.

`off`

: Don't allow the browser or password manager to automatically fill out the password field. Note that some software ignores this value, since it's typically harmful to users' ability to maintain safe password practices.

`current-password`

: Allow the browser or password manager to enter the current password for the site. This provides more information than `on` does, since it lets the browser or password manager automatically enter currently-known password for the site in the field, but not to suggest a new one.

`new-password`

: Allow the browser or password manager to automatically enter a new password for the site; this is used on "change your password" and "new user" forms, on the field asking the user for a new password. The new password may be generated in a variety of ways, depending on the password manager in use. It may fill in a new suggested password, or it might show the user an interface for creating one.

```
<label for="userPassword">Password:</label>
<input id="userPassword" type="password" autocomplete="current-password" />
```

### Making the password mandatory

To tell the user's browser that the password field must have a valid value before the form can be submitted, specify the Boolean `required` attribute.

```
<label for="userPassword">Password: </label>
<input id="userPassword" type="password" required />
<input type="submit" value="Submit" />
```

### Specifying an input mode

If your recommended (or required) password syntax rules would benefit from an alternate text entry interface than the standard keyboard, you can use the `inputmode` attribute to request a specific one. The most obvious use case for this is if the password is required to be numeric (such as a PIN). Mobile devices with virtual keyboards, for example, may opt to switch to a numeric keypad layout instead of a full keyboard, to make entering the password easier. If the PIN is for one-time use, set the `autocomplete` attribute to either `off` or `one-time-code` to suggest that it's not saved.

```
<label for="pin">PIN: </label>
<input id="pin" type="password" inputmode="numeric" />
```

### Setting length requirements

As usual, you can use the `minlength` and `maxlength` attributes to establish minimum and maximum acceptable lengths for the password. This example expands on the previous one by specifying that the user's PIN must be at least four and no more than eight digits. The `size` attribute is used to

ensure that the password entry control is eight characters wide.

```
<label for="pin">PIN:</label>
<input
  id="pin"
  type="password"
  inputmode="numeric"
  minlength="4"
  maxlength="8"
  size="8" />
```

### Selecting text

As with other textual entry controls, you can use the `select()` method to select all the text in the password field.

#### HTML

```
<label for="userPassword">Password: </label>
<input id="userPassword" type="password" size="12" />
<button id="selectAll">Select All</button>
```

#### JavaScript

```
document.getElementById("selectAll").onclick = () => {
  document.getElementById("userPassword").select();
};
```

#### Result

You can also use `selectionStart` and `selectionEnd` to get (or set) what range of characters in the control are currently selected, and `selectionDirection` to know which direction selection occurred in (or will be extended in, depending on your platform; see its documentation for an explanation). However, given that the text is obscured, the usefulness of these is somewhat limited.

### Validation

If your application has character set restrictions or any other requirement for the actual content of the entered password, you can use the `pattern` attribute to establish a regular expression to be used to automatically ensure that your passwords meet those requirements.

In this example, only values consisting of at least four and no more than eight hexadecimal digits are valid.

```
<label for="hexId">Hex ID: </label>
<input
  id="hexId"
  type="password"
  pattern="[0-9a-fA-F]{4,8}"
  title="Enter an ID consisting of 4-8 hexadecimal digits"
  autocomplete="new-password" />
```

## Examples

### Requesting a Social Security number

This example only accepts input which matches the format for a valid United States Social Security Number. These numbers, used for tax and identification purposes in the US, are in the form "123-45-6789". Assorted rules for what values are permitted in each group exist as well.

#### HTML

```
<label for="ssn">SSN:</label>
<input
  type="password"
  id="ssn"
  inputmode="numeric"
  minlength="9"
  maxlength="12"
  pattern="(?!000)([0-6]\d{2}|7([0-6]\d|7[012]))([ -])?(?!00)\d\d\3(?!0000)\d{4}"
  required
  autocomplete="off" />
<br />
<label for="ssn">Value:</label>
<span id="current"></span>
```

This uses a `pattern` which limits the entered value to strings representing legal Social Security numbers. Obviously, this regexp doesn't guarantee a valid SSN (since we don't have access to the Social Security Administration's database), but it does ensure the number could be one; it generally

avoids values that cannot be valid. In addition, it allows the three groups of digits to be separated by a space, a dash ("-"), or nothing.

The `inputmode` is set to `numeric` to encourage devices with virtual keyboards to switch to a numeric keypad layout for easier entry. The `minlength` and `maxlength` attributes are set to 9 and 12, respectively, to require that the value be at least nine and no more than 12 characters (the former without separating characters between the groups of digits and the latter with them). The `required` attribute is used to indicate that this control must have a value. Finally, `autocomplete` is set to `off` to avoid password managers and session restore features trying to set its value, since this isn't a password at all.

### JavaScript

The JavaScript displays the entered SSN onscreen so you can see it. This defeats the purpose of a password field, but it helps experiment with the `pattern`.

```
const ssn = document.getElementById("ssn");
const current = document.getElementById("current");

ssn.oninput = (event) => {
  current.textContent = ssn.value;
};
```

### Result

### Technical summary

| | |
|---|---|
| **Value** | A string representing a password, or empty |
| **Events** | `change` and `input` |
| **Supported Common Attributes** | `autocomplete`, `inputmode`, `maxlength`, `minlength`, `pattern`, `placeholder`, `readonly`, `required`, and `size` |
| **IDL attributes** | `selectionStart`, `selectionEnd`, `selectionDirection`, and `value` |
| **DOM interface** | `HTMLInputElement` |
| **Implicit ARIA Role** | no corresponding role |

## <input type="radio"> HTML attribute value

`<input>` elements of type `radio` are generally used in **radio groups**—collections of radio buttons describing a set of related options.

Only one radio button in a given group can be selected at the same time. Radio buttons are typically rendered as small circles, which are filled or highlighted when selected.

### Try it

```
<fieldset>
  <legend>Select a maintenance drone:</legend>

  <div>
    <input type="radio" id="huey" name="drone" value="huey" checked />
    <label for="huey">Huey</label>
  </div>

  <div>
    <input type="radio" id="dewey" name="drone" value="dewey" />
    <label for="dewey">Dewey</label>
  </div>

  <div>
    <input type="radio" id="louie" name="drone" value="louie" />
    <label for="louie">Louie</label>
  </div>
</fieldset>
```

```
p,
label {
  font:
    1rem "Fira Sans",
    sans-serif;
}

input {
  margin: 0.4rem;
}
```

### Value

The `value` attribute is a string containing the radio button's value. The value is never shown to the user by their user agent. Instead, it's used to identify which radio button in a group is selected.

### Defining a radio group

A radio group is defined by giving each of radio buttons in the group the same `name`. Once a radio group is established, selecting any radio button in that group automatically deselects any currently-selected radio button in the same group.

You can have as many radio groups on a page as you like, as long as each has its own unique `name`.

For example, if your form needs to ask the user for their preferred contact method, you might create three radio buttons, each with the `name` property set to `contact` but one with the value `email`, one with the value `phone`, and one with the value `mail`. The user never sees the `value` or the `name` (unless you expressly add code to display it).

The resulting HTML looks like this:

```
<fieldset>
  <legend>Please select your preferred contact method:</legend>
  <div>
    <input type="radio" id="contactChoice1" name="contact" value="email" />
    <label for="contactChoice1">Email</label>

    <input type="radio" id="contactChoice2" name="contact" value="phone" />
    <label for="contactChoice2">Phone</label>

    <input type="radio" id="contactChoice3" name="contact" value="mail" />
    <label for="contactChoice3">Mail</label>
  </div>
</fieldset>
```

Here you see the three radio buttons, each with the `name` set to `contact` and each with a unique `value` that uniquely identifies that individual radio button within the group. They each also have a unique `id`, which is used by the `<label>` element's `for` attribute to associate the labels with the radio buttons.

You can try out this example here:

### Data representation of a radio group

When a form containing the above controls is submitted with a radio button selected, the form's data includes an entry in the form `contact=value`. For example, if the user clicks on the "Phone" radio button then submits the form, `contact=phone` will be included in the the form's submitted data.

If you omit the `value` attribute in the HTML, the submitted form data assigns the value `on` to the group. In this scenario, if the user clicked on the "Phone" option and submitted the form, the resulting form data would be `contact=on`, which isn't helpful. So don't forget to set your `value` attributes!

> **Note:** If no radio button is selected when the form is submitted, the radio group is not included in the submitted form data at all, since there is no value to report.

It's fairly uncommon to actually want to allow the form to be submitted without any of the radio buttons in a group selected, so it is usually wise to have one default to the `checked` state. See Selecting a radio button by default below.

Let's add a bit of code to our example so we can examine the data generated by these controls. The HTML is revised to wrap the controls in a `<form>`, add a submit button, and add a `<pre>` block to output the form data into:

```
<form>
  <fieldset>
    <legend>Please select your preferred contact method:</legend>
    <div>
      <input type="radio" id="contactChoice1" name="contact" value="email" />
      <label for="contactChoice1">Email</label>
      <input type="radio" id="contactChoice2" name="contact" value="phone" />
      <label for="contactChoice2">Phone</label>
      <input type="radio" id="contactChoice3" name="contact" value="mail" />
      <label for="contactChoice3">Mail</label>
    </div>
    <div>
      <button type="submit">Submit</button>
    </div>
  </fieldset>
</form>
<pre id="log"></pre>
```

Then we add some JavaScript to set up an event listener on the `submit` event, which is sent when the user clicks the "Submit" button:

```
const form = document.querySelector("form");
const log = document.querySelector("#log");

form.addEventListener("submit", (event) => {
  const data = new FormData(form);
  let output = "";
  for (const entry of data) {
    output = `${output}${entry[0]}=${entry[1]}\r`;
  }
  log.innerText = output;
  event.preventDefault();
});
```

Try this example out and see how there's never more than one result for the `contact` group.

### Additional attributes

In addition to the common attributes shared by all `<input>` elements, `radio` inputs support the following attributes.

`checked`

> A Boolean attribute which, if present, indicates that this radio button is the default selected one in the group.
>
> Unlike other browsers, Firefox by default persists the dynamic checked state of an `<input>` across page loads. Use the `autocomplete` attribute to control this feature.

`value`

> The `value` attribute is one which all `<input>`s share; however, it serves a special purpose for inputs of type `radio`: when a form is submitted, only radio buttons which are currently checked are submitted to the server, and the reported value is the value of the `value` attribute. If the `value` is not otherwise specified, it is the string `on` by default. This is demonstrated in the section Value above.

`required`

> The `required` attribute is one which most `<input>`s share. If any radio button in a same-named group of radio buttons has the `required` attribute, a radio button in that group must be checked, although it doesn't have to be the one with the attribute applied.

### Using radio inputs

Radio buttons look and operate in a similar manner to the push buttons on old-fashioned radios, such as the one shown below.

 Shows what radio buttons looked like in the olden days.

Radio buttons are similar to checkboxes, but with an important distinction: radio buttons are designed for selecting one value out of a set, whereas checkboxes let you turn individual values on and off. Where multiple controls exist, radio buttons allow one to be selected out of them all, whereas checkboxes allow multiple values to be selected.

We already covered the fundamentals of radio buttons above. Let's now look at the other common radio-button-related features and techniques you may need to know about.

### Selecting a radio button by default

To make a radio button selected by default, you include `checked` attribute, as shown in this revised version of the previous example:

```
<fieldset>
  <legend>Please select your preferred contact method:</legend>
  <div>
    <input
      type="radio"
      id="contactChoice1"
      name="contact"
      value="email"
      checked />
    <label for="contactChoice1">Email</label>

    <input type="radio" id="contactChoice2" name="contact" value="phone" />
    <label for="contactChoice2">Phone</label>

    <input type="radio" id="contactChoice3" name="contact" value="mail" />
    <label for="contactChoice3">Mail</label>
  </div>
</fieldset>
```

In this case, the first radio button is now selected by default.

> **Note:** If you put the `checked` attribute on more than one radio button, later instances will override earlier ones; that is, the last `checked` radio button will be the one that is selected. This is because only one radio button in a group can ever be selected at once, and the user agent automatically deselects others each time a new one is marked as checked.

### Providing a bigger hit area for your radio buttons

In the above examples, you may have noticed that you can select a radio button by clicking on its associated `<label>` element, as well as on the radio button itself. This is a really useful feature of HTML form labels that makes it easier for users to click the option they want, especially on small-screen devices like smartphones.

Beyond accessibility, this is another good reason to properly set up `<label>` elements on your forms.

### Validation

In the case of a radio button with the `required` attribute set, or a same-named group of radio buttons in which at least one member has `required` set, a radio button needs to be selected for the control to be considered valid. If no radio button is checked, the `valueMissing` property of a `ValidityState` object will return `true` during validation, and the browser will ask the user to select an option.

### Styling radio inputs

The following example shows a slightly more thorough version of the example we've seen throughout the article, with some additional styling, and with better semantics established through use of specialized elements. The HTML looks like this:

```
<fieldset>
  <legend>Please select your preferred contact method:</legend>
  <div>
    <input
      type="radio"
      id="contactChoice1"
      name="contact"
      value="email"
      checked />
    <label for="contactChoice1">Email</label>

    <input type="radio" id="contactChoice2" name="contact" value="phone" />
    <label for="contactChoice2">Phone</label>

    <input type="radio" id="contactChoice3" name="contact" value="mail" />
    <label for="contactChoice3">Mail</label>
  </div>
</fieldset>
```

The CSS involved in this example is a bit more significant:

```
html {
  font-family: sans-serif;
}

div:first-of-type {
  display: flex;
  align-items: flex-start;
  margin-bottom: 5px;
}

label {
  margin-right: 15px;
  line-height: 32px;
}

input {
  appearance: none;

  border-radius: 50%;
  width: 16px;
  height: 16px;

  border: 2px solid #999999;
  transition: 0.2s all linear;
  margin-right: 5px;

  position: relative;
  top: 4px;
}
```

```
input:checked {
  border: 6px solid black;
}

legend {
  color: white;
  background-color: black;
  padding: 5px 10px;
  border-radius: 0;
  border: 0;
  font-size: 14px;
}
```

Most notable here is the use of the `appearance` property (with prefixes needed to support some browsers). By default, radio buttons (and checkboxes) are styled with the operating system's native styles for those controls. By specifying `appearance: none`, you can remove the native styling altogether, and create your own styles for them. Here we've used a `border` along with `border-radius` and a `transition` to create a nice animating radio selection. Notice also how the `:checked` pseudo-class is used to specify the styles for the radio button's appearance when selected.

> **Note:** If you wish to use the `appearance` property, you should test it very carefully. Although it is supported in most modern browsers, its implementation varies widely. In older browsers, even the keyword `none` does not have the same effect across different browsers, and some do not support it at all. The differences are smaller in the newest browsers.

Notice that when clicking on a radio button, there's a nice, smooth fade out/in effect as the two buttons change state. In addition, the style and coloring of the legend are customized to have strong contrast. This might not be a look you'd want in a real web application, but it definitely shows off the possibilities.

### Technical summary

| | |
|---|---|
| **Value** | A string representing the value of the radio button. |
| **Events** | `change` and `input` |
| **Supported common attributes** | `checked`, `value` and `required` |
| **IDL attributes** | `checked` and `value` |
| **DOM interface** | `HTMLInputElement` |
| **Implicit ARIA Role** | `radio` |

### See also

- `<input>` and the `HTMLInputElement` interface that implements it.
- `RadioNodeList`: the interface that describes a list of radio buttons

## <input type="range"> HTML attribute value

`<input>` elements of type **`range`** let the user specify a numeric value which must be no less than a given value, and no more than another given value. The precise value, however, is not considered important. This is typically represented using a slider or dial control rather than a text entry box like the number input type.

Because this kind of widget is imprecise, it should only be used if the control's exact value isn't important.

If the user's browser doesn't support type `range`, it will fall back and treat it as a `text` input.

### Try it

```
<p>Audio settings:</p>

<div>
  <input type="range" id="volume" name="volume" min="0" max="11" />
  <label for="volume">Volume</label>
</div>

<div>
  <input
    type="range"
    id="cowbell"
    name="cowbell"
    min="0"
    max="100"
    value="90"
    step="10" />
```

```
  <label for="cowbell">Cowbell</label>
</div>
```

```
p,
label {
  font:
    1rem "Fira Sans",
    sans-serif;
}

input {
  margin: 0.4rem;
}
```

### Value

The value of an `<input type="range">` element is set using the `value` attribute which accepts a string representing the selected number. The value is never an empty string (`""`). The default value is halfway between the specified minimum and maximum—unless the maximum is actually less than the minimum, in which case the default is set to the value of the `min` attribute. The algorithm for determining the default value is:

```
defaultValue =
  rangeElem.max < rangeElem.min
    ? rangeElem.min
    : rangeElem.min + (rangeElem.max - rangeElem.min) / 2;
```

If an attempt is made to set the value lower than the minimum, it is set to the minimum. Similarly, an attempt to set the value higher than the maximum results in it being set to the maximum.

### Validation

There is no pattern validation available; however, the following forms of automatic validation are performed:

- If the `value` is set to something which can't be converted into a valid floating-point number, validation fails because the input is suffering from a bad input.
- The value won't be less than `min`. The default is 0.
- The value won't be greater than `max`. The default is 100.
- The value will be a multiple of `step`. The default is 1.

### Additional attributes

In addition to the attributes shared by all `<input>` elements, range inputs offer the following attributes.

> **Note:** The following input attributes do not apply to the input range: `accept`, `alt`, `checked`, `dirname`, `formaction`, `formenctype`, `formmethod`, `formnovalidate`, `formtarget`, `height`, `maxlength`, `minlength`, `multiple`, `pattern`, `placeholder`, `readonly`, `required`, `size`, and `src`. Any of these attributes, if included, will be ignored.

### list

The value of the `list` attribute is the `id` of a `<datalist>` element located in the same document. The `<datalist>` provides a list of predefined values to suggest to the user for this input. Any values in the list that are not compatible with the `type` are not included in the suggested options. The values provided are suggestions, not requirements: users can select from this predefined list or provide a different value.

See the adding tick marks below for an example of how the options on a range are denoted in supported browsers.

### max

The greatest value in the range of permitted values. If the `value` entered into the element exceeds this, the element fails constraint validation. If the value of the `max` attribute isn't a number, then the element has no maximum value.

This value must be greater than or equal to the value of the `min` attribute. See the HTML `max` attribute.

### min

The lowest value in the range of permitted values. If the `value` of the element is less than this, the element fails constraint validation. If a value is specified for `min` that isn't a valid number, the input has no minimum value.

This value must be less than or equal to the value of the `max` attribute. See the HTML `min` attribute.

> **Note:** If the `min` and `max` values are equal or the `max` value is lower than the `min` value the user will not be able to interact with the range.

#### step

The `step` attribute is a number that specifies the granularity that the value must adhere to, or the special value `any`, which is described below. Only values which are a whole number of steps from the step base are valid. The step base is `min` if specified, `value` otherwise, or `0` if neither is provided.

The default stepping value for `step` inputs is `1`, allowing only integers to be entered—*unless* the stepping base is not an integer.

A string value of `any` means that no stepping is implied, and any value is allowed (barring other constraints, such as `min` and `max`). See the Setting step to the `any` value example for how this works in supported browsers.

> **Note:** When the value entered by a user doesn't adhere to the stepping configuration, the user agent may round off the value to the nearest valid value, preferring to round numbers up when there are two equally close options.

### Non-standard attributes

#### orient

Similar to the -moz-orient non-standard CSS property impacting the `<progress>` and `<meter>` elements, the `orient` attribute defines the orientation of the range slider. Values include `horizontal`, meaning the range is rendered horizontally, and `vertical`, where the range is rendered vertically.

### Examples

While the `number` type lets users enter a number with optional constraints forcing their value to be between a minimum and a maximum value, it does require that they enter a specific value. The `range` input type lets you ask the user for a value in cases where the user may not even care—or know—what the specific numeric value selected is.

A few examples of situations in which range inputs are commonly used:

- Audio controls such as volume and balance, or filter controls.
- Color configuration controls such as color channels, transparency, brightness, etc.
- Game configuration controls such as difficulty, visibility distance, world size, and so forth.
- Password length for a password manager's generated passwords.

As a rule, if the user is more likely to be interested in the percentage of the distance between minimum and maximum values than the actual number itself, a range input is a great candidate. For example, in the case of a home stereo volume control, users typically think "set volume at halfway to maximum" instead of "set volume to 0.5".

#### Specifying the minimum and maximum

By default, the minimum is 0 and the maximum is 100. If that's not what you want, you can easily specify different bounds by changing the values of the `min` and/or `max` attributes. These can be any floating-point value.

For example, to ask the user for a value between -10 and 10, you can use:

```
<input type="range" min="-10" max="10" />
```

#### Setting the value's granularity

By default, the granularity is 1, meaning the value is always an integer. To control the granularity, you can change the `step` attribute. For example, if you need a value to be halfway between 5 and 10, you should set the value of `step` to 0.5:

#### Setting the step attribute

```
<input type="range" min="5" max="10" step="0.5" />
```

#### Setting step to `any`

If you want to accept any value regardless of how many decimal places it extends to, you can specify a value of `any` for the `step` attribute:

#### HTML

```
<input id="pi_input" type="range" min="0" max="3.14" step="any" />
<p>Value: <output id="value"></output></p>
```

#### JavaScript

```
const value = document.querySelector("#value");
const input = document.querySelector("#pi_input");
value.textContent = input.value;
input.addEventListener("input", (event) => {
```

```
  value.textContent = event.target.value;
});
```

This example lets the user select any value between 0 and π without any restriction on the fractional part of the value selected. JavaScript is used to show how the value changes as the user interacts with the range.

### Adding tick marks

To add tick marks to a range control, include the `list` attribute, giving it the `id` of a `<datalist>` element which defines a series of tick marks on the control. Each point is represented using an `<option>` element with its `value` set to the range's value at which a mark should be drawn.

#### HTML

```
<label for="temp">Choose a comfortable temperature:</label><br />
<input type="range" id="temp" name="temp" list="markers" />

<datalist id="markers">
  <option value="0"></option>
  <option value="25"></option>
  <option value="50"></option>
  <option value="75"></option>
  <option value="100"></option>
</datalist>
```

#### Result

### Using the same datalist for multiple range controls

To help you from repeating code you can reuse that same `<datalist>` for multiple `<input type="range">` elements, and other `<input>` types.

> **Note:** If you also want to show the labels as in the example below then you would need a `datalist` for each range input.

#### HTML

```
<p>
  <label for="temp1">Temperature for room 1:</label>
  <input type="range" id="temp1" name="temp1" list="values" />
</p>
<p>
  <label for="temp2">Temperature for room 2:</label>
  <input type="range" id="temp2" name="temp2" list="values" />
</p>

<p>
  <label for="temp3">Temperature for room 3:</label>
  <input type="range" id="temp3" name="temp3" list="values" />
</p>

<datalist id="values">
  <option value="0" label="0"></option>
  <option value="25" label="25"></option>
  <option value="50" label="50"></option>
  <option value="75" label="75"></option>
  <option value="100" label="100"></option>
</datalist>
```

#### Result

### Adding labels

You can label tick marks by giving the `<option>` elements `label` attributes. However, the label content will not be displayed by default. You can use CSS to show the labels and to position them correctly. Here's one way you could do this.

#### HTML

```
<label for="tempB">Choose a comfortable temperature:</label><br />
<input type="range" id="tempB" name="temp" list="values" />

<datalist id="values">
  <option value="0" label="very cold!"></option>
  <option value="25" label="cool"></option>
  <option value="50" label="medium"></option>
  <option value="75" label="getting warm!"></option>
```

```
  <option value="100" label="hot!"></option>
</datalist>
```

### CSS

```
datalist {
  display: flex;
  flex-direction: column;
  justify-content: space-between;
  writing-mode: vertical-lr;
  width: 200px;
}

option {
  padding: 0;
}

input[type="range"] {
  width: 200px;
  margin: 0;
}
```

### Result

### Creating vertical range controls

By default, browsers render range inputs as sliders with the knob sliding left and right.

To create a vertical range wherein the thumb slides up and down, set the `writing-mode` property with a value of either `vertical-rl` or `vertical-lr`:

```
input[type="range"] {
  writing-mode: vertical-lr;
}
```

This causes the range slider to render vertically:

You can also set the CSS `appearance` property to the non-standard `slider-vertical` value if you want to support older versions of Chrome and Safari, and include the non-standard `orient="vertical"` attribute to support older versions of Firefox.

See Creating vertical form controls for examples.

### Technical summary

| | |
|---|---|
| **Value** | A string containing the string representation of the selected numeric value; use `valueAsNumber` to get the value as a number. |
| **Events** | `change` and `input` |
| **Supported common attributes** | `autocomplete`, `list`, `max`, `min`, `step` |
| **IDL attributes** | `list`, `value`, `valueAsNumber` |
| **DOM interface** | `HTMLInputElement` |
| **Implicit ARIA Role** | `slider` |

### See also

- HTML Forms
- `<input>` and the `HTMLInputElement` interface it's based upon
- `<input type="number">`
- `validityState.rangeOverflow` and `validityState.rangeUnderflow`
- Controlling multiple parameters with ConstantSourceNode
- Creating vertical form controls
- Styling the range element

## <input type="reset"> HTML attribute value

`<input>` elements of type **`reset`** are rendered as buttons, with a default `click` event handler that resets all inputs in the form to their initial values.

### Try it

```
<form>
  <div class="controls">
    <label for="comments">Comments:</label>
```

```
    <textarea id="comments" name="comments"></textarea>

    <input type="reset" value="Reset" />
  </div>
</form>
```

```
.controls {
  padding-top: 1rem;
  display: grid;
  grid-template-rows: repeat(2, auto);
  grid-template-columns: 1fr 2fr;
  gap: 0.7rem;
}

label {
  font-size: 0.8rem;
  justify-self: end;
}

input[type="reset"] {
  width: 5rem;
  justify-self: end;
}

input[type="reset"] {
  grid-column: 2;
  grid-row: 2;
}
```

### Value

An `<input type="reset">` element's `value` attribute contains a string that is used as the button's label providing the button with an accessible description. Buttons such as `reset` don't have a value otherwise.

### Setting the value attribute

```
<input type="reset" value="Reset the form" />
```

### Omitting the value attribute

If you don't specify a `value`, you get a button with the default label (typically "Reset," but this will vary depending on the user agent):

```
<input type="reset" />
```

### Using reset buttons

`<input type="reset">` buttons are used to reset forms. If you want to create a custom button and then customize the behavior using JavaScript, you need to use `<input type="button">`, or better still, a `<button>` element.

You should usually avoid including reset buttons in your forms. They're rarely useful, and are instead more likely to frustrate users who click them by mistake (often while trying to click the submit button).

### A basic reset button

We'll begin by creating a basic reset button:

```
<form>
  <div>
    <label for="example">Type in some sample text</label>
    <textarea id="example"></textarea>
  </div>
  <div>
    <input type="reset" value="Reset the form" />
  </div>
</form>
```

This renders like so:

Try entering some text into the text field, and then pressing the reset button.

### Adding a reset keyboard shortcut

To add a keyboard shortcut to a reset button — just as you would with any `<input>` for which it makes sense — you use the `accesskey` global attribute.

In this example, `r` is specified as the access key (you'll need to press `r` plus the particular modifier keys for your browser/OS combination; see `accesskey` for a useful list of those).

```
<form>
  <div>
    <label for="example">Type in some sample text</label>
    <textarea id="example"></textarea>
  </div>
  <div>
    <input type="reset" value="Reset the form" accesskey="r" />
  </div>
</form>
```

The problem with the above example is that there's no way for the user to know what the access key is! This is especially true since the modifiers are typically non-standard to avoid conflicts. When building a site, be sure to provide this information in a way that doesn't interfere with the site design (for example by providing an easily accessible link that points to information on what the site access keys are). Adding a tooltip to the button (using the `title` attribute) can also help, although it's not a complete solution for accessibility purposes.

### Disabling and enabling a reset button

To disable a reset button, specify the `disabled` attribute on it, like so:

```
<input type="reset" value="Disabled" disabled />
```

You can enable and disable buttons at run time by setting `disabled` to `true` or `false`; in JavaScript this looks like `btn.disabled = true` or `btn.disabled = false`.

> **Note:** See the `<input type="button">` page for more ideas about enabling and disabling buttons.

### Validation

Buttons don't participate in constraint validation; they have no real value to be constrained.

### Examples

We've included basic examples above. There isn't really anything more to say about reset buttons.

### Technical summary

| | |
|---|---|
| **Value** | A string used as the button's label |
| **Events** | `click` |
| **Supported common attributes** | `type` and `value` |
| **IDL attributes** | `value` |
| **DOM interface** | `HTMLInputElement` |
| **Implicit ARIA Role** | `button` |

### See also

- `<input>` and the `HTMLInputElement` interface which implements it.
- Forms and buttons
- HTML forms
- The `<button>` element

## `<input type="search">` HTML attribute value

`<input>` elements of type **`search`** are text fields designed for the user to enter search queries into. These are functionally identical to `text` inputs, but may be styled differently by the user agent.

### Try it

```
<label for="site-search">Search the site:</label>
<input type="search" id="site-search" name="q" />

<button>Search</button>
```

```
label {
  display: block;
  font:
```

```
    1rem "Fira Sans",
    sans-serif;
}

input,
label {
  margin: 0.4rem 0;
}
```

### Value

The `value` attribute contains a string representing the value contained in the search field. You can retrieve this using the `HTMLInputElement.value` property in JavaScript.

```
searchTerms = mySearch.value;
```

If no validation constraints are in place for the input (see Validation for more details), the value can be any text string or an empty string (`""`).

### Additional attributes

In addition to the global attributes and the attributes that operate on all `<input>` elements regardless of their type, search field inputs support the following attributes.

### list

The values of the list attribute is the `id` of a `<datalist>` element located in the same document. The `<datalist>` provides a list of predefined values to suggest to the user for this input. Any values in the list that are not compatible with the `type` are not included in the suggested options. The values provided are suggestions, not requirements: users can select from this predefined list or provide a different value.

### maxlength

The maximum string length (measured in UTF-16 code units) that the user can enter into the search field. This must be an integer value of 0 or higher. If no `maxlength` is specified, or an invalid value is specified, the search field has no maximum length. This value must also be greater than or equal to the value of `minlength`.

The input will fail constraint validation if the length of the text entered into the field is greater than `maxlength` UTF-16 code units long. Constraint validation is only applied when the value is changed by the user.

### minlength

The minimum string length (measured in UTF-16 code units) that the user can enter into the search field. This must be a non-negative integer value smaller than or equal to the value specified by `maxlength`. If no `minlength` is specified, or an invalid value is specified, the search input has no minimum length.

The search field will fail constraint validation if the length of the text entered into the field is fewer than `minlength` UTF-16 code units long. Constraint validation is only applied when the value is changed by the user.

### pattern

The `pattern` attribute, when specified, is a regular expression that the input's `value` must match for the value to pass constraint validation. It must be a valid JavaScript regular expression, as used by the `RegExp` type, and as documented in our guide on regular expressions; the `'u'` flag is specified when compiling the regular expression so that the pattern is treated as a sequence of Unicode code points, instead of as ASCII. No forward slashes should be specified around the pattern text.

If the specified pattern is not specified or is invalid, no regular expression is applied and this attribute is ignored completely.

> **Note:** Use the `title` attribute to specify text that most browsers will display as a tooltip to explain what the requirements are to match the pattern. You should also include other explanatory text nearby.

See the section Specifying a pattern for details and an example.

### placeholder

The `placeholder` attribute is a string that provides a brief hint to the user as to what kind of information is expected in the field. It should be a word or short phrase that demonstrates the expected type of data, rather than an explanatory message. The text *must not* include carriage returns or line feeds.

If the control's content has one directionality (LTR or RTL) but needs to present the placeholder in the opposite directionality, you can use Unicode bidirectional algorithm formatting characters to override directionality within the placeholder; see How to use Unicode controls for bidi text for more information.

> **Note:** Avoid using the `placeholder` attribute if you can. It is not as semantically useful as other ways to explain your form, and can cause unexpected technical issues with your content. See `<input>` labels for more information.

### readonly

A Boolean attribute which, if present, means this field cannot be edited by the user. Its `value` can, however, still be changed by JavaScript code directly setting the `HTMLInputElement` `value` property.

> **Note:** Because a read-only field cannot have a value, `required` does not have any effect on inputs with the `readonly` attribute also specified.

### size

The `size` attribute is a numeric value indicating how many characters wide the input field should be. The value must be a number greater than zero, and the default value is 20. Since character widths vary, this may or may not be exact and should not be relied upon to be so; the resulting input may be narrower or wider than the specified number of characters, depending on the characters and the font (`font` settings in use).

This does *not* set a limit on how many characters the user can enter into the field. It only specifies approximately how many can be seen at a time. To set an upper limit on the length of the input data, use the `maxlength` attribute.

### spellcheck

`spellcheck` is a global attribute that is used to indicate whether to enable spell-checking for an element. It can be used on any editable content, but here we consider specifics related to the use of `spellcheck` on `<input>` elements. The permitted values for `spellcheck` are:

`false`

: Disable spell-checking for this element.

`true`

: Enable spell-checking for this element.

"" (empty string) or no value

: Follow the element's default behavior for spell-checking. This may be based upon a parent's `spellcheck` setting or other factors.

An input field can have spell-checking enabled if it doesn't have the readonly attribute set and is not disabled.

The value returned by reading `spellcheck` may not reflect the actual state of spell-checking within a control, if the user agent's preferences override the setting.

### Non-standard attributes

The following non-standard attributes are available to search input fields. Avoid using them where possible.

### incremental

The Boolean attribute `incremental` is a WebKit and Blink extension (so supported by Safari, Opera, Chrome, etc.) which, if present, tells the user agent to process the input as a live search. As the user edits the value of the field, the user agent sends `search` events to the `HTMLInputElement` object representing the search box. This allows your code to update the search results in real time as the user edits the search.

If `incremental` is not specified, the `search` event is only sent when the user explicitly initiates a search (such as by pressing the `Enter` or `Return` key while editing the field).

The `search` event is rate-limited so that it is not sent more frequently than an implementation-defined interval.

### results

The `results` attribute—supported only by Safari—is a numeric value that lets you override the maximum number of entries to be displayed in the `<input>` element's natively-provided drop-down menu of previous search queries.

The value must be a non-negative decimal number. If not provided, or an invalid value is given, the browser's default maximum number of entries is used.

### Using search inputs

`<input>` elements of type `search` are very similar to those of type `text`, except that they are specifically intended for handling search terms. They are basically equivalent in behavior, but user agents may choose to style them differently by default (and, of course, sites may use stylesheets to apply custom styles to them).

### Basic example

```
<input type="search" id="mySearch" name="q" />
```

This renders like so:

`q` is the most common `name` given to search inputs, although it's not mandatory. If this input is included in a form and submitted, the data name/value pair sent to the server will be `q=searchTerm`.

> **Note:** You must remember to set a `name` for your input, otherwise nothing will be submitted.

### Differences between search and text types

The main basic differences come in the way browsers handle them. The first thing to note is that some browsers show a cross icon that can be clicked on to remove the search term instantly if desired, in Chrome this action is also triggered when pressing escape. The following screenshot comes from Chrome:


Focused search input, with focus ring, with the text 'cats'. There is an x icon in the input abutting the right side.

In addition, modern browsers also tend to automatically store search terms previously entered across domains, which then come up as autocomplete options when subsequent searches are performed in search inputs on that domain. This helps users who tend to do searches on the same or similar search queries over time. This screenshot is from Firefox:


An input in error state with a red focus ring. The user has entered the letter 'h'. A pop-up selection list is open directly under the input box with two options: hello and hermansje.

At this point, let's look at some useful techniques you can apply to your search forms.

### Setting placeholders

You can provide a useful placeholder inside your search input that could give a hint on what to do using the `placeholder` attribute. Look at the following example:

```
<input type="search" id="mySearch" name="q" placeholder="Search the site…" />
```

You can see how the placeholder is rendered below:

### Search form labels and accessibility

One problem with search forms is their accessibility; a common design practice is not to provide a label for the search field (although there might be a magnifying glass icon or similar), as the purpose of a search form is normally fairly obvious for sighted users due to placement (this example shows a typical visual pattern).

This could, however, cause confusion for screen reader users, as they will not receive any verbal indication of what the search input is. One way around this that won't impact your visual design is to use landmark elements.

- Wrap the whole search functionality in a `<search>` element, which creates a landmark region that assistive technologies can announce and quickly navigate to. If your `<input>` is already in a `<form>`, you can alternatively add `role="search"` to the `<form>` element, which also makes the `<form>` a search landmark. The `<search>` element uses native HTML semantics, while `role="search"` has more support and may be terser to type if you already have a `<form>` wrapper.
- If that isn't enough, you can use an `aria-label` attribute on the `<input>` itself. This should be a descriptive text label that will be read out by the screen reader; it's used as a non-visual equivalent to `<label>`.

Let's have a look at a minimal example:

```
<search>
  <input
    type="search"
    id="mySearch"
    name="q"
    placeholder="Search the site…"
    aria-label="Search through site content" />
</search>
```

You can see how this is rendered below:

There is no visual difference from the previous example, but screen reader users have way more information available to them.

> **Note:** See Signposts/Landmarks for more information about such accessibility features.

### Physical input element size

The physical size of the input box can be controlled using the `size` attribute. With it, you can specify the number of characters the input box can display at a time. In this example, for instance, the search box is 30 characters wide:

```
<input
  type="search"
  id="mySearch"
  name="q"
  placeholder="Search the site…"
  size="30" />
```

The result is this wider input box:

### Validation

`<input>` elements of type `search` have the same validation features available to them as regular `text` inputs. It is less likely that you'd want to use validation features in general for search boxes. In many cases, users should just be allowed to search for anything, but there are a few cases to consider, such as searches against data of a known format.

> **Note:** HTML form validation is *not* a substitute for scripts that ensure that the entered data is in the proper format. It's far too easy for someone to make adjustments to the HTML that allow them to bypass the validation, or to remove it entirely. It's also possible for someone to bypass your HTML entirely and submit the data directly to your server. If your server-side code fails to validate the data it receives, disaster could strike when improperly-formatted data (or data which is too large, is of the wrong type, and so forth) is entered into your database.

### A note on styling

There are useful pseudo-classes available for styling valid/invalid form elements: `:valid` and `:invalid`. In this section, we'll use the following CSS, which will place a check (tick) next to inputs containing valid values, and a cross next to inputs containing invalid values.

```
input:invalid ~ span::after {
  content: "✖";
  padding-left: 5px;
  position: absolute;
}

input:valid ~ span::after {
  content: "✓";
  padding-left: 5px;
  position: absolute;
}
```

The technique also requires a `<span>` element to be placed after the form element, which acts as a holder for the icons. This was necessary because some input types on some browsers don't display icons placed directly after them very well.

### Making input required

You can use the `required` attribute as an easy way of making entering a value required before form submission is allowed:

```
<form>
  <div>
    <input
      type="search"
      id="mySearch"
      name="q"
      placeholder="Search the site…"
      required />
    <button>Search</button>
    <span class="validity"></span>
  </div>
</form>
```

This renders like so:

In addition, if you try to submit the form with no search term entered into it, the browser will show a message. The following example is from Firefox:




Different messages will be shown when you try to submit the form with different types of invalid data contained inside the inputs; see the below examples.

### Input value length

You can specify a minimum length, in characters, for the entered value using the `minlength` attribute; similarly, use `maxlength` to set the maximum length of the entered value.

The example below requires that the entered value be 4–8 characters in length.

```
<form>
  <div>
    <label for="mySearch">Search for user</label>
    <input
      type="search"
      id="mySearch"
      name="q"
      placeholder="User IDs are 4–8 characters in length"
      required
      size="30"
      minlength="4"
      maxlength="8" />
    <button>Search</button>
    <span class="validity"></span>
  </div>
</form>
```

This renders like so:

If you try to submit the form with less than 4 characters, you'll be given an appropriate error message (which differs between browsers). If you try to go beyond 8 characters in length, the browser won't let you.

### Specifying a pattern

You can use the `pattern` attribute to specify a regular expression that the inputted value must follow to be considered valid (see Validating against a regular expression for a crash course).

Let's look at an example. Say we wanted to provide a product ID search form, and the IDs were all codes of two letters followed by four numbers. The following example covers it:

```
<form>
  <div>
    <label for="mySearch">Search for product by ID:</label>
    <input
      type="search"
      id="mySearch"
      name="q"
      placeholder="two letters followed by four numbers"
      required
      size="30"
      pattern="[A-z]{2}[0-9]{4}" />
    <button>Search</button>
    <span class="validity"></span>
  </div>
</form>
```

This renders like so:

### Examples

### Accessible search form

This example shows a typical `<nav>` menu that includes a search form. WAI-ARIA features are used to make the form more accessible to assistive technology (AT) users.

### HTML

The main nav items are comprised of a list of links. The search `<form>` is given a `role="search"` attribute, which makes the `<form>` a search landmark, announcing it to AT. We also give the `<input type="search">` element an `aria-label` to give it a clear label that is read out by screen readers.

```
<nav>
  <ul>
    <li><a href="#">Home</a></li>
    <li><a href="#">About</a></li>
    <li><a href="#">Contact</a></li>
  </ul>
  <form role="search">
    <input
      type="search"
      name="q"
```

```
      placeholder="Search query"
      aria-label="Search through site content" />
    <button>Search!</button>
  </form>
</nav>
```

We have hidden the example's CSS and JavaScript for brevity, as they are not relevant to understanding the HTML. You can check them out by pressing the "Play" button in the live example below.

### Result

### Technical summary

| | |
|---|---|
| **Value** | A string representing the value contained in the search field. |
| **Events** | `change` and `input` |
| **Supported Common Attributes** | `autocomplete`, `list`, `maxlength`, `minlength`, `pattern`, `placeholder`, `required`, `size`. |
| **IDL attributes** | `value` |
| **DOM interface** | `HTMLInputElement` |
| **Implicit ARIA Role** | with no `list` attribute: `searchbox`<br>with `list` attribute: `combobox` |

### See also

- HTML Forms
- `<input>` and the `HTMLInputElement` interface it's based upon
- `<input type="text">`

## <input type="submit"> HTML attribute value

`<input>` elements of type **`submit`** are rendered as buttons. When the `click` event occurs (typically because the user clicked the button), the user agent attempts to submit the form to the server.

### Value

An `<input type="submit">` element's `value` attribute contains a string which is displayed as the button's label. Buttons do not have a true value otherwise. The `value` provides the accessible description for the button.

### Setting the value attribute

```
<input type="submit" value="Send Request" />
```

### Omitting the value attribute

If you don't specify a `value`, the button will have a default label, chosen by the user agent. This label is likely to be something along the lines of "Submit" or "Submit Query." Here's an example of a submit button with a default label in your browser:

```
<input type="submit" />
```

### Additional attributes

In addition to the attributes shared by all `<input>` elements, `submit` button inputs support the following attributes.

### formaction

A string indicating the URL to which to submit the data. This takes precedence over the `action` attribute on the `<form>` element that owns the `<input>`.

This attribute is also available on `<input type="image">` and `<button>` elements.

### formenctype

A string that identifies the encoding method to use when submitting the form data to the server. There are three permitted values:

`application/x-www-form-urlencoded`

> This, the default value, sends the form data as a string after percent-encoding the text using an algorithm such as `encodeURI()`.

`multipart/form-data`

> Uses the `FormData` API to manage the data, allowing for files to be submitted to the server. You *must* use this encoding type if your form includes any `<input>` elements of `type` `file` (`<input type="file">`).

`text/plain`

Plain text; mostly useful only for debugging, so you can easily see the data that's to be submitted.

If specified, the value of the `formenctype` attribute overrides the owning form's `enctype` attribute.

This attribute is also available on `<input type="image">` and `<button>` elements.

### formmethod

A string indicating the HTTP method to use when submitting the form's data; this value overrides any `method` attribute given on the owning form. Permitted values are:

`get`

A URL is constructed by starting with the URL given by the `formaction` or `action` attribute, appending a question mark ("?") character, then appending the form's data, encoded as described by `formenctype` or the form's `enctype` attribute. This URL is then sent to the server using an HTTP `get` request. This method works well for forms that contain only ASCII characters and have no side effects. This is the default value.

`post`

The form's data is included in the body of the request that is sent to the URL given by the `formaction` or `action` attribute using an HTTP `post` method. This method supports complex data and file attachments.

`dialog`

This method is used to indicate that the button closes the dialog with which the input is associated, and does not transmit the form data at all.

This attribute is also available on `<input type="image">` and `<button>` elements.

### formnovalidate

A Boolean attribute which, if present, specifies that the form should not be validated before submission to the server. This overrides the value of the `novalidate` attribute on the element's owning form.

This attribute is also available on `<input type="image">` and `<button>` elements.

### formtarget

A string which specifies a name or keyword that indicates where to display the response received after submitting the form. The string must be the name of a **browsing context** (that is, a tab, window, or `<iframe>`). A value specified here overrides any target given by the `target` attribute on the `<form>` that owns this input.

In addition to the actual names of tabs, windows, or inline frames, there are a few special keywords that can be used:

`_self`

Loads the response into the same browsing context as the one that contains the form. This will replace the current document with the received data. This is the default value used if none is specified.

`_blank`

Loads the response into a new, unnamed, browsing context. This is typically a new tab in the same window as the current document, but may differ depending on the configuration of the user agent.

`_parent`

Loads the response into the parent browsing context of the current one. If there is no parent context, this behaves the same as `_self`.

`_top`

Loads the response into the top-level browsing context; this is the browsing context that is the topmost ancestor of the current context. If the current context is the topmost context, this behaves the same as `_self`.

This attribute is also available on `<input type="image">` and `<button>` elements.

### Using submit buttons

`<input type="submit">` buttons are used to submit forms. If you want to create a custom button and then customize the behavior using JavaScript, you need to use `<input type="button">`, or better still, a `<button>` element.

If you choose to use `<button>` elements to create the buttons in your form, keep this in mind: If the `<button>` is inside a `<form>`, that button will be treated as the "submit" button. So you should be in the habit of expressly specifying which button is the submit button.

### A basic submit button

We'll begin by creating a form with a basic submit button:

```
<form>
  <div>
    <label for="example">Let's submit some text</label>
    <input id="example" type="text" name="text" />
  </div>
  <div>
    <input type="submit" value="Send" />
  </div>
</form>
```

This renders like so:

Try entering some text into the text field, and then submitting the form.

Upon submitting, the data name/value pair gets sent to the server. In this instance, the string will be `text=user-text`, where "user-text" is the text entered by the user, encoded to preserve special characters. Where and how the data is submitted depends on the configuration of the `<form>`; see Sending form data for more details.

### Adding a keyboard shortcut to a submit button

Keyboard shortcuts, also known as access keys and keyboard equivalents, let the user trigger a button using a key or combination of keys on the keyboard. To add a keyboard shortcut to a submit button — just as you would with any `<input>` for which it makes sense — you use the `accesskey` global attribute.

In this example, `s` is specified as the access key (you'll need to press `s` plus the particular modifier keys for your browser/OS combination). In order to avoid conflicts with the user agent's own keyboard shortcuts, different modifier keys are used for access keys than for other shortcuts on the host computer. See `accesskey` for further details.

Here's the previous example with the `s` access key added:

```
<form>
  <div>
    <label for="example">Let's submit some text</label>
    <input id="example" type="text" name="text" />
  </div>
  <div>
    <input type="submit" value="Send" accesskey="s" />
  </div>
</form>
```

For example, in Firefox for Mac, pressing `Control-Option-S` triggers the Send button, while Chrome on Windows uses `Alt+S`.

The problem with the above example is that the user will not know what the access key is! This is especially true since the modifiers are typically non-standard to avoid conflicts. When building a site, be sure to provide this information in a way that doesn't interfere with the site design (for example by providing an easily accessible link that points to information on what the site access keys are). Adding a tooltip to the button (using the `title` attribute) can also help, although it's not a complete solution for accessibility purposes.

### Disabling and enabling a submit button

To disable a submit button, specify the `disabled` attribute on it, like so:

```
<input type="submit" value="Send" disabled />
```

You can enable and disable buttons at run time by setting `disabled` to `true` or `false`; in JavaScript this looks like `btn.disabled = true` or `btn.disabled = false`.

> **Note:** See the `<input type="button">` page for more ideas about enabling and disabling buttons.

### Validation

Submit buttons don't participate in constraint validation; they have no real value to be constrained.

### Examples

We've included basic examples above. There isn't really anything more to say about submit buttons. There's a reason this kind of control is sometimes called a "simple button."

### Technical Summary

| | |
|---|---|
| **Value** | A string used as the button's label |
| **Events** | `click` |
| **Supported common attributes** | `type` and `value` |
| **IDL attributes** | `value` |
| **DOM interface** | `HTMLInputElement` |
| **Implicit ARIA Role** | `button` |

### See also

- `<input>` and the `HTMLInputElement` interface which implements it.
- Forms and buttons
- HTML forms
- The `<button>` element

## <input type="tel"> HTML attribute value

`<input>` elements of type **`tel`** are used to let the user enter and edit a telephone number. Unlike `<input type="email">` and `<input type="url">`, the input value is not automatically validated to a particular format before the form can be submitted, because formats for telephone numbers vary so much around the world.

### Try it

```
<label for="phone">
  Enter your phone number:<br />
  <small>Format: 123-456-7890</small>
</label>

<input
  type="tel"
  id="phone"
  name="phone"
  pattern="[0-9]{3}-[0-9]{3}-[0-9]{4}"
  required />
```

```
label {
  display: block;
  font:
    1rem "Fira Sans",
    sans-serif;
}

input,
label {
  margin: 0.4rem 0;
}
```

### Value

The `<input>` element's `value` attribute contains a string that either represents a telephone number or is an empty string (`""`).

### Additional attributes

In addition to the global attributes and the attributes that operate on all `<input>` elements regardless of their type, telephone number inputs support the following attributes.

#### list

The values of the list attribute is the `id` of a `<datalist>` element located in the same document. The `<datalist>` provides a list of predefined values to suggest to the user for this input. Any values in the list that are not compatible with the `type` are not included in the suggested options. The values provided are suggestions, not requirements: users can select from this predefined list or provide a different value.

#### maxlength

The maximum string length (measured in UTF-16 code units) that the user can enter into the telephone number field. This must be an integer value of 0 or higher. If no `maxlength` is specified, or an invalid value is specified, the telephone number field has no maximum length. This value must also be greater than or equal to the value of `minlength`.

The input will fail constraint validation if the length of the text entered into the field is greater than `maxlength` UTF-16 code units long. Constraint validation is only applied when the value is changed by the user.

#### minlength

The minimum string length (measured in UTF-16 code units) that the user can enter into the telephone number field. This must be a non-negative integer value smaller than or equal to the value specified by `maxlength`. If no `minlength` is specified, or an invalid value is specified, the telephone number input has no minimum length.

The telephone number field will fail constraint validation if the length of the text entered into the field is fewer than `minlength` UTF-16 code units long. Constraint validation is only applied when the value is changed by the user.

#### pattern

The `pattern` attribute, when specified, is a regular expression that the input's `value` must match for the value to pass constraint validation. It must be a valid JavaScript regular expression, as used by the `RegExp` type, and as documented in our guide on regular expressions; the `'u'` flag is specified when compiling the regular expression so that the pattern is treated as a sequence of Unicode code points, instead of as ASCII. No forward slashes should be specified around the pattern text.

If the specified pattern is not specified or is invalid, no regular expression is applied and this attribute is ignored completely.

> **Note:** Use the `title` attribute to specify text that most browsers will display as a tooltip to explain what the requirements are to match the pattern. You should also include other explanatory text nearby.

See Pattern validation below for details and an example.

#### placeholder

The `placeholder` attribute is a string that provides a brief hint to the user as to what kind of information is expected in the field. It should be a word or short phrase that demonstrates the expected type of data, rather than an explanatory message. The text *must not* include carriage returns or line feeds.

If the control's content has one directionality (LTR or RTL) but needs to present the placeholder in the opposite directionality, you can use Unicode bidirectional algorithm formatting characters to override directionality within the placeholder; see How to use Unicode controls for bidi text for more information.

> **Note:** Avoid using the `placeholder` attribute if you can. It is not as semantically useful as other ways to explain your form, and can cause unexpected technical issues with your content. See `<input>` labels for more information.

#### readonly

A Boolean attribute which, if present, means this field cannot be edited by the user. Its `value` can, however, still be changed by JavaScript code directly setting the `HTMLInputElement` `value` property.

> **Note:** Because a read-only field cannot have a value, `required` does not have any effect on inputs with the `readonly` attribute also specified.

#### size

The `size` attribute is a numeric value indicating how many characters wide the input field should be. The value must be a number greater than zero, and the default value is 20. Since character widths vary, this may or may not be exact and should not be relied upon to be so; the resulting input may be narrower or wider than the specified number of characters, depending on the characters and the font (`font` settings in use).

This does *not* set a limit on how many characters the user can enter into the field. It only specifies approximately how many can be seen at a time. To set an upper limit on the length of the input data, use the `maxlength` attribute.

### Using tel inputs

Despite the fact that inputs of type `tel` are functionally identical to standard `text` inputs, they do serve useful purposes; the most quickly apparent of these is that mobile browsers — especially on mobile phones — may opt to present a custom keypad optimized for entering phone numbers. Using a specific input type for telephone numbers also makes adding custom validation and handling of phone numbers more convenient.

> **Note:** Browsers that don't support type `tel` fall back to being a standard text input.

Telephone numbers are a very commonly collected type of data on the web. When creating any kind of registration or e-commerce site, for example, you will likely need to ask the user for a telephone number, whether for business purposes or for emergency contact purposes. Given how commonly-entered phone numbers are, it's unfortunate that a "one size fits all" solution for validating phone numbers is not practical.

Fortunately, you can consider the requirements of your own site and implement an appropriate level of validation yourself. See Validation, below, for details.

### Custom keyboards

One of the main advantages of `<input type="tel">` is that it causes mobile browsers to display a special keyboard for entering phone numbers. For example, here's what the keypads look like on a couple of devices.

| Firefox for Android | WebKit iOS (Safari/Chrome/Firefox) |
| --- | --- |
|  Firefox for Android screen shot |  Firefox for iOS screenshot |

### A basic tel input

In its most basic form, a tel input can be implemented like this:

```
<label for="telNo">Phone number:</label>
<input id="telNo" name="telNo" type="tel" />
```

There is nothing magical going on here. When submitted to the server, the above input's data would be represented as, for example, `telNo=+12125553151`.

### Placeholders

Sometimes it's helpful to offer an in-context hint as to what form the input data should take. This can be especially important if the page design doesn't offer descriptive labels for each `<input>`. This is where **placeholders** come in. A placeholder is a value that demonstrates the form the `value` should take by presenting an example of a valid value, which is displayed inside the edit box when the element's `value` is `""`. Once data is entered into the box, the placeholder disappears; if the box is emptied, the placeholder reappears.

Here, we have a `tel` input with the placeholder `123-4567-8901`. Note how the placeholder disappears and reappears as you manipulate the contents of the edit field.

```
<input id="telNo" name="telNo" type="tel" placeholder="123-4567-8901" />
```

### Controlling the input size

You can control not only the physical length of the input box, but also the minimum and maximum lengths allowed for the input text itself.

### Physical input element size

The physical size of the input box can be controlled using the `size` attribute. With it, you can specify the number of characters the input box can display at a time. In this example, for instance, the `tel` edit box is 20 characters wide:

```
<input id="telNo" name="telNo" type="tel" size="20" />
```

### Element value length

The `size` is separate from the length limitation on the entered telephone number. You can specify a minimum length, in characters, for the entered telephone number using the `minlength` attribute; similarly, use `maxlength` to set the maximum length of the entered telephone number.

The example below creates a 20-character wide telephone number entry box, requiring that the contents be no shorter than 9 characters and no longer than 14 characters.

```
<input
  id="telNo"
  name="telNo"
  type="tel"
  size="20"
  minlength="9"
  maxlength="14" />
```

> **Note:** The above attributes do affect Validation — the above example's inputs will count as invalid if the length of the value is less than 9 characters, or more than 14. Most browser won't even let you enter a value over the max length.

### Providing default options

### Providing a single default using the value attribute

As always, you can provide a default value for a `tel` input box by setting its `value` attribute:

```
<input id="telNo" name="telNo" type="tel" value="333-4444-4444" />
```

### Offering suggested values

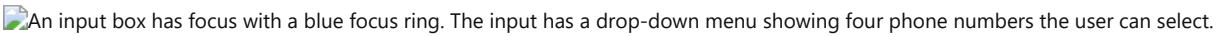
Taking it a step further, you can provide a list of default phone number values from which the user can select. To do this, use the `list` attribute. This doesn't limit the user to those options, but does allow them to select commonly-used telephone numbers more quickly. This also offers hints to `autocomplete`. The `list` attribute specifies the ID of a `<datalist>` element, which in turn contains one `<option>` element per suggested value; each `option`'s `value` is the corresponding suggested value for the telephone number entry box.

```
<label for="telNo">Phone number: </label>
<input id="telNo" name="telNo" type="tel" list="defaultTels" />

<datalist id="defaultTels">
  <option value="111-1111-1111"></option>
  <option value="122-2222-2222"></option>
  <option value="333-3333-3333"></option>
  <option value="344-4444-4444"></option>
</datalist>
```

With the `<datalist>` element and its `<option>`s in place, the browser will offer the specified values as potential values for the phone number; this is typically presented as a popup or drop-down menu containing the suggestions. While the specific user experience may vary from one browser to another, typically clicking in the edit box presents a drop-down of the suggested phone numbers. Then, as the user types, the list is adjusted to show only filtered matching values. Each typed character narrows down the list until the user makes a selection or types a custom value.

Here's a screenshot of what that might look like:


An input box has focus with a blue focus ring. The input has a drop-down menu showing four phone numbers the user can select.

### Validation

As we've touched on before, it's quite difficult to provide a one-size-fits-all client-side validation solution for phone numbers. So what can we do? Let's consider some options.

> **Warning:** HTML form validation is *not* a substitute for server-side scripts that ensure the entered data is in the proper format before it is allowed into the database. It's far too easy for someone to make adjustments to the HTML that allow them to bypass the validation, or to remove it entirely. It's also possible for someone to bypass your HTML entirely and submit the data directly to your server. If your server-side code fails to validate the data it receives, disaster could strike when improperly-formatted data (or data which is too large, is of the wrong type, and so forth) is entered into your database.

### Making telephone numbers required

You can make it so that an empty input is invalid and won't be submitted to the server using the `required` attribute. For example, let's use this HTML:

```
<form>
  <div>
    <label for="telNo">Enter a telephone number (required): </label>
    <input id="telNo" name="telNo" type="tel" required />
    <span class="validity"></span>
  </div>
  <div>
    <button>Submit</button>
  </div>
</form>
```

And let's include the following CSS to highlight valid entries with a checkmark and invalid entries with a cross:

```
div {
  margin-bottom: 10px;
  position: relative;
}

input[type="number"] {
  width: 100px;
}

input + span {
  padding-right: 30px;
}

input:invalid + span::after {
  position: absolute;
  content: "✖";
  padding-left: 5px;
  color: darkred;
}
```

```
input:valid + span::after {
  position: absolute;
  content: "✓";
  padding-left: 5px;
  color: #009000;
}
```

The output looks like this:

### Pattern validation

If you want to further restrict entered numbers so they also have to conform to a specific pattern, you can use the `pattern` attribute, which takes as its value a regular expression that entered values have to match.

In this example we'll use the same CSS as before, but our HTML is changed to look like this:

```
<form>
  <div>
    <label for="telNo">
      Enter a telephone number (in the form xxx-xxx-xxxx):
    </label>
    <input
      id="telNo"
      name="telNo"
      type="tel"
      required
      pattern="[0-9]{3}-[0-9]{3}-[0-9]{4}" />
    <span class="validity"></span>
  </div>
  <div>
    <button>Submit</button>
  </div>
</form>
```

Notice how the entered value is reported as invalid unless the pattern xxx-xxx-xxxx is matched; for instance, 41-323-421 won't be accepted. Neither will 800-MDN-ROCKS. However, 865-555-6502 will be accepted. This particular pattern is obviously only useful for certain locales — in a real application you'd probably have to vary the pattern used depending on the locale of the user.

### Examples

In this example, we present a `<select>` element that lets the user choose which country they're in, and a set of `<input type="tel">` elements to let them enter each part of their phone number; there is no reason why you can't have multiple `tel` inputs.

Each input has a `placeholder` attribute to show a hint to sighted users about what to enter into it, a `pattern` to enforce a specific number of characters for the desired section, and an `aria-label` attribute to contain a hint to be read out to screen reader users about what to enter into it.

```
<div>
  <label for="country">Choose your country:</label>
  <select id="country" name="country">
    <option>UK</option>
    <option selected>US</option>
    <option>Germany</option>
  </select>
</div>
<div>
  <p>Enter your telephone number:</p>
  <span class="areaDiv">
    <input
      id="areaNo"
      name="areaNo"
      type="tel"
      required
      placeholder="Area code"
      pattern="[0-9]{3}"
      aria-label="Area code" />
    <span class="validity"></span>
  </span>
  <span class="number1Div">
    <input
      id="number1"
      name="number1"
      type="tel"
      required
      placeholder="First part"
      pattern="[0-9]{3}"
      aria-label="First part of number" />
```

```
    <span class="validity"></span>
  </span>
  <span class="number2Div">
    <input
      id="number2"
      name="number2"
      type="tel"
      required
      placeholder="Second part"
      pattern="[0-9]{4}"
      aria-label="Second part of number" />
    <span class="validity"></span>
  </span>
</div>
```

The JavaScript contains an `onchange` event handler that, when the `<select>` value is changed, updates the `<input>` element's `pattern`, `placeholder`, and `aria-label` to suit the format of telephone numbers in that country/territory.

```
const selectElem = document.querySelector("select");
const inputElems = document.querySelectorAll("input");

selectElem.onchange = () => {
  for (const e of inputElems) {
    e.value = "";
  }

  if (selectElem.value === "US") {
    inputElems[2].parentNode.style.display = "inline";

    inputElems[0].placeholder = "Area code";
    inputElems[0].pattern = "[0-9]{3}";

    inputElems[1].placeholder = "First part";
    inputElems[1].pattern = "[0-9]{3}";
    inputElems[1].setAttribute("aria-label", "First part of number");

    inputElems[2].placeholder = "Second part";
    inputElems[2].pattern = "[0-9]{4}";
    inputElems[2].setAttribute("aria-label", "Second part of number");
  } else if (selectElem.value === "UK") {
    inputElems[2].parentNode.style.display = "none";

    inputElems[0].placeholder = "Area code";
    inputElems[0].pattern = "[0-9]{3,6}";

    inputElems[1].placeholder = "Local number";
    inputElems[1].pattern = "[0-9]{4,8}";
    inputElems[1].setAttribute("aria-label", "Local number");
  } else if (selectElem.value === "Germany") {
    inputElems[2].parentNode.style.display = "inline";

    inputElems[0].placeholder = "Area code";
    inputElems[0].pattern = "[0-9]{3,5}";

    inputElems[1].placeholder = "First part";
    inputElems[1].pattern = "[0-9]{2,4}";
    inputElems[1].setAttribute("aria-label", "First part of number");

    inputElems[2].placeholder = "Second part";
    inputElems[2].pattern = "[0-9]{4}";
    inputElems[2].setAttribute("aria-label", "Second part of number");
  }
};
```

The example looks like this:

This is an interesting idea, which goes to show a potential solution to the problem of dealing with international phone numbers. You would have to extend the example of course to provide the correct pattern for potentially every country, which would be a lot of work, and there would still be no foolproof guarantee that the users would enter their numbers correctly.

It makes you wonder if it is worth going to all this trouble on the client-side, when you could just let the user enter their number in whatever format they wanted on the client-side and then validate and sanitize it on the server. But this choice is yours to make.

### Technical Summary

| | |
|---|---|
| **Value** | A string representing a telephone number, or empty |

| | |
|---|---|
| **Events** | `change` and `input` |
| **Supported common attributes** | `autocomplete`, `list`, `maxlength`, `minlength`, `pattern`, `placeholder`, `readonly`, and `size` |
| **IDL attributes** | `list`, `selectionStart`, `selectionEnd`, `selectionDirection`, and `value` |
| **DOM interface** | `HTMLInputElement` |
| **Implicit ARIA Role** | with no `list` attribute: `textbox`<br>with `list` attribute: `combobox` |

### See also

- HTML forms guide
- `<input>`
  - `<input type="text">`
  - `<input type="email">`

## `<input type="text">` HTML attribute value

`<input>` elements of type **`text`** create basic single-line text fields.

### Try it

```
<label for="name">Name (4 to 8 characters):</label>

<input
  type="text"
  id="name"
  name="name"
  required
  minlength="4"
  maxlength="8"
  size="10" />
```

```
label {
  display: block;
  font:
    1rem "Fira Sans",
    sans-serif;
}

input,
label {
  margin: 0.4rem 0;
}
```

### Value

The `value` attribute is a string that contains the current value of the text entered into the text field. You can retrieve this using the `HTMLInputElement` `value` property in JavaScript.

```
let theText = myTextInput.value;
```

If no validation constraints are in place for the input (see Validation for more details), the value may be an empty string (`""`).

### Additional attributes

In addition to the global attributes and the attributes that operate on all `<input>` elements regardless of their type, text inputs support the following attributes.

#### `<code>list</code>`

The values of the list attribute is the `id` of a `<datalist>` element located in the same document. The `<datalist>` provides a list of predefined values to suggest to the user for this input. Any values in the list that are not compatible with the `type` are not included in the suggested options. The values provided are suggestions, not requirements: users can select from this predefined list or provide a different value.

#### `<code>maxlength</code>`

The maximum string length (measured in UTF-16 code units) that the user can enter into the `text` input. This must be an integer value of 0 or higher. If no `maxlength` is specified, or an invalid value is specified, the `text` input has no maximum length. This value must also be greater than or equal to the value of `minlength`.

The input will fail constraint validation if the length of the text value of the field is greater than `maxlength` UTF-16 code units long. Constraint validation is only applied when the value is changed by the user.

### <code>minlength</code>

The minimum string length (measured in UTF-16 code units) that the user can enter into the `text` input. This must be a non-negative integer value smaller than or equal to the value specified by `maxlength`. If no `minlength` is specified, or an invalid value is specified, the `text` input has no minimum length.

The input will fail constraint validation if the length of the text entered into the field is fewer than `minlength` UTF-16 code units long. Constraint validation is only applied when the value is changed by the user.

### <code>pattern</code>

The `pattern` attribute, when specified, is a regular expression that the input's `value` must match for the value to pass constraint validation. It must be a valid JavaScript regular expression, as used by the `RegExp` type, and as documented in our guide on regular expressions; the `'u'` flag is specified when compiling the regular expression so that the pattern is treated as a sequence of Unicode code points, instead of as ASCII. No forward slashes should be specified around the pattern text.

If the specified pattern is not specified or is invalid, no regular expression is applied and this attribute is ignored completely.

> **Note:** Use the `title` attribute to specify text that most browsers will display as a tooltip to explain what the requirements are to match the pattern. You should also include other explanatory text nearby.

See Specifying a pattern for further details and an example.

### <code>placeholder</code>

The `placeholder` attribute is a string that provides a brief hint to the user as to what kind of information is expected in the field. It should be a word or short phrase that demonstrates the expected type of data, rather than an explanatory message. The text *must not* include carriage returns or line feeds.

If the control's content has one directionality (LTR or RTL) but needs to present the placeholder in the opposite directionality, you can use Unicode bidirectional algorithm formatting characters to override directionality within the placeholder; see How to use Unicode controls for bidi text for more information.

> **Note:** Avoid using the `placeholder` attribute if you can. It is not as semantically useful as other ways to explain your form, and can cause unexpected technical issues with your content. See `<input>` accessibility concerns for more information.

### <code>readonly</code>

A Boolean attribute which, if present, means this field cannot be edited by the user. Its `value` can, however, still be changed by JavaScript code directly setting the `HTMLInputElement` `value` property.

> **Note:** Because a read-only field cannot have a value, `required` does not have any effect on inputs with the `readonly` attribute also specified.

### <code>size</code>

The `size` attribute is a numeric value indicating how many characters wide the input field should be. The value must be a number greater than zero, and the default value is 20. Since character widths vary, this may or may not be exact and should not be relied upon to be so; the resulting input may be narrower or wider than the specified number of characters, depending on the characters and the font (`font` settings in use).

This does *not* set a limit on how many characters the user can enter into the field. It only specifies approximately how many can be seen at a time. To set an upper limit on the length of the input data, use the `maxlength` attribute.

### <code>spellcheck</code>

The `spellcheck` global attribute is used to indicate whether to enable spell-checking for an element. It can be used on any editable content, but here we consider specifics related to the use of `spellcheck` on `<input>` elements. The permitted values for `spellcheck` are:

`false`

: Disable spell-checking for this element.

`true`

: Enable spell-checking for this element.

`""` (empty string) or no value

: Follow the element's default behavior for spell-checking. This may be based upon a parent's `spellcheck` setting or other factors.

An input field can have spell-checking enabled if it doesn't have the readonly attribute set and is not disabled.

The value returned by reading `spellcheck` may not reflect the actual state of spell-checking within a control, if the user agent's preferences override the setting.

### Using text inputs

`<input>` elements of type `text` create basic, single-line inputs. You should use them anywhere you want the user to enter a single-line value and there isn't a more specific input type available for collecting that value (for example, if it's a date, URL, email, or search term, you've got better options available).

### Basic example

```
<div>
  <label for="uname">Choose a username: </label>
  <input type="text" id="uname" name="name" />
</div>
```

This renders like so:

If this input is included in a form and submitted, the data name/value pair sent to the server will be `name=Chris` (if "Chris" was entered as the input value before submission). You must remember to include `name` attribute on the `<input>` element, otherwise the text field's value won't be included with the submitted data.

### Setting placeholders

You can provide a useful placeholder inside your text input that can provide a hint as to what to enter by including using the `placeholder` attribute. Look at the following example:

```
<div>
  <label for="uname">Choose a username: </label>
  <input
    type="text"
    id="uname"
    name="name"
    placeholder="Lower case, all one word" />
</div>
```

You can see how the placeholder is rendered below:

The placeholder is typically rendered in a lighter color than the element's foreground color, and automatically vanishes when the user begins to enter text into the field (or whenever the field has a value set programmatically by setting its `value` attribute).

### Physical input element size

The physical size of the input box can be controlled using the `size` attribute. With it, you can specify the number of characters the text input can display at a time. This affects the width of the element, letting you specify the width in terms of characters rather than pixels. In this example, for instance, the input is 30 characters wide:

```
<div>
  <label for="uname">Choose a username: </label>
  <input
    type="text"
    id="uname"
    name="name"
    placeholder="Lower case, all one word"
    size="30" />
</div>
```

### Validation

`<input>` elements of type `text` have no automatic validation applied to them (since a basic text input needs to be capable of accepting any arbitrary string), but there are some client-side validation options available, which we'll discuss below.

> **Note:** HTML form validation is *not* a substitute for server-scripts that ensure the entered data is in the proper format. It's far too easy for someone to make adjustments to the HTML that allow them to bypass the validation, or to remove it entirely. It's also possible for someone to bypass your HTML entirely and submit the data directly to your server. If your server-side code fails to validate the data it receives, disaster could strike when improperly-formatted data (or data which is too large, is of the wrong type, and so forth) is entered into your database.

### A note on styling

There are useful pseudo-classes available for styling form elements to help the user see when their values are valid or invalid. These are `:valid` and `:invalid`. In this section, we'll use the following CSS, which will place a check (tick) mark next to inputs containing valid values, and a cross (X) next to inputs containing invalid values.

```
div {
  margin-bottom: 10px;
  position: relative;
}

input + span {
  padding-right: 30px;
}

input:invalid + span::after {
  position: absolute;
  content: "✕";
  padding-left: 5px;
}

input:valid + span::after {
  position: absolute;
  content: "✓";
  padding-left: 5px;
}
```

The technique also requires a `<span>` element to be placed after the form element, which acts as a holder for the icons. This was necessary because some input types on some browsers don't display icons placed directly after them very well.

### Making input required

You can use the `required` attribute as an easy way of making entering a value required before form submission is allowed:

```
<form>
  <div>
    <label for="uname">Choose a username: </label>
    <input type="text" id="uname" name="name" required />
    <span class="validity"></span>
  </div>
  <div>
    <button>Submit</button>
  </div>
</form>
```

This renders like so:

If you try to submit the form with no username entered into it, the browser will show an error message.

### Input value length

You can specify a minimum length (in characters) for the entered value using the `minlength` attribute; similarly, use `maxlength` to set the maximum length of the entered value, in characters.

The example below requires that the entered value be 4–8 characters in length.

```
<form>
  <div>
    <label for="uname">Choose a username: </label>
    <input
      type="text"
      id="uname"
      name="name"
      required
      size="10"
      placeholder="Username"
      minlength="4"
      maxlength="8" />
    <span class="validity"></span>
  </div>
  <div>
    <button>Submit</button>
  </div>
</form>
```

This renders like so:

If you try to submit the form with less than 4 characters, you'll be given an appropriate error message (which differs between browsers). If you try to enter more than 8 characters, the browser won't let you.

> **Note:** If you specify a `minlength` but do not specify `required`, the input is considered valid, since the user is not required to specify a value.

### Specifying a pattern

You can use the `pattern` attribute to specify a regular expression that the inputted value must match in order to be considered valid (see Validating against a regular expression for a crash course on using regular expressions to validate inputs).

The example below restricts the value to 4-8 characters and requires that it contain only lower-case letters.

```
<form>
  <div>
    <label for="uname">Choose a username: </label>
    <input
      type="text"
      id="uname"
      name="name"
      required
      size="45"
      pattern="[a-z]{4,8}" />
    <span class="validity"></span>
    <p>Usernames must be lowercase and 4-8 characters in length.</p>
  </div>
  <div>
    <button>Submit</button>
  </div>
</form>
```

This renders like so:

### Examples

You can see good examples of text inputs used in context in our Your first HTML form and How to structure an HTML form articles.

### Technical summary

| | |
|---|---|
| **Value** | A string representing the text contained in the text field. |
| **Events** | `change` and `input` |
| **Supported Common Attributes** | `autocomplete`, `list`, `maxlength`, `minlength`, `pattern`, `placeholder`, `readonly`, `required` and `size` |
| **IDL attributes** | `list`, `value` |
| **DOM interface** | `HTMLInputElement` |
| **Implicit ARIA Role** | with no `list` attribute: `textbox`<br>with `list` attribute: `combobox` |

### See also

- HTML Forms
- `<input>` and the `HTMLInputElement` interface it's based upon.
- `<input type="search">`
- `<textarea>`: Multi-line text input

## <input type="time"> HTML attribute value

`<input>` elements of type **`time`** create input fields designed to let the user easily enter a time (hours and minutes, and optionally seconds).

While the control's user interface appearance is based on the browser and operating system, the features are the same. The value is always a 24-hour `HH:mm` or `HH:mm:ss` formatted time, with leading zeros, regardless of the UI's input format.

### Try it

```
<label for="appointment">Choose a time for your meeting:</label>

<input
  type="time"
  id="appointment"
  name="appointment"
  min="09:00"
  max="18:00"
```

```
  required />

<small>Office hours are 9am to 6pm</small>
```

```
label {
  display: block;
  font:
    1rem "Fira Sans",
    sans-serif;
}

input,
label {
  margin: 0.4rem 0;
}
```

### Additional attributes

In addition to the attributes common to all `<input>` elements, `time` inputs offer the following attributes.

> **Note:** Unlike many data types, time values have a **periodic domain**, meaning that the values reach the highest possible value, then wrap back around to the beginning again. For example, specifying a `min` of `14:00` and a `max` of `2:00` means that the permitted time values start at 2:00 PM, run through midnight to the next day, ending at 2:00 AM. See more in the making min and max cross midnight section of this article.

#### list

The values of the list attribute is the `id` of a `<datalist>` element located in the same document. The `<datalist>` provides a list of predefined values to suggest to the user for this input. Any values in the list that are not compatible with the `type` are not included in the suggested options. The values provided are suggestions, not requirements: users can select from this predefined list or provide a different value.

#### max

A string indicating the latest time to accept, specified in the same time value format as described above. If the specified string isn't a valid time, no maximum value is set.

#### min

A string specifying the earliest time to accept, given in the time value format described previously. If the value specified isn't a valid time string, no minimum value is set.

#### readonly

A Boolean attribute which, if present, means this field cannot be edited by the user. Its `value` can, however, still be changed by JavaScript code directly setting the `HTMLInputElement` `value` property.

> **Note:** Because a read-only field cannot have a value, `required` does not have any effect on inputs with the `readonly` attribute also specified.

#### step

The `step` attribute is a number that specifies the granularity that the value must adhere to, or the special value `any`, which is described below. Only values which are a whole number of steps from the step base are valid. The step base is `min` if specified, `value` otherwise, or `0` (`00:00:00`) if neither is provided.

For `time` inputs, the value of `step` is given in seconds and is treated as a number of milliseconds equal to 1000 times the `step` value (the underlying numeric value is in milliseconds). The default value is 60, indicating 1 minute.

A string value of `any` means that no stepping is implied, and any value is allowed (barring other constraints, such as `min` and `max`). In reality, it has the same effect as `60` for `time` inputs because the picker UI in this case only allows selecting whole minutes.

> **Note:** When the data entered by the user doesn't adhere to the stepping configuration, the user agent may round to the nearest valid value, preferring numbers in the positive direction when there are two equally close options.

### Validation

By default, `<input type="time">` does not apply any validation to entered values, other than the user agent's interface generally not allowing you to enter anything other than a time value. This is helpful, but you can't entirely rely on the value to be a proper time string, since it might be an empty string (`""`), which is allowed. For examples of constraint validation using the `min`, `max`, `step`, and `required` attributes, see the setting maximum and minimum times section.

## Examples

### Basic uses of time

The most basic use of `<input type="time">` involves a basic `<input>` and `<label>` element combination, as seen below:

```
<label for="appointment-time">Choose an appointment time: </label>
<input id="appointment-time" type="time" name="appointment-time" />
```

### Creating a time picker interface

In this example, we create an interface element for choosing time using the native picker created with `<input type="time">`:

```
<label for="appointment-time">
  Choose an appointment time (opening hours 12:00 to 18:00):
</label>
<input
  id="appointment-time"
  type="time"
  name="appointment-time"
  min="12:00"
  max="18:00"
  required />
<span class="validity"></span>
```

```
input[type="time"] {
  width: 100px;
}

input + span {
  padding-right: 30px;
}

input:invalid + span::after {
  position: absolute;
  content: "✖";
  padding-left: 5px;
}

input:valid + span::after {
  position: absolute;
  content: "✓";
  padding-left: 5px;
}
```

### Controlling input size

`<input type="time">` doesn't support form sizing attributes such as `size`, since times are always about the same number of characters long. You'll have to resort to CSS for sizing needs.

### Setting the value attribute

You can set a default value for the input by including a valid time in the `value` attribute when creating the `<input>` element, like so:

```
<label for="appointment-time">Choose an appointment time: </label>
<input
  id="appointment-time"
  type="time"
  name="appointment-time"
  value="13:30" />
```

### Setting the value using JavaScript

You can also get and set the time value in JavaScript using the `HTMLInputElement` `value` property, for example:

```
const timeControl = document.querySelector('input[type="time"]');
timeControl.value = "15:30";
```

### Time value format

The `value` of the `time` input is always in 24-hour format that includes leading zeros: `HH:mm`, regardless of the input format, which is likely to be selected based on the user's locale (or by the user agent). If the time includes seconds (see Using the step attribute), the format is always `HH:mm:ss`. You can learn more about the format of the time value used by this input type in Time strings.

In this example, you can see the time input's value by entering a time and seeing how it changes afterward.

First, a look at the HTML. We include a label and input, and add a `<p>` element with a `<span>` to display the value of the `time` input:

```
<label for="startTime">Start time: </label>
<input type="time" id="startTime" />
<p>
  Value of the <code>time</code> input:
  <code>"<span id="value">n/a</span>"</code>.
</p>
```

The JavaScript code adds code to the time input to watch for the `input` event, which is triggered every time the contents of an input element change. When this happens, the contents of the `<span>` are replaced with the new value of the input element.

```
const startTime = document.getElementById("startTime");
const valueSpan = document.getElementById("value");

startTime.addEventListener("input", () => {
  valueSpan.innerText = startTime.value;
});
```

When a form including a `time` input is submitted, the value is encoded before being included in the form's data. The form's data entry for a time input will always be in the form `name=HH%3Amm`, or `name=HH%3Amm%3Ass` if seconds are included (see Using the step attribute).

### Using the step attribute

You can use the `step` attribute to vary the amount of time jumped whenever the time is incremented or decremented (for example, so the time moves by 10 minutes at a time when clicking the little arrow widgets).

It takes an integer value defining the number of seconds you want to increment by; the default value is 60 seconds. With this as the default, most user agent time UIs display hours and minutes but not seconds. Including the `step` attribute with any numeric value other than a value divisible by `60` adds seconds to the UI, if the `min` or `max` value has not already caused the seconds to be visible.

```
<label for="appointment-time">Choose an appointment time: </label>
<input id="appointment-time" type="time" name="appointment-time" step="2" />
```

To specify minutes or hours as a step, specify the number of minutes or hours in seconds, such as 120 for 2 minutes, or 7200 for 2 hours.

### Setting maximum and minimum times

You can use the `min` and `max` attributes to restrict the valid times that can be chosen by the user. In the following example we are setting a minimum time of `12:00` and a maximum time of `18:00`:

```
<label for="appointment-time">
  Choose an appointment time (opening hours 12:00 to 18:00):
</label>
<input
  id="appointment-time"
  type="time"
  name="appointment-time"
  min="12:00"
  max="18:00" />
<span class="validity"></span>
```

Here's the CSS used in the above example. Here we make use of the `:valid` and `:invalid` CSS properties to style the input based on whether the current value is valid. We add an icon as generated content icon on a `<span>` next to the input.

```
div {
  margin-bottom: 10px;
  position: relative;
}

input[type="number"] {
  width: 100px;
}

input + span {
  padding-right: 30px;
}

input:invalid + span::after {
  position: absolute;
  content: "✖";
  padding-left: 5px;
```

```
}

input:valid + span::after {
  position: absolute;
  content: "✓";
  padding-left: 5px;
}
```

The result here is that:

- Only times between 12:00 and 18:00 will be seen as valid; times outside that range will be denoted as invalid.

### Making min and max cross midnight

By setting a `min` attribute greater than the `max` attribute, the valid time range will wrap around midnight to produce a valid time range. This functionality is not supported by any other input types.

```
const input = document.createElement("input");
input.type = "time";
input.min = "23:00";
input.max = "01:00";
input.value = "23:59";

if (input.validity.valid && input.type === "time") {
  // <input type=time> reversed range supported
} else {
  // <input type=time> reversed range unsupported
}
```

### Making times required

In addition, you can use the `required` attribute to make filling in the time mandatory. Browsers will display an error if you try to submit a time that is outside the set bounds, or an empty time field.

Let's look at an example; here we've set minimum and maximum times, and also made the field required:

```
<form>
  <div>
    <label for="appointment-time">
      Choose an appointment time (opening hours 12:00 to 18:00):
    </label>
    <input
      id="appointment-time"
      type="time"
      name="appointment-time"
      min="12:00"
      max="18:00"
      required />
    <span class="validity"></span>
  </div>
  <div>
    <input type="submit" value="Submit form" />
  </div>
</form>
```

If you try to submit the form with an incomplete time (or with a time outside the set bounds), the browser displays an error. Try playing with the example now:

> **Warning:** HTML form validation is *not* a substitute for scripts that ensure that the entered data is in the proper format. It's far too easy for someone to make adjustments to the HTML that allow them to bypass the validation, or to remove it entirely. It's also possible for someone to bypass your HTML entirely and submit the data directly to your server. If your server-side code fails to validate the data it receives, disaster could strike when improperly-formatted data is submitted (or data which is too large, of the wrong type, and so forth).

### Technical Summary

| | |
|---|---|
| **Value** | A string representing a time, or empty. |
| **Events** | `change` and `input` |
| **Supported common attributes** | `autocomplete`, `list`, `readonly`, `step` |
| **IDL attributes** | `list`, `value`, `valueAsDate`, `valueAsNumber` |
| **DOM interface** | `HTMLInputElement` |

| Implicit ARIA Role | no corresponding role |
| --- | --- |

### See also

- `<input type="date">`
- `<input type="datetime-local">`
- `<input type="week">`
- `<input type="month">`
- The generic `<input>` element and the interface used to manipulate it, `HTMLInputElement`
- Date and time formats used in HTML
- Date and Time picker tutorial

## `<input type="url">` HTML attribute value

`<input>` elements of type **`url`** are used to let the user enter and edit a URL.

The input value is automatically validated to ensure that it's either empty or a properly-formatted URL before the form can be submitted. The `:valid` and `:invalid` CSS pseudo-classes are automatically applied as appropriate to visually denote whether the current value of the field is a valid URL or not.

### Try it

```
<label for="url">Enter an https:// URL:</label>
<input
  type="url"
  name="url"
  id="url"
  placeholder="https://example.com"
  pattern="https://.*"
  size="30"
  required />
```

```
label {
  display: block;
  font:
    1rem "Fira Sans",
    sans-serif;
}

input,
label {
  margin: 0.4rem 0;
}
```

### Value

The `<input>` element's `value` attribute contains a string that is automatically validated as conforming to URL syntax. More specifically, there are two possible value formats that will pass validation:

1. An empty string ("") indicates the user did not enter a value or that the value was removed.
2. A single properly-formed absolute URL. This doesn't necessarily mean the URL address exists, but it is at least formatted correctly. An entry matching `urlscheme://rest-of-url` can be valid, even if the `urlscheme` entered doesn't exist.

See Validation for details on how URLs are validated to ensure that they're formatted properly.

### Additional attributes

In addition to the global attributes, and the attributes that operate on all `<input>` elements regardless of their type, `url` inputs support the following attributes.

> **Note:** The `autocorrect` global attribute can be added to url inputs, but the stored state is always `off`.

#### list

The values of the list attribute is the `id` of a `<datalist>` element located in the same document. The `<datalist>` provides a list of predefined values to suggest to the user for this input. Any values in the list that are not compatible with the `type` are not included in the suggested options. The values provided are suggestions, not requirements: users can select from this predefined list or provide a different value.

#### maxlength

The maximum string length (measured in UTF-16 code units) that the user can enter into the `url` input. This must be an integer value of `0` or higher. If no `maxlength` is specified, or an invalid value is specified, the `url` input has no maximum length. This value must also be greater than or equal to the value of `minlength`.

The input will fail constraint validation if the length of the text value of the field is greater than `maxlength` UTF-16 code units long. Constraint validation is only applied when the value is changed by the user.

### minlength

The minimum string length (measured in UTF-16 code units) that the user can enter into the `url` input. This must be a non-negative integer value smaller than or equal to the value specified by `maxlength`. If no `minlength` is specified, or an invalid value is specified, the `url` input has no minimum length.

The input will fail constraint validation if the length of the text entered into the field is fewer than `minlength` UTF-16 code units long. Constraint validation is only applied when the value is changed by the user.

### pattern

The `pattern` attribute, when specified, is a regular expression that the input's `value` must match for the value to pass constraint validation. It must be a valid JavaScript regular expression, as used by the `RegExp` type, and as documented in our guide on regular expressions; the `'u'` flag is specified when compiling the regular expression so that the pattern is treated as a sequence of Unicode code points, instead of as ASCII. No forward slashes should be specified around the pattern text.

If the specified pattern is not specified or is invalid, no regular expression is applied and this attribute is ignored completely.

> **Note:** Use the `title` attribute to specify text that most browsers will display as a tooltip to explain what the requirements are to match the pattern. You should also include other explanatory text nearby.

See the section Pattern validation for details and an example.

### placeholder

The `placeholder` attribute is a string that provides a brief hint to the user as to what kind of information is expected in the field. It should be a word or short phrase that demonstrates the expected type of data, rather than an explanatory message. The text *must not* include carriage returns or line feeds.

If the control's content has one directionality (LTR or RTL) but needs to present the placeholder in the opposite directionality, you can use Unicode bidirectional algorithm formatting characters to override directionality within the placeholder; see How to use Unicode controls for bidi text for more information.

> **Note:** Avoid using the `placeholder` attribute if you can. It is not as semantically useful as other ways to explain your form, and can cause unexpected technical issues with your content. See `<input>` labels for more information.

### readonly

The `readonly` Boolean attribute, if present, means this field cannot be edited by the user. Its `value` can, however, still be changed by JavaScript code directly setting the `HTMLInputElement` `value` property.

> **Note:** Because a read-only field cannot have a value, `required` does not have any effect on inputs with the `readonly` attribute also specified.

### size

The `size` attribute is a numeric value indicating how many characters wide the input field should be. The value must be a number greater than zero, and the default value is 20. Since character widths vary, this may or may not be exact and should not be relied upon to be so; the resulting input may be narrower or wider than the specified number of characters, depending on the characters and the font (`font` settings in use).

This does *not* set a limit on how many characters the user can enter into the field. It only specifies approximately how many can be seen at a time. To set an upper limit on the length of the input data, use the `maxlength` attribute.

### spellcheck

The `spellcheck` global attribute is used to indicate whether to enable spell-checking for an element. It can be used on any editable content, but here we consider specifics related to the use of `spellcheck` on `<input>` elements. The permitted values for `spellcheck` are:

`false`

Disable spell-checking for this element.

`true`

Enable spell-checking for this element.

"" (empty string) or no value

Follow the element's default behavior for spell-checking. This may be based upon a parent's `spellcheck` setting or other factors.

An input field can have spell-checking enabled if it doesn't have the readonly attribute set and is not disabled.

The value returned by reading `spellcheck` may not reflect the actual state of spell-checking within a control if the user agent's preferences override the setting.

### Using URL inputs

When you create a URL input with the proper `type` value, `url`, you get automatic validation that the entered text is at least in the correct form to potentially be a legitimate URL. This can help avoid cases in which the user mistypes their website's address, or provides an invalid one.

It's important, however, to note that this is not enough to ensure that the specified text is a URL which actually exists, corresponds to the user of the site, or is acceptable in any other way. It ensures that the value of the field is properly formatted to be a URL.

> **Note:** A user can tinker with your HTML behind the scenes, so your site *must not* use this validation for any security purposes. You *must* verify the URL on the server-side of any transaction in which the provided text may have any security implications of any kind.

### A basic URL input

This element is implemented as a standard text input field with basic validation features. In its most basic form, a URL input can be implemented like this:

```
<input id="myURL" name="myURL" type="url" />
```

Notice that it's considered valid when empty and when a single validly-formatted URL address is entered, but is otherwise not considered valid. By adding the `required` attribute, only properly-formed URLs are allowed; the input is no longer considered valid when empty.

There is nothing magical going on here. Submitting this form would cause the following data to be sent to the server: `myURL=http%3A%2F%2Fwww.example.com`. Note how characters are escaped as necessary.

### Placeholders

Sometimes it's helpful to offer an in-context hint as to what form the input data should take. This can be especially important if the page design doesn't offer descriptive labels for each `<input>`. This is where **placeholders** come in. A placeholder is a value that demonstrates the form the `value` should take by presenting an example of a valid value, which is displayed inside the edit box when the element's `value` is "". Once data is entered into the box, the placeholder disappears; if the box is emptied, the placeholder reappears.

Here, we have a `url` input with the placeholder `http://www.example.com`. Note how the placeholder disappears and reappears as you manipulate the contents of the edit field.

```
<input
  id="myURL"
  name="myURL"
  type="url"
  placeholder="http://www.example.com" />
```

### Controlling the input size

You can control both the physical length of the input box and the minimum and maximum lengths allowed for the input text.

### Physical input element size

The physical size of the input box can be controlled using the `size` attribute. With it, you can specify the number of characters the input box can display at a time. In this example, for instance, the `url` edit box is 30 characters wide:

```
<input id="myURL" name="myURL" type="url" size="30" />
```

### Element value length

The `size` is separate from the length limitation on the entered URL itself. You can specify a minimum length, in characters, for the entered URL using the `minlength` attribute; similarly, use `maxlength` to set the maximum length of the entered URL. If `maxLength` exceeds `size`, the input box's contents will scroll as needed to show the current selection or insertion point as the content is manipulated.

The example below creates a 30-character wide URL address entry box, requiring that the contents be no shorter than 10 characters and no longer than 80 characters.

```
<input
  id="myURL"
  name="myURL"
  type="url"
  size="30"
  minlength="10"
  maxlength="80" />
```

> **Note:** These attributes also affect validation; a value shorter or longer than the specified minimum/maximum lengths will be classified as invalid; in addition most browsers will refuse to let the user enter a value longer than the specified maximum length.

### Providing default options

#### Providing a single default using the value attribute

As always, you can provide a default value for a `url` input box by setting its `value` attribute:

```
<input id="myURL" name="myURL" type="url" value="http://www.example.com" />
```

#### Offering suggested values

Taking it a step further, you can provide a list of default options from which the user can select by specifying the `list` attribute. This doesn't limit the user to those options, but does allow them to select commonly-used URLs more quickly. This also offers hints to `autocomplete`. The `list` attribute specifies the ID of a `<datalist>`, which in turn contains one `<option>` element per suggested value; each `option`'s `value` is the corresponding suggested value for the URL entry box.

```
<input id="myURL" name="myURL" type="url" list="defaultURLs" />

<datalist id="defaultURLs">
  <option value="https://developer.mozilla.org/"></option>
  <option value="http://www.google.com/"></option>
  <option value="http://www.microsoft.com/"></option>
  <option value="https://www.mozilla.org/"></option>
  <option value="http://w3.org/"></option>
</datalist>
```

With the `<datalist>` element and its `<option>`s in place, the browser will offer the specified values as potential values for the URL; this is typically presented as a popup or drop-down menu containing the suggestions. While the specific user experience may vary from one browser to another, typically clicking in the edit box presents a drop-down of the suggested URLs. Then, as the user types, the list is adjusted to show only matching values. Each typed character narrows down the list until the user makes a selection or types a custom value.

#### Using labels for suggested values

You can opt to include the `label` attribute on one or all of your `<option>` elements to provide textual labels. Some browsers may display only the labels, while others may display both the label and the URL.

```
<input id="myURL" name="myURL" type="url" list="defaultURLs" />

<datalist id="defaultURLs">
  <option value="https://developer.mozilla.org/" label="MDN Web Docs"></option>
  <option value="http://www.google.com/" label="Google"></option>
  <option value="http://www.microsoft.com/" label="Microsoft"></option>
  <option value="https://www.mozilla.org/" label="Mozilla"></option>
  <option value="http://w3.org/" label="W3C"></option>
</datalist>
```

### Validation

There are two levels of content validation available for `url` inputs. First, there's the standard level of validation offered to all `<input>`s, which automatically ensures that the contents meet the requirements to be a valid URL. But there's also the option to add additional filtering to ensure that your own specialized needs are met, if you have any.

> **Warning:** HTML form validation is *not* a substitute for scripts that ensure that the entered data is in the proper format. It's far too easy for someone to make adjustments to the HTML that allow them to bypass the validation, or to remove it entirely. It's also possible for someone to

> bypass your HTML entirely and submit the data directly to your server. If your server-side code fails to validate the data it receives, disaster could strike when improperly-formatted data (or data which is too large, is of the wrong type, and so forth) is entered into your database.

### Basic validation

Browsers that support the `url` input type automatically provide validation to ensure that only text that matches the standard format for URLs is entered into the input box.

The syntax of a URL is fairly intricate. It's defined by WHATWG's URL Living Standard and is described for newcomers in our article What is a URL?

### Making a URL required

As mentioned earlier, to make a URL entry required before the form can be submitted (you can't leave the field blank), you just need to include the `required` attribute on the input.

### Pattern validation

If you need the entered URL to be restricted further than just "any string that looks like a URL," you can use the `pattern` attribute to specify a regular expression the value must match for the value to be valid.

### Examples

### URL validation

In this example, we make sure that a URL is filled in using the `required` attribute and that the URL is something on `mozilla.org` using the `pattern` attribute for illustration.

### HTML

In the `url` input, we set `pattern` to `".*\.mozilla\.org.*"`. This regular expression validates a string that has any number of characters, followed by ".mozilla.org", followed by any number of characters. Because the browser runs both the standard URL filter *and* our custom pattern against the specified text, we wind up with a validation that says "make sure this is a valid URL, and also contains `.mozilla.org`."

Note that a strict pattern like `https://developer\.mozilla\.org.*` would be more robust, but that would make the `type="url"` attribute redundant in this case.

The `title` attribute also describes the `pattern` for users with assistive technologies.

```
<form>
  <label for="myURL">
    Enter a url from this site:
    <input
      id="myURL"
      name="myURL"
      type="url"
      required
      pattern=".*\.mozilla\.org.*"
      title="URL should include mozilla.org" />
    <span class="validity"></span>
  </label>
  <button>Submit</button>
</form>
```

### CSS

The CSS gives visual clues to show the user if the content is `:valid` or `:invalid` by adding an appropriate `content` property and includes alternative text for users with assistive technologies.

```
input:focus:invalid {
  outline: 2px solid red;
}

input:focus:valid {
  outline: 2px solid green;
}

input + span {
  padding: 0 0.3rem;
}

input:invalid + span::after {
  content: "✖" / "Content is not valid";
  color: red;
}
```

```
input:valid + span::after {
  content: "✓" / "Content is valid";
  color: green;
}
```

### Result

Copy this page's url and paste it in the input field and you will see a green outline and green tick. Enter any other URL that doesn't contain **mozilla.org** or an invalid URL and you will see a red outline and red cross.

Check the Pattern validation and Using URL inputs sections for other examples.

### Technical summary

| **Value** | A string representing a URL, or empty |
|---|---|
| **Events** | `change` and `input` |
| **Supported Common Attributes** | `autocomplete`, `list`, `maxlength`, `minlength`, `pattern`, `placeholder`, `readonly`, `required` and `size` |
| **IDL attributes** | `list`, `value`, `selectionEnd`, `selectionDirection` |
| **DOM interface** | `HTMLInputElement` |
| **Implicit ARIA Role** | with no `list` attribute: `textbox`<br>with `list` attribute: `combobox` |

### See also

- HTML forms guide
- `<input>`
- `<input type="tel">`
- `<input type="email">`

## <input type="week"> HTML attribute value

`<input>` elements of type **`week`** create input fields allowing easy entry of a year plus the ISO 8601 week number during that year (i.e., week 1 to 52 or 53).

### Try it

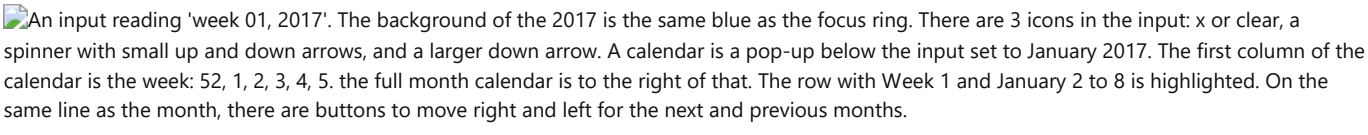
```
<label for="camp-week">Choose a week in May or June:</label>

<input
  type="week"
  name="week"
  id="camp-week"
  min="2018-W18"
  max="2018-W26"
  required />
```

```
label {
  display: block;
  font:
    1rem "Fira Sans",
    sans-serif;
}

input,
label {
  margin: 0.4rem 0;
}
```

The control's user interface varies from browser to browser; cross-browser support is currently a bit limited, with only Chrome/Opera and Microsoft Edge supporting it at this time. In non-supporting browsers, the control degrades gracefully to function identically to `<input type="text">`.


An input reading 'week 01, 2017'. The background of the 2017 is the same blue as the focus ring. There are 3 icons in the input: x or clear, a spinner with small up and down arrows, and a larger down arrow. A calendar is a pop-up below the input set to January 2017. The first column of the calendar is the week: 52, 1, 2, 3, 4, 5. the full month calendar is to the right of that. The row with Week 1 and January 2 to 8 is highlighted. On the same line as the month, there are buttons to move right and left for the next and previous months.

### Value

A string representing the value of the week/year entered into the input. The format of the date and time value used by this input type is described in Week strings.

You can set a default value for the input by including a value inside the `value` attribute, like so:

```
<label for="week">What week would you like to start?</label>
<input id="week" type="week" name="week" value="2017-W01" />
```

One thing to note is that the displayed format may differ from the actual `value`, which is always formatted `yyyy-Www`. When the above value is submitted to the server, for example, browsers may display it as `Week 01, 2017`, but the submitted value will always look like `week=2017-W01`.

You can also get and set the value in JavaScript using the input element's `value` property, for example:

```
const weekControl = document.querySelector('input[type="week"]');
weekControl.value = "2017-W45";
```

### Additional attributes

In addition to the attributes common to `<input>` elements, week inputs offer the following attributes.

#### max

The latest (time-wise) year and week number, in the string format discussed in the Value section above, to accept. If the `value` entered into the element exceeds this, the element fails constraint validation. If the value of the `max` attribute isn't a valid week string, then the element has no maximum value.

This value must be greater than or equal to the year and week specified by the `min` attribute.

#### min

The earliest year and week to accept. If the `value` of the element is less than this, the element fails constraint validation. If a value is specified for `min` that isn't a valid week string, the input has no minimum value.

This value must be less than or equal to the value of the `max` attribute.

#### readonly

A Boolean attribute which, if present, means this field cannot be edited by the user. Its `value` can, however, still be changed by JavaScript code directly setting the `HTMLInputElement` `value` property.

> **Note:** Because a read-only field cannot have a value, `required` does not have any effect on inputs with the `readonly` attribute also specified.

#### step

The `step` attribute is a number that specifies the granularity that the value must adhere to, or the special value `any`, which is described below. Only values which are a whole number of steps from the step base are valid. The step base is `min` if specified, `value` otherwise, or −259,200,000 (the start of week `1970-W01`) if neither is provided.

For `week` inputs, the value of `step` is given in weeks and is treated as a number of milliseconds equal to 604,800,000 times the `step` value (the underlying numeric value is in milliseconds). The default value is 1, indicating 1 week.

A string value of `any` means that no stepping is implied, and any value is allowed (barring other constraints, such as `min` and `max`). In reality, it has the same effect as `1` for `week` inputs because the picker UI only allows selecting whole weeks.

> **Note:** When the data entered by the user doesn't adhere to the stepping configuration, the user agent may round to the nearest valid value, preferring numbers in the positive direction when there are two equally close options.

### Using week inputs

Week inputs sound convenient at first glance, since they provide an easy UI for choosing weeks, and they normalize the data format sent to the server, regardless of the user's browser or locale. However, there are issues with `<input type="week">` because browser support is not guaranteed across all browsers.

We'll look at basic and more complex uses of `<input type="week">`, then offer advice on mitigating the browser support issue later on (see Handling browser support).

### Basic uses of week

The most basic use of `<input type="week">` involves a basic `<input>` and `<label>` element combination, as seen below:

```
<label for="week">What week would you like to start?</label>
<input id="week" type="week" name="week" />
```

### Controlling input size

`<input type="week">` doesn't support form sizing attributes such as `size`. You'll have to resort to CSS for sizing needs.

### Using the step attribute

You should be able to use the `step` attribute to vary the number of weeks jumped whenever they are incremented or decremented, however it doesn't seem to have any effect on supporting browsers.

### Validation

By default, `<input type="week">` does not apply any validation to entered values. The UI implementations generally don't let you specify anything that isn't a valid week/year, which is helpful, but it's still possible to submit with the field empty, and you might want to restrict the range of choosable weeks.

### Setting maximum and minimum weeks

You can use the `min` and `max` attributes to restrict the valid weeks that can be chosen by the user. In the following example we are setting a minimum value of `Week 01, 2017` and a maximum value of `Week 52, 2017`:

```
<label for="week">What week would you like to start?</label>
<input id="week" type="week" name="week" min="2017-W01" max="2017-W52" />
<span class="validity"></span>
```

Here's the CSS used in the above example. Here we make use of the `:valid` and `:invalid` CSS properties to style the input based on whether the current value is valid. We had to put the icons on a `<span>` next to the input, not on the input itself, because in Chrome the generated content is placed inside the form control, and can't be styled or shown effectively.

```
div {
  margin-bottom: 10px;
  position: relative;
}

input[type="number"] {
  width: 100px;
}

input + span {
  padding-right: 30px;
}

input:invalid + span::after {
  position: absolute;
  content: "✖";
  padding-left: 5px;
}

input:valid + span::after {
  position: absolute;
  content: "✓";
  padding-left: 5px;
}
```

The result here is that only weeks between W01 and W52 in 2017 will be seen as valid and be selectable in supporting browsers.

### Making week values required

In addition you can use the `required` attribute to make filling in the week mandatory. As a result, supporting browsers will display an error if you try to submit an empty week field.

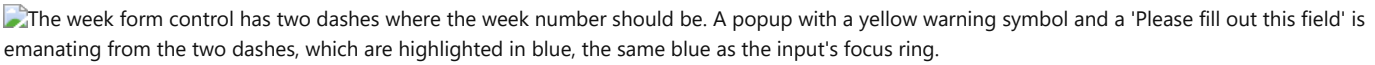
Let's look at an example; here we've set minimum and maximum weeks, and also made the field required:

```
<form>
  <div>
    <label for="week">What week would you like to start?</label>
    <input
      id="week"
      type="week"
      name="week"
      min="2017-W01"
      max="2017-W52"
      required />
    <span class="validity"></span>
  </div>
  <div>
```

```
    <input type="submit" value="Submit form" />
  </div>
</form>
```

If you try to submit the form with no value, the browser displays an error. Try playing with the example now:

Here is a screenshot for those of you who aren't using a supporting browser:

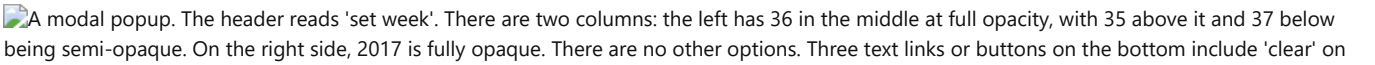

The week form control has two dashes where the week number should be. A popup with a yellow warning symbol and a 'Please fill out this field' is emanating from the two dashes, which are highlighted in blue, the same blue as the input's focus ring.

> **Warning:** HTML form validation is *not* a substitute for scripts that ensure that the entered data is in the proper format. It's far too easy for someone to make adjustments to the HTML that allow them to bypass the validation, or to remove it entirely. It's also possible for someone to bypass your HTML entirely and submit the data directly to your server. If your server-side code fails to validate the data it receives, disaster could strike when improperly-formatted data is submitted (or data which is too large, of the wrong type, and so forth).

### Handling browser support

As mentioned above, the major problem with using week inputs right now is browser support: Safari and Firefox don't support it on desktop, and old versions of IE don't support it.

Mobile platforms such as Android and iOS make perfect use of such input types, providing specialist UI controls that make it really easy to select values in a touchscreen environment. For example, the `week` picker on Chrome for Android looks like this:


A modal popup. The header reads 'set week'. There are two columns: the left has 36 in the middle at full opacity, with 35 above it and 37 below being semi-opaque. On the right side, 2017 is fully opaque. There are no other options. Three text links or buttons on the bottom include 'clear' on the 'left' and 'cancel' and 'set' on the right.

Non-supporting browsers gracefully degrade to a text input, but this creates problems both in terms of consistency of user interface (the presented control will be different), and data handling.

The second problem is the more serious. As mentioned earlier, with a `week` input the actual value is always normalized to the format `yyyy-Www`. When the browser falls back to a generic text input, there's nothing to guide the user toward correctly formatting the input (and it's certainly not intuitive). There are multiple ways in which people could write week values; for example:

- `Week 1 2017`
- `Jan 2-8 2017`
- `2017-W01`
- etc.

The best way to deal with week/years in forms in a cross-browser way at the moment is to get the user to enter the week number and year in separate controls (`<select>` elements being popular; see below for an example), or use JavaScript libraries such as jQuery date picker.

### Examples

In this example we create two sets of UI elements for choosing weeks: a native picker created using `<input type="week">`, and a set of two `<select>` elements for choosing weeks/years in older browsers that don't support the `week` input type.

The HTML looks like so:

```
<div class="nativeWeekPicker">
  <label for="week">What week would you like to start?</label>
  <input
    id="week"
    type="week"
    name="week"
    min="2017-W01"
    max="2018-W52"
    required />
  <span class="validity"></span>
</div>
<p class="fallbackLabel">What week would you like to start?</p>
<div class="fallbackWeekPicker">
  <div>
    <span>
      <label for="week">Week:</label>
      <select id="fallbackWeek" name="week"></select>
    </span>
    <span>
      <label for="year">Year:</label>
      <select id="year" name="year">
        <option value="2017" selected>2017</option>
```

```
        <option value="2018">2018</option>
      </select>
    </span>
  </div>
</div>
```

The week values are dynamically generated by the JavaScript code below.

The other part of the code that may be of interest is the feature detection code. To detect whether the browser supports `<input type="week">`, we create a new `<input>` element, try setting its `type` to `week`, then immediately check what its `type` is set to. Non-supporting browsers will return `text`, because the `week` type falls back to type `text`. If `<input type="week">` is not supported, we hide the native picker and show the fallback picker UI (`<select>`s) instead.

```
// Get UI elements
const nativePicker = document.querySelector(".nativeWeekPicker");
const fallbackPicker = document.querySelector(".fallbackWeekPicker");
const fallbackLabel = document.querySelector(".fallbackLabel");

const yearSelect = document.querySelector("#year");
const weekSelect = document.querySelector("#fallbackWeek");

// Hide fallback initially
fallbackPicker.style.display = "none";
fallbackLabel.style.display = "none";

// Test whether a new date input falls back to a text input or not
const test = document.createElement("input");

try {
  test.type = "week";
} catch (e) {
  console.log(e.description);
}

// If it does, run the code inside the if () {} block
if (test.type === "text") {
  // Hide the native picker and show the fallback
  nativePicker.style.display = "none";
  fallbackPicker.style.display = "block";
  fallbackLabel.style.display = "block";

  // populate the weeks dynamically
  populateWeeks();
}

function populateWeeks() {
  // Populate the week select with 52 weeks
  for (let i = 1; i <= 52; i++) {
    const option = document.createElement("option");
    option.textContent = i < 10 ? `0${i}` : i;
    weekSelect.appendChild(option);
  }
}
```

> **Note:** Remember that some years have 53 weeks in them (see Weeks per year)! You'll need to take this into consideration when developing production apps.

### Technical summary

| | |
|---|---|
| **Value** | A string representing a week and year, or empty |
| **Events** | `change` and `input` |
| **Supported common attributes** | `autocomplete`, `list`, `readonly`, `step` |
| **IDL attributes** | `list`, `value`, `valueAsDate`, `valueAsNumber` |
| **DOM interface** | `HTMLInputElement` |
| **Implicit ARIA Role** | no corresponding role |

### See also

- The generic `<input>` element and the interface used to manipulate it, `HTMLInputElement`
- Date and time formats used in HTML

- `<input type="datetime-local">`, `<input type="date">`, `<input type="time">`, and `<input type="month">`

### <ins> HTML inserted text element

The **`<ins>`** HTML element represents a range of text that has been added to a document. You can use the `<del>` element to similarly represent a range of text that has been deleted from the document.

#### Try it

```
<p>&ldquo;You're late!&rdquo;</p>
<del>
  <p>&ldquo;I apologize for the delay.&rdquo;</p>
</del>
<ins cite="../how-to-be-a-wizard.html" datetime="2018-05">
  <p>&ldquo;A wizard is never late &hellip;&rdquo;</p>
</ins>
```

```
del,
ins {
  display: block;
  text-decoration: none;
  position: relative;
}

del {
  background-color: #ffbbbb;
}

ins {
  background-color: #d4fcbc;
}

del::before,
ins::before {
  position: absolute;
  left: 0.5rem;
  font-family: monospace;
}

del::before {
  content: "−";
}

ins::before {
  content: "+";
}

p {
  margin: 0 1.8rem;
  font-family: "Georgia", serif;
  font-size: 1rem;
}
```

#### Attributes

This element includes the global attributes.

`cite`

> This attribute defines the URI of a resource that explains the change, such as a link to meeting minutes or a ticket in a troubleshooting system.

`datetime`

> This attribute indicates the time and date of the change and must be a valid date with an optional time string. If the value cannot be parsed as a date with an optional time string, the element does not have an associated timestamp. For the format of the string without a time, see Format of a valid date string. The format of the string if it includes both date and time is covered in Format of a valid local date and time string.

#### Accessibility

The presence of the `<ins>` element is not announced by most screen reading technology in its default configuration. It can be made to be announced by using the CSS `content` property, along with the `::before` and `::after` pseudo-elements.

```
ins::before,
ins::after {
```

```
  clip-path: inset(100%);
  clip: rect(1px, 1px, 1px, 1px);
  height: 1px;
  overflow: hidden;
  position: absolute;
  white-space: nowrap;
  width: 1px;
}

ins::before {
  content: " [insertion start] ";
}

ins::after {
  content: " [insertion end] ";
}
```

Some people who use screen readers deliberately disable announcing content that creates extra verbosity. Because of this, it is important to not abuse this technique and only apply it in situations where not knowing content has been inserted would adversely affect understanding.

- Tweaking Text Level Styles | Adrian Roselli

### Examples

```
<ins>This text has been inserted</ins>
```

### Result

### Technical summary

| | |
|---|---|
| **Content categories** | Phrasing content, flow content. |
| **Permitted content** | Transparent. |
| **Tag omission** | None, both the starting and ending tag are mandatory. |
| **Permitted parents** | Any element that accepts phrasing content. |
| **Implicit ARIA role** | `insertion` |
| **Permitted ARIA roles** | Any |
| **DOM interface** | `HTMLModElement` |

### See also

- `<del>` element for marking deletion into a document

## <kbd> HTML keyboard input element

The **`<kbd>`** HTML element represents user input (typically keyboard input). By default, the content text is displayed using the user agent's default monospace font.

`<kbd>` may be nested in various combinations with the `<samp>` (Sample Output) element to represent various forms of input or output based on visual cues.

### Try it

```
<p>
  Please press <kbd>Ctrl</kbd> + <kbd>Shift</kbd> + <kbd>R</kbd> to re-render an
  MDN page.
</p>
```

```
kbd {
  background-color: #eeeeee;
  border-radius: 3px;
  border: 1px solid #b4b4b4;
  box-shadow:
    0 1px 1px rgb(0 0 0 / 0.2),
    0 2px 0 0 rgb(255 255 255 / 0.7) inset;
  color: #333333;
  display: inline-block;
  font-size: 0.85em;
  font-weight: bold;
  line-height: 1;
  padding: 2px 4px;
```

```
  white-space: nowrap;
}
```

### Attributes

This element only includes the global attributes.

### Usage notes

Other elements can be used in tandem with `<kbd>` to represent more specific scenarios:

- Nesting a `<kbd>` element within another `<kbd>` element represents an actual key or other unit of input as a portion of a larger input. See Representing keystrokes within an input below.
- Nesting a `<kbd>` element inside a `<samp>` element represents input that has been echoed back to the user by the system. See Echoed input, below, for an example.
- Nesting a `<samp>` element inside a `<kbd>` element, on the other hand, represents input which is based on text presented by the system, such as the names of menus and menu items, or the names of buttons displayed on the screen. See the example under Representing onscreen input options below.

> **Note:** You can define a custom style to override the browser's default font selection for the `<kbd>` element, although the user's preferences may potentially override your CSS.

### Examples

### Basic example

```
<p>
  Use the command <kbd>help my-command</kbd> to view documentation for the
  command "my-command".
</p>
```

### Result

### Representing keystrokes within an input

To describe an input comprised of multiple keystrokes, you can nest multiple `<kbd>` elements, with an outer `<kbd>` element representing the overall input and each individual keystroke or component of the input enclosed within its own `<kbd>`.

### Unstyled

First, let's look at what this looks like as just plain HTML.

### HTML

```
<p>
  You can also create a new document using the
  <kbd><kbd>Ctrl</kbd>+<kbd>N</kbd></kbd> keyboard shortcut.
</p>
```

This wraps the entire key sequence in an outer `<kbd>` element, then each individual key within its own, in order to denote the components of the sequence.

> **Note:** You don't need to do all this wrapping; you can choose to simplify it by leaving out the external `<kbd>` element. In other words, simplifying this to just `<kbd>Ctrl</kbd>+<kbd>N</kbd>` would be perfectly valid.
>
> Depending on your style sheet, though, you may find it useful to do this kind of nesting.

### Result

The output looks like this without a style sheet applied:

### With custom styles

We can make more sense of this by adding some CSS:

### CSS

We add a new selector for nested `<kbd>` elements, `kbd>kbd`, which we can apply when rendering keyboard keys:

```
kbd > kbd {
  border-radius: 3px;
  padding: 1px 2px 0;
  border: 1px solid black;
}
```

#### HTML

Then we update the HTML to use this class on the keys in the output to be presented:

```
<p>
  You can also create a new document by pressing the
  <kbd><kbd>Ctrl</kbd>+<kbd>N</kbd></kbd> shortcut.
</p>
```

#### Result

The result is just what we want!

### Echoed input

Nesting a `<kbd>` element inside a `<samp>` element represents input that has been echoed back to the user by the system.

```
<p>
  If a syntax error occurs, the tool will output the initial command you typed
  for your review:
</p>
<blockquote>
  <samp><kbd>custom-git ad my-new-file.cpp</kbd></samp>
</blockquote>
```

#### Result

### Representing onscreen input options

Nesting a `<samp>` element inside a `<kbd>` element represents input which is based on text presented by the system, such as the names of menus and menu items, or the names of buttons displayed on the screen.

For example, you can explain how to choose the "New Document" option in the "File" menu using HTML that looks like this:

```
<p>
  To create a new file, choose the <kbd><kbd><samp>File</samp></kbd>
  ⇒<kbd><samp>New Document</samp></kbd></kbd> menu option.
</p>

<p>
  Don't forget to click the <kbd><samp>OK</samp></kbd> button to confirm once
  you've entered the name of the new file.
</p>
```

This does some interesting nesting. For the menu option description, the entire input is enclosed in a `<kbd>` element. Then, inside that, both the menu and menu item names are contained within both `<kbd>` and `<samp>`, indicating an input which is selected from a screen widget.

#### Result

### Technical summary

| | |
|---|---|
| **Content categories** | Flow content, phrasing content, palpable content. |
| **Permitted content** | Phrasing content. |
| **Tag omission** | None, both the starting and ending tag are mandatory. |
| **Permitted parents** | Any element that accepts phrasing content. |
| **Implicit ARIA role** | No corresponding role |
| **Permitted ARIA roles** | Any |
| **DOM interface** | `HTMLElement` |

### See also

- `<code>`

## <label> HTML label element

The **`<label>`** HTML element represents a caption for an item in a user interface.

### Try it

```
<div class="preference">
  <label for="cheese">I like cheese.</label>
  <input type="checkbox" name="cheese" id="cheese" />
</div>

<div class="preference">
  <label for="peas">I like peas.</label>
  <input type="checkbox" name="peas" id="peas" />
</div>
```

```
.preference {
  display: flex;
  justify-content: space-between;
  width: 60%;
  margin: 0.5rem;
}
```

### Attributes

This element includes the global attributes.

`for`

> The value is the `id` of the labelable form control in the same document, associating the `<label>` with that form control. Note that its JavaScript reflection property is `htmlFor`.

### Usage notes

#### Associating a label with a form control

The first element in the document with an `id` attribute matching the value of the `for` attribute is the *labeled control* for this `label` element — if the element with that `id` is actually a labelable element. If it is *not* a labelable element, then the `for` attribute has no effect. If there are other elements that also match the `id` value, later in the document, they are not considered.

Multiple `<label>` elements can be associated with the same form control by having multiple `<label>` elements with the same `for` attribute value, which gives the form control multiple labels.

Associating a `<label>` with a form control, such as `<input>` or `<textarea>` offers some major advantages:

- The label text is not only visually associated with its corresponding text input; it is programmatically associated with it too. This means that, for example, a screen reader will read out the label when the user is focused on the form input, making it easier for an assistive technology user to understand what data should be entered.
- When a user clicks or touches/taps a label, the browser passes the focus to its associated input (the resulting event is also raised for the input). That increased hit area for focusing the input provides an advantage to anyone trying to activate it — including those using a touch-screen device.

There are two ways to associate a `<label>` with a form control, commonly referred to as *explicit* and *implicit* association.

To explicitly associate a `<label>` element with an `<input>` element, you first need to add the `id` attribute to the `<input>` element. Next, you add the `for` attribute to the `<label>` element, where the value of `for` is the same as the `id` in the `<input>` element.

```
<label for="peas">I like peas.</label>
<input type="checkbox" name="peas" id="peas" />
```

Alternatively, you can nest the `<input>` directly inside the `<label>`, in which case the `for` and `id` attributes are not needed because the association is implicit:

```
<label>
  I like peas.
  <input type="checkbox" name="peas" />
</label>
```

> **Note:** A `<label>` element can have both a `for` attribute and a contained control element, as long as the `for` attribute points to the contained control element.

These two methods are equivalent, but there are a few considerations:

- While common browser and screen reader combinations support implicit association, not all assistive technologies do.
- Depending on your design, the type of association may impact stylability. Making the `<label>` and form control sibling elements instead of parent-child means they are separate, adjacent boxes, enabling more customizable layout such as lining them up with grid or flex layout methods.
- Explicit association requires the form control to have an `id`, which must be unique in the whole document. This is hard especially in a componentized application. Frameworks often provide their own solutions, such as React's `useId()`, but it still requires extra orchestration to get right.

Generally, we recommend using explicit association with the `for` attribute, to ensure compatibility with external tools and assistive technologies. In fact, you can simultaneously nest *and* provide `id`/`for` for maximum compatibility.

The form control that a label is labeling is called the *labeled control* of the label element. Multiple labels can be associated with the same form control:

```
<label for="username">Enter your username:</label>
<input id="username" name="username" type="text" />
<label for="username">Forgot your username?</label>
```

Elements that can be associated with a `<label>` element include `<button>`, `<input>` (except for `type="hidden"`), `<meter>`, `<output>`, `<progress>`, `<select>` and `<textarea>`.

### Accessibility

#### Interactive content

Other than the implicitly associated form control, don't place additional interactive elements such as anchors or buttons inside a `<label>`. Doing so makes it difficult for people to activate the form input associated with the `label`.

**Don't do this:**

```
<label for="tac">
  <input id="tac" type="checkbox" name="terms-and-conditions" />
  I agree to the <a href="terms-and-conditions.html">Terms and Conditions</a>
</label>
```

**Prefer this:**

```
<p>
  <a href="terms-and-conditions.html">Read our Terms and Conditions</a>
</p>
<label for="tac">
  <input id="tac" type="checkbox" name="terms-and-conditions" />
  I agree to the Terms and Conditions
</label>
```

> **Note:** It is a good practice to place any necessary context, such as the link to the terms and conditions, before the form control, so that the user can read it before they interact with the control.

#### Headings

Placing heading elements within a `<label>` interferes with many kinds of assistive technology, because headings are commonly used as a navigation aid. If the label's text needs to be adjusted visually, use CSS classes applied to the `<label>` element instead.

If a form, or a section of a form needs a title, use the `<legend>` element placed within a `<fieldset>`.

**Don't do this:**

```
<label for="your-name">
  <h3>Your name</h3>
  <input id="your-name" name="your-name" type="text" />
</label>
```

**Prefer this:**

```
<label class="large-label" for="your-name">
  Your name
  <input id="your-name" name="your-name" type="text" />
</label>
```

#### Buttons

An `<input>` element with a `type="button"` declaration and a valid `value` attribute does not need a label associated with it. Doing so may actually interfere with how assistive technology parses the button input. The same applies for the `<button>` element.

### Examples

#### Defining an implicit label

```
<label>Click me <input type="text" /></label>
```

#### Defining an explicit label with the "for" attribute

```
<label for="username">Click me to focus on the input field</label>
<input type="text" id="username" />
```

### Technical summary

| | |
|---|---|
| **Content categories** | Flow content, phrasing content, interactive content, form-associated element, palpable content. |
| **Permitted content** | Phrasing content, but no descendant `label` elements. No labelable elements other than the labeled control are allowed. |
| **Tag omission** | None, both the starting and ending tags are mandatory. |
| **Permitted parents** | Any element that accepts phrasing content. |
| **Implicit ARIA role** | No corresponding role |
| **Permitted ARIA roles** | No `role` permitted |
| **DOM interface** | `HTMLLabelElement` |

## <legend> HTML field set legend element

The **`<legend>`** HTML element represents a caption for the content of its parent `<fieldset>`.

In customizable `<select>` elements, the `<legend>` element is allowed as a child of `<optgroup>`, to provide a label that is easy to target and style. This replaces any text set in the `<optgroup>` element's `label` attribute, and it has the same semantics.

### Try it

```
<fieldset>
  <legend>Choose your favorite monster</legend>

  <input type="radio" id="kraken" name="monster" value="K" />
  <label for="kraken">Kraken</label><br />

  <input type="radio" id="sasquatch" name="monster" value="S" />
  <label for="sasquatch">Sasquatch</label><br />

  <input type="radio" id="mothman" name="monster" value="M" />
  <label for="mothman">Mothman</label>
</fieldset>
```

```
legend {
  background-color: black;
  color: white;
  padding: 3px 6px;
}

input {
  margin: 0.4rem;
}
```

### Attributes

This element only includes the global attributes.

### Examples

See `<form>` for examples on `<legend>`.

### Technical summary

| | |
|---|---|
| **Content categories** | None. |

| | |
|---|---|
| **Permitted content** | Phrasing content and headings (h1–h6 elements). |
| **Tag omission** | None, both the starting and ending tag are mandatory. |
| **Permitted parents** | A `<fieldset>` whose first child is this `<legend>` element. In customizable select elements, a `<legend>` element is permitted as a child of `<optgroup>` . |
| **Implicit ARIA role** | No corresponding role |
| **Permitted ARIA roles** | No `role` permitted |
| **DOM interface** | `HTMLLegendElement` |

### See also

- ARIA: Form role

## <li> HTML list item element

The **`<li>`** HTML element is used to represent an item in a list. It must be contained in a parent element: an ordered list (`<ol>`), an unordered list (`<ul>`), or a menu (`<menu>`). In menus and unordered lists, list items are usually displayed using bullet points. In ordered lists, they are usually displayed with an ascending counter on the left, such as a number or letter.

### Try it

```
<p>Apollo astronauts:</p>

<ul>
  <li>Neil Armstrong</li>
  <li>Alan Bean</li>
  <li>Peter Conrad</li>
  <li>Edgar Mitchell</li>
  <li>Alan Shepard</li>
</ul>
```

```
p,
li {
  font:
    1rem "Fira Sans",
    sans-serif;
}

p {
  font-weight: bold;
}
```

### Attributes

This element includes the global attributes.

`value`

> This integer attribute indicates the current ordinal value of the list item as defined by the `<ol>` element. The only allowed value for this attribute is a number, even if the list is displayed with Roman numerals or letters. List items that follow this one continue numbering from the value set. This attribute has no meaning for unordered lists (`<ul>`) or for menus (`<menu>`).

`type`

> This character attribute indicates the numbering type:
>
> - `a`: lowercase letters
> - `A`: uppercase letters
> - `i`: lowercase Roman numerals
> - `I`: uppercase Roman numerals
> - `1`: numbers
>
> This type overrides the one used by its parent `<ol>` element, if any.
>
> > **Note:** This attribute has been deprecated; use the CSS `list-style-type` property instead.

### Examples

For more detailed examples, see the `<ol>` and `<ul>` pages.

### Ordered list

```
<ol>
  <li>first item</li>
  <li>second item</li>
  <li>third item</li>
</ol>
```

### Result

### Ordered list with a custom value

```
<ol type="I">
  <li value="3">third item</li>
  <li>fourth item</li>
  <li>fifth item</li>
</ol>
```

### Result

### Unordered list

```
<ul>
  <li>first item</li>
  <li>second item</li>
  <li>third item</li>
</ul>
```

### Result

### Technical summary

| **Content categories** | None. |
| --- | --- |
| **Permitted content** | Flow content. |
| **Tag omission** | The end tag can be omitted if the list item is immediately followed by another `<li>` element, or if there is no more content in its parent element. |
| **Permitted parents** | An `<ul>`, `<ol>`, or `<menu>` element. Though not a conforming usage, the obsolete `<dir>` can also be a parent. |
| **Implicit ARIA role** | `listitem` when child of an `ol`, `ul` or `menu` |
| **Permitted ARIA roles** | `menuitem`, `menuitemcheckbox`, `menuitemradio`, `option`, `none`, `presentation`, `radio`, `separator`, `tab`, `treeitem` |
| **DOM interface** | `HTMLLIElement` |

### See also

- Other list-related HTML Elements: `<ul>`, `<ol>`, `<menu>`, and the obsolete `<dir>`;
- CSS properties that may be specially useful to style the `<li>` element:
  - the `list-style` property, to choose the way the ordinal is displayed,
  - CSS counters, to handle complex nested lists,
  - the `margin` property, to control the indent of the list item.

## <link> HTML external resource link element

The **`<link>`** HTML element specifies relationships between the current document and an external resource. This element is most commonly used to link to stylesheets, but is also used to establish site icons (both "favicon" style icons and icons for the home screen and apps on mobile devices) among other things.

### Try it

```
<link href="https://developer.mozilla.org/shared-assets/misc/link-element-example.css" rel="stylesheet" />

<p>This text will be red as defined in the external stylesheet.</p>
<p style="color: blue">
  The <code>style</code> attribute can override it, though.
</p>
```

To link an external stylesheet, you'd include a `<link>` element inside your `<head>` like this:

```
<link href="main.css" rel="stylesheet" />
```

This example provides the path to the stylesheet inside an `href` attribute and a `rel` attribute with a value of `stylesheet`. The `rel` stands for "relationship", and is one of the key features of the `<link>` element — the value denotes how the item being linked to is related to the containing document.

There are a number of other common types you'll come across. For example, a link to the site's favicon:

```
<link rel="icon" href="favicon.ico" />
```

There are a number of other icon `rel` values, mainly used to indicate special icon types for use on various mobile platforms, e.g.:

```
<link
  rel="apple-touch-icon"
  sizes="114x114"
  href="apple-icon-114.png"
  type="image/png" />
```

The `sizes` attribute indicates the icon size, while the `type` contains the MIME type of the resource being linked. These provide useful hints to allow the browser to choose the most appropriate icon available.

You can also provide a media type or query inside a `media` attribute; this resource will then only be loaded if the media condition is true. For example:

```
<link href="print.css" rel="stylesheet" media="print" />
<link href="mobile.css" rel="stylesheet" media="screen and (width <= 600px)" />
```

Some interesting new performance and security features have been added to the `<link>` element too. Take this example:

```
<link
  rel="preload"
  href="myFont.woff2"
  as="font"
  type="font/woff2"
  crossorigin="anonymous" />
```

A `rel` value of `preload` indicates that the browser should preload this resource (see `rel="preload"` for more details), with the `as` attribute indicating the specific class of content being fetched. The `crossorigin` attribute indicates whether the resource should be fetched with a CORS request.

Other usage notes:

- A `<link>` element can occur either in the `<head>` or `<body>` element, depending on whether it has a link type that is **body-ok**. For example, the `stylesheet` link type is body-ok, and therefore `<link rel="stylesheet">` is permitted in the body. However, this isn't a good practice to follow; it makes more sense to separate your `<link>` elements from your body content, putting them in the `<head>`.
- When using `<link>` to establish a favicon for a site, and your site uses a Content Security Policy (CSP) to enhance its security, the policy applies to the favicon. If you encounter problems with the favicon not loading, verify that the `Content-Security-Policy` header's `img-src` directive is not preventing access to it.
- The HTML and XHTML specifications define event handlers for the `<link>` element, but it is unclear how they would be used.
- Under XHTML 1.0, void elements such as `<link>` require a trailing slash: `<link />`.
- WebTV supports the use of the value `next` for `rel` to preload the next page in a document series.

### Attributes

This element includes the global attributes.

`as`

This attribute is required when `rel="preload"` has been set on the `<link>` element, optional when `rel="modulepreload"` has been set, and otherwise should not be used. It specifies the type of content being loaded by the `<link>`, which is necessary for request matching, application of correct content security policy, and setting of correct `Accept` request header.

Furthermore, `rel="preload"` uses this as a signal for request prioritization. The table below lists the valid values for this attribute and the elements or resources they apply to.

| `As` value | `Rel` value | Applies To |
|---|---|---|
| audioworklet | modulepreload | AudioWorklet modules |
| fetch | preload | fetch, XHR<br><br>**Note:** This value also requires `<link>` to contain the crossorigin attribute, see CORS-enabled fetches. |

| `As` value | `Rel` value | Applies To |
| --- | --- | --- |
| font | preload | CSS @font-face<br><br>**Note:** This value also requires `<link>` to contain the crossorigin attribute, see CORS-enabled fetches. |
| image | preload | `<img>` and `<picture>` elements with srcset or imageset attributes, SVG `<image>` elements, CSS `*-image` rules |
| json | modulepreload | Supplementary JSON file |
| paintworklet | modulepreload | PaintWorklet modules |
| script | preload or modulepreload | `<script>` elements, Worker `importScripts`, and `modulepreload` destinations. |
| serviceworker | modulepreload | ServiceWorker modules |
| sharedworker | modulepreload | SharedWorker |
| style | preload or modulepreload | `<link rel=stylesheet>` elements, CSS `@import` and `modulepreload` destinations. |
| text | modulepreload | Supplementary plain text file |
| track | preload | `<track>` elements (WebVTT, MIME type `text/vtt`) |
| worker | modulepreload | Worker modules |

`blocking`

This attribute explicitly indicates that certain operations should be blocked until specific conditions are met. It must only be used when the `rel` attribute contains the `expect` or `stylesheet` keywords. With `rel="expect"`, it indicates that operations should be blocked until a specific DOM node has been parsed. With `rel="stylesheet"`, it indicates that operations should be blocked until an external stylesheet and its critical subresources have been fetched and applied to the document. The operations that are to be blocked must be a space-separated list of blocking tokens listed below. Currently there is only one token:

- `render`: The rendering of content on the screen is blocked.

> **Note:** Only `link` elements in the document's `<head>` can possibly block rendering. By default, a `link` element with `rel="stylesheet"` in the `<head>` blocks rendering when the browser discovers it during parsing. If such a `link` element is added dynamically via script, you must additionally set `blocking = "render"` for it to block rendering.

`crossorigin`

This enumerated attribute indicates whether CORS must be used when fetching the resource. CORS-enabled images can be reused in the `<canvas>` element without being *tainted*. The allowed values are:

`anonymous`

A cross-origin request (i.e., with an `Origin` HTTP header) is performed, but no credential is sent (i.e., no cookie, X.509 certificate, or HTTP Basic authentication). If the server does not give credentials to the origin site (by not setting the `Access-Control-Allow-Origin` HTTP header) the resource will be tainted and its usage restricted.

`use-credentials`

A cross-origin request (i.e., with an `Origin` HTTP header) is performed along with a credential sent (i.e., a cookie, certificate, and/or HTTP Basic authentication is performed). If the server does not give credentials to the origin site (through `Access-Control-Allow-Credentials` HTTP header), the resource will be *tainted* and its usage restricted.

If the attribute is not present, the resource is fetched without a CORS request (i.e., without sending the `Origin` HTTP header), preventing its non-tainted usage. If invalid, it is handled as if the enumerated keyword **anonymous** was used. See CORS settings attributes for additional information.

`disabled`

For `rel="stylesheet"` only, the `disabled` Boolean attribute indicates whether the described stylesheet should be loaded and applied to the document. If `disabled` is specified in the HTML when it is loaded, the stylesheet will not be loaded during page load. Instead, the stylesheet will be loaded on-demand, if and when the `disabled` attribute is changed to `false` or removed.

Setting the `disabled` property in the DOM causes the stylesheet to be removed from the document's `Document.styleSheets` list.

`fetchpriority`

Provides a hint of the relative priority to use when fetching a resource of a particular type. Allowed values:

`high`

Fetch the resource at a high priority relative to other resources of the same type.

`low`

Fetch the resource at a low priority relative to other resources of the same type.

`auto`

Don't set a preference for the fetch priority. This is the default. It is used if no value or an invalid value is set.

`href`

This attribute specifies the URL of the linked resource. A URL can be absolute or relative.

`hreflang`

This attribute indicates the language of the linked resource. It is purely advisory. Values should be valid BCP 47 language tags. Use this attribute only if the `href` attribute is present.

`imagesizes`

For `rel="preload"` and `as="image"` only, the `imagesizes` attribute has similar syntax and semantics as the `sizes` attribute that indicates to preload the appropriate resource used by an `img` element with corresponding values for its `srcset` and `sizes` attributes.

`imagesrcset`

For `rel="preload"` and `as="image"` only, the `imagesrcset` attribute has similar syntax and semantics as the `srcset` attribute that indicates to preload the appropriate resource used by an `img` element with corresponding values for its `srcset` and `sizes` attributes.

`integrity`

This attribute contains one or more hashes of the resource. It is used to ensure that the content of the resource is what the developer expects it to be, and has not been replaced with a malicious copy in a supply chain attack. The attribute must only be specified when the `rel` attribute is set to `stylesheet`, `preload`, or `modulepreload`. See Subresource Integrity.

`media`

This attribute specifies the media that the linked resource applies to. Its value must be a media type / media query. This attribute is mainly useful when linking to external stylesheets — it allows the user agent to pick the best adapted one for the device it runs on.

`referrerpolicy`

A string indicating which referrer to use when fetching the resource. For detailed explanations and examples of each policy, see the `Referrer-Policy` header documentation.

- `no-referrer` means that the `Referer` header will not be sent.
- `no-referrer-when-downgrade` means that no `Referer` header will be sent when navigating to an origin without TLS (HTTPS). This is a user agent's default behavior, if no policy is otherwise specified.
- `origin` means that the referrer will be the origin of the page, which is roughly the scheme, the host, and the port.
- `origin-when-cross-origin` means that navigating to other origins will be limited to the scheme, the host, and the port, while navigating on the same origin will include the referrer's path.
- `same-origin` means that the referrer (origin, path, and query string) is sent for same-origin requests, but no referrer is sent for cross-origin requests.
- `strict-origin` means that only the origin is sent when the protocol security level stays the same (HTTPS→HTTPS). No referrer is sent to less secure destinations (HTTPS→HTTP). This is important for HTTPS pages because it prevents leaking referrer information to insecure origins.
- `strict-origin-when-cross-origin` means that the full referrer is sent for same-origin requests. For cross-origin requests, only the origin is sent when the protocol stays the same (HTTPS→HTTPS), and no referrer is sent when downgrading to HTTP. This is the default value, which balances functionality with privacy and security for HTTPS sites.
- `unsafe-url` means that the referrer will include the origin and the path (but not the fragment, password, or username). This case is unsafe because it can leak origins and paths from TLS-protected resources to insecure origins.

`rel`

This attribute names a relationship of the linked document to the current document. The attribute must be a space-separated list of link type values.

`sizes`

This attribute defines the sizes of the icons for visual media contained in the resource. It must be present only if the `rel` contains a value of `icon` or a non-standard type such as Apple's `apple-touch-icon`. It may have the following values:

- `any`, meaning that the icon can be scaled to any size as it is in a vector format, like `image/svg+xml`.
- a white-space separated list of sizes, each in the format `<width in pixels>x<height in pixels>` or `<width in pixels>X<height in pixels>`. Each of these sizes must be contained in the resource.

> **Note:** Most icon formats are only able to store one single icon; therefore, most of the time, the `sizes` attribute contains only one entry. Microsoft's ICO format and Apple's ICNS format can store multiple icon sizes in a single file. ICO has better browser support, so you should use this format if cross-browser support is a concern.

`title`

The `title` attribute has special semantics on the `<link>` element. When used on a `<link rel="stylesheet">` it defines a default or an alternate stylesheet.

`type`

This attribute is used to define the type of the content linked to. The value of the attribute should be a MIME type such as **text/html**, **text/css**, and so on. The common use of this attribute is to define the type of stylesheet being referenced (such as **text/css**), but given that CSS is the only stylesheet language used on the web, not only is it possible to omit the `type` attribute, but is actually now recommended practice. It is also used on `rel="preload"` link types, to make sure the browser only downloads file types that it supports.

### Non-standard attributes

`target`

Defines the frame or window name that has the defined linking relationship or that will show the rendering of any linked resource.

### Obsolete attributes

`charset`

This attribute defines the character encoding of the linked resource. The value is a space- and/or comma-delimited list of character sets as defined in RFC 2045. The default value is `iso-8859-1`.

> **Note:** To produce the same effect as this obsolete attribute, use the `Content-Type` HTTP header on the linked resource.

`rev`

The value of this attribute shows the relationship of the current document to the linked document, as defined by the `href` attribute. The attribute thus defines the reverse relationship compared to the value of the `rel` attribute. Link type values for the attribute are similar to the possible values for `rel`.

> **Note:** Instead of `rev`, you should use the `rel` attribute with the opposite link type value. For example, to establish the reverse link for `made`, specify `author`. Also, this attribute doesn't stand for "revision" and must not be used with a version number, even though many sites misuse it in this way.

## Examples

### Including a stylesheet

To include a stylesheet in a page, use the following syntax:

```
<link href="style.css" rel="stylesheet" />
```

### Providing alternative stylesheets

You can also specify alternative style sheets.

The user can choose which style sheet to use by choosing it from the **View > Page Style** menu. This provides a way for users to see multiple versions of a page.

```
<link href="default.css" rel="stylesheet" title="Default Style" />
<link href="fancy.css" rel="alternate stylesheet" title="Fancy" />
<link href="basic.css" rel="alternate stylesheet" title="Basic" />
```

### Providing icons for different usage contexts

You can include links to several icons on the same page, and the browser will choose which one works best for its particular context using the `rel` and `sizes` values as hints.

```
<!-- iPad Pro with high-resolution Retina display: -->
<link
  rel="apple-touch-icon"
  sizes="167x167"
  href="https://developer.mozilla.org/apple-touch-icon-167x167.png" />
<!-- 3x resolution iPhone: -->
<link
  rel="apple-touch-icon"
  sizes="180x180"
  href="https://developer.mozilla.org/apple-touch-icon-180x180.png" />
<!-- non-Retina iPad, iPad mini, etc.: -->
<link
  rel="apple-touch-icon"
  sizes="152x152"
  href="https://developer.mozilla.org/apple-touch-icon-152x152.png" />
<!-- 2x resolution iPhone and other devices: -->
<link rel="apple-touch-icon" href="https://developer.mozilla.org/apple-touch-icon-120x120.png" />
<!-- basic favicon -->
<link rel="icon" href="https://developer.mozilla.org/favicon.ico" />
```

For information about what `sizes` to choose for Apple icons, see Apple's documentation on configuring web applications and the referenced Apple human interface guidelines. Usually, it is sufficient to provide a large image, such as 192x192, and let the browser scale it down as needed, but you may want to provide images with different levels of detail for different sizes, as the Apple design guideline recommends. Providing smaller icons for lower resolutions also saves bandwidth.

It may not be necessary to provide `<link>` elements at all. For example, browsers automatically request `/favicon.ico` from the root of a site, and Apple also automatically requests `/apple-touch-icon-[size].png`, `/apple-touch-icon.png`, etc. However, providing explicit links protects you against changes to these conventions.

### Conditionally loading resources with media queries

You can provide a media type or query inside a `media` attribute; this resource will then only be loaded if the media condition is true. For example:

```
<link href="print.css" rel="stylesheet" media="print" />
<link href="mobile.css" rel="stylesheet" media="all" />
<link href="desktop.css" rel="stylesheet" media="screen and (width >= 600px)" />
<link
  href="highres.css"
  rel="stylesheet"
  media="screen and (resolution >= 300dpi)" />
```

### Stylesheet load events

You can determine when a style sheet has been loaded by watching for a `load` event to fire on it; similarly, you can detect if an error has occurred while processing a style sheet by watching for an `error` event:

```
<link rel="stylesheet" href="mystylesheet.css" id="my-stylesheet" />
```

```
const stylesheet = document.getElementById("my-stylesheet");

stylesheet.onload = () => {
  // Do something interesting; the sheet has been loaded
};

stylesheet.onerror = () => {
  console.log("An error occurred loading the stylesheet!");
};
```

> **Note:** The `load` event fires once the stylesheet and all of its imported content has been loaded and parsed, and immediately before the styles start being applied to the content.

### Preload examples

You can find a number of `<link rel="preload">` examples in Preloading content with `rel="preload"`.

### Blocking rendering till a resource is fetched

You can include `render` token inside a `blocking` attribute; the rendering of the page will be blocked till the resource and its critical subresources are fetched and applied to the document. For example:

```
<link blocking="render" rel="stylesheet" href="example.css" crossorigin />
```

### Technical summary

| | |
|---|---|
| **Content categories** | Metadata content. If `itemprop` is present: Flow content and phrasing content. |
| **Permitted content** | None; it is a void element. |
| **Tag omission** | Must have a start tag and must not have an end tag. |
| **Permitted parents** | Any element that accepts metadata elements. If itemprop is present: any element that accepts phrasing content. |
| **Implicit ARIA role** | `link` with `href` attribute |
| **Permitted ARIA roles** | No `role` permitted |
| **DOM interface** | `HTMLLinkElement` |

### See also

- `Link` HTTP header
- `Referrer-Policy` HTTP header

## <main> HTML main element

The **`<main>`** HTML element represents the dominant content of the `<body>` of a document. The main content area consists of content that is directly related to or expands upon the central topic of a document, or the central functionality of an application.

### Try it

```
<header>Gecko facts</header>

<main>
  <p>
    Geckos are a group of usually small, usually nocturnal lizards. They are
    found on every continent except Antarctica.
  </p>

  <p>
    Many species of gecko have adhesive toe pads which enable them to climb
    walls and even windows.
  </p>
</main>
```

```
header {
  font:
    bold 7vw "Arial",
    sans-serif;
}
```

### Attributes

This element only includes the global attributes.

### Usage notes

A document mustn't have more than one `<main>` element that doesn't have the `hidden` attribute specified.

The content of a `<main>` element should be unique to the document. Content that is repeated across a set of documents or document sections such as sidebars, navigation links, copyright information, site logos, and search forms shouldn't be included unless the search form is the main function of the page.

`<main>` doesn't contribute to the document's outline; that is, unlike elements such as `<body>`, headings such as h2, and such, `<main>` doesn't affect the DOM's concept of the structure of the page. It's strictly informative.

### Accessibility

### Landmark

The `<main>` element behaves like a `main` landmark role. Landmarks can be used by assistive technology to quickly identify and navigate to large sections of the document. Prefer using the `<main>` element over declaring `role="main"`, unless there are legacy browser support concerns.

### Skip navigation

Skip navigation, also known as "skipnav", is a technique that allows an assistive technology user to quickly bypass large sections of repeated content (main navigation, info banners, etc.). This lets the user access the main content of the page faster.

Adding an `id` attribute to the `<main>` element lets it be a target of a skip navigation link.

```
<body>
  <a href="#main-content">Skip to main content</a>

  <!-- navigation and header content -->

  <main id="main-content">
    <!-- main page content -->
  </main>
</body>
```

- WebAIM: "Skip Navigation" Links

### Reader mode

Browser reader mode functionality looks for the presence of the `<main>` element, as well as heading and content sectioning elements when converting content into a specialized reader view.

- Building websites for Safari Reader Mode and other reading apps.

### Examples

```
<!-- other content -->

<main>
  <h1>Apples</h1>
  <p>The apple is the pomaceous fruit of the apple tree.</p>

  <article>
    <h2>Red Delicious</h2>
    <p>
      These bright red apples are the most common found in many supermarkets.
    </p>
    <p>…</p>
    <p>…</p>
  </article>

  <article>
    <h2>Granny Smith</h2>
    <p>These juicy, green apples make a great filling for apple pies.</p>
    <p>…</p>
    <p>…</p>
  </article>
</main>

<!-- other content -->
```

### Result

### Technical summary

| | |
|---|---|
| **Content categories** | Flow content, palpable content. |
| **Permitted content** | Flow content. |
| **Tag omission** | None; both the starting and ending tags are mandatory. |
| **Permitted parents** | Where flow content is expected, but only if it is a hierarchically correct `main` element. |
| **Implicit ARIA role** | `main` |
| **Permitted ARIA roles** | No `role` permitted |
| **DOM interface** | `HTMLElement` |

### See also

- Basic structural elements: `<html>`, `<head>`, `<body>`
- Section-related elements: `<article>`, `<aside>`, `<footer>`, `<header>`, or `<nav>`
- ARIA: Main role

## <map> HTML image map element

The **`<map>`** HTML element is used with `<area>` elements to define an image map (a clickable link area).

### Try it

```
<map name="infographic">
  <area
    shape="poly"
    coords="130,147,200,107,254,219,130,228"
    href="https://developer.mozilla.org/docs/Web/HTML"
    alt="HTML" />
  <area
    shape="poly"
    coords="130,147,130,228,6,219,59,107"
    href="https://developer.mozilla.org/docs/Web/CSS"
    alt="CSS" />
  <area
    shape="poly"
    coords="130,147,200,107,130,4,59,107"
    href="https://developer.mozilla.org/docs/Web/JavaScript"
    alt="JavaScript" />
</map>
<img
  usemap="#infographic"
  src="https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/map//shared-assets/images/examples/mdn-info2.png"
  alt="MDN infographic" />
```

```
img {
  display: block;
  margin: 0 auto;
  width: 260px;
  height: 232px;
}
```

### Attributes

This element includes the global attributes.

`name`

> The `name` attribute gives the map a name so that it can be referenced. The attribute must be present and must have a non-empty value with no space characters. The value of the `name` attribute must not be equal to the value of the `name` attribute of another `<map>` element in the same document. If the `id` attribute is also specified, both attributes must have the same value.

### Examples

#### Image map with two areas

Click the left-hand parrot for JavaScript, or the right-hand parrot for CSS.

#### HTML

```
<!-- Photo by Juliana e Mariana Amorim on Unsplash -->
<map name="primary">
  <area
    shape="circle"
    coords="75,75,75"
    href="https://developer.mozilla.org/docs/Web/JavaScript"
    target="_blank"
    alt="JavaScript" />
  <area
    shape="circle"
    coords="275,75,75"
    href="https://developer.mozilla.org/docs/Web/CSS"
    target="_blank"
    alt="CSS" />
</map>
<img
  usemap="#primary"
  src="https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/map/parrots.jpg"
  alt="350 x 150 picture of two parrots" />
```

#### Result

### Technical summary

| | |
|---|---|
| **Content categories** | Flow content, phrasing content, palpable content. |
| **Permitted content** | Any transparent element. |
| **Tag omission** | None, both the starting and ending tag are mandatory. |

| | |
|---|---|
| **Permitted parents** | Any element that accepts phrasing content. |
| **Implicit ARIA role** | No corresponding role |
| **Permitted ARIA roles** | No `role` permitted |
| **DOM interface** | `HTMLMapElement` |

### See also

- `<a>`
- `<area>`

## <mark> HTML mark text element

The **`<mark>`** HTML element represents text which is **marked** or **highlighted** for reference or notation purposes due to the marked passage's relevance in the enclosing context.

### Try it

```
<p>Search results for "salamander":</p>

<hr />

<p>
  Several species of <mark>salamander</mark> inhabit the temperate rainforest of
  the Pacific Northwest.
</p>

<p>
  Most <mark>salamander</mark>s are nocturnal, and hunt for insects, worms, and
  other small creatures.
</p>
```

```
mark {
  /* Add your styles here */
}
```

### Attributes

This element only includes the global attributes.

### Usage notes

Typical use cases for `<mark>` include:

- When used in a quotation (`<q>`) or block quote (`<blockquote>`), it generally indicates text which is of special interest but is not marked in the original source material, or material which needs special scrutiny even though the original author didn't think it was of particular importance. Think of this like using a highlighter pen in a book to mark passages that you find of interest.
- Otherwise, `<mark>` indicates a portion of the document's content which is likely to be relevant to the user's current activity. This might be used, for example, to indicate the words that matched a search operation.
- Don't use `<mark>` for syntax highlighting purposes; instead, use the `<span>` element with appropriate CSS applied to it.

> **Note:** Don't confuse `<mark>` with the `<strong>` element; `<mark>` is used to denote content which has a degree of *relevance*, while `<strong>` indicates spans of text of *importance*.

### Accessibility

The presence of the `mark` element is not announced by most screen reading technology in its default configuration. It can be made to be announced by using the CSS `content` property, along with the `::before` and `::after` pseudo-elements.

```
mark::before,
mark::after {
  clip-path: inset(100%);
  clip: rect(1px, 1px, 1px, 1px);
  height: 1px;
  overflow: hidden;
  position: absolute;
  white-space: nowrap;
  width: 1px;
}

mark::before {
```

```
  content: " [highlight start] ";
}

mark::after {
  content: " [highlight end] ";
}
```

Some people who use screen readers deliberately disable announcing content that creates extra verbosity. Because of this, it is important to not abuse this technique and only apply it in situations where not knowing content has been highlighted would adversely affect understanding.

- Tweaking Text Level Styles, Reprised via Adrian Roselli (2025)

### Examples

#### Marking text of interest

In this first example, a `<mark>` element is used to mark some text within a quote which is of particular interest to the user.

```
<blockquote>
  It is a period of civil war. Rebel spaceships, striking from a hidden base,
  have won their first victory against the evil Galactic Empire. During the
  battle, <mark>Rebel spies managed to steal secret plans</mark> to the Empire's
  ultimate weapon, the DEATH STAR, an armored space station with enough power to
  destroy an entire planet.
</blockquote>
```

#### Result

#### Identifying context-sensitive passages

This example demonstrates using `<mark>` to mark search results within a passage.

```
<p>
  It is a dark time for the Rebellion. Although the Death Star has been
  destroyed, <mark class="match">Imperial</mark> troops have driven the Rebel
  forces from their hidden base and pursued them across the galaxy.
</p>

<p>
  Evading the dreaded <mark class="match">Imperial</mark> Starfleet, a group of
  freedom fighters led by Luke Skywalker has established a new secret base on
  the remote ice world of Hoth.
</p>
```

To help distinguish the use of `<mark>` for search results from other potential usage, this example applies the custom class `"match"` to each match.

#### Result

### Technical summary

| | |
|---|---|
| **Content categories** | Flow content, phrasing content, palpable content. |
| **Permitted content** | Phrasing content. |
| **Tag omission** | None, both the starting and ending tag are mandatory. |
| **Permitted parents** | Any element that accepts phrasing content. |
| **Implicit ARIA role** | No corresponding role |
| **Permitted ARIA roles** | Any |
| **DOM interface** | `HTMLElement` |

## <marquee> HTML marquee element

The **`<marquee>`** HTML element is used to insert a scrolling area of text. You can control what happens when the text reaches the edges of its content area using its attributes.

The HTML `<marquee>` element is deprecated and its use is strongly discouraged. If you must create the effect of scrolling text or continuous elements, consider using CSS animations with CSS transforms instead of `<marquee>` elements to smoothly animate content. Additionally, include the `prefers-reduced-motion` CSS `@media` query to stop the animation based on user preference, thereby improving user experience and accessibility.

### Attributes

`behavior`

Sets how the text is scrolled within the marquee. Possible values are `scroll`, `slide` and `alternate`. If no value is specified, the default value is `scroll`.

`bgcolor`

Sets the background color through color name or hexadecimal value.

`direction`

Sets the direction of the scrolling within the marquee. Possible values are `left`, `right`, `up` and `down`. If no value is specified, the default value is `left`.

`height`

Sets the height in pixels or percentage value.

`hspace`

Sets the horizontal margin

`loop`

Sets the number of times the marquee will scroll. If no value is specified, the default value is −1, which means the marquee will scroll continuously.

`scrollamount`

Sets the amount of scrolling at each interval in pixels. The default value is 6.

`scrolldelay`

Sets the interval between each scroll movement in milliseconds. The default value is 85. Note that any value smaller than 60 is ignored and the value 60 is used instead unless `truespeed` is specified.

`truespeed`

By default, `scrolldelay` values lower than 60 are ignored. If `truespeed` is present, those values are not ignored.

`vspace`

Sets the vertical margin in pixels or percentage value.

`width`

Sets the width in pixels or percentage value.

### Examples

```
<marquee>This text will scroll from right to left</marquee>

<marquee direction="up">This text will scroll from bottom to top</marquee>

<marquee
  direction="down"
  width="250"
  height="200"
  behavior="alternate"
  class="outlined">
  <marquee behavior="alternate">This text will bounce</marquee>
</marquee>
```

```
.outlined {
  border: solid;
}
```

### Result

### Technical summary

| **DOM interface** | `HTMLMarqueeElement` |
| --- | --- |

### See also

- CSS `transform` property
- CSS `translate` property

- CSS transforms module
- CSS animations module
- `HTMLMarqueeElement`

## <menu> HTML menu element

The **`<menu>`** HTML element is described in the HTML specification as a semantic alternative to `<ul>`, but treated by browsers (and exposed through the accessibility tree) as no different than `<ul>`. It represents an unordered list of items (which are represented by `<li>` elements).

### Try it

```
<div class="news">
  <a href="#">NASA's Webb Delivers Deepest Infrared Image of Universe Yet</a>
  <menu>
    <li><button id="save">Save for later</button></li>
    <li><button id="share">Share this news</button></li>
  </menu>
</div>
```

```
.news {
  background-color: bisque;
  padding: 1em;
  border: solid thin black;
}

menu {
  list-style-type: none;
  display: flex;
  padding: 0;
  margin-bottom: 0;
  gap: 1em;
}
```

### Attributes

This element also accepts the global attributes.

`compact`

> This Boolean attribute hints that the list should be rendered in a compact style. The interpretation of this attribute is browser-specific. Use CSS instead: to give a similar effect as the `compact` attribute, the CSS property `line-height` can be used with a value of `80%`.

### Usage notes

The `<menu>` and `<ul>` elements both represent an unordered list of items. The key difference is that `<ul>` primarily contains items for display, while `<menu>` represents a toolbar containing commands that the user can perform or activate.

> **Note:** In early versions of the HTML specification, the `<menu>` element had an additional use case as a context menu. This functionality is considered obsolete and is not in the specification.

### Examples

#### Toolbar

In this example, a `<menu>` is used to create a toolbar for an editing application.

#### HTML

```
<menu>
  <li><button onclick="copy()">Copy</button></li>
  <li><button onclick="cut()">Cut</button></li>
  <li><button onclick="paste()">Paste</button></li>
</menu>
```

Note that this is functionally no different from:

```
<ul>
  <li><button onclick="copy()">Copy</button></li>
  <li><button onclick="cut()">Cut</button></li>
  <li><button onclick="paste()">Paste</button></li>
</ul>
```

### CSS

```
menu,
ul {
  display: flex;
  list-style: none;
  padding: 0;
  width: 400px;
}

li {
  flex-grow: 1;
}

button {
  width: 100%;
}
```

### Result

### Technical summary

| | |
|---|---|
| **Content categories** | Flow content. If the element's children include at least one `<li>` element: Palpable content. |
| **Permitted content** | Zero or more occurrences of `<li>`, `<script>`, and `<template>`. |
| **Tag omission** | None, both the starting and ending tag are mandatory. |
| **Permitted parents** | Any element that accepts flow content. |
| **Implicit ARIA role** | `list` |
| **Permitted ARIA roles** | `directory`, `group`, `listbox`, `menu`, `menubar`, `none`, `presentation`, `radiogroup`, `tablist`, `toolbar` or `tree` |
| **DOM interface** | `HTMLMenuElement` |

### See also

- Other list-related HTML Elements: `<ol>`, `<ul>`, and `<li>`.

## <meta> HTML metadata element

The **`<meta>`** HTML element represents metadata that cannot be represented by other meta-related elements, such as `<base>`, `<link>`, `<script>`, `<style>`, or `<title>`.

The type of metadata provided by the `<meta>` element can be one of the following:

- If the `name` attribute is set, the `<meta>` element provides *document-level metadata* that applies to the whole page.
- If the `http-equiv` attribute is set, the `<meta>` element acts as a *pragma directive* to simulate directives that could otherwise be given by an HTTP header.
- If the `charset` attribute is set, the `<meta>` element is a *charset declaration*, giving the character encoding in which the document is encoded.
- If the `itemprop` attribute is set, the `<meta>` element provides *user-defined metadata*.

### Attributes

This element includes the global attributes.

> **Note:** The `name` attribute has a specific meaning for the `<meta>` element. The `itemprop` attribute must not be set on a `<meta>` element that includes a `name`, `http-equiv`, or `charset` attribute.

`charset`

> This attribute declares the document's character encoding. If the attribute is present, its value must be an ASCII case-insensitive match for the string `"utf-8"`, because UTF-8 is the only valid encoding for HTML5 documents. `<meta>` elements which declare a character encoding must be located entirely within the first 1024 bytes of the document.

`content`

> This attribute contains the value for the `http-equiv` or `name` attribute, depending on which is used.

`http-equiv`

Defines a pragma directive, which are instructions for the browser for processing the document. The attribute's name is short for `http-equivalent` because the allowed values are names of equivalent HTTP headers.

`media`

The `media` attribute defines which media the theme color defined in the `content` attribute should be applied to. Its value is a media query, which defaults to `all` if the attribute is missing. This attribute is only relevant when the element's `name` attribute is set to `theme-color`. Otherwise, it has no effect, and should not be included.

`name`

The `name` and `content` attributes can be used together to provide document metadata in terms of name-value pairs, with the `name` attribute giving the metadata name, and the `content` attribute giving the value.

### Examples

#### Setting a meta description

The following `<meta>` tag provides a `description` as metadata for the web page:

```
<meta
  name="description"
  content="The HTML reference describes all elements and attributes of HTML, including global attributes that apply to all elements." />
```

#### Setting a page redirect

The following example uses `http-equiv="refresh"` to direct the browser to perform a redirect. The `content="3;url=https://www.mozilla.org"` attribute will redirect page to `https://www.mozilla.org` after 3 seconds:

```
<meta http-equiv="refresh" content="3;url=https://www.mozilla.org" />
```

### Technical summary

| | |
|---|---|
| **Content categories** | Metadata content. If the `itemprop` attribute is present: flow content, phrasing content. |
| **Permitted content** | None; it is a void element. |
| **Tag omission** | Must have a start tag and must not have an end tag. |
| **Permitted parents** | • `<meta charset>`, `<meta http-equiv>`: a `<head>` element. If the `http-equiv` is not an encoding declaration, it can also be inside a `<noscript>` element, itself inside a `<head>` element.<br>• `<meta name>`: any element that accepts metadata content.<br>• `<meta itemprop>`: any element that accepts metadata content or flow content. |
| **Implicit ARIA role** | No corresponding role |
| **Permitted ARIA roles** | No `role` permitted |
| **DOM interface** | `HTMLMetaElement` |

### See also

- Standard metadata names
- Learn: `<meta>`

## <meta http-equiv> HTML attribute

The **`http-equiv`** attribute of the `<meta>` element allows you to provide processing instructions for the browser as if the response that returned the document included certain HTTP headers. The metadata is *document-level metadata* that applies to the whole page.

When a `<meta>` element has an `http-equiv` attribute, a `content` attribute defines the corresponding `http-equiv` value. For example, the following `<meta>` tag tells the browser to refresh the page after 5 minutes:

```
<meta http-equiv="Refresh" content="300" />
```

### Value

Only a subset of the HTTP headers are supported as `http-equiv` values. These include:

`content-language`

Sets a default language for the document used by assistive technologies or styling by the browser. Similar to the `Content-Language` HTTP header. Use the `lang` attribute instead.

`content-type`

Declares the document's media type (MIME type) and character encoding. The `content` attribute must be `"text/html; charset=utf-8"` if specified. This is equivalent to a `<meta>` element with the `charset` attribute specified and carries the same restriction on placement within the document. Can only be used in documents served with a `text/html` media type — not in documents served with an XML (`application/xml` or `application/xhtml+xml`) type. See the `Content-Type` HTTP header.

`content-security-policy`

Allows page authors to define a content security policy (CSP) for the current page, typically to specify allowed origins and script endpoints to guard against cross-site scripting attacks. See the `Content-Security-Policy` HTTP header.

`default-style`

Sets the name of the default CSS style sheet set.

`refresh`

Equivalent to the `Refresh` HTTP header. This instruction specifies:

- The number of seconds until the page should be reloaded if the `content` attribute is a non-negative integer.
- The number of seconds until the page should redirect to another URL if the `content` attribute is a non-negative integer followed by `;url=` and a valid URL.

The timer starts when the page is *completely loaded*, which is after the `load` and `pageshow` events have both fired. See Accessibility concerns for more information.

`set-cookie`

Sets a cookie for the document. Browsers now ignore this pragma; use the `Set-Cookie` HTTP response header or `document.cookie` instead.

`x-ua-compatible`

Used by legacy versions of the now-retired Microsoft Internet Explorer so that it more closely followed specified behavior. If specified, the `content` attribute must have the value `"IE=edge"`. User agents now ignore this pragma. The name derives from the `X-UA-Compatible` HTTP header.

> **Warning:** Some browsers process additional headers that are not listed above. Since unrecognized headers or invalid values are ignored, this can lead to inconsistent behavior across browser implementations. In particular, **Do not set other security headers** using `<meta http-equiv=`, as this can lead to a false sense of security!

### Accessibility concerns

Pages set with a `http-equiv="Refresh"` value run the risk of having the refresh interval being too short. People navigating with the aid of assistive technology such as a screen reader may be unable to read through and understand the page's content before being automatically redirected. Abrupt, unannounced page updates may also be disorienting for people experiencing low vision conditions.

- MDN Understanding WCAG, Guideline 2.2 explanations
- MDN Understanding WCAG, Guideline 3.2 explanations
- Understanding Success Criterion 2.2.1 | W3C Understanding WCAG 2.0
- Understanding Success Criterion 2.2.4 | W3C Understanding WCAG 2.0
- Understanding Success Criterion 3.2.5 | W3C Understanding WCAG 2.0

## Examples

### Disable unsafe inline code and only allow HTTPS resources

This HTML `<meta>` element sets the default CSP to only allow resource loading (images, fonts, scripts, etc.) over HTTPS. Because the `unsafe-inline` and `unsafe-eval` directives are not set, inline scripts will be blocked:

```
<meta http-equiv="Content-Security-Policy" content="default-src https:" />
```

The same restrictions can be applied using the HTTP `Content-Security-Policy` header:

```
Content-Security-Policy: default-src https:
```

### Setting a page redirect

The following example uses `http-equiv="refresh"` to direct the browser to perform a redirect. The `content="3;url=https://www.mozilla.org"` attribute will redirect page to `https://www.mozilla.org` after 3 seconds:

```
<meta http-equiv="refresh" content="3;url=https://www.mozilla.org" />
```

### See also

- `<meta name="referrer">`
- Metadata: the `<meta>` element
- Preventing attacks using `<meta>` httparchive.org (2022)

## `<meta name>` HTML attribute

The **`name`** attribute of the `<meta>` element provides metadata in name-value pairs. When a `<meta>` element has a `name` attribute, a `content` attribute defines the corresponding value. The metadata is *document-level metadata* that applies to the whole page.

For example, the following `<meta>` tag provides a `description` as metadata for a document:

```
<meta
  name="description"
  content="The HTML reference describes all elements and attributes of HTML, including global attributes that apply to all elements." />
```

### Value

### Meta names defined in the HTML specification

The HTML specification defines the following set of standard metadata names:

`application-name`

> Browsers may use this to identify the application running in the web page. It is different from the `<title>` element, which may contain an application (or website) name, but a `<title>` may add contextual information like a document name or a status. Individual pages shouldn't define their own, unique `application-name`. To provide translations, use multiple `<meta>` tags with the `lang` attribute for each language:
>
> ```
> <meta name="application-name" content="Weather Wizard" lang="en" />
> <meta name="application-name" content="Mago del Clima" lang="es" />
> ```

`author`

> The document author's name.

`color-scheme`

> Specifies one or more color schemes with which the document is compatible. The browser will use this information in tandem with the user's browser or device settings to determine what colors to use for everything from background and foregrounds to form controls and scrollbars. The primary use for `<meta name="color-scheme">` is to indicate compatibility and order of preference for light and dark color modes.

`description`

> A short and accurate summary of the content of the page usually referred to as a "meta description". Search engines like Google use this metadata to adjust the appearance of a webpage in search results.

`generator`

> The identifier of the software that generated the page.

`keywords`

> Words relevant to the page's content separated by commas.

`referrer`

> Controls the HTTP `Referer` header of requests sent from the document.

`theme-color`

> Indicates a suggested color that user agents should use to customize the display of the page or of the surrounding user interface. The `content` attribute contains a valid CSS `<color>`. The `media` attribute with a valid media query list can be included to set the media that the theme color metadata applies to.

### Meta names defined in other specifications

`responsive-embedded-sizing`

Opts in an embedded document to sharing its size information with the parent page. Defined in the CSS box sizing module.

`text-scale`

Enables opting the page in to having the `<html>` root element's `font-size` scale in proportion to OS and browser-level text scale settings. Defined in the CSS fonts module.

`viewport`

Gives hints about the initial size of the viewport. Defined in the CSS viewport module.

### Meta names defined in the WHATWG MetaExtensions wiki

The WHATWG Wiki MetaExtensions page contains a large set of non-standard metadata names. Some of the names included are used quite commonly in practice, notably the following:

`creator`

The name of the creator of the document, such as an organization or institution. If there are more than one, several `<meta>` elements should be used.

`googlebot`

A synonym of `robots`, is only followed by Googlebot (the indexing crawler for Google).

`publisher`

The document publisher's name.

`robots`

A comma-separated list of values defining the crawl behavior that cooperative crawlers (or "robots") should use with the page.

## See also

- Metadata: the `<meta>` element

## `<meta name="color-scheme">` HTML attribute value

The **`color-scheme`** value for the `name` attribute of the `<meta>` element indicates a suggested color scheme that user agents should use for a page. If specified, you define the color scheme using a `content` attribute in the `<meta>` element with a valid CSS `color-scheme` value.

The theme color works at the *document level* in the same way that the CSS `color-scheme` property specifies the preferred and accepted color schemes of *individual elements*. The primary use for `<meta name="color-scheme">` is to indicate compatibility and order of preference for light and dark color modes. For example, to indicate that a document prefers dark mode but also supports light mode:

```
<meta name="color-scheme" content="dark light" />
```

Your styles can adapt to the current color scheme using the `prefers-color-scheme` CSS media feature.

### Usage notes

A `<meta name="color-scheme">` element has the following additional attributes:

`content`

A `<meta>` element with `name=color-scheme` must have a `content` attribute that defines the color scheme as a CSS `color-scheme` value. The `content` attribute may be one of the following:

`normal`

The document is unaware of color schemes and should be rendered using the default color palette.

`light`, `dark`, `light dark`, `dark light`

One or more color schemes supported by the document. Multiple color schemes indicates that the first scheme is preferred by the document, but that the second scheme is acceptable if the user prefers it. Specifying the same color scheme multiple times has the same effect as specifying it once.

`only light`

Indicates that the document *only* supports light mode, with a light background and dark foreground colors. `only dark` *is invalid*, because forcing a document to render in dark mode when it isn't compatible can result in unreadable content and all major browsers default to

light mode if not otherwise configured.

### Examples

#### Using a color-scheme keyword

The following example indicates to the browser that the page supports both light and dark themes. Whether the light or dark color scheme is used depends on user preferences such as OS-level settings or the browser settings:

```
<meta name="color-scheme" content="light dark" />
```

### See also

- `color-scheme` CSS property
- `prefers-color-scheme` media query

## `<meta name="referrer">` HTML attribute value

The **`referrer`** value for the `name` attribute of the `<meta>` element controls the HTTP `Referer` header of requests sent from the document. If specified, you define the referrer using a `content` attribute in the `<meta>` element as a keyword value.

For example, the following `<meta>` element sends the origin of the document as the referrer:

```
<meta name="referrer" content="origin" />
```

> **Warning:** Dynamically inserting `<meta name="referrer">` (with `document.write()` or `appendChild()`) makes the referrer behavior unpredictable. When several conflicting policies are defined, the `no-referrer` policy is applied.

### Usage notes

A `<meta name="referrer">` element has the following additional attributes:

`content`

Sets the document referrer. You must define this attribute. Accepts one of the following values:

`no-referrer`

Does not send an HTTP `Referer` header.

`origin`

Sends the origin of the document.

`no-referrer-when-downgrade`

Sends the full URL when the destination is at least as secure as the current page (HTTP(S)→HTTPS), but sends no referrer when it's less secure (HTTPS→HTTP). This is the default behavior.

`origin-when-cross-origin`

Sends the full URL (stripped of parameters) for same-origin requests, but only sends the origin for other cases.

`same-origin`

Sends the full URL (stripped of parameters) for same-origin requests. Cross-origin requests will contain no referrer header.

`strict-origin`

Sends the origin when the destination is at least as secure as the current page (HTTP(S)→HTTPS), but sends no referrer when it's less secure (HTTPS→HTTP).

`strict-origin-when-cross-origin`

Sends the full URL (stripped of parameters) for same-origin requests. Sends the origin when the destination is at least as secure as the current page (HTTP(S)→HTTPS). Otherwise, sends no referrer.

`unsafe-URL`

Sends the full URL (stripped of parameters) for same-origin or cross-origin requests.

### Examples

### Removing a referrer from requests

The following `<meta>` element specifies that the document shouldn't send a `Referer` header with HTTP requests from the document:

```
<meta name="referrer" content="no-referrer" />
```

### See also

- HTTP `Referer` header

## <meta name="responsive-embedded-sizing">

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`responsive-embedded-sizing`** value for the `name` attribute of a `<meta>` element opts in a document embedded in an `<iframe>` to sharing its size information with the parent document. The frame can then be sized relative to its contained document's layout size using the `frame-sizing` CSS property.

### Description

For security and privacy reasons, `<iframe>` elements do not by default expose any information to the parent document about the size of the content in the document they are embedding.

To enable responsive sizing of `<iframe>` elements based on their content, the `<meta name="responsive-embedded-sizing">` tag can be included in an embedded document to opt it in to sharing its size information with the parent document.

The `frame-sizing` CSS property can then be set on the `<iframe>` to cause it to adopt the same horizontal or vertical size as the embedded document's actual content size (termed the **internal layout intrinsic size** in the spec, but abbreviated to "layout size" in our documentation). This allows `<iframe>` content to fit seamlessly into the embedding element and avoid unnecessary scrollbars.

To resize the `<iframe>` dynamically as the embedded document changes layout size, you can call the `Window.requestResize()` method from the embedded document to make it report an updated size.

### Examples

See the `frame-sizing` and `Window.requestResize()` pages for complete examples.

### See also

- `Window.requestResize()`
- `frame-sizing` CSS property
- CSS box sizing module

## <meta name="robots"> HTML attribute value

The **`robots`** value for the `name` attribute of the `<meta>` element (often called a "robots tag") defines the crawl and indexing behavior that web crawlers should use with the page. If specified, you define instructions for crawlers in the `content` attribute of the `<meta>` element as a comma-separated list of one or more rules.

For example, to hint to crawlers that a page should be excluded from their search indexes, a `noindex` value can be used:

```
<meta name="robots" content="noindex" />
```

> **Note:** Only cooperative robots follow these rules. A crawler still needs to access the resource to read headers and meta elements (see X-Robots-Tag: Interaction with robots.txt). If you want to prevent bandwidth consumption by crawlers, a restrictive robots.txt file is more effective than indexing rules as it blocks resources from being crawled entirely.

### Usage notes

A `<meta name="robots">` element can have the following additional attributes:

`content`

> The `content` attribute must be defined, and its value sets indexing and crawling behavior for cooperative search engine robots. Accepts one or more of the following keywords as a comma-separated list:
>
> `index`
>
> > Allows the robot to index the page. This is the default behavior. Used by all major crawlers.

`noindex`

Requests the robot not to index the page. Used by all major crawlers.

`follow`

Allows the robot to follow links on the page. This is the default behavior. Used by all major crawlers.

`nofollow`

Requests the robot not to follow the links on the page. Used by all major crawlers.

`all`

Equivalent to `index, follow`. Used by: Google.

`none`

Equivalent to `noindex, nofollow`. Used by: Google.

`noarchive`

Requests that the search engine not cache the page content. Used by: Google, Yahoo, Bing.

`nosnippet`

Prevents displaying any description of the page in search engine results. Used by: Google, Bing.

`noimageindex`

Requests that this page not appear as the referring page of an indexed image. Used by: Google.

`nocache`

Synonym of `noarchive`. Used by: Bing.

### Description

There are several important considerations to note when setting a `robots` meta value:

- Only cooperative robots follow these rules. They won't prevent malicious actors like email harvesters from ignoring the directives.
- If defined in a `<meta>` tag, robots still need to access the page to read these rules. To reduce bandwidth, consider using a robots.txt file instead.
- The `<meta name="robots">` tag and `robots.txt` serve different roles: `robots.txt` controls crawling, while the `robots` meta tag influences indexing and other behavior.
- A page blocked by `robots.txt` may still be indexed if linked from other sources.
- The `noindex` directive will only take effect after the robot revisits the page, so ensure `robots.txt` doesn't prevent this.
- Some values, such as `index` vs. `noindex` or `follow` vs. `nofollow`, are mutually exclusive. Behavior is undefined when conflicting values are used.
- Robots like Google, Yahoo, and Bing also support these directives in the HTTP header `X-Robots-Tag`, which is useful for non-HTML content such as PDFs or images.

### Examples

#### Using a robots keyword

The following example uses `nofollow` to request that a crawler doesn't follow links on a page and `noindex` to request that the page is excluded from indexing:

```
<meta name="robots" content="nofollow, noindex" />
```

### Specifications

While not part of any specification, it is a de-facto standard method for communicating with search bots, web crawlers, and similar user agents.

### Browser compatibility

This feature is intended for crawlers to observe, so "browser" compatibility doesn't apply.

### See also

- `X-Robots-Tag` HTTP header
- robots.txt configuration guide
- robots.txt glossary entry
- Search engine glossary entry
- RFC 9309: Robots Exclusion Protocol
- WHATWG Wiki MetaExtensions page

- Using the robots meta tag on developers.google.com

### <meta name="text-scale">

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`text-scale`** value for the `name` attribute of a `<meta>` element enables opting the page in to having the `<html>` root element's initial `font-size` scale in proportion to OS and browser-level text scale settings.

> **Warning:** If you include `<meta name="text-scale" content="scale" />` on your website to opt in to this text scaling behavior, you must test that it supports text sizes up to the maximum text-scaling factor for your target platforms. This typically ranges from 200% to over 300% on mobile platforms, with some accessibility features enabling even larger text scaling. Make sure your website does not look broken for users setting larger or smaller operating system font-size preferences.

#### Usage notes

A `<meta name="text-scale">` element has the following additional attributes:

`content`

Specifies the `font-size` scale opt-in behavior. Its value is a keyword, which can be one of the following:

`scale`

Opts the page in to having the `<html>` root element's initial `font-size` scale in proportion to OS and browser-level text scale settings. It also causes the browser to disable existing browser-based mechanisms and heuristics (for example, text auto-sizing on mobile).

`legacy`

The default value. The page is not opted in to the root element's `font-size` scaling in proportion to OS and browser-level text scale settings. This has the same effect as not including the `<meta>` element at all (OS-level font preferences are ignored.)

#### Description

The `<meta name="text-scale" content="scale" />` element can be included in a document's `<head>` to signal to the browser that the page is sized in a way that will scale well across various user-selected font size preferences; it also disables existing browser-based mechanisms and heuristics.

Specifically, it defines the value of the `<html>` root element's initial `font-size` to scale in proportion to user-defined OS and browser-level font size settings. The `initial` value of the root `font-size` is `medium`, which defines the `rem` unit's value. Provided you set or allow the root `font-size` to default to a local or root font-relative `<length>` unit, any subsequent keyword (like `medium`) or font-relative length (such as `em` and `rem`) will be scaled in proportion to OS or browser font size settings.

For example, with `<meta name="text-scale" content="scale">` included on the page, the following rule:

```
p {
  font-size: medium;
}
```

would result in all `<p>` elements receiving a scaled font size. You could also set `font-size` to `initial` to get the same effect.

On mobile platforms, this is not the case by default. `<meta name="text-scale" content="scale" />` enables this scaling. On desktop platforms, the effect is that the `env(preferred-text-scale)` environment variable is made to reflect the multiplier that corresponds to browser font size settings, but other than that, it has no discernable advantage.

#### Usage summary

To summarize, set `scale` only if your app is designed to support font scaling. Recommended usage is to:

1. Include `<meta name="text-scale" content="scale" />` in the `<head>` of your page.
2. Don't override the initial `:root` `font-size` with an absolute length value (like `16px`).
3. Use only font-relative units such as `em`/`rem` or keywords such as `small`, `x-large`, etc. to size content.

#### <code><meta name="text-scale"></code> versus <code>env(preferred-text-scale)</code>

Using `<meta name="text-scale" />` to size dimensions relative to OS text scale settings is recommended over the `env(preferred-text-scale)` environment variable. They have similar effects on mobile, but `<meta>` also provides improvements for desktop browsers (and is easier to use).

Avoid using both features together, as text scaling may be applied twice, making small font-relative dimensions smaller and large font-relative dimensions larger.

### Examples

#### Meta viewport text-scale demonstration

This example demonstrates the effect of `<meta name="text-scale" content="scale">`.

#### HTML

We include the `<meta name="text-scale" content="scale">` element in the document `<head>`, plus a `<meta name="viewport">` element to ensure correct display on mobile devices. We also include some text content inside `<p>` elements with different `class` attributes set on them to allow us to target them with different styles.

```
<!doctype html>
<html lang="en-US">
  <head>
    <meta name="text-scale" content="scale" />
    <meta name="viewport" content="width=device-width" />
  </head>
  <body>
    <p class="text-scale">
      This font size obeys the user's font preferences, whether those
      preferences are specified at the operating system level or the user agent
      level.
    </p>
    <p class="fixed">
      This font size does NOT respect the user's font preferences, even with
      text-scale set.
      <span class="text-scale">But this font size does!</span>
    </p>
  </body>
</html>
```

#### CSS

Text containers with a `class` of `text-scale` are given a `font-size` of `1rem`, which means that in browsers that support `<meta name="text-scale" content="scale">`, that text will scale as the OS/browser font settings are changed. Text containers with a `class` of `fixed` are given a `font-size` of `20px`, which means that this text will remain at a fixed size as the OS/browser font settings are changed.

```
.text-scale {
  font-size: 1rem;
}

.fixed {
  font-size: 20px;
}
```

#### Result

This version has the `<meta name="text-scale">` element included:

This version does NOT have the `<meta name="text-scale">` element included:

Test these examples in a mobile browser. Change the preferred font size in the mobile device's display or accessibility settings. Note how, in the first example, when `<meta name="text-scale">` is included, the top and bottom lines of text scale in proportion to the OS settings, whereas the middle line, with the `font-size` set using absolute units, doesn't change size. Without the `<meta name="text-scale">` element, text does not scale proportionally with OS settings.

To make testing easier, you can open both versions full screen in a separate tab using the links below:

- Example with `<meta name="text-scale">`
- Example without `<meta name="text-scale">`

#### A text-scale-responsive layout

This example demonstrates that with `<meta name="text-scale">` applied to a page, font-relative sizes can be used inside `@media` queries to cause mobile browsers to automatically adjust breakpoints as the OS text size is changed.

#### HTML

Like the previous example, our markup again includes the `<meta name="text-scale">` and `<meta name="viewport">` elements in the `<head>`. In this demo, the body content contains two elements — `<main>` and `<aside>` — to represent a main content column and a sidebar.

```
<!doctype html>
<html lang="en-US">
```

```
  <head>
    <meta name="text-scale" content="scale" />
    <meta name="viewport" content="width=device-width" />
  </head>
  <body>
    <main>Main content</main>
    <aside>Aside content</aside>
  </body>
</html>
```

### CSS

By default, the main content and sidebar are laid out one below the other. We include a `@media` query that lays the elements out side-by-side using CSS Grid when the viewport becomes wider than `35rem`.

```
@media (width > 35rem) {
  body {
    display: grid;
    gap: 24px;
    grid-template-columns: 1fr 18.75rem;
  }
}
```

### Result

Test this in a mobile browser. You'll see that, as the OS font size is increased, the breakpoint size increases in proportion to it. At larger OS font sizes, the main and aside will start to appear on top of each other whereas previously they appeared side-by-side. You may have to look at it in landscape orientation to see the effect.

You can open the demo in a separate tab using the link below to make testing easier:

Responsive layout example

### See also

- `env(preferred-text-scale)`
- [meta text-scale] Support for WebView on chromium.googlesource.com (2026)

## <meta name="theme-color"> HTML attribute value

The **`theme-color`** value for the `name` attribute of the `<meta>` element indicates a suggested color that user agents should use to customize the display of the page or of the surrounding user interface. If specified, you define a theme color using a `content` attribute in the `<meta>` element as a CSS `<color>` value.

For example, to indicate that a document should use `cornflowerblue` as a theme color, set the `<meta>` as:

```
<meta name="theme-color" content="cornflowerblue" />
```

To set the media to which the theme color metadata applies, include the `media` attribute with a valid media query list (see the `theme-color` media query example).

### Usage notes

A `<meta name="theme-color">` element has the following additional attributes:

`content`

> A `<meta>` element with `name=theme-color` must have a `content` attribute that defines the theme color. The value of `content` attribute is as follows:
>
> `<color>` value
>
> > A valid color value, such as hexadecimal, RGB, named color, etc.

`media` Optional

> Any valid media type or query. If provided, the options for the document's theme color defined in the `content` attribute are suggested to the browser when the media query matches.

### Examples

### Setting a color value

Consider the following code that uses `<meta>` to set a theme color:

```
<meta name="theme-color" content="#4285f4" />
```

The following image shows the effect of this setting in Chrome on an Android mobile device:


Image showing the effect of using theme-color

*Image credit: from Icons & Browser Colors, created and shared by Google and used according to terms described in the Creative Commons 4.0 Attribution License.*

### Using a media query with <code>theme-color</code>

You can provide a media type or query inside the `media` attribute. The `theme-color` will then only be set if the media condition is true. For example:

```
<meta
  name="theme-color"
  content="cornflowerblue"
  media="(prefers-color-scheme: light)" />
<meta
  name="theme-color"
  content="dimgray"
  media="(prefers-color-scheme: dark)" />
```

### See also

- `<meta>` `name` attribute `color-scheme` value
- `color-scheme` CSS property
- `prefers-color-scheme` media query

## <meta name="viewport"> HTML attribute value

The **`viewport`** value for the `name` attribute of a `<meta>` element gives hints about how the viewport should be sized.

If specified, you define viewport-related behaviors using a `content` attribute in the `<meta>` element as a comma-separated list of one or more values.

### Usage notes

A `<meta name="viewport">` element has the following additional attributes:

`content`

> The `content` attribute must be defined, and its value sets various viewport-related behaviors. Its value is a comma-separated list of one or more key-value pairs specified as `key=value`. The following keys are defined:
>
> `width`
>
> > Controls the (minimum) pixel width of the viewport (see viewport width and screen width). It can be set to a positive whole number of pixels between 1 and 10000 (like `width=600`) or to the special value `device-width`, which is the physical size of the device screen in CSS pixels. This value establishes the value of the `vw` unit.
>
> `height`
>
> > Controls the (minimum) pixel height of the viewport (see viewport width and screen width). It can be set to a positive whole number of pixels between 1 and 10000 (like `height=400`) or to the special value `device-height`, which is the physical size of the device screen in CSS pixels. This value establishes the value of the `vh` unit.
>
> `initial-scale`
>
> > Defines the ratio between the device width (`device-width` in portrait mode or `device-height` in landscape mode) and the viewport size. It can be a number between `0.0` and `10.0`.
>
> `maximum-scale`
>
> > Defines the maximum amount to zoom in. It must be greater or equal to the `minimum-scale` or the behavior is undefined. Browser settings can ignore this rule, and iOS10+ ignores it by default. It can be a number between `0.0` and `10.0`.
>
> `minimum-scale`
>
> > Defines the minimum zoom level. It must be smaller or equal to the `maximum-scale` or the behavior is undefined. Browser settings can ignore this rule, and iOS10+ ignores it by default. It can be a number between `0.0` and `10.0`.
>
> `user-scalable`
>
> > A boolean indicating if the user can zoom the web page. Browser settings can ignore this rule, and iOS10+ ignores it by default. It can be either `yes` or `no`, defaulting to `yes`.

> **Warning:** Disabling zooming capabilities by setting `user-scalable` to a value of `no` prevents people experiencing low vision conditions from being able to read and understand page content. Additionally, WCAG requires a minimum of 2× scaling; however, the best practice is to enable a 5× zoom. For more information, see:
>
> - MDN Understanding WCAG, Guideline 1.4 explanations
> - Understanding Success Criterion 1.4.4 | W3C Understanding WCAG 2.0

`interactive-widget`

Specifies the effect that interactive UI widgets, such as virtual keyboards, have on a page's viewport. It can be the keyword `resizes-visual`, `resizes-content`, or `overlays-content`.

- `resizes-visual`: The visual viewport gets resized by the interactive widget. This is the default.
- `resizes-content`: The viewport gets resized by the interactive widget.
- `overlays-content`: Neither the viewport nor the visual viewport gets resized by the interactive widget.

When the viewport gets resized, the initial containing block also gets resized, thereby affecting the computed size of viewport units.

`viewport-fit`

Defines the viewable portions of the web page. It can be one of the keywords `auto`, `contain`, or `cover`.

- `auto`: Doesn't affect the initial layout viewport, and the whole web page is viewable.
- `contain`: The viewport is scaled to fit the largest rectangle inscribed within the display.
- `cover`: The viewport is scaled to fill the device display. It's highly recommended to use the safe area inset variables to ensure that important content doesn't end up outside the display.

### Viewport width and screen width

The browser's viewport is the area of the window in which web content can be seen. The viewport size needs to be computed before the page's content can be laid out—the page can overflow the viewport, in which case the browser provides scrollbars for the user to scroll around and access all the content, but the viewport size is the guideline for how much space, especially horizontally, the content should fit into.

Some mobile devices and other narrow screens render pages in a virtual window or viewport that is wider than the screen, and then shrink the rendered result down to fit the screen size. Users can then zoom and pan to look more closely at different areas of the page. For example, if a mobile screen has a width of 640px, pages might be rendered with a virtual viewport of 980px, and then it will be shrunk down to fit into the 640px space. This is done because not all pages are optimized for mobile and break (or at least look bad) when rendered at a small viewport width. This virtual viewport is a way to make non-mobile-optimized sites in general look better on narrow screen devices. However, this mechanism is not so good for pages that are optimized for narrow screens using media queries — if the virtual viewport is 980px for example, media queries that kick in at 640px or 480px or less will never be used, limiting the effectiveness of such responsive design techniques. The viewport `<meta>` element mitigates this problem of virtual viewport on narrow screen devices.

The recommended setting is the following, which sets the viewport to match the device's width:

```
<meta name="viewport" content="width=device-width" />
```

> **Note:** This example omits `initial-scale=1`, a common practice that's usually unnecessary. When it is omitted, the browser determines the initial zoom level automatically and may zoom out when content overflows the viewport. The amount of scaling is browser-dependent and may depend on both the content's width and height. Add `initial-scale=1` if overflowing content causes unwanted shrinking and you want the page to start at 100% zoom instead; however, you should usually avoid overflowing content horizontally.

Sites can set their viewport to a specific size. For example, the definition `"width=320, initial-scale=1"` can be used to fit precisely onto a small phone display in portrait mode. This can cause problems when the browser renders a page at a larger size. To fix this, browsers will expand the viewport width if necessary to fill the screen at the requested scale. This is especially useful on large-screen devices.

For pages that set an initial or maximum scale, this means the `width` property actually translates into a *minimum* viewport width. For example, if your layout needs at least 500 pixels of width then you can use the following markup. When the screen is more than 500 pixels wide, the browser will expand the viewport (rather than zoom in) to fit the screen:

```
<meta name="viewport" content="width=500, initial-scale=1" />
```

### Screen density

Screen resolutions have risen to the size that individual pixels are indistinguishable by the human eye. For example, smartphones often have small screens with resolutions upwards of 1920–1080 pixels (≈400dpi). Because of this, many browsers can display their pages in a smaller physical size by translating multiple hardware pixels for each CSS "pixel". Initially, this caused usability and readability problems on many touch-optimized websites.

On high dpi screens, pages with `initial-scale=1` will effectively be zoomed by browsers. Their text will be smooth and crisp, but their bitmap images may not take advantage of the full screen resolution. To get sharper images on these screens, web developers may want to design images – or whole

layouts – at a higher scale than their final size and then scale them down using CSS or viewport properties.

The default pixel ratio depends on the display density. On a display with density less than 200dpi, the ratio is 1.0. On displays with density between 200 and 300dpi, the ratio is 1.5. For displays with density over 300dpi, the ratio is the integer floor (*density*/150dpi). Note that the default ratio is true only when the viewport scale equals 1. Otherwise, the relationship between CSS pixels and device pixels depends on the current zoom level.

### Examples

#### Using a meta viewport size

The following example indicates to the browser that the page should be rendered at the device width:

```
<meta name="viewport" content="width=device-width" />
```

#### Using a media query with a viewport meta

The following `content` value uses multiple keywords that hint to the browser to use fullscreen mode, along with `viewport-fit`, which helps avoid display cutouts such as mobile device notches:

```
<meta name="viewport" content="width=device-width, viewport-fit=cover" />
```

#### The effect of interactive UI widgets

Interactive UI widgets of the browser can influence the size of the page's viewports. The most common such UI widget is a virtual keyboard. To control which resize behavior the browser should use, set the `interactive-widget` property.

By default, the virtual keyboard only resizes the visual viewport, which doesn't affect the layout of the page. You can make the page's layout adapt to the presence of the virtual keyboard by setting the `interactive-widget` property to `resizes-content`:

```
<meta name="viewport" content="interactive-widget=resizes-content" />
```

### See also

- viewport glossary term
- Prepare for viewport resize behavior changes coming to Chrome on Android on developer.chrome.com
- Mobile viewports for responsive experiences on Adobe Experience League

## <meter> HTML meter element

The **`<meter>`** HTML element represents either a scalar value within a known range or a fractional value.

### Try it

```
<label for="fuel">Fuel level:</label>

<meter id="fuel" min="0" max="100" low="33" high="66" optimum="80" value="50">
  at 50/100
</meter>
```

```
label {
  padding-right: 10px;
  font-size: 1rem;
}
```

### Attributes

This element includes the global attributes.

`value`

The current numeric value. This must be between the minimum and maximum values (`min` attribute and `max` attribute) if they are specified. If unspecified or malformed, the value is `0`. If specified, but not within the range given by the `min` attribute and `max` attribute, the value is equal to the nearest end of the range.

> **Note:** Unless the `value` attribute is between `0` and `1` (inclusive), the `min` and `max` attributes should define the range so that the `value` attribute's value is within it.

`min`

The lower numeric bound of the measured range. This must be less than the maximum value (`max` attribute), if specified. If unspecified, the minimum value is `0`.

`max`

The upper numeric bound of the measured range. This must be greater than the minimum value (`min` attribute), if specified. If unspecified, the maximum value is `1`.

`low`

The upper numeric bound of the low end of the measured range. This must be greater than the minimum value (`min` attribute), and it also must be less than the high value and maximum value (`high` attribute and `max` attribute, respectively), if any are specified. If unspecified, or if less than the minimum value, the `low` value is equal to the minimum value.

`high`

The lower numeric bound of the high end of the measured range. This must be less than the maximum value (`max` attribute), and it also must be greater than the low value and minimum value (`low` attribute and `min` attribute, respectively), if any are specified. If unspecified, or if greater than the maximum value, the `high` value is equal to the maximum value.

`optimum`

This attribute indicates the optimal numeric value. It must be within the range (as defined by the `min` attribute and `max` attribute). When used with the `low` attribute and `high` attribute, it gives an indication where along the range is considered preferable. For example, if it is between the `min` attribute and the `low` attribute, then the lower range is considered preferred. The browser may color the meter's bar differently depending on whether the value is less than or equal to the optimum value.

### Examples

#### Basic Example

#### HTML

```
<p>Battery level: <meter min="0" max="100" value="75">75%</meter></p>
```

#### Result

#### High and low range example

Note that in this example the `min` attribute is omitted. This is allowed, as it will default to `0`.

#### HTML

```
<p>
  Student's exam score:
  <meter low="50" high="80" max="100" value="84">84%</meter>
</p>
```

#### Result

### Technical Summary

| | |
|---|---|
| **Content categories** | Flow content, phrasing content, labelable content, palpable content. |
| **Permitted content** | Phrasing content, but there must be no `<meter>` element among its descendants. |
| **Tag omission** | None, both the starting and ending tag are mandatory. |
| **Permitted parents** | Any element that accepts phrasing content. |
| **Implicit ARIA role** | `meter` |
| **Permitted ARIA roles** | No `role` permitted |
| **DOM interface** | `HTMLMeterElement` |

### See also

- Creating vertical form controls
- `<progress>`
- `::-webkit-meter-bar`, `::-webkit-meter-inner-element`, `::-webkit-meter-even-less-good-value`, `::-webkit-meter-optimum-value`, `::-webkit-meter-suboptimum-value`: non-standard pseudo-elements

## <nav> HTML navigation section element

The **`<nav>`** HTML element represents a section of a page whose purpose is to provide navigation links, either within the current document or to other documents. Common examples of navigation sections are menus, tables of contents, and indexes.

### Try it

```
<nav class="crumbs">
  <ol>
    <li class="crumb"><a href="#">Bikes</a></li>
    <li class="crumb"><a href="#">BMX</a></li>
    <li class="crumb">Jump Bike 3000</li>
  </ol>
</nav>

<h1>Jump Bike 3000</h1>
<p>
  This BMX bike is a solid step into the pro world. It looks as legit as it
  rides and is built to polish your skills.
</p>
```

```
nav {
  border-bottom: 1px solid black;
}

.crumbs ol {
  list-style-type: none;
  padding-left: 0;
}

.crumb {
  display: inline-block;
}

.crumb a::after {
  display: inline-block;
  color: black;
  content: ">";
  font-size: 80%;
  font-weight: bold;
  padding: 0 3px;
}
```

### Attributes

This element only includes the global attributes.

### Usage notes

- It's not necessary for all links to be contained in a `<nav>` element. `<nav>` is intended only for a major block of navigation links; typically the `<footer>` element often has a list of links that don't need to be in a `<nav>` element.
- A document may have several `<nav>` elements, for example, one for site navigation and one for intra-page navigation. `aria-labelledby` can be used in such case to promote accessibility, see example.
- User agents, such as screen readers targeting disabled users, can use this element to determine whether to omit the initial rendering of navigation-only content.

### Examples

In this example, a `<nav>` block is used to contain an unordered list (`<ul>`) of links. With appropriate CSS, this can be presented as a sidebar, navigation bar, or drop-down menu.

```
<nav class="menu">
  <ul>
    <li><a href="#">Home</a></li>
    <li><a href="#">About</a></li>
    <li><a href="#">Contact</a></li>
  </ul>
</nav>
```

The semantics of the `nav` element is that of providing links. However a `nav` element doesn't have to contain a list, it can contain other kinds of content as well. In this navigation block, links are provided in prose:

```
<nav>
  <h2>Navigation</h2>
  <p>
    You are on my home page. To the north lies <a href="https://developer.mozilla.org/blog">my blog</a>, from
    whence the sounds of battle can be heard. To the east you can see a large
    mountain, upon which many <a href="https://developer.mozilla.org/school">school papers</a> are littered.
    Far up this mountain you can spy a little figure who appears to be me,
    desperately scribbling a <a href="https://developer.mozilla.org/school/thesis">thesis</a>.
  </p>
```

```
  <p>
    To the west are several exits. One fun-looking exit is labeled
    <a href="https://games.example.com/">"games"</a>. Another more
    boring-looking exit is labeled <a href="https://isp.example.net/">ISP™</a>.
  </p>
  <p>
    To the south lies a dark and dank <a href="https://developer.mozilla.org/about">contacts page</a>.
    Cobwebs cover its disused entrance, and at one point you see a rat run
    quickly out of the page.
  </p>
</nav>
```

### Technical summary

| | |
|---|---|
| **Content categories** | Flow content, sectioning content, palpable content. |
| **Permitted content** | Flow content. |
| **Tag omission** | None, both the starting and ending tag are mandatory. |
| **Permitted parents** | Any element that accepts flow content. |
| **Implicit ARIA role** | `navigation` |
| **Permitted ARIA roles** | No `role` permitted |
| **DOM interface** | `HTMLElement` |

### See also

- Other section-related elements: `<body>`, `<article>`, `<section>`, `<aside>`, h1, h2, h3, h4, h5, h6, `<hgroup>`, `<header>`, `<footer>`, `<address>`;
- Sections and outlines of an HTML document.
- ARIA: Navigation role

## <nobr> HTML non-breaking text element

The **`<nobr>`** HTML element prevents the text it contains from automatically wrapping across multiple lines, potentially resulting in the user having to scroll horizontally to see the entire width of the text.

> **Warning:** Although this element is widely supported, it was *never* standard HTML, so you shouldn't use it. Instead, use the CSS property `white-space` like this:

```
<span class="nobr">Long line with no breaks</span>
```

```
.nobr {
  white-space: nowrap;
}
```

### See also

- `white-space`
- `overflow`

## <noembed> HTML embed fallback element

The **`<noembed>`** HTML element is an obsolete, non-standard way to provide alternative, or "fallback", content for browsers that do not support the `<embed>` element or do not support the type of embedded content an author wishes to use. This element was deprecated in HTML 4.01 and above in favor of placing fallback content between the opening and closing tags of an `<object>` element.

> **Note:** While this element currently still works in many browsers, it is obsolete and should not be used. Use `<object>` instead, with fallback content between the opening and closing tags of the element.

### Examples

The message inside `<noembed>` tag will appear only when your browser does not support `<embed>` tag.

### Show an alternative content

```
<embed
  type="video/webm"
  src="https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/noembed//media/examples/flower.mp4"
  width="200"
```

```
  height="200" />
<noembed>
  <h1>Alternative content</h1>
</noembed>
```

## <noframes> HTML frame fallback element

The **`<noframes>`** HTML element provides content to be presented in browsers that don't support (or have disabled support for) the `<frame>` element. Although most commonly-used browsers support frames, there are exceptions, including certain special-use browsers including some mobile browsers, as well as text-mode browsers.

A `<noframes>` element can contain any HTML elements that are allowed within the body of an HTML document, except for the `<frameset>` and `<frame>` elements, since using frames when they aren't supported doesn't make sense.

`<noframes>` can be used to present a message explaining that the user's browser doesn't support frames, but ideally should be used to present an alternate form of the site that doesn't use frames but still offers the same or similar functionality.

> **Note:** This element is obsolete and shouldn't be used, since the `<frame>` and `<frameset>` elements are also obsolete. When frames are needed at all, they should be presented using the `<iframe>` element.

### Attributes

Like all other HTML elements, this element supports the global attributes. It has no other attributes available.

### Example

In this example, we see a frameset with two frames. In addition, `<noframes>` is used to present an explanatory message if the user agent doesn't support frames.

```
<!doctype html>
<html lang="en-US">
  <head>
    <!-- Document metadata goes here -->
  </head>
  <frameset rows="45%, 55%">
    <frame src="https://developer.mozilla.org/en/HTML/Element/frameset" />
    <frame src="https://developer.mozilla.org/en/HTML/Element/frame" />
    <noframes>
      <p>
        It seems your browser does not support frames or is configured to not
        allow them.
      </p>
    </noframes>
  </frameset>
</html>
```

### See also

- `<frameset>`
- `<frame>`

## <noscript> HTML noscript element

The **`<noscript>`** HTML element defines a section of HTML to be inserted if a script type on the page is unsupported or if scripting is currently turned off in the browser.

### Attributes

This element only includes the global attributes.

### Examples

```
<noscript>
  <!-- anchor linking to external file -->
  <a href="https://www.mozilla.org/">External Link</a>
</noscript>
<p>Rocks!</p>
```

### Result with scripting enabled

Rocks!

### Result with scripting disabled

External Link

Rocks!

### Usage notes

The `<noscript>` element represents its children differently depending on if scripting is enabled:

- If scripting is disabled the `<noscript>` element represents its children as HTML content.
- If scripting is enabled, the `<noscript>` element represents its children as text.

### Technical summary

| | |
|---|---|
| **Content categories** | Metadata content, flow content, phrasing content. |
| **Permitted content** | When scripting is disabled and when it is a descendant of the `<head>` element: in any order, zero or more `<link>` elements, zero or more `<style>` elements, and zero or more `<meta>` elements.<br>When scripting is disabled and when it isn't a descendant of the `<head>` element: any transparent content, but no `<noscript>` element must be among its descendants.<br>Otherwise: flow content or phrasing content. |
| **Tag omission** | None, both the starting and ending tag are mandatory. |
| **Permitted parents** | Any element that accepts phrasing content, if there are no ancestor `<noscript>` element, or in a `<head>` element (but only for an HTML document), here again if there are no ancestor `<noscript>` element. |
| **Implicit ARIA role** | No corresponding role |
| **Permitted ARIA roles** | No `role` permitted |
| **DOM interface** | `HTMLElement` |

## <object> HTML external object element

The **`<object>`** HTML element represents an external resource, which can be treated as an image, a nested browsing context, or a resource to be handled by a plugin.

### Try it

```
<object
  type="video/mp4"
  data="/shared-assets/videos/flower.mp4"
  width="250"
  height="200"></object>
```

### Attributes

This element includes the global attributes.

`archive`

A space-separated list of URIs for archives of resources for the object.

`border`

The width of a border around the control, in pixels.

`classid`

The URI of the object's implementation. It can be used together with, or in place of, the **data** attribute.

`codebase`

The base path used to resolve relative URIs specified by **classid**, **data**, or **archive**. If not specified, the default is the base URI of the current document.

`codetype`

The content type of the data specified by **classid**.

`data`

The address of the resource as a valid URL. At least one of **data** and **type** must be defined.

`declare`

The presence of this Boolean attribute makes this element a declaration only. The object must be instantiated by a subsequent `<object>` element. Repeat the `<object>` element completely each time the resource is reused.

`form`

The form element, if any, that the object element is associated with (its *form owner*). The value of the attribute must be an ID of a `<form>` element in the same document.

`height`

The height of the displayed resource, as in `<integer>` in CSS pixels.

`name`

The name of valid browsing context (HTML5), or the name of the control (HTML 4). The name becomes a property of the `Window` and `Document` objects, containing a reference to the embedded window or the element itself.

`standby`

A message that the browser can show while loading the object's implementation and data.

`type`

The content type of the resource specified by **data**. At least one of **data** and **type** must be defined.

`usemap`

A hash-name reference to a `<map>` element; that is a '#' followed by the value of a `name` of a map element.

`width`

The width of the display resource, as in `<integer>` in CSS pixels.

### Usage notes

The `<object>` element can display PDFs using the browser's built-in PDF viewer. Its children provide fallback content when the resource cannot be displayed. You can use this to provide a link to the PDF, but also provide a link outside the element so it remains available if the viewer loads but fails to display the PDF.

Unlike `<iframe>`, `<object>` has no attributes that control loading. The embedding page's Content Security Policy controls its sources through `object-src`.

### Examples

#### Embed a video

#### HTML

```
<object
  type="video/webm"
  data="/shared-assets/videos/flower.webm"
  width="600"
  height="140">
  <img
    src="https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/object//shared-assets/images/examples/flowers.jpg"
    alt="Some beautiful flowers" />
</object>
```

#### Result

If the video in the example fails to load, the user will be provided with an image as fallback content. The `<img>` tag is used to display an image. We include the `src` attribute set to the path to the image we want to embed. We also include the `alt` attribute, which provides the image with an accessible name. If the image also fails to load, the content of the `alt` attribute will be displayed.

### Technical summary

| | |
|---|---|
| **Content categories** | Flow content; phrasing content; embedded content, palpable content; if the element has a `usemap` attribute, interactive content; listed, submittable form-associated element. |
| **Permitted content** | zero or more `<param>` elements, then transparent. |

| | |
|---|---|
| **Tag omission** | None, both the starting and ending tag are mandatory. |
| **Permitted parents** | Any element that accepts embedded content. |
| **Implicit ARIA role** | No corresponding role |
| **Permitted ARIA roles** | `application`, `document`, `img` |
| **DOM interface** | `HTMLObjectElement` |

### See also

- `<embed>`
- `<param>`

## <ol> HTML ordered list element

The **`<ol>`** HTML element represents an ordered list of items — typically rendered as a numbered list.

### Try it

```
<ol>
  <li>Mix flour, baking powder, sugar, and salt.</li>
  <li>In another bowl, mix eggs, milk, and oil.</li>
  <li>Stir both mixtures together.</li>
  <li>Fill muffin tray 3/4 full.</li>
  <li>Bake for 20 minutes.</li>
</ol>
```

```
li {
  font:
    1rem "Fira Sans",
    sans-serif;
  margin-bottom: 0.5rem;
}
```

### Attributes

This element also accepts the global attributes.

`compact`

This Boolean attribute hints that the list should be rendered in a compact style. The interpretation of this attribute is browser-specific. Use CSS instead: to give a similar effect as the `compact` attribute, the CSS property `line-height` can be used with a value of `80%`.

`reversed`

This Boolean attribute specifies that the list's items are in reverse order. Items will be numbered from high to low.

`start`

An integer to start counting from for the list items. Always an Arabic numeral (1, 2, 3, etc.), even when the numbering `type` is letters or Roman numerals. For example, to start numbering elements from the letter "d" or the Roman numeral "iv," use `start="4"`.

`type`

Sets the numbering type:

- `a` for lowercase letters
- `A` for uppercase letters
- `i` for lowercase Roman numerals
- `I` for uppercase Roman numerals
- `1` for numbers (default)

The specified type is used for the entire list unless a different `type` attribute is used on an enclosed `<li>` element.

> **Note:** Unless the type of the list number matters (like legal or technical documents where items are referenced by their number/letter), use the CSS `list-style-type` property instead.

### Usage notes

Typically, ordered list items display with a preceding marker, such as a number or letter.

The `<ol>` and `<ul>` (or the synonym `<menu>`) elements may nest as deeply as desired, alternating between `<ol>`, `<ul>` (or `<menu>`) as needed. To nest a list, place it inside an `<li>` element of the parent list; a list element can not be a direct child of another `<ul>` or `<ol>`.

The `<ol>` and `<ul>` elements both represent a list of items. The difference is with the `<ol>` element, the order is meaningful. For example:

- Steps in a recipe
- Turn-by-turn directions
- The list of ingredients in decreasing proportion on nutrition information labels

To determine which list to use, try changing the order of the list items; if the meaning changes, use the `<ol>` element — otherwise you can use `<ul>` otherwise, or `<menu>` if your list is a menu.

### Examples

#### Basic example

```
<ol>
  <li>Fee</li>
  <li>Fi</li>
  <li>Fo</li>
  <li>Fum</li>
</ol>
```

#### Result

#### Using Roman Numeral type

```
<ol type="i">
  <li>Introduction</li>
  <li>List of Grievances</li>
  <li>Conclusion</li>
</ol>
```

#### Result

#### Using the start attribute

```
<p>Finishing places of contestants not in the winners' circle:</p>

<ol start="4">
  <li>Speedwalk Stu</li>
  <li>Saunterin' Sam</li>
  <li>Slowpoke Rodriguez</li>
</ol>
```

#### Result

#### Nesting lists

```
<ol>
  <li>first item</li>
  <li>
    second item
    <!-- closing </li> tag is not here! -->
    <ol>
      <li>second item first subitem</li>
      <li>second item second subitem</li>
      <li>second item third subitem</li>
    </ol>
  </li>
  <!-- Here's the closing </li> tag -->
  <li>third item</li>
</ol>
```

#### Result

#### Unordered list inside ordered list

```
<ol>
  <li>first item</li>
  <li>
    second item
```

```
    <!-- closing </li> tag is not here! -->
    <ul>
      <li>second item first subitem</li>
      <li>second item second subitem</li>
      <li>second item third subitem</li>
    </ul>
  </li>
  <!-- Here's the closing </li> tag -->
  <li>third item</li>
</ol>
```

### Result

### Technical summary

| | |
|---|---|
| **Content categories** | Flow content, and if the `<ol>` element's children include at least one `<li>` element, palpable content. |
| **Permitted content** | Zero or more `<li>`, `<script>` and `<template>` elements. |
| **Tag omission** | None, both the starting and ending tag are mandatory. |
| **Permitted parents** | Any element that accepts flow content. |
| **Implicit ARIA role** | `list` |
| **Permitted ARIA roles** | `directory`, `group`, `listbox`, `menu`, `menubar`, `none`, `presentation`, `radiogroup`, `tablist`, `toolbar`, `tree` |
| **DOM interface** | `HTMLOListElement` |

### See also

- Other list-related HTML Elements: `<ul>`, `<li>`, `<menu>`
- CSS properties that may be specially useful to style the `<ol>` element:
  - the `list-style` property, to choose the way the ordinal displays
  - CSS counters, to handle complex nested lists
  - the `line-height` property, to simulate the deprecated `compact` attribute
  - the `margin` property, to control the list indentation

## <optgroup> HTML option group element

The **`<optgroup>`** HTML element creates a grouping of options within a `<select>` element.

### Try it

```
<label for="dino-select">Choose a dinosaur:</label>
<select id="dino-select">
  <optgroup label="Theropods">
    <option>Tyrannosaurus</option>
    <option>Velociraptor</option>
    <option>Deinonychus</option>
  </optgroup>
  <optgroup label="Sauropods">
    <option>Diplodocus</option>
    <option>Saltasaurus</option>
    <option>Apatosaurus</option>
  </optgroup>
</select>
```

```
label {
  display: block;
  margin-bottom: 10px;
}
```

### Attributes

This element includes the global attributes.

`disabled`

> If this Boolean attribute is set, none of the items in this option group is selectable. Often browsers gray out such control and it won't receive any browsing events, like mouse clicks or focus-related ones.

`label`

> The name of the group of options, which the browser can use when labeling the options in the user interface. This attribute is mandatory if this element is used.

### Usage notes

Optgroup elements may not be nested.

In customizable `<select>` elements, the `<legend>` element is allowed as a child of `<optgroup>`, to provide a label that is easy to target and style. This replaces any text set in the `<optgroup>` element's `label` attribute, and it has the same semantics.

### Examples

```
<select>
  <optgroup label="Group 1">
    <option>Option 1.1</option>
  </optgroup>
  <optgroup label="Group 2">
    <option>Option 2.1</option>
    <option>Option 2.2</option>
  </optgroup>
  <optgroup label="Group 3" disabled>
    <option>Option 3.1</option>
    <option>Option 3.2</option>
    <option>Option 3.3</option>
  </optgroup>
</select>
```

### Result

### Technical summary

| | |
|---|---|
| **Content categories** | None. |
| **Permitted content** | Zero or more `<option>` elements. In customizable select elements, a `<legend>` element is permitted as a child of `<optgroup>`. |
| **Tag omission** | The start tag is mandatory. The end tag is optional if this element is immediately followed by another `<optgroup>` element, or if the parent element has no more content. |
| **Permitted parents** | A `<select>` element. |
| **Implicit ARIA role** | `group` |
| **Permitted ARIA roles** | No `role` permitted |
| **DOM interface** | `HTMLOptGroupElement` |

### See also

- Other form-related elements: `<form>`, `<legend>`, `<label>`, `<button>`, `<select>`, `<datalist>`, `<option>`, `<fieldset>`, `<textarea>`, `<input>`, `<output>`, `<progress>` and `<meter>`.
- Customizable select elements

## <option> HTML option element

The **`<option>`** HTML element is used to define an item contained in a `<select>`, an `<optgroup>`, or a `<datalist>` element. As such, `<option>` can represent menu items in popups and other lists of items in an HTML document.

### Try it

```
<label for="pet-select">Choose a pet:</label>

<select id="pet-select">
  <option value="">--Please choose an option--</option>
  <option value="dog">Dog</option>
  <option value="cat">Cat</option>
  <option value="hamster">Hamster</option>
  <option value="parrot">Parrot</option>
  <option value="spider">Spider</option>
  <option value="goldfish">Goldfish</option>
</select>
```

```
label {
  font-family: sans-serif;
  font-size: 1rem;
```

```
  padding-right: 10px;
}

select {
  font-size: 0.9rem;
  padding: 2px 5px;
}
```

### Attributes

This element includes the global attributes.

`disabled`

> If this Boolean attribute is set, this option is not checkable. Often browsers gray out such control and it won't receive any browsing event, like mouse clicks or focus-related ones. If this attribute is not set, the element can still be disabled if one of its ancestors is a disabled `<optgroup>` element.

`label`

> This attribute is text for the label indicating the meaning of the option. If the `label` attribute isn't defined, its value is that of the element text content.

`selected`

> If present, this Boolean attribute indicates that the option is initially selected. If the `<option>` element is the descendant of a `<select>` element whose `multiple` attribute is not set, only one single `<option>` of this `<select>` element may have the `selected` attribute.

`value`

> The content of this attribute represents the value to be submitted with the form, should this option be selected. If this attribute is omitted, the value is taken from the text content of the option element.

### Styling with CSS

Styling `<option>` elements has historically been highly limited. Customizable select elements explains newer features that enable their full customization, just like any regular DOM element.

### Legacy option styling

In browsers that don't support the modern customization features (or legacy codebases where they can't be used), the styling available on `<option>` elements depends on the browser and operating system. Depending on the operating system, the `font-size` of the owning `<select>` is respected in Firefox and Chromium. Chromium may additionally allow `color`, `background-color`, `font-family`, `font-variant`, and `text-align` to be set.

You can find more details about legacy `<option>` styling in our guide to advanced form styling.

### Examples

See `<select>` for examples.

### Technical summary

| | |
|---|---|
| **Content categories** | None. |
| **Permitted content** | In traditional `<select>` elements, only text content is permitted, possibly with escaped characters (like `é`). In customizable select elements, `<option>` elements can have any arbitrary content. |
| **Tag omission** | The start tag is mandatory. The end tag is optional if this element is immediately followed by another `<option>` element or an `<optgroup>`, or if the parent element has no more content. |
| **Permitted parents** | A `<select>`, an `<optgroup>` or a `<datalist>` element. |
| **Implicit ARIA role** | `option` |
| **Permitted ARIA roles** | No `role` permitted |
| **DOM interface** | `HTMLOptionElement` |

### See also

- Other form-related elements: `<form>`, `<legend>`, `<label>`, `<button>`, `<select>`, `<datalist>`, `<optgroup>`, `<fieldset>`, `<textarea>`, `<input>`, `<output>`, `<progress>` and `<meter>`.
- Customizable select elements

## <output> HTML output element

The **<output>** HTML element is a container element into which a site or app can inject the results of a calculation or the outcome of a user action.

### Attributes

This element includes the global attributes.

`for`

A space-separated list of other elements' `id`s, indicating that those elements contributed input values to (or otherwise affected) the calculation.

`form`

The `<form>` element to associate the output with (its *form owner*). The value of this attribute must be the `id` of a `<form>` in the same document. (If this attribute is not set, the `<output>` is associated with its ancestor `<form>` element, if any.)

This attribute lets you associate `<output>` elements to `<form>`s anywhere in the document, not just inside a `<form>`. It can also override an ancestor `<form>` element. The `<output>` element's name and content are not submitted when the form is submitted.

`name`

The element's name. Used in the `form.elements` API.

The `<output>` value, name, and contents are NOT submitted during form submission.

### Accessibility

Many browsers implement this element as an `aria-live` region. Assistive technology will thereby announce the results of UI interactions posted inside it without requiring that focus is switched away from the controls that produce those results.

Live-region updates are generally announced as plain text, so links, buttons, and other semantics nested inside `<output>` may not be conveyed in the announcement.

### Examples

In the following example, we provide a slider whose value can range between `0` and `100`, and an `<input>` element into which you can enter a second number. The two numbers are added together, and the result is displayed in the `<output>` element each time the value of any of the controls changes.

```
<input type="range" id="b" name="b" value="50" /> +
<input type="number" id="a" name="a" value="10" /> =
<output id="result" for="a b">60</output>
```

```
const a = document.getElementById("a");
const b = document.getElementById("b");
const result = document.getElementById("result");

function updateResult() {
  const aValue = a.valueAsNumber;
  const bValue = b.valueAsNumber;
  result.value = aValue + bValue;
}

a.addEventListener("input", updateResult);
b.addEventListener("input", updateResult);

updateResult();
```

### Result

### Technical summary

| | |
|---|---|
| **Content categories** | Flow content, phrasing content, listed, labelable, resettable form-associated element, palpable content. |
| **Permitted content** | Phrasing content. |
| **Tag omission** | None, both the starting and ending tag are mandatory. |
| **Permitted parents** | Any element that accepts phrasing content. |
| **Implicit ARIA role** | `status` |
| **Permitted ARIA roles** | Any |
| **DOM interface** | `HTMLOutputElement` |

## <p> HTML paragraph element

The **<p>** HTML element represents a paragraph. Paragraphs are usually represented in visual media as blocks of text separated from adjacent blocks by blank lines and/or first-line indentation, but HTML paragraphs can be any structural grouping of related content, such as images or form fields.

Paragraphs are block-level elements, and notably will automatically close if another block-level element is parsed before the closing `</p>` tag. See "Tag omission" below.

### Try it

```
<p>
  Geckos are a group of usually small, usually nocturnal lizards. They are found
  on every continent except Antarctica.
</p>

<p>
  Some species live in houses where they hunt insects attracted by artificial
  light.
</p>
```

```
p {
  margin: 10px 0;
  padding: 5px;
  border: 1px solid #999999;
}
```

### Attributes

This element only includes the global attributes.

> **Note:** The `align` attribute on `<p>` tags is obsolete and shouldn't be used.

### Accessibility

Breaking up content into paragraphs helps make a page more accessible. Screen-readers and other assistive technology provide shortcuts to let their users skip to the next or previous paragraph, letting them skim content like how white space lets visual users skip around.

Using empty `<p>` elements to add space between paragraphs is problematic for people who navigate with screen-reading technology. Screen readers may announce the paragraph's presence, but not any content contained within it — because there is none. This can confuse and frustrate the person using the screen reader.

If extra space is desired, use CSS properties like `margin` to create the effect:

```
p {
  margin-bottom: 2em; /* increase white space after a paragraph */
}
```

### Examples

#### HTML

```
<p>
  This is the first paragraph of text. This is the first paragraph of text. This
  is the first paragraph of text. This is the first paragraph of text.
</p>
<p>
  This is the second paragraph. This is the second paragraph. This is the second
  paragraph. This is the second paragraph.
</p>
```

#### Result

### Styling paragraphs

By default, browsers separate paragraphs with a single blank line. Alternate separation methods, such as first-line indentation, can be achieved with CSS:

#### HTML

```
<p>
  Separating paragraphs with blank lines is easiest for readers to scan, but
  they can also be separated by indenting their first lines. This is often used
```

```
  to take up less space, such as to save paper in print.
</p>

<p>
  Writing that is intended to be edited, such as school papers and rough drafts,
  uses both blank lines and indentation for separation. In finished works,
  combining both is considered redundant and amateurish.
</p>

<p>
  In very old writing, paragraphs were separated with a special character: ¶,
  the <i>pilcrow</i>. Nowadays, this is considered claustrophobic and hard to
  read.
</p>

<p>
  How hard to read? See for yourself:
  <button data-toggle-text="Oh no! Switch back!">
    Use pilcrow for paragraphs
  </button>
</p>
```

### CSS

```
p {
  margin: 0;
  text-indent: 3ch;
}

p.pilcrow {
  text-indent: 0;
  display: inline;
}
p.pilcrow + p.pilcrow::before {
  content: " ¶ ";
}
```

### JavaScript

```
document.querySelector("button").addEventListener("click", (event) => {
  document.querySelectorAll("p").forEach((paragraph) => {
    paragraph.classList.toggle("pilcrow");
  });

  [event.target.innerText, event.target.dataset.toggleText] = [
    event.target.dataset.toggleText,
    event.target.innerText,
  ];
});
```

### Result

### Technical summary

| | |
|---|---|
| **Content categories** | Flow content, palpable content. |
| **Permitted content** | Phrasing content. |
| **Tag omission** | The start tag is required. The end tag may be omitted if the `<p>` element is immediately followed by an `<address>`, `<article>`, `<aside>`, `<blockquote>`, `<details>`, `<div>`, `<dl>`, `<fieldset>`, `<figcaption>`, `<figure>`, `<footer>`, `<form>`, h1, h2, h3, h4, h5, h6, `<header>`, `<hgroup>`, `<hr>`, `<main>`, `<menu>`, `<nav>`, `<ol>`, `<pre>`, `<search>`, `<section>`, `<table>`, `<ul>` or another `<p>` element, or if there is no more content in the parent element and the parent element is not an `<a>`, `<audio>`, `<del>`, `<ins>`, `<map>`, `<noscript>` or `<video>` element, or an autonomous custom element. |
| **Permitted parents** | Any element that accepts flow content. |
| **Implicit ARIA role** | paragraph |
| **Permitted ARIA roles** | Any |

| DOM interface | HTMLParagraphElement |
| --- | --- |

### See also

- <hr>
- <br>

## <param> HTML object parameter element

The **<param>** HTML element defines parameters for an <object> element.

> **Note:** Use the <object> element with a data attribute to set the URL of an external resource.

### Attributes

This element includes the global attributes.

name

Name of the parameter.

value

Specifies the value of the parameter.

type

Only used if the valuetype is set to ref. Specifies the MIME type of values found at the URI specified by value.

valuetype

Specifies the type of the value attribute. Possible values are:

- data: Default value. The value is passed to the object's implementation as a string.
- ref: The value is a URI to a resource where run-time values are stored.
- object: An ID of another <object> in the same document.

### Technical summary

| Content categories | None. |
| --- | --- |
| Permitted content | None; it is a void element. |
| Tag omission | Must have a start tag and must not have an end tag. |
| Permitted parents | An <object> before any flow content. |
| Implicit ARIA role | No corresponding role |
| Permitted ARIA roles | No role permitted |
| DOM interface | HTMLParamElement |

### See also

- <object>

## <picture> HTML picture element

The **<picture>** HTML element contains zero or more <source> elements and one <img> element to offer alternative versions of an image for different display/device scenarios.

The browser will consider each child <source> element and choose the best match among them. If no matches are found—or the browser doesn't support the <picture> element—the URL of the <img> element's src attribute is selected. The selected image is then presented in the space occupied by the <img> element.

### Try it

```
<!--Change the browser window width to see the image change.-->

<picture>
  <source
    srcset="/shared-assets/images/examples/surfer.jpg"
    media="(orientation: portrait)" />
```

```
  <img src="https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/picture//shared-assets/images/examples/painted-hand.jpg" alt="" />
</picture>
```

### Attributes

This element includes only global attributes.

### Usage notes

To decide which URL to load, the user agent examines each `<source>`'s `srcset`, `media`, and `type` attributes to select a compatible image that best matches the current layout and capabilities of the display device.

The `<img>` element serves two purposes:

1. It describes the size and other attributes of the image and its presentation.
2. It provides a fallback in case none of the offered `<source>` elements are able to provide a usable image.

Common use cases for `<picture>`:

- **Art direction.** Cropping or modifying images for different `media` conditions (for example, loading a simpler version of an image which has too many details, on smaller displays).
- **Offering alternative image formats**, for cases where certain formats are not supported.

  > **Note:** For example, newer formats like AVIF or WEBP have many advantages, but might not be supported by the browser. A list of supported image formats can be found in: Image file type and format guide.

- **Saving bandwidth and speeding page load times** by loading the most appropriate image for the viewer's display.

If providing higher-density versions of an image for high-DPI (Retina) display, use `srcset` on the `<img>` element instead. This lets browsers opt for lower-density versions in data-saving modes, and you don't have to write explicit `media` conditions.

You can use the `object-position` property to adjust the positioning of the image within the element's frame, and the `object-fit` property to control how the image is resized to fit within the frame.

> **Note:** Use these properties on the child `<img>` element, **not** the `<picture>` element.

### Examples

These examples demonstrate how different attributes of the `<source>` element change the selection of the image inside `<picture>`.

#### The media attribute

The `media` attribute specifies a media condition (similar to a media query) that the user agent will evaluate for each `<source>` element.

If the `<source>`'s media condition evaluates to `false`, the browser skips it and evaluates the next element inside `<picture>`.

```
<picture>
  <source srcset="mdn-logo-wide.png" media="(width >= 600px)" />
  <img src="https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/picture/mdn-logo-narrow.png" alt="MDN" />
</picture>
```

You can swap image assets for light and dark themes using the `prefers-color-scheme` media feature:

```
<picture>
  <source srcset="logo-dark.png" media="(prefers-color-scheme: dark)" />
  <source srcset="logo-light.png" media="(prefers-color-scheme: light)" />
  <img src="https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/picture/logo-light.png" alt="Product logo" />
</picture>
```

#### The srcset attribute

The srcset attribute is used to offer a list of possible images based on size or the display's pixel density.

It is composed of a comma-separated list of image descriptors. Each image descriptor is composed of a URL of the image, and *either*:

- a *width descriptor*, followed by a `w` (such as `300w`); *OR*
- a *pixel density descriptor*, followed by an `x` (such as `2x`) to serve a high-res image for high-DPI screens.

Make sure to note that:

- width and pixel density descriptors should not be used together
- a missing pixel density descriptor implies 1x
- duplicate descriptor values are not allowed (2x & 2x, 100w & 100w)

The following example illustrates the usage of `srcset` attribute with the `<source>` element to specify a high-density and standard-resolution image:

```
<picture>
  <source srcset="logo.png, logo-1.5x.png 1.5x" />
  <img src="https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/picture/logo.png" alt="MDN Web Docs logo" height="320"
width="320" />
</picture>
```

The `srcset` attribute can also be used on the `<img>` element without needing the `<picture>` element. The following example demonstrates how to use the `srcset` attribute to specify standard-resolution and high-density images, respectively:

```
<img
  srcset="logo.png, logo-2x.png 2x"
  src="https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/picture/logo.png"
  height="320"
  width="320"
  alt="MDN Web Docs logo" />
```

### The sizes attribute

The `sizes` attribute of the `<source>` element allows you to specify a set of media condition-length pairs and indicate the image display size for each condition. This helps the browser select the most appropriate image from the `srcset` attribute, which lists images with their intrinsic widths.

The browser evaluates the media conditions in the sizes attribute before it downloads any images. See the sizes attribute of the `<img>` and `<source>` elements for more information.

For example:

```
<picture>
  <source
    srcset="small.jpg 480w, medium.jpg 800w, large.jpg 1200w"
    sizes="(max-width: 600px) 400px, 800px"
    type="image/jpeg" />
  <img src="https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/picture/fallback.jpg" alt="Example image" />
</picture>
```

In this example:

- If the viewport is 600px wide or less, the slot size is 400px; otherwise, it is 800px.
- The browser multiplies the slot size by the device pixel ratio to determine the ideal image width, then picks the closest available image from `srcset`.

Without sizes, the browser will use the default size of the image as specified by its dimensions in pixels. This may not be the best fit for all devices, especially if the image is displayed on different screen sizes or in different contexts.

Please note that sizes will have its effect only if width dimension descriptors are provided with srcset instead of pixel ratio values (200w instead of 2x for example). For more information on using `srcset`, see the Responsive images documentation.

### The type attribute

The `type` attribute specifies a MIME type for the resource URL(s) in the `<source>` element's `srcset` attribute. If the user agent does not support the given type, the `<source>` element is skipped.

```
<picture>
  <source srcset="photo.avif" type="image/avif" />
  <source srcset="photo.webp" type="image/webp" />
  <img src="https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/picture/photo.jpg" alt="photo" />
</picture>
```

### Technical summary

| | |
|---|---|
| **Content categories** | Flow content, phrasing content, embedded content |
| **Permitted content** | Zero or more `<source>` elements, followed by one `<img>` element, optionally intermixed with script-supporting elements. |
| **Tag omission** | None, both the starting and ending tag are mandatory. |
| **Permitted parents** | Any element that allows embedded content. |
| **Implicit ARIA role** | No corresponding role |

| | |
|---|---|
| **Permitted ARIA roles** | No `role` permitted |
| **DOM interface** | `HTMLPictureElement` |

### See also

- `<img>` element
- `<source>` element
- Positioning and sizing the picture within its frame: `object-position` and `object-fit`
- Image file type and format guide
- `prefers-color-scheme` media feature

## <plaintext> HTML plain text element

The **`<plaintext>`** HTML element renders everything following the start tag as raw text, ignoring any following HTML. There is no closing tag, since everything after it is considered raw text.

> **Warning:** Do not use this element.
>
> - `<plaintext>` is deprecated since HTML 2, and not all browsers implemented it. Browsers that did implement it didn't do so consistently.
> - `<plaintext>` is obsolete; browsers that accept it may instead treat it as a `<pre>` element that still interprets HTML within.
> - If `<plaintext>` is the first element on the page (other than any non-displayed elements, like `<head>`), do not use HTML at all. Instead serve a text file with the `text/plain` MIME-type.
> - Instead of `<plaintext>`, use the `<pre>` element or, if semantically accurate (such as for inline text), the `<code>` element. Escape any `<`, `>` and `&` characters, to prevent browsers inadvertently parsing the element content as HTML.
> - A monospaced font can be applied to any HTML element via a CSS `font-family` style with the `monospace` generic value.

### Attributes

This element has no other attributes than the global attributes common to all elements.

### DOM interface

This element implements the `HTMLElement` interface.

### See also

- The `<pre>` and `<code>` elements, which should be used instead.
- The `<xmp>` element, also obsolete, similar to `<plaintext>`.

## <pre> HTML preformatted text element

The **`<pre>`** HTML element represents preformatted text which is to be presented exactly as written in the HTML file. The text is typically rendered using a non-proportional, or monospaced font.

Whitespace inside this element is displayed as written, with one exception. If one or more leading newline characters are included immediately following the opening `<pre>` tag, the *first* newline character is stripped. This transformation is done by the HTML parser and does not apply when using XHTML.

`<pre>` elements' text content is parsed as HTML, so if you want to ensure that your text content stays as plain text, some syntax characters, such as `<`, may need to be escaped using their respective character references. See escaping ambiguous characters for more information.

`<pre>` elements commonly contain `<code>`, `<samp>`, and `<kbd>` elements, to represent computer code, computer output, and user input, respectively.

By default, `<pre>` is a block-level element, i.e., its default `display` value is `block`.

### Try it

```
<pre>
             S
             A
            LUT
             M
            O N
            D  E
            DONT
          JE SUIS
          LA  LAN
          G U E  É
         L O Q U E N
        TE      QUESA
```

```
      B  O  U  C  H  E
     O       P A R I S
    T I R E   ET   TIRERA
   T O U            JOURS
  AUX                 A  L
 LEM                    ANDS   - Apollinaire
</pre>
```

```
pre {
  font-size: 0.7rem;
  margin: 0;
}
```

### Attributes

This element only includes the global attributes.

`width`

> Contains the *preferred* count of characters that a line should have. Though technically still implemented, this attribute has no visual effect; to achieve such an effect, use CSS `width` instead.

`wrap`

> Is a *hint* indicating how the overflow must happen. In modern browser this hint is ignored and no visual effect results in its present; to achieve such an effect, use CSS `white-space` instead.

### Accessibility

It is important to provide an alternate description for any images or diagrams created using preformatted text. The alternate description should clearly and concisely describe the image or diagram's content.

People experiencing low vision conditions and browsing with the aid of assistive technology such as a screen reader may not understand what the preformatted text characters are representing when they are read out in sequence.

A combination of the `<figure>` and `<figcaption>` elements, supplemented by the ARIA `role` and `aria-label` attributes on the `pre` element allow the preformatted ASCII art to be announced as an image with alternative text, and the `figcaption` serving as the image's caption.

### Example

```
<figure>
  <pre role="img" aria-label="ASCII COW">
      _____________________________
  &lt; I'm an expert in my field. &gt;
      -----------------------------
          \   ^__^
           \  (oo)\_______
              (__)\       )\/\
                  ||----w |
                  ||     ||
  </pre>
  <figcaption id="cow-caption">
    A cow saying, "I'm an expert in my field." The cow is illustrated using
    preformatted text characters.
  </figcaption>
</figure>
```

- MDN Understanding WCAG, Guideline 1.1 explanations
- H86: Providing text alternatives for ASCII art, emoticons, and leetspeak | W3C Techniques for WCAG 2.0

### Examples

### Basic example

### HTML

```
<p>Using CSS to change the font color is easy.</p>
<pre><code>
body {
  color: red;
}
</code></pre>
```

### Result

### Escaping ambiguous characters

Suppose you want to demonstrate HTML code in a `<pre>` element. The character sequences that define valid HTML tags (starting with `<` and ending with `>`) will not be displayed. To display the tag characters as text, you need to escape (at least) the `<` character using its character reference, so that the sequences no longer define valid tags.

In reality, the HTML parser treats most characters as plain text unless in specific contexts. For example, `< code` is fine, but `<code` would be misparsed; `&am;` is fine, but `&` is not. However, it's a good practice to escape all ambiguous characters to avoid any confusion, especially if you are programmatically generating HTML and injecting the `<pre>` content. In this case, here's a good rule of thumb for how to escape characters:

1. First, write the content out, as you would like it to appear in the HTML document.
2. Replace any ampersands (`&`) with `&`. Do this step first, so that new `&` characters generated in the next step don't get escaped.
3. Replace any `<` characters with `<`.

This should result in the content being displayed as you intended. The replacement of other HTML syntax characters is optional (like `>` to `>`, `"` to `"`, and `'` to `'`), but will do no harm.

#### HTML

```
<pre><code>
let i = 5;

if (i &lt; 10 &amp;amp;&amp;amp; i &gt; 0)
  return &amp;quot;Single Digit Number&amp;quot;;
</code></pre>
```

#### Result

#### Technical summary

| | |
|---|---|
| **Content categories** | Flow content, palpable content. |
| **Permitted content** | Phrasing content. |
| **Tag omission** | None, both the starting and ending tag are mandatory. |
| **Permitted parents** | Any element that accepts flow content. |
| **Implicit ARIA role** | `generic` |
| **Permitted ARIA roles** | Any |
| **DOM interface** | `HTMLPreElement` |

#### See also

- CSS: `white-space`, `word-break`
- Character reference
- Related element: `<code>`, `<samp>`, `<kbd>`

## <progress> HTML progress indicator element

The **`<progress>`** HTML element displays an indicator showing the completion progress of a task, typically displayed as a progress bar.

#### Try it

```
<label for="file">File progress:</label>

<progress id="file" max="100" value="70">70%</progress>
```

```
label {
  padding-right: 10px;
  font-size: 1rem;
}
```

#### Attributes

This element includes the global attributes.

`max`

> This attribute describes how much work the task indicated by the `progress` element requires. The `max` attribute, if present, must have a value greater than `0` and be a valid floating point number. The default value is `1`.

`value`

This attribute specifies how much of the task that has been completed. It must be a valid floating point number between `0` and `max`, or between `0` and `1` if `max` is omitted. If there is no `value` attribute, the progress bar is indeterminate; this indicates that an activity is ongoing with no indication of how long it is expected to take.

> **Note:** Unlike the `<meter>` element, the minimum value is always 0, and the `min` attribute is not allowed for the `<progress>` element.

> **Note:** The `:indeterminate` pseudo-class can be used to match against indeterminate progress bars. To change the progress bar to indeterminate after giving it a value you must remove the value attribute with `element.removeAttribute('value')`.

### Accessibility

#### Labeling

In most cases you should provide an accessible label when using `<progress>`. While you can use the standard ARIA labeling attributes `aria-labelledby` or `aria-label` as you would for any element with `role="progressbar"`, when using `<progress>` you can alternatively use the `<label>` element.

> **Note:** Text placed between the element's tags is not an accessible label, it is only recommended as a fallback for old browsers that do not support this element.

#### Examples

```
<label>
  Uploading Document: <progress value="70" max="100">70 %</progress>
</label>

<!-- OR -->
<br />

<label for="progress-bar">Uploading Document</label>
<progress id="progress-bar" value="70" max="100">70 %</progress>
```

#### Result

#### Describing a particular region

If the `<progress>` element is describing the loading progress of a section of a page, use `aria-describedby` to point to the status, and set `aria-busy="true"` on the section that is being updated, removing the `aria-busy` attribute when it has finished loading.

#### Examples

```
<div aria-busy="true" aria-describedby="progress-bar">
  <!-- content is for this region is loading -->
</div>

<!-- ... -->

<progress id="progress-bar" aria-label="Content loading…"></progress>
```

#### Result

#### Examples

```
<progress value="70" max="100">70 %</progress>
```

#### Result

### Technical summary

| | |
|---|---|
| **Content categories** | Flow content, phrasing content, labelable content, palpable content. |
| **Permitted content** | Phrasing content, but there must be no `<progress>` element among its descendants. |
| **Tag omission** | None, both the starting and ending tag are mandatory. |
| **Permitted parents** | Any element that accepts phrasing content. |
| **Implicit ARIA role** | `progressbar` |
| **Permitted ARIA roles** | No `role` permitted |

| DOM interface | `HTMLProgressElement` |
| --- | --- |

### See also

- Creating vertical form controls
- `<meter>`
- `:indeterminate`
- `-moz-orient`
- `::-moz-progress-bar`
- `::-webkit-progress-bar`
- `::-webkit-progress-value`
- `::-webkit-progress-inner-element`

## <q> HTML inline quotation element

The **`<q>`** HTML element indicates that the enclosed text is a short inline quotation. Most modern browsers implement this by surrounding the text in quotation marks. This element is intended for short quotations that don't require paragraph breaks; for long quotations use the `<blockquote>` element.

### Try it

```
<p>
  When Dave asks HAL to open the pod bay door, HAL answers:
  <q
    cite="https://www.imdb.com/title/tt0062622/quotes/?item=qt0396921&ref_=ext_shr_lnk">
    I'm sorry, Dave. I'm afraid I can't do that.
  </q>
</p>
```

```
q {
  font-style: italic;
}
```

### Attributes

This element includes the global attributes.

`cite`

> The value of this attribute is a URL that designates a source document or message for the information quoted. This attribute is intended to point to information explaining the context or the reference for the quote.

### Examples

```
<p>
  According to Mozilla's website,
  <q cite="https://www.mozilla.org/en-US/about/history/details/"
    >Firefox 1.0 was released in 2004 and became a big success.</q
  >
</p>
```

### Result

### Technical summary

| **Content categories** | Flow content, phrasing content, palpable content. |
| --- | --- |
| **Permitted content** | Phrasing content. |
| **Tag omission** | None, both the starting and ending tag are mandatory. |
| **Permitted parents** | Any element that accepts phrasing content. |
| **Implicit ARIA role** | `generic` |
| **Permitted ARIA roles** | Any |
| **DOM interface** | `HTMLQuoteElement` |

### See also

- The `<blockquote>` element for long quotations.
- The `<cite>` element for source citations.

## <rb> HTML ruby base element

The **<rb>** HTML element is used to delimit the base text component of a <ruby> annotation, i.e., the text that is being annotated. One <rb> element should wrap each separate atomic segment of the base text.

### Attributes

This element only includes the global attributes.

### Usage notes

- Ruby annotations are for showing pronunciation of East Asian characters, like using Japanese furigana or Taiwanese bopomofo characters. The <rb> element is used to separate out each segment of the ruby base text.
- Even though <rb> is not a void element, it is common to just include the opening tag of each element in the source code, so that the ruby markup is less complex and easier to read. The browser can then fill in the full element in the rendered version.
- You need to include one <rt> element for each base segment/<rb> element that you want to annotate.

### Examples

### Using rb

In this example, we provide an annotation for the original character equivalent of "Kanji":

```
<ruby>
  <rb>漢</rb><rb>字 </rb><rp>(</rp><rt>kan</rt><rt>ji</rt><rp>)</rp>
</ruby>
```

Note how we've included two <rb> elements, to delimit the two separate parts of the ruby base text. The annotation on the other hand is delimited by two <rt> elements.

### Result

### Separate annotations

Note that we could also write this example with the two base text parts annotated completely separately. In this case we don't need to include <rb> elements:

```
<ruby>
  漢 <rp>(</rp><rt>Kan</rt><rp>)</rp> 字 <rp>(</rp><rt>ji</rt><rp>)</rp>
</ruby>
```

### Result

See the article about the <ruby> element for further examples.

### Technical summary

| | |
|---|---|
| **Content categories** | None. |
| **Permitted content** | As a child of a <ruby> element. |
| **Tag omission** | The end tag can be omitted if the element is immediately followed by an <rt>, <rtc>, or <rp> element or another <rb> element, or if there is no more content in the parent element. |
| **Permitted parents** | A <ruby> element. |
| **Permitted ARIA roles** | Any |
| **DOM interface** | HTMLElement |

### See also

- <ruby>
- <rt>
- <rp>
- <rtc>
- CSS ruby layout module

## <rp> HTML ruby fallback parenthesis element

The **<rp>** HTML element is used to provide fall-back parentheses for browsers that do not support display of ruby annotations using the <ruby> element. One <rp> element should enclose each of the opening and closing parentheses that wrap the <rt> element that contains the annotation's

text.

### Try it

```
<ruby>
  漢 <rp>(</rp><rt>kan</rt><rp>)</rp> 字 <rp>(</rp><rt>ji</rt><rp>)</rp>
</ruby>
```

```
ruby {
  font-size: 2em;
}
```

### Attributes

This element only includes the global attributes.

### Usage notes

- Ruby annotations are for showing pronunciation of East Asian characters, like using Japanese furigana or Taiwanese bopomofo characters. The `<rp>` element is used in the case of lack of `<ruby>` element support; the `<rp>` content provides what should be displayed in order to indicate the presence of a ruby annotation, usually parentheses.

### Examples

### Using ruby annotations

This example uses ruby annotations to display the Romaji equivalents for each character.

```
<ruby>
  漢 <rp>(</rp><rt>Kan</rt><rp>)</rp> 字 <rp>(</rp><rt>ji</rt><rp>)</rp>
</ruby>
```

### Result

See the article about the `<ruby>` element for further examples.

### Without ruby support

If your browser does not support ruby annotations, the result looks like this instead:

### Technical summary

| | |
|---|---|
| **Content categories** | None. |
| **Permitted content** | Text |
| **Tag omission** | The end tag can be omitted if the element is immediately followed by an `<rt>` or another `<rp>` element, or if there is no more content in the parent element. |
| **Permitted parents** | A `<ruby>` element. `<rp>` must be positioned immediately before or after an `<rt>` element. |
| **Implicit ARIA role** | No corresponding role |
| **Permitted ARIA roles** | Any |
| **DOM interface** | `HTMLElement` |

### See also

- `<ruby>`
- `<rt>`
- `<rb>`
- `<rtc>`
- CSS ruby layout module

## <rt> HTML ruby text element

The **`<rt>`** HTML element specifies the ruby text component of a ruby annotation, which is used to provide pronunciation, translation, or transliteration information for East Asian typography. The `<rt>` element must always be contained within a `<ruby>` element.

### Try it

```
<ruby>
  漢 <rp>(</rp><rt>kan</rt><rp>)</rp> 字 <rp>(</rp><rt>ji</rt><rp>)</rp>
</ruby>
```

```
ruby {
  font-size: 2em;
}
```

### Attributes

This element only includes the global attributes.

### Examples

### Using ruby annotations

This example provides Romaji transliteration for the kanji characters within the `<ruby>` element:

```
<ruby> 漢 <rt>Kan</rt> 字 <rt>ji</rt> </ruby>
```

### Result

See the `<ruby>` element for more examples.

### Technical summary

| | |
|---|---|
| **Content categories** | None. |
| **Permitted content** | Phrasing content. |
| **Tag omission** | The end tag may be omitted if the `<rt>` element is immediately followed by an `<rt>` or `<rp>` element, or if there is no more content in the parent element |
| **Permitted parents** | A `<ruby>` element. |
| **Implicit ARIA role** | No corresponding role |
| **Permitted ARIA roles** | Any |
| **DOM interface** | `HTMLElement` |

### See also

- `<ruby>`
- `<rp>`
- `<rb>`
- `<rtc>`
- `text-transform: full-size-kana`
- CSS ruby layout module

## <rtc> HTML ruby text container element

The **`<rtc>`** HTML element embraces semantic annotations of characters presented in a ruby of `<rb>` elements used inside of `<ruby>` element. `<rb>` elements can have both pronunciation (`<rt>`) and semantic (`<rtc>`) annotations.

### Try it

```
<ruby lang="zh-Hant">
  <rbc>
    <rb>馬</rb><rp>(</rp><rt>mǎ</rt><rp>)</rp>
    <rb>來</rb><rp>(</rp><rt>lái</rt><rp>)</rp>
    <rb>西</rb><rp>(</rp><rt>xī</rt><rp>)</rp>
    <rb>亞</rb><rp>(</rp><rt>yà</rt><rp>)</rp>
  </rbc>
  <rtc lang="en">
    <rp>(</rp><rt>Malaysia</rt><rp>)</rp>
  </rtc>
</ruby>
```

```
ruby {
  font-size: 2em;
```

```
  ruby-position: under;
}

rtc {
  ruby-position: over;
}
```

### Attributes

This element only includes the global attributes.

### Examples

```
<div class="info">
  <ruby>
    <rbc>
      <rb>旧</rb><rt>jiù</rt> <rb>金</rb><rt>jīn</rt> <rb>山</rb><rt>shān</rt>
    </rbc>
    <rtc>San Francisco</rtc>
  </ruby>
</div>
```

### Result

### Technical summary

| | |
|---|---|
| **Content categories** | None. |
| **Permitted content** | Phrasing content or `<rt>` elements. |
| **Tag omission** | The closing tag can be omitted if it is immediately followed by a `<rb>`, `<rtc>` or `<rt>` element opening tag or by its parent closing tag. |
| **Permitted parents** | A `<ruby>` element. |
| **Permitted ARIA roles** | Any |
| **DOM interface** | `HTMLElement` |

### See also

- `<ruby>`
- `<rp>`
- `<rb>`
- `<rt>`
- CSS ruby layout module

## <ruby> HTML ruby annotation element

The **`<ruby>`** HTML element represents small annotations that are rendered above, below, or next to base text, usually used for showing the pronunciation of East Asian characters. It can also be used for annotating other kinds of text, but this usage is less common.

The term *ruby* originated as a unit of measurement used by typesetters, representing the smallest size that text can be printed on newsprint while remaining legible.

### Try it

```
<ruby> 明日 <rp>(</rp><rt>Ashita</rt><rp>)</rp> </ruby>
```

```
ruby {
  font-size: 2em;
}
```

### Attributes

This element only includes the global attributes.

### Examples

### Example 1: Character

```
<ruby>
  漢 <rp>(</rp><rt>Kan</rt><rp>)</rp> 字 <rp>(</rp><rt>ji</rt><rp>)</rp>
</ruby>
```

### Result

### Example 2: Word

```
<ruby> 明日 <rp>(</rp><rt>Ashita</rt><rp>)</rp> </ruby>
```

### Result

### Technical summary

| | |
|---|---|
| **Content categories** | Flow content, phrasing content, palpable content. |
| **Permitted content** | One or more groups, each consisting of two parts:<br>1. The base text, which is either:<br>  ◦ Phrasing content, but with no `<ruby>` elements and with no `<ruby>` element descendants, or<br>  ◦ A single `<ruby>` element that itself has no `<ruby>` element descendants.<br>2. The annotations for the base text, which is either:<br>  ◦ One or more `<rt>` elements, or<br>  ◦ An `<rp>` element followed by one or more `<rt>` elements, each of which is itself followed by an `<rp>` element (that is, `rp, rt, rp, rt, ..., rp`). |
| **Tag omission** | None, both the starting and ending tag are mandatory. |
| **Permitted parents** | Any element that accepts phrasing content. |
| **Implicit ARIA role** | No corresponding role |
| **Permitted ARIA roles** | Any |
| **DOM interface** | `HTMLElement` |

### See also

- `<rt>`
- `<rp>`
- `ruby-overhang`
- `text-transform`: full-size-kana
- CSS ruby layout module

## <s> HTML strikethrough element

The **`<s>`** HTML element renders text with a strikethrough, or a line through it. Use the `<s>` element to represent things that are no longer relevant or no longer accurate. However, `<s>` is not appropriate when indicating document edits; for that, use the `<del>` and `<ins>` elements, as appropriate.

### Try it

```
<p><s>There will be a few tickets available at the box office tonight.</s></p>

<p>SOLD OUT!</p>
```

```
s {
  /* Add your styles here */
}
```

### Attributes

This element only includes the global attributes.

### Accessibility

The presence of the `s` element is not announced by most screen reading technology in its default configuration. It can be made to be announced by using the CSS `content` property, along with the `::before` and `::after` pseudo-elements.

```
s::before,
s::after {
  clip-path: inset(100%);
  clip: rect(1px, 1px, 1px, 1px);
  height: 1px;
  overflow: hidden;
```

```
  position: absolute;
  white-space: nowrap;
  width: 1px;
}

s::before {
  content: " [start of stricken text] ";
}

s::after {
  content: " [end of stricken text] ";
}
```

Some people who use screen readers deliberately disable announcing content that creates extra verbosity. Because of this, it is important to not abuse this technique and only apply it in situations where not knowing content has been struck out would adversely affect understanding.

- Tweaking Text Level Styles, Reprised | Adrian Roselli

### Examples

```
.sold-out {
  text-decoration: line-through;
}
```

```
<s>Today's Special: Salmon</s> SOLD OUT<br />
<span class="sold-out">Today's Special: Salmon</span> SOLD OUT
```

### Result

### Technical summary

| | |
|---|---|
| **Content categories** | Phrasing content, flow content. |
| **Permitted content** | Phrasing content. |
| **Tag omission** | None, both the starting and ending tag are mandatory. |
| **Permitted parents** | Any element that accepts phrasing content. |
| **Implicit ARIA role** | `deletion` |
| **Permitted ARIA roles** | Any |
| **DOM interface** | `HTMLElement` |

### See also

- The `<strike>` element, alter ego of the `<s>` element, is obsolete and should not be used on websites anymore.
- The `<del>` element is to be used instead if the data has been *deleted*.
- The CSS `text-decoration-line` property is to be used to achieve the former visual aspect of the `<s>` element.

## <samp> HTML sample output element

The **`<samp>`** HTML element is used to enclose inline text which represents sample (or quoted) output from a computer program. Its contents are typically rendered using the browser's default monospaced font (such as Courier or Lucida Console).

### Try it

```
<p>I was trying to boot my computer, but I got this hilarious message:</p>

<p>
  <samp>Keyboard not found <br />Press F1 to continue</samp>
</p>
```

```
samp {
  font-weight: bold;
}
```

### Attributes

This element only includes the global attributes.

### Usage notes

You can use a CSS rule to override the browser's default font face for the `<samp>` element; however, it's possible that the browser's preferences may take precedence over any CSS you specify.

The CSS to override the default font face would look like this:

```
samp {
  font-family: "Courier";
}
```

> **Note:** If you need an element which will serve as a container for output generated by your website or app's JavaScript code, you should instead use the `<output>` element.

### Examples

#### Basic example

In this basic example, a paragraph includes an example of the output of a program.

```
<p>
  When the process is complete, the utility will output the text
  <samp>Scan complete. Found <em>N</em> results.</samp> You can then proceed to
  the next step.
</p>
```

#### Result

#### Sample output including user input

You can nest the `<kbd>` element within a `<samp>` block to present an example that includes text entered by the user. For example, consider this text presenting a transcript of a Linux (or macOS) console session:

#### HTML

```
<pre>
<samp><span class="prompt">mike@interwebz:~$</span> <kbd>md5 -s "Hello world"</kbd>
MD5 ("Hello world") = 3e25960a79dbc69b674cd4ec67a72c62

<span class="prompt">mike@interwebz:~$</span> <span class="cursor">&#9608;</span></samp></pre>
```

Note the use of `<span>` to allow customization of the appearance of specific portions of the sample text such as the shell prompts and the cursor. Note also the use of `<kbd>` to represent the command the user entered at the prompt in the sample text.

#### CSS

The CSS that achieves the appearance we want is:

```
.prompt {
  color: #bb0000;
}

samp > kbd {
  font-weight: bold;
}

.cursor {
  color: #0000bb;
}
```

This gives the prompt and cursor fairly subtle colorization and emboldens the keyboard input within the sample text.

#### Result

The resulting output is this:

### Technical summary

| | |
|---|---|
| **Content categories** | Flow content, phrasing content, palpable content. |
| **Permitted content** | Phrasing content. |
| **Tag omission** | None, both the starting and ending tag are mandatory. |
| **Permitted parents** | Any element that accepts phrasing content. |
| **Implicit ARIA role** | `generic` |
| **Permitted ARIA roles** | Any |

| DOM interface | HTMLElement |
| --- | --- |

### See also

- Related elements: <kbd>, <code>, <pre>
- The <output> element: a container for script-generated output

## <script> HTML script element

The **<script>** HTML element is used to embed executable code or data; this is typically used to embed or refer to JavaScript code. The <script> element can also be used with other languages, such as WebGL's GLSL shader programming language and JSON.

### Attributes

This element includes the global attributes.

async

For classic scripts, if the async attribute is present, then the classic script will be fetched in parallel to parsing and evaluated as soon as it is available.

For module scripts, if the async attribute is present then the scripts and all their dependencies will be fetched in parallel to parsing and evaluated as soon as they are available.

> **Warning:** This attribute must not be used if the src attribute is absent (i.e., for inline scripts) for classic scripts, in this case it would have no effect.

This attribute allows the elimination of **parser-blocking JavaScript** where the browser would have to load and evaluate scripts before continuing to parse. defer has a similar effect in this case.

If the attribute is specified with the defer attribute, the element will act as if only the async attribute is specified.

This is a boolean attribute: the presence of a boolean attribute on an element represents the true value, and the absence of the attribute represents the false value.

See Browser compatibility for notes on browser support. See also Async scripts for asm.js.

attributionsrc

Specifies that you want the browser to send an Attribution-Reporting-Eligible header along with the script resource request. On the server-side this is used to trigger sending an Attribution-Reporting-Register-Source or Attribution-Reporting-Register-Trigger header in the response, to register a JavaScript-based attribution source or attribution trigger, respectively. Which response header should be sent back depends on the value of the Attribution-Reporting-Eligible header that triggered the registration.

> **Note:** Alternatively, JavaScript-based attribution sources or triggers can be registered by sending a fetch() request containing the attributionReporting option (either set directly on the fetch() call or on a Request object passed into the fetch() call), or by sending an XMLHttpRequest with setAttributionReporting() invoked on the request object.

There are two versions of this attribute that you can set:

- Boolean, i.e., just the attributionsrc name. This specifies that you want the Attribution-Reporting-Eligible header sent to the same server as the src attribute points to. This is fine when you are handling the attribution source or trigger registration on the same server. When registering an attribution trigger this property is optional, and an empty string value will be used if it is omitted.
- Value containing one or more URLs, for example:

```
<script
  src="https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/script/myscript.js"
  attributionsrc="https://a.example/register-source https://b.example/register-source"></script>
```

This is useful in cases where the requested resource is not on a server you control, or you just want to handle registering the attribution source on a different server. In this case, you can specify one or more URLs as the value of attributionsrc. When the resource request occurs the Attribution-Reporting-Eligible header will be sent to the URL(s) specified in attributionSrc in addition to the resource origin. These URLs can then respond with an Attribution-Reporting-Register-Source or Attribution-Reporting-Register-Trigger header as appropriate to complete registration.

> **Note:** Specifying multiple URLs means that multiple attribution sources can be registered on the same feature. You might for example have different campaigns that you are trying to measure the success of, which involve generating different reports on

> different data.

See the Attribution Reporting API for more details.

`blocking`

This attribute explicitly indicates that certain operations should be blocked until the script has executed. The operations that are to be blocked must be a space-separated list of blocking tokens. Currently there is only one token:

- `render`: The rendering of content on the screen is blocked.

> **Note:** Only `script` elements in the document's `<head>` can possibly block rendering. Scripts are not render-blocking by default; if a `script` element does not include `type="module"`, `async`, or `defer`, then it blocks *parsing*, not *rendering*. If such a `script` element is added dynamically via script, you must set `blocking = "render"` for it to block rendering.

`crossorigin`

Normal `script` elements pass minimal information to the `window.onerror` for scripts which do not pass the standard CORS checks. To allow error logging for sites which use a separate domain for static media, use this attribute. See CORS settings attributes for a more descriptive explanation of its valid arguments.

`defer`

This Boolean attribute is set to indicate to a browser that the script is meant to be executed after the document has been parsed, but before firing `DOMContentLoaded` event.

Scripts with the `defer` attribute will prevent the `DOMContentLoaded` event from firing until the script has loaded and finished evaluating.

> **Warning:** This attribute must not be used if the `src` attribute is absent (i.e., for inline scripts), in this case it would have no effect.
>
> The `defer` attribute has no effect on module scripts — they defer by default.

Scripts with the `defer` attribute will execute in the order in which they appear in the document.

This attribute allows the elimination of **parser-blocking JavaScript** where the browser would have to load and evaluate scripts before continuing to parse. `async` has a similar effect in this case.

If the attribute is specified with the `async` attribute, the element will act as if only the `async` attribute is specified.

`fetchpriority`

Provides a hint of the relative priority to use when fetching an external script. Allowed values:

`high`

Fetch the external script at a high priority relative to other external scripts.

`low`

Fetch the external script at a low priority relative to other external scripts.

`auto`

Don't set a preference for the fetch priority. This is the default. It is used if no value or an invalid value is set.

`integrity`

This attribute contains one or more hashes of the script. It is used to ensure that the content of the script is what the developer expects it to be, and has not been replaced with a malicious script in a supply chain attack. The attribute must not be specified when the `src` attribute is absent. See also Subresource Integrity.

`nomodule`

This Boolean attribute is set to indicate that the script should not be executed in browsers that support ES modules — in effect, this can be used to serve fallback scripts to older browsers that do not support modular JavaScript code.

`nonce`

A cryptographic nonce (number used once) to allow scripts in a script-src Content-Security-Policy. The server must generate a unique nonce value each time it transmits a policy. It is critical to provide a nonce that cannot be guessed as bypassing a resource's policy is otherwise trivial.

`referrerpolicy`

Indicates which referrer to send when fetching the script, or resources fetched by the script:

- `no-referrer`: The `Referer` header will not be sent.
- `no-referrer-when-downgrade`: The `Referer` header will not be sent to origins without TLS (HTTPS).
- `origin`: The sent referrer will be limited to the origin of the referring page: its scheme, host, and port.
- `origin-when-cross-origin`: The referrer sent to other origins will be limited to the scheme, the host, and the port. Navigations on the same origin will still include the path.
- `same-origin`: A referrer will be sent for same origin, but cross-origin requests will contain no referrer information.
- `strict-origin`: Only send the origin of the document as the referrer when the protocol security level stays the same (HTTPS→HTTPS), but don't send it to a less secure destination (HTTPS→HTTP).
- `strict-origin-when-cross-origin` (default): Send a full URL when performing a same-origin request, only send the origin when the protocol security level stays the same (HTTPS→HTTPS), and send no header to a less secure destination (HTTPS→HTTP).
- `unsafe-url`: The referrer will include the origin *and* the path (but not the fragment, password, or username). **This value is unsafe**, because it leaks origins and paths from TLS-protected resources to insecure origins.

> **Note:** An empty string value (`""`) is both the default value, and a fallback value if `referrerpolicy` is not supported. If `referrerpolicy` is not explicitly specified on the `<script>` element, it will adopt a higher-level referrer policy, i.e., one set on the whole document or domain. If a higher-level policy is not available, the empty string is treated as being equivalent to `strict-origin-when-cross-origin`.

`src`

This attribute specifies the URI of an external script; this can be used as an alternative to embedding a script directly within a document.

`type`

This attribute indicates the type of script represented. The value of this attribute will be one of the following:

#### Attribute is not set (default), an empty string, or a JavaScript MIME type

Indicates that the script is a "classic script", containing JavaScript code. Authors are encouraged to omit the attribute if the script refers to JavaScript code rather than specify a MIME type. JavaScript MIME types are listed in the IANA media types specification.

`importmap`

This value indicates that the body of the element contains an import map. The import map is a JSON object that developers can use to control how the browser resolves module specifiers when importing JavaScript modules.

`module`

This value causes the code to be treated as a JavaScript module. The processing of the script contents is deferred. The `charset` and `defer` attributes have no effect. For information on using `module`, see our JavaScript modules guide. Unlike classic scripts, module scripts require the use of the CORS protocol for cross-origin fetching.

`speculationrules`

This value indicates that the body of the element contains speculation rules. Speculation rules take the form of a JSON object that determine what resources should be prefetched or prerendered by the browser. This is part of the Speculation Rules API.

#### Any other value

The embedded content is treated as a data block, and won't be processed by the browser. Developers must use a valid MIME type that is not a JavaScript MIME type to denote data blocks. All of the other attributes will be ignored, including the `src` attribute.

### Deprecated attributes

`charset`

If present, its value must be an ASCII case-insensitive match for `utf-8`. It's unnecessary to specify the `charset` attribute, because documents must use UTF-8, and the `script` element inherits its character encoding from the document.

`language`

Like the `type` attribute, this attribute identifies the scripting language in use. Unlike the `type` attribute, however, this attribute's possible values were never standardized. The `type` attribute should be used instead.

### Notes

Scripts without `async`, `defer` or `type="module"` attributes, as well as inline scripts without the `type="module"` attribute, are fetched and executed immediately before the browser continues to parse the page.

The script should be served with the `text/javascript` MIME type, but browsers are lenient and only block them if the script is served with an image type (`image/*`), a video type (`video/*`), an audio type (`audio/*`), or `text/csv`. If the script is blocked, an `error` event is sent to the element; otherwise, a

`load` event is sent.

### Examples

#### Basic usage

This example shows how to import (an external) script using the `<script>` element:

```
<script src="https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/script/javascript.js"></script>
```

The following example shows how to put (an inline) script inside the `<script>` element:

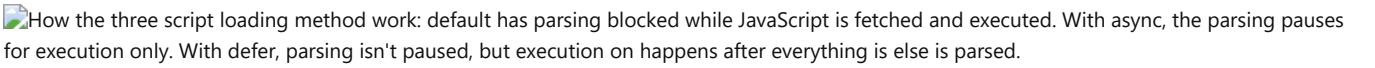
```
<script>
  alert("Hello World!");
</script>
```

#### async and defer

Scripts loaded using the `async` attribute will download the script without blocking the page while the script is being fetched. However, once the download is complete, the script will execute, which blocks the page from rendering. This means that the rest of the content on the web page is prevented from being processed and displayed to the user until the script finishes executing. You get no guarantee that scripts will run in any specific order. It is best to use `async` when the scripts in the page run independently from each other and depend on no other script on the page.

Scripts loaded with the `defer` attribute will load in the order they appear on the page. They won't run until the page content has all loaded, which is useful if your scripts depend on the DOM being in place (e.g., they modify one or more elements on the page).

Here is a visual representation of the different script loading methods and what that means for your page:

 How the three script loading method work: default has parsing blocked while JavaScript is fetched and executed. With async, the parsing pauses for execution only. With defer, parsing isn't paused, but execution on happens after everything is else is parsed.

*This image is from the HTML spec, copied and cropped to a reduced version, under CC BY 4.0 license terms.*

For example, if you have the following script elements:

```
<script async src="https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/script/js/vendor/jquery.js"></script>
<script async src="https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/script/js/script2.js"></script>
<script async src="https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/script/js/script3.js"></script>
```

You can't rely on the order the scripts will load in. `jquery.js` may load before or after `script2.js` and `script3.js` and if this is the case, any functions in those scripts depending on `jquery` will produce an error because `jquery` will not be defined at the time the script runs.

`async` should be used when you have a bunch of background scripts to load in, and you just want to get them in place as soon as possible. For example, maybe you have some game data files to load, which will be needed when the game actually begins, but for now you just want to get on with showing the game intro, titles, and lobby, without them being blocked by script loading.

Scripts loaded using the `defer` attribute (see below) will run in the order they appear in the page and execute them as soon as the script and content are downloaded:

```
<script defer src="https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/script/js/vendor/jquery.js"></script>
<script defer src="https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/script/js/script2.js"></script>
<script defer src="https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/script/js/script3.js"></script>
```

In the second example, we can be sure that `jquery.js` will load before `script2.js` and `script3.js` and that `script2.js` will load before `script3.js`. They won't run until the page content has all loaded, which is useful if your scripts depend on the DOM being in place (e.g., they modify one or more elements on the page).

To summarize:

- `async` and `defer` both instruct the browser to download the script(s) in a separate thread, while the rest of the page (the DOM, etc.) is downloading, so the page loading is not blocked during the fetch process.
- scripts with an `async` attribute will execute as soon as the download is complete. This blocks the page and does not guarantee any specific execution order.
- scripts with a `defer` attribute will load in the order they are in and will only execute once everything has finished loading.
- If your scripts should be run immediately and they don't have any dependencies, then use `async`.
- If your scripts need to wait for parsing and depend on other scripts and/or the DOM being in place, load them using `defer` and put their corresponding `<script>` elements in the order you want the browser to execute them.

#### Module fallback

Browsers that support the `module` value for the `type` attribute ignore any script with a `nomodule` attribute. That enables you to use module scripts while providing `nomodule`-marked fallback scripts for non-supporting browsers.

```
<script type="module" src="https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/script/main.js"></script>
<script nomodule src="https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/script/fallback.js"></script>
```

### Importing modules with importmap

When importing modules in scripts, if you don't use the `type=importmap` feature, then each module must be imported using a module specifier that is either an absolute or relative URL. In the example below, the first module specifier is an absolute URL, while the second (`"./shapes/square.js"`) resolves relative to the base URL of the document.

```
import { name as circleName } from "https://example.com/shapes/circle.js";
import { name as squareName, draw } from "./shapes/square.js";
```

An import map allows you to provide a mapping that, if matched, can replace the text in the module specifier. The import map below defines keys `circle` and `square` that can be used as aliases for the module specifiers shown above.

```
<script type="importmap">
  {
    "imports": {
      "circle": "https://example.com/shapes/circle.js",
      "square": "./shapes/square.js"
    }
  }
</script>
```

This allows us to import modules using names in the module specifier (rather than absolute or relative URLs).

```
import { name as circleName } from "circle";
import { name as squareName, draw } from "square";
```

For more examples of what you can do with import maps, see the Importing modules using import maps section in the JavaScript modules guide.

### Embedding data in HTML

You can also use the `<script>` element to embed data in HTML with server-side rendering by specifying a valid non-JavaScript MIME type in the `type` attribute.

```
<!-- Generated by the server -->
<script id="data" type="application/json">
  {
    "userId": 1234,
    "userName": "Maria Cruz",
    "memberSince": "2000-01-01T00:00:00.000Z"
  }
</script>

<!-- Static -->
<script>
  const userInfo = JSON.parse(document.getElementById("data").text);
  console.log("User information: %o", userInfo);
</script>
```

### Blocking rendering till a script is fetched and executed

You can include `render` token inside a `blocking` attribute; the rendering of the page will be blocked till the script is fetched and executed. In the example below, we block rendering on an async script, so that the script doesn't block parsing but is guaranteed to be evaluated before rendering starts.

```
<script blocking="render" async src="https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/script/async-script.js">
</script>
```

### Technical summary

| | |
|---|---|
| **Content categories** | Metadata content, Flow content, Phrasing content. |
| **Permitted content** | Dynamic script such as `text/javascript`. |
| **Tag omission** | None, both the starting and ending tag are mandatory. |
| **Permitted parents** | Any element that accepts metadata content, or any element that accepts phrasing content. |
| **Implicit ARIA role** | No corresponding role |
| **Permitted ARIA roles** | No `role` permitted |
| **DOM interface** | `HTMLScriptElement` |

### See also

- `document.currentScript`
- Flavio Copes' article on loading JavaScript efficiently and explaining the differences between `async` and `defer`
- JavaScript modules guide

## `<script type>` HTML attribute

The **`type`** attribute of the `<script>` element indicates the *type* of script represented by the element: a classic script, an import map, a JavaScript module, speculation rules, or a data block.

### Value

The value of this attribute indicates the type of data represented by the script, and will be one of the following:

#### Attribute is not set (default), an empty string, or a JavaScript MIME type

Indicates that the script is a "classic script", containing JavaScript code. Authors are encouraged to omit the attribute if the script refers to JavaScript code rather than specify a MIME type. JavaScript MIME types are listed in the IANA media types specification.

`importmap`

This value indicates that the body of the element contains an import map. The import map is a JSON object that developers can use to control how the browser resolves module specifiers when importing JavaScript modules.

`module`

This value causes the code to be treated as a JavaScript module. The processing of the script contents is deferred. The `charset` and `defer` attributes have no effect. For information on using `module`, see our JavaScript modules guide. Unlike classic scripts, module scripts require the use of the CORS protocol for cross-origin fetching.

`speculationrules`

This value indicates that the body of the element contains speculation rules. Speculation rules take the form of a JSON object that determine what resources should be prefetched or prerendered by the browser. This is part of the Speculation Rules API.

#### Any other value

The embedded content is treated as a data block, and won't be processed by the browser. Developers must use a valid MIME type that is not a JavaScript MIME type to denote data blocks. All of the other attributes will be ignored, including the `src` attribute.

> **Note:** In earlier browsers, the type identified the scripting language of the embedded or imported (via the `src` attribute) code.

## `<script type="importmap">` HTML attribute value

The **`importmap`** value of the `type` attribute of the `<script>` element indicates that the body of the element contains an import map.

An import map is a JSON object that allows developers to control how the browser resolves module specifiers when importing JavaScript modules. It provides a mapping between the text used as the module specifier in an `import` statement or `import()` operator, and the corresponding value that will replace the text when resolving the specifier. The JSON object must conform to the Import map JSON representation format.

An import map is used to resolve module specifiers in static and dynamic imports, and therefore must be declared and processed before any `<script>` elements that import modules using specifiers declared in the map. Note that the import map applies only to module specifiers in the `import` statement or `import()` operator for modules loaded into documents; it does not apply to the path specified in the `src` attribute of a `<script>` element or to modules loaded into workers or worklets.

For more information, see the Importing modules using import maps section in the JavaScript modules guide.

### Syntax

```
<script type="importmap">
  // JSON object defining import
</script>
```

The `src`, `async`, `nomodule`, `defer`, `crossorigin`, `integrity`, and `referrerpolicy` attributes must not be specified.

### Exceptions

`TypeError`

The import map definition is not a JSON object, the `importmap` key is defined but its value is not a JSON object, or the `scopes` key is defined but its value is not a JSON object.

Browsers generate console warnings for other cases where the import map JSON does not conform to the import map schema.

### Description

When importing a JavaScript module, both the `import` statement and `import()` operator have a "module specifier" that indicates the module to be imported. A browser must be able to resolve this specifier to an absolute URL in order to import the module.

For example, the following statements import elements from the module specifier `"https://example.com/shapes/circle.js"`, which is an absolute URL, and the module specifier `"./modules/shapes/square.js"`, which is a path relative to the base URL of the document.

```
import { name as circleName } from "https://example.com/shapes/circle.js";
import { name as squareName, draw } from "./modules/shapes/square.js";
```

Import maps allow developers to specify (almost) any text they want in the module specifier; the map provides a corresponding value that will replace the text when the module specifier is resolved.

### Bare modules

The import map below defines an `imports` key that has a "module specifier map" with properties `circle` and `square`.

```
<script type="importmap">
  {
    "imports": {
      "circle": "https://example.com/shapes/circle.js",
      "square": "./modules/shapes/square.js"
    }
  }
</script>
```

With this import map we can import the same modules as above, but using "bare modules" in our module specifiers:

```
import { name as circleName } from "circle";
import { name as squareName, draw } from "square";
```

### Mapping path prefixes

A module specifier map key can also be used to remap a path prefix in a module specifier. Note that in this case the property and mapped path must both have a trailing forward slash (`/`).

```
<script type="importmap">
  {
    "imports": {
      "shapes/": "./modules/shapes/",
      "other-shapes/": "https://example.com/modules/shapes/"
    }
  }
</script>
```

We could then import a circle module as shown.

```
import { name as circleName } from "shapes/circle.js";
```

### Paths in the module specifier map key

Module specifier keys do not have to be single word names ("bare names"). They can also contain or end with path separators, or be absolute URLs, or be relative URL paths that start with `/`, `./`, or `../`.

```
{
  "imports": {
    "modules/shapes/": "./modules/src/shapes/",
    "modules/square": "./modules/src/other/shapes/square.js",
    "https://example.com/modules/square.js": "./modules/src/other/shapes/square.js",
    "../modules/shapes/": "/modules/shapes/"
  }
}
```

If there are several module specifier keys in a module specifier map that might match, then the most specific key will be selected (i.e., the one with the longer path/value).

A module specifier of `./foo/../js/app.js` would be resolved to `./js/app.js` before matching. This means that a module specifier key of `./js/app.js` would match the module specifier even though they are not exactly the same.

### Scoped module specifier maps

You can use the `scopes` key to provide mappings that are only used if the script importing the module contains a particular URL path. If the URL of the loading script matches the supplied path, the mapping associated with the scope will be used. This allows different versions of the module to be used depending on what code is doing the importing.

For example, the map below will only use the scoped map if the loading module has a URL that includes the path: "/modules/custom-shapes/".

```
<script type="importmap">
  {
    "imports": {
      "square": "./modules/shapes/square.js"
    },
    "scopes": {
      "/modules/custom-shapes/": {
        "square": "https://example.com/modules/shapes/square.js"
      }
    }
  }
</script>
```

If multiple scopes match the referrer URL, then the most specific scope path is used (the scope key name with the longest name). The browser falls back to the next most specific scoped path if there is no matching specifier, and so on, eventually falling back to the module specifier map in the `imports` key.

### Integrity metadata map

You can use the `integrity` key to provide mapping for module integrity metadata. This enables you to ensure the integrity of dynamically or statically imported modules. `integrity` also enables you to provide a fallback for top-level or preloaded modules, in case they don't already include an `integrity` attribute.

The map keys represent module URLs, which can be absolute or relative (starting with `/`, `./`, or `../`). The map values represent integrity metadata, identical to that used in `integrity` attribute values.

For example, the map below defines integrity metadata for the `square.js` module (directly) and its bare specifier (transitively, via the `imports` key).

```
<script type="importmap">
  {
    "imports": {
      "square": "./modules/shapes/square.js"
    },
    "integrity": {
      "./modules/shapes/square.js": "sha384-oqVuAfXRKap7fdgcCY5uykM6+R9GqQ8K/uxy9rx7HNQlGYl1kPzQho1wx4JwY8wC"
    }
  }
</script>
```

### Merging multiple import maps

Supporting browsers can declare one or more import maps anywhere in the document, provided they are defined before any module that depends on them is loaded (some browser versions allow only a single import map declaration, which must appear before any module is loaded).

Internally, browsers maintain a single global import map representation. When multiple import maps are included in a document, their contents are merged into the global import map when they are registered.

For example, consider the following two import maps:

```
<script type="importmap">
  {
    "imports": {
      "/app/": "./original-app/"
    }
  }
</script>
```

```
<script type="importmap">
  {
    "imports": {
      "/app/helper": "./helper/index.mjs"
    },
    "scopes": {
      "/js": {
        "/app/": "./js-app/"
      }
```

```
    }
  }
</script>
```

These are equivalent to the following single import map:

```
<script type="importmap">
  {
    "imports": {
      "/app/": "./original-app/",
      "/app/helper": "./helper/index.mjs"
    },
    "scopes": {
      "/js": {
        "/app/": "./js-app/"
      }
    }
  }
</script>
```

Module specifiers in each registered map that were already resolved beforehand are dropped. Subsequent resolutions of these specifiers will provide the same results as their previous resolutions.

For example, if the module specifier `/app/helper.js` was already resolved, the following new import map:

```
<script type="importmap">
  {
    "imports": {
      "/app/helper.js": "./helper/index.mjs",
      "lodash": "/node_modules/lodash-es/lodash.js"
    }
  }
</script>
```

Would be equivalent to:

```
<script type="importmap">
  {
    "imports": {
      "lodash": "/node_modules/lodash-es/lodash.js"
    }
  }
</script>
```

The `/app/helper.js` rule was ignored and not incorporated into the map.

Similarly, module specifiers in a registered map that were already mapped to URLs in the global map are dropped; their previous mapping prevails.

For example, the following two import maps:

```
<script type="importmap">
  {
    "imports": {
      "/app/helper": "./helper/index.mjs",
      "lodash": "/node_modules/lodash-es/lodash.js"
    }
  }
</script>
```

```
<script type="importmap">
  {
    "imports": {
      "/app/helper": "./main/helper/index.mjs"
    }
  }
</script>
```

Are equivalent to the following single import map:

```
<script type="importmap">
  {
    "imports": {
      "/app/helper": "./helper/index.mjs",
      "lodash": "/node_modules/lodash-es/lodash.js"
```

```
    }
  }
</script>
```

The `/app/helper/` rule was dropped from the second map.

> **Note:** In non-supporting browsers (check the compatibility data), a polyfill can be used to avoid issues related to module resolution.

### Import map JSON representation

The following is a "formal" definition of the import map JSON representation.

The import map must be a valid JSON object that can define any of the optional keys `imports`, `scopes` and `integrity`. Each key's value must be an object, which may be empty.

`imports` Optional

The value is a module specifier map, which provides the mappings between module specifier text that might appear in an `import` statement or `import()` operator, and the text that will replace it when the specifier is resolved.

This is the fallback map that is searched for matching module specifiers if no `scopes` path URLs match, or if module specifier maps in matching `scopes` paths do not contain a key that matches the module specifier.

`<module specifier map>`

A "module specifier map" is a valid JSON object where the *keys* are text that may be present in the module specifier when importing a module, and the corresponding *values* are the URLs or paths that will replace this text when the module specifier is resolved to an address.

The module specifier map JSON object has the following requirements:

- None of the keys may be empty.
- All of the values must be strings, defining either a valid absolute URL or a valid URL string that starts with `/`, `./`, or `../`.
- If a key ends with `/`, then the corresponding value must also end with `/`. A key with a trailing `/` can be used as a prefix for when mapping (or remapping) modules addresses.
- The object properties' ordering is irrelevant: if multiple keys can match the module specifier, the most specific key is used (in other words, a specifier "olive/branch/" would match before "olive/").

`integrity` Optional

Defines a valid JSON object where the *keys* are strings containing valid absolute or relative URLs (starting with `/`, `./`, or `../`), and the corresponding *values* are valid integrity metadata.

If the URL of a script importing or preloading a module matches a key in the `integrity` object, the corresponding integrity metadata is applied to the script's fetch options, unless they already have integrity metadata attached to them.

`scopes` Optional

Scopes define path-specific module specifier maps, allowing the choice of map to depend on the path of the code importing the module.

The scopes object is a valid JSON object where each property is a `<scope key>`, which is a URL path, with a corresponding value that is a `<module specifier map>`.

If the URL of a script importing a module matches a `<scope key>` path, then the `<module specifier map>` value associated with the key is checked for matching specifiers first. If there are multiple matching scope keys, then the value associated with the most specific/nested scope paths are checked for matching module specifiers first. The fallback module specifier map in `imports` is used if there are no matching module specifier keys in any of the matching scoped module specifier maps.

Note that the scope does not change how an address is resolved; relative addresses are always resolved to the import map base URL.

## See also

- JavaScript modules > Importing modules using import maps
- The `type` attribute of HTML `<script>` elements
- `import` statement
- `import()` operator

## `<script type="speculationrules">` HTML attribute value

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`speculationrules`** value of the `type` attribute of the `<script>` element indicates that the body of the element contains speculation rules.

Speculation rules take the form of a JSON structure that determine what resources should be prefetched or prerendered by the browser. This is part of the Speculation Rules API.

> **Note:** Speculation rules can be defined inside external text files referenced by the `Speculation-Rules` HTTP header, using the same JSON representation provided below. Specifying an HTTP header is useful in cases where developers are not able to directly modify the document itself.

### Syntax

```
<script type="speculationrules">
  // JSON object defining rules
</script>
```

> **Note:** The `src`, `async`, `nomodule`, `defer`, `crossorigin`, `integrity`, and `referrerpolicy` attributes must not be specified.

### Exceptions

`TypeError`

The speculation rules definition is not a valid JSON object.

### Description

A `<script type="speculationrules">` element must contain a valid JSON structure that defines speculation rules. The following examples show separate prefetch and prerender rules:

```
<script type="speculationrules">
  {
    "prefetch": [
      {
        "urls": ["next.html", "next2.html"],
        "requires": ["anonymous-client-ip-when-cross-origin"],
        "referrer_policy": "no-referrer"
      }
    ]
  }
</script>
```

```
<script type="speculationrules">
  {
    "prerender": [
      {
        "where": { "href_matches": "/next" },
        "eagerness": "eager"
      }
    ]
  }
</script>
```

### Speculation rules JSON representation

The JSON structure contains one or more fields at the top level, each one representing an action to define speculation rules for. At present the supported actions are:

`"prefetch"` Optional

Rules for potential future navigations that should have their associated document response body downloaded, leading to significant performance improvements when those documents are navigated to. Note that none of the subresources referenced by the page are downloaded.

`"prerender"` Optional

Rules for potential future navigations that should have their associated documents fully downloaded, rendered, and loaded into an invisible tab. This includes loading all subresources, running all JavaScript, and even loading subresources and performing data fetches started by JavaScript. When those documents are navigated to, navigations will be instant, leading to major performance improvements.

> **Note:** Consult the Speculation Rules API main page for full details on how to use prefetch and prerender effectively.

Each action field contains an array, which in turn contains one or more objects. Each object contains a single rule defining a set of URLs and related parameters.

Each object can contain the following properties:

`"source"`

A string indicating the source of the URLs to which the rule applies. This is optional because the value can always be inferred from other properties.

This can be one of:

`"document"`

Specifies that the URLs will be matched from navigation links in the associated document (as defined in `<a>` and `<area>` elements), based on the conditions described by a `"where"` key. Note that the presence of a `"where"` key implies `"source": "document"`, so it is optional.

`"list"`

Specifies that the URLs will come from a list, specified in the `"urls"` key. Note that the presence of a `"urls"` key implies `"source": "list"`, so it is optional.

`"urls"`

An array of strings representing a list of URLs to apply the rule to. These can be absolute or relative URLs. Relative URLs will be parsed relative to the document base URL (if inline in a document) or relative to the external resource URL (if externally fetched). `"urls"` and `"where"` cannot both be set in the same rule.

`"where"`

An object representing the conditions by which the rule matches URLs contained in the associated document. Effectively, the `"where"` object represents a test that is performed on every link on the page to see whether the speculation rule is applied to it. `"where"` and `"urls"` cannot both be set in the same rule.

This object can contain exactly one of the following properties:

`"href_matches"`

A string containing a URL pattern, or an array containing multiple URL pattern strings, which follow the standard URL Pattern API syntax. Links in the document whose URLs match the pattern(s) will have the rule applied.

`"relative_to"`

In the case of an `"href_matches"` condition, this can specify where you want that condition to be matched relative to. This works in exactly the same way as the rule-level `"relative_to"` key, except that it only affects a single `"href_matches"` condition inside a `"where"` key.

`"selector_matches"`

A string containing a CSS selector, or an array containing multiple CSS selectors. Links in the document matched by those selectors will have the rule applied.

`"and"`

An array containing one or more objects containing conditions (`"href_matches"`, `"selector_matches"`, `"and"`, `"not"`, or `"or"`), all of which must match for the rule to be applied to them.

`"not"`

An object containing one condition (`"href_matches"`, `"selector_matches"`, `"and"`, `"not"`, or `"or"`) which, if it matches, will *not* have the rule applied to it. All links that *do not* match the condition *will* have the rule applied.

`"or"`

An array containing one or more objects containing conditions (`"href_matches"`, `"selector_matches"`, `"and"`, `"not"`, or `"or"`), any of which can match for the rule to be applied to them.

`"where"` conditions can be nested multiple levels deep to create complex conditions, or you can choose to split them into separate rules to keep them simple. See where examples for more explanation, and multiple examples of use.

`"eagerness"`

A string providing a hint to the browser as to how eagerly it should prefetch/prerender link targets in order to balance performance advantages against resource overheads. Possible values are:

`"immediate"`

The author thinks the link is very likely to be followed, and/or the document may take significant time to fetch. Prefetch/prerender should start as soon as possible, subject only to considerations such as user preferences and resource limits.

`"eager"`

The author wants to prefetch/prerender a large number of navigations, as early as possible. Prefetch/prerender should start on any slight suggestion that a link may be followed. For example, the user could move their mouse cursor towards the link, hover/focus it for a moment, or pause scrolling with the link in a prominent place.

`"moderate"`

The author is looking for a balance between `eager` and `conservative`. Prefetch/prerender should start when there is a reasonable suggestion that the user will follow a link in the near future. For example, the user could scroll a link into the viewport and hover/focus it for some time.

`"conservative"`

The author wishes to get some benefit from speculative loading with a fairly small tradeoff of resources. Prefetch/prerender should start only when the user is starting to click on the link, for example on `mousedown` or `pointerdown`.

If `"eagerness"` is not explicitly specified, list (`"urls"`) rules default to `immediate` and document (`"where"`) rules default to `conservative`. The browser takes this hint into consideration along with its own heuristics, so it may select a link that the author has hinted as less eager than another, if the less eager candidate is considered a better choice.

`"expects_no_vary_search"`

A string providing a hint to the browser as to what `No-Vary-Search` header value will be set on responses for documents that it is receiving prefetch/prerender requests for. The browser can use this to determine ahead of time whether it is more useful to wait for an existing prefetch/prerender to finish, or start a new fetch request when the speculation rule is matched. See the `"expects_no_vary_search"` example for more explanation of how this can be used.

`"referrer_policy"`

A string representing a specific referrer policy string to use when requesting the URLs specified in the rule — see `Referrer-Policy` for possible values. The purpose of this is to allow the referring page to set a stricter policy specifically for the speculative request than the policy the page already has set (either by default, or by using `Referrer-Policy`).

> **Note:** A cross-site prefetch requires a referrer policy that is at least as strict as the default `"strict-origin-when-cross-origin"` value — so `"strict-origin-when-cross-origin"`, `"same-origin"`, `"strict-origin"`, or `"no-referrer"`. A laxer policy set in the speculation rules will override a stricter policy set on the referring page as long as it is still sufficiently strict for the cross-site case.

> **Note:** In the case of document rules, the matched link's specified referrer policy (e.g., using the `referrerpolicy` attribute) will be used, unless the rule specifies a policy that overrides it.

`"relative_to"`

A string specifying where you want links matched by URL to be matched relative to. The value can be one of:

`document`

URLs should be matched relative to the document the speculation rules are being set on.

`ruleset`

URLs should be matched relative to the file the rules are specified in. This is the default value.

This key setting is only relevant for rules defined in an external file (set using the `Speculation-Rules` header). When rules are specified inside the same document they are being set for (i.e., in an inline `<script>` element), it makes no difference.

`"requires"`

An array of strings representing capabilities of the browser parsing the rule, which must be available if the rule is to be applied to the specified URLs.

> **Warning:** Prefetches will automatically fail in browsers that cannot meet a specified requirement, even if they support the Speculation Rules API.

Possible values are:

`"anonymous-client-ip-when-cross-origin"`

(prefetch-only) Specifies that the rule matches only if the user agent can prevent the client IP address from being visible to the origin server if a cross-origin prefetch request is issued. Exactly how this works is dependent on browser implementation specifics. For example:

- Chrome's implementation hides the IP address using a Google-owned proxy, therefore by default it only works for Google-controlled referrers (since in that case, sending the URLs of the destination to Google is not an additional privacy leak). When used on a non-Google-owned site, rules that include this will only match for users that turn on "Enhanced preloading" in `chrome://settings/preloading`.
- Other Chromium-based browsers will have to provide their own solutions. Thorough testing in all target browsers is advised.
- A future Safari implementation may possibly use something along the lines of iCloud Private Relay.
- A future Firefox implementation might use something based on the Mozilla VPN product.

`"tag"`

A string used to identify a rule or ruleset. This will be included in the `Sec-Speculation-Tags` request header for any speculations covered by that rule.

`"target_hint"`

A string indicating where the page expects the prerendered content to be activated. The directive not supported for prefetch speculations. Allowed values are:

`"target_hint": "_blank"`

Open rerendered content in a new page.

`"target_hint": "_self"`

Open rerendered content in the current page. This is the default, if not specified.

> **Note:** As speculation rules use a `<script>` element, they need to be explicitly allowed in the `Content-Security-Policy script-src` directive if the site includes it. This is done by adding the `"inline-speculation-rules"` value along with a hash- or nonce-source.

### Examples

#### Prefetch and prerender in the same set of rules

The basic examples shown in the description section included separate speculation rules defined for prefetch and prerender. It is possible to define both in a single set of rules:

```
<script type="speculationrules">
  {
    "prefetch": [
      {
        "urls": ["next.html", "next2.html"],
        "requires": ["anonymous-client-ip-when-cross-origin"],
        "referrer_policy": "no-referrer"
      }
    ],
    "prerender": [
      {
        "where": { "selector_matches": ".product-link" },
        "eagerness": "eager"
      }
    ]
  }
</script>
```

> **Note:** This code snippet provides a list (`"urls"`) rule and a document (`"where"`) rule example.

#### Multiple rule sets

It is also allowable to include multiple sets of rules in a single HTML file:

```
<script type="speculationrules">
  {
    "prefetch": [
      {
        "urls": ["next.html", "next2.html"],
        "requires": ["anonymous-client-ip-when-cross-origin"],
        "referrer_policy": "no-referrer"
      }
    ]
  }
</script>
<script type="speculationrules">
  {
    "prerender": [
      {
        "where": { "selector_matches": ".product-link" },
        "eagerness": "eager"
      }
    ]
  }
</script>
```

And multiple rules in a single result set:

```
<script type="speculationrules">
  {
    "prerender": [
      {
        "urls": ["one.html"]
      },
      {
        "urls": ["two.html"]
      }
    ]
  }
</script>
```

### Dynamic rule insertion

Below is an example that feature detects speculation rules and, if supported, dynamically adds a prerender speculation rule via JavaScript:

```
if (
  HTMLScriptElement.supports &&
  HTMLScriptElement.supports("speculationrules")
) {
  const specScript = document.createElement("script");
  specScript.type = "speculationrules";
  const specRules = {
    prerender: [
      {
        urls: ["/next.html"],
      },
    ],
  };
  specScript.textContent = JSON.stringify(specRules);
  console.log("added speculation rules to: next.html");
  document.body.append(specScript);
}
```

### <code>where</code> syntax examples

A document-sourced rule contains a `"where"` property, which is an object containing criteria that define which links in the document are matched. Effectively, the `"where"` object represents a test that is performed on every link on the page to see whether the speculation rule is applied to it.

The most basic version will match a single URL pattern or CSS selector:

```
{ "where": { "href_matches": "/next" } }
```

```
{ "where": { "selector_matches": ".important-link" } }
```

`"href_matches"` and `"selector_matches"` can also be set to an array of values, so multiple URL patterns or CSS selectors can be matched simultaneously:

```
{ "where": { "href_matches": ["/next", "/profile"] } }
```

```
{ "where": { "selector_matches": [".important-link", "#unique-link"] } }
```

URL patterns and selectors can also contain wildcard (`*`) characters, allowing a single value to match multiple URLs. For example, the object below could match `user/`, `user/settings`, `user/stats`, etc.

```
{ "where": { "href_matches": "/user/*" } }
```

Search parameters (or query strings) can also be targeted in `href_matches`. For example, the object below could match all same-origin URLs with a `category` search parameter (as the first or a subsequent parameter):

```
{ "where": { "href_matches": "/*\\?*(^|&)category=*" } }
```

Any condition can be negated by placing it inside a `"not"` condition — this means that, when matched, a link *won't* have the speculation rule applied to it, but when *not* matched, it *will*. The following example will cause all links that *don't* match the URL pattern `/logout` to have the rule applied to them, but not links that match `/logout`:

```
{ "where": { "not": { "href_matches": "/logout" } } }
```

### Combining multiple `"where"` conditions with `"and"` or `"or"`

Multiple conditions can be combined inside `"and"` or `"or"` conditions — these take the value of arrays containing multiple conditions, all or any of which (respectively) have to match for the speculation rules to apply to a link. Using `"and"` or `"or"`, conditions can be nested multiple levels deep — there is no specified limit on allowed nesting levels.

It is useful to think of the `"where"` object as being equivalent to an `if` statement. So

```
{ and: [A, B, { or: [C, { not: D }] }] }
```

is equivalent to

```
if (A && B && (C || !D)) {
  apply speculation rule
}
```

In the following complete speculation rule example, all same-origin pages are marked for prefetching except those known to be problematic — the `/logout` page, and any links marked up with a class of `.no-prerender`:

```
<script type="speculationrules">
  {
    "prefetch": [
      {
        "where": {
          "and": [
            { "href_matches": "/*" },
            { "not": { "href_matches": "/logout" } },
            { "not": { "selector_matches": ".no-prerender" } }
          ]
        }
      }
    ]
  }
</script>
```

> **Note:** The `where` pattern above does not include cross-site links, which are supported for prefetching (provided the user has no cookies set for the destination site, to protect against tracking) but not for prerendering.

### <code>"relative_to"</code> example

For rule sets that are externally fetched (i.e., via the `Speculation-Rules`) response header, URLs in list rules and URL patterns in document rules are parsed relative to the containing external text file's URL by default. To parse URLs in a list rule relative to the document's base URL, `"relative_to"` is used like this:

```
{
  "urls": ["/home", "/about"],
  "relative_to": "document"
}
```

For document rules, `"relative_to"` can be paired directly with `"href_matches"` and the document's base URL would only be used for patterns in that particular condition:

```
{
  "where": {
    "or": [
      { "href_matches": "/home", "relative_to": "document" },
      { "href_matches": "/about" }
    ]
  }
}
```

In the above example, only the first `"href_matches"` will be matched relative to the document's base URL.

`relative_to` is mainly relevant if the speculation rules JSON file is on a different origin to the document you wish to apply them to:

1. If the document is located at `https://example.com/some/subpage.html` and the rules are at `https://example.com/resources/rules.json`, then `/home` always equates to `https://example.com/home` regardless of whether `relative_to` is set to `document` or `ruleset`.
2. However, if the document is located at `https://example.com/some/subpage.html` and the rules are at `https://other.example/resources/rules.json` (for example, on a third-party or cookieless resource origin), then:
   - `"relative_to": "document"` will cause `/home` to equate to `https://example.com/home`.
   - `"relative_to": "ruleset"` will cause `/home` to equate to `https://other.example/home`.

   This is the typical use case for `"relative_to"`.
3. Another potential (but rarer) use case is when your URLs are specified in the form `home` instead of `/home`. If the document is located at `https://example.com/some/subpage.html` and the rules are at `https://example.com/resources/rules.json`, then:
   - `"relative_to": "document"` would cause `home` to equate to `https://example.com/some/home`.
   - `"relative_to": "ruleset"` would cause `home` to equate to `https://example.com/resources/home`.

### <code>"expects_no_vary_search"</code> example

Consider the case of a user directory landing page, `/users`, which has an `id` parameter added to bring up information on a specific user, for example `/users?id=345`. Whether this URL should be considered identical for caching purposes depends on the behavior of the application:

1. If this parameter has the effect of loading a completely new page containing the information for the specified user, then the URL should be cached separately.
2. If this parameter has the effect of highlighting the specified user on the same page, and perhaps revealing a pullout panel displaying their data, then the URL should be considered the same for caching purposes. This could result in performance improvements around the loading of the user pages and could be achieved via a `No-Vary-Search` with a value of `params=("id")`.

How does this affect speculation rules? Consider the following code:

```
<script type="speculationrules">
  {
    "prefetch": [
      {
        "urls": ["/users"]
      }
    ]
  }
</script>
<a href="https://developer.mozilla.org/users?id=345">User Bob</a>
```

What would happen in this case when the user starts a navigation to `/users?id=345` when the headers for the prefetch of `/users` have not been received yet? At this point, the browser doesn't know what the `No-Vary-Search` value will be, if anything. If there was no `No-Vary-Search` value set, and the application behavior was more like Option 1 above, the prefetch would be wasted and the browser would need to go and fetch the separate `/users?id=345` page from scratch.

To solve this, we can provide a hint as to what the page author expects the `No-Vary-Search` value to be. A speculation rule can have an `"expects_no_vary_search"` field, which contains a string representation of the expected header value:

```
<script type="speculationrules">
  {
    "prefetch": [
      {
        "urls": ["/users"],
        "expects_no_vary_search": "params=(\"id\")"
      }
    ]
  }
</script>
<a href="https://developer.mozilla.org/users?id=345">User Bob</a>
```

This indicates that Option 2 described above is what the server is expected to produce. If a navigation starts while there is an ongoing prefetch of `/users`, this informs the browser that it is appropriate to wait for the prefetch, instead of immediately starting another fetch for `/users?id=345`.

Document rules can also be used in conjunction with `"expects_no_vary_search"`, depending on the pattern used. For example, in the case of:

```
<script type="speculationrules">
  {
    "prefetch": [
      {
        { "where": { "href_matches": "/users?id=*" } },
        "expects_no_vary_search": "params=(\"id\")"
      }
    ]
  }
</script>
<a href="https://developer.mozilla.org/users?id=012">User Bill</a>
<a href="https://developer.mozilla.org/users?id=345">User Bob</a>
<a href="https://developer.mozilla.org/users?id=678">User Ben</a>
```

If a link is hovered over, the browser will start prefetching that specific link.

If the user hovers over another link before the prefetch completes, the `expects_no_vary_search` pattern tells the browser that there is no need to cancel the current prefetch, because all `/users` URLs with `id` URL parameter values effectively point to the same page for this context (and for caching purposes).

> **Warning:** Additional care must be taken when using prerender with `No-Vary-Search` since the page may initially be prerendered with different URL parameters. `No-Vary-Search` is used for URL parameters that deliver the same resource from the server, but are used by the client for various reasons (client-side rendering, UTM parameters for analytics measurement, etc.). As the initial prerender may be for different URL parameters, any code depending on them should only run after prerender activation.

Multiple params can be provided in a space-separated array:

```
<script type="speculationrules">
  {
    "prefetch": [
      {
        { "where": { "href_matches": "/users?id=*" } },
        "expects_no_vary_search": "params=(\"id\" \"order\" \"lang\")"
      }
    ]
  }
</script>
```

> **Note:** As a structured field, the parameters should be space-separated, quoted strings — as shown above — and not comma-separated, which developers may be more used to.

### <code>eagerness</code> example

The following set of document rules shows how `eagerness` can be used to hint at the eagerness with which the browser should prerender each matching set of links.

```
<script type="speculationrules">
  {
    "prerender": [
      {
        "where": { "href_matches": "/*" },
        "eagerness": "conservative"
      },
      {
        "where": { "selector_matches": ".product-link" },
        "eagerness": "eager"
      }
    ]
  }
</script>
```

Here we are hinting that:

- All same-site links contained in the document should be conservatively prerendered (i.e., when the user starts to activate them).

- Any product links (in this case, those with a `class` of `.product-link`) in the document should be eagerly prerendered (i.e., if the user makes any kind of move towards navigating to them).

> **Note:** The effects of eagerness settings are less useful for list rules. By default, list rule URLs are prefetched/prerendered immediately as soon as the rules are parsed, which is what you'd expect — they are intended for explicit listing of high-priority URLs that you want to make available as soon as possible. For this reason, `eager` has the same effect as `immediate` in current implementations. Lower eagerness settings are for prefetching/prerendering when links are interacted with, and for these you are more likely to use document rules to find them on the page.

### <code>tag</code> example

A `tag` can be included at the top level, to identify the overall ruleset:

```
<script type="speculationrules">
  {
    "tag": "my-rules",
    "prerender": [
      {
        "where": { "href_matches": "/*" },
        "eagerness": "conservative"
      }
    ]
  }
</script>
```

Or to identify individual rules:

```
<script type="speculationrules">
  {
    "prefetch": [
      "tag": "my-prefetch-rule",
      "urls": ["next.html"]
    ],
    "prerender": [
      "tag": "my-prerender-rule",
      "urls": ["next2.html"]
    ],
  }
</script>
```

See `Sec-Speculation-Tags` for more examples.

### <code>target_hint</code> example

A `target_hint` can be included to indicate the target window in which matching prerender speculations will be opened:

```
<script type="speculationrules">
  {
    "tag": "my-rules",
    "prerender": [
      {
        "eagerness": "eager",
        "target_hint": "_blank",
        "urls": ["page2.html"]
      }
    ]
  }
</script>
```

The rules above allow the following links to be prerendered correctly in the appropriate targets:

```
<a href="page1.html">Open link in this window</a>
<a target="_blank" href="page2.html">Open link in new window</a>
```

`target_hint` is only needed for list rules, which use `urls`. They are not needed for document rules (which use `where`) since in these the target can be known from the `target` attribute of the `<a>` link element.

### See also

- Prerender pages in Chrome for instant page navigations on developer.chrome.com
- Speculative loading
- Speculation Rules API

## <search> HTML generic search element

The **<search>** HTML element is a container representing the parts of the document or application with form controls or other content related to performing a search or filtering operation. The <search> element semantically identifies the purpose of the element's contents as having search or filtering capabilities. The search or filtering functionality can be for the website or application, the current web page or document, or the entire Internet or subsection thereof.

### Attributes

This element only includes the global attributes.

### Usage notes

The <search> element is not for presenting search results. Rather, search or filtered results should be presented as part of the main content of that web page. That said, suggestions and links that are part of "quick search" functionality within the search or filtering functionality are appropriately nested within the contents of the <search> element as they are search features.

### Accessibility

The <search> element defines a search landmark. This removes the need for adding role=search to a <form> element.

### Examples

### Header search form

This example demonstrates the use of <search> as the container for a search within a website header to perform a site-wide search. The <search> is a semantic container for the <form> that submits the user-entered search query to a server.

#### HTML

```
<header>
  <h1>Movie website</h1>
  <search>
    <form action="./search/">
      <label for="movie">Find a Movie</label>
      <input type="search" id="movie" name="q" />
      <button type="submit">Search</button>
    </form>
  </search>
</header>
```

#### Result

### Web app search

This example demonstrates potential DOM content when dynamically including JavaScript search functionality in a web application. When search functionality is implemented entirely with JavaScript, if no form is submitted, neither a <form> element nor a submit <button> is required. For semantics, the <search> element is included to contain the search and filtering capabilities.

#### HTML

```
<search>
  <label>
    Find and filter your query
    <input type="search" id="query" />
  </label>
  <label>
    <input type="checkbox" id="exact-only" />
    Exact matches only
  </label>

  <section>
    <h3>Results:</h3>
    <ul id="results">
      <!-- search result content -->
    </ul>
    <output id="no-results">
      <!-- no results content -->
    </output>
  </section>
</search>
```

#### Result

> **Note:** Remember that some users don't have JavaScript, and none of your users have JavaScript running until the JavaScript is successfully downloaded, parsed, and executed. Ensure your users can access the content of your site with JavaScript disabled.

### Multiple searches

This example demonstrates a page with two search features. The first is a global site search located on the header. The second is a search and filter based on the page context, in our example a car search.

#### HTML

```
<body>
  <header>
    <h1>Car rental agency</h1>
    <search title="Website">...</search>
  </header>
  <main>
    <h2>Cars available for rent</h2>
    <search title="Cars">
      <h3>Filter results</h3>
      ...
    </search>
    <article>
      <!-- search result content -->
    </article>
  </main>
</body>
```

#### Result

#### Technical summary

| | |
|---|---|
| **Content categories** | Flow content, palpable content. |
| **Permitted content** | Flow content. |
| **Tag omission** | None, both the starting and ending tag are mandatory. |
| **Implicit ARIA role** | `search` |
| **Permitted ARIA roles** | `form`, `group`, `none`, `presentation`, `region`, `search` |
| **DOM interface** | `HTMLElement` |

### See also

- Other search-related elements: `<header>`, `<footer>`, `<aside>`, `<nav>`, `<form>`
- ARIA: Search role

## <section> HTML generic section element

The **`<section>`** HTML element represents a generic standalone section of a document, which doesn't have a more specific semantic element to represent it. Sections should always have a heading, with very few exceptions.

#### Try it

```
<h1>Choosing an Apple</h1>
<section>
  <h2>Introduction</h2>
  <p>
    This document provides a guide to help with the important task of choosing
    the correct Apple.
  </p>
</section>

<section>
  <h2>Criteria</h2>
  <p>
    There are many different criteria to be considered when choosing an Apple —
    size, color, firmness, sweetness, tartness...
  </p>
</section>
```

```
h1,
h2 {
```

```
  margin: 0;
}
```

### Attributes

This element only includes the global attributes.

### Usage notes

As mentioned above, `<section>` is a generic sectioning element, and should only be used if there isn't a more specific element to represent it. As an example, a navigation menu should be wrapped in a `<nav>` element, but a list of search results or a map display and its controls don't have specific elements, and could be put inside a `<section>`.

Also consider these cases:

- If the contents of the element represent a standalone, atomic unit of content that makes sense syndicated as a standalone piece (e.g., a blog post or blog comment, or a newspaper article), the `<article>` element would be a better choice.
- If the contents represent useful tangential information that works alongside the main content, but is not directly part of it (like related links, or an author bio), use an `<aside>`.
- If the contents represent the main content area of a document, use `<main>`.
- If you are only using the element as a styling wrapper, use a `<div>` instead.

To reiterate, each `<section>` should be identified, typically by including a heading (h1 - h6 element) as a child of the `<section>` element, wherever possible. See below for examples of where you might see a `<section>` without a heading.

### Examples

#### Basic usage example

#### Before

```
<div>
  <h2>Heading</h2>
  <p>Bunch of awesome content</p>
</div>
```

#### Result

#### After

```
<section>
  <h2>Heading</h2>
  <p>Bunch of awesome content</p>
</section>
```

#### Result

#### Using a section without a heading

Circumstances where you might see `<section>` used without a heading are typically found in web application/UI sections rather than in traditional document structures. In a document, it doesn't really make any sense to have a separate section of content without a heading to describe its contents. Such headings are useful for all readers, but particularly useful for users of assistive technologies like screen readers, and they are also good for SEO.

Consider however a secondary navigation mechanism. If the global navigation is already wrapped in a `<nav>` element, you could conceivably wrap a previous/next menu in a `<section>`:

```
<section>
  <a href="#">Previous article</a>
  <a href="#">Next article</a>
</section>
```

Or what about some kind of button bar for controlling your app? This might not necessarily want a heading, but it is still a distinct section of the document:

```
<section>
  <button class="reply">Reply</button>
  <button class="reply-all">Reply to all</button>
  <button class="fwd">Forward</button>
  <button class="del">Delete</button>
</section>
```

### Result

Depending on the content, including a heading could also be good for SEO, so it is an option to consider.

### Technical summary

| | |
|---|---|
| **Content categories** | Flow content, Sectioning content, palpable content. |
| **Permitted content** | Flow content. |
| **Tag omission** | None, both the starting and ending tag are mandatory. |
| **Permitted parents** | Any element that accepts flow content. Note that a `<section>` element must not be a descendant of an `<address>` element. |
| **Implicit ARIA role** | `region` if the element has an accessible name, otherwise `generic` |
| **Permitted ARIA roles** | `alert`, `alertdialog`, `application`, `banner`, `complementary`, `contentinfo`, `dialog`, `document`, `feed`, `log`, `main`, `marquee`, `navigation`, `none`, `note`, `presentation`, `search`, `status`, `tabpanel` |
| **DOM interface** | `HTMLElement` |

### See also

- Other section-related elements: `<body>`, `<nav>`, `<article>`, `<aside>`, h1, h2, h3, h4, h5, h6, `<hgroup>`, `<header>`, `<footer>`, `<address>`
- Using HTML sections and outlines
- ARIA: Region role
- Why You Should Choose HTML5 article Over section, by Bruce Lawson

## <select> HTML select element

The **`<select>`** HTML element represents a control that provides a menu of options.

### Try it

```
<label for="pet-select">Choose a pet:</label>

<select name="pets" id="pet-select">
  <option value="">--Please choose an option--</option>
  <option value="dog">Dog</option>
  <option value="cat">Cat</option>
  <option value="hamster">Hamster</option>
  <option value="parrot">Parrot</option>
  <option value="spider">Spider</option>
  <option value="goldfish">Goldfish</option>
</select>
```

```
label {
  font-family: sans-serif;
  font-size: 1rem;
  padding-right: 10px;
}

select {
  font-size: 0.9rem;
  padding: 2px 5px;
}
```

### Attributes

This element includes the global attributes.

`autocomplete`

> A string providing a hint for a user agent's autocomplete feature. See The HTML autocomplete attribute for a complete list of values and details on how to use autocomplete.

`autofocus`

> This Boolean attribute lets you specify that a form control should have input focus when the page loads. Only one form element in a document can have the `autofocus` attribute.

`disabled`

This Boolean attribute indicates that the user cannot interact with the control. If this attribute is not specified, the control inherits its setting from the containing element, for example `<fieldset>`; if there is no containing element with the `disabled` attribute set, then the control is enabled.

`form`

The `<form>` element to associate the `<select>` with (its *form owner*). The value of this attribute must be the `id` of a `<form>` in the same document. (If this attribute is not set, the `<select>` is associated with its ancestor `<form>` element, if any.)

This attribute lets you associate `<select>` elements to `<form>`s anywhere in the document, not just inside a `<form>`. It can also override an ancestor `<form>` element.

`multiple`

This Boolean attribute indicates that zero or more options can be selected in the list. If it is not specified, then only one option can be selected at a time. Multiple selected options are submitted using the `URLSearchParams` array convention, i.e., `name=value1&name=value2`. If `multiple` is specified, `size` defaults to `4` instead of `1`.

`name`

This attribute is used to specify the name of the control.

`required`

This Boolean attribute indicates that the user must select at least one option before the form can be submitted. The `<select>` has no selected options if it has no options, `multiple` is specified and the user deselects all options, the select's value is programmatically set to `""`, or only the *placeholder label option* is selected. Any option other than the placeholder label option is considered valid, even if its value is also empty.

The placeholder label option is the text shown in the box before the user makes a choice, like the "--Please choose an option--" in the Try it demo above. Semantically, it is considered equivalent to the `placeholder` attribute and is not considered an actual option. It is defined as the first option in the options list which is a direct child of the `<select>` (not inside an `<optgroup>`) and has an empty string as its value. It is only relevant when the `size` is `1` and `multiple` is not specified; in all other cases, such an `<option>` is just a regular option due to the way the `<select>` is rendered.

`size`

Specifies the number of options to display in the closed state as a positive integer. If the attribute is not specified, the default value is `1`, unless the `multiple` attribute is specified, in which case the default value is `4`. If the value is greater than `1` or the `multiple` attribute is present, browsers render a scrolling list box that has the specified number of rows visible; otherwise the options render as a drop-down list. For backwards compatibility, the `size` property always returns `0` as the default value.

### Usage notes

Typically, like other form controls, a `<select>` element is associated with a `<label>` for accessibility purposes, as well as a `name` attribute to represent the name of the associated data point submitted to the server. Each menu option is defined by an `<option>` element nested inside the `<select>`.

Each `<option>` element should have a `value` attribute containing the data value to submit to the server when that option is selected. If no `value` attribute is included, the value defaults to the text contained inside the element. You can include a `selected` attribute on an `<option>` element to make it selected by default when the page first loads. If no `selected` attribute is specified, the first `<option>` element will be selected by default.

A `<select>` element is represented in JavaScript by an `HTMLSelectElement` object, and this object has a `value` property which contains the value of the selected `<option>`.

You can further nest `<option>` elements inside `<optgroup>` elements to create separate groups of options inside the dropdown. You can also include `<hr>` elements to create separators that add visual breaks between options.

For further examples, see The native form widgets: Drop-down content.

### Options inside wrapper elements

The `<select>` element builds its list of options from all `<option>` descendants, not just its direct children. This means that options can be wrapped in other elements, such as `<div>` elements, and they will still appear as selectable options in the dropdown and be included in form submission. Wrapper elements are useful for styling in customizable select elements but have no effect on the select's behavior: they don't create groups, labels, or separators. To group options under a heading, use an `<optgroup>`; an `<option>` counts as part of an `<optgroup>` if the group is an ancestor, so wrapper elements may also be used inside a group without breaking the association.

> **Note:** Browsers with modern parsing behavior preserve all elements written inside a `<select>` in the DOM — including wrapper elements, `<button>`, and `<selectedcontent>`. Older browsers instead remove non-permitted elements while parsing, keeping only the `<option>`, `<optgroup>`, and `<hr>` structure. As a result, styling, markup, or scripting that depends on the removed elements will not work on older browsers.

### Selecting multiple options

A `<select>` element with the `multiple` attribute lets the user choose zero or more options. How the control is rendered depends on the `size` attribute:

- If the `multiple` attribute is set, browsers show a scrolling list box, unless the `size` is set to `1`. When `size` is omitted, the list box is four options tall, even if there are fewer options.
- If `size` is `1`, supporting browsers display a drop-down that lets the user select more than one option when the control is active. If exactly one option is selected, the selected option is displayed, otherwise, the control displays the count of its selected options as a single-line list box. Browsers that don't support these features display a multiple option menu list within a control that is the height of a single-line box.

When using the `multiple` attribute, always inform users that more than one option can be selected.

The following example uses `<select multiple size="1">` to create a drop-down that allows multiple selections:

```
<label for="flavors">Choose one or more ice cream flavors:</label>
<select id="flavors" name="flavors" multiple size="1">
  <option value="chocolate">Chocolate</option>
  <option value="strawberry">Strawberry</option>
  <option value="vanilla">Vanilla</option>
</select>
```

### Selecting options in a list box

On a desktop computer, there are a number of ways to select multiple options in a `<select>` element with a `multiple` attribute and a `size` attribute greater than `1`.

Mouse users can hold the `Ctrl` (`Command` on macOS) or `Shift` key (depending on what makes sense for your operating system) and then click multiple options to select/deselect them.

> **Note:** The keyboard mechanisms described below are not standardized and depend on the browser and operating system.
>
> For example, Firefox on macOS additionally supports using `Ctrl`, while Safari on macOS does not support using `Space` for non-contiguous selection.

Keyboard users can select multiple contiguous items by:

- Focusing on the `<select>` element (e.g., using `Tab`).
- Selecting an item at the top or bottom of the range they want to select using the `Up` and `Down` cursor keys to go up and down the options.
- Holding down the `Shift` key and then using the `Up` and `Down` cursor keys to increase or decrease the range of items selected.

Keyboard users can select multiple non-contiguous items by:

- Focusing on the `<select>` element (e.g., using `Tab`).
- Holding down the `Ctrl` key (`Command` on macOS) then using the `Up` and `Down` cursor keys to change the "focused" select option, i.e., the one that will be selected if you choose to do so. The "focused" select option is highlighted with a dotted outline, in the same way as a keyboard-focused link.
- Pressing `Space` to select/deselect "focused" select options.

### Styling with CSS

The `<select>` element has historically been difficult to style effectively with CSS. The following guides contain information about features that enable fully customizable select elements:

- Customizable select elements
- Customizable select listboxes

### Legacy select styling

In browsers that don't support the modern customization features (or legacy codebases where they can't be used), you are limited to manipulating the box model, the displayed font, etc. You can also use the `appearance` property to remove the default system `appearance`.

It is however, hard to get a consistent result across browsers with traditional `<select>` elements. If you want to get full control, you should consider using a library with good facilities for styling form widgets, or try rolling your own dropdown menu using non-semantic elements, JavaScript, and WAI-ARIA to provide semantics.

You can use the `:open` pseudo-class to style `<select>` elements in the open state, that is, when the drop-down options list is displayed. This doesn't apply to multi-line `<select>` elements (those with the `multiple` attribute set) — they tend to render as a scrolling list box rather than a drop-down, so don't have an open state.

For more information on legacy `<select>` styling, see:

- Styling HTML forms
- Advanced styling for HTML forms

- The `field-sizing` property, which controls how `<select>` elements are sized in relation to their contained options.

### Accessibility

The `<hr>` within a `<select>` should be considered purely decorative, as they are currently not exposed within the accessibility tree and therefore not exposed to assistive technologies.

Setting `size="1"` on a multiple select (i.e., `<select multiple size="1">`) renders a drop-down that lets users select multiple options. Some browsers don't expand the option list when the control is active, displaying multiple options in a menu list the height of a single-line box. This harms usability. When including a multiple-select, inform users they can select more than one option, even when multiple options are displayed.

### Examples

#### Basic select

The following example creates a three-value dropdown menu. The second option includes the `selected` attribute, making that option selected by default.

```
<select name="choice">
  <option value="first">First Value</option>
  <option value="second" selected>Second Value</option>
  <option value="third">Third Value</option>
</select>
```

#### Result

#### Select with grouping options

The following example creates a dropdown menu with grouping using `<optgroup>` and `<hr>` to make it easier for the user to understand the content in the dropdown.

```
<label for="hr-select">Your favorite food</label> <br />

<select name="foods" id="hr-select">
  <option value="">Choose a food</option>
  <hr />
  <optgroup label="Fruit">
    <option value="apple">Apples</option>
    <option value="banana">Bananas</option>
    <option value="cherry">Cherries</option>
    <option value="damson">Damsons</option>
  </optgroup>
  <hr />
  <optgroup label="Vegetables">
    <option value="artichoke">Artichokes</option>
    <option value="broccoli">Broccoli</option>
    <option value="cabbage">Cabbages</option>
  </optgroup>
  <hr />
  <optgroup label="Meat">
    <option value="beef">Beef</option>
    <option value="chicken">Chicken</option>
    <option value="pork">Pork</option>
  </optgroup>
  <hr />
  <optgroup label="Fish">
    <option value="cod">Cod</option>
    <option value="haddock">Haddock</option>
    <option value="salmon">Salmon</option>
    <option value="turbot">Turbot</option>
  </optgroup>
</select>
```

#### Result

#### Advanced select with multiple features

The following example is more complex, showing off more features you can use on a `<select>` element:

- The `multiple` attribute enables selecting more than one option.
- The `size` attribute is set to `4`, which means 4 lines are displayed at a time. Users can scroll to view all the options.
- Two `<optgroup>` elements are included, creating two visual groupings, generally with the group name being bolded and nested options being indented.
- The `disabled` attribute is included on the "Hamster" option, making that option not selectable.

```
<label>
  Please choose one or more pets:
  <select name="pets" multiple size="4">
    <optgroup label="4-legged pets">
      <option value="dog">Dog</option>
      <option value="cat">Cat</option>
      <option value="hamster" disabled>Hamster</option>
    </optgroup>
    <optgroup label="Flying pets">
      <option value="parrot">Parrot</option>
      <option value="macaw">Macaw</option>
      <option value="albatross">Albatross</option>
    </optgroup>
  </select>
</label>
```

### Result

### Technical summary

| | |
|---|---|
| **Content categories** | Flow content, phrasing content, interactive content, listed, labelable, resettable, and submittable form-associated element |
| **Permitted content** | • `<option>`, `<optgroup>`, or `<hr>` elements, optionally preceded by a `<button>` element with a nested `<selectedcontent>` element if a dropdown box.<br>• `<div>`, `<script>`, `<template>`, and `<noscript>` elements. |
| **Tag omission** | None, both the starting and ending tag are mandatory. |
| **Permitted parents** | Any element that accepts phrasing content. |
| **Implicit ARIA role** | `combobox` with **no** `multiple` attribute and **no** `size` attribute greater than 1, otherwise `listbox`. |
| **Permitted ARIA roles** | `menu` with **no** `multiple` attribute and **no** `size` attribute greater than 1, otherwise `combobox` is permitted but not recommended. |
| **DOM interface** | `HTMLSelectElement` |

### See also

- The `<option>` element
- The `<optgroup>` element
- Customizable select elements
- Events fired by `<select>`: `change`, `input`

## <selectedcontent> HTML selected option display element

The **`<selectedcontent>`** HTML is used inside a `<select>` element to display the contents of its currently selected `<option>` within its first child `<button>`. This enables you to style all parts of a `<select>` element, referred to as "customizable selects".

### Attributes

This element includes the global attributes.

### Description

You use the `<selectedcontent>` element as the only child of a `<button>` element, which must be the first child of the `<select>` element. Any `<option>` element, the only other valid child of `<select>`, must come after the `<button>` and nested `<selectedcontent>` pair.

```
<select>
  <button>
    <selectedcontent></selectedcontent>
  </button>
  <option></option>
  ...
</select>
```

### How <code><selectedcontent></code> works behind the scenes

The `<selectedcontent>` element contains a clone of the content of the currently selected `<option>`. The browser renders this clone using `cloneNode()`. When the selected `<option>` changes, such as during a `change` event, the contents of `<selectedcontent>` are replaced with a clone of the newly selected `<option>`. Being aware of this behavior is important, especially when working with dynamic content.

> **Warning:** Since the browser updates `<selectedcontent>` only when the selected `<option>` changes, any dynamic modifications to the content of the selected `<option>` after the `<select>` is rendered won't be cloned to `<selectedcontent>`. You'll need to update `<selectedcontent>` manually. Watch out if you're using any of the popular front-end JavaScript frameworks where `<option>` elements are updated dynamically after the initial render–the result may not match what you expect in `<selectedcontent>`.

#### <code><selectedcontent></code> inertness

By default, any `<button>` inside a `<select>` element is inert. As a result, all content inside that button—including `<selectedcontent>`—is also inert. This means users can't interact with or focus on content inside `<selectedcontent>`.

#### Styling the selected option's content with CSS

You can target the content of the currently selected `<option>` element and style how it appears inside the select button. Styling the button doesn't affect the styling of the content of the `<option>` that was cloned. This lets you customize how the selected option appears in the button, separately from how it appears in the drop-down list.

For example, your `<option>` elements may contain icons, images, or even videos that render nicely inside the drop-down, but could cause the select `<button>` to increase in size, look untidy, and affect the surrounding layout. By targeting the content inside `<selectedcontent>`, you can hide elements such as images in the button, without affecting how they appear in the drop-down, as shown in the following snippet:

```
selectedcontent img {
  display: none;
}
```

> **Note:** If the `<button>` and/or `<selectedcontent>` elements are not included inside `<select>`, the browser creates an implicit `<button>` to display the contents of the selected `<option>`. This fallback button cannot be targeted with CSS using the `button` or `selectedcontent` type selector.

### Examples

You can see a full example that includes the `<selectedcontent>` element in our Customizable select elements guide.

### Technical summary

| | |
|---|---|
| **Content categories** | None |
| **Permitted content** | Mirrors content from the selected `<option>`. |
| **Tag omission** | None, both the starting and ending tag are mandatory. |
| **Permitted parents** | A `<button>` element that is the first child of a `<select>` element. |
| **Implicit ARIA role** | None |
| **Permitted ARIA roles** | None |
| **DOM interface** | `HTMLSelectedContentElement` |

### See also

- The `<select>` element
- The `<option>` element
- The `<optgroup>` element
- Customizable select elements

## <slot> HTML web component slot element

The **`<slot>`** HTML element is a placeholder inside a Web Component that you can fill with your own markup when the component is used. This lets you create separate DOM trees and present them together.

Slots can contain plain text, other HTML elements, or other web components. A slot can also contain default content, which is displayed if the slot is not assigned other content when the web component is used.

### Attributes

This element includes the global attributes.

`name`

> The slot's name. A *named slot* is a `<slot>` element with a `name` attribute, while an *unnamed slot* has no `name` attribute, and the name defaults to the empty string.

When a shadow root uses named slot assignment, top-level child elements of its host are rendered in slots that have a matching name in their `slot` attribute. Slot names should be unique per shadow root: if you have two slots with the same name, all of the elements with a matching `slot` attribute are rendered in the *first* slot. All top-level child elements that don't have a `slot` attribute are rendered in the first unnamed `<slot>` element, which is referred to as the *default slot*. The `name` has no effect if the shadow root uses manual slot assignment.

For more information see `shadowrootslotassignment` on the `<template>` element and `Element.attachShadow()`.

## Examples

### Basic usage

This HTML shows how a number of named slots might be declared within a `<template>` element. Note that these slots are only used as slots when the template is used inside a shadow root.

```
<template id="element-details-template">
  <style>
    details {
      font-family: "Open Sans Light", "Helvetica", "Arial", sans-serif;
    }
    .name {
      font-weight: bold;
      color: #217ac0;
      font-size: 120%;
    }
    h4 {
      margin: 10px 0 -8px 0;
      background: #217ac0;
      color: white;
      padding: 2px 6px;
      border: 1px solid #cee9f9;
      border-radius: 4px;
    }
    .attributes {
      margin-left: 22px;
      font-size: 90%;
    }
    .attributes p {
      margin-left: 16px;
      font-style: italic;
    }
  </style>
  <details>
    <summary>
      <code class="name">
        &lt;<slot name="element-name">NEED NAME</slot>&gt;
      </code>
      <span class="desc"><slot name="description">NEED DESCRIPTION</slot></span>
    </summary>
    <div class="attributes">
      <h4>Attributes</h4>
      <slot name="attributes"><p>None</p></slot>
    </div>
  </details>
  <hr />
</template>
```

> **Note:** You can see this complete example in action at element-details (see it running live). In addition, you can find an explanation at Using templates and slots.

## Technical summary

| | |
|---|---|
| **Content categories** | Flow content, phrasing content |
| **Permitted content** | Transparent |
| **Events** | `slotchange` |
| **Tag omission** | None, both the starting and ending tag are mandatory. |
| **Permitted parents** | Any element that accepts phrasing content |
| **Implicit ARIA role** | No corresponding role |
| **Permitted ARIA roles** | No `role` permitted |
| **DOM interface** | `HTMLSlotElement` |

### See also

- HTML `<template>` element
- HTML `slot` attribute
- CSS `::slotted` pseudo-element
- CSS `:has-slotted` pseudo-class
- CSS scoping module
- Using templates and slots

## <small> HTML side comment element

The **`<small>`** HTML element represents side-comments and small print, like copyright and legal text, independent of its styled presentation. By default, it renders text within it one font-size smaller, such as from `small` to `x-small`.

### Try it

```
<p>
  MDN Web Docs is a learning platform for Web technologies and the software that
  powers the Web.
</p>

<hr />

<p>
  <small
    >The content is licensed under a Creative Commons Attribution-ShareAlike 2.5
    Generic License.</small
  >
</p>
```

```
small {
  font-size: 0.7em;
}
```

### Attributes

This element only includes the global attributes.

### Examples

### Basic usage

```
<p>
  This is the first sentence.
  <small>This whole sentence is in small letters.</small>
</p>
```

### Result

### CSS alternative

```
<p>
  This is the first sentence.
  <span class="small">This whole sentence is in small letters.</span>
</p>
```

```
.small {
  font-size: 0.8em;
}
```

### Result

### Notes

Although the `<small>` element, like the `<b>` and `<i>` elements, may be perceived to violate the principle of separation between structure and presentation, all three are valid in HTML. Authors are encouraged to use their best judgment when determining whether to use `<small>` or CSS.

### Technical summary

| | |
|---|---|
| **Content categories** | Flow content, phrasing content. |
| **Permitted content** | Phrasing content |

| | |
|---|---|
| **Tag omission** | None; must have both a start tag and an end tag. |
| **Permitted parents** | Any element that accepts phrasing content, or any element that accepts flow content. |
| **Implicit ARIA role** | `generic` |
| **Permitted ARIA roles** | Any |
| **DOM interface** | `HTMLElement` |

### See also

- `<b>`
- `<sub>` and `<sup>`
- `<font>`
- `<style>`

## <source> HTML media or image source element

The **`<source>`** HTML element specifies one or more media resources for the `<picture>`, `<audio>`, and `<video>` elements. It is a void element, which means that it has no content and does not require a closing tag. This element is commonly used to offer the same media content in multiple file formats in order to provide compatibility with a broad range of browsers given their differing support for image file formats and media file formats.

### Try it

```
<video controls width="250" height="200" muted>
  <source src="https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/source//shared-assets/videos/flower.webm"
type="video/webm" />
  <source src="https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/source//shared-assets/videos/flower.mp4"
type="video/mp4" />
  Download the
  <a href="https://developer.mozilla.org/shared-assets/videos/flower.webm">WEBM</a>
  or
  <a href="https://developer.mozilla.org/shared-assets/videos/flower.mp4">MP4</a>
  video.
</video>
```

### Attributes

This element supports all global attributes. In addition, the following attributes can be used with it:

`type`

Specifies the MIME media type of the image or other media type, optionally including a `codecs` parameter.

`src`

Specifies the URL of the media resource. Required if the parent of `<source>` is `<audio>` or `<video>`. Not allowed if the parent is `<picture>`.

`srcset`

Specifies a comma-separated list of one or more image URLs and their descriptors. Required if the parent of `<source>` is `<picture>`. Not allowed if the parent is `<audio>` or `<video>`.

The list consists of strings separated by commas, indicating a set of possible images for the browser to use. Each string is composed of:

- A URL specifying an image location.
- An optional width descriptor—a positive integer directly followed by `"w"`, such as `300w`.
- An optional pixel density descriptor—a positive floating number directly followed by `"x"`, such as `2x`.

Each string in the list must have either a width descriptor or a pixel density descriptor to be valid. These two descriptors should not be used together; only one should be used consistently throughout the list. The value of each descriptor in the list must be unique. The browser chooses the most adequate image to display at a given point of time based on these descriptors. If the descriptors are not specified, the default value used is `1x`. If the `sizes` attribute is also present, then each string must include a width descriptor. If the browser does not support `srcset`, then `src` will be used for the default image source.

`sizes`

Specifies a list of source sizes that describe the final rendered width of the image. Allowed if the parent of `<source>` is `<picture>`. Not allowed if the parent is `<audio>` or `<video>`.

The list consists of source sizes separated by commas. Each source size is media condition-length pair. Before laying the page out, the browser uses this information to determine which image defined in `srcset` to display. Note that `sizes` will take effect only if width descriptors are provided with `srcset`, not pixel density descriptors (i.e., `200w` should be used instead of `2x`).

`media`

Specifies the media query for the resource's intended media.

`height`

Specifies the intrinsic height of the image in pixels. Allowed if the parent of `<source>` is a `<picture>`. Not allowed if the parent is `<audio>` or `<video>`.

The height value must be an integer without any units.

`width`

Specifies the intrinsic width of the image in pixels. Allowed if the parent of `<source>` is a `<picture>`. Not allowed if the parent is `<audio>` or `<video>`.

The width value must be an integer without any units.

### Usage notes

The `<source>` element is a **void element**, which means that it not only has no content but also has no closing tag. That is, you *never* use `</source>` in your HTML.

The browser goes through a list of `<source>` elements to find a format it supports. It uses the first one it can display. For each `<source>` element:

- If the `type` attribute isn't specified, the browser retrieves the media's type from the server and determines if it can be displayed. If the media can't be rendered, the browser checks the next `<source>` in the list.
- If the `type` attribute is specified, the browser immediately compares it with the media types it can display. If the type is not supported, the browser skips querying the server and directly checks the next `<source>` element.

If none of the `<source>` elements provide a usable source:

- In the case of a `<picture>` element, the browser will fall back to using the image specified in the `<picture>` element's `<img>` child.
- In the case of an `<audio>` or `<video>` element, the browser will fall back to displaying the content included between the element's opening and closing tags.

For information about image formats supported by web browsers and guidance on selecting appropriate formats to use, see our Image file type and format guide. For details on the video and audio media types you can use, see the Media type and format guide.

### Examples

#### Using the <code>type</code> attribute with <code><video></code>

This example demonstrates how to offer a video in different formats: WebM for browsers that support it, Ogg for those that support Ogg, and QuickTime for browsers that support QuickTime. If the `<audio>` or `<video>` element is not supported by the browser, a notice is displayed instead. If the browser supports the element but does not support any of the specified formats, an `error` event is raised on the `<audio>` or `<video>` element and the default media controls (if enabled) will indicate an error. For more details on which media file formats to use and their browser support, refer to our Media type and format guide.

```
<video controls>
  <source src="https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/source/foo.webm" type="video/webm" />
  <source src="https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/source/foo.ogg" type="video/ogg" />
  <source src="https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/source/foo.mov" type="video/quicktime" />
  I'm sorry; your browser doesn't support HTML video.
</video>
```

#### Using the <code>media</code> attribute with <code><video></code>

This example demonstrates how to offer an alternate source file for viewports above a certain width. When a user's browsing environment meets the specified `media` condition, the associated `<source>` element is chosen. The contents of its `src` attribute are then requested and rendered. If the `media` condition does not match, the browser will move on to the next `<source>` in the list. The second `<source>` option in the code below has no `media` condition, so it will be selected for all other browsing contexts.

```
<video controls>
  <source src="https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/source/foo-large.webm" media="(width >= 800px)" />
  <source src="https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/source/foo.webm" />
  I'm sorry; your browser doesn't support HTML video.
</video>
```

For more examples, the HTML video and audio article in the Learn area is a great resource.

#### Using the <code>media</code> attribute with <code><picture></code>

In this example, two `<source>` elements are included within `<picture>`, providing versions of an image to use when the available space exceeds certain widths. If the available width is less than the smallest of these widths, the browser will fall back to the image specified in the `<img>` element.

```
<picture>
  <source srcset="mdn-logo-wide.png" media="(width >= 800px)" />
  <source srcset="mdn-logo-medium.png" media="(width >= 600px)" />
  <img src="https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/source/mdn-logo-narrow.png" alt="MDN Web Docs" />
</picture>
```

With the `<picture>` element, you must always include an `<img>` with a fallback image. Also, make sure to add an `alt` attribute for accessibility, unless the image is purely decorative and irrelevant to the content.

### Using <code>height</code> and <code>width</code> attributes with <code><picture></code>

In this example, three `<source>` elements with `height` and `width` attributes are included in a `<picture>` element. A media query allows the browser to select an image to display with the `height` and `width` attributes based on the viewport size.

```
<picture>
  <source
    srcset="landscape.png"
    media="(width >= 1000px)"
    width="1000"
    height="400" />
  <source
    srcset="square.png"
    media="(width >= 800px)"
    width="800"
    height="800" />
  <source
    srcset="portrait.png"
    media="(width >= 600px)"
    width="600"
    height="800" />
  <img
    src="https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/source/fallback.png"
    alt="Image used when the browser does not support the sources"
    width="500"
    height="400" />
</picture>
```

### Technical summary

| | |
|---|---|
| **Content categories** | None. |
| **Permitted content** | None; it is a void element. |
| **Tag omission** | Must have a start tag and must not have an end tag. |
| **Permitted parents** | A media element—`<audio>` or `<video>`—and it must be placed before any flow content or `<track>` element. A `<picture>` element, and it must be placed before the `<img>` element. |
| **Implicit ARIA role** | No corresponding role |
| **Permitted ARIA roles** | No `role` permitted |
| **DOM interface** | `HTMLSourceElement` |

### See also

- `<audio>` element
- `<picture>` element
- `<video>` element
- Image file type and format guide
- Media type and format guide
- Web performance

##  HTML content span element

The **`<span>`** HTML element is a generic inline container for phrasing content, which does not inherently represent anything. It can be used to group elements for styling purposes (using the `class` or `id` attributes), or because they share attribute values, such as `lang`. It should be used only when no other semantic element is appropriate. `<span>` is very much like a `<div>` element, but `<div>` is a block-level element whereas a `<span>` is an inline-level element.

### Try it

```
<p>
  Add the <span class="ingredient">basil</span>,
  <span class="ingredient">pine nuts</span> and
  <span class="ingredient">garlic</span> to a blender and blend into a paste.
```

```
</p>

<p>
  Gradually add the <span class="ingredient">olive oil</span> while running the
  blender slowly.
</p>
```

```
span.ingredient {
  color: red;
}
```

### Attributes

This element only includes the global attributes.

### Example

### Example 1

### HTML

```
<p><span>Some text</span></p>
```

### Result

### Example 2

### HTML

```
<li>
  <span>
    <a href="portfolio.html" target="_blank">See my portfolio</a>
  </span>
</li>
```

### CSS

```
li span {
  background: gold;
}
```

### Result

### Technical summary

| | |
|---|---|
| **Content categories** | Flow content, phrasing content. |
| **Permitted content** | Phrasing content. |
| **Tag omission** | None, both the starting and ending tag are mandatory. |
| **Permitted parents** | Any element that accepts phrasing content, or any element that accepts flow content. |
| **Implicit ARIA role** | No corresponding role |
| **Permitted ARIA roles** | Any |
| **DOM interface** | `HTMLSpanElement` |

### See also

- HTML `<div>` element

## <strike> HTML strikethrough element

The **`<strike>`** HTML element places a strikethrough (horizontal line) over text.

> **Warning:** This element is deprecated in HTML 4 and XHTML 1, and obsoleted in the HTML Living Standard. If semantically appropriate, i.e., if it represents *deleted* content, use `<del>` instead. In all other cases use `<s>`.

### Attributes

This element includes the global attributes.

### Examples

```
&lt;strike&gt;: <strike>Today's Special: Salmon</strike> SOLD OUT<br />
&lt;s&gt;: <s>Today's Special: Salmon</s> SOLD OUT
```

### Result

### Technical summary

| DOM interface | `HTMLElement` |
| --- | --- |

### See also

- The `<s>` element.
- The `<del>` element should be used if the data has been *deleted*.
- The CSS `text-decoration` property can be used to style text with a strikethrough.

##  HTML strong importance element

The **`<strong>`** HTML element indicates that its contents have strong importance, seriousness, or urgency. Browsers typically render the contents in bold type.

### Try it

```
<p>
  ... the most important rule, the rule you can never forget, no matter how much
  he cries, no matter how much he begs:
  <strong>never feed him after midnight</strong>.
</p>
```

```
p {
  font-size: 1rem;
}
```

### Attributes

This element only includes the global attributes.

### Usage notes

The `<strong>` element is for content that is of "strong importance," including things of great seriousness or urgency (such as warnings). This could be a sentence that is of great importance to the whole page, or you could merely try to point out that some words are of greater importance compared to nearby content.

Typically this element is rendered by default using a bold font weight. However, it should *not* be used to apply bold styling; use the CSS `font-weight` property for that purpose. Use the `<b>` element to draw attention to certain text without indicating a higher level of importance. Use the `<em>` element to mark text that has stress emphasis.

Another accepted use for `<strong>` is to denote the labels of paragraphs which represent notes or warnings within the text of a page.

#### <b> vs. <strong>

It is often confusing to new developers why there are so many ways to express the same thing on a rendered website. `<b>` and `<strong>` are perhaps one of the most common sources of confusion, causing developers to ask "Should I use `<b>` or `<strong>`? Don't they both do the same thing?"

Not exactly. The `<strong>` element is for content that is of greater importance, while the `<b>` element is used to draw attention to text without indicating that it's more important.

It may help to realize that both are valid and semantic elements in HTML and that it's a coincidence that they both have the same default styling (boldface) in most browsers (although some older browsers actually underline `<strong>`). Each element is meant to be used in certain types of scenarios, and if you want to bold text for decoration, you should instead actually use the CSS `font-weight` property.

The intended meaning or purpose of the enclosed text should be what determines which element you use. Communicating meaning is what semantics are all about.

#### <em> vs. <strong>

Adding to the confusion is the fact that while HTML 4 defined `<strong>` as indicating a stronger emphasis, HTML 5 defines `<strong>` as representing "strong importance for its contents." This is an important distinction to make.

While `<em>` is used to change the meaning of a sentence as spoken emphasis does ("I *love* carrots" vs. "I love *carrots*"), `<strong>` is used to give portions of a sentence added importance (e.g., "**Warning!** This is **very dangerous.**") Both `<strong>` and `<em>` can be nested to increase the relative

degree of importance or stress emphasis, respectively.

### Examples

#### Basic example

```
<p>
  Before proceeding, <strong>make sure you put on your safety goggles</strong>.
</p>
```

#### Result

#### Labeling warnings

```
<p>
  <strong>Important:</strong> Before proceeding, make sure you add plenty of
  butter.
</p>
```

#### Result

#### Technical summary

| | |
|---|---|
| **Content categories** | Flow content, phrasing content, palpable content. |
| **Permitted content** | Phrasing content. |
| **Tag omission** | None; must have both a start tag and an end tag. |
| **Permitted parents** | Any element that accepts phrasing content, or any element that accepts flow content. |
| **Implicit ARIA role** | `strong` |
| **Permitted ARIA roles** | Any |
| **DOM interface** | `HTMLElement` |

### See also

- The `<b>` element
- The `<em>` element
- The `font-weight` property

## <style> HTML style information element

The **`<style>`** HTML element contains style information for a document, or part of a document. It contains CSS, which is applied to the contents of the document containing the `<style>` element.

### Try it

```
<style>
  p {
    color: #26b72b;
  }
  code {
    font-weight: bold;
  }
</style>

<p>
  This text will be green. Inline styles take precedence over CSS included
  externally.
</p>

<p style="color: blue">
  The <code>style</code> attribute can override it, though.
</p>
```

```
p {
  color: red;
}
```

### Attributes

This element includes the global attributes.

`blocking`

This attribute explicitly indicates that certain operations should be blocked on the fetching of critical subresources and the application of the stylesheet to the document. `@import`-ed stylesheets are generally considered as critical subresources, whereas `background-image` and fonts are not. The operations that are to be blocked must be a space-separated list of blocking tokens listed below. Currently there is only one token:

- `render`: The rendering of content on the screen is blocked.

> **Note:** Only `style` elements in the document's `<head>` can possibly block rendering. By default, a `style` element in the `<head>` blocks rendering when the browser discovers it during parsing. If such a `style` element is added dynamically via script, you must additionally set `blocking = "render"` for it to block rendering.

`media`

This attribute defines which media the style should be applied to. Its value is a media query, which defaults to `all` if the attribute is missing.

`nonce`

A cryptographic nonce (number used once) used to allow inline styles in a style-src Content-Security-Policy. The server must generate a unique nonce value each time it transmits a policy. It is critical to provide a nonce that cannot be guessed as bypassing a resource's policy is otherwise trivial.

`title`

This attribute specifies alternative style sheet sets.

### Deprecated attributes

`type`

This attribute should not be provided: if it is, the only permitted values are the empty string or a case-insensitive match for `text/css`.

### Usage notes

The `<style>` element is typically included inside the `<head>` of the document. It can also be used in the template contents of a `<template>` element, which can include any elements.

If you include multiple `<style>` and `<link>` elements in your document, they will be applied to the DOM in the order they are included in the document — make sure you include them in the correct order, to avoid unexpected cascade issues.

In the same manner as `<link>` elements, `<style>` elements can include `media` attributes that contain media queries, allowing you to selectively apply internal stylesheets to your document depending on media features such as viewport width.

### Examples

#### A basic stylesheet

In the following example, we apply a short stylesheet to a document:

```
<!doctype html>
<html lang="en-US">
  <head>
    <meta charset="UTF-8" />
    <title>Test page</title>
    <style>
      p {
        color: red;
      }
    </style>
  </head>
  <body>
    <p>This is my paragraph.</p>
  </body>
</html>
```

#### Result

#### Using <code><style></code> inside <code><template></code>

A `<style>` element can also be placed inside a `<template>` element. The styles remain inactive until the template content is instantiated and inserted into the document.

```
<template id="card-template">
  <style>
    .card {
      border: 1px solid #cccccc;
      padding: 1rem;
      border-radius: 0.5rem;
    }
  </style>

  <div class="card">Template content</div>
</template>
```

### Multiple style elements

In this example we've included two `<style>` elements — notice how the conflicting declarations in the later `<style>` element override those in the earlier one, if they have equal specificity.

```
<!doctype html>
<html lang="en-US">
  <head>
    <meta charset="UTF-8" />
    <title>Test page</title>
    <style>
      p {
        color: white;
        background-color: blue;
        padding: 5px;
        border: 1px solid black;
      }
    </style>
    <style>
      p {
        color: blue;
        background-color: yellow;
      }
    </style>
  </head>
  <body>
    <p>This is my paragraph.</p>
  </body>
</html>
```

### Result

### Including a media query

In this example we build on the previous one, including a `media` attribute on the second `<style>` element so it is only applied when the viewport is less than 500px in width.

```
<!doctype html>
<html lang="en-US">
  <head>
    <meta charset="UTF-8" />
    <title>Test page</title>
    <style>
      p {
        color: white;
        background-color: blue;
        padding: 5px;
        border: 1px solid black;
      }
    </style>
    <style media="(width < 500px)">
      p {
        color: blue;
        background-color: yellow;
      }
    </style>
  </head>
  <body>
    <p>This is my paragraph.</p>
  </body>
</html>
```

### Result

### Technical summary

| | |
|---|---|
| **Content categories** | Metadata content. |
| **Permitted content** | Text content matching the `type` attribute, that is `text/css`. |
| **Tag omission** | Neither tag is omissible. |
| **Permitted parents** | Any element that accepts metadata content. |
| **Implicit ARIA role** | No corresponding role |
| **Permitted ARIA roles** | No `role` permitted |
| **DOM interface** | `HTMLStyleElement` |

### See also

- The `<link>` element, which allows us to apply external stylesheets to a document.
- Alternative Style Sheets

## <sub> HTML subscript element

The **`<sub>`** HTML element specifies inline text which should be displayed as subscript for solely typographical reasons. Subscripts are typically rendered with a lowered baseline using smaller text.

### Try it

```
<p>
  Almost every developer's favorite molecule is
  C<sub>8</sub>H<sub>10</sub>N<sub>4</sub>O<sub>2</sub>, also known as
  "caffeine."
</p>
```

```
p {
  font:
    1rem "Fira Sans",
    sans-serif;
}
```

### Attributes

This element only includes the global attributes.

### Usage notes

The `<sub>` element should be used only for typographical reasons—that is, to change the position of the text to comply with typographical conventions or standards, rather than solely for presentation or appearance purposes.

For example, using `<sub>` to style the name of a company which uses altered baselines in their wordmark would not be appropriate; instead, CSS should be used. For example, you could use the `vertical-align` property with a declaration like `vertical-align: sub` or, to more precisely control the baseline shift, `vertical-align: -25%`.

Appropriate use cases for `<sub>` include (but aren't necessarily limited to):

- Marking up footnote numbers. See Footnote numbers for an example.
- Marking up the subscript in mathematical variable numbers (although you may also consider using a MathML formula for this). See Variable subscripts.
- Denoting the number of atoms of a given element within a chemical formula (such as every developer's best friend, $C_8H_{10}N_4O_2$, otherwise known as "caffeine"). See Chemical formulas.

### Examples

#### Footnote numbers

Traditional footnotes are denoted using numbers which are rendered in subscript. This is a common use case for `<sub>`:

```
<p>
  According to the computations by Nakamura, Johnson, and Mason<sub>1</sub> this
  will result in the complete annihilation of both particles.
</p>
```

#### Result

#### Variable subscripts

In mathematics, families of variables related to the same concept (such as distances along the same axis) are represented using the same variable name with a subscript following. For example:

```
<p>
  The horizontal coordinates' positions along the X-axis are represented as
  <var>x<sub>1</sub></var> … <var>x<sub>n</sub></var>.
</p>
```

#### Result

#### Chemical formulas

When writing a chemical formula, such as $H_2O$, the number of atoms of a given element within the described molecule is represented using a subscripted number; in the case of water, the subscripted "2" indicates that there are two atoms of hydrogen in the molecule.

Another example:

```
<p>
  Almost every developer's favorite molecule is
  C<sub>8</sub>H<sub>10</sub>N<sub>4</sub>O<sub>2</sub>, which is commonly known
  as "caffeine."
</p>
```

#### Result

#### Technical summary

| | |
|---|---|
| **Content categories** | Flow content, phrasing content, palpable content. |
| **Permitted content** | Phrasing content. |
| **Tag omission** | None, both the starting and ending tag are mandatory. |
| **Permitted parents** | Any element that accepts phrasing content. |
| **Implicit ARIA role** | `subscript` |
| **Permitted ARIA roles** | Any |
| **DOM interface** | `HTMLElement` |

#### See also

- The `<sup>` HTML element that produces superscript. Note that you cannot use `sup` and `sub` both at the same time: you need to use MathML to produce both a superscript directly above a subscript next to the chemical symbol of an element, representing its atomic number and its nuclear number.
- The `<msub>`, `<msup>`, and `<msubsup>` MathML elements.
- The CSS `vertical-align` property.

### <summary> HTML disclosure summary element

The **`<summary>`** HTML element specifies a summary, caption, or legend for a `<details>` element's disclosure box. Clicking the `<summary>` element toggles the state of the parent `<details>` element open and closed.

#### Try it

```
<details>
  <summary>
    I have keys but no doors. I have space but no room. You can enter but can't
    leave. What am I?
  </summary>
  A keyboard.
</details>
```

```
details {
  border: 1px solid #aaaaaa;
  border-radius: 4px;
  padding: 0.5em 0.5em 0;
}

summary {
  font-weight: bold;
  margin: -0.5em -0.5em 0;
  padding: 0.5em;
}
```

```
details[open] {
  padding: 0.5em;
}

details[open] summary {
  border-bottom: 1px solid #aaaaaa;
  margin-bottom: 0.5em;
}
```

### Attributes

This element only includes the global attributes.

### Usage notes

The `<summary>` element's contents can be any heading content, plain text, or HTML that can be used within a paragraph.

A `<summary>` element may *only* be used as the first child of a `<details>` element. When the user clicks on the summary, the parent `<details>` element is toggled open or closed, and then a `toggle` event is sent to the `<details>` element, which can be used to let you know when this state change occurs.

The content of the `<details>` provides the accessible description for the `<summary>`.

### Default label text

If a `<details>` element's first child is not a `<summary>` element, the user agent will use a default string (typically "Details") as the label for the disclosure box.

### Default style

Per the HTML specification, the default style for `<summary>` elements includes `display: list-item`. This makes it possible to change or remove the icon displayed as the disclosure widget next to the label from the default, which is typically a triangle.

You can also change the style to `display: block` to remove the disclosure triangle.

See the Browser compatibility section for details, as not all browsers support full functionality of this element yet.

For WebKit-based browsers, such as Safari, it is possible to control the icon display through the non-standard CSS pseudo-element `::-webkit-details-marker`. To remove the disclosure triangle, use `summary::-webkit-details-marker { display: none }`.

### Examples

Below are some examples showing `<summary>` in use. You can find more examples in the documentation for the `<details>` element.

### Basic example

A basic example showing the use of `<summary>` in a `<details>` element:

```
<details open>
  <summary>Overview</summary>
  <ol>
    <li>Cash on hand: $500.00</li>
    <li>Current invoice: $75.30</li>
    <li>Due date: 5/6/19</li>
  </ol>
</details>
```

### Result

### Summaries as headings

You can use heading elements in `<summary>`, like this:

```
<details open>
  <summary><h4>Overview</h4></summary>
  <ol>
    <li>Cash on hand: $500.00</li>
    <li>Current invoice: $75.30</li>
    <li>Due date: 5/6/19</li>
  </ol>
</details>
```

### Result

This currently has some spacing issues that could be addressed using CSS.

> **Warning:** The role assigned to the `<summary>` element varies across browsers. Some still assign it a default `button` role, which removes all roles from its children. This inconsistency can cause issues for users of assistive technologies such as screen readers (`<h4>` in the previous example will have its role removed and will not be treated as a heading for these users). You should test your `<summary>` implementation on multiple platforms to ensure there's consistent accessibility support.

### HTML in summaries

This example adds some semantics to the `<summary>` element to indicate the label as important:

```
<details open>
  <summary><strong>Overview</strong></summary>
  <ol>
    <li>Cash on hand: $500.00</li>
    <li>Current invoice: $75.30</li>
    <li>Due date: 5/6/19</li>
  </ol>
</details>
```

### Result

### Changing the summary's icon

The `<summary>` element's marker, the disclosure triangle, can be customized with CSS. The marker can be targeted using the `::marker` pseudo-element. which accepts the `list-style` shorthand property and its longhand component properties, such as `list-style-type`. This enables changing the triangle to an image (usually with `list-style-image`) or a string (including emojis). In this example, we replace the content of one disclosure widget and remove the icon from another by setting `list-style: none` before adding a custom disclosure icon via generated content.

### CSS

In the first disclosure widget, we style the `::marker`, changing the `content` based on the `<details>` element's `[open]` attribute. For the second widget, we remove the marker with `list-style` properties, then add styled generated content with the `::after` pseudo-element. We also include styles for `::-webkit-details-marker` to target Safari. The selector for the browser-specific pseudo-element is included in an `:is()` pseudo-class so it doesn't invalidate the selector list.

```
details {
  font-size: 1rem;
  font-family: "Open Sans", "Calibri", sans-serif;
  border: solid;
  padding: 2px 6px;
  margin-bottom: 1em;
}

details:first-of-type summary::marker,
:is(::-webkit-details-marker) {
  content: "+ ";
  font-family: monospace;
  color: red;
  font-weight: bold;
}

details[open]:first-of-type summary::marker {
  content: "- ";
}

details:last-of-type summary {
  list-style: none;
  &::after {
    content: "+";
    color: white;
    background-color: darkgreen;
    border-radius: 1em;
    font-weight: bold;
    padding: 0 5px;
    margin-inline-start: 5px;
  }
  [open] &::after {
    content: "-";
  }
}
details:last-of-type summary::-webkit-details-marker {
  display: none;
}
```

The CSS includes the `[open]` attribute selector, matching only when the `open` attribute is present (when the `<details>` are open). The `:first-of-type` and `:last-of-type` pseudo-classes target the first and sibling elements of the same type. We included the prefixed `-webkit-` pseudo-element within a `:is()` pseudo-class as it takes a forgiving selector list, so if the prefixed pseudo-element is invalid in a browser, the whole selector block will not be invalid. We also used CSS nesting. See the CSS selectors module.

### HTML

```
<h1>Quotes from Helen Keller</h1>

<details>
  <summary>On women's rights</summary>
  <p>
    <q>We have prayed, we have coaxed, we have begged, for the vote, with the
      hope that men, out of chivalry, would bestow equal rights upon women and
      take them into partnership in the affairs of the state. We hoped that
      their common sense would triumph over prejudices and stupidity. We thought
      their boasted sense of justice would overcome the errors that so often
      fetter the human spirit; but we have always gone away empty-handed. We
      shall beg no more.</q>
  </p>
</details>

<details>
  <summary>On optimism</summary>
  <p>
    <q>Optimism is the faith that leads to achievement; nothing can be done
      without hope.</q>
  </p>
</details>
```

### Result

### Technical summary

| | |
|---|---|
| **Content categories** | none |
| **Permitted content** | Phrasing content, optionally intermixed with Heading content |
| **Tag omission** | None; both the start tag and the end tag are mandatory. |
| **Permitted parents** | The `<details>` element. |
| **Implicit ARIA role** | No corresponding role |
| **Permitted ARIA roles** | No `role` permitted |
| **DOM interface** | `HTMLElement` |

### See also

- `<details>`

## <sup> HTML superscript element

The **`<sup>`** HTML element specifies inline text which is to be displayed as superscript for solely typographical reasons. Superscripts are usually rendered with a raised baseline using smaller text.

### Try it

```
<p>
  The <em>Pythagorean theorem</em> is often expressed as the following equation:
</p>

<p>
  <var>a<sup>2</sup></var> + <var>b<sup>2</sup></var> = <var>c<sup>2</sup></var>
</p>
```

```
p {
  font:
    1rem "Fira Sans",
    sans-serif;
}
```

### Attributes

This element only includes the global attributes.

### Usage notes

The `<sup>` element should only be used for typographical reasons—that is, to change the position of the text to comply with typographical conventions or standards, rather than solely for presentation or appearance purposes.

For example, to style the wordmark of a business or product which uses a raised baseline should be done using CSS (most likely `vertical-align`) rather than `<sup>`. This would be done using, for example, `vertical-align: super` or, to shift the baseline up 50%, `vertical-align: 50%`.

Appropriate use cases for `<sup>` include (but aren't necessarily limited to):

- Displaying exponents, such as "$x^3$". It may be worth considering the use of MathML for these, especially in more complex cases. See Exponents under Examples below.
- Displaying superior lettering, which is used in some languages when rendering certain abbreviations. For example, in French, the word "mademoiselle" can be abbreviated "M$^{lle}$"; this is an acceptable use case. See Superior lettering for examples.
- Representing ordinal numbers, such as "4$^{th}$" instead of "fourth." See Ordinal numbers for examples.

### Examples

#### Exponents

Exponents, or powers of a number, are among the most common uses of superscripted text. For example:

```
<p>
  One of the most common equations in all of physics is <var>E</var>=<var>m</var
  ><var>c</var><sup>2</sup>.
</p>
```

#### Result

#### Superior lettering

Superior lettering is not technically the same thing as superscript. However, it is common to use `<sup>` to present superior lettering in HTML. Among the most common uses of superior lettering is the presentation of certain abbreviations in French:

```
<p>Robert a présenté son rapport à M<sup>lle</sup> Bernard.</p>
```

#### Result

#### Ordinal numbers

Ordinal numbers, such as "fourth" in English or "quinto" in Spanish may be abbreviated using numerals and language-specific text rendered in superscript:

```
<p>
  The ordinal number "fifth" can be abbreviated in various languages as follows:
</p>
<ul>
  <li>English: 5<sup>th</sup></li>
  <li>French: 5<sup>ème</sup></li>
</ul>
```

#### Result

### Technical summary

| | |
|---|---|
| **Content categories** | Flow content, phrasing content, palpable content. |
| **Permitted content** | Phrasing content. |
| **Tag omission** | None, both the starting and ending tag are mandatory. |
| **Permitted parents** | Any element that accepts phrasing content. |
| **Implicit ARIA role** | `superscript` |
| **Permitted ARIA roles** | Any |
| **DOM interface** | `HTMLElement` |

### See also

- The `<sub>` HTML element that produces subscripts. Note that you cannot use `sub` and `sup` at the same time: you need to use MathML to produce both a superscript and a subscript next to the chemical symbol of an element, representing its atomic number and its nuclear number.
- The `<msub>`, `<msup>`, and `<msubsup>` MathML elements.

- The CSS `vertical-align` property.

## <table> HTML table element

The **`<table>`** HTML element represents tabular data—that is, information presented in a two-dimensional table comprised of rows and columns of cells containing data.

### Try it

```
<table>
  <caption>
    Front-end web developer course 2021
  </caption>
  <thead>
    <tr>
      <th scope="col">Person</th>
      <th scope="col">Most interest in</th>
      <th scope="col">Age</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <th scope="row">Chris</th>
      <td>HTML tables</td>
      <td>22</td>
    </tr>
    <tr>
      <th scope="row">Dennis</th>
      <td>Web accessibility</td>
      <td>45</td>
    </tr>
    <tr>
      <th scope="row">Sarah</th>
      <td>JavaScript frameworks</td>
      <td>29</td>
    </tr>
    <tr>
      <th scope="row">Karen</th>
      <td>Web performance</td>
      <td>36</td>
    </tr>
  </tbody>
  <tfoot>
    <tr>
      <th scope="row" colspan="2">Average age</th>
      <td>33</td>
    </tr>
  </tfoot>
</table>
```

```
table {
  border-collapse: collapse;
  border: 2px solid rgb(140 140 140);
  font-family: sans-serif;
  font-size: 0.8rem;
  letter-spacing: 1px;
}

caption {
  caption-side: bottom;
  padding: 10px;
  font-weight: bold;
}

thead,
tfoot {
  background-color: rgb(228 240 245);
}

th,
td {
  border: 1px solid rgb(160 160 160);
  padding: 8px 10px;
}

td:last-of-type {
```

```
  text-align: center;
}

tbody > tr:nth-of-type(even) {
  background-color: rgb(237 238 242);
}

tfoot th {
  text-align: right;
}

tfoot td {
  font-weight: bold;
}
```

### Attributes

This element includes the global attributes.

### Deprecated attributes

The following attributes are deprecated and should not be used. They are documented below for reference when updating existing code and for historical interest only.

`align`

Specifies the horizontal alignment of the table within its parent element. The possible enumerated values are `left`, `center`, and `right`. Use the `margin-inline-start` and `margin-inline-end` CSS properties instead, as this attribute is deprecated.

`bgcolor`

Defines the background color of the table. The value is an HTML color; either a 6-digit hexadecimal RGB code, prefixed by a `#`, or a color keyword. Other CSS `<color>` values are not supported. Use the `background-color` CSS property instead, as this attribute is deprecated.

`border`

Defines, as a non-negative integer value (in pixels), the size of the frame surrounding the table. If set to `0`, the `frame` attribute is set to void. Use the `border` CSS property instead, as this attribute is deprecated.

`cellpadding`

Defines the space between the content of a cell and its border. This attribute is obsolete: instead of using it, apply the `padding` CSS property to the `<th>` and `<td>` elements.

`cellspacing`

Defines the size of the space between two cells. This attribute is obsolete: instead of using it, set the `border-spacing` CSS property on the `<table>` element. Note that this has no effect if the `<table>` element's `border-collapse` CSS property is set to `collapse`.

`frame`

Defines which side of the frame surrounding the table must be displayed. The possible enumerated values are `void`, `above`, `below`, `hsides`, `vsides`, `lhs`, `rhs`, `box` and `border`. Use the `border-style` and `border-width` CSS properties instead, as this attribute is deprecated.

`rules`

Defines where rules (borders) are displayed in the table. The possible enumerated values are `none` (default value), `groups` (`<thead>`, `<tbody>`, and `<tfoot>` elements), `rows` (horizontal lines), `cols` (vertical lines), and `all` (border around every cell). Use the `border` CSS property on the appropriate table-related elements, as well as on the `<table>` itself, instead, as this attribute is deprecated.

`summary`

Defines an alternative text that summarizes the content of the table. Use the `<caption>` element instead, as this attribute is deprecated.

`width`

Specifies the width of the table. Use the `width` CSS property instead, as this attribute is deprecated.

> **Note:** While no HTML specification includes `height` as a `<table>` attribute, some browsers support a non-standard interpretation of `height`. The unitless value sets a minimum absolute height in pixels. If set as a percent value, the minimum table height will be relative to the parent container's height. Use the `min-height` CSS property instead, as this attribute is deprecated.

### Visual layout of table contents

Following elements are part of the table structure:

- `<caption>`
- `<thead>`
- `<colgroup>`
- `<col>`
- `<th>`
- `<tbody>`
- `<tr>`
- `<td>`
- `<tfoot>`

The `<table>` box establishes a table formatting context. Elements inside the `<table>` generate rectangular boxes. Each box occupies a number of table cells according to the following rules:

1. The row boxes fill the table in the source code order from top to bottom. Each row box occupies one row of cells.
2. A row group box occupies one or more row boxes.
3. Column boxes are placed next to each other in source code order. Depending on the value of the `dir` attribute, the columns are laid in left-to-right or right-to-left direction. A column box occupies one or more columns of table cells.
4. A column group box occupies one or more column boxes.
5. A cell box may span over multiple rows and columns. User agents trim cells to fit in the available number of rows and columns.

Table cells do have padding. Boxes that make up a table do not have margins.

### Table layers and transparency

For styling purpose the table elements may be thought of as being put on six superimposed layers:

 Table element layers

The background set on an element in one layer will be visible only if the layers above it have transparent background. A missing cell is rendered as if an anonymous table-cell box occupied that place.

### Accessibility

### Captions

By supplying a `<caption>` element whose value clearly and concisely describes the table's purpose, it helps the people decide if they need to check the rest of the table content or skip over it.

This helps people navigating with the aid of assistive technology such as a screen reader, people experiencing low vision conditions, and people with cognitive concerns.

- MDN Adding a caption to your table with <caption>
- Caption & Summary • Tables • W3C WAI Web Accessibility Tutorials

### Scoping rows and columns

The `scope` attribute on header cells (`<th>` elements) is redundant in simple contexts, because scope is inferred. However, some assistive technologies may fail to draw correct inferences, so specifying header scope may improve user experiences. In complex tables, `scope` can be specified to provide necessary information about the cells related to a header.

- MDN table accessibility guide
- Tables with two headers • Tables • W3C WAI Web Accessibility Tutorials
- Tables with irregular headers • Tables • W3C WAI Web Accessibility Tutorials
- H63: Using the scope attribute to associate header cells and data cells in data tables | W3C Techniques for WCAG 2.0

### Complicated tables

Assistive technologies such as screen readers may have difficulty parsing tables that are so complex that header cells can't be associated in a strictly horizontal or vertical way. This is typically indicated by the presence of the `colspan` and `rowspan` attributes.

Ideally, consider alternate ways to present the table's content, including breaking it apart into a collection of smaller, related tables that don't have to rely on using the `colspan` and `rowspan` attributes. In addition to helping people who use assistive technology understand the table's content, this may also benefit people with cognitive concerns who may have difficulty understanding the associations the table layout is describing.

If the table cannot be broken apart, use a combination of the `id` and `headers` attributes to programmatically associate each table cell with the header(s) (`<th>` elements) the cell is associated with.

- MDN table accessibility guide
- Tables with multi-level headers • Tables • W3C WAI Web Accessibility Tutorials
- H43: Using id and headers attributes to associate data cells with header cells in data tables | Techniques for W3C WCAG 2.0

### Examples

The examples below include tables of progressively increasing complexity. See also our beginner's Styling tables guide for table styling information including common, useful techniques.

Since the structure of a `<table>` involves the use of several table-related HTML elements along with various associated attributes, the following examples are intended to provide a simplified explanation that covers the basics and common standards. Additional and more detailed information can be found on the corresponding linked pages.

These table examples demonstrate how to create an accessible table that is structured with HTML and styled with CSS.

Because of how HTML tables are structured, the markup can quickly grow. For this reason, it is important to clearly define the table's purpose and final appearance to create the appropriate structure. A logical structure developed with semantic markup is not only easier to style, but enables useful and accessible tables that can be understood and navigated by everyone, including search engines and users of assistive technologies.

The first example is basic, with subsequent examples growing in complexity. First, we will develop a very basic HTML table structure for the table. The first two examples contain no table section groups such as a defined head, body, or foot, and involve no cell spanning or explicitly defined cell relationships. Not even a caption is provided. As we work through the examples, they will be progressively enhanced to include all the table features that a complex data table should possess.

#### Basic table

This example includes a *very* basic table with three rows and two columns. To demonstrate default browser table styles, no CSS has been included in this example.

#### HTML

The table rows are defined with `<tr>` elements, and the columns are defined with table header and data cells within them. The first row contains the header cells (`<th>` elements) that serve as column headers for the data cells (`<td>` elements). Each element (`<th>` or `<td>`) per row is in its respective column—that is, the first element of a row is in the first column, and the second element of that row is in the second column.

```
<table>
  <tr>
    <th>Name</th>
    <th>Age</th>
  </tr>
  <tr>
    <td>Maria Sanchez</td>
    <td>28</td>
  </tr>
  <tr>
    <td>Michael Johnson</td>
    <td>34</td>
  </tr>
</table>
```

#### Result

There is no custom CSS or user stylesheet applied to this table. The styling results purely from the user-agent stylesheet.

#### Expanded table with header cells

This example extends the basic table, extending the content and adding basic CSS styles.

#### HTML

The table comprises four rows (`<tr>` elements) now, with four columns each. The first row is a row of header cells (The first row contains only `<th>` elements). Subsequent rows include a header column (`<th>` elements as the first child elements of each row) and three data columns (`<td>` elements). As table sectioning elements are not used, the browser automatically defines the content group structure, i.e., all rows are wrapped within the body of the table of an implicit `<tbody>` element.

```
<table>
  <tr>
    <th>Name</th>
    <th>ID</th>
    <th>Member Since</th>
    <th>Balance</th>
  </tr>
  <tr>
    <th>Margaret Nguyen</th>
    <td>427311</td>
    <td><time datetime="2010-06-03">June 3, 2010</time></td>
    <td>0.00</td>
  </tr>
```

```
  <tr>
    <th>Edvard Galinski</th>
    <td>533175</td>
    <td><time datetime="2011-01-13">January 13, 2011</time></td>
    <td>37.00</td>
  </tr>
  <tr>
    <th>Hoshi Nakamura</th>
    <td>601942</td>
    <td><time datetime="2012-07-23">July 23, 2012</time></td>
    <td>15.00</td>
  </tr>
</table>
```

### CSS

With CSS, we provide the basic styling to create lines around the components of the table to make the data structure clearer. The CSS adds a solid border around the `<table>` and around each of the table's cells, including those specified with both `<th>` and `<td>` elements, demarcating every header and data cells.

```
table {
  border: 2px solid rgb(140 140 140);
}

th,
td {
  border: 1px solid rgb(160 160 160);
}
```

### Result

## Specifying table cell relations

Before moving on to extend the table in more advanced ways, it's advisable to improve accessibility by defining relationships between the header and data cells (`<th>` and `<td>` elements).

### HTML

This is accomplished by introducing the `scope` attribute on the `<th>` elements and setting the values to the corresponding `col` (column) or `row` value.

```
<table>
  <tr>
    <th scope="col">Name</th>
    <th scope="col">ID</th>
    <th scope="col">Member Since</th>
    <th scope="col">Balance</th>
  </tr>
  <tr>
    <th scope="row">Margaret Nguyen</th>
    <td>427311</td>
    <td><time datetime="2010-06-03">June 3, 2010</time></td>
    <td>0.00</td>
  </tr>
  <tr>
    <th scope="row">Edvard Galinski</th>
    <td>533175</td>
    <td><time datetime="2011-01-13">January 13, 2011</time></td>
    <td>37.00</td>
  </tr>
  <tr>
    <th scope="row">Hoshi Nakamura</th>
    <td>601942</td>
    <td><time datetime="2012-07-23">July 23, 2012</time></td>
    <td>15.00</td>
  </tr>
</table>
```

The CSS and visual result are unchanged—the adaptation provides valuable contextual information for assistive technologies such as screen readers to help identify which cells the headers relate to.

> **Note:** If the table structure is even more complex, the (additional) use of the `headers` attribute on the `<th>` and `<td>` elements may improve accessibility and help assistive technologies identify the relationships between cells; see Complicated tables.

### Explicitly specifying table section groups

In addition to improving accessibility by specifying cell relations, the semantics of the table can be improved by introducing table section groups.

#### HTML

Since the first row (`<tr>` element) contains only column header cells and provides the header for the rest of the table's contents, it can be enclosed in the `<thead>` element to explicitly specify that row as the head section of the table. Moreover, what is automatically accomplished by the browser can also be defined explicitly—the body section of the table, which contains the main data of the table, is specified by enclosing the corresponding rows in the `<tbody>` element. The explicit use of the `<tbody>` element helps the browser to create the intended table structure, avoiding unwanted results.

```
<table>
  <thead>
    <tr>
      <th scope="col">Name</th>
      <th scope="col">ID</th>
      <th scope="col">Member Since</th>
      <th scope="col">Balance</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <th scope="row">Margaret Nguyen</th>
      <td>427311</td>
      <td><time datetime="2010-06-03">June 3, 2010</time></td>
      <td>0.00</td>
    </tr>
    <tr>
      <th scope="row">Edvard Galinski</th>
      <td>533175</td>
      <td><time datetime="2011-01-13">January 13, 2011</time></td>
      <td>37.00</td>
    </tr>
    <tr>
      <th scope="row">Hoshi Nakamura</th>
      <td>601942</td>
      <td><time datetime="2012-07-23">July 23, 2012</time></td>
      <td>15.00</td>
    </tr>
  </tbody>
</table>
```

Once again, the CSS and visual result are unchanged—specifying such table section groups provides valuable contextual information for assistive technologies, including screen readers and search engines, as well as for styling in the CSS, which will be shown in a later example.

### Column and row spanning

In this example, we extend the table even more by adding a column and introducing a multi-row head section.

#### HTML

Building on the table created so far, a new column for a "Membership End Date" is added in each body row with the `<td>` element. An additional row (`<tr>` element) is also added within the head section (`<thead>` element) to introduce a "Membership Dates" header as a heading for the "Joined" and "Canceled" columns.

The creation of the second header row involves adding `colspan` and `rowspan` attributes to the `<th>` elements to allocate the header cells to the correct columns and rows.

```
<table>
  <thead>
    <tr>
      <th scope="col" rowspan="2">Name</th>
      <th scope="col" rowspan="2">ID</th>
      <th scope="col" colspan="2">Membership Dates</th>
      <th scope="col" rowspan="2">Balance</th>
    </tr>
    <tr>
      <th scope="col">Joined</th>
      <th scope="col">Canceled</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <th scope="row">Margaret Nguyen</th>
      <td>427311</td>
```

```
      <td><time datetime="2010-06-03">June 3, 2010</time></td>
      <td>n/a</td>
      <td>0.00</td>
    </tr>
    <tr>
      <th scope="row">Edvard Galinski</th>
      <td>533175</td>
      <td><time datetime="2011-01-13">January 13, 2011</time></td>
      <td><time datetime="2017-04-08">April 8, 2017</time></td>
      <td>37.00</td>
    </tr>
    <tr>
      <th scope="row">Hoshi Nakamura</th>
      <td>601942</td>
      <td><time datetime="2012-07-23">July 23, 2012</time></td>
      <td>n/a</td>
      <td>15.00</td>
    </tr>
  </tbody>
</table>
```

### Result

The head section now has two rows, one with the headers (`<th>` elements) "Name", "ID", "Membership Dates", and "Balance", and a "Membership Dates" header with two subheaders that are in a second row: "Joined" and "Canceled". This is accomplished by:

- The first row's "Name", "ID", and "Balance" header cells span both table header rows by using the `rowspan` attribute, making them each two rows tall.
- The first row's "Membership Dates" header cell spans two columns using the `colspan` attribute, causing it to be two columns wide.
- The second row contains only the two header cells "Joined" and "Canceled" because the other three columns are merged with the cells in the first row that span two rows. The two header cells are correctly positioned under the "Membership Dates" header.

### Table caption and column summary

It's a common and advisable practice to provide a summary for the table's content, allowing users to quickly determine the table's relevance. Furthermore, the "Balance" column is summarized by displaying the sum of the balances of the individual members.

### HTML

A table summary is added by using a table caption (`<caption>` element) as the first child element of the `<table>`. The caption provides the accessible name or accessible description for the table.

Lastly, a table foot section (`<tfoot>` element) is added below the body, with a row that summarizes the "Balance" column by displaying a sum. The elements and attributes introduced earlier are applied.

```
<table>
  <caption>
    Status of the club members 2021
  </caption>
  <thead>
    <tr>
      <th scope="col" rowspan="2">Name</th>
      <th scope="col" rowspan="2">ID</th>
      <th scope="col" colspan="2">Membership Dates</th>
      <th scope="col" rowspan="2">Balance</th>
    </tr>
    <tr>
      <th scope="col">Joined</th>
      <th scope="col">Canceled</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <th scope="row">Margaret Nguyen</th>
      <td>427311</td>
      <td><time datetime="2010-06-03">June 3, 2010</time></td>
      <td>n/a</td>
      <td>0.00</td>
    </tr>
    <tr>
      <th scope="row">Edvard Galinski</th>
      <td>533175</td>
      <td><time datetime="2011-01-13">January 13, 2011</time></td>
      <td><time datetime="2017-04-08">April 8, 2017</time></td>
      <td>37.00</td>
    </tr>
```

```
    <tr>
      <th scope="row">Hoshi Nakamura</th>
      <td>601942</td>
      <td><time datetime="2012-07-23">July 23, 2012</time></td>
      <td>n/a</td>
      <td>15.00</td>
    </tr>
  </tbody>
  <tfoot>
    <tr>
      <th scope="row" colspan="4">Total balance</th>
      <td>52.00</td>
    </tr>
  </tfoot>
</table>
```

#### Result

### Basic table styling

Let's apply a basic style to the table to adjust the typeface and add a `background-color` to the head and foot rows. The HTML is unchanged this time, so let's dive right into the CSS.

### CSS

While a `font` property is added to the `<table>` element here to set a more visually appealing typeface (or an abominable sans-serif typeface, depending on your personal opinion), the interesting part is the second style, where the `<tr>` elements located within the `<thead>` and `<tfoot>` are styled adding a light blue `background-color`. This is a way to quickly apply a background color to all the cells in specific sections simultaneously.

```
table {
  border: 2px solid rgb(140 140 140);
  font:
    16px "Open Sans",
    "Helvetica",
    "Arial",
    sans-serif;
}

thead > tr,
tfoot > tr {
  background-color: rgb(228 240 245);
}

th,
td {
  border: 1px solid rgb(160 160 160);
}
```

#### Result

### Advanced table styling

Now we'll go all-out, with styles on rows in the header and body areas both, including alternating row colors, cells with different colors depending on position within a row, and so forth. Let's take a look at the result first this time.

#### Result

Here's what the final table will look like:

There is no change to the HTML again. See what proper preparation of the HTML structure can do?

### CSS

The CSS is much more involved this time. It's not complicated, but there's a lot going on. Let's break it down.

Here the `border-collapse` and `border-spacing` properties are added to eliminate spacing between cells and collapse borders that touch one another to be a single border instead of winding up with double borders. Additionally, the `<caption>` is placed at the `bottom` of the table using the `caption-side` property:

```
table {
  border-collapse: collapse;
  border-spacing: 0;
  border: 2px solid rgb(140 140 140);
  font:
    16px "Open Sans",
```

```
    "Helvetica",
    "Arial",
    sans-serif;
}

caption {
  caption-side: bottom;
  padding: 10px;
  font-weight: bold;
}
```

Next, the `padding` property is used to give all the table cells space around their content. The `vertical-align` property aligns the content of the header cells to the `bottom` of the cell, which can be seen on the cells in the head that span two rows:

```
th,
td {
  border: 1px solid rgb(160 160 160);
  padding: 4px 6px;
}

th {
  vertical-align: bottom;
}
```

The next CSS rule sets the `background-color` of all `<tr>` elements in the table's head (as specified using `<thead>`). Then the bottom border of the head is set to be a two-pixel wide line. Notice, however, that we're using the `:nth-of-type` selector to apply the `border-bottom` property to the *second* row in the head. Why? Because the head is made of two rows that are spanned by some of the cells. That means there are actually two rows there; applying the style to the first row would not give us the expected result:

```
thead > tr {
  background-color: rgb(228 240 245);
}

thead > tr:nth-of-type(2) {
  border-bottom: 2px solid rgb(140 140 140);
}
```

Let's style the two header cells "Joined" and "Canceled" with green and red hues to represent the "good" of a new member and the "bummer" of a canceled membership. Here we dig into the last row of the table's head section using the `:last-of-type` selector and give the first header cell in it (the "Joined" header) a greenish color, and the second header cell in it (the "Canceled" header) a reddish hue:

```
thead > tr:last-of-type > th:nth-of-type(1) {
  background-color: rgb(225 255 225);
}

thead > tr:last-of-type > th:nth-of-type(2) {
  background-color: rgb(255 225 225);
}
```

Since the first column should stand out as well, some custom styling is added here too. This CSS rule styles the first header cell in each row of the table's body with the `text-align` property to left-justify the member names, and with a somewhat different `background-color`:

```
tbody > tr > th:first-of-type {
  text-align: left;
  background-color: rgb(225 229 244);
}
```

It's common to help improve the readability of table data by alternating row colors—this is sometimes referred to as "zebra striping". Let's add a bit of `background-color` to every even row:

```
tbody > tr:nth-of-type(even) {
  background-color: rgb(237 238 242);
}
```

Since it's standard practice to right-justify currency values in tables, let's do that here. This just sets the `text-align` property for the last `<td>` in each body row to `right`:

```
tbody > tr > td:last-of-type {
  text-align: right;
}
```

Finally, some styling similar to the head is applied to the foot section of the table to make it stand out as well:

```
tfoot > tr {
  border-top: 2px dashed rgb(140 140 140);
  background-color: rgb(228 240 245);
}

tfoot th,
tfoot td {
  text-align: right;
  font-weight: bold;
}
```

### Displaying large tables in small spaces

A common issue with tables on the web is that they don't natively work very well on small screens when the amount of content is large, and the way to make them scrollable isn't obvious, especially when the markup may come from a CMS and cannot be modified to have a wrapper.

This example provides one way to display tables in small spaces. We've hidden the HTML content as it is very large, and there is nothing remarkable about it. The CSS is more useful to inspect in this example.

### CSS

When looking at these styles you'll notice that table's `display` property has been set to `block`. While this allows scrolling, the table loses some of its integrity, and table cells try to become as small as possible. To mitigate this issue we've set `white-space` to `nowrap` on the `<tbody>`. However, we don't do this for the `<thead>` to avoid long titles forcing columns to be wider than they need to be to display the data.

To keep the table headers on the page while scrolling down we've set `position` to sticky on the `<th>` elements. Note that we have **not** set `border-collapse` to `collapse`, as if we do the header cannot be separated correctly from the rest of the table.

Given that the `<table>` has a fixed size, the `overflow` set to `auto` is the important part here, as it makes the table scrollable.

```
table,
th,
td {
  border: 1px solid black;
}

table {
  overflow: auto;
  width: 100%;
  max-width: 400px;
  height: 240px;
  display: block;
  margin: 0 auto;
  border-spacing: 0;
}

tbody {
  white-space: nowrap;
}

th,
td {
  padding: 5px 10px;
  border-top-width: 0;
  border-left-width: 0;
}

th {
  position: sticky;
  top: 0;
  background: white;
  vertical-align: bottom;
}

th:last-child,
td:last-child {
  border-right-width: 0;
}

tr:last-child td {
  border-bottom-width: 0;
}
```

### Result

### Technical summary

| **Content categories** | Flow content |
| --- | --- |
| **Permitted content** | In this order:<br>1. an optional `<caption>` element,<br>2. zero or more `<colgroup>` elements,<br>3. an optional `<thead>` element,<br>4. either one of the following:<br>   - zero or more `<tbody>` elements<br>   - one or more `<tr>` elements<br>5. an optional `<tfoot>` element |
| **Tag omission** | None, both the starting and ending tag are mandatory. |
| **Permitted parents** | Any element that accepts flow content |
| **Implicit ARIA role** | `table` |
| **Permitted ARIA roles** | Any |
| **DOM interface** | `HTMLTableElement` |

### See also

- Learn: HTML table basics
- `<caption>`, `<col>`, `<colgroup>`, `<tbody>`, `<td>`, `<tfoot>`, `<th>`, `<thead>`, `<tr>`: Other table-related elements
- `background-color`: CSS property to set the background color of the table
- `border`, `border-collapse`, `border-spacing`: CSS properties to control the aspect of cell borders, rules, and frame
- `margin`, `padding`: CSS properties to align the table and set spacing on cell content
- `text-align`: CSS property to horizontally align table cell content
- `vertical-align`: CSS property to vertically align table cell content
- `width`: CSS property to control the width of the table

## <tbody> HTML table body element

The **`<tbody>`** HTML element encapsulates a set of table rows (`<tr>` elements), indicating that they comprise the body of a table's (main) data.

### Try it

```
<table>
  <caption>
    Council budget (in £) 2018
  </caption>
  <thead>
    <tr>
      <th scope="col">Items</th>
      <th scope="col">Expenditure</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <th scope="row">Donuts</th>
      <td>3,000</td>
    </tr>
    <tr>
      <th scope="row">Stationery</th>
      <td>18,000</td>
    </tr>
  </tbody>
  <tfoot>
    <tr>
      <th scope="row">Totals</th>
      <td>21,000</td>
    </tr>
  </tfoot>
</table>
```

```
thead,
tfoot {
  background-color: #2c5e77;
  color: white;
}

tbody {
```

```
  background-color: #e4f0f5;
}

table {
  border-collapse: collapse;
  border: 2px solid rgb(140 140 140);
  font-family: sans-serif;
  font-size: 0.8rem;
  letter-spacing: 1px;
}

caption {
  caption-side: bottom;
  padding: 10px;
}

th,
td {
  border: 1px solid rgb(160 160 160);
  padding: 8px 10px;
}

td {
  text-align: center;
}
```

### Attributes

This element includes the global attributes.

### Deprecated attributes

The following attributes are deprecated and should not be used. They are documented below for reference when updating existing code and for historical interest only.

`align`

> Specifies the horizontal alignment of each body cell. The possible enumerated values are `left`, `center`, `right`, `justify`, and `char`. When supported, the `char` value aligns the textual content on the character defined in the `char` attribute and on offset defined by the `charoff` attribute. Use the `text-align` CSS property instead, as this attribute is deprecated.

`bgcolor`

> Defines the background color of each body cell. The value is an HTML color; either a 6-digit hexadecimal RGB code, prefixed by a `#`, or a color keyword. Other CSS `<color>` values are not supported. Use the `background-color` CSS property instead, as this attribute is deprecated.

`char`

> Specifies the alignment of the content to a character of each body cell. Typical values for this include a period (`.`) when attempting to align numbers or monetary values. If `align` is not set to `char`, this attribute is ignored.

`charoff`

> Specifies the number of characters to offset the body cell content from the alignment character specified by the `char` attribute.

`valign`

> Specifies the vertical alignment of each body cell. The possible enumerated values are `baseline`, `bottom`, `middle`, and `top`. Use the `vertical-align` CSS property instead, as this attribute is deprecated.

### Usage notes

- The `<tbody>` is placed after any `<caption>`, `<colgroup>`, and `<thead>` elements.
- If `<tr>` elements are specified as direct children of the `<table>` (see "tag omission" in the technical summary for a description of when this is valid), then the markup generated by the browser will include a `<tbody>` element that encapsulates them. As a result, CSS selectors such as `table > tr` will not select these elements. See also the Not specifying a body example.
- It's permitted to use more than one `<tbody>` per table as long as they are all consecutive. This allows you to divide the rows (`<tr>` elements) in large tables into sections, each of which may be separately formatted if so desired. If not marked up to be consecutive elements, browsers will correct this author error, ensuring any `<thead>` and `<tfoot>` elements are rendered as the first and last elements of the table, respectively.
- Along with its related `<thead>` and `<tfoot>` elements, the `<tbody>` element provides useful semantic information and can be used when rendering for either screen or print. Specifying such table content groups also provides valuable contextual information for assistive technologies, including screen readers and search engines.
- When printing a document, in the case of a multipage table, the `<thead>` and `<tfoot>` elements usually specify information that remains the same—or at least very similar—on each page, while the `<tbody>` element's contents generally will differ from page to page.

- When a table is presented in a screen context (such as a window) that is not large enough to display the entire table, the user agent may let the user scroll the contents of the `<thead>`, `<tbody>`, `<tfoot>`, and `<caption>` blocks separately from one another for the same parent `<table>`.

### Examples

See `<table>` for a complete table example introducing common standards and best practices.

#### Not specifying a body

This example demonstrates that the browser automatically encapsulates `<tr>` elements within a `<tbody>` element if the rows are not nested within a table grouping element (`<tbody>`, `<tfoot>`, or `<thead>`) and are, as in this example, the direct children of the `<table>` element.

#### HTML

Here, a very basic table with some table rows (`<tr>` elements) containing data (`<td>` elements) about students is created.

```
<table>
  <tr>
    <td>3741255</td>
    <td>Jones, Martha</td>
    <td>Computer Science</td>
    <td>240</td>
  </tr>
  <tr>
    <td>3971244</td>
    <td>Nim, Victor</td>
    <td>Russian Literature</td>
    <td>220</td>
  </tr>
  <tr>
    <td>4100332</td>
    <td>Petrov, Alexandra</td>
    <td>Astrophysics</td>
    <td>260</td>
  </tr>
</table>
```

#### CSS

Note the CSS in the example, where a `background-color` is specified for the `<tbody>` element and the `tbody` is used as a part of an additional CSS selector. Alternatively, browser developer tools can be used to check the presence of the `<tbody>` element in the DOM.

```
tbody {
  background-color: #e4f0f5;
}

tbody > tr > td:last-of-type {
  width: 60px;
  text-align: center;
}
```

#### Result

#### Basic body structure

This example extends and enhances the basic table from the previous example.

#### HTML

We introduce a table head (`<thead>` element) and explicitly use a `<tbody>` element to structure the table into semantic sections. The table head contains the column headers (`<th>` elements). The `<tbody>` element represents the body section of the table, which contains a number of rows (`<tr>` elements) with the table's main data, which is the data of each student.

The use of such table content groups and semantic markup is not only useful for visual presentation (via CSS styling) and contextual information for assistive technologies; moreover, the explicit use of the `<tbody>` element helps the browser to create the intended table structure, avoiding unwanted results.

```
<table>
  <thead>
    <tr>
      <th>Student ID</th>
      <th>Name</th>
      <th>Major</th>
      <th>Credits</th>
```

```
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>3741255</td>
      <td>Jones, Martha</td>
      <td>Computer Science</td>
      <td>240</td>
    </tr>
    <tr>
      <td>3971244</td>
      <td>Nim, Victor</td>
      <td>Russian Literature</td>
      <td>220</td>
    </tr>
    <tr>
      <td>4100332</td>
      <td>Petrov, Alexandra</td>
      <td>Astrophysics</td>
      <td>260</td>
    </tr>
  </tbody>
</table>
```

### CSS

The CSS is nearly unchanged from the previous example, except for some basic styling to highlight the table head so that the headers of the columns stand out from the data in the table body. As in the example above, the `tbody` type selector is used to style the body cells.

```
tbody {
  background-color: #e4f0f5;
}

tbody > tr > td:last-of-type {
  text-align: center;
}

thead {
  border-bottom: 2px solid rgb(160 160 160);
  background-color: #2c5e77;
  color: white;
}
```

### Result

## Multiple bodies

This example extends and enhances the table from the example above even more by introducing multiple body sections.

Using multiple `<tbody>` elements allows creating row groupings within a table. Each table body can potentially have its own head row or rows; however, *there may only be one `<thead>` element per table*! Because of that, `<tr>` with `<th>` elements can be used to create headers within each `<tbody>`.

### HTML

Building on the table in the previous basic example, more students are added and, instead of listing each student's major on each row, the students are grouped by major. Note that each major is enclosed within its own `<tbody>` block, with the first row (`<tr>` element) serving as the head of the block, displaying the major title within a `<th>` element that uses the `colspan` attribute to span the header across all three columns of the table. Each remaining row within each major's `<tbody>` represents one student.

```
<table>
  <thead>
    <tr>
      <th>Student ID</th>
      <th>Name</th>
      <th>Credits</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <th colspan="3">Computer Science</th>
    </tr>
    <tr>
      <td>3741255</td>
      <td>Jones, Martha</td>
```

```
      <td>240</td>
    </tr>
    <tr>
      <td>4077830</td>
      <td>Pierce, Benjamin</td>
      <td>200</td>
    </tr>
    <tr>
      <td>5151701</td>
      <td>Kirk, James</td>
      <td>230</td>
    </tr>
  </tbody>
  <tbody>
    <tr>
      <th colspan="3">Russian Literature</th>
    </tr>
    <tr>
      <td>3971244</td>
      <td>Nim, Victor</td>
      <td>220</td>
    </tr>
  </tbody>
  <tbody>
    <tr>
      <th colspan="3">Astrophysics</th>
    </tr>
    <tr>
      <td>4100332</td>
      <td>Petrov, Alexandra</td>
      <td>260</td>
    </tr>
    <tr>
      <td>8892377</td>
      <td>Toyota, Hiroko</td>
      <td>240</td>
    </tr>
  </tbody>
</table>
```

### CSS

Most of the CSS is unchanged. However, a slightly more subtle style is added for header cells contained directly within a `<tbody>` (as opposed to those that reside in the `<thead>`). This is used for the headers indicating each table section's corresponding major.

```
tbody > tr > th {
  border-top: 2px solid rgb(160 160 160);
  border-bottom: 1px solid rgb(140 140 140);
  background-color: #e4f0f5;
  font-weight: normal;
}

tbody {
  background-color: whitesmoke;
}

thead {
  background-color: #2c5e77;
  color: white;
}
```

### Result

### Technical summary

| | |
|---|---|
| **Content categories** | None. |
| **Permitted content** | Zero or more `<tr>` elements. |
| **Tag omission** | A `<tbody>` element's start tag can be omitted if the first thing inside the `<tbody>` element is a `<tr>` element, and if the element is not immediately preceded by a `<tbody>`, `<thead>`, or `<tfoot>` element whose end tag has been omitted. (It can't be omitted if the element is empty.) A `<tbody>` element's end tag can be omitted if the `<tbody>` element is immediately followed by a `<tbody>` or `<tfoot>` element, or if there is no more content in the parent element. |

| | |
|---|---|
| **Permitted parents** | Within the required parent `<table>` element, the `<tbody>` element can be added after any `<caption>`, `<colgroup>`, and `<thead>` elements. |
| **Implicit ARIA role** | `rowgroup` |
| **Permitted ARIA roles** | Any |
| **DOM interface** | `HTMLTableSectionElement` |

See also

- Learn: HTML table basics
- `<caption>`, `<col>`, `<colgroup>`, `<table>`, `<td>`, `<tfoot>`, `<th>`, `<thead>`, `<tr>`: Other table-related elements
- `background-color`: CSS property to set the background color of each body cell
- `border`: CSS property to control borders of body cells
- `text-align`: CSS property to horizontally align each body cell content
- `vertical-align`: CSS property to vertically align each body cell content

## <td> HTML table data cell element

The **`<td>`** HTML element defines a cell of a table that contains data and may be used as a child of the `<tr>` element.

Try it

```
<table>
  <caption>
    Alien football stars
  </caption>
  <thead>
    <tr>
      <th scope="col">Player</th>
      <th scope="col">Gloobles</th>
      <th scope="col">Za'taak</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <th scope="row">TR-7</th>
      <td>7</td>
      <td>4,569</td>
    </tr>
    <tr>
      <th scope="row">Khiresh Odo</th>
      <td>7</td>
      <td>7,223</td>
    </tr>
    <tr>
      <th scope="row">Mia Oolong</th>
      <td>9</td>
      <td>6,219</td>
    </tr>
  </tbody>
</table>
```

```
th,
td {
  border: 1px solid rgb(160 160 160);
  padding: 8px 10px;
}

th[scope="col"] {
  background-color: #505050;
  color: white;
}

th[scope="row"] {
  background-color: #d6ecd4;
}

td {
  text-align: center;
}
```

```
tr:nth-of-type(even) {
  background-color: #eeeeee;
}

table {
  border-collapse: collapse;
  border: 2px solid rgb(140 140 140);
  font-family: sans-serif;
  font-size: 0.8rem;
  letter-spacing: 1px;
}

caption {
  caption-side: bottom;
  padding: 10px;
}
```

### Attributes

This element includes the global attributes.

`colspan`

> Contains a non-negative integer value that indicates how many columns the data cell spans or extends. The default value is `1`. Values higher than `1000` are clipped to `1000`.

`headers`

> Contains a list of space-separated strings, each corresponding to the `id` attribute of the `<th>` elements that provide headings for this table cell.

`rowspan`

> Contains a non-negative integer value that indicates for how many rows the data cell spans or extends. The default value is `1`; if its value is set to `0`, it extends until the end of the table grouping section (`<thead>`, `<tbody>`, `<tfoot>`, even if implicitly defined), that the cell belongs to. Values higher than `65534` are clipped to `65534`.

### Deprecated attributes

The following attributes are deprecated and should not be used. They are documented below for reference when updating existing code and for historical interest only.

`abbr`

> Contains a short abbreviated description of the data cell's content. Some user-agents, such as speech readers, may present this description before the content itself. Put the abbreviated content inside the cell and place the (longer) description in the `title` attribute, as this attribute is deprecated. Or, preferably, include the content within the data cell, and use CSS to visually clip overflowing text.

`align`

> Specifies the horizontal alignment of the data cell. The possible enumerated values are `left`, `center`, `right`, `justify`, and `char`. When supported, the `char` value aligns the textual content on the character defined in the `char` attribute and the offset defined by the `charoff` attribute. Use the `text-align` CSS property instead, as this attribute is deprecated.

`axis`

> Contains a list of space-separated strings, each corresponding to the `id` attribute of a group of cells that the data cell applies to.

`bgcolor`

> Defines the background color of the data cell. The value is an HTML color; either a 6-digit hexadecimal RGB code, prefixed by a `#`, or a color keyword. Other CSS `<color>` values are not supported. Use the `background-color` CSS property instead, as this attribute is deprecated.

`char`

> Does nothing. It was originally intended to specify the alignment of the content to a character of the data cell. Typical values for this include a period (`.`) when attempting to align numbers or monetary values. If `align` is not set to `char`, this attribute is ignored.

`charoff`

> Does nothing. It was originally intended to specify the number of characters to offset the data cell content from the alignment character specified by the `char` attribute.

`height`

> Defines a recommended data cell height. Use the `height` CSS property instead, as this attribute is deprecated.

`scope`

Defines the cells that the header (defined in the `<th>`) element relates to. The possible enumerated values are `row`, `col`, `rowgroup`, and `colgroup`. Only use this attribute with the `<th>` element to define the row or column for which it is a header, as this attribute is deprecated for the `<td>` element.

`valign`

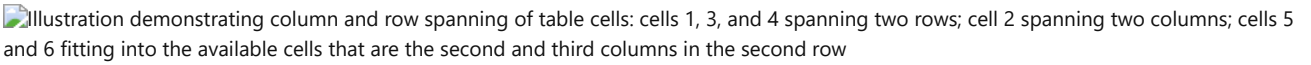
Specifies the vertical alignment of the data cell. The possible enumerated values are `baseline`, `bottom`, `middle`, and `top`. Use the `vertical-align` CSS property instead, as this attribute is deprecated.

`width`

Defines a recommended data cell width. Use the `width` CSS property instead, as this attribute is deprecated.

### Usage notes

- The `<td>` may only be used within a `<tr>` element.
- When using the `colspan` and `rowspan` attributes to span data cells across multiple columns and rows, cells without these attributes defined (with a default value of `1`) are automatically fitted into free available spaces in the table structure that span 1x1 cells, as illustrated in the following figure:

   Illustration demonstrating column and row spanning of table cells: cells 1, 3, and 4 spanning two rows; cell 2 spanning two columns; cells 5 and 6 fitting into the available cells that are the second and third columns in the second row

  > **Note:** These attributes must not be used to overlap cells.

### Examples

See `<table>` for a complete table example introducing common standards and best practices.

### Basic data cells

This example uses `<td>` elements along with other table-related elements to introduce a basic table with data about the phonetic alphabet.

### HTML

Some table rows (`<tr>` elements) contain both header cells (`<th>` elements) and data cell `<td>` elements. The `<th>` element that is the first child of each row forms the first column of the table, with each `<th>` providing the row header for the data cells within that row. Each corresponding `<td>` element contains data aligned with its respective column header and row header cell.

> **Note:** Normally, a table head group with column headers would be implemented to make it easier to understand the information in the columns. The `<thead>` and `<tbody>` elements would be used to group such rows of headers and data into the respective table head and body sections. This is not implemented in this example to focus on the data cells and reduce the complexity of this example.

```
<table>
  <tr>
    <th scope="row">A</th>
    <td>Alfa</td>
    <td>AL fah</td>
  </tr>
  <tr>
    <th scope="row">B</th>
    <td>Bravo</td>
    <td>BRAH voh</td>
  </tr>
  <tr>
    <th scope="row">C</th>
    <td>Charlie</td>
    <td>CHAR lee</td>
  </tr>
  <tr>
    <th scope="row">D</th>
    <td>Delta</td>
    <td>DELL tah</td>
  </tr>
</table>
```

### CSS

Some basic CSS is used to style the table and its cells. CSS attribute selectors and the `:nth-of-type` pseudo-class are used to alternate the appearance of the cells to make the information in the table easier to understand and identify.

```
td,
th {
  border: 1px solid rgb(160 160 160);
  padding: 8px 10px;
}

tr:nth-of-type(odd) td {
  background-color: #eeeeee;
}

tr th[scope="row"] {
  background-color: #d6ecd4;
}
```

### Result

### Column and row spanning

This example extends and enhances the basic table from the previous example by adding an additional "ABC" cell.

### HTML

An additional data cell (`<td>` element) is introduced within the first row (`<tr>` element). This creates a fourth column in the table.

Using the `rowspan` attribute, the "ABC" cell is spanned across the first three rows of the table. The last data cells of the subsequent rows each span two columns. This is done using the `colspan` attribute, aligning them correctly within the table structure. Note that an additional row (`<tr>` element) is added to the table to illustrate this.

```
<table>
  <tr>
    <th scope="row">A</th>
    <td>Alfa</td>
    <td>AL fah</td>
    <td rowspan="3">ABC</td>
  </tr>
  <tr>
    <th scope="row">B</th>
    <td>Bravo</td>
    <td>BRAH voh</td>
  </tr>
  <tr>
    <th scope="row">C</th>
    <td>Charlie</td>
    <td>CHAR lee</td>
  </tr>
  <tr>
    <th scope="row">D</th>
    <td>Delta</td>
    <td colspan="2">DELL tah</td>
  </tr>
  <tr>
    <th scope="row">E</th>
    <td>Echo</td>
    <td colspan="2">ECK oh</td>
  </tr>
</table>
```

### CSS

The `:first-of-type` and `:last-of-type` pseudo-classes are used in the CSS to select and style the added "ABC" data cell.

```
tr:first-of-type td:last-of-type {
  width: 60px;
  background-color: #505050;
  color: white;
  font-weight: bold;
  text-align: center;
}

td,
th {
  border: 1px solid rgb(160 160 160);
```

```
  padding: 8px 10px;
}

tr:nth-of-type(odd) td {
  background-color: #eeeeee;
}

tr th[scope="row"] {
  background-color: #d6ecd4;
}
```

### Result

### Associate data cells with header cells

For more complex relationships between data cells (`<td>` elements) and header cells (`<th>` elements), using `<th>` elements with the `scope` attribute alone may not be sufficient for assistive technologies, especially screen readers.

### HTML

To improve the accessibility of the previous example and to allow screen readers, for example, to speak the headers associated with each data cell, the `headers` attribute can be introduced along with `id` attributes. Each row header cell (`<th>` element) associated with the "ABC" data cell, i.e., the letters "A", "B", and "C", is given a unique identifier with the `id` attribute. The "ABC" data cell (`<td>` element) then uses these `id` values in a space-separated list for the `headers` attribute.

> **Note:** It's recommended to use more descriptive and useful values for the `id` attribute. Each `id` in a document must be unique to that document. In this example, the `id` values are single characters to maintain focus on the concept of the `headers` attribute.

```
<table>
  <tr>
    <th id="a" scope="row">A</th>
    <td>Alfa</td>
    <td>AL fah</td>
    <td headers="a b c" rowspan="3">ABC</td>
  </tr>
  <tr>
    <th id="b" scope="row">B</th>
    <td>Bravo</td>
    <td>BRAH voh</td>
  </tr>
  <tr>
    <th id="c" scope="row">C</th>
    <td>Charlie</td>
    <td>CHAR lee</td>
  </tr>
  <tr>
    <th scope="row">D</th>
    <td>Delta</td>
    <td colspan="2">DELL tah</td>
  </tr>
  <tr>
    <th scope="row">E</th>
    <td>Echo</td>
    <td colspan="2">ECK oh</td>
  </tr>
</table>
```

### Result

While the visual result is unchanged from the previous example table, each data cell (`<td>`) is now explicitly associated with its row header cell (`<th>`).

### Technical summary

| | |
|---|---|
| **Content categories** | Sectioning root. |
| **Permitted content** | Flow content. |
| **Tag omission** | The start tag is mandatory.<br>The end tag may be omitted, if it is immediately followed by a `<th>` or `<td>` element or if there are no more data in its parent element. |
| **Permitted parents** | A `<tr>` element. |
| **Implicit ARIA role** | `cell` if a descendant of a `<table>` element, or `gridcell` if a descendant of an element with `grid` role |

| | |
|---|---|
| **Permitted ARIA roles** | Any |
| **DOM interface** | `HTMLTableCellElement` |

### See also

- Learn: HTML table basics
- `<caption>`, `<col>`, `<colgroup>`, `<table>`, `<tbody>`, `<tfoot>`, `<th>`, `<thead>`, `<tr>`: Other table-related elements
- `background-color`: CSS property to set the background color of each data cell
- `border`: CSS property to control borders of data cells
- `height`: CSS property to control the recommended data cell height
- `text-align`: CSS property to horizontally align each data cell content
- `vertical-align`: CSS property to vertically align each data cell content
- `width`: CSS property to control the recommended data cell width
- `:nth-of-type`, `:first-of-type`, `:last-of-type`: CSS pseudo-classes to select the desired data cells

## `<template>` HTML content template element

The **`<template>`** HTML element serves as a mechanism for holding HTML fragments, which can either be used later via JavaScript, generated immediately and inserted into a shadow DOM, or used as part of out-of-order patching with `<template for="...">`.

### Attributes

This element includes the global attributes.

`for`

The `for` attribute is used for out-of-order patching with `<template for="...">`, matching an equivalent `<?start id="...">` or `<?marker "...">` marker. See the out-of-order patching section and examples section.

`shadowrootmode`

Creates a shadow root for the parent element. It is a declarative version of the `Element.attachShadow()` method and accepts the same enumerated values.

`open`

Exposes the internal shadow root DOM for JavaScript (recommended for most use cases).

`closed`

Hides the internal shadow root DOM from JavaScript.

> **Note:** The HTML parser creates a `ShadowRoot` object in the DOM for the first `<template>` in a node with this attribute set to an allowed value. If the attribute is not set, or not set to an allowed value — or if a `ShadowRoot` has already been declaratively created in the same parent — then an `HTMLTemplateElement` is constructed. A `HTMLTemplateElement` cannot subsequently be changed into a shadow root after parsing, for example, by setting `HTMLTemplateElement.shadowRootMode`.

> **Note:** You may find the non-standard `shadowroot` attribute in older tutorials and examples that used to be supported in Chrome 90-110. This attribute has since been removed and replaced by the standard `shadowrootmode` attribute.

`shadowrootclonable`

Sets the value of the `clonable` property of a `ShadowRoot` created using this element to `true`. If set, a clone of the shadow host (the parent element of this `<template>`) created with `Node.cloneNode()` or `Document.importNode()` will include a shadow root in the copy.

`shadowrootcustomelementregistry`

Sets the `customElementRegistry` property of a `ShadowRoot` created using this element to `null`, rather than the document's custom element registry. This allows a scoped `CustomElementRegistry` to be attached later using `CustomElementRegistry.initialize()`.

`shadowrootdelegatesfocus`

Sets the value of the `delegatesFocus` property of a `ShadowRoot` created using this element to `true`. If this is set and a non-focusable element in the shadow tree is selected, then focus is delegated to the first focusable element in the tree. The value defaults to `false`.

`shadowrootreferencetarget`

Sets the value of the `referenceTarget` property of a `ShadowRoot` created using this element. The value should be the ID of an element inside the shadow DOM. If set, target references to the host element from outside the shadow DOM will cause the referenced target element to become

the effective target of the reference to the host element.

`shadowrootserializable`

Sets the value of the `serializable` property of a `ShadowRoot` created using this element to `true`. If set, the shadow root may be serialized by calling the `Element.getHTML()` or `ShadowRoot.getHTML()` methods with the `options.serializableShadowRoots` parameter set `true`. The value defaults to `false`.

`shadowrootslotassignment`

Sets the `slotAssignment` property of a `ShadowRoot` created using this element. This is the declarative equivalent of the `slotAssignment` option of the `Element.attachShadow()` method.

`named`

Elements are automatically assigned to `<slot>` elements within this shadow root. This is the default value.

Elements with the `slot` attribute are assigned to the first `<slot>` in the template that has the corresponding `name` attribute. If multiple elements specify the same slot name, they are all added to the first slot in the template that has that name, and rendered in the order they are declared. All unnamed elements — elements that don't specify a `slot` attribute — are assigned to the default slot in the order they are declared. This is the first unnamed `<slot>` in the template.

`manual`

Elements are manually assigned to particular slot elements using `HTMLSlotElement.assign()`. No automatic assignment takes place.

### Usage notes

This element has no permitted content, because everything nested inside it in the HTML source does not actually become the children of the `<template>` element. The `Node.childNodes` property of the `<template>` element is always empty, and you can only access said nested content via the special `content` property. However, if you call `Node.appendChild()` or similar methods on the `<template>` element, then you would be inserting children into the `<template>` element itself, which is a violation of its content model and does not actually update the `DocumentFragment` returned by the `content` property.

Due to the way the `<template>` element is parsed, all `<html>`, `<head>`, and `<body>` opening and closing tags inside the template are syntax errors and are ignored by the parser, so `<template><head><title>Test</title></head></template>` is the same as `<template><title>Test</title></template>`.

There are three main ways to use the `<template>` element.

### Template document fragment

By default, the element's content is not rendered. The corresponding `HTMLTemplateElement` interface includes a standard `content` property (without an equivalent content/markup attribute). This `content` property is read-only and holds a `DocumentFragment` that contains the DOM subtree represented by the template.

The `Node.cloneNode()` and `Document.importNode()` methods both create a copy of a node. The difference is that `importNode()` clones the node in the context of the calling document, whereas `cloneNode()` uses the document of the node being cloned. The document context determines the `CustomElementRegistry` for constructing any custom elements. For this reason, use `document.importNode()` to clone the `content` fragment so that custom element descendants are constructed using the definitions in the current document, rather than the separate document that owns the template content. See the `Node.cloneNode()` page's examples for more details.

Note that the `DocumentFragment` container itself should not have data attached to it. See the Data on the DocumentFragment is not cloned example for more details.

### Declarative Shadow DOM

If the `<template>` element contains the `shadowrootmode` attribute with a value of either `open` or `closed`, the HTML parser will immediately generate a shadow DOM. The element is replaced in the DOM by its content wrapped in a `ShadowRoot`, which is attached to the parent element. This is the declarative equivalent of calling `Element.attachShadow()` to attach a shadow root to an element.

If the element has any other value for `shadowrootmode`, or does not have the `shadowrootmode` attribute, the parser generates a `HTMLTemplateElement`. Similarly, if there are multiple declarative shadow roots, only the first one is replaced by a `ShadowRoot` — subsequent instances are parsed as `HTMLTemplateElement` objects.

Other attributes prefixed with `shadowroot` allow declarative customization of the `ShadowRoot`, such as controlling how slots are assigned.

### Out-of-order patching

> **Note:** This use case is still experimental and has limited browser support. See the browser compatibility table for browser support information.

Traditionally, HTML is delivered in order and read, processed, and displayed from top to bottom. To change that order, you can either hide or rearrange elements with CSS or update the DOM produced by the HTML afterwards with JavaScript. However, many pages are composed of multiple

parts that may be ready to render at different times, or that may be more important to deliver to the user earlier.

The `<template>` element allows for delivering HTML out-of-order, which involves replacing processing instruction markers with the contents of the `<template>` element (also referred to as **patching**).

For example, a `<?marker name="my-identifier">` processing instruction marker can be patched with the contents of a `<template for="my-identifier">` element supplied much later in the HTML. See the Using `<template for>` for patching example.

As well as the `<?marker>` processing instructions marker, a `<?start>` and `<?end>` pair can be used to contain temporary content (for example, `<?start name="my-identifier">Loading...<?end>`), which is temporarily shown until the `<template for="my-identifier">` is processed and the whole section is replaced. See the Using `<template for>` for range patching example.

When written in HTML, processing instructions can be provided with or without the trailing `?`, and the browser will add it if not supplied when parsing the DOM. Both `<?start?>` and `<?start>` are therefore valid and parsed as `<?start?>`. XML is stricter and requires the trailing `?`.

If the `for` attribute does not match any marker processing instruction `name`, the `<template>` content will remain hidden in the DOM and won't be used in any patch.

To prevent components from updating unrelated parts of the DOM, `<template for="...">` elements can only patch markers inside the `<template>` parent's DOM tree. The only exception is `<template>` elements that are direct children of the `<body>` element — they can also patch `<head>` elements to allow updating `<title>` and other `<head>` elements.

### Examples

#### Generating table rows

First we start with the HTML portion of the example.

```
<table id="producttable">
  <thead>
    <tr>
      <th>UPC_Code</th>
      <th>Product_Name</th>
    </tr>
  </thead>
  <tbody>
    <!-- existing data could optionally be included here -->
  </tbody>
</table>

<template id="productrow">
  <tr>
    <td class="record"></td>
    <td></td>
  </tr>
</template>
```

First, we have a table into which we will later insert content using JavaScript code. Then comes the template, which describes the structure of an HTML fragment representing a single table row.

Now that the table has been created and the template defined, we use JavaScript to insert rows into the table, with each row being constructed using the template as its basis.

```
// Instantiate the table with the existing HTML tbody
// and the row with the template
const tbody = document.querySelector("tbody");
const template = document.querySelector("#productrow");

// Clone the new row and insert it into the table
const clone = document.importNode(template.content, true);
let td = clone.querySelectorAll("td");
td[0].textContent = "1235646565";
td[1].textContent = "Stuff";

tbody.appendChild(clone);

// Clone the new row and insert it into the table
const clone2 = document.importNode(template.content, true);
td = clone2.querySelectorAll("td");
td[0].textContent = "0384928528";
td[1].textContent = "Acme Kidney Beans 2";

tbody.appendChild(clone2);
```

The result is the original HTML table, with two new rows appended to it via JavaScript:

### Implementing a declarative shadow DOM

In this example, a hidden support warning is included at the beginning of the markup. This warning is later set to be displayed via JavaScript if the browser doesn't support the `shadowrootmode` attribute. Next, there are two `<article>` elements, each containing nested `<style>` elements with different behaviors. The first `<style>` element is global to the whole document. The second one is scoped to the shadow root generated in place of the `<template>` element because of the presence of the `shadowrootmode` attribute.

```
<p hidden>
  ⛔ Your browser doesn't support <code>shadowrootmode</code> attribute yet.
</p>
<article>
  <style>
    p {
      padding: 8px;
      background-color: wheat;
    }
  </style>
  <p>I'm in the DOM.</p>
</article>
<article>
  <template shadowrootmode="open">
    <style>
      p {
        padding: 8px;
        background-color: plum;
      }
    </style>
    <p>I'm in the shadow DOM.</p>
  </template>
</article>
```

```
const isShadowRootModeSupported = Object.hasOwn(
  HTMLTemplateElement.prototype,
  "shadowRootMode",
);

document
  .querySelector("p[hidden]")
  .toggleAttribute("hidden", isShadowRootModeSupported);
```

### Declarative Shadow DOM with delegated focus

This example demonstrates how `shadowrootdelegatesfocus` is applied to a shadow root that is created declaratively, and the effect this has on focus.

The code first declares a shadow root inside a `<div>` element, using the `<template>` element with the `shadowrootmode` attribute. This displays both a non-focusable `<div>` containing text and a focusable `<input>` element. It also uses CSS to style elements with `:focus` to blue, and to set the normal styling of the host element.

```
<div>
  <template shadowrootmode="open">
    <style>
      :host {
        display: block;
        border: 1px dotted black;
        padding: 10px;
        margin: 10px;
      }
      :focus {
        outline: 2px solid blue;
      }
    </style>
    <div>Clickable Shadow DOM text</div>
    <input type="text" placeholder="Input inside Shadow DOM" />
  </template>
</div>
```

The second code block is identical except that it sets the `shadowrootdelegatesfocus` attribute, which delegates focus to the first focusable element in the tree if a non-focusable element in the tree is selected.

```
<div>
  <template shadowrootmode="open" shadowrootdelegatesfocus>
    <style>
      :host {
        display: block;
```

```
        border: 1px dotted black;
        padding: 10px;
        margin: 10px;
      }
      :focus {
        outline: 2px solid blue;
      }
    </style>
    <div>Clickable Shadow DOM text</div>
    <input type="text" placeholder="Input inside Shadow DOM" />
  </template>
</div>
```

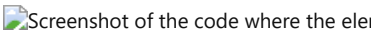
Last of all we use the following CSS to apply a red border to the parent `<div>` element when it has focus.

```
div:focus {
  border: 2px solid red;
}
```

The results are shown below. When the HTML is first rendered, the elements have no styling, as shown in the first image. For the shadow root that does not have `shadowrootdelegatesfocus` set you can click anywhere except the `<input>` and the focus does not change (if you select the `<input>` element it will look like the second image).


Screenshot of code with no focus set

For the shadow root with `shadowrootdelegatesfocus` set, clicking on the text (which is non-focusable) selects the `<input>` element, as this is the first focusable element in the tree. This also focuses the parent element as shown below.


Screenshot of the code where the element has focus

### Declarative shadow DOM with named slot assignment

This example shows how elements can be assigned to slots in a shadow DOM based on their `slot` attribute (matched against the slot's `name` attribute).

#### HTML

First we define an `<article>` element that presents title, metadata, and article body information.

The article contains a `<template>` element that will become a shadow root, because of the presence of the `shadowrootmode` attribute. We don't need to set its `shadowrootslotassignment` attribute because named slot assignment is the default.

The template defines elements that have named slots for "header" and "meta" information, and an unnamed slot for "body" information. The elements are styled differently so it is easy to differentiate them.

```
<article id="host">
  <template shadowrootmode="open" shadowrootslotassignment="named">
    <style>
      .header {
        background-color: plum;
      }
      .meta {
        background-color: green;
      }
      .body {
        background-color: lightblue;
      }
    </style>

    <h2 class="header">
      <slot name="title"></slot>
    </h2>

    <div class="meta">
      <slot name="meta"></slot>
    </div>

    <div class="body">
      <slot></slot>
    </div>
  </template>

  <p>
    Text 1 with no slot attribute. Goes into default (unnamed) slot inside the
    "body" div.
```

```
  </p>
  <span slot="title">Text for the title slot</span>
  <span slot="meta">Text for the meta slot</span>
  <p>
    Text 2 with no slot attribute. Also goes into default (unnamed) slot inside
    the "body" div.
  </p>
</article>
```

Inside the same host, below the template, we have four elements for populating the slots. The `<span>` elements have `slot` attributes that match the `name` attributes on slots in the template, and will populate the corresponding slots. The two `<p>` elements are unnamed, so are both inserted into the unnamed `<slot>` in the "body" element.

### Results

The example below should show the content of the slots displayed in the appropriate sections.

### Declarative shadow DOM with manual slot assignment

This example shows how elements can be assigned to slots in a shadow DOM using manual slot assignment.

With this approach, each element must manually be assigned to a particular slot. There is no default assignment, so any slot that is not assigned will be empty.

### HTML

First we have a hidden support warning. This warning is later set to be displayed via JavaScript if the browser doesn't support the `shadowrootslotassignment` attribute.

```
<p id="support-warning" hidden>
  ⛔ Your browser doesn't support the
  <code>shadowrootslotassignment</code> attribute yet.
</p>
```

Next, we define an `<article>` element that presents title, metadata, and article body information. This contains a `<template>` element that will become a shadow root, because of the presence of the `shadowrootmode` attribute, and will use manual slot assignment because `shadowrootslotassignment="manual"` is set.

The template defines elements that have slots for "header", "meta", and "body" information, that can be separately referenced by their `id` attribute. The elements are styled differently so it is easy to differentiate them.

```
<article id="host">
  <template shadowrootmode="open" shadowrootslotassignment="manual">
    <style>
      .header {
        background-color: plum;
      }
      .meta {
        background-color: green;
      }
      .body {
        background-color: lightblue;
      }
    </style>

    <h2 class="header">
      <slot id="titleSlot"></slot>
    </h2>

    <div class="meta">
      <slot id="metaSlot"></slot>
    </div>

    <div class="body">
      <slot id="bodySlot"></slot>
    </div>
  </template>

  <span id="text_title">Text for the title slot</span>
  <span id="text_meta">Text for the meta slot</span>
  <p id="text_body_1">Text 1 for body slot.</p>
  <p id="text_body_2">Text 2 for body slot.</p>
</article>
```

Inside the same host, below the template, we have four elements for populating the slots. These are also identified by id.

### JavaScript

The JavaScript for manual slot assignment is shown below. First the code gets the slots within the shadow root, then the text to be inserted, and finally assigns the text to the slot. Note that you can only assign a node once to any particular slot, and that if you assign multiple nodes to a single slot using `HTMLSlotElement.assign()`, the order they are specified controls the order they are added.

```
const host = document.querySelector("#host");
const shadow = host.shadowRoot;

// 1. Target your slots
const titleSlot = shadow.querySelector("#titleSlot");
const metaSlot = shadow.querySelector("#metaSlot");
const bodySlot = shadow.querySelector("#bodySlot");

// 2. Target the Elements to slot
const body1Text = document.querySelector("#text_body_1");
const body2Text = document.querySelector("#text_body_2");
const titleText = document.querySelector("#text_title");
const metaText = document.querySelector("#text_meta");

// 3. Manually assign them
titleSlot.assign(titleText);
metaSlot.assign(metaText);
bodySlot.assign(body2Text, body1Text);
```

The code displays the hidden support warning if slot assignment is not supported.

```
const isShadowRootSlotAssignmentSupported = Object.hasOwn(
  HTMLTemplateElement.prototype,
  "shadowRootSlotAssignment",
);

document
  .querySelector("p[hidden]")
  .toggleAttribute("hidden", isShadowRootSlotAssignmentSupported);
```

### Results

The example below should show the content of the slots displayed in the appropriate sections.

> **Note:** If the `shadowrootslotassignment` attribute is not supported, a warning note is displayed and the browser will use `named` assignment. However, because none of the slots or elements to be inserted are named, all the elements will be inserted into the title slot (because this is the first unnamed slot, and hence is the "default" slot).

## Data on the DocumentFragment is not cloned

When a `DocumentFragment` value is passed, `Node.appendChild` and similar methods move only the *child nodes* of that value into the target node. Therefore, it is usually preferable to attach event handlers to the children of a `DocumentFragment`, rather than to the `DocumentFragment` itself.

Consider the following HTML and JavaScript:

### HTML

```
<div id="container"></div>

<template id="template">
  <div>Click me</div>
</template>
```

### JavaScript

```
const container = document.getElementById("container");
const template = document.getElementById("template");

function clickHandler(event) {
  event.target.append(" — Clicked this div");
}

const firstClone = document.importNode(template.content, true);
firstClone.addEventListener("click", clickHandler);
container.appendChild(firstClone);

const secondClone = document.importNode(template.content, true);
```

```
secondClone.children[0].addEventListener("click", clickHandler);
container.appendChild(secondClone);
```

### Result

Since `firstClone` is a `DocumentFragment`, only its children are added to `container` when `appendChild` is called; the event handlers of `firstClone` are not copied. In contrast, because an event handler is added to the first *child node* of `secondClone`, the event handler is copied when `appendChild` is called, and clicking on it works as one would expect.

### Using <code><template for></code> for patching

This example uses the `<?marker name="placeholder">` processing instruction as a placeholder and later on fills in the contents using `<template for="placeholder">`.

```
<body>
  <div>
    <?marker name="placeholder">
  </div>
  ...
  <template for="placeholder">Lorem Ipsum...</template>
  ...
</body>
```

Initially, this results in an empty `<div>` being rendered. It is then updated to the following after the `<template>` element is parsed and processed:

```
  <div>
    Lorem Ipsum...
  </div>
```

### Using <code><template for></code> for range patching

This example uses the `<?start>` and `<?end>` processing instructions to contain placeholder content, which is initially displayed then replaced by the `<template for>` content later on.

```
<body>
  <div>
    <?start name="placeholder">
    Loading...
    <?end>
  </div>
  ...
  <template for="placeholder">Lorem Ipsum...</template>
  ...
</body>
```

Initially the `<div>` is rendered with the `Loading...` placeholder content. This is then updated to the following after the `<template>` is parsed and processed:

```
  <div>
    Lorem Ipsum...
  </div>
```

This example also demonstrates the lack of processing instruction children and nesting. The `<?start>` and `<?end>` processing instructions, although linked in terms of their relationship with `<template for>`, are separate nodes and not opening and closing tags. They therefore do not contain the `Loading...` content as a child (as demonstrated by the lack of indentation).

### Using <code><template for></code> for patching <code><head></code> elements

This example shows that `<template for>` elements that are direct children of the `<body>` element can patch `<head>` markers.

```
<head>
  ...
  <?start name="title"><title>Loading...</title><?end>
  <?start name="meta-description"><meta name="description" contents="Loading..."><?end>
  ...
</head>
<body>
  ...
  <template for="title"><title>The actual title of the page</title></template>
  <template for="meta-description"><meta name="description" contents="This is a meaningful description..."></template>
  ...
</body>
```

Results in the following once the `<template>` elements have been parsed:

```
<head>
  ...
  <title>The actual title of the page</title>
  <meta name="description" contents="This is a meaningful description...">
  ...
</head>
<body>
  ...
</body>
```

### Including markers in <code><template for></code> to allow contents to be repatched later

You can also insert markers inside `<template for>` elements, creating new placeholders to enable patching the same content multiple times. You can reuse existing `name` attributes.

For example, if you are building a Single Page Application (SPA) with `<template for>`, you may wish to allow the `<title>` to be patched on each route update, which could be achieved like this:

```
<head>
  ...
  <?start name="title">
  <title>Loading...</title>
  <?end>
  ...
</head>
<body>
  ...
  <template for="title"><?start name="title"><title>The actual title of the page</title><?end></template>
  ...
</body>
```

This will result in the following once the `<template>` element is parsed:

```
<head>
  ...
  <?start name="title"><title>The actual title of the page</title><?end>
  ...
</head>
<body>
  ...
</body>
```

Later on, a new `<template for="title">` could be inserted into the DOM to replace the `<title>` again.

### Technical summary

| | |
|---|---|
| **Content categories** | Metadata content, flow content, phrasing content, script-supporting element |
| **Permitted content** | Nothing (see Usage notes) |
| **Tag omission** | None, both the starting and ending tag are mandatory. |
| **Permitted parents** | Any element that accepts metadata content, phrasing content, or script-supporting elements. Also allowed as a child of a `<colgroup>` element that does *not* have a `span` attribute. |
| **Implicit ARIA role** | No corresponding role |
| **Permitted ARIA roles** | No `role` permitted |
| **DOM interface** | `HTMLTemplateElement` |

### See also

- `part` and `exportparts` HTML attributes
- `<slot>` HTML element
- `:has-slotted`, `:host`, `:host()`, and `:host-context()` CSS pseudo-classes
- `::part` and `::slotted` CSS pseudo-elements
- `ShadowRoot` interface
- Using templates and slots

- CSS scoping module
- Declarative Shadow DOM (with html) in *Using Shadow DOM*
- Declarative shadow DOM on web.dev (2023)

### <textarea> HTML textarea element

The **`<textarea>`** HTML element represents a multi-line plain-text editing control, useful when you want to allow users to enter a sizeable amount of free-form text, for example a comment on a review or feedback form.

#### Try it

```
<label for="story">Tell us your story:</label>

<textarea id="story" name="story" rows="5" cols="33">
It was a dark and stormy night...
</textarea>
```

```
label,
textarea {
  font-size: 0.8rem;
  letter-spacing: 1px;
}

textarea {
  padding: 10px;
  max-width: 100%;
  line-height: 1.5;
  border-radius: 5px;
  border: 1px solid #cccccc;
  box-shadow: 1px 1px 1px #999999;
}

label {
  display: block;
  margin-bottom: 10px;
}
```

The above example demonstrates a number of features of `<textarea>`:

- An `id` attribute to allow the `<textarea>` to be associated with a `<label>` element for accessibility purposes
- A `name` attribute to set the name of the associated data point submitted to the server when the form is submitted.
- `rows` and `cols` attributes to allow you to specify an exact size for the `<textarea>` to take. Setting these is a good idea for consistency, as browser defaults can differ.
- The `<textarea>` element specifies its content differently in HTML and JavaScript contexts:
  - In HTML, the initial content of a `<textarea>` is specified between its opening and closing tags, not as a `value` attribute.
  - In JavaScript, `<textarea>` elements have a `value` property that can be used to get or set the current content, and `defaultValue` to get and set its initial value (equivalent to accessing the HTML element's text content).

The `<textarea>` element also accepts several attributes common to form `<input>`s, such as `autocapitalize`, `autocomplete`, `autofocus`, `disabled`, `placeholder`, `readonly`, and `required`.

#### Attributes

This element includes the global attributes.

`autocapitalize`

> Controls whether inputted text is automatically capitalized and, if so, in what manner.

`autocomplete`

> Controls whether entered text can be automatically completed by the browser. Possible values are:
>
> - `off`: The user must explicitly enter a value into this field for every use, or the document provides its own auto-completion method; the browser does not automatically complete the entry.
> - `on`: The browser can automatically complete the value based on values that the user has entered during previous uses.
> - `<token-list>`: An ordered set of space-separated autofill detail tokens, optionally preceded by a sectioning token, a billing or shipping grouping token, and/or a token identifying the type of recipient.
>
> `<textarea>` elements that don't specify the `autocomplete` attribute inherit the `autocomplete` `on` or `off` status set on the `<textarea>`'s form owner. The form owner is either the `<form>` element that this `<textarea>` element is a descendant of or the form element whose `id` is specified by the `form` attribute of the input element. For more information, see the `autocomplete` attribute in `<form>`.

`autocorrect`

Controls whether automatic spelling correction and processing of text is enabled while the user is editing this `textarea`. Permitted values are:

`on`

Enable automatic spelling correction and text substitutions.

`off`

Disable automatic spelling correction and text substitutions.

`autofocus`

This Boolean attribute lets you specify that a form control should have input focus when the page loads. Only one form-associated element in a document can have this attribute specified.

`cols`

The visible width of the text control, in average character widths. If it is specified, it must be a positive integer. If it is not specified, the default value is `20`.

`dirname`

This attribute is used to indicate the text directionality of the element contents. For more information, see the `dirname` attribute.

`disabled`

This Boolean attribute indicates that the user cannot interact with the control. If this attribute is not specified, the control inherits its setting from the containing element, for example `<fieldset>`; if there is no containing element when the `disabled` attribute is set, the control is enabled.

`form`

The form element that the `<textarea>` element is associated with (its "form owner"). The value of the attribute must be the `id` of a form element in the same document. If this attribute is not specified, the `<textarea>` element must be a descendant of a form element. This attribute enables you to place `<textarea>` elements anywhere within a document, not just as descendants of form elements.

`maxlength`

The maximum string length (measured in UTF-16 code units) that the user can enter. If this value isn't specified, the user can enter an unlimited number of characters.

`minlength`

The minimum string length (measured in UTF-16 code units) required that the user should enter.

`name`

The name of the control.

`placeholder`

A hint to the user of what can be entered in the control. Carriage returns or line-feeds within the placeholder text must be treated as line breaks when rendering the hint.

> **Note:** Placeholders should only be used to show an example of the type of data that should be entered into a form; they are *not* a substitute for a proper `<label>` element tied to the input. See `<input>` labels for a full explanation.

`readonly`

This Boolean attribute indicates that the user cannot modify the value of the control. Unlike the `disabled` attribute, the `readonly` attribute does not prevent the user from clicking or selecting in the control. The value of a read-only control is still submitted with the form.

`required`

This attribute specifies that the user must fill in a value before submitting a form.

`rows`

The number of visible text lines for the control. If it is specified, it must be a positive integer. If it is not specified, the default value is 2.

`spellcheck`

Specifies whether the `<textarea>` is subject to spell-checking by the underlying browser/OS. The value can be:

- `true`: Indicates that the element needs to have its spelling and grammar checked.

- `default` : Indicates that the element is to act according to a default behavior, possibly based on the parent element's own `spellcheck` value.
- `false` : Indicates that the element should not be spell-checked.

`wrap`

Indicates how the control should wrap the value for form submission. Possible values are:

- `hard`: The browser automatically inserts line breaks (CR+LF) so that each line is no longer than the width of the control; the `cols` attribute must be specified for this to take effect
- `soft`: The browser ensures that all line breaks in the entered value are a `CR+LF` pair, but no additional line breaks are added to the value.
- `off` : Like `soft` but changes appearance to `white-space: pre` so line segments exceeding `cols` are not wrapped and the `<textarea>` becomes horizontally scrollable.

If this attribute is not specified, `soft` is its default value.

### Styling with CSS

`<textarea>` is a replaced element — it has intrinsic dimensions, like a raster image. By default, its `display` value is `inline-block`. Compared to other form elements it is relatively easy to style, with its box model, fonts, color scheme, etc. being easily manipulable using regular CSS.

Styling HTML forms provides some useful tips on styling `<textarea>`s.

### Baseline inconsistency

The HTML specification doesn't define where the baseline of a `<textarea>` is, so different browsers set it to different positions. For Gecko, the `<textarea>` baseline is set on the baseline of the first line of the textarea, on another browser it may be set on the bottom of the `<textarea>` box. Don't use `vertical-align: baseline` on it; the behavior is unpredictable.

### Controlling whether a textarea is resizable

In most browsers, `<textarea>`s are resizable — you'll notice the drag handle in the right-hand corner, which can be used to alter the size of the element on the page. This is controlled by the `resize` CSS property — resizing is enabled by default, but you can explicitly disable it using a `resize` value of `none`:

```
textarea {
  resize: none;
}
```

### Styling valid and invalid values

Valid and invalid values of a `<textarea>` element (e.g., those within, and outside the bounds set by `minlength`, `maxlength`, or `required`) can be highlighted using the `:valid` and `:invalid` pseudo-classes. For example, to give your textarea a different border depending on whether it is valid or invalid:

```
textarea:invalid {
  border: 2px dashed red;
}

textarea:valid {
  border: 2px solid lime;
}
```

### Examples

### Basic example

The following example shows a textarea with a set number of rows and columns, some default content, and CSS styles that prevent users from resizing the element more than 500px wide and 130px high:

```
<textarea name="textarea" rows="5" cols="15">Write something here</textarea>
```

```
textarea {
  max-height: 130px;
  max-width: 500px;
}
```

### Result

### Example using "minlength" and "maxlength"

This example has a minimum and maximum number of characters — of 10 and 20 respectively. Try it and see.

```
<textarea name="textarea" rows="5" cols="30" minlength="10" maxlength="20">
Write something here…
</textarea>
```

```
textarea {
  max-height: 130px;
  max-width: 500px;
}
```

### Result

Note that `minlength` doesn't stop the user from removing characters so that the number entered goes past the minimum, but it does make the value entered into the `<textarea>` invalid. Also note that even if you have a `minlength` value set (3, for example), an empty `<textarea>` is still considered valid unless you also have the `required` attribute set.

### Example using "placeholder"

This example has a placeholder set. Notice how it disappears when you start typing into the box.

```
<textarea
  name="textarea"
  rows="5"
  cols="30"
  placeholder="Comment text."></textarea>
```

```
textarea {
  max-height: 130px;
  max-width: 500px;
}
```

### Result

> **Note:** Placeholders should only be used to show an example of the type of data that should be entered into a form; they are *not* a substitute for a proper `<label>` element tied to the input. See `<input>` labels for a full explanation.

### Disabled and readonly text areas

This example shows two `<textarea>`s — one is `readonly` and one is `disabled`. You cannot edit the contents of either element, but the `readonly` element is focusable and its value is submitted in forms. The `disabled` element's value is not submitted and it's not focusable.

```
<textarea name="textarea" rows="5" cols="30" readonly>
I am a read-only textarea.
</textarea>
<textarea name="textarea" rows="5" cols="30" disabled>
I am a disabled textarea.
</textarea>
```

```
textarea {
  display: block;
  resize: horizontal;
  max-width: 500px;
}
```

### Result

### Technical summary

| | |
|---|---|
| **Content categories** | Flow content, phrasing content, Interactive content, listed, labelable, resettable, and submittable form-associated element. |
| **Permitted content** | Text |
| **Tag omission** | None, both the starting and ending tag are mandatory. |
| **Permitted parents** | Any element that accepts phrasing content. |
| **Implicit ARIA role** | `textbox` |
| **Permitted ARIA roles** | No `role` permitted |
| **DOM interface** | `HTMLTextAreaElement` |

### See also

- Other form-related elements:
  - `<form>`
  - `<button>`
  - `<datalist>`
  - `<legend>`
  - `<label>`
  - `<select>`
  - `<optgroup>`
  - `<option>`
  - `<input>`
  - `<fieldset>`
  - `<output>`
  - `<progress>`
  - `<meter>`

## <tfoot> HTML table foot element

The **`<tfoot>`** HTML element encapsulates a set of table rows (`<tr>` elements), indicating that they comprise the foot of a table with information about the table's columns. This is usually a summary of the columns, e.g., a sum of the given numbers in a column.

### Try it

```
<table>
  <caption>
    Council budget (in £) 2018
  </caption>
  <thead>
    <tr>
      <th scope="col">Items</th>
      <th scope="col">Expenditure</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <th scope="row">Donuts</th>
      <td>3,000</td>
    </tr>
    <tr>
      <th scope="row">Stationery</th>
      <td>18,000</td>
    </tr>
  </tbody>
  <tfoot>
    <tr>
      <th scope="row">Totals</th>
      <td>21,000</td>
    </tr>
  </tfoot>
</table>
```

```
thead,
tfoot {
  background-color: #2c5e77;
  color: white;
}

tbody {
  background-color: #e4f0f5;
}

table {
  border-collapse: collapse;
  border: 2px solid rgb(140 140 140);
  font-family: sans-serif;
  font-size: 0.8rem;
  letter-spacing: 1px;
}

caption {
  caption-side: bottom;
  padding: 10px;
}
```

```
th,
td {
  border: 1px solid rgb(160 160 160);
  padding: 8px 10px;
}

td {
  text-align: center;
}
```

### Attributes

This element includes the global attributes.

### Deprecated attributes

The following attributes are deprecated and should not be used. They are documented below for reference when updating existing code and for historical interest only.

`align`

> Specifies the horizontal alignment of each foot cell. The possible enumerated values are `left`, `center`, `right`, `justify`, and `char`. When supported, the `char` value aligns the textual content on the character defined in the `char` attribute and on offset defined by the `charoff` attribute. Use the `text-align` CSS property instead, as this attribute is deprecated.

`bgcolor`

> Defines the background color of each foot cell. The value is an HTML color; either a 6-digit hexadecimal RGB code, prefixed by a `#`, or a color keyword. Other CSS `<color>` values are not supported. Use the `background-color` CSS property instead, as this attribute is deprecated.

`char`

> Does nothing. It was originally intended to specify the alignment of the content to a character of each foot cell. Typical values for this include a period (`.`) when attempting to align numbers or monetary values. If `align` is not set to `char`, this attribute is ignored.

`charoff`

> Does nothing. It was originally intended to specify the number of characters to offset the foot cell content from the alignment character specified by the `char` attribute.

`valign`

> Specifies the vertical alignment of each foot cell. The possible enumerated values are `baseline`, `bottom`, `middle`, and `top`. Use the `vertical-align` CSS property instead, as this attribute is deprecated.

### Usage notes

- The `<tfoot>` is placed after any `<caption>`, `<colgroup>`, `<thead>`, `<tbody>`, and `<tr>` elements.
- Along with its related `<thead>` and `<tbody>` elements, the `<tfoot>` element provides useful semantic information and can be used when rendering for either screen or print. Specifying such table content groups also provides valuable contextual information for assistive technologies, including screen readers and search engines.
- When printing a document, in the case of a multipage table, the table foot usually specifies information that is output as an intermediate summary on each page.

### Example

See `<table>` for a complete table example introducing common standards and best practices.

### Table with footer

This example demonstrates a table divided into a head section with column headers, a body section with the table's main data, and a foot section summarizing data of one column.

### HTML

The `<thead>`, `<tbody>`, and `<tfoot>` elements are used to structure a basic table into semantic sections. The `<tfoot>` element represents the foot section of the table, which contains a row (`<tr>`) representing the calculated average of the values in the "Credits" column.

To allocate the cells in the foot to the correct columns, the `colspan` attribute is used on the `<th>` element to span row header cell across the first three table columns. The single data cell (`<td>`) in the foot is automatically placed in the correct location, i.e., the fourth column, with the omitted `colspan` attribute value defaulting to `1`. Additionally, the `scope` attribute is set to `row` on the header cell (`<th>`) in the foot to explicitly define that this foot header cell relates to the table cells within the same row, which in our example is the one data cell in the foot row that contains the calculated average.

```
<table>
  <thead>
    <tr>
      <th>Student ID</th>
      <th>Name</th>
      <th>Major</th>
      <th>Credits</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>3741255</td>
      <td>Jones, Martha</td>
      <td>Computer Science</td>
      <td>240</td>
    </tr>
    <tr>
      <td>3971244</td>
      <td>Nim, Victor</td>
      <td>Russian Literature</td>
      <td>220</td>
    </tr>
    <tr>
      <td>4100332</td>
      <td>Petrov, Alexandra</td>
      <td>Astrophysics</td>
      <td>260</td>
    </tr>
  </tbody>
  <tfoot>
    <tr>
      <th colspan="3" scope="row">Average Credits</th>
      <td>240</td>
    </tr>
  </tfoot>
</table>
```

### CSS

Some basic CSS is used to style and highlight the table foot so that the foot cells stand out from the data in the table body.

```
tfoot {
  border-top: 3px dotted rgb(160 160 160);
  background-color: #2c5e77;
  color: white;
}

tfoot th {
  text-align: right;
}

tfoot td {
  font-weight: bold;
}

thead {
  border-bottom: 2px solid rgb(160 160 160);
  background-color: #2c5e77;
  color: white;
}

tbody {
  background-color: #e4f0f5;
}
```

### Result

### Technical summary

| | |
|---|---|
| **Content categories** | None. |
| **Permitted content** | Zero or more `<tr>` elements. |

| | |
|---|---|
| **Tag omission** | The start tag is mandatory. The end tag may be omitted if there is no more content in the parent `<table>` element. |
| **Permitted parents** | A `<table>` element. The `<tfoot>` must appear after any `<caption>`, `<colgroup>`, `<thead>`, `<tbody>`, and `<tr>` elements. Note that this is the requirement in HTML.<br>Originally, in HTML4, the opposite was true: the `<tfoot>` element could not be placed after any `<tbody>` and `<tr>` elements. |
| **Implicit ARIA role** | `rowgroup` |
| **Permitted ARIA roles** | Any |
| **DOM interface** | `HTMLTableSectionElement` |

See also

- Learn: HTML table basics
- `<caption>`, `<col>`, `<colgroup>`, `<table>`, `<tbody>`, `<td>`, `<th>`, `<thead>`, `<tr>`: Other table-related elements
- `background-color`: CSS property to set the background color of each foot cell
- `border`: CSS property to control borders of foot cells
- `text-align`: CSS property to horizontally align each foot cell content
- `vertical-align`: CSS property to vertically align each foot cell content

## <th> HTML table header element

The **`<th>`** HTML element defines a cell as the header of a group of table cells and may be used as a child of the `<tr>` element. The exact nature of this group is defined by the `scope` and `headers` attributes.

Try it

```
<table>
  <caption>
    Alien football stars
  </caption>
  <thead>
    <tr>
      <th scope="col">Player</th>
      <th scope="col">Gloobles</th>
      <th scope="col">Za'taak</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <th scope="row">TR-7</th>
      <td>7</td>
      <td>4,569</td>
    </tr>
    <tr>
      <th scope="row">Khiresh Odo</th>
      <td>7</td>
      <td>7,223</td>
    </tr>
    <tr>
      <th scope="row">Mia Oolong</th>
      <td>9</td>
      <td>6,219</td>
    </tr>
  </tbody>
</table>
```

```
th,
td {
  border: 1px solid rgb(160 160 160);
  padding: 8px 10px;
}

th[scope="col"] {
  background-color: #505050;
  color: white;
}

th[scope="row"] {
  background-color: #d6ecd4;
}
```

```
td {
  text-align: center;
}

tr:nth-of-type(even) {
  background-color: #eeeeee;
}

table {
  border-collapse: collapse;
  border: 2px solid rgb(140 140 140);
  font-family: sans-serif;
  font-size: 0.8rem;
  letter-spacing: 1px;
}

caption {
  caption-side: bottom;
  padding: 10px;
}
```

### Attributes

This element includes the global attributes.

`abbr`

A short, abbreviated description of the header cell's content provided as an alternative label to use for the header cell when referencing the cell in other contexts. Some user-agents, such as screen readers, may present this description before the content itself.

`colspan`

A non-negative integer value indicating how many columns the header cell spans or extends. The default value is `1`. Values higher than `1000` are clipped to `1000`.

`headers`

A list of space-separated strings corresponding to the `id` attributes of the `<th>` elements that provide the headers for this header cell.

`rowspan`

A non-negative integer value indicating how many rows the header cell spans or extends. The default value is `1`; if its value is set to `0`, the header cell will extend to the end of the table grouping section (`<thead>`, `<tbody>`, `<tfoot>`, even if implicitly defined), that the `<th>` belongs to. Values higher than `65534` are clipped at `65534`.

`scope`

Defines the cells that the header (defined in the `<th>`) element relates to. Possible enumerated values are:

- `row`: the header relates to all cells of the row it belongs to;
- `col`: the header relates to all cells of the column it belongs to;
- `rowgroup`: the header belongs to a rowgroup and relates to all of its cells;
- `colgroup`: the header belongs to a colgroup and relates to all of its cells.

If the `scope` attribute is not specified, or its value is not `row`, `col`, `rowgroup`, or `colgroup`, then browsers automatically select the set of cells to which the header cell applies.

### Deprecated attributes

The following attributes are deprecated and should not be used. They are documented below for reference when updating existing code and for historical interest only.

`align`

Specifies the horizontal alignment of the header cell. The possible enumerated values are `left`, `center`, `right`, `justify`, and `char`. When supported, the `char` value aligns the textual content on the character defined in the `char` attribute and the offset defined by the `charoff` attribute. Use the `text-align` CSS property instead, as this attribute is deprecated.

`axis`

Contains a list of space-separated strings, each corresponding to the `id` attribute of a group of cells that the header cell applies to. Use the `scope` attribute instead, as this attribute is deprecated.

`bgcolor`

Defines the background color of the header cell. The value is an HTML color; either a 6-digit hexadecimal RGB code, prefixed by a `#`, or a color keyword. Other CSS `<color>` values are not supported. Use the `background-color` CSS property instead, as this attribute is deprecated.

`char`

Does nothing. It was originally intended to specify the alignment of the content to a character of the header cell. Typical values for this include a period (`.`) when attempting to align numbers or monetary values. If `align` is not set to `char`, this attribute is ignored.

`charoff`

Does nothing. It was originally intended to specify the number of characters to offset the header cell content from the alignment character specified by the `char` attribute.

`height`

Defines a recommended header cell height. Use the `height` CSS property instead, as this attribute is deprecated.

`valign`

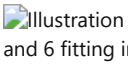
Specifies the vertical alignment of the header cell. The possible enumerated values are `baseline`, `bottom`, `middle`, and `top`. Use the `vertical-align` CSS property instead, as this attribute is deprecated.

`width`

Defines a recommended header cell width. Use the `width` CSS property instead, as this attribute is deprecated.

### Usage notes

- The `<th>` may only be used within a `<tr>` element.
- In simple contexts, using the `scope` attribute on header cells (`<th>` elements) is redundant because `scope` is inferred. However, certain assistive technologies may fail to infer correctly, so specifying header scope may improve user experiences.
- When using the `colspan` and `rowspan` attributes to span header cells across multiple columns and rows, cells without these attributes defined (with a default value of `1`) are automatically fitted into free available spaces in the table structure that span 1x1 cells, as illustrated in the following figure:

   Illustration demonstrating column and row spanning of table cells: cells 1, 3, and 4 spanning two rows; cell 2 spanning two columns; cells 5 and 6 fitting into the available cells that are the second and third columns in the second row

  > **Note:** These attributes must not be used to overlap cells.

### Examples

See `<table>` for a complete table example introducing common standards and best practices.

### Basic column and row headers

This example uses `<th>` elements to introduce column and row headers in a basic table structure.

### HTML

The first row (`<tr>` element) contains the column headers (`<th>` elements), which act as "titles" for the columns to make it easier to understand the information in the columns and identify the data. To indicate that each column header relates to all cells in the corresponding column, the `scope` attribute is set to `col` (column).

The remaining rows contain the main data of the table. Each of these rows has a row header (`<th>` element) introduced as the first cell. This creates a column with row headers as the first column of the table. Similar to the column headers, the `scope` attribute is set to `row` to specify which cells each row header relates to, which in the example below are all data cells (`<td>` elements) in each `row`.

> **Note:** Normally, the grouping elements `<thead>` and `<tbody>` are used to group rows with headers into the respective table head and body sections. These elements are omitted in this example to reduce complexity and enable focusing on the use of header cells.

```
<table>
  <tr>
    <th scope="col">Symbol</th>
    <th scope="col">Code word</th>
    <th scope="col">Pronunciation</th>
  </tr>
  <tr>
    <th scope="row">A</th>
```

```
    <td>Alfa</td>
    <td>AL fah</td>
  </tr>
  <tr>
    <th scope="row">B</th>
    <td>Bravo</td>
    <td>BRAH voh</td>
  </tr>
  <tr>
    <th scope="row">C</th>
    <td>Charlie</td>
    <td>CHAR lee</td>
  </tr>
  <tr>
    <th scope="row">D</th>
    <td>Delta</td>
    <td>DELL tah</td>
  </tr>
</table>
```

### CSS

Some basic CSS is used to style the table and its cells. We use CSS attribute selectors to target header cells based on their `scope` attribute values, highlighting column and row headers (`<th>` elements) and differentiating them each other and from the data cells (`<td>`).

```
th,
td {
  border: 1px solid rgb(160 160 160);
  padding: 8px 10px;
}

th[scope="col"] {
  background-color: #505050;
  color: white;
}

th[scope="row"] {
  background-color: #d6ecd4;
}

tr:nth-of-type(odd) td {
  background-color: #eeeeee;
}
```

### Result

### Column and row spanning

This example extends and enhances the basic table from the previous example by adding a second row for additional column headers.

### HTML

An additional table row (`<tr>` element) is added as the second header row of the table with two additional column headers (`<th>` elements). In this way, the "Pronunciation" column is split into two columns, one for the IPA (International Phonetic Alphabet) notation and one for the respelling (the original pronunciation column). The corresponding data cells (`<td>` elements) are added to each subsequent row.

As shown in the usage notes, the `colspan` and `rowspan` attributes can be used for the `<th>` elements to allocate the header cells to the correct columns and rows. To achieve a "two-row" header in the table structure, the first two header cells within the first `<tr>` element are spanned across two rows. The third header cell is spanned two columns wide (remaining in the first row). This setup leaves two available areas in the third and fourth columns in the second row, where the two headers within the second `<tr>` element are automatically placed, with the default value being `1` for the `colspan` and `rowspan` attributes.

> **Note:** Normally, `<thead>` and `<tbody>` elements are used to group rows with headers into the respective table head and body sections. This is not implemented in this example to focus on the headers and spanning and reduce the example's complexity.

```
<table>
  <tr>
    <th scope="col" rowspan="2">Symbol</th>
    <th scope="col" rowspan="2">Code word</th>
    <th scope="col" colspan="2">Pronunciation</th>
  </tr>
  <tr>
    <th scope="col">IPA</th>
```

```
    <th scope="col">Respelling</th>
  </tr>
  <tr>
    <th scope="row">A</th>
    <td>Alfa</td>
    <td>ˈælfa</td>
    <td>AL fah</td>
  </tr>
  <tr>
    <th scope="row">B</th>
    <td>Bravo</td>
    <td>ˈbraːˈvo</td>
    <td>BRAH voh</td>
  </tr>
  <tr>
    <th scope="row">C</th>
    <td>Charlie</td>
    <td>ˈtʃɑːli</td>
    <td>CHAR lee</td>
  </tr>
  <tr>
    <th scope="row">D</th>
    <td>Delta</td>
    <td>ˈdeltɑ</td>
    <td>DELL tah</td>
  </tr>
</table>
```

### CSS

The CSS is unchanged from the previous example.

### Result

### Associate header cells with other header cells

For more complex relationships between header cells, using `th` elements with the `scope` attribute alone may not be sufficient for assistive technologies, especially screen readers.

### HTML

To improve the accessibility of the previous example and to allow screen readers, for example, to speak the headers associated with each header cell, the `headers` attribute can be introduced along with `id` attributes. Because of the way the "Pronunciation" column is split into two columns in the example, introducing a "two row" header, assistive technologies such as screen readers may not be able to identify which additional header cells (`th` elements) the "Pronunciation" header cell is related to and vice versa. Therefore, the `headers` attribute is used on the "Pronunciation", "IPA", and "Respelling" header cells to associate the related header cells based on the values of the unique identifiers from the added `id` attributes in the form of a space-separated list.

> **Note:** It's recommended to use more descriptive and useful values for the `id` attribute. Each `id` in a document must be unique to that document. In this example, the `id` values are single characters to maintain focus on the concept of the `headers` attribute.

```
<table>
  <tr>
    <th scope="col" rowspan="2">Symbol</th>
    <th scope="col" rowspan="2">Code word</th>
    <th scope="col" colspan="2" id="p" headers="i r">Pronunciation</th>
  </tr>
  <tr>
    <th scope="col" id="i" headers="p">IPA</th>
    <th scope="col" id="r" headers="p">Respelling</th>
  </tr>
  <tr>
    <th scope="row">A</th>
    <td>Alfa</td>
    <td>ˈælfa</td>
    <td>AL fah</td>
  </tr>
  <tr>
    <th scope="row">B</th>
    <td>Bravo</td>
    <td>ˈbraːˈvo</td>
    <td>BRAH voh</td>
  </tr>
  <tr>
```

```
    <th scope="row">C</th>
    <td>Charlie</td>
    <td>ˈtʃɑːli</td>
    <td>CHAR lee</td>
  </tr>
  <tr>
    <th scope="row">D</th>
    <td>Delta</td>
    <td>ˈdeltɑ</td>
    <td>DELL tah</td>
  </tr>
</table>
```

### Result

The visual result is unchanged from the previous example table.

### Technical summary

| | |
|---|---|
| **Content categories** | None. |
| **Permitted content** | Flow content, but with no header, footer, sectioning content, or heading content descendants. |
| **Tag omission** | The start tag is mandatory.<br>The end tag may be omitted, if it is immediately followed by a `<th>` or `<td>` element or if there are no more data in its parent element. |
| **Permitted parents** | A `<tr>` element. |
| **Implicit ARIA role** | `columnheader` or `rowheader` |
| **Permitted ARIA roles** | Any |
| **DOM interface** | `HTMLTableCellElement` |

### See also

- Learn: HTML table basics
- `<caption>`, `<col>`, `<colgroup>`, `<table>`, `<tbody>`, `<td>`, `<tfoot>`, `<thead>`, `<tr>`: Other table-related elements
- `background-color`: CSS property to set the background color of each header cell
- `border`: CSS property to control borders of header cells
- `height`: CSS property to control the recommended header cell height
- `text-align`: CSS property to horizontally align each header cell content
- `vertical-align`: CSS property to vertically align each header cell content
- `width`: CSS property to control the recommended header cell width
- `:nth-of-type`, `:first-of-type`, `:last-of-type`: CSS pseudo-classes to select the desired header cells

## <thead> HTML table head element

The **`<thead>`** HTML element encapsulates a set of table rows (`<tr>` elements), indicating that they comprise the head of a table with information about the table's columns. This is usually in the form of column headers (`<th>` elements).

### Try it

```
<table>
  <caption>
    Council budget (in £) 2018
  </caption>
  <thead>
    <tr>
      <th scope="col">Items</th>
      <th scope="col">Expenditure</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <th scope="row">Donuts</th>
      <td>3,000</td>
    </tr>
    <tr>
      <th scope="row">Stationery</th>
      <td>18,000</td>
    </tr>
  </tbody>
  <tfoot>
    <tr>
```

```
      <th scope="row">Totals</th>
      <td>21,000</td>
    </tr>
  </tfoot>
</table>
```

```
thead,
tfoot {
  background-color: #2c5e77;
  color: white;
}

tbody {
  background-color: #e4f0f5;
}

table {
  border-collapse: collapse;
  border: 2px solid rgb(140 140 140);
  font-family: sans-serif;
  font-size: 0.8rem;
  letter-spacing: 1px;
}

caption {
  caption-side: bottom;
  padding: 10px;
}

th,
td {
  border: 1px solid rgb(160 160 160);
  padding: 8px 10px;
}

td {
  text-align: center;
}
```

### Attributes

This element includes the global attributes.

### Deprecated attributes

The following attributes are deprecated and should not be used. They are documented below for reference when updating existing code and for historical interest only.

`align`

> Specifies the horizontal alignment of each head cell. The possible enumerated values are `left`, `center`, `right`, `justify`, and `char`. When supported, the `char` value aligns the textual content on the character defined in the `char` attribute and the offset defined by the `charoff` attribute. Use the `text-align` CSS property instead, as this attribute is deprecated.

`bgcolor`

> Defines the background color of each head cell. The value is an HTML color; either a 6-digit hexadecimal RGB code, prefixed by a `#`, or a color keyword. Other CSS `<color>` values are not supported. Use the `background-color` CSS property instead, as this attribute is deprecated.

`char`

> Does nothing. It was originally intended to specify the alignment of the content to a character of each head cell. If `align` is not set to `char`, this attribute is ignored.

`charoff`

> Does nothing. It was originally intended to specify the number of characters to offset the head cell content from the alignment character specified by the `char` attribute.

`valign`

> Specifies the vertical alignment of each head cell. The possible enumerated values are `baseline`, `bottom`, `middle`, and `top`. Use the `vertical-align` CSS property instead, as this attribute is deprecated.

### Usage notes

- The `<thead>` is placed after any `<caption>` and `<colgroup>` elements, but before any `<tbody>`, `<tfoot>`, and `<tr>` elements.
- Along with its related `<tbody>` and `<tfoot>` elements, the `<thead>` element provides useful semantic information and can be used when rendering for either screen or print. Specifying such table content groups also provides valuable contextual information for assistive technologies, including screen readers and search engines.
- When printing a document, in the case of a multipage table, the table head usually specifies information that remains the same on each page.

### Examples

See `<table>` for a complete table example introducing common standards and best practices.

#### Basic head structure

This example demonstrates a table divided into a head section with column headers and a body section with the table's main data.

#### HTML

The `<thead>` and `<tbody>` elements are used to structure the table rows into semantic sections. The `<thead>` element represents the head section of the table, which contains a row (`<tr>`) of column headers cells (`<th>`).

```
<table>
  <thead>
    <tr>
      <th>Student ID</th>
      <th>Name</th>
      <th>Major</th>
      <th>Credits</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>3741255</td>
      <td>Jones, Martha</td>
      <td>Computer Science</td>
      <td>240</td>
    </tr>
    <tr>
      <td>3971244</td>
      <td>Nim, Victor</td>
      <td>Russian Literature</td>
      <td>220</td>
    </tr>
    <tr>
      <td>4100332</td>
      <td>Petrov, Alexandra</td>
      <td>Astrophysics</td>
      <td>260</td>
    </tr>
  </tbody>
</table>
```

#### CSS

Some basic CSS is used to style and highlight the table head so that the headings of the columns stand out from the data in the table body.

```
thead {
  border-bottom: 2px solid rgb(160 160 160);
  text-align: center;
  background-color: #2c5e77;
  color: white;
}

tbody {
  background-color: #e4f0f5;
}
```

#### Result

#### Multiple head rows

This example demonstrates a table head section with two rows.

#### HTML

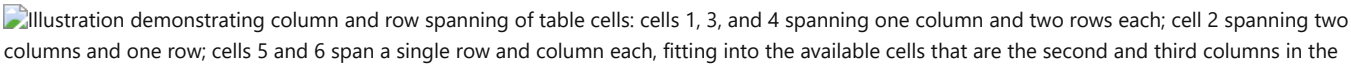
We extend the markup the table from the basic example in this example by including two table rows (`<tr>`) within the `<thead>` element creating a multi-row table head. We included an additional column, splitting the student names into first and last names.

```
<table>
  <thead>
    <tr>
      <th rowspan="2">Student ID</th>
      <th colspan="2">Student</th>
      <th rowspan="2">Major</th>
      <th rowspan="2">Credits</th>
    </tr>
    <tr>
      <th>First name</th>
      <th>Last name</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>3741255</td>
      <td>Martha</td>
      <td>Jones</td>
      <td>Computer Science</td>
      <td>240</td>
    </tr>
    <tr>
      <td>3971244</td>
      <td>Victor</td>
      <td>Nim</td>
      <td>Russian Literature</td>
      <td>220</td>
    </tr>
    <tr>
      <td>4100332</td>
      <td>Alexandra</td>
      <td>Petrov</td>
      <td>Astrophysics</td>
      <td>260</td>
    </tr>
  </tbody>
</table>
```

### Cell spanning

In order to associate and line up the header cells with the correct columns and rows, the `colspan` and `rowspan` attributes are used on the `<th>` elements. The values set in these attributes specify how many cells each header cell (`<th>`) spans. Due to the way these attributes are set, the two second-row header cells are lined up with the columns they head. These each span one row and one column as the default values for the `colspan` and `rowspan` attributes are both `1`.

The column and row spanning demonstrated by this example are illustrated in the following figure:


Illustration demonstrating column and row spanning of table cells: cells 1, 3, and 4 spanning one column and two rows each; cell 2 spanning two columns and one row; cells 5 and 6 span a single row and column each, fitting into the available cells that are the second and third columns in the second row

### CSS

The CSS is unchanged from the previous example.

### Result

### Technical summary

| | |
|---|---|
| **Content categories** | None. |
| **Permitted content** | Zero or more `<tr>` elements. |
| **Tag omission** | The start tag is mandatory. The end tag may be omitted if the `<thead>` element is immediately followed by a `<tbody>` or `<tfoot>` element. |
| **Permitted parents** | A `<table>` element. The `<thead>` must appear after any `<caption>` and `<colgroup>` elements, even implicitly defined, but before any `<tbody>`, `<tfoot>`, and `<tr>` elements. |
| **Implicit ARIA role** | `rowgroup` |
| **Permitted ARIA roles** | Any |

| DOM interface | `HTMLTableSectionElement` |
|---|---|

### See also

- Learn: HTML table basics
- `<caption>`, `<col>`, `<colgroup>`, `<table>`, `<tbody>`, `<td>`, `<tfoot>`, `<th>`, `<tr>`: Other table-related elements
- `background-color`: CSS property to set the background color of each head cell
- `border`: CSS property to control borders of head cells
- `text-align`: CSS property to horizontally align each head cell content
- `vertical-align`: CSS property to vertically align each head cell content

## <time> HTML time (date) element

The **`<time>`** HTML element represents a specific period in time. It may include the `datetime` attribute to translate dates into machine-readable format, allowing for better search engine results or custom features such as reminders.

It may represent one of the following:

- A time on a 24-hour clock.
- A precise date in the Gregorian calendar (with optional time and timezone information).
- A valid time duration.

### Try it

```
<p>
  The Cure will be celebrating their 40th anniversary on
  <time datetime="2018-07-07">July 7</time> in London's Hyde Park.
</p>

<p>
  The concert starts at <time datetime="20:00">20:00</time> and you'll be able
  to enjoy the band for at least <time datetime="PT2H30M">2h 30m</time>.
</p>
```

```
time {
  font-weight: bold;
}
```

### Attributes

Like all other HTML elements, this element supports the global attributes.

`datetime`

This attribute indicates the time and/or date of the element and must be in one of the formats described below.

### Usage notes

This element is for presenting dates and times in a machine-readable format. For example, this can help a user agent offer to add an event to a user's calendar.

This element should not be used for dates prior to the introduction of the Gregorian calendar (due to complications in calculating those dates).

The *datetime value* (the machine-readable value of the datetime) is the value of the element's `datetime` attribute, which must be in the proper format (see below). If the element does not have a `datetime` attribute, **it must not have any element descendants**, and the *datetime value* is the element's child text content.

### Valid datetime values

| Description | Microsyntax | Examples |
|---|---|---|
| Valid month string | *YYYY-MM* | 2011-11, 2013-05 |
| Valid date string | *YYYY-MM-DD* | 1887-12-01 |
| Valid yearless date string | *MM-DD* | 11-12 |
| Valid time string | *HH:MM*<br>*HH:MM:SS*<br>*HH:MM:SS.mmm* | 23:59<br>12:15:47<br>12:15:52.998 |
| Valid local date and time string | *YYYY-MM-DD HH:MM*<br>*YYYY-MM-DD HH:MM:SS*<br>*YYYY-MM-DD HH:MM:SS.mmm*<br>*YYYY-MM-DDTHH:MM* | 2013-12-25 11:12<br>1972-07-25 13:43:07<br>1941-03-15 07:06:23.678<br>2013-12-25T11:12 |

| Description | Microsyntax | Examples |
| --- | --- | --- |
| | *YYYY*-*MM*-*DD*T*HH*:*MM*:*SS*<br>*YYYY*-*MM*-*DD*T*HH*:*MM*:*SS*.*mmm* | 1972-07-25T13:43:07<br>1941-03-15T07:06:23.678 |
| Valid time-zone offset string | Z<br>+*HHMM*<br>+*HH*:*MM*<br>-*HHMM*<br>-*HH*:*MM* | Z<br>+0200<br>+04:30<br>-0300<br>-08:00 |
| Valid global date and time string | Any combination of a valid local date and time string followed by a valid time-zone offset string | 2013-12-25 11:12+0200<br>1972-07-25 13:43:07+04:30<br>1941-03-15 07:06:23.678Z<br>2013-12-25T11:12-08:00 |
| Valid week string | *YYYY*-W*WW* | 2013-W46 |
| Four or more ASCII digits | *YYYY* | 2013, 0001 |
| Valid duration string | P*d*DT*h*H*m*M*s*S<br>P*d*DT*h*H*m*M*s*.*X*S<br>P*d*DT*h*H*m*M*s*.*XX*S<br>P*d*DT*h*H*m*M*s*.*XXX*S<br>PT*h*H*m*M*s*S<br>PT*h*H*m*M*s*.*X*S<br>PT*h*H*m*M*s*.*XX*S<br>PT*h*H*m*M*s*.*XXX*S<br>*ww dd hh mm ss* | P12DT7H12M13S<br>P12DT7H12M13.3S<br>P12DT7H12M13.45S<br>P12DT7H12M13.455S<br>PT7H12M13S<br>PT7H12M13.2S<br>PT7H12M13.56S<br>PT7H12M13.999S<br>7d 5h 24m 13s |

### Examples

#### Basic example

#### HTML

```
<p>The concert starts at <time datetime="2018-07-07T20:00:00">20:00</time>.</p>
```

#### Result

#### <code>datetime</code> example

#### HTML

```
<p>
  The concert took place on <time datetime="2001-05-15T19:00">May 15</time>.
</p>
```

#### Result

### Technical summary

| | |
| --- | --- |
| **Content categories** | Flow content, phrasing content, palpable content. |
| **Permitted content** | Phrasing content. |
| **Tag omission** | None, both the starting and ending tag are mandatory. |
| **Permitted parents** | Any element that accepts phrasing content. |
| **Implicit ARIA role** | `time` |
| **Permitted ARIA roles** | Any |
| **DOM interface** | `HTMLTimeElement` |

### See also

- The `<data>` element, allowing you to signal other kinds of values.

## <title> HTML document title element

The **`<title>`** HTML element defines the document's title that is shown in a browser's title bar or a page's tab. It only contains text; HTML tags within the element, if any, are also treated as plain text.

```
<title>Grandma's Heavy Metal Festival Journal</title>
```

### Attributes

This element only includes the global attributes.

### Usage notes

The `<title>` element is always used within a page's `<head>` block.

### Page titles and SEO

The contents of a page title can have significant implications for search engine optimization (SEO). In general, a longer, descriptive title performs better than short or generic titles. The content of the title is one of the components used by search engine algorithms to decide the order in which to list pages in search results. Also, the title is the initial "hook" by which you grab the attention of readers glancing at the search results page.

A few guidelines and tips for composing good titles:

- Avoid one- or two-word titles. Use a descriptive phrase, or a term-definition pairing for glossary or reference-style pages.
- Search engines typically display about the first 55–60 characters of a page title. Text beyond that may be lost, so try not to have titles longer than that. If you must use a longer title, make sure the important parts come earlier and that nothing critical is in the part of the title that is likely to be dropped.
- Don't use "keyword blobs." If your title is just a list of words, algorithms often reduce your page's position in the search results.
- Try to make sure your titles are as unique as possible within your own site. Duplicate—or near-duplicate—titles can contribute to inaccurate search results.

### Accessibility

It is important to provide an accurate and concise title to describe the page's purpose.

A common navigation technique for users of assistive technology is to read the page title and infer the content the page contains. This is because navigating into a page to determine its content can be a time-consuming and potentially confusing process. Titles should be unique to every page of a website, ideally surfacing the primary purpose of the page first, followed by the name of the website. Following this pattern will help ensure that the primary purpose of the page is announced by a screen reader first. This provides a far better experience than having to listen to the name of a website before the unique page title, for every page a user navigates to in the same website.

### Examples

```
<title>Menu - Blue House Chinese Food - FoodYum: Online takeout today!</title>
```

If a form submission contains errors and the submission re-renders the current page, the title can be used to help make users aware of any errors with their submission. For instance, update the page `title` value to reflect significant page state changes (such as form validation problems).

```
<title>
  2 errors - Your order - Sea Food Store - Food: Online takeout today!
</title>
```

> **Note:** Presently, dynamically updating a page's title will not be automatically announced by screen readers. If you are going to update the page title to reflect significant changes to a page's state, then the use of ARIA Live Regions may be necessary, as well.

- MDN Understanding WCAG, Guideline 2.4 explanations
- Understanding Success Criterion 2.4.2 | W3C Understanding WCAG 2.1

### Examples

```
<title>Awesome interesting stuff</title>
```

This example establishes a page whose title (as displayed at the top of the window or in the window's tab) as "Awesome interesting stuff".

### Technical summary

| | |
|---|---|
| **Content categories** | Metadata content. |
| **Permitted content** | Text that is not inter-element whitespace. |
| **Tag omission** | Both opening and closing tags are required. Note that leaving off `</title>` should cause the browser to ignore the rest of the page. |
| **Permitted parents** | A `<head>` element that contains no other `<title>` element. |
| **Implicit ARIA role** | No corresponding role |
| **Permitted ARIA roles** | No `role` permitted. |

| DOM interface | HTMLTitleElement |
| --- | --- |

### See also

- SVG `<title>` element

## `<tr>` HTML table row element

The **`<tr>`** HTML element defines a row of cells in a table. The row's cells can then be established using a mix of `<td>` (data cell) and `<th>` (header cell) elements.

### Try it

```
<table>
  <caption>
    Alien football stars
  </caption>
  <thead>
    <tr>
      <th scope="col">Player</th>
      <th scope="col">Gloobles</th>
      <th scope="col">Za'taak</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <th scope="row">TR-7</th>
      <td>7</td>
      <td>4,569</td>
    </tr>
    <tr>
      <th scope="row">Khiresh Odo</th>
      <td>7</td>
      <td>7,223</td>
    </tr>
    <tr>
      <th scope="row">Mia Oolong</th>
      <td>9</td>
      <td>6,219</td>
    </tr>
  </tbody>
</table>
```

```
th,
td {
  border: 1px solid rgb(160 160 160);
  padding: 8px 10px;
}

th[scope="col"] {
  background-color: #505050;
  color: white;
}

th[scope="row"] {
  background-color: #d6ecd4;
}

td {
  text-align: center;
}

tr:nth-of-type(even) {
  background-color: #eeeeee;
}

table {
  border-collapse: collapse;
  border: 2px solid rgb(140 140 140);
  font-family: sans-serif;
  font-size: 0.8rem;
  letter-spacing: 1px;
}

caption {
  caption-side: bottom;
```

```
  padding: 10px;
}
```

### Attributes

This element includes the global attributes.

### Deprecated attributes

The following attributes are deprecated and should not be used. They are documented below for reference when updating existing code and for historical interest only.

`align`

> Specifies the horizontal alignment of each row cell. The possible enumerated values are `left`, `center`, `right`, `justify`, and `char`. When supported, the `char` value aligns the textual content on the character defined in the `char` attribute and on offset defined by the `charoff` attribute. Use the `text-align` CSS property instead, as this attribute is deprecated.

`bgcolor`

> Defines the background color of each row cell. The value is an HTML color; either a 6-digit hexadecimal RGB code, prefixed by a `#`, or a color keyword. Other CSS `<color>` values are not supported. Use the `background-color` CSS property instead, as this attribute is deprecated.

`char`

> Specifies the alignment of the content to a character of each row cell. Typical values for this include a period (`.`) when attempting to align numbers or monetary values. If `align` is not set to `char`, this attribute is ignored.

`charoff`

> Specifies the number of characters to offset the row cell content from the alignment character specified by the `char` attribute.

`valign`

> Specifies the vertical alignment of each row cell. The possible enumerated values are `baseline`, `bottom`, `middle`, and `top`. Use the `vertical-align` CSS property instead, as this attribute is deprecated.

### Usage notes

- The `<tr>` element is valid as a child of a `<thead>`, `<tbody>`, or `<tfoot>` element only.
- If the `<tr>` is placed as a direct child of its parent `<table>` element, the `<tbody>` parent is implied and browsers will add the `<tbody>` to the markup.
- The implied `<tbody>` parent is only supported if the `<table>` otherwise has no child `<tbody>` elements, and only if the `<tr>` is included after any `<caption>`, `<colgroup>`, and `<thead>` elements.
- The CSS pseudo-classes `:nth-of-type`, `:first-of-type`, and `:last-of-type` are often useful for selecting the desired set of rows and their data and header cells (`<td>` and `<th>` elements).
- When a `<tr>` is included as a direct child of the `<table>`, as the browser adds a `<tbody>` to the markup, CSS selectors such as `table > tr` may not work as expected or at all.

### Examples

See `<table>` for a complete table example introducing common standards and best practices.

### Basic row setup

This example demonstrates a table with four rows and three columns, where the first column contains headers for the row data cells.

### HTML

Four `<tr>` elements are used to create four table rows. Each row contains three cells - one header cell (`<th>`) and two data cells (`<td>`) - creating three columns. The `scope` attribute set on each header cell specifies which cells they relate to, which in this example is all data cells within the `row`.

```
<table>
  <tbody>
    <tr>
      <th scope="row">A</th>
      <td>Alfa</td>
      <td>AL fah</td>
    </tr>
    <tr>
      <th scope="row">B</th>
      <td>Bravo</td>
      <td>BRAH voh</td>
    </tr>
```

```
    <tr>
      <th scope="row">C</th>
      <td>Charlie</td>
      <td>CHAR lee</td>
    </tr>
    <tr>
      <th scope="row">D</th>
      <td>Delta</td>
      <td>DELL tah</td>
    </tr>
  </tbody>
</table>
```

### CSS

The CSS `:nth-of-type` pseudo-class is used to select every `odd` row and set the `background-color` of those rows to a slightly darker tone, creating a so-called "zebra stripe" effect. This alternating background makes the rows of data in the table easier to parse and read—imagine having many rows and columns and trying to find some data in a particular row. In addition, the row header cells (`<th>` elements) are highlighted with a `background-color` to distinguish them from the data cells (`<td>` elements).

```
tr:nth-of-type(odd) {
  background-color: #eeeeee;
}

tr th[scope="row"] {
  background-color: #d6ecd4;
}
```

### Result

## Header row

This example extends the basic table from the previous example by adding a header row as the first row of the table.

### HTML

An additional table row (`<tr>`) is added as the first row of the table with column header cells (`<th>`) providing a header for each column. We put this row in a `<thead>` grouping element to indicate this is the header of the table. The `scope` attribute is added to each header cell (`<th>`) within this head row to explicitly specify that each header cell relates to all the cells within its own column, even though those cells are in the `<tbody>`.

```
<table>
  <thead>
    <tr>
      <th scope="col">Symbol</th>
      <th scope="col">Code word</th>
      <th scope="col">Pronunciation</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <th scope="row">A</th>
      <td>Alfa</td>
      <td>AL fah</td>
    </tr>
    <tr>
      <th scope="row">B</th>
      <td>Bravo</td>
      <td>BRAH voh</td>
    </tr>
    <tr>
      <th scope="row">C</th>
      <td>Charlie</td>
      <td>CHAR lee</td>
    </tr>
    <tr>
      <th scope="row">D</th>
      <td>Delta</td>
      <td>DELL tah</td>
    </tr>
  </tbody>
</table>
```

### CSS

The CSS is nearly unchanged from the previous example, except for some additional styling to highlight the "header row" so that the headers of the columns stand out from the other cells.

```
tr:nth-of-type(odd) {
  background-color: #eeeeee;
}

tr th[scope="col"] {
  background-color: #505050;
  color: white;
}

tr th[scope="row"] {
  background-color: #d6ecd4;
}
```

#### Result

### Sorting rows

There are no native methods for sorting the rows (`<tr>` elements) of a `<table>`. But using `Array.prototype.sort()`, `Node.removeChild`, and `Node.appendChild`, a custom `sort()` function can be implemented in JavaScript to sort an `HTMLCollection` of `<tr>` elements.

#### HTML

A `<tbody>` element is used in this basic table to mark the body section of the table and to include three rows (`<tr>` elements) with data (`<td>` elements), creating one column with numbers in descending order.

```
<table>
  <tbody>
    <tr>
      <td>3</td>
    </tr>
    <tr>
      <td>2</td>
    </tr>
    <tr>
      <td>1</td>
    </tr>
  </tbody>
</table>
```

#### JavaScript

In the JavaScript code below, the created `sort()` function is attached to the `<tbody>` element so that it sorts the table cells in order of increasing value and updates the display accordingly.

```
HTMLTableSectionElement.prototype.sort = function (cb) {
  Array.from(this.rows)
    .sort(cb)
    .forEach((e) => this.appendChild(this.removeChild(e)));
};

document
  .querySelector("table")
  .tBodies[0].sort((a, b) => a.textContent.localeCompare(b.textContent));
```

#### Result

### Sorting rows with a click on header cells

This example extends the basic table from the previous example by making the sorting interactive and independent for multiple columns.

#### HTML

An additional data cell (`<td>` element) is added to each row (`<tr>` element) within the table body (`<tbody>` element) to create a second column with letters in ascending order. Using the `<thead>` element, a head section is added before the body section to introduce a head row with table header cells (`<th>` element). These header cells are used in the JavaScript code below to make them clickable and then to perform the corresponding sorting when activated per click.

```
<table>
  <thead>
    <tr>
      <th>Numbers</th>
```

```
      <th>Letters</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>3</td>
      <td>A</td>
    </tr>
    <tr>
      <td>2</td>
      <td>B</td>
    </tr>
    <tr>
      <td>1</td>
      <td>C</td>
    </tr>
  </tbody>
</table>
```

### JavaScript

A click event handler is added to each table header (`<th>` element) of each `<table>` in the `document`; it sorts all the rows (`<tr>` elements) of the `<tbody>` based on the contents of the data cells (`<td>` elements) contained in the rows.

> **Note:** This solution assumes that the `<td>` elements are populated by raw text with no descendant elements.

```
const allTables = document.querySelectorAll("table");

for (const table of allTables) {
  const tBody = table.tBodies[0];
  const rows = Array.from(tBody.rows);
  const headerCells = table.tHead.rows[0].cells;

  for (const th of headerCells) {
    const cellIndex = th.cellIndex;

    th.addEventListener("click", () => {
      rows.sort((tr1, tr2) => {
        const tr1Text = tr1.cells[cellIndex].textContent;
        const tr2Text = tr2.cells[cellIndex].textContent;
        return tr1Text.localeCompare(tr2Text);
      });

      tBody.append(...rows);
    });
  }
}
```

### Result

> **Note:** To be usable and accessible, the header cell of each sortable column must be identifiable as a sorting button and each must define whether the column is currently sorted in ascending or descending order visually and with the `aria-sort` attribute. See the ARIA Authoring Practices Guide's sortable table example for more information.

### Technical summary

| | |
|---|---|
| **Content categories** | None |
| **Permitted content** | Zero or more `<td>` and/or `<th>` elements; script-supporting elements (`<script>` and `<template>`) are also allowed. |
| **Tag omission** | Start tag is mandatory. End tag may be omitted if the `<tr>` element is immediately followed by a `<tr>` element, or if the row is the last element in its parent table group (`<thead>`, `<tbody>` or `<tfoot>`) element. |
| **Permitted parents** | `<table>` (only if the table has no child `<tbody>` element, and even then only after any `<caption>`, `<colgroup>`, and `<thead>` elements); otherwise, the parent must be a `<thead>`, `<tbody>` or `<tfoot>` element. |
| **Implicit ARIA role** | `row` |
| **Permitted ARIA roles** | Any |

| DOM interface | `HTMLTableRowElement` |
| --- | --- |

### See also

- Learn: HTML table basics
- `<caption>`, `<col>`, `<colgroup>`, `<table>`, `<tbody>`, `<td>`, `<tfoot>`, `<th>`, `<thead>`: Other table-related elements
- `background-color`: CSS property to set the background color of each row cell
- `border`: CSS property to control borders of row cells
- `text-align`: CSS property to horizontally align each row cell content
- `vertical-align`: CSS property to vertically align each row cell content
- `:nth-of-type`, `:first-of-type`, `:last-of-type`: CSS pseudo-classes to select the desired row cells

## <track> HTML embed text track element

The **`<track>`** HTML element is used as a child of the media elements, `<audio>` and `<video>`. Each track element lets you specify a timed text track (or time-based data) that can be displayed in parallel with the media element, for example to overlay subtitles or closed captions on top of a video or alongside audio tracks.

Multiple tracks can be specified for a media element, containing different kinds of timed text data, or timed text data that has been translated for different locales. The data that is used will either be the track that has been set to be the default, or a kind and translation based on user preferences.

The tracks are formatted in WebVTT format (`.vtt` files) — Web Video Text Tracks.

### Try it

```
<video controls src="https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/track//shared-assets/videos/friday.mp4">
  <track
    default
    kind="captions"
    srclang="en"
    label="English"
    src="https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/track//shared-assets/misc/friday.vtt" />
  Download the
  <a href="https://developer.mozilla.org/shared-assets/videos/friday.mp4">MP4</a>
  video, and
  <a href="https://developer.mozilla.org/shared-assets/misc/friday.vtt">subtitles</a>.
</video>
```

```
video {
  width: 250px;
}

video::cue {
  font-size: 1rem;
}
```

### Attributes

This element includes the global attributes.

`default`

> This attribute indicates that the track should be enabled unless the user's preferences indicate that another track is more appropriate. This may only be used on one `track` element per media element.

`kind`

> How the text track is meant to be used. If omitted the default kind is `subtitles`. If the attribute contains an invalid value, it will use `metadata`. The following keywords are allowed:
>
> `subtitles`
>
> > Subtitles provide transcription or translation of the dialog. They are suitable for when the sound is available but not understood, such as speech or text that is not English in an English language film. Subtitles may contain additional content, usually extra background information. For example the text at the beginning of the Star Wars films, or the date, time, and location of a scene. Subtitles' information complements the audio and video. It is often embedded in the video itself, but can also be provided separately, especially for whole-film translations.
>
> `captions`
>
> > Closed captions provide transcription or translation of the dialog, sound effects, relevant musical cues, and other relevant audio information, such as the cue's source (e.g., character, environment). They are suitable for when sound is unavailable or not clearly audible (e.g., because it is muted, drowned-out by ambient noise, or because the user is deaf).

`descriptions`

Descriptions summarize the *video* component of the media resource. They are intended to be synthesized as audio when the visual component is obscured, unavailable, or not usable (e.g., because the user is interacting with the application without a screen while driving, or because the user is blind).

`chapters`

Chapter titles are intended to be used when the user is navigating the media resource.

`metadata`

Tracks used by scripts. Not visible to the user.

`label`

A user-readable title of the text track which is used by the browser when listing available text tracks.

`src`

Address of the track (`.vtt` file). Must be a valid URL. This attribute must be specified and its URL value must have the same origin as the document — unless the `<audio>` or `<video>` parent element of the `track` element has a `crossorigin` attribute.

`srclang`

Language of the track text data. It must be a valid BCP 47 language tag. If the `kind` attribute is set to `subtitles`, then `srclang` must be defined.

### Usage notes

#### Track data types

The type of data that `track` adds to the media is set in the `kind` attribute, which can take values of `subtitles`, `captions`, `chapters` or `metadata`. The element points to a source file containing timed text that the browser exposes when the user requests additional data.

A media element cannot have more than one `track` with the same `kind`, `srclang`, and `label`.

#### Detecting cue changes

The underlying `TextTrack`, indicated by the `track` property, receives a `cuechange` event every time the currently-presented cue is changed. This happens even if the track isn't associated with a media element.

If the track *is* associated with a media element, using the `<track>` element as a child of the `<audio>` or `<video>` element, the `cuechange` event is also sent to the `HTMLTrackElement`.

```
let textTrackElem = document.getElementById("text-track");

textTrackElem.addEventListener("cuechange", (event) => {
  let cues = event.target.track.activeCues;
});
```

#### Adding text tracks programmatically

The JavaScript interface that represents the `<track>` element is `HTMLTrackElement`. The text track associated with an element can be obtained from the `HTMLTrackElement.track` property, and is of type `TextTrack`.

`TextTrack` objects also can be added to a `HTMLVideoElement` or `HTMLAudioElement` elements using the `HTMLMediaElement.addTextTrack()` method. The `TextTrack` objects associated with a media element stored in a `TextTrackList`, which can be retrieved using the `HTMLMediaElement.textTracks` property.

### Examples

```
<video controls poster="/images/sample.gif">
  <source src="https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/track/sample.mp4" type="video/mp4" />
  <source src="https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/track/sample.ogv" type="video/ogv" />
  <track kind="captions" src="https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/track/sampleCaptions.vtt"
srclang="en" />
  <track kind="chapters" src="https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/track/sampleChapters.vtt"
srclang="en" />
  <track kind="subtitles" src="https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/track/sampleSubtitles_de.vtt"
srclang="de" />
  <track kind="subtitles" src="https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/track/sampleSubtitles_en.vtt"
srclang="en" />
  <track kind="subtitles" src="https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/track/sampleSubtitles_ja.vtt"
srclang="ja" />
  <track kind="subtitles" src="https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/track/sampleSubtitles_oz.vtt"
```

```
srclang="oz" />
  <track kind="metadata" src="https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/track/keyStage1.vtt" srclang="en"
label="Key Stage 1" />
  <track kind="metadata" src="https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/track/keyStage2.vtt" srclang="en"
label="Key Stage 2" />
  <track kind="metadata" src="https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/track/keyStage3.vtt" srclang="en"
label="Key Stage 3" />
  <!-- Fallback -->
  …
</video>
```

### Technical summary

| | |
|---|---|
| **Content categories** | None |
| **Permitted content** | None; it is a void element. |
| **Tag omission** | Must have a start tag and must not have an end tag. |
| **Permitted parents** | A media element, `<audio>` or `<video>`. |
| **Implicit ARIA role** | No corresponding role |
| **Permitted ARIA roles** | No `role` permitted |
| **DOM interface** | `HTMLTrackElement` |

### See also

- WebVTT text track format
- `HTMLMediaElement.textTracks`

## <tt> HTML teletype text element

The **`<tt>`** HTML element creates inline text which is presented using the user agent's default monospace font face. This element was created for the purpose of rendering text as it would be displayed on a fixed-width display such as a teletype, text-only screen, or line printer.

The terms **non-proportional**, **monotype**, and **monospace** are used interchangeably and have the same general meaning: they describe a typeface whose characters are all the same number of pixels wide.

This element is obsolete, however. You should use the more semantically helpful `<code>`, `<kbd>`, `<samp>`, or `<var>` elements for inline text that needs to be presented in monospace type, or the `<pre>` tag for content that should be presented as a separate block.

> **Note:** If none of the semantic elements are appropriate for your use case (for example, if you need to show some content in a non-proportional font), you should consider using the `<span>` element, styling it as desired using CSS. The `font-family` property is a good place to start.

### Attributes

This element only includes the global attributes

### Examples

### Basic example

This example uses `<tt>` to show text entered into, and output by, a terminal application.

```
<p>
  Enter the following at the telnet command prompt:
  <code>set localecho</code><br />

  The telnet client should display: <tt>Local Echo is on</tt>
</p>
```

### Result

### Overriding the default font

You can override the browser's default font—if the browser permits you to do so, which it isn't required to do—using CSS:

### CSS

```
tt {
  font-family: "Lucida Console", "Menlo", "Monaco", "Courier New", monospace;
```

```
}
```

### HTML

```
<p>
  Enter the following at the telnet command prompt:
  <code>set localecho</code><br />

  The telnet client should display: <tt>Local Echo is on</tt>
</p>
```

### Result

### Usage notes

The `<tt>` element is, by default, rendered using the browser's default non-proportional font. You can override this using CSS by creating a rule using the `tt` selector, as seen in the example Overriding the default font above.

> **Note:** User-configured changes to the default monospace font setting may take precedence over your CSS.

Although this element wasn't officially deprecated in HTML 4.01, its use was discouraged in favor of the semantic elements and/or CSS. The `<tt>` element is obsolete in HTML 5.

### Technical summary

| | |
|---|---|
| **Content categories** | Flow content, phrasing content, palpable content. |
| **Permitted content** | Phrasing content. |
| **Tag omission** | None, both the starting and ending tag are mandatory. |
| **Permitted parents** | Any element that accepts phrasing content. |
| **Permitted ARIA roles** | Any |
| **DOM interface** | `HTMLElement` |

### See also

- The semantic `<code>`, `<var>`, `<kbd>`, and `<samp>` elements
- The `<pre>` element for displaying preformatted text blocks

## <u> HTML unarticulated annotation (underline) element

The **`<u>`** HTML element represents a span of inline text which should be rendered in a way that indicates that it has a non-textual annotation. This is rendered by default as a single solid underline, but may be altered using CSS.

> **Warning:** This element used to be called the "Underline" element in older versions of HTML, and is still sometimes misused in this way. To underline text, you should instead apply a style that includes the CSS `text-decoration` property set to `underline`.

### Try it

```
<p>
  You could use this element to highlight <u>speling</u> mistakes, so the writer
  can <u>corect</u> them.
</p>
```

```
p {
  margin: 0;
}

u {
  text-decoration: red wavy underline;
}
```

See the Usage notes section for further details on when it's appropriate to use `<u>` and when it isn't.

### Attributes

This element only includes the global attributes.

### Usage notes

Along with other pure styling elements, the original HTML Underline (`<u>`) element was deprecated in HTML 4; however, `<u>` was restored in HTML 5 with a new, semantic, meaning: to mark text as having some form of non-textual annotation applied.

> **Note:** Avoid using the `<u>` element with its default styling (of underlined text) in such a way as to be confused with a hyperlink, which is also underlined by default.

### Use cases

Valid use cases for the `<u>` element include annotating spelling errors, applying a proper name mark to denote proper names in Chinese text, and other forms of annotation.

You should *not* use `<u>` to underline text for presentation purposes, or to denote titles of books.

### Other elements to consider using

In most cases, you should use an element other than `<u>`, such as:

- `<em>` to denote stress emphasis
- `<b>` to draw attention to text
- `<mark>` to mark key words or phrases
- `<strong>` to indicate that text has strong importance
- `<cite>` to mark the titles of books or other publications
- `<i>` to denote technical terms, transliterations, thoughts, or names of vessels in Western texts

To provide textual annotations (as opposed to the non-textual annotations created with `<u>`), use the `<ruby>` element.

To apply an underlined appearance without any semantic meaning, use the `text-decoration` property's value `underline`.

### Examples

### Indicating a spelling error

This example uses the `<u>` element and some CSS to display a paragraph which includes a misspelled error, with the error indicated in the red wavy underline style which is fairly commonly used for this purpose.

#### HTML

```
<p>This paragraph includes a <u class="spelling">wrnogly</u> spelled word.</p>
```

In the HTML, we see the use of `<u>` with a class, `spelling`, which is used to indicate the misspelling of the word "wrongly".

#### CSS

```
u.spelling {
  text-decoration: red wavy underline;
}
```

This CSS indicates that when the `<u>` element is styled with the class `spelling`, it should have a red wavy underline underneath its text. This is a common styling for spelling errors. Another common style can be presented using `red dashed underline`.

#### Result

The result should be familiar to anyone who has used any of the more popular word processors available today.

### Avoiding <u>

Most of the time, you actually don't want to use `<u>`. Here are some examples that show what you should do instead in several cases.

### Non-semantic underlines

To underline text without implying any semantic meaning, use a `<span>` element with the `text-decoration` property set to `"underline"`, as shown below.

#### HTML

```
<span class="underline">Today's Special</span>
<br />
Chicken Noodle Soup With Carrots
```

#### CSS

```
.underline {
  text-decoration: underline;
}
```

#### Result

#### Presenting a book title

Book titles should be presented using the `<cite>` element instead of `<u>` or even `<i>`.

#### Using the cite element

```
<p>The class read <cite>Moby-Dick</cite> in the first term.</p>
```

#### Styling the cite element

The default styling for the `<cite>` element renders the text in italics. You can override that using CSS:

```
<p>The class read <cite>Moby-Dick</cite> in the first term.</p>
```

```
cite {
  font-style: normal;
  text-decoration: underline;
}
```

#### Technical summary

| | |
|---|---|
| **Content categories** | Flow content, phrasing content, palpable content. |
| **Permitted content** | Phrasing content. |
| **Tag omission** | None, both the starting and ending tag are mandatory. |
| **Permitted parents** | Any element that accepts phrasing content. |
| **Implicit ARIA role** | `generic` |
| **Permitted ARIA roles** | Any |
| **DOM interface** | `HTMLElement` |

#### See also

- The `<span>`, `<i>`, `<em>`, `<b>`, and `<cite>` elements should usually be used instead.
- The CSS `text-decoration` property should be used for non-semantic underlining.

### <ul> HTML unordered list element

The **`<ul>`** HTML element represents an unordered list of items, typically rendered as a bulleted list.

#### Try it

```
<ul>
  <li>Milk</li>
  <li>
    Cheese
    <ul>
      <li>Blue cheese</li>
      <li>Feta</li>
    </ul>
  </li>
</ul>
```

```
li {
  list-style-type: circle;
}

li li {
  list-style-type: square;
}
```

#### Attributes

This element includes the global attributes.

`compact`

This Boolean attribute hints that the list should be rendered in a compact style. The interpretation of this attribute is browser-specific. Use CSS instead: to give a similar effect as the `compact` attribute, the CSS property `line-height` can be used with a value of `80%`.

`type`

This attribute sets the bullet style for the list. The values defined under HTML3.2 and the transitional version of HTML 4.0/4.01 are:

- `circle`
- `disc`
- `square`

A fourth bullet type has been defined in the WebTV interface, but not all browsers support it: `triangle`.

If not present and if no CSS `list-style-type` property applies to the element, the user agent selects a bullet type depending on the nesting level of the list.

> **Warning:** Do not use this attribute, as it has been deprecated; use the CSS `list-style-type` property instead.

### Usage notes

- The `<ul>` element is for grouping a collection of items that do not have a numerical ordering, and their order in the list is meaningless. Typically, unordered-list items are displayed with a bullet, which can be of several forms, like a dot, a circle, or a square. The bullet style is not defined in the HTML description of the page, but in its associated CSS, using the `list-style-type` property.
- The `<ul>` and `<ol>` elements may be nested as deeply as desired. Moreover, the nested lists may alternate between `<ol>` and `<ul>` without restriction. To nest a list, place it inside an `<li>` element of the parent list; a `<ul>` or `<ol>` can't be a direct child of another `<ul>` or `<ol>`.
- The `<ol>` and `<ul>` elements both represent a list of items. They differ in that, with the `<ol>` element, the order is meaningful. To determine which one to use, try changing the order of the list items; if the meaning is changed, the `<ol>` element should be used, otherwise you can use `<ul>`.

### Examples

#### Basic example

```
<ul>
  <li>first item</li>
  <li>second item</li>
  <li>third item</li>
</ul>
```

#### Result

#### Nesting a list

```
<ul>
  <li>first item</li>
  <li>
    second item
    <!-- Look, the closing </li> tag is not placed here! -->
    <ul>
      <li>second item first subitem</li>
      <li>
        second item second subitem
        <!-- Same for the second nested unordered list! -->
        <ul>
          <li>second item second subitem first sub-subitem</li>
          <li>second item second subitem second sub-subitem</li>
          <li>second item second subitem third sub-subitem</li>
        </ul>
      </li>
      <!-- Closing </li> tag for the li that
                  contains the third unordered list -->
      <li>second item third subitem</li>
    </ul>
    <!-- Here is the closing </li> tag -->
  </li>
  <li>third item</li>
</ul>
```

#### Result

#### Ordered list inside unordered list

```
<ul>
  <li>first item</li>
  <li>
    second item
    <!-- Look, the closing </li> tag is not placed here! -->
    <ol>
      <li>second item first subitem</li>
      <li>second item second subitem</li>
      <li>second item third subitem</li>
    </ol>
    <!-- Here is the closing </li> tag -->
  </li>
  <li>third item</li>
</ul>
```

### Result

### Technical summary

| | |
|---|---|
| **Content categories** | Flow content, and if the `<ul>` element's children include at least one `<li>` element, palpable content. |
| **Permitted content** | Zero or more `<li>`, `<script>` and `<template>` elements. |
| **Tag omission** | None, both the starting and ending tag are mandatory. |
| **Permitted parents** | Any element that accepts flow content. |
| **Implicit ARIA role** | `list` |
| **Permitted ARIA roles** | `directory`, `group`, `listbox`, `menu`, `menubar`, `none`, `presentation`, `radiogroup`, `tablist`, `toolbar`, `tree` |
| **DOM Interface** | `HTMLUListElement` |

### See also

- Other list-related HTML Elements: `<ol>`, `<li>`, `<menu>`
- CSS properties that may be specially useful to style the `<ul>` element:
  - the `list-style` property, to choose the way the ordinal displays.
  - CSS counters, to handle complex nested lists.
  - the `line-height` property, to simulate the deprecated `compact` attribute.
  - the `margin` property, to control the list indentation.

## <var> HTML variable element

The **`<var>`** HTML element represents the name of a variable in a mathematical expression or a programming context. It's typically presented using an italicized version of the current typeface, although that behavior is browser-dependent.

### Try it

```
<p>
  The volume of a box is <var>l</var> × <var>w</var> × <var>h</var>, where
  <var>l</var> represents the length, <var>w</var> the width and
  <var>h</var> the height of the box.
</p>
```

```
var {
  font-weight: bold;
}
```

### Attributes

This element only includes the global attributes.

### Usage notes

### Related elements

Other elements that are used in contexts in which `<var>` is commonly used include:

- `<code>`: The HTML Code element
- `<kbd>`: The HTML Keyboard input element
- `<samp>`: The HTML Sample Output element

If you encounter code that is mistakenly using `<var>` for style purposes rather than semantic purposes, you should either use a `<span>` with appropriate CSS or, an appropriate semantic element among the following:

- `<em>`
- `<i>`
- `<q>`

### Default style

Most browsers apply `font-style` to `"italic"` when rendering `<var>`. This can be overridden in CSS, like this:

```
var {
  font-style: normal;
}
```

### Examples

### Basic example

Here's a basic example, using `<var>` to denote variable names in a mathematical equation.

```
<p>An algebraic equation: <var>x</var> = <var>y</var> + 2</p>
```

### Result

### Overriding the default style

Using CSS, you can override the default style for the `<var>` element. In this example, variable names are rendered in bold, using Courier if it's available, otherwise it falls back to the default monospace font.

### CSS

```
var {
  font:
    bold 15px "Courier",
    "Courier New",
    monospace;
}
```

### HTML

```
<p>
  The variables <var>minSpeed</var> and <var>maxSpeed</var> control the minimum
  and maximum speed of the apparatus in revolutions per minute (RPM).
</p>
```

This HTML uses `<var>` to enclose the names of two variables.

### Result

### Technical summary

| **Content categories** | Flow content, phrasing content, palpable content. |
|---|---|
| **Permitted content** | Phrasing content. |
| **Tag omission** | None, both the starting and ending tag are mandatory. |
| **Permitted parents** | Any element that accepts phrasing content. |
| **Implicit ARIA role** | No corresponding role |
| **Permitted ARIA roles** | Any |
| **DOM interface** | `HTMLElement` |

## `<video>` HTML video embed element

The **`<video>`** HTML element embeds a media player which supports video playback into the document. You can use `<video>` for audio content as well, but the `<audio>` element may provide a more appropriate user experience.

### Try it

```
<video controls width="250">
  <source src="https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/video//shared-assets/videos/flower.webm"
type="video/webm" />

  <source src="https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/video//shared-assets/videos/flower.mp4"
type="video/mp4" />
```

```
  Download the
  <a href="https://developer.mozilla.org/shared-assets/videos/flower.webm">WEBM</a>
  or
  <a href="https://developer.mozilla.org/shared-assets/videos/flower.mp4">MP4</a>
  video.
</video>
```

The above example shows how to use the `<video>` element. Similar to the `<img>` element, we include a path to the media we want to display inside the `src` attribute; we can include other attributes to specify information such as video width and height, whether we want it to autoplay and loop, or to show the browser's default video controls, and so on.

The content inside the opening and closing `<video></video>` tags is shown as a fallback in browsers that don't support the element.

### Attributes

Like all other HTML elements, this element supports the global attributes.

`autoplay`

A Boolean attribute; if specified, the video automatically begins to play back as soon as it can without stopping to finish loading the data.

> **Note:** Modern browsers block audio (or videos with an unmuted audio track) from autoplaying, as sites that automatically play audio can be an unpleasant experience for users. See our autoplay guide for additional information about how to properly use autoplay.

To disable video autoplay, `autoplay="false"` will not work; the video will autoplay if the attribute is there in the `<video>` tag at all. To remove autoplay, the attribute needs to be removed altogether.

> **Note:** Videos with the `loading="lazy"` attribute set will not start downloading and autoplaying until the element is near or within the viewport.

`controls`

If this attribute is present, the browser will offer controls to allow the user to control video playback, including volume, seeking, and pause/resume playback.

`controlslist`

The `controlslist` attribute, when specified, helps the browser select what controls to show for the `video` element whenever the browser shows its own set of controls (that is, when the `controls` attribute is specified).

The allowed values are `nodownload`, `nofullscreen` and `noremoteplayback`.

Use the `disablepictureinpicture` attribute if you want to disable the Picture-In-Picture mode (and the control).

`crossorigin`

This enumerated attribute indicates whether to use CORS to fetch the related video. CORS-enabled resources can be reused in the `<canvas>` element without being *tainted*. The allowed values are:

`anonymous`

Sends a cross-origin request without a credential. In other words, it sends the `Origin:` HTTP header without a cookie, X.509 certificate, or performing HTTP Basic authentication. If the server does not give credentials to the origin site (by not setting the `Access-Control-Allow-Origin:` HTTP header), the resource will be *tainted*, and its usage restricted.

`use-credentials`

Sends a cross-origin request with a credential. In other words, it sends the `Origin:` HTTP header with a cookie, a certificate, or performing HTTP Basic authentication. If the server does not give credentials to the origin site (through `Access-Control-Allow-Credentials:` HTTP header), the resource will be *tainted* and its usage restricted.

When not present, the resource is fetched without a CORS request (i.e., without sending the `Origin:` HTTP header), preventing its non-tainted use in `<canvas>` elements. If invalid, it is handled as if the enumerated keyword `anonymous` was used. See CORS settings attributes for additional information.

`disablepictureinpicture`

Prevents the browser from suggesting a Picture-in-Picture context menu or to request Picture-in-Picture automatically in some cases.

`disableremoteplayback`

A Boolean attribute used to disable the capability of remote playback in devices that are attached using wired (HDMI, DVI, etc.) and wireless technologies (Miracast, Chromecast, DLNA, AirPlay, etc.).

In Safari, you can use `x-webkit-airplay="deny"` as a fallback.

`height`

The height of the video's display area, in CSS pixels (absolute values only; no percentages).

`loading`

Indicates how the browser should load the video (including any poster image):

`eager`

Loads the video immediately, regardless of whether or not the video is currently within the visible viewport (this is the default value).

`lazy`

Defers loading the video until it reaches a calculated distance from the viewport, as defined by the browser.

Lazy loading avoids the network and storage bandwidth required to handle the video until it's reasonably certain that it will be needed. This improves the performance in most typical use cases.

While explicit `width` and `height` attributes are recommended for all videos to avoid layout shift, they are especially important for lazy-loaded ones. Lazy-loaded videos will never be loaded if they do not intersect a visible part of an element, even if loading them would change that, because unloaded videos have a `width` and `height` of `0`. It creates an even more disruptive user experience when the content visible in the viewport reflows in the middle of reading it.

Lazy-loaded videos located in the visual viewport may not yet be visible when the Window `load` event is fired. This is because the event is fired based on eager-loaded videos — lazy-loaded videos are not considered even if they are located within the visual viewport upon initial page load.

Loading is only deferred when JavaScript is enabled. This is an anti-tracking measure, because if a user agent supported lazy loading when scripting is disabled, it would still be possible for a site to track a user's approximate scroll position throughout a session, by strategically placing videos in a page's markup such that a server can track how many videos are requested and when.

> **Note:** The `loading="lazy"` attribute also impacts the `autoplay`, `poster`, and `preload` attributes, as described in each of those sections of the page.

`loop`

A Boolean attribute; if specified, the browser will automatically seek back to the start upon reaching the end of the video.

`muted`

A Boolean attribute that indicates the default audio mute setting contained in the video. If set, the audio will be initially silenced. Its default value is `false`, meaning the audio will be heard when the video is played.

> **Note:** To unmute, setting `muted="false"` will not work; the audio will be muted if the attribute is present at all. To unmute, the attribute must be removed entirely.

`playsinline`

A Boolean attribute indicating that the video is to be played "inline", that is, within the element's playback area. Note that the absence of this attribute *does not* imply that the video will always be played in fullscreen.

`poster`

A URL for an image to be shown while the video is downloading. If this attribute isn't specified, nothing is displayed until the first frame is available, then the first frame is shown as the poster frame.

> **Note:** Videos with the `loading="lazy"` attribute set will only download the `poster` resource once the video is near or within the viewport.

`preload`

This enumerated attribute is intended to provide a hint to the browser about what the author thinks will lead to the best user experience regarding what content is loaded before the video is played. It may have one of the following values:

- `none`: Indicates that the video should not be preloaded.

- `metadata`: Indicates that only video metadata (e.g., length) is fetched.
- `auto`: Indicates that the whole video file can be downloaded, even if the user is not expected to use it.
- *empty string*: A synonym of the `auto` value.

The default value is different for each browser. The spec advises it to be set to `metadata`.

> **Note:**
>
> - Videos with the `loading="lazy"` attribute set will only apply the `preload` behavior once the video is near or within the viewport.
> - The `autoplay` attribute has precedence over `preload`. If `autoplay` is specified, the browser would obviously need to start downloading the video for playback.
> - The specification does not force the browser to follow the value of this attribute; it is a mere hint.

`src`

The URL of the video to embed. This is optional; you may instead use the `<source>` element within the video block to specify the video to embed.

`width`

The width of the video's display area, in CSS pixels (absolute values only; no percentages).

### Events

`audioprocess`

The input buffer of a `ScriptProcessorNode` is ready to be processed.

`canplay`

The browser can play the media, but estimates that not enough data has been loaded to play the media up to its end without having to stop for further buffering of content.

`canplaythrough`

The browser estimates it can play the media up to its end without stopping for content buffering.

`complete`

The rendering of an `OfflineAudioContext` is terminated.

`durationchange`

The `duration` attribute has been updated.

`emptied`

The media has become empty; for example, this event is sent if the media has already been loaded (or partially loaded), and the `load()` method is called to reload it.

`ended`

Playback has stopped because the end of the media was reached.

`error`

An error occurred while fetching the media data, or the type of the resource is not a supported media format.

`loadeddata`

The first frame of the media has finished loading.

`loadedmetadata`

The metadata has been loaded.

`loadstart`

Fired when the browser has started to load the resource.

`pause`

Playback has been paused.

`play`

Playback has begun.

`playing`

Playback is ready to start after having been paused or delayed due to lack of data.

`progress`

Fired periodically as the browser loads a resource.

`ratechange`

The playback rate has changed.

`seeked`

A *seek* operation completed.

`seeking`

A *seek* operation began.

`stalled`

The user agent is trying to fetch media data, but data is unexpectedly not forthcoming.

`suspend`

Media data loading has been suspended.

`timeupdate`

The time indicated by the `currentTime` attribute has been updated.

`volumechange`

The volume has changed.

`waiting`

Playback has stopped because of a temporary lack of data.

### Usage notes

Browsers don't all support the same video formats; you can provide multiple sources inside nested `<source>` elements, and the browser will then use the first one it understands.

```
<video controls>
  <source src="https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/video/myVideo.webm" type="video/webm" />
  <source src="https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/video/myVideo.mp4" type="video/mp4" />
  <p>
    Your browser doesn't support HTML video. Here is a
    <a href="myVideo.mp4" download="myVideo.mp4">link to the video</a> instead.
  </p>
</video>
```

When using `<source>` elements, the browser attempts to load each source sequentially. If a source fails (e.g., due to an invalid URL or unsupported format), the next source is attempted, and so on. An `error` event fires on the `<video>` element after all sources have failed; `error` events are not fired on each individual `<source>` element.

We offer a substantive and thorough guide to media file types and the guide to the codecs supported for video. Also available is a guide to audio codecs that can be used with them.

Other usage notes:

- If you don't specify the `controls` attribute, the video won't include the browser's default controls; you can create your own custom controls using JavaScript and the `HTMLMediaElement` API. See Creating a cross-browser video player for more details.
- To allow precise control over your video (and audio) content, `HTMLMediaElement`s fire many different events. In addition to providing controllability, these events let you monitor the progress of both download and playback of the media, as well as the playback state and position.
- You can use the `object-position` property to adjust the positioning of the video within the element's frame, and the `object-fit` property to control how the video's size is adjusted to fit within the frame.
- To show subtitles/captions along with your video, you can use some JavaScript along with the `<track>` element and the WebVTT format. See Adding captions and subtitles to HTML video for more information.

- You can play audio files using a `<video>` element. This can be useful if, for example, you need to perform audio with a WebVTT transcript, since the `<audio>` element doesn't allow captions using WebVTT.
- To test the fallback content on browsers that support the element, you can replace `<video>` with a non-existing element like `<notavideo>`.

A good general source of information on using HTML `<video>` is the HTML video and audio beginner's tutorial.

### Styling with CSS

The `<video>` element is a replaced element — its `display` value is `inline` by default — but its default width and height in the viewport is defined by the video being embedded.

There are no special considerations for styling `<video>`; a common strategy is to give it a `display` value of `block` to make it easier to position, size, etc., and then provide styling and layout information as required. Video player styling basics provides some useful styling techniques.

### Adding subtitles and other timed text tracks

Timed text tracks for subtitles, closed captions, chapter headings, and so on, can be added declaratively by nesting the `<track>` element. The tracks are specified in Web Video Text Tracks File Format (WebVTT) (`.vtt` files).

For example, the HTML below includes the file "captions.vtt", which will be used to overlay closed captions on the video if captions are enabled by the user.

```
<video controls src="https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/video/video.webm">
  <track default kind="captions" src="https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/video/captions.vtt" />
</video>
```

Timed text tracks can also be added programmatically using the WebVTT API.

### Detecting track addition and removal

You can detect when tracks are added to and removed from a `<video>` element using the `addtrack` and `removetrack` events. However, these events aren't sent directly to the `<video>` element itself. Instead, they're sent to the track list object within the `<video>` element's `HTMLMediaElement` that corresponds to the type of track that was added to the element:

`HTMLMediaElement.audioTracks`

> An `AudioTrackList` containing all of the media element's audio tracks. Add a listener for `addtrack` to this object to be notified when new audio tracks are added to the element.

`HTMLMediaElement.videoTracks`

> A `VideoTrackList` containing all of the media element's video tracks. Add an `addtrack` listener to this object to be notified when video tracks are added to the element.

`HTMLMediaElement.textTracks`

> A `TextTrackList` containing all of the media element's text tracks (which are used for subtitles, closed captions, and so on). Add an `addtrack` listener to this object to be notified when text tracks are added to the element.

For example, to detect when audio tracks are added to or removed from a `<video>` element, you can use code like this:

```
const elem = document.querySelector("video");

elem.audioTracks.onaddtrack = (event) => {
  trackEditor.addTrack(event.track);
};

elem.audioTracks.onremovetrack = (event) => {
  trackEditor.removeTrack(event.track);
};
```

This code watches for audio tracks to be added to and removed from the element, and calls a hypothetical function on a track editor to register and remove the track from the editor's list of available tracks.

You can also use `addEventListener()` to listen for the `addtrack` and `removetrack` events.

### Server support for video

If the MIME type for the video is not set correctly on the server, the video may not show or show a gray box containing an X (if JavaScript is enabled).

If you use Apache Web Server to serve WebM videos, you can fix this problem by adding the video file type extensions to the `video/webm` MIME type (the most common WebM file extension is `.webm`). To do this, edit the `mime.types` file in `/etc/apache` or use the `AddType` configuration directive in `httpd.conf`:

```
AddType video/webm .webm
```

Your web host may provide an easy interface to MIME type configuration changes for new technologies until a global update naturally occurs.

### Accessibility

Videos should provide both captions and transcripts that accurately describe their content (see Adding captions and subtitles to HTML video for more information on how to implement these). Captions allow people who are experiencing hearing loss to understand a video's audio content as the video is being played, while transcripts allow people who need additional time to be able to review audio content at a pace and format that is comfortable for them.

It's worth noting that while you can caption audio-only media, you can only do so when playing audio in a `<video>` element since the video region of the element is used to present the captions. This is one of the special scenarios in which it's useful to play audio in a video element.

If automatic captioning services are used, it is important to review the generated content to ensure it accurately represents the source video.

In addition to spoken dialog, subtitles and transcripts should also identify music and sound effects that communicate important information. This includes emotion and tone:

```
14
00:03:14 --> 00:03:18
[Dramatic rock music]

15
00:03:19 --> 00:03:21
[whispering] What's that off in the distance?

16
00:03:22 --> 00:03:24
It's… it's a…

16 00:03:25 --> 00:03:32
[Loud thumping]
[Dishes clattering]
```

Captions should not obstruct the main subject of the video. They can be positioned using the `align` VTT cue setting.

- Web Video Text Tracks Format (WebVTT)
- WebAIM: Captions, Transcripts, and Audio Descriptions
- MDN Understanding WCAG, Guideline 1.2 explanations
- Understanding Success Criterion 1.2.1 | W3C Understanding WCAG 2.0
- Understanding Success Criterion 1.2.2 | W3C Understanding WCAG 2.0

### Examples

### Single source

This example plays a video when activated, providing the user with the browser's default video controls to control playback.

### HTML

```
<!-- Basic video example -->
<!-- 'Big Buck Bunny' licensed under CC 3.0 by the Blender foundation. Hosted by archive.org -->
<!-- Poster from peach.blender.org -->
<video
  controls
  src="https://archive.org/download/BigBuckBunny_124/Content/big_buck_bunny_720p_surround.mp4"
  poster="https://peach.blender.org/wp-content/uploads/title_anouncement.jpg?x11217"
  width="620">
  Sorry, your browser doesn't support embedded videos, but don't worry, you can
  <a href="https://archive.org/details/BigBuckBunny_124">download it</a>
  and watch it with your favorite video player!
</video>
```

### Result

Until the video starts playing, the image provided in the `poster` attribute is displayed in its place. If the browser doesn't support video playback, it displays the fallback text.

### Multiple sources

This example builds on the last one, offering three different sources for the media; this allows the video to be watched regardless of which video codecs are supported by the browser.

### HTML

```
<!-- Using multiple sources as fallbacks for a video tag -->
<!-- 'Elephants Dream' by Orange Open Movie Project Studio, licensed under CC-3.0, hosted by archive.org -->
<!-- Poster hosted by Wikimedia -->
<video
  width="620"
  controls
  poster="https://upload.wikimedia.org/wikipedia/commons/e/e8/Elephants_Dream_s5_both.jpg">
  <source
    src="https://archive.org/download/ElephantsDream/ed_hd.avi"
    type="video/avi" />
  <source
    src="https://archive.org/download/ElephantsDream/ed_1024_512kb.mp4"
    type="video/mp4" />

  Sorry, your browser doesn't support embedded videos, but don't worry, you can
  <a
    href="https://archive.org/download/ElephantsDream/ed_1024_512kb.mp4"
    download="ed_1024_512kb.mp4">
    download the MP4
  </a>
  and watch it with your favorite video player!
</video>
```

### Result

First AVI is tried. If that can't be played, MP4 is tried. A fallback message is displayed if the video element isn't supported, but not if all sources fail.

Some media file types let you provide more specific information using the `codecs` parameter as part of the file's type string. For example, `video/webm; codecs="vp8, vorbis"` says that the file is a WebM video using VP8 for its video and Vorbis for audio.

### Technical summary

| | |
|---|---|
| **Content categories** | Flow content, phrasing content, embedded content. If it has a `controls` attribute: interactive content and palpable content. |
| **Permitted content** | If the element has a `src` attribute: zero or more `<track>` elements, followed by transparent content that contains no media elements–that is no `<audio>` or `<video>`.<br><br>Else: zero or more `<source>` elements, followed by zero or more `<track>` elements, followed by transparent content that contains no media elements–that is no `<audio>` or `<video>`. |
| **Tag omission** | None, both the starting and ending tag are mandatory. |
| **Permitted parents** | Any element that accepts embedded content. |
| **Implicit ARIA role** | No corresponding role |
| **Permitted ARIA roles** | `application` |
| **DOM interface** | `HTMLVideoElement` |

### See also

- Guide to media types and formats on the web
  - Media container formats (file types)
  - Web video codec guide
  - Web audio codec guide
- Positioning and sizing the picture within its frame: `object-position` and `object-fit`
- `<audio>`
- HTML video and audio
- Manipulating video using canvas
- Configuring servers for Ogg media

## <wbr> HTML line break opportunity element

The **<wbr>** HTML element represents a word break opportunity—a position within text where the browser may optionally break a line, though its line-breaking rules would not otherwise create a break at that location.

### Try it

```
<div id="example-paragraphs">
  <p>Fernstraßenbauprivatfinanzierungsgesetz</p>
  <p>Fernstraßen<wbr />bau<wbr />privat<wbr />finanzierungs<wbr />gesetz</p>
  <p>Fernstraßen&shy;bau&shy;privat&shy;finanzierungs&shy;gesetz</p>
</div>
```

```
#example-paragraphs {
  background-color: white;
  overflow: hidden;
  resize: horizontal;
  width: 9rem;
  border: 2px dashed #999999;
}
```

### Attributes

This element only includes the global attributes.

### Notes

`<wbr>` behaves like the `U+200B ZERO-WIDTH SPACE` code point. In particular, it behaves like a Unicode bidi BN code point, meaning it has no effect on bidi-ordering: `<div dir=rtl>123,<wbr>456</div>` displays, when not broken on two lines, `123,456` and not `456,123`.

For the same reason, the `<wbr>` element does not introduce a hyphen at the line break point. To make a hyphen appear only at the end of a line, use the soft hyphen character entity (``) instead.

### Examples

*The Yahoo Style Guide* recommends breaking a URL *before* punctuation, to avoid leaving a punctuation mark at the end of the line, which the reader might mistake for the end of the URL.

```
<p>
  http://this<wbr />.is<wbr />.a<wbr />.really<wbr />.long<wbr />.example<wbr />.com/With<wbr />/deeper<wbr />/level<wbr />/pages<wbr
/>/deeper<wbr />/level<wbr />/pages<wbr />/deeper<wbr />/level<wbr />/pages<wbr />/deeper<wbr />/level<wbr />/pages<wbr />/deeper<wbr
/>/level<wbr />/pages
</p>
```

### Result

### Technical summary

| | |
|---|---|
| **Content categories** | Flow content, phrasing content. |
| **Permitted content** | Empty |
| **Tag omission** | Must have a start tag and must not have an end tag. |
| **Permitted parents** | Any element that accepts phrasing content. |
| **Implicit ARIA role** | No corresponding role |
| **Permitted ARIA roles** | Any |
| **DOM interface** | `HTMLElement` |

### See also

- `overflow-wrap`
- `word-break`
- `hyphens`
- The `<br>` element

## <xmp> HTML plain text rendering element

### Summary

The **<xmp>** HTML element renders text between the start and end tags without interpreting the HTML in between and using a monospaced font. The HTML2 specification recommended that it should be rendered wide enough to allow 80 characters per line.

> **Note:** Do not use this element.

- It has been deprecated since HTML3.2 and was not implemented in a consistent way. It was completely removed from current HTML.
- Use the `<pre>` element or, if semantically adequate, the `<code>` element instead. Note that you will need to escape the `<` character as `<` and the `&` character as `&` to make sure they are not interpreted as markup.
- A monospaced font can also be obtained on any element, by applying an adequate CSS style using `monospace` as the generic-font value for the `font-family` property.

### Attributes

This element has no other attributes than the global attributes, common to all elements.

### DOM interface

This element implements the `HTMLElement` interface.

### See also

- The `<pre>` and `<code>` elements to be used instead.
- The `<plaintext>` element, similar to `<xmp>` but also obsolete.

## Global attributes

**Global attributes** are attributes common to all HTML elements; they can be used on all elements, though they may have no effect on some elements.

Global attributes may be specified on all HTML elements, *even those not specified in the standard*. That means that any non-standard elements must still permit these attributes, even though using those elements means that the document is no longer HTML5-compliant. For example, HTML5-compliant browsers hide content marked as `<foo hidden>…</foo>`, even though `<foo>` is not a valid HTML element.

In addition to the basic HTML global attributes, the following global attributes also exist:

- `xml:lang` and `xml:base` — these are inherited from the XHTML specifications and deprecated, but kept for compatibility purposes.
- The ARIA `role` attribute and the multiple `aria-*` states and properties, used for ensuring accessibility.
- The event handler attributes listed below.

### List of global attributes

`accesskey`

> Provides a hint for generating a keyboard shortcut for the current element. This attribute consists of a space-separated list of characters. The browser should use the first one that exists on the computer keyboard layout.

`anchor`

> Associates a positioned element with an anchor element. The attribute's value is the `id` value of the element you want to anchor the positioned element to. The element can then be positioned using CSS anchor positioning.

`autocapitalize`

> Controls whether inputted text is automatically capitalized and, if so, in what manner.

`autocorrect`

> Controls whether input text is automatically corrected for spelling errors. This can be applied to elements that have editable text except for `<input>` elements with the attribute: `type="password"`, `type="email"`, or `type="url"`.

`autofocus`

> Indicates that an element is to be focused on page load, or as soon as the `<dialog>` it is part of is displayed. This attribute is a boolean, initially false.

`class`

> A space-separated list of the classes of the element. Classes allow CSS and JavaScript to select and access specific elements via the class selectors or functions like the method `Document.getElementsByClassName()`.

`contenteditable`

> An enumerated attribute indicating if the element should be editable by the user. If so, the browser modifies its widget to allow editing. The attribute must take one of the following values:
>
> - `true` or the *empty string*, which indicates that the element must be editable;
> - `false`, which indicates that the element must not be editable.
> - `plaintext-only`, which indicates the element's raw text is editable, but rich text formatting is disabled.

`data-*`

Forms a class of attributes, called custom data attributes, that allow proprietary information to be exchanged between the HTML and its DOM representation that may be used by scripts. All such custom data are available via the `HTMLElement` interface of the element the attribute is set on. The `HTMLElement.dataset` property gives access to them.

`dir`

An enumerated attribute indicating the directionality of the element's text. It can have the following values:

- `ltr`, which means *left to right* and is to be used for languages that are written from the left to the right (like English);
- `rtl`, which means *right to left* and is to be used for languages that are written from the right to the left (like Arabic);
- `auto`, which lets the user agent decide. It uses a basic algorithm as it parses the characters inside the element until it finds a character with a strong directionality, then it applies that directionality to the whole element.

`draggable`

An enumerated attribute indicating whether the element can be dragged, using the Drag and Drop API. It can have the following values:

- `true`, which indicates that the element may be dragged
- `false`, which indicates that the element may not be dragged.

`enterkeyhint`

Hints what action label (or icon) to present for the enter key on virtual keyboards.

`exportparts`

Used to transitively export shadow parts from a nested shadow tree into a containing light tree.

`headingoffset`

Increases the computed heading level of the heading elements inside the element, without changing the elements used to write them.

`headingreset`

A boolean attribute that stops the `headingoffset` values of ancestor elements from applying to the headings inside the element.

`hidden`

An enumerated attribute indicating that the element is not yet, or is no longer, *relevant*. For example, it can be used to hide elements of the page that can't be used until the login process has been completed. The browser won't render such elements. This attribute must not be used to hide content that could legitimately be shown.

`id`

Defines a unique identifier (ID) which must be unique in the whole document. Its purpose is to identify the element when linking (using a fragment identifier), scripting, or styling (with CSS).

`inert`

A boolean value that makes the browser disregard user input events for the element. Useful when click events are present.

`inputmode`

Provides a hint to browsers about the type of virtual keyboard configuration to use when editing this element or its contents. Used primarily on `<input>` elements, but is usable on any element while in `contenteditable` mode.

`is`

Allows you to specify that a standard HTML element should behave like a registered customized built-in element (see Using custom elements for more details).

> **Note:** The `item*` attributes are part of the WHATWG HTML Microdata feature.

`itemid`

The unique, global identifier of an item.

`itemprop`

Used to add properties to an item. Every HTML element may have an `itemprop` attribute specified, where an `itemprop` consists of a name and value pair.

`itemref`

Properties that are not descendants of an element with the `itemscope` attribute can be associated with the item using an `itemref`. It provides a list of element ids (not `itemid`s) with additional properties elsewhere in the document.

`itemscope`

`itemscope` (usually) works along with `itemtype` to specify that the HTML contained in a block is about a particular item. `itemscope` creates the Item and defines the scope of the `itemtype` associated with it. `itemtype` is a valid URL of a vocabulary (such as schema.org) that describes the item and its properties context.

`itemtype`

Specifies the URL of the vocabulary that will be used to define `itemprop`s (item properties) in the data structure. `itemscope` is used to set the scope of where in the data structure the vocabulary set by `itemtype` will be active.

`lang`

Helps define the language of an element: the language that non-editable elements are in, or the language that editable elements should be written in by the user. The attribute should contain a valid BCP 47 language tag. `xml:lang` has priority over it.

`nonce`

A cryptographic nonce ("number used once") which can be used by Content Security Policy to determine whether or not a given fetch will be allowed to proceed.

`part`

A space-separated list of the part names of the element. Part names allows CSS to select and style specific elements in a shadow tree via the `::part` pseudo-element.

`popover`

Used to designate an element as a popover element (see Popover API). Popover elements are hidden via `display: none` until opened via an invoking/control element (i.e., a `<button>` or `<input type="button">` with a `popovertarget` attribute) or a `HTMLElement.showPopover()` call.

`role`

Roles define the semantic meaning of content, allowing screen readers and other tools to present and support interaction with an object in a way that is consistent with user expectations of that type of object. `role`s are added to HTML elements using `role="role_type"`, where `role_type` is the name of a role in the ARIA specification.

`slot`

Assigns a slot in a shadow DOM shadow tree to an element: An element with a `slot` attribute is assigned to the slot created by the `<slot>` element whose `name` attribute's value matches that `slot` attribute's value.

`spellcheck`

An enumerated attribute defines whether the element may be checked for spelling errors. It may have the following values:

- empty string or `true`, which indicates that the element should be, if possible, checked for spelling errors;
- `false`, which indicates that the element should not be checked for spelling errors.

`style`

Contains CSS styling declarations to be applied to the element. Note that it is recommended for styles to be defined in a separate file or files. This attribute and the `<style>` element have mainly the purpose of allowing for quick styling, for example for testing purposes.

`tabindex`

An integer attribute indicating if the element can take input focus (is *focusable*), if it should participate to sequential keyboard navigation, and if so, at what position. It can take several values:

- a *negative value* means that the element should be focusable, but should not be reachable via sequential keyboard navigation;
- `0` means that the element should be focusable and reachable via sequential keyboard navigation, but its relative order is defined by the platform convention;
- a *positive value* means that the element should be focusable and reachable via sequential keyboard navigation; the order in which the elements are focused is the increasing value of the `tabindex`. If several elements share the same tabindex, their relative order follows their relative positions in the document.

`title`

Contains a text representing advisory information related to the element it belongs to. Such information can typically, but not necessarily, be presented to the user as a tooltip.

`translate`

An enumerated attribute that is used to specify whether an element's attribute values and the values of its `Text` node children are to be translated when the page is localized, or whether to leave them unchanged. It can have the following values:

- empty string or `yes`, which indicates that the element will be translated.
- `no`, which indicates that the element will not be translated.

`virtualkeyboardpolicy`

An enumerated attribute used to control the on-screen virtual keyboard behavior on devices such as tablets, mobile phones, or other devices where a hardware keyboard may not be available for elements that its content is editable (for example, it is an `<input>` or `<textarea>` element, or an element with the `contenteditable` attribute set).

- `auto` or an *empty string*, which automatically shows the virtual keyboard when the element is focused or tapped.
- `manual`, which decouples focus and tap on the element from the virtual keyboard's state.

`writingsuggestions`

An enumerated attribute indicating if browser-provided writing suggestions should be enabled under the scope of the element or not.

- `false`, which disables the browser's writing suggestions.
- `true` or an *empty string*, which enables writing suggestions.

### List of global event handler attributes

HTML event handler attributes are discouraged; see HTML attribute reference for how they work.

While the attributes listed below apply to all elements, they are not useful on all elements. For example, `onvolumechange` HTML attribute is accepted by all elements and attaches an event listener for `volumechange`, but only media elements will ever receive a `volumechange` event fired by the browser. For other elements, you can only use `EventTarget.dispatchEvent()` to manually dispatch one. Some attributes can be specified on `<body>`, but they would instead listen to events on `window`.

- `onabort`
- `onanimationcancel`
- `onanimationend`
- `onanimationiteration`
- `onanimationstart`
- `onauxclick`
- `onbeforeinput`
- `onbeforematch`
- `onbeforetoggle`
- `onblur`
- `oncancel`
- `oncanplay`
- `oncanplaythrough`
- `onchange`
- `onclick`
- `onclose`
- `oncommand`
- `oncontentvisibilityautostatechange`
- `oncontextlost`
- `oncontextmenu`
- `oncontextrestored`
- `oncopy`
- `oncuechange`
- `oncut`
- `ondblclick`
- `ondrag`
- `ondragend`
- `ondragenter`
- `ondragleave`
- `ondragover`
- `ondragstart`
- `ondrop`
- `ondurationchange`
- `onemptied`
- `onended`
- `onerror`
- `onfocus`
- `onfocusin`
- `onfocusout`
- `onformdata`

- onfullscreenchange
- onfullscreenerror
- ongesturechange
- ongestureend
- ongesturestart
- ongotpointercapture
- oninput
- oninvalid
- onkeydown
- onkeypress
- onkeyup
- onload
- onloadeddata
- onloadedmetadata
- onloadstart
- onlostpointercapture
- onmousedown
- onmouseenter
- onmouseleave
- onmousemove
- onmouseout
- onmouseover
- onmouseup
- onmousewheel
- onpaste
- onpause
- onplay
- onplaying
- onpointercancel
- onpointerdown
- onpointerenter
- onpointerleave
- onpointermove
- onpointerout
- onpointerover
- onpointerrawupdate
- onpointerup
- onprogress
- onratechange
- onreset
- onresize
- onscroll
- onscrollend
- onscrollsnapchange
- onscrollsnapchanging
- onsecuritypolicyviolation
- onseeked
- onseeking
- onselect
- onselectionchange
- onselectstart
- onslotchange
- onstalled
- onsubmit
- onsuspend
- ontimeupdate
- ontoggle
- ontouchcancel
- ontouchend
- ontouchmove
- ontouchstart
- ontransitioncancel
- ontransitionend
- ontransitionrun
- ontransitionstart
- onvolumechange

- `onwaiting`
- `onwebkitmouseforcechanged`
- `onwebkitmouseforcedown`
- `onwebkitmouseforceup`
- `onwebkitmouseforcewillbegin`
- `onwheel`

### See also

- `Element` interface that allows querying most global attributes.

## accesskey HTML global attribute

The **`accesskey`** global attribute provides a hint for generating a keyboard shortcut for the current element. The attribute value must consist of a single printable character (which includes accented and other characters that can be generated by the keyboard).

### Try it

```
<p>If you need to relax, press the <b>S</b>tress reliever!</p>
<button accesskey="s">Stress reliever</button>
```

```
b {
  text-decoration: underline;
}
```

The way to activate the accesskey depends on the browser and its platform:

| | Windows | Linux | Mac |
|---|---|---|---|
| **Firefox** | `Alt` + `Shift` + *`key`* | | `Control` + `Option` + *`key`* or `Control` + `Alt` + *`key`* |
| **MS Edge** | `Alt` + *`key`* | `Control` + `Option` + *`key`* or `Control` + `Option` + `Shift` + *`key`* | `Control` + `Option` + *`key`* |
| **Google Chrome** | | | |
| **Safari** | n/a | | `Control` + `Option` + *`key`* |
| **Opera** | `Alt` + *`key`* | `Alt` + `Shift` + *`key`* | `Control` + `Alt` + *`key`* |

### Accessibility concerns

There are numerous concerns with the `accesskey` attribute:

- An `accesskey` value can conflict with a system or browser keyboard shortcut, or assistive technology functionality. What may work for one combination of operating system, assistive technology, and browser may not work with other combinations.
- Certain `accesskey` values may not be present on certain keyboards, especially when internationalization is a concern. So adapting to specific languages could cause further problems.
- `accesskey` values that rely on numbers may be confusing to individuals experiencing cognitive concerns, where the number doesn't have a logical association with the functionality it triggers.
- Informing the user that `accesskeys` are present, so that they are aware of the functionality. If the system lacks a method of notifying the user about this feature, the user might accidentally activate `accesskeys`.

Because of these issues, it is generally advised not to use `accesskeys` for most general-purpose websites and web apps.

- WebAIM: Keyboard Accessibility - Accesskey

### See also

- `HTMLElement.accessKey`
- `HTMLElement.accessKeyLabel`
- All global attributes.
- `aria-keyshortcuts`

## anchor HTML global attribute

> **Non-standard:** This feature is not standardized. We do not recommend using non-standard features in production, as they have limited browser support, and may change or be removed. However, they can be a suitable alternative in specific cases where no standard option exists.

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`anchor`** global attribute is used to associate a positioned element with an anchor element. The attribute's value is the `id` value of the element you want to anchor the positioned element to. The element can then be positioned using CSS anchor positioning.

> **Note:** Alternatively, you can associate a positioned element with an anchor element via CSS, using the `anchor-name` and `position-anchor` properties. If both anchoring techniques are used on the same element, the CSS technique takes precedence over the HTML technique.

## Examples

### Basic <code>anchor</code> attribute usage

The following example uses HTML to associate a positioned element with an anchor. CSS is then used to tether the positioned element to the right of the anchor.

### HTML

We create a `<div>` element with an `id` of `example-anchor`. This is our anchor element. We then include another `<div>` with the `anchor` attribute set to `example-anchor`. This designates the first `<div>` as the anchor for the second `<div>`, associating the two together.

We also include some filler text around the two `<div>`s to make the `<body>` taller so that it will scroll.

```
<p>
  Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor
  incididunt ut labore et dolore magna aliqua. Dui nunc mattis enim ut tellus
  elementum sagittis vitae et.
</p>

<div class="anchor" id="example-anchor">⚓</div>

<div class="infobox" anchor="example-anchor">
  <p>This is an information box.</p>
</div>

<p>
  Nisi quis eleifend quam adipiscing vitae proin sagittis nisl rhoncus. In arcu
  cursus euismod quis viverra nibh cras pulvinar. Vulputate ut pharetra sit amet
  aliquam.
</p>

<p>
  Malesuada nunc vel risus commodo viverra maecenas accumsan lacus. Vel elit
  scelerisque mauris pellentesque pulvinar pellentesque habitant morbi
  tristique. Porta lorem mollis aliquam ut porttitor. Turpis cursus in hac
  habitasse platea dictumst quisque. Dolor sit amet consectetur adipiscing elit.
  Ornare lectus sit amet est placerat. Nulla aliquet porttitor lacus luctus
  accumsan.
</p>
```

### CSS

We use CSS to convert the `infobox` element into an *anchor-positioned element* and position it relative to its anchor. We set its:

- `position` property to `fixed`, converting it to a positioned element so it can be positioned relative to the anchor's position.
- `left` property to an `anchor()` function with a value of `right`. This tethers the positioned element to its anchor, positioning its left edge flush to the anchor's right edge.
- `align-self` property to `anchor-center`. This causes the infobox to be aligned centrally to the center of the anchor in the inline direction.
- `margin-left` to `10px`, creating space between the anchor-positioned element and its anchor.

```
.infobox {
  position: fixed;
  left: anchor(right);
  align-self: anchor-center;
  margin-left: 10px;
}
```

### Result

Scroll the example to see how the infobox is tethered to the anchor. When the `anchor` attribute is supported, the infobox will be fixed to the right of the anchor. If not supported, the infobox will be fixed to the viewport.

## Specifications

This attribute is not currently part of the HTML specification. Read the discussion about adding the `anchor` attribute at https://github.com/whatwg/html/pull/9144.

### See also

- `HTMLElement.anchorElement`
- CSS `anchor-name` property
- CSS `position-anchor` property
- CSS anchor positioning module

## autocapitalize HTML global attribute

The **`autocapitalize`** global attribute is an enumerated attribute that controls whether inputted text is automatically capitalized and, if so, in what manner. This is relevant to:

- `<input>` and `<textarea>` elements.
- Any element with `contenteditable` set on it.

`autocapitalize` doesn't affect behavior when typing on a physical keyboard. It affects the behavior of other input mechanisms such as virtual keyboards on mobile devices and voice input. This can assist users by making data entry quicker and easier, for example by automatically capitalizing the first letter of each sentence.

### Value

Possible values are:

`none` or `off`

> Do not automatically capitalize any text.

`sentences` or `on`

> Automatically capitalize the first character of each sentence.

`words`

> Automatically capitalize the first character of each word.

`characters`

> Automatically capitalize every character.

### Usage notes

- `autocapitalize` can be set on `<input>` and `<textarea>` elements, *and* on their containing `<form>` elements. When `autocapitalize` is set on a `<form>` element, it sets the autocapitalize behavior for all contained `<input>`s and `<textarea>`s, overriding any `autocapitalize` values set on contained elements.
- `autocapitalize` has no effect on the `url`, `email`, or `password` `<input>` types, where autocapitalization is never enabled.
- Where `autocapitalize` is not specified, the adopted default behavior varies between browsers. For example:
  - Chrome and Safari default to `on`/`sentences`
  - Firefox defaults to `off`/`none`.

### Examples

#### HTML

```
<p>Form to test different autocapitalize settings:</p>

<form>
  <div>
    <label for="default">Default: no autocapitalize set</label>
    <input type="text" id="default" name="default" />
  </div>
  <div>
    <label for="off">autocapitalize="off"</label>
    <input type="text" id="off" name="off" autocapitalize="off" />
  </div>
  <div>
    <label for="none">autocapitalize="none"</label>
    <input type="text" id="none" name="none" autocapitalize="none" />
  </div>
  <div>
    <label for="on">autocapitalize="on"</label>
    <input type="text" id="on" name="on" autocapitalize="on" />
  </div>
```

```
  <div>
    <label for="sentences">autocapitalize="sentences"</label>
    <input
      type="text"
      id="sentences"
      name="sentences"
      autocapitalize="sentences" />
  </div>
  <div>
    <label for="words">autocapitalize="words"</label>
    <input type="text" id="words" name="words" autocapitalize="words" />
  </div>
  <div>
    <label for="characters">autocapitalize="characters"</label>
    <input
      type="text"
      id="characters"
      name="characters"
      autocapitalize="characters" />
  </div>
  <div>
    <label for="characters-ta">autocapitalize="characters" on textarea</label>
    <textarea
      type="text"
      id="characters-ta"
      name="characters-ta"
      autocapitalize="characters">
    </textarea>
  </div>
</form>

<hr />

<p contenteditable autocapitalize="characters">
  This content is editable and has autocapitalize="characters" set on it
</p>
```

### Result

Test the effect on each input using a virtual keyboard or voice entry (keyboard entry will not work).

## autocorrect HTML global attribute

The **`autocorrect`** global attribute is an enumerated attribute that controls whether autocorrection of editable text is enabled for spelling and/or punctuation errors.

The specific autocorrection behavior, including which words are substituted, depends on the user agent and the services provided by the underlying device. For example, on macOS a user agent might rely on registered replacement text and punctuation. Other devices and browsers may use a different approach.

Autocorrection is relevant to editable text elements:

- `<input>` elements, except for `password`, `email`, and `url`, which do not support autocorrection.
- `<textarea>` elements.
- Any element that has the `contenteditable` attribute set.

Editable elements have auto-correction enabled by default, except for within a `<form>` element, where the default value may be inherited from the form. Explicitly setting the attribute overrides the default.

### Value

Possible values are:

`on` or `""` (the empty string)

> Enable automatic correction of spelling and punctuation errors.

`off`

> Disable automatic correction of editable text.

The `<input>` element types that don't support auto-correction always have the `off` state: `password`, `email` and `url`.

For all other editable elements, setting any other value than those listed above is always treated as `on`. The default value for elements that are not nested inside a `<form>` is `on`.

When nested in a `<form>`, the following elements inherit their default value of `autocorrect` from the form if it has been set: `<button>`, `<fieldset>`, `<input>`, `<output>`, `<select>`, and `<textarea>`.

### Examples

#### Basic example

This example demonstrates basic `autocorrect` attribute usage.

#### HTML

We include two text `<input>` elements with different values for their `autocorrect` attributes:

```
<label for="vegetable">A vegetable: </label>
<input id="vegetable" name="vegetable" type="text" autocorrect="on" />

<label for="fruit">A fruit: </label>
<input id="fruit" name="fruit" type="text" autocorrect="off" />
```

#### Results

Enter invalid text into the fruit and vegetable text entry boxes above. If auto-correction is supported on your browser, and there is an appropriate substitution provided by the underlying device, a typo in a vegetable name input should be auto-corrected. Typos should not be corrected in the fruit name field.

#### Enabling and disabling autocorrection

This example shows how you can enable and disable autocorrection using the `autocorrect` attribute.

#### HTML

The HTML markup defines a `<button>`, a "name" `<input>` element of `type="text"`, a "bio" `<textarea>` element, and two `<label>` elements.

The "username" element has `autocorrect="off"` set because auto-correcting a name would be annoying! The bio does not specify a value for `autocorrect`, which means that it is enabled (we could have set any value other than `off`).

```
<button id="reset">Reset</button>
<label for="username">Name: </label>
<input id="username" name="username" type="text" autocorrect="off" />
<label for="bio">Biography: </label>
<textarea id="bio" name="bio"></textarea>
```

#### JavaScript

The code checks whether the `autocorrect` is supported by checking if it is present on the prototype. If it is not present this fact is logged. If it is present then the value of the `autocorrect` property for each of the text-entry elements is logged.

A click handler is added for the button, which allows you to reset the entered text and the log.

```
const resetButton = document.querySelector("#reset");
const userNameElement = document.querySelector("#username");
const bioElement = document.querySelector("#bio");

if (!("autocorrect" in HTMLElement.prototype)) {
  log("autocorrect not supported");
} else {
  log(`userNameElement.autocorrect: ${userNameElement.autocorrect}`);
  log(`bioElement.autocorrect: ${bioElement.autocorrect}`);
}

resetButton.addEventListener("click", (e) => {
  userNameElement.value = "";
  bioElement.value = "";
});
```

#### Results

If auto-correction is supported by your browser, the log area below the "Biography" and "Name" inputs should show that it is enabled for "Biography" inputs but not "Name" inputs.

Enter invalid text into the name and biography text entry boxes. If the device has a substitute for the entered word, this will be used to autocorrect text in the "Biography" input (only).

### See also

- All global attributes.
- `spellcheck`.

## autofocus HTML global attribute

The **`autofocus`** global attribute is a Boolean attribute indicating whether the element should be focused on page load or, if it is nested inside a `<dialog>` or `popover` element, when the `<dialog>` or popover is shown.

```
<input name="q" autofocus />
```

Only one element inside a document, `<dialog>` element, or popover may have this attribute specified. If applied to multiple elements, the first focusable one will receive focus.

> **Note:** The `autofocus` attribute applies to all elements, not just form controls. For example, it might be used on a contenteditable area.

> **Note:** On page load, if a URI fragment identifier is specified and identifies an element, the element with the `autofocus` attribute does not receive focus via the `autofocus` attribute. Generally, the element indicated by the fragment receives focus instead.

### Accessibility concerns

Automatically focusing a form control can confuse visually-impaired people using screen-reading technology and people with cognitive impairments. When `autofocus` is assigned, screen-readers "teleport" their user to the form control without warning them beforehand.

Use careful consideration for accessibility when applying the `autofocus` attribute. Automatically focusing on a control can cause the page to scroll on load. The focus can also cause dynamic keyboards to display on some touch devices. While a screen reader will announce the label of the form control receiving focus, the screen reader will not announce anything before the label, and the sighted user on a small device will equally miss the context created by the preceding content.

## class HTML global attribute

The **`class`** global attribute is a list of the classes of the element, separated by ASCII whitespace.

### Try it

```
<p>Narrator: This is the beginning of the play.</p>

<p class="note editorial">Above point sounds a bit obvious. Remove/rewrite?</p>

<p>Narrator: I must warn you now folks that this beginning is very exciting.</p>

<p class="note">[Lights go up and wind blows; Caspian enters stage right]</p>
```

```
.note {
  font-style: italic;
  font-weight: bold;
}

.editorial {
  background: rgb(255 0 0 / 0.25);
  padding: 10px;
}

.editorial::before {
  content: "Editor: ";
}
```

### Syntax

The `class` attribute is a list of class values separated by ASCII whitespace.

Each class value may contain any Unicode characters (except, of course, ASCII whitespace). However, when used in CSS selectors, either from JavaScript using APIs like `Document.querySelector()` or in CSS stylesheets, class attribute values must be valid CSS identifiers. This means that if a class attribute value is not a valid CSS identifier (for example, `my?class` or `1234`) then it must be escaped before being used in a selector, either using the `CSS.escape()` method or manually.

For this reason, it's recommended that developers choose values for class attributes that are valid CSS identifiers that don't require escaping.

### Description

Classes allow CSS and JavaScript to select and access specific elements via the class selectors or functions like the `document.getElementsByClassName()`.

Though the specification doesn't put requirements on the name of classes, web developers are encouraged to use names that describe the semantic purpose of the element, rather than the presentation of the element. For example, *attribute* to describe an attribute rather than *italics*, although an element of this class may be presented by *italics*. Semantic names remain logical even if the presentation of the page changes.

### See also

- All global attributes.
- `element.className`
- `element.classList`
- Introduction to CSS

## contenteditable HTML global attribute

The **`contenteditable`** global attribute is an enumerated attribute indicating if the element should be editable by the user. If so, the browser modifies its widget to allow editing.

### Try it

```
<blockquote contenteditable="true">
  <p>Edit this content to add your own quote</p>
</blockquote>

<cite contenteditable="true">-- Write your own name here</cite>
```

```
blockquote {
  background: #eeeeee;
  border-radius: 5px;
  margin: 16px 0;
}

blockquote p {
  padding: 15px;
}

cite {
  margin: 16px 32px;
  font-weight: bold;
}

blockquote p::before {
  content: "\201C";
}

blockquote p::after {
  content: "\201D";
}

[contenteditable="true"] {
  caret-color: red;
}
```

### Value

The attribute must take one of the following values:

- `true` or an *empty string*, which indicates that the element is editable.
- `false`, which indicates that the element is not editable.
- `plaintext-only`, which indicates that the element's raw text is editable, but rich text formatting is disabled.

If the attribute is given without a value, like `<label contenteditable>Example Label</label>`, its value is treated as an empty string.

If this attribute is missing or its value is invalid, its value is *inherited* from its parent element: so the element is editable if its parent is editable.

Note that although its allowed values include `true` and `false`, this attribute is an *enumerated* one and not a *Boolean* one.

You can set the color used to draw the text insertion caret with the CSS `caret-color` property.

Elements that are made editable, and therefore interactive, by using the `contenteditable` attribute can be focused. They participate in sequential keyboard navigation. However, elements with the `contenteditable` attribute nested within other `contenteditable` elements are not added to the tabbing sequence by default. You can add the nested `contenteditable` elements to the keyboard navigation sequence by specifying the `tabindex` value (`tabindex="0"`).

If content is pasted into an element with `contenteditable="true"`, all formatting is retained. If content is pasted into an element with `contenteditable="plaintext-only"`, all formatting is removed.

### Examples

#### Pasting content into contenteditable

This example has two `<div>` elements with `contenteditable`, the first with the value `true` and the second with the value `plaintext-only`. Copy the content below and paste it into each `div` to see their effects.

#### HTML

```
<h2>Pasting areas</h2>
<section class="pasting">
  <div class="wrapper">
    <h3>contenteditable="true"</h3>
    <div contenteditable="true"></div>
  </div>
  <div class="wrapper">
    <h3>contenteditable="plaintext-only"</h3>
    <div contenteditable="plaintext-only"></div>
  </div>
</section>
```

### See also

- All global attributes
- `HTMLElement.contentEditable` and `HTMLElement.isContentEditable`
- The CSS `caret-color` property
- HTMLElement `input` event

## dir HTML global attribute

The **`dir`** global attribute is an enumerated attribute that sets the element's *base text direction* and directionally isolates its contents from surrounding text.

### Try it

```
<p dir="rtl">
  This paragraph is in English but incorrectly goes right to left.
</p>
<p dir="ltr">This paragraph is in English and correctly goes left to right.</p>

<hr />

<p lang="ar">هذه الفقرة باللغة العربية ولكن بشكل خاطئ من اليسار إلى اليمين.</p>
<p lang="ar" dir="auto">
  هذه الفقرة باللغة العربية ، لذا يجب الانتقال من اليمين إلى اليسار.
</p>
```

### Values

It can have the following values:

- `ltr`, which sets a *left-to-right* base direction;
- `rtl`, which sets a *right-to-left* base direction;
- `auto`, which lets the user agent determine the base direction from the text, generally using the first character with a strong directionality (skipping `<bdi>`, `<script>`, `<style>`, `<textarea>`, and elements with valid `dir` attributes). For `<textarea>` and `<pre>`, the rendering direction is determined separately for each paragraph of text.

> **Note:** The `auto` value should be used for data with an unknown directionality, like data coming from user input or external data.
>
> If the `dirname` attribute is used when users submit input, it may be possible to render the data with a specified directionality instead of relying on automatic detection.

If unspecified or invalid, the direction is generally inherited from the parent element. Omitting `dir` does not enable automatic direction detection.

### Inheritance

If an element has no `dir` attribute, it generally inherits the direction of its parent element, or the `host` if the parent node is a `ShadowRoot`. If no ancestor sets a direction, the default is left-to-right.

There are exceptions:

- A `<bdi>` element determines its direction from its content, as if `dir="auto"` were set.
- An `<input type="tel">` element uses left-to-right direction.

### Usage notes

The base direction is used by the Unicode Bidirectional Algorithm (BiDi). While characters strongly typed as LTR or RTL (such as Latin, Hebrew, or Arabic letters) establish directionality for themselves and all neutral characters in between (creating "runs" of text), the base direction is necessary in two scenarios:

- It is assumed by neutral characters (like spaces or punctuation) at the boundaries of runs of text with different directionalities, often including at the very beginnings or ends.
- It is used to order runs of text.

For example, consider the first English paragraph in the Try it demo. There are two runs of text: the English text (LTR due to the Latin characters) and the final period (which is at the end of the `p` element, so assumes the base direction RTL). These runs of text are laid out right-to-left, so the text appears first on the right edge, followed to its left by the period. Similarly, the first Arabic paragraph inherits the LTR base direction from the HTML document, so the RTL Arabic text appears on the left edge, followed to its right by the period (with LTR direction). Both paragraphs are typographically incorrect.

Even for documents in a single script, it is recommended to explicitly set `dir` on the root element, which is especially important for RTL scripts, because the default LTR base direction is wrong. The `lang` attribute declares the language, but does not imply the base direction.

This attribute can be overridden by the CSS properties `direction` and `unicode-bidi`, if a CSS page is active and the element supports these properties.

As the directionality of the text is semantically related to its content and not to its presentation, it is recommended that web developers use this attribute instead of the related CSS properties when possible. That way, the text will display correctly even on a browser that doesn't support CSS or has the CSS deactivated.

An image can have its `dir` property set to `"rtl"` in which case the HTML attributes `title` and `alt` will be formatted and defined as `"rtl"`.

When a table has its `dir` set to `"rtl"`, the column order is arranged from right to left.

The `<bdo>` element requires `dir="ltr"` or `dir="rtl"`. On this element, the attribute overrides the characters' intrinsic directionality instead of only setting a base direction.

Browsers might allow users to change the directionality of `<input>` and `<textarea>` elements in order to assist with authoring content. Chrome and Safari provide a directionality option in the contextual menu of input fields. Firefox uses `Ctrl` (Windows)/`Cmd` (macOS) + `Shift` + `X` inside a `<textarea>` to toggle text direction. These features toggle the `dir` attribute value between `ltr` and `rtl`.

### Examples

#### Setting document-level direction

Set `dir="rtl"` on the `<html>` element when the page is predominantly written in a right-to-left script, such as Arabic or Hebrew. Use `dir` on a block within the page when that block needs a different base direction.

In this Arabic document, the English paragraph needs both `dir="ltr"` and `lang="en"`.

```
<!doctype html>
<html dir="rtl" lang="ar">
  <head>
    <meta charset="utf-8" />
    <title>صفحة عربية</title>
  </head>
  <body>
    <p>محتوى الصفحة باللغة العربية.</p>
    <p dir="ltr" lang="en">This paragraph is in English.</p>
  </body>
</html>
```

#### Setting text direction explicitly

This example explicitly sets the base direction of two paragraphs. The base direction affects the default alignment and the placement of punctuation.

```
<p dir="ltr" lang="en">This sentence is in English and reads left to right.</p>
<p dir="rtl" lang="ar">
  هذه الجملة باللغة العربية وتُقرأ من اليمين إلى اليسار.
</p>
```

#### Inline bidirectional text

When an inline phrase has a different base direction from the surrounding text, tightly wrap the whole phrase in an element with the appropriate `dir` value. This also isolates the phrase's direction from its surroundings, so punctuation and numbers outside it are not treated as part of the phrase. Use an existing semantic element, such as `<cite>` for a book title, or a `<bdi>` if no other element is appropriate.

```
<p dir="rtl" lang="ar">
  اقرأ <cite dir="ltr" lang="en">How the Grinch Stole Christmas!</cite> اليوم.
</p>
```

Note that the exclamation mark, a part of the title, is inside the `<cite>` element. If outside, it will adopt the base direction and therefore appear on the left edge of the title.

### Using dir="auto" for user-generated content

When the direction of text is not known in advance, such as for user comments, use `dir="auto"`. The browser uses the first strongly directional character to determine the element's base direction. This is a heuristic, not language detection: a comment that starts with an English name and continues in Arabic will get a left-to-right base direction. Use an explicit direction when it is known.

Here, the paragraphs represent two possible comments. The comments are written by users, so the site is not aware of their languages during rendering.

```
<p dir="auto">This comment is in English.</p>
<p dir="auto">هذا التعليق باللغة العربية.</p>
```

The `<bdi>` element offers the same effect, and is terser for inline isolation, especially when no semantic element is appropriate.

### See also

- All global attributes.
- `HTMLElement.dir` that reflects this attribute.
- Handling different text directions
- Creating HTML pages in Arabic, Hebrew and other right-to-left scripts on w3.org

## draggable HTML global attribute

The **`draggable`** global attribute is an enumerated attribute that indicates whether the element can be dragged, either with native browser behavior or the HTML Drag and Drop API.

The `draggable` attribute may be applied to elements that strictly fall under the HTML namespace, which means that it cannot be applied to SVGs. For more information about what namespace declarations look like, and what they do, see Namespace crash course.

`draggable` can have the following values:

- `true`: the element can be dragged.
- `false`: the element cannot be dragged.

> **Warning:** This attribute is *enumerated* and not *Boolean*. A value of `true` or `false` is mandatory, and shorthand like `<img draggable>` is forbidden. The correct usage is `<img draggable="true">`.

If this attribute is not set, its default value is `auto`, which means drag behavior is the default browser behavior: only text selections, images, and links can be dragged. For other elements, the event `ondragstart` must be set for drag and drop to work, as shown in this comprehensive example.

### See also

- All global attributes.

## enterkeyhint HTML global attribute

The **`enterkeyhint`** global attribute is an enumerated attribute defining what action label (or icon) to present for the enter key on virtual keyboards.

### Try it

```
<input enterkeyhint="go" />

<p contenteditable enterkeyhint="go">https://example.org</p>
```

### Description

Form controls (such as `<textarea>` or `<input>` elements) or elements using `contenteditable` can specify an `inputmode` attribute to control what kind of virtual keyboard will be used. To further improve the user's experience, the enter key can be customized specifically by providing an `enterkeyhint`

attribute indicating how the enter key should be labeled (or which icon should be shown). The enter key usually represents what the user should do next; typical actions are: sending text, inserting a new line, or searching.

If no `enterkeyhint` attribute is provided, the user agent might use contextual information from the `inputmode`, `type`, or `pattern` attributes to display a suitable enter key label (or icon).

### Value

The `enterkeyhint` attribute is an enumerated attribute and only accepts the following values:

| Value | Description | Example label (depends on user agent and user language) |
|---|---|---|
| `enterkeyhint="enter"` | Typically inserting a new line. | `return`, ↵ |
| `enterkeyhint="done"` | Typically meaning there is nothing more to input and the input method editor (IME) will be closed. | `done`, ✅ |
| `enterkeyhint="go"` | Typically meaning to take the user to the target of the text they typed. | `go`, → |
| `enterkeyhint="next"` | Typically taking the user to the next field that will accept text. | `next`, ⇥ |
| `enterkeyhint="previous"` | Typically taking the user to the previous field that will accept text. | `return`, ⇤ |
| `enterkeyhint="search"` | Typically taking the user to the results of searching for the text they have typed. | `search`, 🔍 |
| `enterkeyhint="send"` | Typically delivering the text to its target. | `send` |

### See also

- `HTMLElement.enterKeyHint` property reflecting this attribute
- `inputmode` global attribute
- `contenteditable` global attribute
- `type` and `pattern` attributes on `<input>` elements

## exportparts HTML global attribute

The **`exportparts`** global attribute allows you to select and style elements existing in nested shadow trees, by exporting their `part` names.

The shadow tree is an isolated structure where identifiers, classes, and styles cannot be reached by selectors or queries belonging to a regular DOM. There are two HTML attributes that can be applied to shadow tree elements that enable targeting CSS styles from outside to shadow tree: `part` and `exportparts`.

The global `part` attribute makes a shadow tree element visible to its parent DOM. A `part` name is used as the parameter of the `::part()` pseudo-element. In this way, you can apply CSS styles to elements in the shadow tree from outside of it. However, the `::part()` pseudo-element is only visible to the parent DOM. This means that when a shadow tree is nested, the parts are not visible to any ancestors other than the direct parent. The `exportparts` attribute solves this limitation.

The `exportparts` attribute enables shadow tree parts to be visible outside the shadow DOM. This concept is referred to as "exporting". The `exportparts` attribute is placed on the element's *shadow host*, which is the element to which the *shadow tree* is attached. The value of this attribute is a comma-separated list of `part` names present in the shadow tree. These names are made available to the DOMs outside the current structure.

```
<template id="ancestor-component">
  <nested-component exportparts="part1, part2, part5"></nested-component>
</template>
```

When exporting a `part`, you have the option to assign a different name to the part, as shown in the snippet below. The value of the `exportparts` attribute is really a comma-separated list of part-name mappings. So the `exportparts` attribute in the above code snippet is the equivalent of `exportparts="part1:part1, part2:part2, part5:part5`, indicating that each `part` is exported with the same name. In each mapping, the first string specifies the name of the part within the shadow tree, and the second string specifies the name with which the part will be exposed externally.

```
<template id="ancestor-component">
  <nested-component
    exportparts="part1:exposed1, part2:exposed2"></nested-component>
</template>
```

### Examples

#### Basic component

To demonstrate how `exportparts` is used to enable targeting parts within nested components, we will create a component, and then nest it within another component.

### HTML

First, let's create a card component that we will then wrap with another component. We also use the new element we created, populating the slots with plain text as content.

```
<template id="card-component-template">
  <style>
    :host {
      display: block;
    }
  </style>
  <div class="base" part="base">
    <div part="header"><slot name="header_slot"></slot></div>
    <div part="body"><slot name="body_slot"></slot></div>
    <div part="footer"><slot name="footer_slot"></slot></div>
  </div>
</template>

<card-component>
  <p slot="header_slot">This is the header</p>
  <p slot="body_slot">This is the body</p>
  <p slot="footer_slot">This is the footer</p>
</card-component>
```

### JavaScript

We use JavaScript to define our web component defined in the HTML above:

```
customElements.define(
  "card-component",
  class extends HTMLElement {
    constructor() {
      super(); // Always call super first in constructor
      const template = document.getElementById("card-component-template");
      const shadowRoot = this.attachShadow({
        mode: "open",
      });
      shadowRoot.appendChild(document.importNode(template.content, true));
    }
  },
);
```

### CSS

We style parts of the `<card-component>` shadow tree using the `::part` pseudo-element:

```
::part(body) {
  color: red;
  font-style: italic;
}
```

### Results

### Nested component

Continuing the above `<card-component>` example, we create a nested component by wrapping the `<card-component>` within another component; in this case, the `<card-wrapper>` component. We then export the parts from the nested component that we want to make stylable from outside the component's shadow tree with the `exportparts` attribute.

### HTML

```
<template id="card-wrapper">
  <style>
    :host {
      display: block;
    }
  </style>
  <card-component exportparts="base, header, body">
    <slot name="H" slot="header_slot"></slot>
    <slot name="B" slot="body_slot"></slot>
    <slot name="F" slot="footer_slot"></slot>
  </card-component>
</template>
```

We include a `<card-wrapper>` custom element, and a `<card-component>` for comparison:

```
<h2>Card wrapper</h2>

<card-wrapper>
  <p slot="H">This is the header</p>
  <p slot="B">This is the body</p>
  <p slot="F">This is the footer</p>
</card-wrapper>

<h2>Card component</h2>

<card-component>
  <p slot="header_slot">This is the header</p>
  <p slot="body_slot">This is the body</p>
  <p slot="footer_slot">This is the footer</p>
</card-component>
```

### JavaScript

```
customElements.define(
  "card-wrapper",
  class extends HTMLElement {
    constructor() {
      super(); // Always call super first in constructor
      const template = document.getElementById("card-wrapper");
      const shadowRoot = this.attachShadow({
        mode: "open",
      });
      shadowRoot.appendChild(document.importNode(template.content, true));
    }
  },
);
```

### CSS

Now, we can target parts of the `<card-component>` directly and when nested within a `<card-wrapper>` like so:

```
h2 {
  background-color: #dedede;
}

card-wrapper,
card-component {
  border: 1px dashed blue;
  width: fit-content;
}

::part(body) {
  color: red;
  font-style: italic;
}

::part(header),
::part(footer) {
  font-weight: bold;
}
```

### Results

Note `footer` is not bold when nested, as we did not include it in `exportparts`.

### Exposing mapped parts

To rename exported parts, we include a comma-separated list of mapped parts, with each mapped part including the original name and exported name separated by a colon (`:`):

### HTML

We update the prior `<card-wrapper>` custom element with the remapping syntax (omitting `body` from the exported parts list):

```
<template id="card-wrapper">
  <card-component
    exportparts="
       base:card__base,
```

```
      header:card__header,
      footer:card__footer
    ">
    <span slot="header_slot"><slot name="H"></slot></span>
    <span slot="body_slot"><slot name="B"></slot></span>
    <span slot="footer_slot"><slot name="F"></slot></span>
  </card-component>
</template>
```

#### JavaScript

```
customElements.define(
  "card-wrapper",
  class extends HTMLElement {
    constructor() {
      super(); // Always call super first in constructor
      const template = document.getElementById("card-wrapper");
      const shadowRoot = this.attachShadow({
        mode: "open",
      });
      shadowRoot.appendChild(document.importNode(template.content, true));
    }
  },
);
```

#### CSS

In targeting the parts of the `<card-component>` from within the `<card-wrapper>`, we can only style the exported parts via their exposed part names:

```
/* selects the exported parts name */
::part(card__header) {
  font-weight: bold;
}
/* selects nothing: these part names were not exported */
::part(footer),
::part(body) {
  font-weight: bold;
}
```

#### Results

#### See also

- `part` HTML attribute
- `<template>` and `<slot>` HTML elements
- `::part` and `::slotted` pseudo-elements
- `:host` pseudo-class
- `ShadowRoot` interface
- `Element.part` property
- Using templates and slots
- CSS scoping module

### headingoffset HTML global attribute

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`headingoffset`** global attribute increases the computed heading level of the heading elements inside the element it is set on, without changing the elements used to write them.

#### Values

A valid non-negative integer between `0` and `8`, inclusive. A value that cannot be parsed as a non-negative integer is treated as an offset of `0`.

#### Description

Each heading element has a **computed heading level**, which is the level assistive technologies expose to users. Without `headingoffset`, that level is the number in the element's name: `1` for `<h1>`, `2` for `<h2>`, and so on.

The `headingoffset` attribute adds to that number. To find the offset for a heading, the browser starts at the heading element itself, walks up through its ancestors – crossing shadow boundaries into the shadow host – and adds up every `headingoffset` value it finds. The offsets accumulate: an `<h1>` inside an element with `headingoffset="1"` that is itself inside an element with `headingoffset="2"` has a computed heading level of 4.

Because the walk starts at the heading element, an offset on the heading itself also counts: `<h1 headingoffset="2">` has a computed heading level of 3.

The walk stops at the first element that has the `headingreset` attribute, after adding that element's own `headingoffset`.

The computed heading level never exceeds `9`, even in cases where offsets add up to more. Because HTML has no heading element above `<h6>`, heading levels of 7, 8, and 9 can be produced only by the `headingoffset` attribute or by the `aria-level` attribute.

This attribute affects only the computed heading level. It does not change the following:

- The element's name, so CSS selectors such as `h1` still match, and the default styling of the heading is unchanged.
- The element's ARIA role, which stays `heading`.
- An explicit `aria-level` attribute on the heading, which takes precedence over the computed heading level.

Because the offset comes from the heading and its ancestors, a reusable component can always use the same heading markup, such as an `<h1>` for its title. You can then use the same markup at any depth of a page without editing its headings. This also avoids the accessibility problem caused by picking a heading element for its font size.

### Accessibility

Screen reader users navigate by headings and rely on heading levels to understand how a page is structured. Use `headingoffset` to make those levels match the visual structure of the page, and check the result with a screen reader or the browser's accessibility inspector.

In browsers that do not support this attribute, headings keep the level of their element name, so the markup must still make sense without the offset.

### Examples

#### Offsetting headings in a component

In this example, the markup uses the same component structure twice – an `<article>` with an `<h1>` title. The second one is nested in a `<section>` that offsets its headings by one level.

```
<h1>Insect guide</h1>

<article>
  <h1>Beetles</h1>
  <p>A beetle has a hardened forewing.</p>
</article>

<section headingoffset="1">
  <h1>Appendix</h1>
  <article>
    <h1>Beetles, revisited</h1>
    <p>The same component, one level deeper.</p>
  </article>
</section>
```

The computed heading levels are:

- `Insect guide`: level 1
- `Beetles`: level 1
- `Appendix`: level 2
- `Beetles, revisited`: level 2

#### Accumulating offsets

Offsets from nested elements are added together, so this `<h2>` has a computed heading level of 5:

```
<article headingoffset="1">
  <section headingoffset="2">
    <h2>Level 5</h2>
  </section>
</article>
```

#### Stopping the offset

An element with the `headingreset` attribute stops the offsets of its ancestors from applying to its descendants. This is useful for content that is not part of the surrounding document structure, such as a dialog:

```
<section headingoffset="2">
  <h1>Level 3</h1>

  <dialog headingreset>
    <h1>Level 1</h1>
```

```
  </dialog>
</section>
```

### See also

- `headingreset` global attribute
- `<h1>–<h6>` elements
- `aria-level` attribute
- ARIA: heading role

## headingreset HTML global attribute

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`headingreset`** global attribute stops the `headingoffset` values of ancestor elements from applying to the heading elements inside the element it is set on.

### Values

This is a boolean attribute; it takes effect when it is present. A value for it is ignored, so `headingreset`, `headingreset=""`, and `headingreset="false"` all have the same effect.

### Description

When a browser computes the heading level of a heading element, it walks up the ancestor chain and adds up the `headingoffset` values it finds. The `headingreset` attribute stops that walk: offsets set on the element itself still count, but no ancestor above it contributes an offset.

Use this attribute for content that is not part of the structure of the surrounding document, such as a `<dialog>`, a popover, or markup inserted from another source. Their headings then take the level of their element name.

If the element carries both `headingoffset` and `headingreset`, its own offset applies and everything above it is discarded.

### Examples

#### Resetting heading levels in a dialog

In this example, the `<section>` offsets its headings by `2`, so the heading directly inside it is at level 3. The `<dialog>` element has `headingreset`, so the section's offset never reaches the heading inside the dialog, which stays at level 1:

```
<section headingoffset="2">
  <h1>Level 3</h1>

  <dialog headingreset>
    <h1>Level 1</h1>
    <p>The offset of the section does not apply here.</p>
  </dialog>
</section>
```

#### Using <code>headingreset</code> and <code>headingoffset</code> on the same element

In this example, the outer offset of `3` never reaches the heading. Only the offset on the element with `headingreset` applies, so the heading is at level 2:

```
<div headingoffset="3">
  <div headingreset headingoffset="1">
    <h1>Level 2</h1>
  </div>
</div>
```

### See also

- `headingoffset` global attribute
- `<h1>–<h6>` elements
- `aria-level` attribute

## hidden HTML global attribute

The **`hidden`** global attribute is an enumerated attribute indicating that the browser should not render the contents of the element. For example, it can be used to hide elements of the page that can't be used until the login process has been completed.

### Try it

```
<p>
  This content should be read right now, as it is important. I am so glad you
  are able to find it!
</p>

<p hidden>
  This content is not relevant to this page right now, so should not be seen.
  Nothing to see here. Nada.
</p>
```

```
p {
  background: #ffe8d4;
  border: 1px solid #f69d3c;
  padding: 5px;
  border-radius: 5px;
}
```

### Description

The `hidden` attribute indicates that the content of an element should not be presented to the user. The attribute takes any one of the following values:

- the keyword `hidden`
- the keyword `until-found`
- an empty string or no value

Invalid `hidden` attribute values also place the element in the *hidden* state. Therefore, all the following elements are in the *hidden* state:

```
<span hidden>I'm hidden</span>
<span hidden="">I'm also hidden</span>
<span hidden="hidden">I'm hidden too!</span>
<span hidden="bananas">I'm equally as hidden!</span>
```

The keyword `until-found` sets the element to the *hidden until found* state:

```
<span hidden="until-found">I'm hidden until found</span>
```

### The hidden state

The *hidden* state indicates that the element is not currently relevant to the page, or that it is being used to declare content for reuse by other parts of the page and should not be directly presented to the user. The browser will not render elements that are in the *hidden* state.

Web browsers may implement the *hidden* state using `display: none`, in which case the element will not participate in page layout. Additionally, changing the value of the CSS `display` property on a hidden element will override the *hidden* state. For instance, elements styled `display: block` will be displayed despite the `hidden` attribute's presence.

### The hidden until found state

In the *hidden until found* state, the element is hidden but its content will be accessible to the browser's "Find in page" feature or to fragment navigation. When these features cause a scroll to an element in a *hidden until found* subtree, the browser will:

1. Fire a `beforematch` event on the hidden element
2. Remove the `hidden` attribute from the element
3. Scroll to the element

This lets you collapse a section of content while still allowing users to find it through search or navigation.

Browsers typically implement *hidden until found* using `content-visibility: hidden`. This means that, unlike elements in the *hidden* state, elements in the *hidden until-found* state generate boxes, and:

- they participate in page layout
- their margin, borders, padding, and background are rendered

Also, the element needs to be affected by layout containment in order to be revealed. If the element in the *hidden until found* state has a `display` value of `none`, `contents`, or `inline`, then the element will not be revealed by "Find in page" or fragment navigation.

### Usage notes

The `hidden` attribute must not be used to hide content just from one presentation. If something is marked hidden, it is hidden from all presentations, including, for instance, screen readers.

Hidden elements shouldn't be linked from visible elements unless using `hidden="until-found"`. For example, it would be incorrect to use the `href` attribute to link to a section with the `hidden` attribute. If the content is not applicable or relevant, it shouldn't be linked.

It is fine, however, to use the ARIA `aria-describedby` attribute to refer to hidden descriptions. While hiding the descriptions implies that they're not useful on their own, they can provide helpful context when referenced in this way.

Similarly, a canvas element with the `hidden` attribute could be used by a scripted graphics engine as an off-screen buffer, and a form control could refer to a hidden form element using its form attribute.

Finally, note that elements that are descendants of a hidden element are still active, which means that script elements can still execute, and form elements can still submit:

```
<div hidden>
  <script>
    console.warn("Boo! I'm hidden *and* running!");
  </script>
</div>
```

### Examples

#### Using the hidden attribute

In this example, we have three `<div>` elements. The first and the third are not hidden, while the second has a `hidden` attribute. Note that the hidden element has no generated box.

```
<div>I'm not hidden</div>
<div hidden>I'm hiding!</div>
<div>I'm not hidden, either</div>
```

#### Using the until-found value

In this example, we have three `<div>` elements. The first and the third are visible, while the second has the `hidden="until-found"` and `id="until-found-box"` attributes. The element with a `until-found-box` id has a dotted red border and a gray background.

We also have a link that targets the `"until-found-box"` fragment and JavaScript that listens for the `beforematch` event firing on that hidden element. The event handler changes the text content of the box to illustrate an action that can occur when the *hidden until found* state is about to be removed.

#### HTML

```
<a href="#until-found-box">Go to hidden content</a>

<div>I'm not hidden</div>
<div id="until-found-box" hidden="until-found">Hidden until found</div>
<div>I'm not hidden, either</div>
```

#### CSS

```
div {
  height: 40px;
  width: 300px;
  border: 5px dashed black;
  margin: 1rem 0;
  padding: 1rem;
  font-size: 2rem;
}

div#until-found-box {
  color: red;
  border: 5px dotted red;
  background-color: lightgray;
}
```

#### JavaScript

```
const untilFound = document.querySelector("#until-found-box");
untilFound.addEventListener(
  "beforematch",
  () => (untilFound.textContent = "I've been revealed!"),
);
```

#### Result

Clicking the "Go to hidden content" link navigates to the *hidden until found* element. The `beforematch` event fires, the text content is updated, and the element becomes visible. Note that although the content of the element is hidden, the element still has a generated box, occupying space in the layout and with background and borders rendered.

To run the example again, click "Reset".

### See also

- `HTMLElement.hidden`
- All global attributes
- The `aria-hidden` attribute
- The `beforematch` event

## id HTML global attribute

The **`id`** global attribute defines an identifier (ID) that must be unique within the entire document.

### Try it

```
<p>A normal, boring paragraph. Try not to fall asleep.</p>

<p id="exciting">The most exciting paragraph on the page. One of a kind!</p>
```

```
#exciting {
  background: linear-gradient(to bottom, #ffe8d4, #f69d3c);
  border: 1px solid dimgrey;
  padding: 10px;
  border-radius: 10px;
  box-shadow: 2px 2px 1px black;
}

#exciting::before {
  content: "ℹ️";
  margin-right: 5px;
}
```

### Syntax

An ID attribute's value must not contain ASCII whitespace characters. Browsers treat non-conforming IDs that contain whitespace as if the whitespace is part of the ID. In contrast to the `class` attribute, which allows space-separated values, elements can only have one single ID value.

Technically, the value for an ID attribute may contain any other Unicode character. However, when used in CSS selectors, either from JavaScript using APIs like `Document.querySelector()` or in CSS stylesheets, ID attribute values must be valid CSS identifiers. This means that if an ID attribute value is not a valid CSS identifier (for example, `my?id` or `1234`) then it must be escaped before being used in a selector, either using the `CSS.escape()` method or manually.

For this reason, it's recommended that developers choose values for ID attributes that are valid CSS identifiers that don't require escaping.

Also, not all valid ID attribute values are valid JavaScript identifiers. For example, `1234` is a valid attribute value but not a valid JavaScript identifier. This means that the value is not a valid variable name, so you can't access the element using code like `window.1234`. However, you can access it using `window["1234"]`.

### Description

The purpose of the ID attribute is to identify a single element when linking (using a fragment identifier), scripting, or styling (with CSS).

You can access elements with ID attributes as global properties of the `window` object, where the property name is the ID value, and the property value is the corresponding element. For example, given this markup:

```
<p id="preamble"></p>
```

You can access this paragraph element in JavaScript using the following code:

```
const content = window.preamble.textContent;
```

> **Warning:** Relying on the `window["id-value"]` or `window.idValue` pattern is dangerous and discouraged because it can lead to unexpected conflicts with existing or future APIs in the browser. For example, if a browser introduces a built-in global property named `preamble` in the future, your code may no longer be able to access the HTML element. To avoid such conflicts, always use the `Document.getElementById()` or `Document.querySelector()` method to access elements by ID.

### See also

- All global attributes.
- `Element.id` that reflects this attribute.

- The `Document.getElementById` method.
- CSS ID selectors.

## inert HTML global attribute

The **`inert`** global attribute is a Boolean attribute indicating that the element and all of its flat tree descendants become *inert*. The `inert` attribute can be added to sections of content that should not be interactive. When an element is inert, it along with all of the element's descendants, including normally interactive elements such as links, buttons, and form controls are disabled because they cannot receive focus or be clicked. The `inert` attribute can also be added to elements that should be offscreen or hidden. An inert element, along with its descendants, gets removed from the tab order and accessibility tree.

Modal `<dialog>`s generated with `showModal()` escape inertness, meaning that they don't inherit inertness from their ancestors, but can be made inert by having the `inert` attribute explicitly set on themselves. No other element can escape inertness.

> **Note:** While `inert` is a global attribute and can be applied to any element, it is generally used for sections of content. To make individual controls "inert", consider using the `disabled` attribute, along with CSS `:disabled` styles, instead.

Inert HTML elements and their flat tree descendants:

- Do not have `click` events fired when clicked on.
- Cannot be focused and `focus` events cannot be fired on them.
- Are not searchable via browser find-in-page features (none of their content is found/matched).
- Disallow users from selecting text contained within their content — akin to using the CSS property `user-select` to disable text selection.
- Cannot have otherwise-editable content edited. This includes, for example, the contents of textual `<input>` fields, and text elements with `contenteditable` set on them.
- Are hidden from assistive technologies as they are excluded from the accessibility tree.

The following other features can be used to set an element and its descendants to an inert state:

- The CSS `interactivity` property.
- The `HTMLElement.inert` DOM property.

### Accessibility concerns

Use careful consideration for accessibility when applying the `inert` attribute. By default, there is no visual way to tell whether or not an element or its subtree is inert. As a web developer, it is your responsibility to clearly indicate the content parts that are active and those that are inert.

While providing visual and non-visual cues about content inertness, also remember that the visual viewport may contain only sections of content. Users may be zoomed in to a small section of content, or users may not be able to view the content at all. Inert sections not being obviously inert can lead to frustration and bad user experience.

### Examples

In this example, the second `<div>` and all of its descendants are made inert via the `inert` attribute:

```
<div>
  <label for="button1">Button 1</label>
  <button id="button1">I am not inert</button>
</div>
<div inert>
  <label for="button2">Button 2</label>
  <button id="button2">I am inert</button>
</div>
```

### See also

- HTML `<dialog>` element
- `HTMLElement.inert` DOM property
- CSS `interactivity` property

## inputmode HTML global attribute

The **`inputmode`** global attribute is an enumerated attribute that hints at the type of data that might be entered by the user while editing the element or its contents. This allows a browser to display an appropriate virtual keyboard.

It is used primarily on `<input>` elements, but is usable on any element in `contenteditable` mode.

It's important to understand that the `inputmode` attribute doesn't cause any validity requirements to be enforced on input. To require that input conforms to a particular data type, choose an appropriate `<input>` element type. For specific guidance on choosing `<input>` types, see the Value section.

### Value

The attribute can have any of the following values:

`none`

No virtual keyboard. For when the page implements its own keyboard input control.

`text` (default value)

Standard input keyboard for the user's current locale.

`decimal`

Fractional numeric input keyboard containing the digits and decimal separator for the user's locale (typically `.` or `,`). Devices may or may not show a minus key (`-`).

`numeric`

Numeric input keyboard, but only requires the digits 0–9. Devices may or may not show a minus key.

`tel`

A telephone keypad input, including the digits 0–9, the asterisk (`*`), and the pound (`#`) key. Inputs that *require* a telephone number should typically use `<input type="tel">` instead.

`search`

A virtual keyboard optimized for search input. For instance, the return/submit key may be labeled "Search", along with possible other optimizations. Inputs that *require* a search query should typically use `<input type="search">` instead.

`email`

A virtual keyboard optimized for entering email addresses. Typically includes the `@`character as well as other optimizations. Inputs that *require* email addresses should typically use `<input type="email">` instead.

`url`

A keypad optimized for entering URLs. This may have the `/` key more prominent, for example. Enhanced features could include history access and so on. Inputs that *require* a URL should typically use `<input type="url">` instead.

### See also

- All global attributes.
- `enterkeyhint` global attribute

## is HTML global attribute

> **Note:** Safari does not plan to support customized built-in elements and browser vendors are exploring alternative solutions to customizing built-ins. Check the browser compatibility section for support information.

The **`is`** global attribute allows you to specify that a standard HTML element should behave like a defined customized built-in element (see Using custom elements for more details).

This attribute can only be used if the specified custom element name has been successfully defined in the current document, and extends the element type it is being applied to.

### Examples

The following code is taken from our word-count-web-component example (see it live also).

```
// Create a class for the element
class WordCount extends HTMLParagraphElement {
  constructor() {
    // Always call super first in constructor
    super();

    // Constructor contents omitted for brevity
    // …
  }
}

// Define the new element
customElements.define("word-count", WordCount, { extends: "p" });
```

```
<p is="word-count"></p>
```

### See also

- All global attributes.

## itemid HTML global attribute

The **`itemid`** global attribute provides microdata in the form of a unique, global identifier of an item.

An `itemid` attribute can only be specified for an element that has both `itemscope` and `itemtype` attributes. Also, `itemid` can only be specified on elements that possess an `itemscope` attribute whose corresponding `itemtype` refers to or defines a vocabulary that supports global identifiers.

The exact meaning of an `itemtype`'s global identifier is provided by the definition of that identifier within the specified vocabulary. The vocabulary defines whether several items with the same global identifier can coexist and, if so, how items with the same identifier are handled.

> **Note:** The WHATWG definition specifies that an `itemid` must be a URL. However, the following example correctly illustrates that a URN may also be used. This inconsistency may reflect the incomplete nature of the Microdata specification.

### Examples

#### Representing structured data for a book

This example uses microdata attributes to represent the following structured data:

| itemscope | itemtype: itemid | https://schema.org/Book: urn:isbn:0-374-22848-5 | |
|---|---|---|---|
| | itemprop | title | Owls of the Eastern Ice |
| | itemprop | author | Jonathan C Slaght |
| | itemprop | datePublished | 2020-08-04 |

#### HTML

```
<dl
  itemscope
  itemtype="https://schema.org/Book"
  itemid="urn:isbn:0-374-22848-5<">
  <dt>Title</dt>
  <dd itemprop="title">Owls of the Eastern Ice</dd>
  <dt>Author</dt>
  <dd itemprop="author">Jonathan C Slaght</dd>
  <dt>Publication date</dt>
  <dd>
    <time itemprop="datePublished" datetime="2020-08-04">August 4 2020</time>
  </dd>
</dl>
```

#### Result

### See also

- All global attributes.
- Other microdata related global attributes:
  - `itemprop`
  - `itemref`
  - `itemscope`
  - `itemtype`

## itemprop HTML global attribute

The **`itemprop`** global attribute is used to add properties to an item. Every HTML element can have an `itemprop` attribute specified, and an `itemprop` consists of a name-value pair. Each name-value pair is called a **property**, and a group of one or more properties forms an **item**. Property values are either a string or a URL and can be associated with a very wide range of elements including `<audio>`, `<embed>`, `<iframe>`, `<img>`, `<link>`, `<object>`, `<source>`, `<track>`, and `<video>`.

### Examples

The example below shows the source for a set of elements marked up with `itemprop` attributes, followed by a table showing the resulting structured data.

### HTML

```
<div itemscope itemtype="http://schema.org/Movie">
  <h1 itemprop="name">Avatar</h1>
  <span>
    Director:
    <span itemprop="director">James Cameron</span>
    (born August 16, 1954)
  </span>
  <span itemprop="genre">Science fiction</span>
  <a href="../movies/avatar-theatrical-trailer.html" itemprop="trailer">
    Trailer
  </a>
</div>
```

### Structured data

| | Item | |
|---|---|---|
| | **itemprop name** | **itemprop value** |
| itemprop | name | Avatar |
| itemprop | director | James Cameron |
| itemprop | genre | Science fiction |
| itemprop | trailer | ../movies/avatar-theatrical-trailer.html |

### Properties

Properties have values that are either a string or a URL. When a string value is a URL, it is expressed using the `<a>` element and its `href` attribute, the `<img>` element and its `src` attribute, or other elements that link to or embed external resources.

### Three properties with values that are strings

```
<div itemscope>
  <p>My name is <span itemprop="name">Neil</span>.</p>
  <p>My band is called <span itemprop="band">Four Parts Water</span>.</p>
  <p>I am <span itemprop="nationality">British</span>.</p>
</div>
```

### One property, "image", whose value is a URL

```
<div itemscope>
  <img itemprop="image" src="https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Global_attributes/itemprop/google-logo.png"
alt="Google" />
</div>
```

When a string value can't be easily read and understood by a person (e.g., a long string of numbers and letters), it can be displayed using the value attribute of the data element, with the more easily-understood-by-a human-version given in the element's contents (which is not part of the structured data - see example below).

### An item with a property whose value is a product ID

The ID is not human-friendly, so the product's name is used instead.

```
<h1 itemscope>
  <data itemprop="product-id" value="9678AOU879">The Instigator 2000</data>
</h1>
```

For numeric data, the meter element and its value attribute can be used.

### A meter element

```
<div itemscope itemtype="http://schema.org/Product">
  <span itemprop="name">Panasonic White 60L Refrigerator</span>
  <img src="https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Global_attributes/itemprop/panasonic-fridge-60l-white.jpg"
alt="" />
  <div
    itemprop="aggregateRating"
    itemscope
    itemtype="http://schema.org/AggregateRating">
    <meter itemprop="ratingValue" min="0" value="3.5" max="5">
      Rated 3.5/5
    </meter>
```

```
    (based on <span itemprop="reviewCount">11</span>
    customer reviews)
  </div>
</div>
```

Similarly, for date- and time-related data, the time element and its datetime attribute can be used.

### An item with one property, "birthday", whose value is a date

```
<div itemscope>
  I was born on
  <time itemprop="birthday" datetime="1984-05-10">May 10th 1984</time>.
</div>
```

Properties can also be groups of name-value pairs, by putting the itemscope attribute on the element that declares the property. Each value is either a string or a group of name-value pairs (i.e., an item).

### An outer item representing a person, and an inner one representing a band

```
<div itemscope>
  <p>Name: <span itemprop="name">Amanda</span></p>
  <p>
    Band:
    <span itemprop="band" itemscope>
      <span itemprop="name">Jazz Band</span>
      (<span itemprop="size">12</span> players)
    </span>
  </p>
</div>
```

The outer item above has two properties, "name" and "band". The "name" is "Amanda", and the "band" is an item in its own right, with two properties, "name" and "size". The "name" of the band is "Jazz Band", and the "size" is "12". The outer item in this example is a top-level microdata item. Items that are not part of others are called top-level microdata items.

### All the properties separated from their items

This example is the same as the previous one, but all the properties are separated from their items.

```
<div itemscope id="amanda" itemref="a b"></div>
<p id="a">Name: <span itemprop="name">Amanda</span></p>
<div id="b" itemprop="band" itemscope itemref="c"></div>
<div id="c">
  <p>Band: <span itemprop="name">Jazz Band</span></p>
  <p>Size: <span itemprop="size">12</span> players</p>
</div>
```

This gives the same result as the previous example. The first item has two properties, "name", set to "Amanda", and "band", set to another item. That second item has two further properties, "name", set to "Jazz Band", and "size", set to "12".

An item can have multiple properties with the same name and different values.

### Ice cream with two flavors

```
<div itemscope>
  <p>Flavors in my favorite ice cream:</p>
  <ul>
    <li itemprop="flavor">Lemon sorbet</li>
    <li itemprop="flavor">Apricot sorbet</li>
  </ul>
</div>
```

This results in an item with two properties, both with the name "flavor" and having the values "Lemon sorbet" and "Apricot sorbet".

An element introducing a property can also introduce multiple properties at once, to avoid duplication when some of the properties have the same value.

### An item with two properties, "favorite-color" and "favorite-fruit", both set to the value "orange"

```
<div itemscope>
  <span
    itemprop="favorite-color
    favorite-fruit"
    >orange
```

```
  </span>
</div>
```

> **Note:** There is no relationship between the microdata and the content of the document where the microdata is marked up.

### Same structured data marked up in two different ways

There is no semantic difference between the following two examples

```
<figure>
  <img src="https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Global_attributes/itemprop/castle.jpeg" />
  <figcaption>
    <span itemscope><span itemprop="name">The Castle</span></span> (1986)
  </figcaption>
</figure>
```

```
<span itemscope><meta itemprop="name" content="The Castle" /></span>
<figure>
  <img src="https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Global_attributes/itemprop/castle.jpeg" />
  <figcaption>The Castle (1986)</figcaption>
</figure>
```

Both have a figure with a caption, and both, completely unrelated to the figure, have an item with a name-value pair with the name "name" and the value "The Castle". The only difference is that if the user drags the figcaption out of the document, the item will be included in the drag-and-drop data. The image associated with the item won't be included.

### Names and values

A property is an unordered set of unique tokens that are case-sensitive and represent the name-value pairs. The property value must have at least one token. In the example below, each data cell is a token.

### Names examples

| | **Item** | |
|---|---|---|
| | **itemprop name** | **itemprop value** |
| **itemprop** | country | Ireland |
| **itemprop** | Option | 2 |
| **itemprop** | https://www.flickr.com/photos/nlireland/6992065114/ | Ring of Kerry |
| **itemprop** | img | https://www.flickr.com/photos/nlireland/6992065114/ |
| **itemprop** | website | flickr |
| **itemprop** | (token) | (token) |

**Tokens** are either strings or URL's. An item is called a **typed item** if it is a URL. Otherwise, it is a string. Strings cannot contain a period or a colon (see below).

1. If the item is a typed item it must be either:
    1. A defined property name, or
    2. A valid URL, which refers to the vocabulary definition, or
    3. A valid URL that is used as a proprietary item property name (i.e., one not defined in a public specification), or
2. If the item is not a typed item it must be:
    1. A string that contains no `.` (U+002E FULL STOP) characters and no `:` characters (U+003A COLON) and is used as a proprietary item property name (again, one not defined in a public specification).

> **Note:** The rules above disallow ":" characters in non-URL values because otherwise they could not be distinguished from URLs. Values with "." characters are reserved for future extensions. Space characters are disallowed because otherwise the values would be parsed as multiple tokens.

### Value

The property value of a name-value pair is as given for the first matching case in the following list:

- If the element has an `itemscope` attribute
    - The value is the **item** created by the element

- If the element is a `meta` element
    - The value is the value of the element's `content` attribute
- If the element is an `audio`, `embed`, `iframe`, `img`, `source`, `track`, or `video` element
    - The value is the resulting URL string that results from parsing the value of the element's src attribute relative to the node document (part of the Microdata DOM API) of the element at the time the attribute is set
- If the element is an `a`, `area`, or `link` element
    - The value is the resulting URL string that results from parsing the value of the element's href attribute relative to the node document of the element at the time the attribute is set
- If the element is an `object` element
    - The value is the resulting URL string that results from parsing the value of the element's data attribute relative to the node document of the element at the time the attribute is set
- If the element is a `data` element
    - The value is the value of the element's value attribute
- If the element is a `meter` element
    - The value is the value of the element's `value` attribute
- If the element is a `time` element
    - The value is the element's `datetime` value

Otherwise

- The value is the element's *textContent*.

If a property's value is a `URL`, the property must be specified using a URL property element. The URL property elements are the `a`, `area`, `audio`, `embed`, `iframe`, `img`, `link`, `object`, `source`, `track`, and `video` elements.

### Name order

Names are unordered relative to each other, but if a particular name has multiple values, they do have a relative order.

In the following example, the "a" property has the values "1" and "2", *in that order*, but whether the "a" property comes before the "b" property or not is not important.

```
<div itemscope>
  <p itemprop="a">1</p>
  <p itemprop="a">2</p>
  <p itemprop="b">test</p>
</div>
```

Here are several equivalent examples:

```
<div itemscope>
  <p itemprop="b">test</p>
  <p itemprop="a">1</p>
  <p itemprop="a">2</p>
</div>
```

```
<div itemscope>
  <p itemprop="a">1</p>
  <p itemprop="b">test</p>
  <p itemprop="a">2</p>
</div>
```

```
<div id="x">
  <p itemprop="a">1</p>
</div>
<div itemscope itemref="x">
  <p itemprop="b">test</p>
  <p itemprop="a">2</p>
</div>
```

### Representing structured data for a book

This example uses microdata attributes to represent the following structured data:

| itemscope | itemtype: itemid | https://schema.org/Book: urn:isbn:0-374-22848-5 | |
|---|---|---|---|
| | itemprop | title | Owls of the Eastern Ice |
| | itemprop | author | Jonathan C Slaght |
| | itemprop | datePublished | 2020-08-04 |

#### HTML

```
<dl
  itemscope
  itemtype="https://schema.org/Book"
  itemid="urn:isbn:0-374-22848-5<">
  <dt>Title</dt>
  <dd itemprop="title">Owls of the Eastern Ice</dd>
  <dt>Author</dt>
  <dd itemprop="author">Jonathan C Slaght</dd>
  <dt>Publication date</dt>
  <dd>
    <time itemprop="datePublished" datetime="2020-08-04">August 4 2020</time>
  </dd>
</dl>
```

#### Result

#### See also

- Other different global attributes
- Other microdata related global attributes:
  - `itemid`
  - `itemref`
  - `itemscope`
  - `itemtype`

### itemref HTML global attribute

Properties that are not descendants of an element with the `itemscope` attribute can be associated with an item using the global attribute **`itemref`**.

`itemref` provides a list of element IDs (not `itemids`) elsewhere in the document, with additional properties

The `itemref` attribute can only be specified on elements that have an `itemscope` attribute specified.

> **Note:** The `itemref` attribute is not part of the microdata data model. It is merely a syntactic construct to aid authors in adding annotations to pages where the data to be annotated does not follow a convenient tree structure. For example, it allows authors to mark up data in a table so that each column defines a separate item while keeping the properties in the cells.

#### Examples

#### Representing structured data for a band

This example uses microdata attributes to represent the following structured data (in JSON-LD format):

```
{
  "@id": "amanda",
  "name": "Amanda",
  "band": {
    "@id": "b",
    "name": "Jazz Band",
    "size": 12
  }
}
```

#### HTML

```
<div itemscope id="amanda" itemref="a b"></div>
<p id="a">Name: <span itemprop="name">Amanda</span></p>
<div id="b" itemprop="band" itemscope itemref="c"></div>
<div id="c">
  <p>Band: <span itemprop="name">Jazz Band</span></p>
```

```
  <p>Size: <span itemprop="size">12</span> players</p>
</div>
```

### Result

### See also

- Other different global attributes
- Other microdata related global attributes:
  - `itemid`
  - `itemprop`
  - `itemscope`
  - `itemtype`

## itemscope HTML global attribute

**`itemscope`** is a boolean global attribute that defines the scope of associated metadata. Specifying the **`itemscope`** attribute for an element creates a new item, which results in a number of name-value pairs that are associated with the element.

A related attribute, `itemtype`, is used to specify the valid URL of a vocabulary (such as schema.org) that describes the item and its properties context. In each of the following examples, the vocabulary is from schema.org.

Every HTML element may have an `itemscope` attribute specified. An `itemscope` element that does not have an associated `itemtype` must have an associated `itemref`.

> **Note:** Find more about `itemtype` attributes at https://schema.org/Thing

### Usage notes

#### itemscope id attributes

When you specify the `itemscope` attribute for an element, a new item is created. The item consists of a group of name-value pairs. For elements with an `itemscope` attribute and an `itemtype` attribute, you may also specify an `id` attribute. You can use the `id` attribute to set a global identifier for the new item. A global identifier allows the item to relate to other items found on pages across the Web.

### Examples

#### Representing structured data for a movie

The following example specifies the `itemtype` as "http://schema.org/Movie", and specifies four related `itemprop` attributes.

| itemscope | Itemtype | Movie | |
|---|---|---|---|
| | itemprop | (itemprop name) | (itemprop value) |
| | itemprop | director | James Cameron |
| | itemprop | genre | Science Fiction |
| | itemprop | name | Avatar |
| | itemprop | Trailer | https://youtu.be/0AY1XIkX7bY |

```
<div itemscope itemtype="https://schema.org/Movie">
  <h1 itemprop="name">Avatar</h1>
  <span>
    Director: <span itemprop="director">James Cameron</span> (born August 16,
    1954)
  </span>
  <span itemprop="genre">Science fiction</span>
  <a href="https://youtu.be/0AY1XIkX7bY" itemprop="trailer">Trailer</a>
</div>
```

#### Representing structured data for a recipe

There are four `itemscope` attributes in the following example. Each `itemscope` attribute sets the scope of its corresponding `itemtype` attribute. The `itemtypes`, `Recipe`, `AggregateRating`, and `NutritionInformation` in the following example are part of the schema.org structured data for a recipe, as specified by the first `itemtype`, `http://schema.org/Recipe`.

| itemscope | itemtype | Recipe | |
|---|---|---|---|
| | itemprop | name | Grandma's Holiday Apple Pie |
| | itemprop | image | https://c1.staticflickr.com/1/30/42759561_8631e2f905_n.jpg |

| | | | |
|---|---|---|---|
| | itemprop | datePublished | 2022-11-05 |
| | itemprop | description | This is my grandmother's apple pie recipe. I like to add a dash of nutmeg. |
| | itemprop | prepTime | PT30M |
| | itemprop | cookTime | PT1H |
| | itemprop | totalTime | PT1H30M |
| | itemprop | recipeYield | 1 9" pie (8 servings) |
| | itemprop | recipeIngredient | Thinly-sliced apples: 6 cups |
| | itemprop | recipeIngredient | White sugar: 3/4 cup |
| | itemprop | recipeInstructions | 1. Cut and peel apples 2. Mix sugar and cinnamon. Use additional sugar for tart apples. |
| | itemprop | author [Person] | |
| | itemprop | name | Carol Smith |
| itemscope | itemprop[itemtype] | aggregateRating [AggregateRating] | |
| | itemprop | ratingValue | 4.0 |
| | itemprop | reviewCount | 35 |
| itemscope | itemprop[itemtype] | nutrition [NutritionInformation] | |
| | itemprop | servingSize | 1 medium slice |
| | itemprop | calories | 250 cal |
| | itemprop | fatContent | 12 g |

> **Note:** A handy tool for extracting microdata structures from HTML is Google's Rich Results Testing Tool. Try it on the HTML shown here.

HTML

```
<div itemscope itemtype="https://schema.org/Recipe">
  <h2 itemprop="name">Grandma's Holiday Apple Pie</h2>
  <img
    itemprop="image"
    src="https://c1.staticflickr.com/1/30/42759561_8631e2f905_n.jpg"
    width="50"
    height="50" />
  <p>
    By
    <span itemprop="author" itemscope itemtype="https://schema.org/Person">
      <span itemprop="name">Carol Smith</span>
    </span>
  </p>
  <p>
    Published:
    <time datetime="2022-11-05" itemprop="datePublished">
      November 5, 20022
    </time>
  </p>
  <span itemprop="description">
    This is my grandmother's apple pie recipe. I like to add a dash of nutmeg.
  </span>
  <br />
  <span
    itemprop="aggregateRating"
    itemscope
    itemtype="https://schema.org/AggregateRating">
    <span itemprop="ratingValue">4.0</span> stars based on
    <span itemprop="reviewCount">35</span> reviews
  </span>
  <br />
  Prep time: <time datetime="PT30M" itemprop="prepTime">30 min</time>
  <br />
  Cook time: <time datetime="PT1H" itemprop="cookTime">1 hour</time>
  <br />
  Total time: <time datetime="PT1H30M" itemprop="totalTime">1 hour 30 min</time>
  <br />
  Yield: <span itemprop="recipeYield">1 9" pie (8 servings)</span>
  <br />
  <span
    itemprop="nutrition"
```

```
    itemscope
    itemtype="https://schema.org/NutritionInformation">
    Serving size: <span itemprop="servingSize">1 medium slice</span><br />
    Calories per serving: <span itemprop="calories">250 cal</span><br />
    Fat per serving: <span itemprop="fatContent">12 g</span><br />
  </span>
  <p>
    Ingredients:<br />
    <span itemprop="recipeIngredient">Thinly-sliced apples: 6 cups<br /></span>
    <span itemprop="recipeIngredient">White sugar: 3/4 cup<br /></span>
    …
  </p>
  Directions: <br />
  <div itemprop="recipeInstructions">
    1. Cut and peel apples<br />
    2. Mix sugar and cinnamon. Use additional sugar for tart apples. <br />
    …
  </div>
</div>
```

### Result

### See also

- Other different global attributes
- Other microdata related global attributes:
  - `itemid`
  - `itemprop`
  - `itemref`
  - `itemtype`

## itemtype HTML global attribute

The global attribute **`itemtype`** specifies the URL of the vocabulary that will be used to define `itemprop`'s (item properties) in the data structure.

`itemscope` is used to set the scope of where in the data structure the vocabulary set by `itemtype` will be active.

Google and other major search engines support the schema.org vocabulary for structured data. This vocabulary defines a standard set of type names and property names. For example, `MusicEvent` indicates a concert performance, with `startDate` and `location` properties specifying the concert's key details. In this case, `MusicEvent` would be the URL used by `itemtype`, with `startDate` and location as `itemprop`'s which `MusicEvent` defines.

> **Note:** More about `itemtype` attributes can be found at https://schema.org/Thing

- The **itemtype** attribute must have a value that is an unordered set of unique tokens which are case-sensitive, each is a valid and absolute URL, and all defined to use the same vocabulary. The attribute's value must have at least one token.
- The item types must all be types defined in applicable specifications (such as schema.org), and must all be defined to use the same vocabulary.
- The itemtype attribute can only be specified on elements which have an itemscope attribute specified.
- The itemid attribute can only be specified on elements which have both an itemscope attribute and an itemtype attribute specified. They must only be specified on elements with an itemscope attribute, whose itemtype attribute specifies a vocabulary not supporting global identifiers for items, as defined by that vocabulary's specification.
- The exact meaning of a global identifier is determined by the vocabulary's specification. It is left to such specifications to define whether multiple items of the same global identifier (whether on the same page or different pages) are allowed to exist, and what processing rules for that vocabulary are, with respect to handling the case of multiple items with the same ID.

### Examples

#### Representing structured data for a product

This example uses microdata attributes to represent structured data for a product, as follows:

| itemscope | itemtype | Product (https://schema.org/Product) | |
|---|---|---|---|
| | itemprop | name | Executive Anvil |
| | itemprop | image | https://pixabay.com/static/uploads/photo/2015/09/05/18/15/suitcase-924605_960_720.png |
| | itemprop | description | Sleeker than ACME's Classic Anvil, the Executive Anvil is perfect for the business traveler looking for something to drop from a height. |
| | itemprop | mpn | 925872 |
| | itemprop | brand [Thing] | |

| | | | |
|---|---|---|---|
| | itemprop | name | ACME |
| itemscope | itemprop[itemtype] | aggregateRating[AggregateRating] | |
| | itemprop | ratingValue | 4.4 |
| | itemprop | reviewCount | 89 |
| | itemprop | offers [Offer] | https://schema.org/Offer |
| | itemprop | priceCurrency | USD |
| | itemprop | price | 119.99 |
| | itemprop | priceValidUntil | 2020-11-05 |
| | itemprop | itemCondition | https://schema.org/UsedCondition |
| | itemprop | availability | https://schema.org/InStock |
| itemscope | itemprop[itemtype] | seller [Organization] | https://schema.org/Organization |
| | itemprop | name | Executive Objects |

> **Note:** A handy tool for extracting microdata structures from HTML is Google's Structured Data Testing Tool. Try it on the HTML shown here.

### HTML

```
<div itemscope itemtype="https://schema.org/Product">
  <span itemprop="brand">ACME<br /></span>
  <span itemprop="name">Executive Anvil<br /></span>
  <img
    itemprop="image"
    src="https://pixabay.com/static/uploads/photo/2015/09/05/18/15/suitcase-924605_960_720.png"
    width="50"
    height="50"
    alt="Executive Anvil logo" /><br />

  <span itemprop="description">
    Sleeker than ACME's Classic Anvil, the Executive Anvil is perfect for the
    business traveler looking for something to drop from a height.
    <br />
  </span>

  Product #: <span itemprop="mpn">925872<br /></span>
  <span
    itemprop="aggregateRating"
    itemscope
    itemtype="https://schema.org/AggregateRating">
    Rating: <span itemprop="ratingValue">4.4</span> stars, based on
    <span itemprop="reviewCount">89 </span> reviews
  </span>
  <p>
    <span itemprop="offers" itemscope itemtype="https://schema.org/Offer">
      Regular price: $179.99<br />
      <meta itemprop="priceCurrency" content="USD" />
      <span itemprop="price">Sale price: $119.99<br /></span>
      (Sale ends
      <time itemprop="priceValidUntil" datetime="2020-11-05">5 November!</time>)
      <br />
      Available from:
      <span
        itemprop="seller"
        itemscope
        itemtype="https://schema.org/Organization">
        <span itemprop="name">Executive Objects<br /></span>
      </span>
      Condition:
      <link
        itemprop="itemCondition"
        href="https://schema.org/UsedCondition" />Previously owned, in excellent
      condition<br />
      <link itemprop="availability" href="https://schema.org/InStock" />In
      stock! Order now!
    </span>
  </p>
</div>
```

### Result

### See also

- Other different global attributes
- Other microdata related global attributes:
  - `itemid`
  - `itemprop`
  - `itemref`
  - `itemscope`

## lang HTML global attribute

The **`lang`** global attribute helps define the language of an element: the language that non-editable elements are written in, or the language that the editable elements should be written in by the user. The attribute contains a single BCP 47 language tag.

> **Note:** The language is in an unknown state if there's no indication of language anywhere. It is recommended to always specify an appropriate value for this attribute, especially due to accessibility concerns.

### Try it

```
<p>This paragraph is English, but the language is not specifically defined.</p>

<p lang="en-GB">This paragraph is defined as British English.</p>

<p lang="fr">Ce paragraphe est défini en français.</p>
```

```
p::before {
  padding-right: 5px;
}

[lang="en-GB"]::before {
  content: "(In British English) ";
}

[lang="fr"]::before {
  content: "(In French) ";
}
```

### Values

The attribute contains a single BCP 47 language tag. If the attribute value is the *empty string* (`lang=""`), the language is set to *unknown*; if the language tag is not valid according to BCP47, it is set to *invalid*.

If the `xml:lang` attribute is also specified, the HTML spec requires the `lang` and `xml:lang` attributes' values to be equal case-insensitively. The `xml:lang` attribute takes priority.

For the CSS pseudo-class `:lang`, two invalid language names are different if their names are different. So while `:lang(es)` matches both `lang="es-ES"` and `lang="es-419"`, `:lang(xyzzy)` would *not* match `lang="xyzzy-Zorp!"`.

### Inheritance

If an element has no `lang` or `xml:lang` attribute, it inherits the language of its parent element, or the `host` if the parent node is a `ShadowRoot`. If no ancestor sets a language, the language can also be specified by `<meta http-equiv="content-language">` or the HTTP `Content-Language` header. If no single content language can be determined from these cues, the default is the empty string (with *unknown* as the content language).

### Accessibility concerns

WCAG Success Criterion 3.1.1 **requires** that a page language is specified in a way which may be 'programmatically determined' (i.e., via the **`lang`** attribute).

WCAG Success Criterion 3.1.2 requires that pages with **parts** in different languages have the languages of those parts specified too. Again, the **`lang`** attribute is the correct mechanism for this.

The purpose of these requirements is primarily to allow assistive technologies such as screen readers to invoke the correct pronunciation.

For example, the language menu on this site (MDN) includes a **`lang`** attribute for each entry:

```
<div class="dropdown-container language-menu">
  <button
    id="header-language-menu"
    type="button"
    class="dropdown-menu-label"
    aria-haspopup="true"
```

```
    aria-owns="language-menu"
    aria-label="Current language is English. Choose your preferred language.">
    English
    <span class="dropdown-arrow-down" aria-hidden="true">▼</span>
  </button>
  <ul
    id="language-menu"
    class="dropdown-menu-items right show"
    aria-expanded="true"
    role="menu">
    <li role="menuitem">
      <a
        href="https://developer.mozilla.org/ca/docs/Web/HTML/Reference/Global_attributes/lang"
        title="Catalan">
        <bdi lang="ca">Català</bdi>
      </a>
    </li>
    <li role="menuitem">
      <a
        href="https://developer.mozilla.org/de/docs/Web/HTML/Reference/Global_attributes/lang"
        title="German">
        <bdi lang="de">Deutsch</bdi>
      </a>
    </li>
    <li role="menuitem">
      <a
        href="https://developer.mozilla.org/es/docs/Web/HTML/Reference/Global_attributes/lang"
        title="Spanish">
        <bdi lang="es">Español</bdi>
      </a>
    </li>
    <li role="menuitem">
      <a
        href="https://developer.mozilla.org/fr/docs/Web/HTML/Reference/Global_attributes/lang"
        title="French">
        <bdi lang="fr">Français</bdi>
      </a>
    </li>
    <li role="menuitem">
      <a
        href="https://developer.mozilla.org/ja/docs/Web/HTML/Reference/Global_attributes/lang"
        title="Japanese">
        <bdi lang="ja">日本語</bdi>
      </a>
    </li>
    <li role="menuitem">
      <a
        href="https://developer.mozilla.org/ko/docs/Web/HTML/Reference/Global_attributes/lang"
        title="Korean">
        <bdi lang="ko">한국어</bdi>
      </a>
    </li>
    <li role="menuitem">
      <a
        href="https://developer.mozilla.org/pt-BR/docs/Web/HTML/Reference/Global_attributes/lang"
        title="Portuguese (Brazilian)">
        <bdi lang="pt-BR">Português (do Brasil)</bdi>
      </a>
    </li>
    <li role="menuitem">
      <a
        href="https://developer.mozilla.org/ru/docs/Web/HTML/Reference/Global_attributes/lang"
        title="Russian">
        <bdi lang="ru">Русский</bdi>
      </a>
    </li>
    <li role="menuitem">
      <a
        href="https://developer.mozilla.org/uk/docs/Web/HTML/Reference/Global_attributes/lang"
        title="Ukrainian">
        <bdi lang="uk">Українська</bdi>
      </a>
    </li>
    <li role="menuitem">
      <a
        href="https://developer.mozilla.org/zh-CN/docs/Web/HTML/Reference/Global_attributes/lang"
        title="Chinese (Simplified)">
        <bdi lang="zh-Hans">中文 (简体)</bdi>
```

```
      </a>
    </li>
    <li>
      <a
        href="#mdn-Web-HTML-Reference-Global_attributes-lang"
        rel="nofollow"
        id="translations-add">
        Add a translation
      </a>
    </li>
  </ul>
</div>
```

### See also

- All global attributes.
- `Content-Language` HTTP Header
- HTML `translate` attribute

## nonce HTML global attribute

The **`nonce`** global attribute is a content attribute defining a cryptographic nonce ("number used once") which can be used by Content Security Policy to determine whether or not a given fetch will be allowed to proceed for a given element.

### Description

The `nonce` attribute is useful to allowlist specific elements, such as a particular inline script or style elements. It can help you to avoid using the CSP `unsafe-inline` directive, which would allowlist *all* inline scripts or styles.

> **Note:** Only use `nonce` for cases where you have no way around using unsafe inline script or style contents. If you don't need `nonce`, don't use it. If your script is static, you could also use a CSP hash instead. (See usage notes on unsafe inline script.) Always try to take full advantage of CSP protections and avoid nonces or unsafe inline scripts whenever possible.

### Using nonce to allowlist a <script> element

There are a few steps involved to allowlist an inline script using the nonce mechanism:

### Generating values

From your web server, generate a random base64-encoded string of at least 128 bits of data from a cryptographically secure random number generator. Nonces should be generated differently each time the page loads (nonce only once!). For example, in nodejs:

```
import crypto from "node:crypto";

crypto.randomBytes(16).toString("base64");
// '8IBTHwOdqNKAWeKl7plt8g=='
```

### Allowlisting inline script

The nonce generated on your backend code should now be used for the inline script that you'd like to allowlist:

```
<script nonce="8IBTHwOdqNKAWeKl7plt8g==">
  // …
</script>
```

### Sending a nonce with a CSP header

Finally, you'll need to send the nonce value in a `Content-Security-Policy` header (prepend `nonce-`):

```
Content-Security-Policy: script-src 'nonce-8IBTHwOdqNKAWeKl7plt8g=='
```

### Accessing nonces and nonce hiding

For security reasons, the `nonce` content attribute is hidden (an empty string will be returned).

```
script.getAttribute("nonce"); // returns empty string
```

The `nonce` property is the only way to access nonces:

```
script.nonce; // returns nonce value
```

Nonce hiding helps prevent attackers from exfiltrating nonce data via mechanisms that can grab data from content attributes like this:

```
script[nonce~="whatever"] {
  background: url("https://evil.com/nonce?whatever");
}
```

### See also

- `HTMLElement.nonce`
- Content Security Policy
- CSP: `script-src`

## part HTML global attribute

The **`part`** global attribute contains a space-separated list of the part names of the element. Part names allows CSS to select and style specific elements in a shadow tree via the `::part` pseudo-element.

See the `exportparts` attribute for a usage example.

### See also

- `exportparts` HTML attribute
- `<template>` and `<slot>` HTML elements
- `::part` and `::slotted` CSS pseudo-elements
- `ShadowRoot` interface
- `Element.part` property
- Using templates and slots
- CSS scoping module

## popover HTML global attribute

The **`popover`** global attribute is used to designate an element as a popover element.

### Value

The `popover` attribute can take one of the following values:

`"auto"`

> `auto` popovers can be "light dismissed" — this means that you can hide the popover by clicking outside it or pressing the `Esc` key. Showing an `auto` popover will generally close other `auto` popovers that are already displayed, unless they are nested.
>
> > **Note:** Setting an empty value for `popover` — `popover` or `popover=""` — is equivalent to setting `popover="auto"`.

`"hint"`

> `hint` popovers do not close `auto` popovers when they are displayed, but will close other hint popovers that are not its ancestors in the hint stack. They can be light dismissed and will respond to close requests.

`"manual"`

> `manual` popovers cannot be "light dismissed" and are not automatically closed. Popovers must explicitly be displayed and closed using declarative show/hide/toggle buttons or JavaScript. Multiple independent `manual` popovers can be shown simultaneously.

### Description

Popover elements are hidden via `display: none` until opened via an invoking/control element (i.e., a `<button>` or `<input type="button">` with a `popovertarget` attribute) or a `HTMLElement.showPopover()` call.

When open, popover elements will appear above all other elements in the top layer, and won't be influenced by parent elements' `position` or `overflow` styling.

Popovers that have the `auto` state can be shown and hidden using associated controls (designated by the `popovertarget` attribute) and "light dismissed" by clicking outside the popover area, opening another popover, or pressing browser-specific mechanisms such as the `Esc` key.

Generally only one `auto` popover can be displayed on-screen at a time — showing a second popover when one is already shown will hide the first one. The exception to this rule is when you have nested auto popovers. See Nested popovers for more details.

They can also be controlled using JavaScript, for example the `HTMLElement.togglePopover()` method can be used to toggle a popover between shown and hidden.

By contrast, `manual` popovers must be manually shown and hidden — they don't automatically close other popovers when they are displayed and they can't be light dismissed. This allows for use cases where you want to show multiple popovers at the same time.

`hint` popovers do not close `auto` popovers when they are displayed, but will close other hint popovers that are not its ancestors in the hint stack. They can be light dismissed and will respond to close requests.

Usually `hint` popovers are shown and hidden in response to non-click JavaScript events such as `mouseover`/`mouseout` and `focus`/`blur`. Clicking a button to open a `hint` popover would cause an open `auto` popover to light-dismiss.

For detailed information on usage, see the Popover API landing page.

### Examples

#### Making an element a popover

The following code renders a button that, when activated, will open a popover element. This behavior can be achieved using HTML alone.

```
<button popovertarget="my-popover">Open Popover</button>

<div popover id="my-popover">Greetings, one and all!</div>
```

#### Nesting popovers

In this example, a button opens a popover that contains additional nested popovers. The nested popovers can be opened without closing the original menu popover.

#### HTML

In the first part of the HTML, we create a `<button>` that will open the main popover, which is a menu containing a few options.

```
<header>
  <button popovertarget="menu">Open Menu</button>
</header>
<main>
  <!--  Page content goes here  -->
</main>
```

In the second part of the HTML, we create the menu popover that is opened by the button we created in the previous code block. This menu popover contains an unordered list of menu items, each with an info button that opens a nested popover. The menu popover uses `popover="auto"`, which means it will not be closed when the nested popovers are opened.

```
<!-- menu popover -->
<div id="menu" popover="auto">
  <ul>
    <li>
      <a href="#">New thing</a><button popovertarget="new-info">ⓘ</button>
    </li>
    <li>
      <a href="#">Open thing</a><button popovertarget="open-info">ⓘ</button>
    </li>
    <li>
      <a href="#">Save thing</a><button popovertarget="save-info">ⓘ</button>
    </li>
    <li>
      <a href="#">Close thing</a><button popovertarget="close-info">ⓘ</button>
    </li>
  </ul>
</div>
```

In the final part of the HTML, we create the info popovers for each menu item. Each popover includes `popover="hint"`, which means it will not close the original menu popover but will close the other open info popovers.

```
<!-- info popovers -->
<div id="new-info" class="info-popover" popover="hint">
  This is some information about <strong>creating a new</strong> thing.
</div>
<div id="open-info" class="info-popover" popover="hint">
  This is some information about <strong>opening an existing</strong> thing.
</div>
<div id="save-info" class="info-popover" popover="hint">
  This is some information about <strong>saving the current</strong> thing.
</div>
<div id="close-info" class="info-popover" popover="hint">
```

```
  This is some information about <strong>closing the current</strong> thing.
</div>
```

### CSS

We've used anchor positioning to position the menu popover below the `<button>` and grid to lay out the menu items and info buttons.

```
#menu {
  margin: 0;
  margin-top: 0.4rem;
  inset: auto;
  position-area: bottom;
}
#menu ul {
  display: grid;
  grid-template-columns: max-content 1fr;
  gap: 0.4rem;
  padding: 0.4rem;
}
#menu li {
  grid-column: span 2;
  display: grid;
  grid: inherit;
  grid-template-columns: subgrid;
  gap: 1.4rem;
}
li [popovertarget] {
  cursor: pointer;
  font-size: 1.2rem;
}
li button {
  border: none;
  padding: 0;
  background-color: inherit;
}
```

Here, we've used anchor positioning to make the info popovers appear to the right of their respective info buttons.

```
div.info-popover {
  margin: 2rem;
  inset: auto;
  max-width: 300px;
  position-area: right;
}
```

### Result

Click the *Open Menu* button, then click the info icons (ⓘ) next to the menu options to open the info popovers.

> **Note:** See our Popover API examples landing page to access the full collection of MDN popover examples.

### See also

- Popover API
- `popovertarget` HTML attribute
- `popovertargetaction` HTML attribute
- `::backdrop` CSS pseudo-element
- `:popover-open` CSS pseudo-class

## slot HTML global attribute

The **`slot`** global attribute assigns a slot in a shadow DOM shadow tree to an element: An element with a `slot` attribute is assigned to the slot created by the `<slot>` element whose `name` attribute's value matches that `slot` attribute's value. You can have multiple elements assigned to the same slot by using the same slot name. Elements without a `slot` attribute are assigned to the unnamed slot, if one exists.

For examples, see our Using templates and slots guide.

### See also

- HTML global attributes
- HTML `<slot>` element
- HTML `<template>` element

- CSS `::slotted` pseudo-element
- CSS scoping module
- Web components

### spellcheck HTML global attribute

The **`spellcheck`** global attribute is an enumerated attribute that defines whether the element may be checked for spelling errors.

> **Note:** This attribute is merely a hint for the browser: browsers are not required to check for spelling errors. Typically non-editable elements are not checked for spelling errors, even if the `spellcheck` attribute is set to `true` and the browser supports spellchecking.

#### Try it

```
<textarea spellcheck="true">
This exampull will be checkd fur spellung when you try to edit it.</textarea>

<textarea spellcheck="false">
This exampull will nut be checkd fur spellung when you try to edit it.</textarea>
```

It may have the following values:

- empty string or `true`, which indicates that the element should be, if possible, checked for spelling errors;
- `false`, which indicates that the element should not be checked for spelling errors.

If this attribute is not set, its default value is element-type and browser-defined. This default value may also be *inherited*, which means that the element content will be checked for spelling errors only if its nearest ancestor has a *spellcheck* state of `true`.

#### Security and privacy concerns

Using spellchecking can have consequences for users' security and privacy. The specification does not regulate *how* spellchecking is done and the content of the element may be sent to a third party for spellchecking results (see enhanced spellchecking and "spell-jacking").

You should consider setting `spellcheck` to `false` for elements that can contain sensitive information.

#### See also

- All global attributes.
- `autocorrect`.

### style HTML global attribute

The **`style`** global attribute contains CSS styling declarations to be applied to the element. Note that it is recommended for styles to be defined in a separate file or files. This attribute and the `<style>` element have mainly the purpose of allowing for quick styling, for example for testing purposes.

> **Note:** This attribute must not be used to convey semantic information. Even if all styling is removed, a page should remain semantically correct. Typically it shouldn't be used to hide irrelevant information; this should be done using the `hidden` attribute.

#### Try it

```
<div style="background: #ffe7e8; border: 2px solid #e66465">
  <p style="margin: 15px; line-height: 1.5; text-align: center">
    Well, I am the slime from your video<br />
    Oozin' along on your livin' room floor.
  </p>
</div>
```

#### See also

- Global attributes
- `HTMLElement.style`

### tabindex HTML global attribute

The **`tabindex`** global attribute allows developers to make HTML elements focusable, allow or prevent them from being sequentially focusable (usually with the `Tab` key, hence the name) and determine their relative ordering for sequential focus navigation.

#### Try it

```
<p>Click anywhere in this pane, then try tabbing through the elements.</p>
```

```
<label>First in tab order:<input type="text" /></label>

<div tabindex="0">Tabbable due to tabindex.</div>

<div>Not tabbable: no tabindex.</div>

<label>Third in tab order:<input type="text" /></label>
```

```
p {
  font-style: italic;
  font-weight: bold;
}

div,
label {
  display: block;
  letter-spacing: 0.5px;
  margin-bottom: 1rem;
}

div:focus {
  font-weight: bold;
}
```

It accepts an integer as a value, with different results depending on the integer's value:

> **Note:** If an HTML element renders and has `tabindex` attribute with any valid integer value, the element can be focused with JavaScript (by calling the `focus()` method) or visually by clicking with the mouse. The particular `tabindex` value controls whether the element is `tabbable` (i.e., reachable via sequential keyboard navigation, usually with the `Tab` key).

- A *negative value* (the exact negative value doesn't actually matter, usually `tabindex="-1"`) means that the element is not reachable via sequential keyboard navigation.

  > **Note:** `tabindex="-1"` may be useful for elements that should not be navigated to directly using the `Tab` key, but need to have keyboard focus set to them. Examples include an off-screen modal window that should be focused when it comes into view, or a form submission error message that should be immediately focused when an errant form is submitted.

- `tabindex="0"` means that the element should be focusable in sequential keyboard navigation, after any positive `tabindex` values. The focus navigation order of these elements is defined by their order in the document source.
- A *positive value* means the element should be focusable in sequential keyboard navigation, with its order defined by the value of the number. That is, `tabindex="4"` is focused before `tabindex="5"` and `tabindex="0"`, but after `tabindex="3"`. If multiple elements share the same positive `tabindex` value, their order relative to each other follows their position in the document source. The maximum value for `tabindex` is 32767.
- If the `tabindex` attribute is included with no value set, whether the element is focusable is determined by the user agent.

  > **Warning:** You are recommended to only use `0` and `-1` as `tabindex` values. Avoid using `tabindex` values greater than `0` and CSS properties that can change the order of focusable HTML elements (Ordering flex items). Doing so makes it difficult for people who rely on using keyboard for navigation or assistive technology to navigate and operate page content. Instead, write the document with the elements in a logical sequence.

Some focusable HTML elements have a default `tabindex` value of `0` set under the hood by the user agent. These elements are an `<a>` or `<area>` with `href` attribute, `<button>`, `<frame>` , `<iframe>`, `<input>`, `<object>`, `<select>`, `<textarea>`, and SVG `<a>` element, or a `<summary>` element that provides summary for a `<details>` element. Developers shouldn't add the `tabindex` attribute to these elements unless it changes the default behavior (for example, including a negative value will remove the element from the focus navigation order).

> **Warning:** The tabindex attribute must not be used on the `<dialog>` element.

### Accessibility concerns

Avoid using the `tabindex` attribute in conjunction with non-interactive content to make something intended to be interactive focusable by keyboard input. An example of this would be using a `<div>` element to describe a button, instead of the `<button>` element.

Interactive components authored using non-interactive elements are not listed in the accessibility tree. This prevents assistive technology from being able to navigate to and manipulate those components. The content should be semantically described using interactive elements (`<a>`, `<button>`,

`<details>`, `<input>`, `<select>`, `<textarea>`, etc.) instead. These elements have built-in roles and states that communicate status to the accessibility that would otherwise have to be managed by ARIA.

- Using the tabindex attribute | Vispero

### See also

- All global attributes
- `HTMLElement.tabIndex` that reflects this attribute
- Accessibility problems with `tabindex`: see Don't Use Tabindex Greater than 0 by Adrian Roselli
- Reading order

## title HTML global attribute

The **`title`** global attribute contains text representing advisory information related to the element it belongs to.

### Try it

```
<p>
  Use the <code>title</code> attribute on an <code>iframe</code> to clearly
  identify the content of the <code>iframe</code> to screen readers.
</p>

<iframe
  title="Wikipedia page for the HTML language"
  src="https://en.m.wikipedia.org/wiki/HTML"></iframe>
<iframe
  title="Wikipedia page for the CSS language"
  src="https://en.m.wikipedia.org/wiki/CSS"></iframe>
```

```
iframe {
  height: 200px;
  margin-bottom: 24px;
  width: 100%;
}
```

### Description

The main use of the `title` attribute is to label `<iframe>` elements for assistive technology.

The `title` attribute may also be used to label controls in data tables.

The `title` attribute, when added to `<link rel="stylesheet">`, creates an alternate stylesheet. When defining an alternative style sheet with `<link rel="alternate">` the attribute is required and must be set to a non-empty string.

If included on the `<abbr>` opening tag, the `title` must be a full expansion of the abbreviation or acronym. Instead of using `title`, when possible, provide an expansion of the abbreviation or acronym in plain text on first use, using the `<abbr>` to mark up the abbreviation. This enables all users know what name or term the abbreviation or acronym shortens while providing a hint to user agents on how to announce the content.

While `title` can be used to provide a programmatically associated label for an `<input>` element, this is not good practice. Use a `<label>` instead.

### Multiline titles

The `title` attribute may contain several lines. Each `U+000A LINE FEED (LF)` character represents a line break. Some caution must be taken, as this means the following renders across two lines:

### HTML

```
<p>
  Newlines in <code>title</code> should be taken into account. This
  <span
    title="This is a
multiline title">
    example span
  </span>
  has a title attribute with a newline.
</p>
<hr />
<pre id="output"></pre>
```

### JavaScript

We can query the `title` attribute and display it in the empty `<pre>` element as follows:

```
const span = document.querySelector("span");
const output = document.querySelector("#output");
output.textContent = span.title;
```

#### Result

#### Title attribute inheritance

If an element has no `title` attribute, then it inherits it from its parent node, which in turn may inherit it from its parent, and so on.

If this attribute is set to the empty string, it means its ancestors' `title`s are irrelevant and shouldn't be used in the tooltip for this element.

#### HTML

```
<div title="CoolTip">
  <p>Hovering here will show "CoolTip".</p>
  <p title="">Hovering here will show nothing.</p>
</div>
```

#### Result

#### Accessibility concerns

Use of the `title` attribute is highly problematic for:

- People using touch-only devices
- People navigating with keyboards
- People navigating with assistive technology such as screen readers or magnifiers
- People experiencing fine motor control impairment
- People with cognitive concerns

This is due to inconsistent browser support, compounded by the additional assistive technology parsing of the browser-rendered page. If a tooltip effect is desired, it is better to use a more accessible technique that can be accessed with the above browsing methods.

- 3.2.5.1. The title attribute | W3C HTML 5.2: 3. Semantics, structure, and APIs of HTML documents
- Using the HTML title attribute – updated | Vispero
- Tooltips & Toggletips - Inclusive Components
- The Trials and Tribulations of the Title Attribute - 24 Accessibility

#### See also

- All global attributes.
- `HTMLElement.title` that reflects this attribute.

### translate HTML global attribute

The **`translate`** global attribute is an enumerated attribute that is used to specify whether an element's *translatable attribute* values and its `Text` node children should be translated when the page is localized, or whether to leave them unchanged.

It can have the following values:

- empty string or `yes`, which indicates that the element should be translated when the page is localized.
- `no`, which indicates that the element must not be translated.

Although not all browsers recognize this attribute, it is respected by automatic translation systems such as Google Translate, and may also be respected by tools used by human translators. As such it's important that web authors use this attribute to mark content that should not be translated.

#### Examples

In this example, the `translate` attribute is used to ask translation tools not to translate the company's brand name in the footer.

```
<footer>
  <small>© 2020 <span translate="no">BrandName</span></small>
</footer>
```

#### See also

- All global attributes.
- The `HTMLElement.translate` property that reflects this attribute.
- Using HTML's translate attribute.
- HTML `lang` attribute

## virtualkeyboardpolicy HTML global attribute

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`virtualkeyboardpolicy`** global attribute is an enumerated attribute. When specified on an element that the element's content is editable (for example, it is an `<input>` or `<textarea>` element, or an element with the `contenteditable` attribute set), it controls the on-screen virtual keyboard behavior on devices such as tablets, mobile phones, or other devices where a hardware keyboard may not be available.

The attribute must take one of the following values:

- `auto` or an *empty string*, which automatically shows the virtual keyboard when the element is focused or tapped.
- `manual`, which decouples focus and tap on the element from the virtual keyboard's state.

### See also

- All global attributes
- `HTMLElement.contentEditable` and `HTMLElement.isContentEditable`
- The VirtualKeyboard API

## writingsuggestions HTML global attribute

The **`writingsuggestions`** global attribute is an enumerated attribute indicating if browser-provided writing suggestions should be enabled under the scope of the element or not.

Some browsers provide writing suggestions to users as they type in editable fields. Suggestions usually appear as grayed-out text positioned after the text cursor, completing the user's sentence. While this can be helpful to users, developers might want to turn writing suggestions off in some cases, such as when providing site-specific writing suggestions.

The `writingsuggestions` attribute can be set on editable fields such as `<input>` or `<textarea>` elements, or on other HTML elements to control the behavior of the browser's suggestions on sections of a page, or on the entire page.

### Syntax

In browsers that support them, writing suggestions are enabled by default. To disable them, set the `writingsuggestions` attribute's value to `false`. Setting the attribute's value to `true`, or omitting the value, enables writing suggestions.

To disable writing suggestions:

```
<input type="text" writingsuggestions="false" />
```

To enable writing suggestions:

```
<input type="text" />
<input type="text" writingsuggestions />
<input type="text" writingsuggestions="true" />
```

### See also

- `autocomplete` attribute
- `spellcheck` attribute
- `contenteditable` attribute
- `<textarea>`
- `<input>`
- `<datalist>` and `list` attribute